

Modern GPU Architecture Second Edition

Volume Two Compute Acceleration, Tensor Cores, and Advanced Systems

Modern GPU Architecture Second Edition

Volume Two Compute Acceleration, Tensor Cores, and Advanced Systems

Second Edition

Gareth Morgan Thomas
Auckland, New Zealand



Published by Burst Books
Auckland, New Zealand

Copyright © 2025 Gareth Morgan Thomas
All rights reserved.

Preface

Modern GPUs are no longer just engines for rendering pixels—they are the computational backbone of real-time graphics, scientific simulations, and artificial intelligence. This book offers a detailed architectural exploration of these powerful processors, mapping out the intricate hardware structures that drive both visual and machine-learning workloads.

Rather than a step-by-step design manual, this volume provides a theoretical blueprint for understanding how modern GPU systems are built and optimized. From the memory hierarchy and graphics pipeline stages to AI accelerators like tensor cores, each component is examined with clarity and depth.

The content is designed for computer engineers, hardware architects, graphics programmers, and AI researchers who require a foundational understanding of GPU internals—not to build a GPU, but to master the systems that increasingly power modern computing.

As we move deeper into an era of heterogeneous computing, the GPU continues to evolve as a core enabler of innovation. This book aims to be both a technical map and a reference guide for navigating that evolution.

How This Book Was Made

Some people loathe AI-written works, seeing them as a threat to the livelihood of authors, academics, and researchers alike. I, however, urge you to embrace the future. In only a few years, AI-generated scientific works will surpass those created by humans—even by teams that spend years producing them.

Most academic textbooks are written by teams of professors, graduate researchers, and editors over several years. They go through multiple review cycles, institutional approvals, and publishing delays. The result is often accurate but slow to appear—and expensive, with prices commonly exceeding \$100–\$200.

This book takes a different path. It was created using an AI-assisted workflow under direct human supervision. AI tools helped organize material, ensure consistency, and keep the content aligned with the latest research, while every formula, diagram, and code sample was reviewed and verified for accuracy (by a different AI of course).

It may not read with the same polish as a traditionally written work, but it contains an abundance of up-to-date technical information—material that often takes years to reach print in conventional publishing.

This method achieves the same technical depth and reliability as traditional textbooks but delivers it faster, at lower cost—typically \$30–\$40—and with information that reflects the most current advances in science and engineering.

The goal is simple: make advanced STEM knowledge accessible, affordable, and up to date.

Welcome to *Modern GPU Architecture Second Edition*.

About the Author

Gareth Morgan Thomas is a qualified expert with extensive expertise across multiple STEM fields. Holding six university diplomas in electronics, software development, web development, and project management, along with qualifications in computer networking, CAD, diesel engineering, well drilling, and welding, he has built a robust foundation of technical knowledge. Educated in Auckland, New Zealand, Gareth Morgan Thomas also spent three years serving in the New Zealand Army, where he honed his discipline and problem-solving skills. With years of technical training, Gareth Morgan Thomas is now dedicated to sharing his deep understanding of science, technology, engineering, and mathematics through a series of specialized books aimed at both beginners and advanced learners.

Table of Contents

About the Author	vii
1 Compute Architecture	3
1.1 GPGPU Programming Model	3
1.1.1 Kernels, threads, and work-groups	3
1.1.2 Hierarchical execution model	4
1.1.3 Global and shared memory scopes	6
1.1.4 Synchronization mechanisms	8
1.2 Compute Dispatch	9
1.2.1 Kernel launch process	9
1.2.2 Command packet format	11
1.2.3 Work-group distribution	14
1.2.4 Concurrent kernel execution	15
1.3 Occupancy and Resource Management	17
1.3.1 Register pressure and allocation	17
1.3.2 Shared memory partitioning	18
1.3.3 Warp and block occupancy	20
1.3.4 Resource scheduling	21
1.4 Work Distribution	23
1.4.1 Static and dynamic scheduling	23
1.4.2 Load balancing algorithms	24
1.4.3 Persistent threads model	26
1.4.4 Cooperative group synchronization	27
1.5 Data Parallel Processing Patterns	29
1.5.1 Map, reduce, and scan operations	29
1.5.2 Histogram and sort algorithms	30
1.5.3 Matrix operations	30
1.5.4 Prefix sums and reductions	32
1.6 Section 6: Verilog Implementation	33
1.6.1 Compute dispatch unit	33
1.6.2 Work-group scheduler	36
1.6.3 Resource allocator	38
1.6.4 Barrier synchronization logic	41
1.6.5 Compute kernel testbench	42

2	Tensor and Matrix Acceleration	45
2.1	Matrix Multiplication Fundamentals	45
2.1.1	GEMM operation principles	45
2.1.2	Blocking and tiling strategies	46
2.1.3	Data reuse optimization	48
2.1.4	Arithmetic intensity considerations	49
2.2	Tensor Core Architecture	50
2.2.1	Systolic array organization	50
2.2.2	Matrix multiply-accumulate units	53
2.2.3	Dataflow and accumulation patterns	55
2.2.4	Precision and throughput balance	57
2.3	Mixed Precision Support	59
2.3.1	FP16, BF16, TF32 computation	59
2.3.2	INT8 and INT4 quantization	61
2.3.3	Accumulator precision control	63
2.3.4	Conversion and normalization units	66
2.4	Tensor Memory Layout	68
2.4.1	Row-major and column-major ordering	68
2.4.2	Tiled and blocked formats	69
2.4.3	Swizzling for conflict avoidance	71
2.4.4	Efficient memory access patterns	72
2.5	Sparse Matrix Acceleration	73
2.5.1	CSR and COO representations	73
2.5.2	Structured sparsity (2:4, 4:8)	75
2.5.3	Zero-skipping hardware logic	77
2.5.4	Compression and decompression paths	79
2.6	Section 6: Verilog Implementation	79
2.6.1	Systolic array module	79
2.6.2	Matrix multiply-accumulate block	82
2.6.3	Data distribution network	85
2.6.4	Tensor core testbench	86
3	Ray Tracing Hardware	91
3.1	Ray Tracing Fundamentals	91
3.1.1	Ray representation	91
3.1.2	Intersection with primitives	92
3.1.3	BVH construction principles	94
3.1.4	Traversal algorithms	95
3.2	RT Core Architecture	97
3.2.1	Ray-box intersection logic	97
3.2.2	Ray-triangle intersection unit	99
3.2.3	BVH traversal engine	101
3.2.4	Hit and miss determination	102
3.3	Acceleration Structures	104
3.3.1	BVH node hierarchy	104
3.3.2	Memory layout optimization	106
3.3.3	Update and refit mechanisms	107

3.3.4	Build-time vs runtime tradeoffs	108
3.4	Ray Coherence and Sorting	110
3.4.1	Coherent Ray Batching	110
3.4.2	Ray binning and bucketing	111
3.4.3	Cache-aware reordering	113
3.4.4	Wavefront path tracing	114
3.4.5	Wavefront Path Tracing	114
3.5	Integration with Rasterization	116
3.5.1	Hybrid rendering pipeline	116
3.5.2	Shader-based ray generation	117
3.5.3	Shader binding table	118
3.5.4	Payload management	120
3.6	Section 6: Verilog Implementation	122
3.6.1	Ray-box intersection module	122
3.6.2	Ray-triangle intersection unit	124
3.6.3	BVH traversal FSM	127
3.6.4	RT core testbench	130
4	Synchronization and Memory Ordering	135
4.1	Memory Consistency Models	135
4.1.1	Sequential and relaxed models	135
4.1.2	Acquire-release semantics	136
4.1.3	Visibility scopes	138
4.1.4	GPU-specific ordering rules	139
4.2	Barriers and Fences	141
4.2.1	Block and grid-level barriers	141
4.2.2	Memory fence types	143
4.2.3	System-wide synchronization	144
4.2.4	Performance overhead	146
4.3	Cache Coherence	147
4.3.1	Write-invalidate protocols	147
4.3.2	Directory-based coherence	150
4.3.3	Cross-core consistency	152
4.3.4	Heterogeneous CPU-GPU models	153
4.4	Atomic Operations	156
4.4.1	Read-modify-write logic	156
4.4.2	Compare-and-swap	158
4.4.3	Arbitration circuits	161
4.4.4	Performance optimizations	162
4.5	Lock-Free Algorithms	164
4.5.1	Wait-free synchronization	164
4.5.2	ABA problem handling	165
4.5.3	Lock-free queues	167
4.5.4	GPU-specific design considerations	169
4.6	Section 6: Verilog Implementation	170
4.6.1	Barrier synchronization module	170
4.6.2	Atomic operation unit	172

4.6.3	Memory fence controller	176
4.6.4	Synchronization testbench	180
5	Advanced Rendering Features	185
5.1	Tessellation Pipeline	185
5.1.1	Hull shader and control points	185
5.1.2	Fixed-function tessellator	187
5.1.3	Domain shader operations	189
5.1.4	Adaptive tessellation control	191
5.2	Geometry Processing	193
5.2.1	Geometry shader stage	193
5.2.2	Primitive amplification	195
5.2.3	Stream output	196
5.2.4	Layered rendering	199
5.3	Mesh Shaders	201
5.3.1	Meshlet-based processing	201
5.3.2	Task and mesh shader stages	202
5.3.3	Workgroup culling and amplification	204
5.3.4	Hardware resource mapping	205
5.4	Variable Rate Shading	207
5.4.1	Shading rate images	207
5.4.2	Coarse shading patterns	209
5.4.3	Foveated rendering	210
5.4.4	Performance and power gains	211
5.5	Deferred Rendering Architecture	213
5.5.1	G-buffer composition	213
5.5.2	Geometry and lighting passes	215
5.5.3	Tile-based deferred shading	217
5.5.4	Bandwidth and efficiency analysis	219
5.6	Section 6: Verilog Implementation	221
5.6.1	Tessellator hardware	221
5.6.2	Meshlet processor	223
5.6.3	VRS controller	226
5.6.4	G-buffer manager testbench	229
6	Display and Video Engines	233
6.1	Display Controller	233
6.1.1	Timing generation (HSYNC and VSYNC)	233
6.1.2	Frame buffer scanning	235
6.1.3	Pixel pipeline organization	238
6.1.4	Multi-display management	240
6.2	Display Compression	242
6.2.1	Display Stream Compression (DSC)	242
6.2.2	Encoder and decoder design	245
6.2.3	Bandwidth reduction analysis	247
6.2.4	Visual quality metrics	249
6.3	Video Decode Acceleration	251

6.3.1	H.264, H.265, VP9, and AV1 decoding	251
6.3.2	Bitstream parsing and entropy decoding	252
6.3.3	Motion compensation hardware	255
6.3.4	Parallel decode engines	257
6.4	Video Encode Acceleration	259
6.4.1	Motion estimation logic	259
6.4.2	Rate control mechanisms	261
6.4.3	Entropy encoder design	263
6.4.4	Multi-format support	266
6.5	Video Processing Pipeline	268
6.5.1	Scaling and filtering	268
6.5.2	Color space conversion	271
6.5.3	Deinterlacing and denoising	274
6.5.4	HDR tone mapping	275
6.6	Section 6: Verilog Implementation	277
6.6.1	Display timing generator	277
6.6.2	Video decoder FSM	279
6.6.3	Motion estimation module	282
6.6.4	Video pipeline testbench	286
7	Interconnect and Communication	289
7.1	On-Chip Networks	289
7.1.1	Mesh and crossbar topologies	289
7.1.2	Router architecture and buffering	291
7.1.3	Flow control and arbitration	293
7.1.4	Deadlock prevention	294
7.2	Memory Crossbar	296
7.2.1	SM-to-memory partition links	296
7.2.2	Bandwidth scheduling	299
7.2.3	Virtual channels	300
7.2.4	QoS enforcement	302
7.2.5	Error handling	304
7.3	High-Speed Serial Links	307
7.3.1	NVLink and Infinity Fabric	307
7.3.2	CXL and coherent interfaces	308
7.3.3	PHY design and equalization	311
7.3.4	Latency and throughput tuning	313
7.4	Multi-GPU Communication	314
7.4.1	GPU-to-GPU transfers	314
7.4.2	Collective operations	317
7.4.3	Topology optimization	319
7.4.4	Topology Optimization for GPU Interconnects	319
7.4.5	Scalability challenges	320
7.5	Section 6: Verilog Implementation	323
7.5.1	Crossbar switch module	323
7.5.2	Round-robin arbiter	325
7.5.3	PCIe transaction engine	327

7.5.4	NoC router and testbench	328
8	Performance Analysis and Optimization	331
8.1	Performance Metrics	331
8.1.1	Throughput (GFLOPS and TFLOPS)	331
8.1.2	Bandwidth utilization	331
8.1.3	Cache efficiency	333
8.1.4	Power and thermal metrics	334
8.2	Bottleneck Identification	336
8.2.1	Memory-bound and compute-bound workloads	336
8.2.2	Latency versus bandwidth limits	337
8.2.3	Roofline analysis	338
8.2.4	Profiling methodology	340
8.3	Performance Counters	341
8.3.1	Counter and sampler architecture	341
8.3.2	Multiplexing techniques	344
8.3.3	Key hardware metrics	346
8.3.4	PMU software interfaces	347
8.4	Workload Characterization	349
8.4.1	Instruction mix and balance	349
8.4.2	Cache and memory patterns	351
8.4.3	Thread divergence statistics	352
8.4.4	Power behavior profiling	353
8.5	Optimization Techniques	355
8.5.1	Occupancy tuning	355
8.5.2	Coalesced memory access	356
8.5.3	Shared memory utilization	358
8.5.4	Instruction scheduling	359
8.6	Section 6: Power and Thermal Management	361
8.6.1	Dynamic power reduction	361
8.6.2	Workload-based DVFS	362
8.6.3	Thermal throttling	364
8.6.4	Efficiency optimization	365
9	Physical Design and Manufacturing	369
9.1	Floorplanning	369
9.1.1	Die partitioning and hierarchy	369
9.1.2	Block placement and routing	371
9.1.3	Power grid and clock tree	372
9.1.4	Thermal optimization	374
9.2	Synthesis and Timing Closure	376
9.2.1	RTL synthesis flow	376
9.2.2	Timing constraints and setup	377
9.2.3	Clock domain verification	379
9.2.4	Multi-mode optimization	381
9.3	Place and Route	383
9.3.1	Placement algorithms	383

9.3.2	Routing congestion management	384
9.3.3	Signal integrity checks	386
9.3.4	IR drop and EM control	388
9.4	Design for Test	389
9.4.1	Scan insertion	389
9.4.2	Built-in self-test	391
9.4.3	JTAG and boundary scan	394
9.4.4	Yield analysis	397
9.5	Packaging Technologies	399
9.5.1	Flip-chip and BGA packaging	399
9.5.2	Thermal interface materials	401
9.5.3	Multi-chip modules	402
9.5.4	TSV-based stacking	404
9.6	Section 6: Advanced Integration	406
9.6.1	Chiplet architectures	406
9.6.2	Die-to-die interconnects	408
9.6.3	UCIe protocol	411
9.6.4	Heterogeneous integration	413
10	Future Directions and Emerging Technologies	417
10.1	Modern GPU Case Studies	417
10.1.1	NVIDIA Hopper and AMD RDNA3	417
10.1.2	Intel Arc and Apple GPU designs	418
10.1.3	Mobile GPUs (Mali, Adreno)	420
10.1.4	Design tradeoff comparisons	421
10.2	Specialized AI Accelerators	423
10.2.1	Google TPU and Cerebras engine	423
10.2.2	Graphcore IPU and Groq LPU	424
10.2.3	Comparison with general GPUs	426
10.2.4	Domain-specific optimization	427
10.3	Beyond Moores Law	429
10.3.1	Process scaling limits	429
10.3.2	GAA and CFET transistor evolution	430
10.3.3	Advanced packaging methods	432
10.3.4	Economic and design impact	434
10.4	Emerging Memory Technologies	436
10.4.1	HBM4 and next-gen DRAM	436
10.4.2	Processing-in-memory concepts	437
10.4.3	Persistent and near-data memory	440
10.4.4	Memory-centric system design	442
10.5	Novel Computing Paradigms	443
10.5.1	Neuromorphic and optical computing	443
10.5.2	Quantum-accelerated systems	445
10.5.3	Approximate computation	446
10.5.4	Stochastic and hybrid models	448
10.6	Section 6: Sustainability and Green Computing	449
10.6.1	Power efficiency trends	449

10.6.2	Carbon footprint reduction	451
10.6.3	Lifecycle optimization	452
10.6.4	Renewable-powered data centers	455
10.7	Section 7: Research Frontiers	456
10.7.1	AI-driven hardware design	456
10.7.2	Self-optimizing microarchitectures	458
10.7.3	Secure and open GPU initiatives	459
10.7.4	Future scalability challenges	461
Glossary of Terms		465
Key Mathematical Equations		471

Introduction

The evolution of the GPU from a fixed-function graphics processor to a general-purpose parallel computing engine is one of the most transformative developments in modern hardware design. What began as a specialized device for rendering polygons and shading pixels has become the backbone of scientific computing, artificial intelligence, and large-scale data processing. Modern GPUs no longer merely display images—they simulate physics, train neural networks, and accelerate entire industries. This second volume explores that transformation in detail, examining the compute, tensor, and system-level architecture that powers today’s most advanced computational engines.

While Volume One dissected the graphics pipeline and its real-time rendering machinery, this book begins where that pipeline ends: at the programmable core level, where threads, warps, and execution units cooperate to perform massive numbers of arithmetic operations every cycle. It delves into the heart of the GPU as a parallel computer—an intricate network of schedulers, execution pipelines, memory hierarchies, and communication fabrics—each tuned to sustain throughput under extraordinary workloads.

The central focus of this volume is compute acceleration. Modern workloads are not defined by the geometry of pixels but by the algebra of tensors—matrix multiplications, convolutions, and reductions executed at scales that challenge the limits of memory bandwidth and on-die communication. These operations have redefined the architecture of GPUs themselves, giving rise to specialized functional units such as Tensor Cores, matrix-multiply accumulators, and sparsity-aware dataflows. To understand the GPU today, one must understand how these accelerators coexist with traditional shader cores, sharing caches, interconnects, and power budgets while maintaining deterministic scheduling and coherence.

This book provides a systematic exploration of that ecosystem. It examines the shader core microarchitecture, instruction set organization, warp scheduling algorithms, and the intricate register file systems that feed thousands of execution lanes. It studies memory subsystems—L1, L2, shared memory, and DRAM controllers—with the same attention to timing and bandwidth that once applied to texture units. It then extends that analysis to tensor and matrix acceleration, explaining how systolic arrays, mixed-precision arithmetic, and structured sparsity have transformed the definition of parallel efficiency.

In the process, it introduces the Verilog and architectural constructs necessary to implement these features at the RTL level. Each module—whether a warp scheduler, tensor accumulator, or interconnect switch—is presented in synthesizable form, emphasizing clock-domain correctness, resource allocation, and latency control. The goal is not to show toy designs but to demonstrate the engineering reality of scalable parallel systems.

As with Volume One, the book avoids abstraction for its own sake. Every concept is connected to measurable quantities: throughput, latency, occupancy, power, and die area. Each equation, diagram, and hardware listing has been chosen to illustrate the trade-offs that define real silicon. The reader is expected to think as an architect, balancing competing objectives

while preserving deterministic behavior under concurrency. The notation may be mathematical, but the intent is practical—to train the mind to reason in cycles, registers, and datapaths.

Compute acceleration is only one aspect of the story. Modern GPUs are systems-on-chip, with interconnects, DMA engines, coherence protocols, and multi-GPU fabrics that rival supercomputers in complexity. Later chapters explore these global systems—how workloads are dispatched, synchronized, and scaled across clusters of compute units or even across multiple dies. They analyze communication latency, arbitration schemes, and topologies such as crossbars, meshes, and rings. They show how performance monitoring, profiling, and dynamic frequency scaling are implemented in hardware to maintain stability under load.

The second half of this volume extends beyond design to manufacturing and evolution. It surveys physical design considerations such as floorplanning, clock distribution, and chiplet integration, and concludes with a look toward emerging directions in GPU research—heterogeneous compute, neuromorphic acceleration, photonic interconnects, and AI-driven hardware optimization. The intent is not to predict the future but to provide the conceptual grounding from which the next generation of architectures will grow.

Like its predecessor, this book is uncompromisingly technical. It assumes that the reader is comfortable with digital design, Verilog, and performance analysis, and is prepared to follow detailed discussions of dataflow scheduling, arithmetic precision, and memory coherence. It does not simplify the engineering reality; it confronts it directly, exposing both the elegance and the difficulty of designing massively parallel systems that work reliably at scale.

Volume Two is both a continuation and an expansion of the ideas established in Volume One. It moves from pixels to tensors, from rasterization to reduction, from rendering to reasoning. Together, these volumes trace the full arc of GPU design—from the transformation of geometry into pixels to the transformation of data into intelligence. They are written for those who seek not to use the GPU as a black box, but to understand and build it from first principles.

The chapters that follow will challenge you to think about computation as architecture, architecture as mathematics, and mathematics as engineering. In mastering them, you will gain not just knowledge of GPU design, but a deeper insight into the parallel nature of modern computation itself—the fundamental language of performance in the age of acceleration.

Chapter 1

Compute Architecture

1.1 GPGPU Programming Model

1.1.1 Kernels, threads, and work-groups

First we will formalize how a compute kernel is decomposed into threads and work-groups and how those map onto SMs (streaming multiprocessors) or CUs (compute units). The objective is to quantify resource limits and show how scheduling, divergence, and local memory shape performance for graphics and ML kernels.

Problem: choose block (work-group) size and layout to maximize parallelism without exceeding per-SM resources or inducing heavy divergence. Analysis begins by enumerating constraints:

- hardware limits: $T_{\text{SM}}^{\text{max}}$ (max threads per SM), $B_{\text{SM}}^{\text{max}}$ (max blocks per SM), R_{SM} (total registers per SM), S_{SM} (shared memory per SM);
- kernel parameters: t_{blk} (threads per block), r_{th} (registers per thread), s_{blk} (shared memory per block).

The number of resident blocks per SM, B_{res} , is

$$[H]B_{\text{res}} = \min \left(\left\lfloor \frac{R_{\text{SM}}}{r_{\text{th}} t_{\text{blk}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{s_{\text{blk}}} \right\rfloor, \left\lfloor \frac{T_{\text{SM}}^{\text{max}}}{t_{\text{blk}}} \right\rfloor, B_{\text{SM}}^{\text{max}} \right). \quad (1.1)$$

Occupancy, the fraction of active threads relative to the peak, then becomes $\text{occ} = \frac{B_{\text{res}} t_{\text{blk}}}{T_{\text{SM}}^{\text{max}}}$.

Implementation guidance: kernels should expose enough parallelism (large grid size) and choose t_{blk} so warps fully utilize ALUs and TMUs without inducing register pressure that forces spilling. Branch divergence serializes execution within a warp; for NVIDIA warps are 32 threads, while other vendors historically used 64-thread wavefronts, so choose per-thread work to minimize lane divergence. Shared/LDS memory and barriers enable fast intra-work-group cooperation but increase s_{blk} and thus may reduce B_{res} .

Example CUDA pattern that demonstrates mapping, shared memory use, and synchronization:

Listing 1.1: Simple tiled matrix multiply kernel showing block/thread indices and shared memory use.

```
__global__ void tiledGEMM(float* A, float* B, float* C,
```

```

        int N) {
__shared__ float As[32][32]; // tile in shared mem
__shared__ float Bs[32][32];
int bx = blockIdx.x, by = blockIdx.y;
int tx = threadIdx.x, ty = threadIdx.y;
int row = by*blockDim.y + ty;
int col = bx*blockDim.x + tx;
float acc = 0.0f;
for (int m=0; m < N; m += 32) {
    As[ty][tx] = A[row*N + (m+tx)]; // load A tile
    Bs[ty][tx] = B[(m+ty)*N + col]; // load B tile
    __syncthreads(); // barrier for work-group
    for (int k=0; k<32; ++k) acc += As[ty][k]*Bs[k][tx];
    __syncthreads(); // ensure next tile safe
}
C[row*N + col] = acc; // write result
}

```

Implications and trade-offs:

- Increasing threads-per-block raises per-block parallel work but may reduce B_{res} via Eq. (8.27), lowering occupancy.
- High register usage increases single-thread performance but can cause spilling that hurts memory-bound workloads.
- Shared-memory tiling reduces global memory traffic and TMU pressure, improving arithmetic intensity for GEMM and convolution workloads, at the cost of fewer concurrent blocks.
- Branch-heavy kernels suffer when divergence increases active mask management and serialization inside warps, particularly for small warp sizes where per-thread variability matters.

Designers should therefore balance occupancy, register use, shared-memory allocation, and divergence to match workload characteristics (graphics shading, compute kernels, or ML tensor workloads) and the target SM/CU resource profile.

1.1.2 Hierarchical execution model

Continuing from the kernel and thread grouping model, we now map those software-level groups onto hardware resources such as SMs, warps, and schedulers to expose the hierarchical execution model used in modern GPUs.

The core problem is efficient, deterministic mapping of a large logical grid of threads onto a finite set of SMs and SIMT lanes while maximizing utilization and minimizing stalls from divergence, memory latency, and resource contention. The analysis proceeds from the software hierarchy (grid $\rightarrow$ work-groups/blocks $\rightarrow$ threads) to the hardware hierarchy (SM assignment $\rightarrow$ warp formation $\rightarrow$ scheduler dispatch). A common mapping used by CUDA and many runtimes computes a global linear thread index via

$$[H]\text{tid}_{\text{global}} = \text{blockIdx}_x \cdot \text{blockDim}_x + \text{threadIdx}_x, \quad (1.2)$$

which allows contiguous memory access patterns when blockDim aligns with memory coalescing widths.

Resource-driven occupancy determines how many warps an SM can keep active. Let

- $W_{\max}$ be the hardware maximum warps per SM,
- R_{SM} be total registers per SM,
- S_{SM} be shared memory per SM,
- r_b be registers used per block,
- s_b be shared memory used per block,
- w_b be warps per block.

Then active warps per SM can be expressed as

$$[H]W_{\text{active}} = \min \left(W_{\max}, \left\lfloor \frac{R_{\text{SM}}}{r_b} \right\rfloor \cdot w_b, \left\lfloor \frac{S_{\text{SM}}}{s_b} \right\rfloor \cdot w_b \right). \quad (1.3)$$

This formulation makes explicit how register pressure and shared-memory allocation throttle concurrency, which directly affects latency hiding.

Implementation-level considerations follow established patterns:

- Form warps by grouping consecutive threads in a block to preserve memory coalescing and reduce TLB churn.
- Scheduler maintains multiple ready warp queues, selecting warps with ready memory or independent instructions to hide latency.
- Barrier and synchronization semantics are implemented at the work-group level, requiring the entire block to be resident on one SM or supported by heavyweight scheduling if spanning SMs.

Example CUDA-style kernel showing hierarchical indices and shared-memory usage:

Listing 1.2: Work-group to warp mapping example

```
__global__ void matAdd(float *A, float *B, float *C, int N) {
    extern __shared__ float sbuf[];           // per-block shared memory
    int local_x = threadIdx.x;                // thread within block
    int block_x = blockIdx.x;                 // work-group id
    int global_tid = block_x * blockDim.x + local_x; // eq. \eqref{eq:global_tid}
    }
    if (global_tid < N) {
        sbuf[local_x] = A[global_tid] + B[global_tid]; // shared staging
        __syncthreads();                               // block-level barrier
        C[global_tid] = sbuf[local_x];
    }
    // warp formation is implicit: consecutive local_x form warps for SIMT
}
```

Design implications and performance trade-offs:

- Increasing blockDim improves coalescing but may exceed per-SM resources, lowering W_{active} (eq. 1.3).
- High register usage per thread reduces active warps, harming latency hiding; compiler and kernel tuning must balance unrolling vs. register pressure.
- Large shared-memory per block improves data reuse but constrains concurrent blocks per SM, impacting throughput.
- Scheduler complexity (multiple queues, priority policies) trades silicon area and power for better warp selection and fairness.

Choosing block sizes and resource allocations therefore is a joint software–hardware optimization to maximize SM utilization under SIMT constraints.

1.1.3 Global and shared memory scopes

Building on the execution hierarchy and the mapping of kernels to grids and work-groups to SMs/CUs, memory scopes determine where data lives, who can see it, and which synchronization primitives are required. Correct placement between global (device) and shared (per-SM) memory is the primary handle for balancing latency, bandwidth, and programmer-visible consistency in SIMT workloads.

Global memory: problem and characteristics.

- Problem: store large datasets for kernels (textures, weight tensors, output arrays) while servicing many concurrent SMs.
- Characteristics: host-visible, high capacity (GBs), high latency (hundreds of cycles to DRAM), typically cached by L2 and optionally by per-SM L1. Coherency across SMs is maintained at cache-line granularity by the GPU’s memory system, but software must use fences for ordering: use `__threadfence` to make writes visible device-wide, and system fences for CPU-GPU consistency.

Shared memory (per-block/work-group): problem and characteristics.

- Problem: low-latency scratchpad for fine-grained data sharing (tile buffers, staging for tensor cores, warp-level reductions).
- Characteristics: on-chip, banked, low latency (tens of cycles), lifetime scoped to the work-group. On NVIDIA SMs shared memory can be configured with L1; on AMD it’s called LDS. Visibility limited to threads in the same block; synchronization uses `__syncthreads` (block barrier) and more granular fences like `__threadfence_block`.

Banking and conflict analysis drive throughput decisions. For word-sized accesses the bank index mapping is

$$[H]\text{bank} = \left\lfloor \frac{\text{byte_address}}{\text{word_size}} \right\rfloor \bmod B, \quad (1.4)$$

where B is the number of banks. When N_w threads in a warp access addresses mapping to the same bank but different words, accesses serialize; if all threads read the same word, the hardware can broadcast.

Occupancy interacts with shared-memory allocation. The practical occupancy per SM is

$$[H]\text{occupancy} = \min \left(\left\lfloor \frac{T_{\text{SM}}}{T_{\text{block}}} \right\rfloor, \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}} T_{\text{block}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{block}}} \right\rfloor \right), \quad (1.5)$$

where T_{SM} is SM thread capacity, R_{SM} register count, S_{SM} shared memory per SM, and S_{block} shared per block. Choose S_{block} and R_{thread} to avoid reducing active warps needed to hide latency.

Implementation sketch (CUDA-style) showing per-block shared staging, barrier, and atomic global commit:

Listing 1.3: Shared staging and atomic commit in a GPU kernel

```
__global__ void tile_accumulate(float* globalOut, const float* globalIn, int N) {
    extern __shared__ float sdata[]; // dynamic shared memory (per-block)
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int lane = threadIdx.x % warpSize;

    // Load into shared memory tile (coalesced from global)
    if (tid < N) sdata[threadIdx.x] = globalIn[tid]; else sdata[threadIdx.x] = 0.0f;
    __syncthreads(); // ensure tile is ready for all threads

    // Parallel reduce within block (simple tree)
    for (int stride = blockDim.x/2; stride > 0; stride >>= 1) {
        if (threadIdx.x < stride) sdata[threadIdx.x] += sdata[threadIdx.x + stride];
        __syncthreads(); // synchronize at each reduction step
    }

    // Thread 0 atomically updates global accumulator
    if (threadIdx.x == 0) {
        atomicAdd(&globalOut[0], sdata[0]); // global atomic; may need
        __threadfence() when mixing with other scopes
    }
}
```

Design implications and trade-offs:

- Use shared memory to maximize arithmetic intensity and minimize TMU/DRAM pressure; avoid large per-block S_{block} that reduces occupancy via Eq. (2).
- Align and pad data to prevent bank conflicts from Eq. (1); exploit broadcast-friendly patterns when possible.
- Use minimal global atomics and prefer block-local aggregation followed by one atomic to reduce contention and ROP/L2 pressure.
- Choose cache/shared memory allocation based on workload (matrix multiply benefits from large shared tiles; streaming tasks favor small shared memory with high occupancy).

1.1.4 Synchronization mechanisms

The previous subsections established the execution hierarchy and the memory scopes available to kernels; synchronization mechanisms are the glue that maps those scopes to correct visibility and ordering across warps, thread blocks (CTAs), and entire device execution. This subsection analyzes common GPU synchronization primitives, their hardware realization, and practical performance models for SM-level and device-level coordination.

Modern GPUs provide synchronization at several granularities:

- Warp-level primitives such as shuffle and ballot and explicit lane rendezvous via `__syncwarp()`, which avoid global state and are implemented by SM-local data-path wiring.
- Block-level barriers via `__syncthreads()`, realized by an on-SM barrier unit that counts arrivals and issues a release when the active-thread count equals the block size.
- Memory fences (`__threadfence_block`, `__threadfence_device`, `__threadfence_system`) that enforce ordering and visibility across local shared memory, L1/L2 caches, and system memory respectively.
- Atomic read-modify-write primitives (e.g., `atomicAdd`, compare-and-swap) that serialize conflicting accesses in a hardware atomic unit, often at cacheline granularity.
- Kernel- and grid-level coordination via cooperative launch and cooperative groups; otherwise inter-block synchronization relies on kernel termination or software protocols.

Operational model and simple performance equations clarify costs. A block barrier completes when all threads arrive; if t_i is arrival time of thread i and N is block size, the barrier completion time is

$$[H]T_{\text{barrier}} = \max_{i \in [1..N]} t_i + T_{\text{release}}, \quad (1.6)$$

where T_{release} is the hardware notify latency. For atomic contention on a single address, serialized execution implies expected latency

$$[H]E[L] \approx L_0 + (c - 1) \cdot L_{\text{serial}}, \quad (1.7)$$

with c contending threads, baseline latency L_0 , and per-serial-step cost L_{serial} .

Hardware implementations use:

- Per-SM barrier counters and wake-up networks, minimizing cross-SM communication for `__syncthreads()`.
- Atomic units colocated with L1/L2/cache slice controllers that perform read-modify-write and update directory state for coherence across SMs.
- Memory-fence paths that flush or mark cachelines and generate DMA/coherence traffic for system-wide visibility.

Example CUDA kernel showing common primitives and a safe publish pattern using a device fence:

Listing 1.4: Block reduction with device fence and atomic publish.

```

#include
__global__ void reduce_and_publish(int *data, int N, int *out_counter) {
    extern __shared__ int sdata[]; // per-block shared memory
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int lane = threadIdx.x;
    // load
    sdata[lane] = (tid < N) ? data[tid] : 0;
    __syncthreads(); // block-level barrier
    // simple tree reduction in shared memory
    for (int offset = blockDim.x/2; offset>0; offset>>=1) {
        if (lane < offset) sdata[lane] += sdata[lane+offset];
        __syncthreads();
    }
    if (lane==0) {
        // publish block result to global memory, ensure visibility to other
        // blocks/host
        int val = sdata[0];
        atomicAdd(out_counter, val); // serialized at global granularity
        __threadfence_system(); // ensure published value visible system
        //wide
    }
}

```

Design implications and trade-offs:

- Block barriers are cheap inside an SM but cannot synchronize across SMs without cooperative features or kernel boundaries, so algorithms should avoid inter-block dependencies when possible.
- Atomics provide correctness but serialize hot addresses; use reduction trees in shared memory to reduce contention.
- Wider memory-fence scopes (device/system) increase latency and coherence traffic; choose the smallest fence consistent with correctness.
- Hardware complexity (barrier units, atomic units, coherence directories) trades area and power for programmability and correctness guarantees.

1.2 Compute Dispatch

1.2.1 Kernel launch process

Building on the execution model that defines kernels, threads and work-groups, the kernel launch process turns a high-level dispatch into per-SM resource reservations and warp streams that the hardware can execute. This subsection analyzes the packetized command flow, resource arbitration, and the constraints that determine how many work-groups are resident per SM.

Problem statement: convert a host-side kernel invocation into concrete allocations of registers, shared memory, and warp slots across multiple SMs while honoring QoS, context isolation, and concurrency requirements.

Analysis:

- Command ingestion: the host driver writes a command packet into a submission queue. The GPU command processor validates the packet, performs address translations, and enqueues a dispatch entry for the compute dispatch unit.
- Packet contents typically include: kernel identifier, entry offset, grid dimensions, block dimensions, per-block dynamic shared memory, per-thread register footprint, and launch flags (priority, coop groups, stream id).
- Resource eligibility: for each SM the allocator computes how many blocks can be resident using the minimum of three constraints:

$$B_{SM} = \min \left(\left\lfloor \frac{R_{SM}}{r_{block}} \right\rfloor, \left\lfloor \frac{S_{SM}}{s_{block}} \right\rfloor, \left\lfloor \frac{W_{SM}}{w_{block}} \right\rfloor \right), (1.8)$$

where R_{SM} is registers per SM, r_{block} is registers required per block, S_{SM} is shared memory per SM, s_{block} is shared memory per block, W_{SM} is maximum concurrent warps per SM, and w_{block} is warps per block.

- Scheduling: the work-group scheduler issues blocks to SMs via a work-stealing or round-robin policy, considering locality (memory-affinity and page residency) and concurrent kernel fairness.

Implementation (practical dispatcher example): the following host-side packet builder shows fields the driver must populate; the GPU-side parser implements a small FSM that atomically reserves resources and returns a dispatch token before threads are created.

Listing 1.5: Simple host-side command packet builder for kernel dispatch

```
struct KernelPacket {
    // serialized to submission queue
    uint64_t kernel_id;           // kernel code identifier
    uint32_t grid_x, grid_y, grid_z; // grid dims
    uint16_t block_x, block_y, block_z; // block dims
    uint32_t shmem_bytes;         // dynamic shared mem per block
    uint16_t regs_per_thread;     // ABI-reported reg usage
    uint8_t priority;             // stream/priority hint
    uint64_t arg_buffer_addr;     // device pointer to args
};

void submit_kernel(KernelPacket pkt) {
    // push to ring buffer and notify command processor (doorbell)
    ring_push(pkt);               // enqueue packet
    doorbell_ring();              // notify GPU
}
```

Operational implications and trade-offs:

- Aggressive register allocation increases throughput per warp but reduces B_{SM} , lowering concurrency and latency hiding.
- Larger dynamic shared memory boosts data reuse for matrix/ML kernels but constrains the number of resident blocks, affecting occupancy for small kernels.
- Packet format simplicity reduces command processor complexity but limits runtime flexibility (fewer dynamic reconfigurations).
- Hardware support for cooperative groups and grid-level barriers requires the dispatcher to reserve all blocks atomically, making launch latency and resource fragmentation critical design metrics.

1.2.2 Command packet format

The kernel launch discussion established what parameters the driver provides to the GPU; this subsection specifies how those parameters are encoded on the command bus so the dispatch unit and SM/CU-level schedulers can consume them efficiently.

A command packet must pack a fixed header for deterministic parsing plus optional payload words for variable-sized argument buffers or descriptor lists. Key design goals are: minimal header words for low-latency dispatch, byte-aligned GPU virtual addresses (commonly 48-bit canonical), and explicit fields for execution granularity (grid and work-group dimensions), resource requirements (registers, shared memory), and synchronization semantics. A practical header layout uses a fixed N_w -word header where payload length is expressed in 32-bit words; the total packet length is

$$[H]W_{\text{total}} = N_w + \left\lceil \frac{S_{\text{payload}}}{4} \right\rceil \quad (1.9)$$

with S_{payload} in bytes. This supports quick buffer allocation and DMA prefetch by the copy engine or DMA engine prior to SM scheduling.

Typical header fields (one possible real-world mapping):

- 32-bit opcode/version/flags (bitfields for priority, sync/fence type, concurrent-kernel hint).
- 16-bit kernel_id + 16-bit header_word_count.
- Grid dims: three 32-bit words (grid_x, grid_y, grid_z).
- Block dims: three 16-bit fields packed into one 32-bit word (block_x, block_y, block_z).
- Shared memory size (32-bit).
- Argument buffer GPU pointer (48-bit split into two 32-bit words).
- Resource mask and affinity (SM/CU/NUMA hints, 32-bit).

Parsing must be stream-friendly and resilient to variable-length payload. The parser presents a ready/valid interface, emits a decoded metadata word to the work-group scheduler, and streams payload to a DMA or argument buffer unit.

Below is a synthesizable Verilog implementation of a minimal stream parser that decodes a 7-word fixed header and asserts `meta_valid` when decoded metadata is ready. Ports use ready/valid semantics for backpressure.

Listing 1.6: Command packet parser (fixed 7-word header) implemented as a ready/valid stream consumer.

```

module cmd_packet_parser (
    input wire      clk,
    input wire      rst_n,
    // input stream (AXI4-Stream style)
    input wire      in_valid,
    output reg       in_ready,
    input wire [31:0] in_data,
    input wire      in_last, // marks end of payload (optional)
    // decoded metadata output
    output reg       meta_valid,
    input wire      meta_ready,
    output reg [15:0] kernel_id,
    output reg [31:0] grid_x, grid_y, grid_z,
    output reg [15:0] block_x, block_y, block_z,
    output reg [31:0] shared_bytes,
    output reg [47:0] arg_ptr
);

    // FSM states
    localparam IDLE=0, HDR=1, PAYLOAD=2, DONE=3;
    reg [1:0] state;
    reg [3:0] hdr_cnt;
    reg [31:0] tmp_word0;
    reg [31:0] tmp_word1;

    // Header word count fixed to 7 for this parser
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= IDLE; in_ready <= 1'b0; hdr_cnt <= 0;
            meta_valid <= 1'b0;
        end else begin
            case (state)
                IDLE: begin
                    meta_valid <= 1'b0;
                    if (in_valid) begin
                        in_ready <= 1'b1; hdr_cnt <= 1;
                        tmp_word0 <= in_data; // opcode/flags
                        state <= HDR;
                    end else begin

```

```

        in_ready <= 1'b0;
    end
end
HDR: begin
    if (in_valid && in_ready) begin
        hdr_cnt <= hdr_cnt + 1;
        case (hdr_cnt)
            1: begin tmp_word1 <= in_data; kernel_id <= in_data[15:0]; end
            2: grid_x <= in_data;
            3: grid_y <= in_data;
            4: grid_z <= in_data;
            5: begin block_x <= in_data[11:0]; block_y <= in_data[23:12];
                block_z <= in_data[31:24]; end
            6: shared_bytes <= in_data;
            7: begin arg_ptr[31:0] <= in_data; end
        endcase
        if (hdr_cnt==7) begin
            state <= PAYLOAD; in_ready <= 1'b1;
        end
    end
end
PAYLOAD: begin
    // consume or forward payload; here we just wait for in_last
    if (in_valid && in_last) begin
        in_ready <= 1'b0;
        state <= DONE;
    end
end
DONE: begin
    if (!meta_valid && meta_ready) begin
        // second half of arg_ptr would be provided by another header word
        // in real packet
        meta_valid <= 1'b1;
        state <= IDLE;
    end
end
endcase
end
endmodule

```

Design implications and trade-offs:

- Fixed small header (low N_w) minimizes dispatch latency but limits extensibility; variable headers require versioning bits.
- 48-bit address fields cost header words; compressed handles (indices to a descriptor table) reduce header width at the cost of extra lookups.

- Strong alignment and word-count fields enable simple stream parsers in dispatch hardware, improving throughput to SM/CU but requiring driver-side packing discipline.
- Supporting concurrent kernels needs per-packet priority and affinity fields, which increase header overhead but improve overall utilization and QoS.

1.2.3 Work-group distribution

The command packet delivers the kernel descriptor and grid dimensions to the dispatch unit, and the kernel launch routine converts those into per-work-group descriptors and resource footprints used by the scheduler. That per-work-group metadata (work-group size, register and shared-memory requirements) is the primary input to the distribution problem that follows.

The problem is to map G work-groups onto S streaming multiprocessors (SMs, also called compute units or CUs) to maximize SIMT throughput while respecting per-SM resource constraints and maintaining good memory/TMU locality. Key constraints and objectives:

- Resource limits per SM: registers, shared (L1) memory, warp-slot count, and hardware limit on concurrent work-groups W_{hw} .
- Load balancing to avoid idle SMs and to hide DRAM latency.
- Affinity to improve cache and TMU reuse when multiple work-groups touch nearby data.

A compact capacity model per SM is

$$[H]C_{SM} = \min\left(\left\lfloor \frac{R_{SM}}{R_{wg}} \right\rfloor, \left\lfloor \frac{S_{SM}}{S_{wg}} \right\rfloor, W_{hw}\right), \quad (1.10)$$

where R_{SM} is registers per SM, R_{wg} is registers required per work-group, S_{SM} is shared memory per SM, S_{wg} is shared memory per work-group, and W_{hw} is the hardware maximum concurrent work-groups. The scheduler must ensure that for any assignment the sum of resource usage per SM never exceeds the physical budget.

Implementation approaches with operational tradeoffs:

- Static round-robin: compute $\lceil G/S \rceil$ assignments, cheap but ignores resource heterogeneity and dynamic runtime variation.
- Centralized queue: a global work-group queue with per-SM bookkeeping; best for fairness and dynamic balancing but needs scalable arbitration and low-latency atomics.
- Distributed freelists: each SM pulls from a shared pool or per-tile queues to reduce contention and improve locality.

A practical centralized dispatch loop (production-quality C-style pseudocode used in runtime firmware) performs atomic dequeue and per-SM resource checks before assigning a WG:

Listing 1.7: Centralized work-group dispatcher (simplified)

```
int dispatch_loop(int G, WorkGroup *queue, SMState *sms, int S) {
    // queue: work-group descriptors; sms: per-SM resource state
    while (atomic_fetch(&remaining) > 0) {
        int wg_id = atomic_dequeue(queue);           // pop next work-group
```

```

for (int sm = 0; sm < S; ++sm) {
    if (try_acquire_resources(&sms[sm], wg_id)) {    // attempt resource
        accounting
        assign_wg_to_sm(sm, wg_id);                // handshake to SM
        break;
    }
}
// fall back: re-enqueue or stall until resources free (backoff)
}
return 0;
}

```

Operational considerations and trade-offs:

- Centralized dispatch with atomic dequeue scales with hardware atomics and queue depth; implement with lockless rings and per-bank atomics to avoid head contention.
- Favor affinity packing when S_{wg} is large to preserve shared-memory locality; pack multiple adjacent WGs on the same SM when cache/TMU reuse dominates.
- Preemptive rebalancing (migrating WGs) is costly; design for non-preemptive short-lived WGs or hardware checkpointing for long kernels.

Design implications: choose a distribution policy that balances low-latency dispatch arbitration against cache/TMU locality gains. Resource-aware capacity checks per Eq. (1.30) are essential to guarantee correctness and predictable occupancy; implementing per-SM freelists reduces global contention but increases implementation complexity and bookkeeping overhead.

1.2.4 Concurrent kernel execution

The previous discussion of command packets and work-group distribution established how launch metadata and per-work-group placement reach the dispatcher; concurrent kernel execution extends that by multiplexing multiple such launches onto a fleet of SMs while enforcing per-SM resource constraints and minimizing interference between SM-level warp schedulers.

Concurrent execution problem: multiple kernels (contexts) arrive with different resource footprints (registers per thread, shared memory per work-group, warps per work-group) and different total work (work-groups W_i). The dispatcher must:

1. Admit a kernel only if overall device-side tables have capacity for its context and metadata (queueing).
2. Budget and assign work-groups to SMs subject to per-SM limits (register file, shared memory, maximum resident warps).
3. Balance fairness and throughput (prioritize latency-sensitive kernels or maximize aggregate throughput for batch jobs).

Analysis—simple performance model: treat each SM as a service node with capacity C measured in resident work-group slots (determined by registers and SMEM). If kernel i is allocated fraction a_i of total SM capacity ($\sum_i a_i \leq 1$), its completion time equals its total work

divided by service rate:

$$[H]T_i = \frac{W_i}{a_i \cdot N_{\text{SM}} \cdot \mu} \quad (1.11)$$

where N_{SM} is number of SMs and μ is the per-SM work-group throughput (work-groups/sec) under steady-state SIMT scheduling. This highlights trade-offs:

- Increasing a_i (more SM share) reduces latency for kernel i but can hurt others.
- Nonlinear resource packing (registers/SMEM) can make a_i discrete; fragmentation reduces effective a_i .

Implementation—synthesizable dispatcher example: the Verilog below is a compact, production-ready dispatcher that:

- accepts kernel launches into a context table,
- performs per-SM resource accounting,
- issues work-group dispatch when an SM has capacity.

Listing 1.8: Simple concurrent-kernel dispatcher (synthesizable).

```

module concurrent_kernel_dispatch #(
  parameter NUM_SMS = 8,
  parameter MAX_CTXS = 16,
  parameter ID_WIDTH = 32,
  parameter WG_WIDTH = 16
) (
  input wire          clk,
  input wire          rst,
  // launch interface
  input wire          launch_valid,
  input wire [ID_WIDTH-1:0] launch_id,
  input wire [WG_WIDTH-1:0] launch_wg_total,
  input wire [15:0]      launch_regs_per_wg,
  input wire [15:0]      launch_smem_per_wg,
  output reg          launch_accept,
  // SM status inputs (free_slot indicates can accept another WG)
  input wire [NUM_SMS-1:0] sm_free_slot,
  // dispatch outputs (one-hot per SM with context id)
  output reg [NUM_SMS-1:0] dispatch_valid,
  output reg [NUM_SMS*ID_WIDTH-1:0] dispatch_ctx_id
);

// Context table
reg [ID_WIDTH-1:0] ctx_id   [0:MAX_CTXS-1];
reg [WG_WIDTH-1:0] ctx_rem  [0:MAX_CTXS-1];
reg [15:0]         ctx_regs [0:MAX_CTXS-1];
reg [15:0]         ctx_smem [0:MAX_CTXS-1];
reg                ctx_used [0:MAX_CTXS-1];

```



```

integer i, s;
// accept new launch if a free context slot exists
always @(posedge clk) begin
  if (rst) begin
    launch_accept <= 0;
    for (i=0;i0) begin
      // allocate one WG to SM s
      ctx_rem[i] <= ctx_rem[i] - 1;
      dispatch_valid[s] <= 1;
      dispatch_ctx_id[s*ID_WIDTH +: ID_WIDTH] <= ctx_id[i];
      // free context when done
      if (ctx_rem[i]==1) ctx_used[i] <= 0;
      disable for; // stop scanning contexts for this SM
    end
  end
end
end
end
end
endmodule

```

Implications and trade-offs:

- Simple per-WG allocation avoids costly preemption but suffers fragmentation when kernels request large SMEM or register footprints.
- Fairness policies (round-robin, weighted) are orthogonal to this core; choosing weights affects latency vs throughput per equation (1.11).
- Hardware support for finer-grain packing (multiple WGs per SM) and runtime defragmentation improves utilization but increases allocator complexity and metadata size.

1.3 Occupancy and Resource Management

1.3.1 Register pressure and allocation

Building on the dispatch and work-group placement policies discussed earlier, register pressure is the primary per-thread resource that directly limits how many warps an SM can keep resident and thereby determines latency hiding in SIMT execution.

Register allocation problem: each SM contains a finite register file capacity R_{SM} (total physical registers). Compilers allocate $r_{\text{per_thread}}$ registers per thread based on live ranges; hardware assigns these in units that map to threads/warps. The number of threads limited by registers is

$$[H]N_{\text{threads,reg}} = \left\lfloor \frac{R_{\text{SM}}}{r_{\text{per_thread}}} \right\rfloor, \quad \text{occupancy} = \frac{\min(N_{\text{threads,reg}}, N_{\text{threads,max}}, B \cdot t)}{N_{\text{threads,max}}}, \quad (1.12)$$

where $N_{\text{threads,max}}$ is the SM hardware limit, B is blocks per SM, and t is threads per block. Equation (1) explicitly shows how increasing $r_{\text{per_thread}}$ reduces resident threads and therefore reduces the number of active warps available to hide latency.

Analysis: high register pressure leads compilers to spill variables to local memory; on modern GPUs spilled local memory accesses go through the L1/L2 hierarchy and can serialize with global accesses, adding instruction overhead and memory bandwidth pressure. For compute kernels (e.g., GEMM or convolution) this can reduce arithmetic intensity and ROP/TMU utilization, turning compute-bound code into memory-bound. Conversely, overly aggressive register reduction (via compiler flags or manual refactor) can increase instruction count or force extra synchronization when using shared memory as a replacement.

Implementation techniques engineers use:

- Reduce live ranges: scope temporaries tightly or split long kernels into stages so values die earlier.
- Replace per-thread large arrays with tiled shared memory (LDS) to trade registers for on-chip shared memory when shared memory is abundant and contention is manageable.
- Use compiler controls: pass `-maxrregcount` to `nvcc` or annotate kernels with `__launch_bounds__` to guide allocation.
- Refactor math-heavy inner loops to use fused multiply-adds and reuse registers rather than allocate new temporaries.

Example (CUDA) showing a per-thread scratch array moved to shared memory and a launch bounds hint:

Listing 1.9: Reducing register pressure by relocating scratch data to shared memory and applying launch bounds.

```
__global__ __launch_bounds__(256,4) void kernel(float *A, int N){
    __shared__ float s_tile[256];           // shared scratch reduces per-thread
        registers
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    float acc = 0.0f;
    if (tid < N) {
        s_tile[threadIdx.x] = A[tid];       // move temporary storage to shared
            memory
        __syncthreads();
        // use shared array to accumulate values
        for (int i = 0; i < 256; ++i)
            acc += s_tile[i];
        A[tid] = acc;
    }
}
```

1.3.2 Shared memory partitioning

Continuing from register allocation effects on occupancy, shared memory sizing and partitioning is the other major per-SM resource that limits concurrent warps and blocks. The allocator

policy for on-chip shared memory can be the dominant factor in achievable occupancy for tiling and ML micro-kernels that use block-local buffers.

Problem: a fixed pool of shared memory per SM must be partitioned among active cooperative-thread-arrays (CTAs or blocks). Analysis begins by expressing the maximum resident blocks per SM as the minimum of the per-resource limits:

$$[H]N_{\text{blocks}} = \min\left(\left\lfloor \frac{R_{\text{SM}}}{R_{\text{per_block}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{per_block}}} \right\rfloor, N_{\text{max}}\right), \quad (1.13)$$

where R denotes registers, S denotes shared bytes, and N_{max} is architectural block concurrency. For example, with $S_{\text{SM}} = 64$ KiB and $S_{\text{per_block}} = 16$ KiB, the shared-memory limit is four blocks.

Analysis: static fixed-slot partitioning simplifies allocator hardware and avoids fragmentation, but imposes quantization loss when kernels request non-multiples of slot size. Dynamic allocation gives better utilization but requires allocation metadata and may increase allocation latency or complexity of per-block retirement housekeeping. Bank conflicts remain orthogonal but interact: more, smaller partitions reduce intra-block bank pressure, while larger contiguous allocations improve bulk DMA into shared memory for matrix multiplication tiles feeding tensor cores.

Implementation: a synthesizable per-SM fixed-slot allocator can be implemented as a stack of free slot indices. This enables single-cycle allocate/free operations in common cases and bounded logic for verification. The module below is minimal but production-ready for FPGAs/ASIC RTL flows; it supports parameterized slot count and returns a slot index on grant.

Listing 1.10: Compact per-SM shared-memory slot allocator (synthesizable).

```

module sm_slot_allocator #(
  parameter SLOT_COUNT = 8,
  parameter SLOT_BITS   = $clog2(SLOT_COUNT)
) (
  input  wire      clk,
  input  wire      rst_n,
  input  wire      alloc_req,    // request one slot
  input  wire      free_req,     // free one slot
  input  wire [SLOT_BITS-1:0] free_idx, // index to free
  output reg       alloc_gnt,    // grant valid
  output reg [SLOT_BITS-1:0] alloc_idx, // granted index
  output wire      empty,       // all slots used
  output wire      full        // all slots free
);

reg [SLOT_BITS-1:0] stack [0:SLOT_COUNT-1];
reg [$clog2(SLOT_COUNT+1)-1:0] sp; // stack pointer (count)
integer i;

// initialize stack with all indices
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    sp <= SLOT_COUNT;
    for (i = 0; i < SLOT_COUNT; i = i + 1)

```

```

    stack[i] <= i[SLOT_BITS-1:0];
    alloc_gnt <= 1'b0;
end else begin
    alloc_gnt <= 1'b0;
    if (alloc_req && sp > 0) begin
        sp <= sp - 1;
        alloc_idx <= stack[sp - 1];
        alloc_gnt <= 1'b1;
    end else if (free_req && sp < SLOT_COUNT) begin
        stack[sp] <= free_idx;
        sp <= sp + 1;
    end
end
end
end

assign empty = (sp == 0);
assign full  = (sp == SLOT_COUNT);
endmodule

```

1.3.3 Warp and block occupancy

The previous subsections established how shared memory partitioning and register allocation constrain per-block resource footprints; this subsection continues by quantifying how those footprints translate into warp and block occupancy on an SM and why that matters for latency hiding and throughput.

Occupancy measures the fraction of the SM's warp slots that are occupied by ready-to-run warps. Operationally, higher occupancy increases the SM's ability to hide long-latency events (global memory, texture fetches, or synchronization), but it is not a sole predictor of performance. Occupancy is bounded by three primary resource constraints: registers, shared memory (LDS), and the hardware limits on warps and blocks per SM. For a kernel with `threadsPerBlock` = T , registers per thread r , and shared memory per block s , define warps per block $w_b = T/W_s$ where W_s is warp size (typically 32). The maximum resident blocks due to each resource are:

$$\begin{aligned}
 B_{\text{reg}} &= \left\lfloor \frac{R_{\text{SM}}}{r \cdot T} \right\rfloor, \\
 [H] B_{\text{sh}} &= \left\lfloor \frac{S_{\text{SM}}}{s} \right\rfloor, \\
 B_{\text{hw}} &= B_{\text{SM}}^{\text{max}},
 \end{aligned} \tag{1.14}$$

where R_{SM} is the total registers per SM, S_{SM} is shared memory per SM, and $B_{\text{SM}}^{\text{max}}$ is the hardware maximum resident blocks. The resident block count is $B_{\text{res}} = \min(B_{\text{reg}}, B_{\text{sh}}, B_{\text{hw}})$, yielding active warps $W_{\text{act}} = B_{\text{res}} \cdot w_b$. Occupancy is

$$[H]\text{occupancy} = \frac{W_{\text{act}}}{W_{\text{SM}}^{\text{max}}}. \tag{1.15}$$

Example: $R_{\text{SM}} = 65536$, $S_{\text{SM}} = 65536$ bytes, $W_{\text{SM}}^{\text{max}} = 64$, $B_{\text{SM}}^{\text{max}} = 16$, $W_s = 32$. For a kernel with $T=256$, $r = 32$, $s = 16$ kB: $w_b = 8$, $B_{\text{reg}} = 8$, $B_{\text{sh}} = 4$, so $B_{\text{res}} = 4$, $W_{\text{act}} = 32$,

occupancy = $32/64 = 0.5$ (50

To evaluate occupancy programmatically use a small utility that mirrors the hardware calculation:

Listing 1.11: SM occupancy calculator (simple model)

```
#include
int calc_occupancy(int Rsm,int Ssm,int Wsm_max,int Bsm_max,
                  int threadsPerBlock,int regPerThread,int shmemPerBlock){
    int warpSize = 32;
    int warpsPerBlock = threadsPerBlock/warpSize;           // assume
    divisible
    int B_reg = Rsm / (regPerThread * threadsPerBlock);      // regs
    limit
    int B_sh = Ssm / shmemPerBlock;                          // shared
    limit
    int B_res = B_reg < B_sh ? B_reg : B_sh;                 // min of
    resource limits
    if(B_res > Bsm_max) B_res = Bsm_max;                     //
    hardware block cap
    int activeWarps = B_res * warpsPerBlock;
    return (activeWarps * 100) / Wsm_max;                    //
    occupancy % (integer)
}
/* use as: calc_occupancy(65536,65536,64,16,256,32,16384); */
```

Implications and trade-offs:

- For memory-bound kernels, raising occupancy improves latency hiding until the compute pipeline or memory bandwidth saturates; beyond that, extra occupancy yields diminishing returns.
- For compute-bound kernels, reducing register pressure (smaller r) may allow more blocks, but may force register spilling with large performance penalties; measure spill-induced memory traffic before trading registers for occupancy.
- Shared memory tiling that increases s may increase data locality but reduce B_{res} ; tune tile size to balance reuse against occupancy loss.
- Scheduler latency hiding works at warp granularity; achieving many small resident warps often helps when global memory latency dominates, whereas fewer high-ILP threads may serve compute-heavy workloads better.

Design implication: hardware and compiler co-optimization should expose knobs to trade registers per thread and per-block shared memory while providing occupancy-aware heuristics so that kernels target the sweet spot between latency hiding and avoiding spilling or excessive resource fragmentation.

1.3.4 Resource scheduling

The previous analysis of warp and block occupancy and the shared memory partitioning constraints motivates a scheduler that jointly reasons about registers, SM-local memory, and block-

/warp granularity. Practical schedulers must translate static resource requests into dynamic resident allocations while minimizing fragmentation and maximizing SIMT utilization.

Problem statement: decide how many blocks to admit per SM and which warps to issue, given per-block register usage R_{blk} , per-block shared memory S_{blk} , and fixed hardware limits R_{SM} , S_{SM} , and B_{max} . Analysis begins by expressing the basic resource-limited resident-block calculation:

$$[H]B_{\text{res}} = \min\left(\left\lfloor \frac{R_{\text{SM}}}{R_{\text{blk}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{blk}}} \right\rfloor, B_{\text{max}}\right). \quad (1.16)$$

Operationally, occupancy is more meaningful in warp terms. Let T_{blk} be threads per block and W_{size} be warp width (usually 32). Define maximum warps per SM as $W_{\text{SM,max}}$. Then effective occupancy fraction is

$$[H]\eta = \frac{B_{\text{res}} \cdot T_{\text{blk}}}{W_{\text{SM,max}} \cdot W_{\text{size}}}. \quad (1.17)$$

These formulas assume perfect packing and ignore fragmentation. Real hardware faces two additional constraints: (1) banked register files and shared memory partition alignment cause internal fragmentation, reducing usable resources by a factor ϕ ; (2) concurrent kernels may need fair division, requiring QoS in allocation.

Implementation strategy: a two-stage allocator. Stage one computes an upper bound on resident blocks using (1) and (2). Stage two performs greedy packing at block granularity, considering alignment and preemption cost. Greedy selection favors blocks with small R_{blk} and S_{blk} or those that unlock many independent warps to tolerate memory latency.

The following listing shows a compact scheduler emulator used in microarchitecture exploration. It computes resident blocks per SM and performs a simple demand-driven dispatch. Comments indicate correspondence to hardware actions.

Listing 1.12: Resource-aware block admission and warp dispatch (emulator).

```
# Simple resource-aware scheduler emulator
def admit_blocks(R_SM, S_SM, B_max, blocks):
    # blocks: list of (R_blk, S_blk, T_blk)
    admitted = []
    r_avail, s_avail, b_avail = R_SM, S_SM, B_max
    # sort by smaller combined resource to reduce fragmentation
    blocks_sorted = sorted(blocks, key=lambda b: (b[0]+b[1]))
    for R_blk, S_blk, T_blk in blocks_sorted:
        if R_blk <= r_avail and S_blk <= s_avail and b_avail > 0:
            admitted.append((R_blk, S_blk, T_blk))
            r_avail -= R_blk; s_avail -= S_blk; b_avail -= 1
    return admitted

def dispatch_warps(admitted, W_size=32):
    # round-robin warp issue with simple backpressure
    warps = []
    for R_blk, S_blk, T_blk in admitted:
        num_warps = (T_blk + W_size - 1) // W_size
        warps.extend([num_warps]*1) # one entry per block (placeholder)
    return warps # emulator uses warp counts for latency hiding
```

Implications and trade-offs:

- Overcommitting registers can increase context-switch latency or force spilling, so conservative ϕ tuning avoids performance cliffs.
- Fine-grained warp scheduling improves latency hiding but increases scheduler complexity and wiring.
- Allocator fairness across concurrent kernels reduces peak throughput for single heavy kernels but improves overall system throughput under mixed workloads.

1.4 Work Distribution

1.4.1 Static and dynamic scheduling

Following dispatch and occupancy analysis, we now examine how work allocation policies affect SM utilization and end-to-end latency. Static and dynamic scheduling methods trade predictability for load balance, and choosing between them changes hardware complexity and runtime behavior.

Static scheduling fixes a mapping of work-groups (blocks) to SMs or partitions before or at kernel launch, enabling simple resource accounting and low scheduler overhead. Typical static strategies include:

- Round-robin assignment of equal-sized work-group ranges to each SM.
- Compiler-driven tiling that assigns contiguous index ranges to warps for predictable memory strides.
- Grid-stride loops in kernels to implement deterministic iteration without global coordination.

Static scheduling minimizes per-dispatch control: no global queues or atomic contention, and the hardware scheduler remains simple. The downside appears when per-task runtimes vary: imbalance causes some SMs to idle while others finish long-running tasks.

Dynamic scheduling places work items into runtime-managed queues, enabling work stealing or persistent threads to adapt to runtime variability. Dynamic models include:

- Hardware block scheduler that assigns blocks to SMs as resources free (typical in modern GPUs).
- Software-managed global work queue with persistent-thread kernels that dequeue tasks atomically.
- Hierarchical queues where CUs/SMs pull from local then global queues to reduce contention.

Dynamic scheduling improves load balance and latency for irregular workloads such as sparse linear algebra or ray traversal, at the cost of atomic operation overhead and possible contention on the queue or memory subsystem.

To quantify imbalance, let t_w be the runtime of work item w , and let S be the number of SMs. If static mapping partitions work into sets W_s , the makespan is

$$[H]T_{\text{static}} = \max_{s=1..S} \sum_{w \in W_s} t_w, \quad T_{\text{ideal}} = \frac{\sum_w t_w}{S}, \quad (1.18)$$

and the imbalance factor $I = T_{\text{static}}/T_{\text{ideal}} \geq 1$. Dynamic scheduling seeks to drive I toward 1 by redistributing remaining tasks at runtime.

A common practical implementation is the persistent-threads pattern. The example shows a CUDA-style persistent kernel that atomically pops task indices from a global counter:

Listing 1.13: Persistent-thread dynamic scheduler using atomic fetch-add

```
__device__ int task_counter; // global task index // GPU global memory
__global__ void persistent_kernel(Task* tasks, int N) {
    while (true) {
        int idx = atomicAdd(&task_counter, 1); // dequeue next task
        if (idx >= N) break;                    // all tasks consumed
        process_task(tasks[idx]);               // compute, memory ops, TMU usage
    }
}
// launch with many persistent blocks and warps to cover latency
```

Design implications and performance trade-offs:

1. Static scheduling reduces scheduler hardware and atomic contention, benefiting throughput for regular workloads and high occupancy.
2. Dynamic scheduling lowers imbalance for irregular workloads, improving end-to-end latency but increasing atomic traffic and possible memory serialization.
3. Hybrid designs (local queues per SM + global overflow) balance contention versus locality, minimizing TMU/ROP interference while maximizing utilization.
4. Choose static when per-task variance is low and memory coalescing dominates; choose dynamic for sparse, unpredictable, or adaptive algorithms where I would otherwise be large.

1.4.2 Load balancing algorithms

Building on static and dynamic scheduler tradeoffs, load balancing algorithms supply per-SM and cross-SM mechanisms to equalize work when task sizes vary or execution is irregular. These algorithms operate at multiple granularities—warp, block (work-group), and SM—and must trade off balancing quality against synchronization and memory-system overhead.

Problem: minimize makespan (total kernel time) given N_{sm} streaming multiprocessors (SMs) and a set of tasks with work sizes $\{w_j\}$. Analysis centers on makespan $M = \max_i L_i$, where L_i is load assigned to SM i . The theoretical lower bound is the average load

$$[H]L_{\text{avg}} = \frac{\sum_j w_j}{N_{\text{sm}}} \quad (1.19)$$

and practical performance is governed by the imbalance factor $\beta = \max_i L_i / L_{\text{avg}}$, so achieved speedup relative to perfect balancing $\approx 1/\beta$.

Algorithms and mechanisms:

- Static partitioning: contiguous or round-robin assignment minimizes metadata but fails on high variance workloads (e.g., BFS frontier sizes or irregular sparse matrix rows).

- Dynamic queueing: a global or per-SM queue holds task descriptors; workers pop tasks, enabling fine-grain balancing at cost of atomic contention and increased L2 traffic.
- Work stealing: idle SMs steal work from busy SM queues; stealing cost s includes atomic RMW latency and possible cache misses. Total time model:

$$[H]T \approx \max_i L_i + s \cdot S \quad (1.20)$$

where S is number of successful steals.

- Hierarchical balancing: combine per-SM local queues and a global directory for coarse-grain redistribution to reduce cross-SM contention.
- Persistent threads model: a fixed pool of persistent blocks pull tasks from queues, reducing kernel-launch overhead for dynamic workloads.

Implementation example: compact device-side work-stealing loop suitable for persistent-thread kernels. Uses atomics and a circular local queue for low-latency pops; a single global steal pointer provides a simple victim selection policy.

Listing 1.14: Device-side persistent-thread work-stealing loop

```
__device__ void worker_loop(TaskQueue *localQ, GlobalQueues *G) {
    while (true) {
        Task t;
        if (localQ->pop(&t)) { // fast local pop (no global contention)
            execute_task(t);    // run compute or memory-bound work
        } else {
            unsigned victim = atomicAdd(&G->stealPtr, 1) % G->numQueues; // choose
                victim
            if (G->queues[victim].steal(&t)) { // atomic steal from victim queue
                execute_task(t);                // pay steal latency but keep SM
                    busy
            } else if (global_done(G)) break;    // termination check
            // else retry or backoff (exponential) to avoid contention
        }
    }
}
```

Operational considerations and trade-offs:

- Memory-bound tasks amplify steal cost s due to cache miss penalties; prefer coarse-grain tasks or data-affinity-aware stealing to reduce remote traffic.
- Atomic contention on global structures limits scalability; use hierarchical queues with per-SM freelists to amortize.
- For SIMT efficiency, assign tasks that map well to warp execution (regular inner loops); steal at block granularity to avoid warp divergence across tasks.
- Persistent-thread designs reduce kernel launch overhead and improve responsiveness to dynamic workloads, but require careful termination detection.

Design implications: choose balancing granularity to match workload variance and memory behavior; minimize global atomics via hierarchical queues; budget steal cost in performance models (Equation 1.20); and prefer affinity-aware policies for memory-intensive ML and graph kernels to reduce L2/DRAM traffic while keeping β close to one.

1.4.3 Persistent threads model

These ideas build directly on load-balancing and scheduling trade-offs: persistent threads convert dynamic scheduling into an on-chip task-stealing loop that reduces kernel-launch and dispatcher overhead while exposing new contention and resource-fragmentation considerations.

The problem: many small or irregular tasks (ray packets, fine-grained graph updates, adaptive tile shading) suffer from kernel-launch latency, CPU-driven queuing, and poor SM utilization when work sizes vary. Analysis: let N be the number of independent tasks, C the effective concurrency (resident threads across all SMs) and t_{task} the average service time per task. With a perfectly balanced persistent pool the ideal runtime is

$$[H]T_{\text{ideal}} \approx \frac{N t_{\text{task}}}{C} + T_{\text{overhead}}, \quad (1.21)$$

where T_{overhead} includes initial kernel launch and final drain costs. Persistent threads aim to maximize C for long periods and minimize T_{overhead} by keeping threads alive on each SM and letting them pull work from a shared queue or counter.

Implementation pattern (SM-side work-pull with chunking):

- Launch configuration: choose blocks so that the number of resident blocks approximates the number of SMs to hold a steady set of resident warps; this avoids excessive context swapping.
- Work acquisition: use an atomic counter or a work-queue; fetch work in chunks of size k to amortize atomic cost and reduce cache-coherence traffic.
- Termination: when fetched index $\geq N$ threads exit; optionally use cooperative-group or grid-level sync to finalize.

Example CUDA-style persistent kernel with chunked atomic fetch (simplified, production code needs error-checking and bounds tuning):

Listing 1.15: Persistent-thread work-pull kernel with chunking

```
__global__ void persistent_kernel(int N, int chunk, int *tasks, /* ... */) {
    extern __shared__ int sdata[]; // per-block scratch (optional)
    // Local worker id (warp or thread-level work mapping).
    int tid = threadIdx.x;
    // Single atomic counter in global memory (address passed as tasks[0]).
    int base;
    while (true) {
        // Each atomicAdd returns previous value; fetch a chunk of work.
        base = atomicAdd(&tasks[0], chunk); // amortize atomic per chunk
        if (base >= N) break;                // no more work
        int upper = min(base + chunk, N);
        for (int i = base + tid; i < upper; i += blockDim.x) {
```

```

    // process task i (compute, memory ops, etc.) -- keep independent.
    process_task(i); // inline compute or call device function.
}
// optional: __syncthreads() if using block-local accumulators.
}
}

```

Operational considerations and trade-offs:

1. Contention: small chunk k increases fairness but causes more atomic operations and cache-line ping-pong; large k reduces atomic rate but increases load imbalance. Tune k by measuring atomic latency and task variance.
2. Occupancy: persistent blocks pin resources (registers, shared memory) on each SM; over-allocating registers decreases C and hurts throughput. Balance register usage per thread against needed concurrency.
3. Divergence: persistent threads tolerate irregular per-task runtimes, but heavy branch divergence inside the processing loop wastes SIMT lanes; prefer warp-cooperative task processing when possible.
4. Correctness and termination: global counters must be sized for N ; prefer 64-bit counters if N exceeds 32-bit range on hardware that supports 64-bit atomics.

Design implications: persistent threads reduce host-side scheduling overhead and improve utilization for irregular workloads but transfer complexity to kernel-side synchronization and resource budgeting. Optimal performance requires tuning chunk size k , resident thread count C , and per-thread resource (register/shared memory) usage to trade off atomic contention vs. load balance while maintaining high SM occupancy.

1.4.4 Cooperative group synchronization

The persistent threads model and load-balancing approaches reduce idle SM time and even work distribution, but they increase the need for fine-grained, group-scoped coordination. Cooperative group synchronization provides the primitives and patterns to guarantee ordering and data visibility inside and across work-groups while respecting SM resource constraints.

Problem: ensure all threads in a logical group reach a rendezvous point with correct memory visibility without deadlock or starvation in a SIMT SM/warp scheduling environment. Analysis: on-SM barriers (block-level) can use fast shared memory and warp-level primitives to implement scalable barriers with low latency. Grid-level barriers across multiple SMs require either hardware support (cooperative launch with a global barrier network) or software algorithms using global atomics and spin-waiting. Software barriers carry two main risks: (1) deadlock if the number of active blocks is less than the number required by the algorithm (e.g., due to occupancy limits), and (2) high contention on a single atomics hotspot.

A common software barrier uses an atomic counter in global memory and a second-phase release, often implemented as:

$$[H]M = B \cdot C \quad (1.22)$$

where M is global memory used by the barrier, B is number of blocks, and C is bytes per-block state (typically 4–8). Time complexity depends on the algorithm:

- Centralized counter: contention cost roughly proportional to $T_{\text{atomic}} \cdot B$ under heavy spinning.
- Tree-based reduce/broadcast: latency scales like $O(\log B)$, approximate $T_{\text{barrier}} \approx T_{\text{atomic}} \log_2 B$.
- Hardware-assisted barrier: near-constant latency if implemented in the SM interconnect.

Implementation: use hardware cooperative groups when available; otherwise implement a scalable software barrier that avoids a single atomic hotspot and respects occupancy. Example CUDA code shows a grid-level cooperative sync when supported and a fallback global-atomic barrier otherwise:

Listing 1.16: Grid sync with cooperative groups and atomic fallback

```
#include
using namespace cooperative_groups;

__global__ void kernel_with_sync(int *global_bar, int num_blocks) {
    auto tb = this_thread_block();
    // block-local work uses shared memory synchronization
    __syncthreads(); // SM-level barrier
    // try hardware grid sync if launched cooperatively
    #ifdef __CUDA_ARCH__
    grid_group g = this_grid(); // may require cooperative launch
    g.sync(); // blocks all threads in grid (heap semantics)
    #else
    // Fallback: scalable two-phase atomic barrier (simplified)
    if (threadIdx.x==0) atomicAdd(global_bar, 1); // leader increments
    __syncthreads();
    while (*global_bar < num_blocks) { /* spin-wait*/ } // poll with backoff
    __syncthreads(); // ensure all threads see release
    #endif
}
```

Implications and trade-offs:

1. Prefer hardware cooperative groups for grid sync to avoid occupancy-related deadlocks and global-atomic contention.
2. If using software barriers, allocate small per-block state (C minimal) and use tree-based reduce/broadcast to reduce contention and lower T_{barrier} .
3. Design-level trade-offs include extra register/shared-memory per-block that reduces occupancy versus synchronization latency improvements; balance by quantifying occupancy: $\text{blocks_per_SM} = \left\lfloor \frac{\text{SM_shared}}{S} \right\rfloor$.
4. For ML workloads with tensor-core micro-batching, prefer intra-block tiling and warp-level sync to avoid expensive cross-SM coordination.

1.5 Data Parallel Processing Patterns

1.5.1 Map, reduce, and scan operations

Building on prior discussion of work distribution and persistent-thread scheduling, this subsection examines how map, reduce, and scan patterns map onto SIMT hardware and scheduler/SM resources for high throughput compute workloads.

Map operations are embarrassingly parallel: each thread performs an independent transformation on one or more elements, so the primary GPU concerns are memory coalescing and instruction-level utilization. Reductions and scans require coordination across threads and thus expose trade-offs between latency-hiding, shared memory capacity, and synchronization cost. Common use-cases include elementwise activation functions (map), global loss accumulation (reduce), and prefix sums for stream compaction (scan).

Problem → analysis: a binary-tree reduction across P threads can be implemented in $\log_2 P$ parallel steps, each combining pairs of partial sums. The step count is

$$[H]S = \lceil \log_2 P \rceil, \quad (1.23)$$

and the total arithmetic count for a full array of N elements remains $N - 1$ adds. Memory traffic dominates when global reads/writes outnumber arithmetic; a simple two-pass reduction reads N elements then writes one result, giving approximate traffic $8N$ bytes for FP32 reads plus writes. Arithmetic intensity (FLOPs per byte) is therefore low:

$$[H]AI \approx \frac{N - 1}{8N} \approx \frac{1}{8} \text{ FLOP/byte (FP32)}. \quad (1.24)$$

Improving AI requires reusing data in shared memory or combining with compute-heavy map work inside the same kernel.

Implementation: efficient GPU reductions use a hierarchical approach:

1. Each warp performs an intra-warp reduction using shuffle primitives to avoid shared-memory bank conflicts.
2. Warps deposit partial results into shared memory.
3. One warp finishes the inter-warp reduction.

The following CUDA kernel implements a block-level sum using warp shuffles and minimal shared memory:

Listing 1.17: Block-level reduction using warp shuffles and shared memory.

```
extern "C" __global__
void block_reduce_sum(const float* __restrict__ g_in, float* g_out, int N) {
    extern __shared__ float sdata[];                // partials per-warp
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    float val = (tid < N) ? g_in[tid] : 0.0f;        // load element (coalesced)
    // Warp-level reduction using shuffle down (mask full-warp)
    unsigned mask = 0xffffffffu;
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(mask, val, offset);
```

```

// thread lane 0 writes warp partial
int lane = threadIdx.x & 31;
int wid  = threadIdx.x >> 5;
if (lane == 0) sdata[wid] = val;
__syncthreads();
// final reduction by first warp
if (wid == 0) {
    val = (threadIdx.x < ((blockDim.x+31)/32)) ? sdata[threadIdx.x] : 0.0f;
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(mask, val, offset);
    if (threadIdx.x == 0) g_out[blockIdx.x] = val; // per-block result
}
}

```

Scans (prefix sums) typically use two-phase algorithms (upsweep/downsweep) or warp-efficient intrinsics; they emphasize synchronization and bank-aware shared memory layout to avoid serialization.

Design implications and trade-offs:

- Favor hierarchical reductions to minimize global atomics and L2 contention; perform per-block aggregation then reduce on SMs or with a second kernel.
- Use warp shuffle intrinsics to bypass shared-memory bank conflicts and reduce synchronization overhead.
- Shared memory usage reduces global memory traffic but lowers occupancy; balance the tile size for SM register/shared-memory budgets.
- For very large arrays, the extra kernel-launch latency for multi-stage reduction is offset by reduced memory traffic; choose parameters to maximize arithmetic intensity and maintain enough warps to hide global-memory latency.

1.5.2 Histogram and sort algorithms

Building on the scan and map/reduce primitives discussed previously, histogram and sort kernels exploit the same data-parallel building blocks (per-thread map, block-level reduction, and prefix-sum) but require additional attention to memory contention, synchronization, and data layout. These patterns are critical for graphics (tile classification, light binning) and ML pre-processing (feature quantization) where both throughput and deterministic ordering matter.

Problem: compute a histogram of N items into B bins or sort N keys efficiently on an SM. Analysis shows two dominant costs: contention when many threads update the same bin, and memory bandwidth for moving keys. A common mapping from a value x to bin index i is

$$[H]i = \left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} B \right\rfloor, \quad 0 \leq i$$

1.5.3 Matrix operations

Building on the reduction patterns and histogram/sort examples, matrix operations combine map and reduction across two spatial dimensions and are a canonical case of data-parallel tiling

and reuse. They expose high arithmetic intensity when implemented with blocking, making them ideal for SM-level shared memory and tensor-core acceleration.

The problem: compute a General Matrix Multiply (GEMM) $C = A \cdot B$ for $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$. Element-wise,

$$[H]C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}, \quad (1.26)$$

which is a reduction over k for each (i, j) . The challenge on a modern GPU SM is to maximize data reuse while avoiding shared-memory bank conflicts, register pressure, and warp divergence.

Analysis: Blocking converts the triple loop into tiles of size $T_m \times T_n$ with inner panels of length T_k . For a single tile, a threadblock cooperatively:

- loads a $T_m \times T_k$ sub-tile of A and a $T_k \times T_n$ sub-tile of B into shared memory;
- performs T_k -length multiply-accumulate (MAC) ops reusing loaded data;
- writes $T_m \times T_n$ outputs back to global memory.

Arithmetic intensity (flops per byte) for a blocked tile approximates

$$\text{AI} \approx \frac{2 T_m T_n T_k}{4(T_m T_k + T_k T_n + T_m T_n)} \quad (\text{single-precision bytes}=4).$$

Choosing T_m, T_n, T_k balances SM shared-memory size, register usage, and occupancy.

Implementation: a cooperative CUDA-style kernel shows the pattern: per-thread accumulators, shared-memory buffers, and synchronized loads. This is a production-style tiled GEMM (row-major) kernel sketch; it omits host setup for brevity but includes synchronization and bounds checks.

Listing 1.18: Tiled GEMM kernel using shared memory and warp-safe accumulation.

```
extern "C" __global__
void tiledGemm(const float* __restrict__ A,
               const float* __restrict__ B,
               float* __restrict__ C,
               int M, int N, int K) {
    const int TM = 64, TN = 64, TK = 16; // tunable tile sizes
    __shared__ float sA[TM][TK];          // shared-memory tiles
    __shared__ float sB[TK][TN];

    int blockRow = blockIdx.y, blockCol = blockIdx.x;
    int threadRow = threadIdx.y, threadCol = threadIdx.x;
    int globalRow = blockRow * TM + threadRow;
    int globalCol = blockCol * TN + threadCol;

    float acc = 0.0f; // per-thread accumulator

    for (int k0 = 0; k0 < K; k0 += TK) {
        int aCol = k0 + threadCol;
        int bRow = k0 + threadRow;
```

```

// cooperative load with bounds check
sA[threadRow][threadCol] = (globalRow < M && aCol < K)
    ? A[globalRow * K + aCol] : 0.0f;
sB[threadRow][threadCol] = (bRow < K && globalCol < N)
    ? B[bRow * N + globalCol] : 0.0f;

__syncthreads();

// compute partial tile
for (int k1 = 0; k1 < TK; ++k1)
    acc += sA[threadRow][k1] * sB[k1][threadCol];

__syncthreads();
}

if (globalRow < M && globalCol < N)
    C[globalRow * N + globalCol] = acc;
}

```

1.5.4 Prefix sums and reductions

These techniques pick up directly from matrix blocking and histogram accumulation: both rely on fast associative aggregation across lanes or threads, so efficient prefix sums and reductions are the primitive that implements those aggregations across an SM or work-group.

Problem — compute an associative prefix (scan) or reduction across a power-of-two SIMD lane group with minimal latency and predictable resource use. Analysis first contrasts two parallel algorithms used on GPUs: Hillis–Steele (inclusive, $S = \log_2 N$ parallel steps but $O(N \log N)$ work) and Blelloch (upsweep/downsweep, $2 \log_2 N$ parallel steps with $O(N)$ work). Practical hardware inside an SM often implements a tree-like scheme mapped to shared memory and SIMD writes to avoid bank conflicts and divergence. The dominant cost metric is the number of synchronization points and global/shared-memory transactions; minimizing these dominates throughput for graphics, compute, and ML kernels.

A compact performance model gives the number of parallel synchronization steps:

$$[H]S = \lceil \log_2 N \rceil, \quad (1.27)$$

with aggregate work scaling as $W = \Theta(N)$ for the Blelloch-style algorithm. For a simple iterative hardware block that executes one element update per cycle per step, total cycles approximate

$$[H]C \approx N \cdot S + C_{\text{overhead}} = N \log_2 N + O(N). \quad (1.28)$$

Implementation — the following synthesizable Verilog implements a parameterized inclusive prefix-scan engine that consumes a vector of lane inputs and produces per-lane prefix sums. It is designed for integration into a shared-memory controller or as an SM-local accelerator; it assumes power-of-two LANES and fixed WIDTH.

Listing 1.19: Compact parameterized prefix-scan unit (synthesizable).

```

module prefix_scan #(

```



```

parameter integer LANES = 8,
parameter integer WIDTH = 32
) (
    input wire          clk,
    input wire          rst,
    input wire          start,
    input wire [LANES*WIDTH-1:0] in_data,    // concatenated inputs
    output reg [LANES*WIDTH-1:0] out_data,    // concatenated prefix results
    output reg          done
);

function integer clog2(input integer v);
    integer i;
    begin i = 0; while ((1 << i) < v) i = i + 1; clog2 = i; end
endfunction
localparam STAGES = clog2(LANES);

reg [WIDTH-1:0] stage_data [0:STAGES][0:LANES-1];
integer s, i;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        done <= 1'b0;
    end else if (start) begin
        // load input
        for (i = 0; i < LANES; i = i + 1)
            stage_data[0][i] <= in_data[i*WIDTH +: WIDTH];
        // iterative scan
        for (s = 0; s < STAGES; s = s + 1)
            for (i = 0; i < LANES; i = i + 1)
                stage_data[s+1][i] <= (i >= (1 << s))
                    ? stage_data[s][i] + stage_data[s][i - (1 << s)]
                    : stage_data[s][i];
        // output final stage
        for (i = 0; i < LANES; i = i + 1)
            out_data[i*WIDTH +: WIDTH] <= stage_data[STAGES][i];
        done <= 1'b1;
    end else done <= 1'b0;
end
endmodule

```

1.6 Section 6: Verilog Implementation

1.6.1 Compute dispatch unit

This implementation continues from the kernel launch parsing and work-group scheduling logic, taking the scheduler's ready work-groups and performing hardware-safe matching against SM

resources and backpressure. The dispatch unit must ensure correctness under SIMT semantics while maximizing throughput and avoiding resource oversubscription.

Problem: assign work-groups to streaming multiprocessors (SMs) subject to register and shared-memory budgets, while preserving fairness and supporting valid/ready backpressure. Analysis shows the key constraints are per-SM available registers R_i and shared-memory S_i , and per-work-group requirements r_{wg} and s_{wg} . The admission condition for an SM i is $R_i \geq r_{wg}$ and $S_i \geq s_{wg}$. The maximum simultaneously resident work-groups per device, M , for a homogeneous work-group footprint simplifies to

$$[H]M = \min \left(\left\lfloor \frac{R_{tot}}{r_{wg}} \right\rfloor, \left\lfloor \frac{S_{tot}}{s_{wg}} \right\rfloor, N_{SM} \cdot W_{max} \right), \quad (1.29)$$

where R_{tot} and S_{tot} are total device resources and W_{max} is per-SM work-group concurrency.

Implementation: The RTL below implements a FIFO of incoming command packets, a round-robin selection engine that scans `NUM_SM` shader modules for a first-fit match meeting resource constraints, and a valid/ready dispatch handshake per module. The dispatcher consumes one FIFO entry per cycle when a module accepts the work-group, decrements resources via an external resource tracker (not included here), and outputs a one-hot select along with the packet payload.

Listing 1.20: Compact compute dispatch unit with FIFO and round-robin SM selection (synthesizable).

```

module compute_dispatch_unit #(
  parameter NUM_SM = 8,
  parameter FIFO_DEPTH = 16,
  parameter CMD_W = 64, // [63:48]=regs_req, [47:32]=smem_req, [31:0]=meta
  parameter REG_W = 16,
  parameter SMEM_W = 16
) (
  input  wire          clk,
  input  wire          rst,
  input  wire          cmd_valid,
  input  wire [CMD_W-1:0] cmd_data,
  output wire          cmd_ready,
  input  wire [NUM_SM*REG_W-1:0] sm_regs_avail,
  input  wire [NUM_SM*SMEM_W-1:0] sm_smem_avail,
  input  wire [NUM_SM-1:0] sm_ready,
  output reg [NUM_SM-1:0] dispatch_valid,
  output reg [NUM_SM*CMD_W-1:0] dispatch_packet,
  input  wire [NUM_SM-1:0] dispatch_ack
);

// FIFO
reg [CMD_W-1:0] fifo [0:FIFO_DEPTH-1];
reg [$clog2(FIFO_DEPTH):0] wr_ptr, rd_ptr, count;
assign cmd_ready = (count < FIFO_DEPTH);
wire fifo_empty = (count == 0);

always @(posedge clk) begin

```

```

    if (rst) begin
        wr_ptr <= 0; rd_ptr <= 0; count <= 0;
    end else begin
        if (cmd_valid && cmd_ready) begin
            fifo[wr_ptr] <= cmd_data;
            wr_ptr <= wr_ptr + 1;
            count <= count + 1;
        end
    end
end

// decode request
wire [REG_W-1:0] req_regs = fifo[rd_ptr][63:48];
wire [SMEM_W-1:0] req_smem = fifo[rd_ptr][47:32];

// round-robin selection
reg [$clog2(NUM_SM)-1:0] rr_ptr;
reg [$clog2(NUM_SM)-1:0] chosen;
reg found;
integer i;

always @(*) begin
    found = 0; chosen = 0;
    for (i = 0; i < NUM_SM; i = i + 1)
        if (!found) begin
            int idx = (rr_ptr + i) % NUM_SM;
            if (sm_ready[idx] &&
                sm_regs_avail[idx*REG_W +: REG_W] >= req_regs &&
                sm_smem_avail[idx*SMEM_W +: SMEM_W] >= req_smem) begin
                chosen = idx[$clog2(NUM_SM)-1:0];
                found = 1;
            end
        end
    end
end

always @(posedge clk) begin
    if (rst) begin
        rr_ptr <= 0;
        dispatch_valid <= 0;
    end else begin
        dispatch_valid <= 0;
        if (!fifo_empty && found) begin
            dispatch_valid[chosen] <= 1'b1;
            dispatch_packet[chosen*CMD_W +: CMD_W] <= fifo[rd_ptr];
            rr_ptr <= chosen + 1;
            rd_ptr <= rd_ptr + 1;
            count <= count - 1;
        end
    end
end

```

```

    end
  end
end
endmodule

```

1.6.2 Work-group scheduler

The compute dispatch unit presented earlier produces SM-targeted work-group dispatch requests and the work-group scheduler must convert those requests into timely, resource-safe allocations to individual SMs/CUs. This subsection formalizes the scheduler problem, analyzes constraints relevant to SIMT occupancy, and gives a synthesizable Verilog implementation that enforces capacity and fairness.

Problem statement and analysis. A work-group scheduler must:

- enforce per-SM resource limits (register file, shared memory, warp slots);
- maximize throughput by keeping SMs occupied while avoiding oversubscription;
- provide fairness and support low-latency issuance for interactive kernels.

Let R_{sm} , S_{sm} , and W_{sm} be an SM's available registers, shared memory bytes, and warp slots respectively. For a work-group requiring r_{wg} registers, s_{wg} shared memory, and w_{wg} warp slots, the maximum number of resident work-groups per SM satisfies

$$[H]WG_{\max} = \min \left(\left\lfloor \frac{R_{\text{sm}}}{r_{\text{wg}}} \right\rfloor, \left\lfloor \frac{S_{\text{sm}}}{s_{\text{wg}}} \right\rfloor, \left\lfloor \frac{W_{\text{sm}}}{w_{\text{wg}}} \right\rfloor \right). \quad (1.30)$$

Enforcing (1) at issue-time prevents dynamic spills and bank contention. Latency hiding requires issuing new work-groups before running SMs stall on memory, so the scheduler should maintain a small lookahead queue per SM.

Design and implementation choices. Two main architectures exist:

1. centralized scheduler with a global queue and per-SM credit counters — simpler to reason about, scales in hardware cost roughly linearly with number of SMs.
2. distributed per-SM queues with occasional work-stealing — reduces central contention and scales better for many SMs, at the cost of complex steal arbitration.

This implementation chooses a centralized, credit-based scheduler with round-robin arbitration across SMs and a FIFO work-group buffer. Each SM exposes three credits: registers, shared memory, and warp slots. A work-group is issued when the target SM has all three credits available.

Verilog implementation. The following synthesizable module implements a small centralized scheduler with parameterized SM count and resource granularity. It supports enqueue and synchronous grant signals, and updates per-SM credits on grant and on explicit free notifications.

Listing 1.21: Work-group scheduler (centralized, credit-based).

```

module wg_scheduler #(
  parameter NUM_SMS = 8,

```

```

parameter WG_ID_W = 16,
parameter REG_QTY_W = 16,
parameter SHM_QTY_W = 16,
parameter WARP_QTY_W = 8
) (
    input wire clk,
    input wire rst,
    // enqueue interface from dispatch unit
    input wire enq_valid,
    input wire [WG_ID_W-1:0] enq_wg_id,
    input wire [REG_QTY_W-1:0] enq_regs, // regs required
    input wire [SHM_QTY_W-1:0] enq_shm, // shared mem required
    input wire [WARP_QTY_W-1:0] enq_warps, // warp slots required
    output reg enq_ready,
    // grant outputs: one SM index and WG assigned
    output reg grant_valid,
    output reg [$clog2(NUM_SMS)-1:0] grant_sm,
    output reg [WG_ID_W-1:0] grant_wg_id,
    // per-SM credit updates (free notifications)
    input wire [NUM_SMS-1:0] free_valid,
    input wire [REG_QTY_W-1:0] free_regs [NUM_SMS-1:0],
    input wire [SHM_QTY_W-1:0] free_shm [NUM_SMS-1:0],
    input wire [WARP_QTY_W-1:0] free_warps [NUM_SMS-1:0],
    // current SM capacities read-init (static at reset or via config)
    input wire [REG_QTY_W-1:0] sm_regs_init [NUM_SMS-1:0],
    input wire [SHM_QTY_W-1:0] sm_shm_init [NUM_SMS-1:0],
    input wire [WARP_QTY_W-1:0] sm_warps_init [NUM_SMS-1:0]
);

// Simple FIFO depth 16 for pending work-groups
localparam FIFO_D = 16;
reg [WG_ID_W-1:0] fifo_wg_id [FIFO_D-1:0];
reg [REG_QTY_W-1:0] fifo_regs [FIFO_D-1:0];
reg [SHM_QTY_W-1:0] fifo_shm [FIFO_D-1:0];
reg [WARP_QTY_W-1:0] fifo_warps [FIFO_D-1:0];
reg [$clog2(FIFO_D+1)-1:0] fifo_wr_ptr, fifo_rd_ptr;
reg [4:0] fifo_count;

// per-SM dynamic credits
reg [REG_QTY_W-1:0] sm_regs [NUM_SMS-1:0];
reg [SHM_QTY_W-1:0] sm_shm [NUM_SMS-1:0];
reg [WARP_QTY_W-1:0] sm_warps [NUM_SMS-1:0];

integer i;
// init and free handling
always @(posedge clk) begin
    if (rst) begin

```

```

fifo_wr_ptr <= 0; fifo_rd_ptr <= 0; fifo_count <= 0;
grant_valid <= 0; enq_ready <= 1;
for (i=0;i0) begin
    // check head-of-fifo fits into SM i
    if ((sm_regs[i] >= fifo_regs[fifo_rd_ptr]) &&
        (sm_shm[i] >= fifo_shm[fifo_rd_ptr]) &&
        (sm_warps[i] >= fifo_warps[fifo_rd_ptr])) begin
        // issue grant
        grant_valid <= 1;
        grant_sm <= i[$clog2(NUM_SMS)-1:0];
        grant_wg_id <= fifo_wg_id[fifo_rd_ptr];
        // consume FIFO head
        fifo_rd_ptr <= fifo_rd_ptr + 1;
        fifo_count <= fifo_count - 1;
        // deduct credits
        sm_regs[i] <= sm_regs[i] - fifo_regs[fifo_rd_ptr];
        sm_shm[i] <= sm_shm[i] - fifo_shm[fifo_rd_ptr];
        sm_warps[i] <= sm_warps[i] - fifo_warps[fifo_rd_ptr];
        disable for; // exit loop after grant
    end
end
end
end
end
endmodule

```

Concrete design implications and trade-offs:

- A centralized credit tracker simplifies correctness proofs but adds single-point contention; scale with `NUM_SMS` increases wire area.
- FIFO depth trades off latency for buffering during bursts; deeper buffers reduce dispatch stalls but cost area.
- Enforcing equation (1.30) at issue time avoids runtime spills, improving sustained TFLOPS for ML kernels.
- Distributed, steal-enabled schedulers reduce latency on large `NUM_SMS` designs, at the expense of more complex coherence and arbitration logic.

1.6.3 Resource allocator

The work-group scheduler decides which wavefronts should run and the compute dispatch unit turns those decisions into active SM assignments; the resource allocator must therefore map a work-group’s register and shared-memory needs onto physical SM resources atomically and with minimal latency to avoid stalling dispatch.

The allocator problem: given per-work-group requirements R_{req} registers and S_{req} bytes of shared memory, assign disjoint regions in the per-SM register file and shared memory (LDS)

so that multiple work-groups can coexist without bank conflicts or oversubscription. Key constraints are:

1. Resource exclusivity: regions assigned to different work-groups must not overlap.
2. Alignment and granularity: registers and LDS are often allocated in fixed-size blocks to simplify banking and reduce metadata.
3. Low-latency allocation: dispatch pipeline expects grant/deny within a few cycles to keep SM occupancy high.

Analysis uses the occupancy bound per SM:

$$[H]\text{Occupancy}_{\max} = \left\lfloor \min \left(\frac{R_{\text{total}}}{R_{\text{per_wg}}}, \frac{S_{\text{total}}}{S_{\text{per_wg}}}, W_{\text{slots}} \right) \right\rfloor, \quad (1.31)$$

where R_{total} and S_{total} are physical resources, and W_{slots} is available warp-slot count. The allocator must enforce this bound while minimizing fragmentation.

Implementation approach: a first-fit block allocator with bitmaps for registers and LDS. The allocator services `req_valid/ req_ready` handshakes from the compute dispatch, scans bitmaps for a contiguous run of free blocks, and returns base indices. Releases are atomic and mark blocks free. This design is simple, synthesizable, and maps to SM-local storage, allowing each SM to run independently without cross-SM coherence.

Design summary in synthesizable Verilog implementing block-based first-fit allocation, single-issue per cycle for simplicity (can be pipelined for throughput):

Listing 1.22: Compact SM-local resource allocator using first-fit search for registers and shared memory.

```

module resource_allocator #(
  parameter REG_BLOCKS = 512,
  parameter SHM_BLOCKS = 256,
  parameter BLOCK_ID_W = 10
) (
  input  wire      clk,
  input  wire      rstn,
  input  wire      req_valid,
  input  wire [15:0] req_reg_blocks,
  input  wire [15:0] req_shm_blocks,
  output reg      req_ready,
  output reg      grant,
  output reg [BLOCK_ID_W-1:0] grant_reg_base,
  output reg [BLOCK_ID_W-1:0] grant_shm_base,
  input  wire      rel_valid,
  input  wire [15:0] rel_reg_blocks,
  input  wire [15:0] rel_shm_blocks,
  input  wire [BLOCK_ID_W-1:0] rel_base_reg,
  input  wire [BLOCK_ID_W-1:0] rel_base_shm
);

reg [REG_BLOCKS-1:0] reg_bmap;

```

```

reg [SHM_BLOCKS-1:0] shm_bmap;
integer i;

// simple first-fit allocation
function automatic [BLOCK_ID_W-1:0] first_fit;
  input [REG_BLOCKS-1:0] bmap;
  input [15:0] nblocks;
  integer j,k;
  reg free;
begin
  first_fit = {BLOCK_ID_W{1'b0}};
  for (j=0;j<REG_BLOCKS-nblocks;j=j+1) begin
    free = 1;
    for (k=0;k<nblocks;k=k+1)
      if (bmap[j+k]) free = 0;
    if (free) begin
      first_fit = j[BLOCK_ID_W-1:0];
      disable first_fit;
    end
  end
end
endfunction

always @(posedge clk or negedge rstn) begin
  if (!rstn) begin
    reg_bmap <= 0; shm_bmap <= 0;
    grant <= 0; req_ready <= 1;
  end else begin
    grant <= 0;
    if (req_valid && req_ready) begin
      grant_reg_base <= first_fit(reg_bmap, req_reg_blocks);
      grant_shm_base <= first_fit(shm_bmap, req_shm_blocks);
      grant <= 1'b1;
    end
    if (rel_valid) begin
      for (i=0;i<rel_reg_blocks;i=i+1)
        reg_bmap[rel_base_reg+i] <= 1'b0;
      for (i=0;i<rel_shm_blocks;i=i+1)
        shm_bmap[rel_base_shm+i] <= 1'b0;
    end
  end
end
endmodule

```


1.6.4 Barrier synchronization logic

The work-group scheduler and resource allocator place and size work-groups and assign warps to SM slots; the barrier logic therefore must map to those allocations, track the number of participating warps, and interoperate with the SM's warp context queues and shared memory banks. In particular, the scheduler provides a work-group ID and an expected participant count; the allocator fixes warp-to-SM mappings that the barrier hardware uses to broadcast release events.

Barrier synchronization solves the problem of stalling multiple warps until every thread in a work-group reaches a synchronization point (for example, CUDA `__syncthreads`). Analysis: implement a per-work-group barrier bank indexed by the scheduler-assigned slot. Each bank stores:

- `expected_count`: number of participating warps W .
- `arrival_count`: running tally of warps that have signaled arrival.
- `gen`: a generation bit used for sense-reversing to distinguish successive uses of the same barrier slot.

The release condition is simple and robust:

$$[H]\text{release when } arrival_count = expected_count. \quad (1.32)$$

On release the bank toggles `gen` and clears `arrival_count` to zero atomically so reused work-groups do not observe stale arrivals.

Implementation notes: warps assert a single-cycle pulse `barrier_req` with their `work_group_id` and a `warp_active` flag indicating participation (compilers ensure structured barriers so masked threads are accounted for). The barrier bank performs an atomic increment on arrival, compares to expected, and, on equality, drives a per-warp release vector or a release token that the SM distributes to the warps mapped to that work-group slot. A generation bit is returned with the release so that stalled warps can observe a change in generation rather than relying on resetting signals.

A synthesizable Verilog implementation follows. It implements a simple multi-slot barrier bank with parameterizable slots and maximum warps per group. Each slot handles increment, compare, and generation toggle. Comments explain signal roles.

Listing 1.23: Compact work-group barrier bank (synthesizable).

```

module barrier_bank #(
  parameter SLOTS = 8,
  parameter MAX_WARPS = 16,
  parameter ID_W = 3
) (
  input  wire      clk,
  input  wire      reset,
  input  wire      req_valid,
  input  wire [ID_W-1:0] req_slot,
  input  wire [$clog2(MAX_WARPS)-1:0] req_inc,
  input  wire      set_expected,
  input  wire [ID_W-1:0] set_slot,

```

```

input wire [$clog2(MAX_WARPS)-1:0] set_expected_count,
output reg          release_valid,
output reg  [ID_W-1:0] release_slot
);
reg [$clog2(MAX_WARPS)-1:0] expected [0:SLOTS-1];
reg [$clog2(MAX_WARPS)-1:0] arrived  [0:SLOTS-1];
integer i;

always @(posedge clk) begin
  if (reset) begin
    release_valid <= 0;
    for (i=0; i<SLOTS; i=i+1) begin
      expected[i] <= 0;
      arrived[i]  <= 0;
    end
  end else begin
    release_valid <= 0;
    if (set_expected)
      expected[set_slot] <= set_expected_count;
    if (req_valid) begin
      arrived[req_slot] <= arrived[req_slot] + req_inc;
      if (arrived[req_slot] + req_inc == expected[req_slot]) begin
        arrived[req_slot] <= 0;
        release_valid <= 1'b1;
        release_slot <= req_slot;
      end
    end
  end
end
endmodule

```

1.6.5 Compute kernel testbench

The test scenario combines the resource allocation constraints and barrier synchronization behavior discussed previously, exercising how a dispatched work-group uses allocated register and shared-memory resources and stalls at barriers under contention.

A robust compute-kernel testbench must solve: how to deterministically drive the dispatch path, emulate a small SIMT execution stream, verify barrier release semantics, and measure occupancy-driven throughput. Analysis shows resident work-groups per SM is bounded by multiple resources; a compact expression is

$$[H]N_{\text{resident}} = \min\left(\left\lfloor \frac{R_{\text{SM}}}{R_{\text{wg}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{wg}}} \right\rfloor, W_{\text{max}}\right) \quad (1.33)$$

where R_{SM} is registers per SM, R_{wg} registers per work-group, S_{SM} shared memory per SM, S_{wg} shared memory per work-group, and W_{max} is hardware warp-slot capacity. The testbench should vary R_{wg} and S_{wg} to validate allocation decisions, and inject thread divergence patterns to exercise reconvergences and barrier correctness.

Implementation below provides:

- a parameterized synthesizable compute kernel core that simulates a small SIMT workload (per-thread increment and a barrier);
- a simple resource allocator modeling resident work-group counting and grant/return protocol;
- a compact barrier module implementing arrival counting and release pulse; and
- a testbench harness that launches multiple work-groups, toggles reset and clock for simulation, and reports completion timing.

Listing 1.24: Compact compute-kernel testbench with allocator and barrier simulation.

```

module compute_kernel_tb;
  reg clk = 0, rst_n = 0;
  always #5 clk = ~clk;

  parameter WG_SIZE = 8;
  parameter MAX_RESIDENT = 2;
  parameter NUM_WG_LAUNCH = 4;

  reg [$clog2(MAX_RESIDENT+1)-1:0] resident_cnt = 0;
  reg grant, release, launch_req;
  reg [3:0] barrier_count;
  reg barrier_release;
  reg [3:0] thread_idx;
  reg running, done;

  wire wg_start = running;
  wire wg_done = done;

  // allocator
  always @(posedge clk) begin
    if (!rst_n) begin
      resident_cnt <= 0; grant <= 0; launch_req <= 0;
    end else begin
      grant <= 0;
      if (launch_req && resident_cnt < MAX_RESIDENT) begin
        grant <= 1; resident_cnt <= resident_cnt + 1;
        launch_req <= 0;
      end
      if (release) resident_cnt <= resident_cnt - 1;
    end
  end

  // barrier
  always @(posedge clk) begin

```

```

if (!rst_n) begin barrier_count <= 0; barrier_release <= 0; end
else begin
    barrier_release <= 0;
    if (wg_start) begin
        barrier_count <= barrier_count + 1;
        if (barrier_count + 1 == WG_SIZE) begin
            barrier_release <= 1; barrier_count <= 0;
        end
    end
end
end

// kernel simulation
always @(posedge clk) begin
    if (!rst_n) begin thread_idx <= 0; running <= 0; done <= 0; release <= 0;
    end
else begin
    if (grant) begin running <= 1; thread_idx <= 0; done <= 0; end
    if (running) begin
        if (barrier_release) begin
            if (thread_idx == WG_SIZE-1) begin running <= 0; done <= 1; end
            else thread_idx <= thread_idx + 1;
        end else if (thread_idx < WG_SIZE-1)
            thread_idx <= thread_idx + 1;
        end
        release <= done;
    end
end

// launch sequence
integer i;
initial begin
    rst_n = 0; #20 rst_n = 1;
    for (i=0; i<NUM_WG_LAUNCH; i=i+1) begin
        launch_req = 1; #10;
        launch_req = 0; #100;
    end
    #100 $finish;
end
endmodule

```

Chapter 2

Tensor and Matrix Acceleration

2.1 Matrix Multiplication Fundamentals

2.1.1 GEMM operation principles

Building on the compute dispatch and occupancy topics in Chapter 11, the GEMM discussion here ties those runtime concerns to the microarchitectural choices that maximize throughput on SMs and tensor cores. The following explains the canonical GEMM semantics, its computational and memory costs, and a practical tiled implementation for SIMT GPUs.

GEMM (general matrix multiply) is commonly expressed in linear-algebra libraries as

$$[H]C \leftarrow \alpha A B + \beta C, \quad (2.1)$$

where A is $M \times K$, B is $K \times N$, and C is $M \times N$. For floating-point multiply-add (FMA) counted as two FLOPs, the total floating-point work is

$$[H]\text{FLOPs} = 2 M N K. \quad (2.2)$$

A naive implementation streams matrices from DRAM and achieves poor arithmetic intensity (FLOPs per byte). On modern GPUs, the goal is to raise arithmetic intensity by maximizing reuse in on-chip storage (registers, shared memory/LDS, and tile-level caches) and by mapping computations to specialized units (tensor cores) when precision and data layout allow.

Analysis steps:

1. Memory traffic model — for a blocked algorithm with block sizes $M_b \times N_b$ and inner tile K_b , each tile loads $M_b \times K_b$ elements of A and $K_b \times N_b$ of B once per accumulation into $M_b \times N_b$ outputs. Effective bytes moved per FLOP reduce with larger K_b and reuse.
2. Compute/memory balance — arithmetic intensity I can be approximated as

$$[H]I \approx \frac{2 M_b N_b K_b}{(M_b K_b + K_b N_b + M_b N_b) \times s}, \quad (2.3)$$

where s is bytes per element (4 for FP32). Increasing K_b increases I until on-chip capacity limits reuse.

3. Mapping to GPU hardware — choose tile sizes to:
 - Fill register file per warp for per-thread accumulators.

- Fit tiles of A and B into shared memory to avoid bank conflicts.
- Keep enough active warps for latency hiding without exceeding shared-memory or register limits.

Implementation: a standard tiled SIMT kernel using shared memory. This is intentionally portable FP32 code; replace with WMMA or vendor intrinsics to use tensor cores for mixed precision.

Listing 2.1: Tiled FP32 GEMM kernel (CUDA) using shared memory

```
__global__ void sgemm_tiled(int M, int N, int K, const float *A, const float *
    B, float *C,
                           float alpha, float beta) {
    const int TILE = 32; // tile size; tune for SM register/shared limits
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float acc = 0.0f; // per-thread accumulator in register
    for (int t = 0; t < (K + TILE - 1) / TILE; ++t) {
        int a_col = t * TILE + threadIdx.x;
        int b_row = t * TILE + threadIdx.y;
        // load tiles with bounds check
        As[threadIdx.y][threadIdx.x] = (row < M && a_col < K) ? A[row * K + a_col]
            : 0.0f;
        Bs[threadIdx.y][threadIdx.x] = (b_row < K && col < N) ? B[b_row * N + col]
            : 0.0f;
        __syncthreads();
        // multiply-accumulate inner tile
        for (int k = 0; k < TILE; ++k)
            acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    if (row < M && col < N)
        C[row * N + col] = alpha * acc + beta * C[row * N + col];
}
```

2.1.2 Blocking and tiling strategies

The previous subsection established GEMM’s operation count and the critical gap between compute and memory traffic; blocking and tiling directly attack that gap by trading on-chip storage to increase data reuse. Here we analyze tiling parameters, show a practical CUDA-style blocked kernel, and derive the arithmetic-intensity benefit that drives SM and tensor-core utilization.

Problem. A naive row-column GEMM issues $\mathcal{O}(N^3)$ FLOPs while moving $\mathcal{O}(N^2)$ matrix elements repeatedly across high-latency VRAM, making the kernel memory-bound on modern GPUs with high SM throughput. Blocking reduces redundant loads by keeping small tiles

resident in fast on-chip storage (registers or shared memory/LDS) so each loaded element participates in many MACs.

Analysis. For square tile size b (computing a b -by- b C tile), a single k -step multiplies one A tile and one B tile, producing $\mathcal{O}(b^3)$ multiply-add operations while loading $\mathcal{O}(b^2)$ elements from each matrix once per step. If C remains resident in registers across steps, the per-step arithmetic intensity (FLOPs per byte transferred) grows linearly with b . Formally, using element size s bytes and counting one multiply-add as two FLOPs,

$$[H]AI_{\text{per-step}} \approx \frac{2b^3}{2b^2 \cdot s} = \frac{b}{s}. \quad (2.4)$$

Thus doubling tile dimension doubles AI, which improves the chance of being compute-bound and increases SM utilization and tensor-core occupancy. Practical limits on b arise from shared memory capacity, register pressure, and bank conflicts.

Implementation. Below is a production-ready CUDA-style tiled GEMM kernel that demonstrates shared-memory blocking and k -loop tiling; it targets SM-level shared memory reuse and is amenable to tensor-core intrinsics with small changes.

Listing 2.2: Shared-memory tiled GEMM kernel

```
extern "C" __global__ void tiled_gemm(const float* A, const float* B, float* C
,
                                int M, int N, int K, int lda, int ldb,
                                int ldc) {
    const int TILE = 32; // tile dimension tuned per-SM for shared mem and
        registers
    __shared__ float sA[TILE][TILE]; // shared memory tiles
    __shared__ float sB[TILE][TILE];
    int bx = blockIdx.x, by = blockIdx.y;
    int tx = threadIdx.x, ty = threadIdx.y;
    int row = by * TILE + ty;
    int col = bx * TILE + tx;
    float acc = 0.0f; // scalar accumulator in register
    for (int k0 = 0; k0 < K; k0 += TILE) {
        // cooperative load with bounds check
        int aidx = row < M && (k0 + tx) < K ? row*lda + (k0 + tx) : -1;
        int bidx = (k0 + ty) < K && col < N ? (k0 + ty)*ldb + col : -1;
        sA[ty][tx] = (aidx >= 0) ? A[aidx] : 0.0f; // load A tile
        sB[ty][tx] = (bidx >= 0) ? B[bidx] : 0.0f; // load B tile
        __syncthreads();
        // compute inner product for tile
        #pragma unroll 8
        for (int t = 0; t < TILE; ++t) acc += sA[ty][t] * sB[t][tx];
        __syncthreads();
    }
    if (row < M && col < N) C[row*ldc + col] = acc; // writeback
}
```

Implications and trade-offs:

- Increasing b increases arithmetic intensity (eq. (1)) and improves SM/tensor-core throughput, but bumps shared memory usage and register pressure, limiting occupancy.
- Bank conflicts and misaligned accesses in shared memory reduce effective reuse; choose TILE to align with bank count and use warp-friendly layouts.
- For tensor cores, use tile shapes matching hardware (e.g., $16 \times 16 \times 16$) and move accumulation to registers to minimize global traffic.
- On multi-GPU systems, larger tiles reduce PCIe/HBM transfer fraction but increase latency for load balancing and dynamic scheduling.

2.1.3 Data reuse optimization

The preceding discussion on blocking and tiling established how work is partitioned across SMs and warps, and the GEMM fundamentals set the flop and memory cost per multiply-accumulate. Building on those, this subsection quantifies how tile parameters and memory placement drive reuse and shows an implementation pattern that maximizes on-chip reuse while respecting SIMT constraints.

Problem: maximize arithmetic intensity on each SM by reusing A and B tiles from shared memory or registers so that DRAM and TMU transactions are amortized across many FMAs (fused multiply-adds). **Analysis:** consider a micro-kernel that multiplies tiles of sizes $M_t \times K_t$ and $K_t \times N_t$ to update a $M_t \times N_t$ output tile. **Operational relevance:** larger M_t, N_t, K_t increase reuse but consume more shared memory and registers, reducing occupancy.

Define compute and data traffic per tile:

$$[H]AI = \frac{\text{FLOPs}}{\text{elements moved from DRAM}} = \frac{2 M_t N_t K_t}{M_t K_t + K_t N_t + \alpha M_t N_t}, \quad (2.5)$$

where $\alpha = 1$ if the output tile is written once (kept on-chip during accumulation), and $\alpha = 2$ if read and written back. Equation (1) shows AI scales approximately with tile dimensions; for square tiles $M_t = N_t = K_t = T$ and single-write outputs, $AI \approx \frac{2T}{3}$, so doubling T nearly doubles arithmetic intensity.

Implementation: a canonical CUDA-style SM micro-kernel uses shared memory double-buffering and warp-level accumulation to maximize reuse and hide global memory latency. The code below demonstrates the pattern (simplified, production kernels add bank conflict avoidance and prefetch pipelining):

Listing 2.3: Tiled GEMM micro-kernel using shared memory

```
#define TILE 32
__global__ void gemm_tiled(const float* A, const float* B, float* C,
                          int M, int N, int K) {
    __shared__ float sA[TILE][TILE]; // shared reuse of A
    __shared__ float sB[TILE][TILE]; // shared reuse of B
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float acc = 0.0f; // register accumulation
    for (int kt = 0; kt < (K + TILE - 1) / TILE; ++kt) {
        int a_col = kt * TILE + threadIdx.x;
```



```

int b_row = kt * TILE + threadIdx.y;
sA[threadIdx.y][threadIdx.x] = (row < M && a_col < K) ? A[row * K + a_col]
    : 0.0f;
sB[threadIdx.y][threadIdx.x] = (b_row < K && col < N) ? B[b_row * N + col]
    : 0.0f;
__syncthreads();
for (int k = 0; k < TILE; ++k)
    acc += sA[threadIdx.y][k] * sB[k][threadIdx.x];
__syncthreads();
}
if (row < M && col < N)
    C[row * N + col] = acc;
}

```

2.1.4 Arithmetic intensity considerations

The previous subsections showed how tiling and careful reuse keep C blocks resident in fast storage, and how data-layout choices expose contiguous accesses; arithmetic intensity ties both ideas to the Roofline model by quantifying compute per byte moved. Here we analyze arithmetic intensity for blocked GEMM, derive a practical rule-of-thumb tied to tile size, and give an implementation pattern that realizes the savings on modern SM/CU hardware.

Problem and analysis: arithmetic intensity (AI) is FLOPs transferred divided by bytes moved between DRAM and the compute engine; it determines whether a kernel is memory-bound or compute-bound on a given GPU with bandwidth B and peak compute P . For dense square GEMM of size N with square block (tile) size b , each full matrix multiply performs $2N^3$ FLOPs. Using a blocking scheme that keeps the destination $C_{i,j}$ block in shared memory or registers across the K -loop, A and B blocks must be reloaded for each k -step, while each C block is loaded/written only once. Summing DRAM transfers yields total elements moved approximately $2N^3/b + 2N^2$. Therefore the arithmetic intensity (in FLOPs per element) becomes

$$[H]AI_{\text{elements}} = \frac{2N^3}{2N^3/b + 2N^2} = \frac{b}{1 + b/N}. \quad (2.6)$$

Converting to FLOPs per byte, for element size s bytes (single precision $s = 4$), gives

$$[H]AI_{\text{bytes}} \approx \frac{b}{s(1 + b/N)} \xrightarrow{N \gg b} \frac{b}{s}. \quad (2.7)$$

Thus, for large matrices the arithmetic intensity grows linearly with tile size b (until on-chip capacity limits reuse). Increasing b raises AI and shifts the kernel toward the compute-bound region of the Roofline.

Implementation: the canonical CUDA-style tiled GEMM uses shared memory per SM to buffer A and B tiles; registers hold partial C accumulators. The snippet below shows the core loop using double-buffered shared-memory tiles and FMA-friendly accumulation. Comments indicate mapping to SM resources.

Listing 2.4: Tiled GEMM kernel sketch using shared memory and register accumulators

```
__global__ void gemm_tile(const float *A, const float *B, float *C,
```

```

        int N) {

    const int b = TILE;                                // tile size
    __shared__ float As[TILE][TILE];                  // shared memory for A
    __shared__ float Bs[TILE][TILE];                  // shared memory for B
    float c_reg[TILE];                                // register accumulators per
        thread row

    int tile_i = blockIdx.y * b;
    int tile_j = blockIdx.x * b;
    int local_row = threadIdx.y;
    int local_col = threadIdx.x;

    // init accumulators
    for (int i = 0; i < TILE; ++i)
        c_reg[i] = 0.0f;

    // loop over tiles
    for (int t = 0; t < (N + b - 1) / b; ++t) {
        int a_col = t * b + local_col;
        int b_row = t * b + local_row;

        As[local_row][local_col] = (tile_i + local_row < N && a_col < N)
            ? A[(tile_i + local_row) * N + a_col]
            : 0.0f;
        Bs[local_row][local_col] = (b_row < N && tile_j + local_col < N)
            ? B[b_row * N + tile_j + local_col]
            : 0.0f;

        __syncthreads();

        for (int k = 0; k < b; ++k)
            c_reg[k] += As[local_row][k] * Bs[k][local_col];
        __syncthreads();
    }

    // store results
    if (tile_i + local_row < N && tile_j + local_col < N)
        C[(tile_i + local_row) * N + tile_j + local_col] = c_reg[local_col];
}

```

2.2 Tensor Core Architecture

2.2.1 Systolic array organization

The preceding discussion of blocking, tiling, and arithmetic intensity motivates a spatial compute fabric that maximizes data reuse across cycles. Systolic arrays implement that fabric by streaming operands through a grid of pipelined processing elements to convert memory band-

width into sustained MAC throughput.

A systolic array is a two-dimensional mesh of processing elements (PEs) that forward intermediate operands rhythmically on each clock. Each PE performs a local multiply-accumulate and shifts partials to neighbors, so a single loaded operand participates in many MACs before eviction. Typical dataflows are:

- weight-stationary: weights stay in PEs, activations stream through;
- output-stationary: partial sums remain local while inputs stream;
- input-stationary: activations stay local and weights stream.

Choice of dataflow affects on-chip buffering, register pressure, and memory traffic.

Throughput and utilization are driven by array shape and clock. For an $N \times N$ systolic array executing fused multiply-add (FMA), peak arithmetic rate (counting FMA as two floating operations) is

$$[H]\text{Peak FLOPS} = 2 \cdot N^2 \cdot f, \quad (2.8)$$

where f is array clock frequency. Sustained FLOPS require matching memory and staging bandwidth; otherwise utilization collapses.

Operational intensity for a tiled GEMM mapping onto a $T \times T$ tile can be approximated as

$$[H]I \approx \frac{2T^3}{(T_w^2 + T_a^2 + T_o^2) \cdot B_e}, \quad (2.9)$$

where T_w, T_a, T_o are sub-tile footprints and B_e is bytes per element. Increasing T improves reuse but increases on-chip buffer size.

Implementation notes for GPUs:

1. Map each warp or group of warps to feed one or more array tiles, using shared memory as double-buffered staging.
2. Use SIMD-friendly operand packing to amortize operand fetch latency across lanes.
3. Avoid bank conflicts in shared memory by swizzling tile layout so sequential threads access sequential banks.
4. Pipeline accumulation into higher-precision accumulators to avoid saturation for mixed-precision paths.

A compact, synthesizable integer systolic-array example follows. It instantiates PE cells with registered inputs, producing streamed outputs and accumulating partial sums. This demonstrates the datapath and handshake signals used when integrating into an SM tile engine.

Listing 2.5: Parameterizable systolic array (integer MAC) example

```

module systolic_pe #(parameter W=16) (
  input   wire clk, rst,
  input   wire signed [W-1:0] a_in, b_in,
  input   wire signed [2*W-1:0] acc_in,
  output  reg signed [W-1:0] a_out, b_out,
  output  reg signed [2*W-1:0] acc_out
);

```

```

// local multiply-accumulate with pipeline registers
reg signed [W-1:0] a_reg, b_reg;
reg signed [2*W-1:0] acc_reg;
always @(posedge clk) begin
    if (rst) begin
        a_reg <= 0; b_reg <= 0; acc_reg <= 0;
        a_out <= 0; b_out <= 0; acc_out <= 0;
    end else begin
        a_reg <= a_in; b_reg <= b_in;
        // multiply then add to incoming accumulator
        acc_reg <= acc_in + (a_reg * b_reg);
        a_out <= a_reg; b_out <= b_reg; acc_out <= acc_reg;
    end
end
endmodule

module systolic_array #(parameter M=4, N=4, W=16) (
    input wire clk, rst,
    input wire signed [W-1:0] a_in [0:M-1], // input column stream
    input wire signed [W-1:0] b_in [0:N-1], // input row stream
    output wire signed [2*W-1:0] c_out [0:M-1][0:N-1]
);
    // instantiate grid of PEs
    genvar i, j;
    wire signed [W-1:0] a_stream [0:M][0:N];
    wire signed [W-1:0] b_stream [0:M][0:N];
    wire signed [2*W-1:0] acc_stream [0:M][0:N];

    // initialize input streams at top row/left column
    generate
        for (j = 0; j < N; j = j + 1) begin
            assign b_stream[0][j] = b_in[j];
        end
        for (i = 0; i < M; i = i + 1) begin
            assign a_stream[i][0] = a_in[i];
        end
    endgenerate

    // connect array of processing elements
    generate
        for (i = 0; i < M; i = i + 1) begin : row_loop
            for (j = 0; j < N; j = j + 1) begin : col_loop
                systolic_pe #(.W(W)) pe (
                    .clk(clk),
                    .rst(rst),
                    .a_in(a_stream[i][j]),
                    .b_in(b_stream[i][j]),

```

```

        .acc_in(acc_stream[i][j]),
        .a_out(a_stream[i][j+1]),
        .b_out(b_stream[i+1][j]),
        .acc_out(acc_stream[i+1][j+1])
    );
end
end
endgenerate

// assign outputs
generate
    for (i = 0; i < M; i = i + 1)
        for (j = 0; j < N; j = j + 1)
            assign c_out[i][j] = acc_stream[i+1][j+1];
        endgenerate
    endgenerate
endmodule

```

2.2.2 Matrix multiply-accumulate units

Building on the spatial dataflow described for systolic arrays, the multiply-accumulate (MAC) cell implements the elemental operation that the array tiles and schedules. The remainder of this subsection analyzes MAC functionality, shows a compact pipelined implementation suitable for GPU tensor cores (integer/fixed-point MACs used in quantized inference and accumulation), and draws engineering trade-offs that impact SM-level throughput, precision, and area.

Problem: provide a high-throughput, pipelined MAC primitive that supports streaming operand supply from shared/L1 memory, minimizes register-file pressure, and preserves accumulator headroom to avoid overflow when K (reduction length) is large. Analysis begins by stating the core operation for a single output element of a GEMM tile:

$$[H]C_{i,j} \leftarrow C_{i,j} + \sum_{k=0}^{K-1} A_{i,k} \cdot B_{k,j}, \quad (2.10)$$

where each summand is produced by a MAC cell. For fixed-point or integer MACs used in INT8/INT4 accelerators, the accumulator width must be sized to hold the worst-case sum: $\text{ACC_BITS} \geq \text{WIDTH_A} + \text{WIDTH_B} + \lceil \log_2 K \rceil$.

Throughput model: when a tile contains N_{mac} MAC cells operating in lockstep with one result per cycle, and each fused multiply-add (FMA) is counted as two FLOPs by common vendor metrics, peak throughput is

$$[H]\text{Throughput}_{\text{FLOP/s}} = 2 \cdot N_{\text{mac}} \cdot f_{\text{clk}}. \quad (2.11)$$

Implementation constraints: MAC latency must balance clock frequency and pipeline depth, while valid-ready handshakes limit backpressure between the tensor core and SM load-store units. The Verilog below is a synthesizable, parameterized streaming dot-product MAC suitable for a tile cell; it supports signed operands, parameterized widths, and a `last` token to end accumulation for an output element.

Listing 2.6: Streaming pipelined dot-product MAC (synthesizable).

```

module mac_stream #(
    parameter WIDTH = 8,           // input operand width
    parameter ACC_WIDTH = 32      // accumulator width
) (
    input  wire      clk,
    input  wire      rst_n,
    input  wire      valid_in, // new pair available
    input  wire signed [WIDTH-1:0] a,
    input  wire signed [WIDTH-1:0] b,
    input  wire      last,      // end of reduction for this output
    output reg      ready,      // can accept next pair
    output reg      valid_out, // output valid
    output reg signed [ACC_WIDTH-1:0] out // accumulated result
);

// pipeline registers for multiply and accumulate
reg signed [2*WIDTH-1:0] mul; // product width
reg signed [ACC_WIDTH-1:0] acc;
reg active;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        mul <= 0;
        acc <= 0;
        out <= 0;
        ready <= 1;
        valid_out <= 0;
        active <= 0;
    end else begin
        valid_out <= 0;
        if (valid_in && ready) begin
            mul <= a * b; // combinational multiply clocked
            // begin accumulation on first valid pair
            if (!active) begin
                acc <= {{(ACC_WIDTH-2*WIDTH){mul[2*WIDTH-1]}}, mul}; // sign-extend
                product
            active <= 1;
            end else begin
                acc <= acc + {{(ACC_WIDTH-2*WIDTH){mul[2*WIDTH-1]}}, mul};
            end
            // accept or deassert ready depending on consumer (simple flow control
            )
            ready <= 1;
            if (last) begin
                out <= acc; // present accumulated result
                valid_out <= 1;
                acc <= 0;
            end
        end
    end

```

```

        active <= 0;
    end
end
end
end
endmodule

```

Implications and trade-offs:

- Wider accumulators reduce overflow risk for large K but increase area and routing capacitance inside each tensor core.
- Deep pipelining raises f_{clk} but increases end-to-end latency and register pressure in the SM register file if results are forwarded to many threads.
- For TF32/FP16 operators, floating-point FMA hardware replaces the integer multiplier and requires normalization and rounding units, increasing power and complexity but enabling higher numerical range.
- System-level balance: maximize N_{mac} and f_{clk} only while keeping memory bandwidth and operand delivery (L1/shared memory, TMU prefetch) sufficient to saturate MAC units; otherwise, peak GFLOPS from Equation (2) is unattainable.

2.2.3 Dataflow and accumulation patterns

Dataflow selection sets which operand stays stationary inside a processing element and which streams, trading local register pressure for reuse and reducing off-chip bandwidth. Common patterns are:

- output-stationary: partial sums for C remain in the local accumulator while A and B tiles stream through, minimizing writebacks to tile memory.
- weight-stationary: filter or weight elements remain local (used in CNNs), maximizing reuse of B across multiple A streams.
- row-/column-stationary: one matrix dimension is held to balance register usage and interconnect contention.

These patterns implement the block GEMM kernel $C_{\text{tile}}(i, j)$ accumulation:

$$[H]C_{\text{tile}}(i, j) + = \sum_{k=0}^{K-1} A_{\text{tile}}(i, k) B_{\text{tile}}(k, j). \quad (2.12)$$

Operationally, each product $A \cdot B$ becomes a partial sum that must be reduced into the accumulator with high throughput and controlled latency.

Two practical accumulation implementations dominate:

1. streaming FMA chains: each product is fused into an accumulator register with one FMA per cycle per lane, giving minimal buffering and simple control, but the accumulator becomes a hotspot for timing or precision scaling.

2. pipelined adder-tree aggregation: parallel products produced by multiple MAC lanes are reduced by a $\log_2(N)$ stage pipeline, achieving single-cycle throughput after pipeline fill and bounded reduction latency equal to the tree depth.

For N parallel partials, a fully pipelined binary adder tree has depth $D = \lceil \log_2 N \rceil$, so the reduction latency scales as $O(\log N)$ while area scales as $O(N)$ for adders.

Precision and rounding policy matter: mixed-precision designs often compute products in FP16 or INT8 and accumulate in FP32 or FP32-style accumulator to avoid catastrophic cancellation. Accumulator width, rounding mode, and normalization pipeline stages must be matched to the chosen dataflow, since output-stationary designs can amortize accumulator normalization across many MACs.

A compact hardware example follows: an N -input, fully pipelined adder-tree that accepts a vector of parallel products and emits a summed partial in $\log_2$ latency. This maps naturally to a systolic tile where each cycle produces N lane products.

Listing 2.7: Pipelined adder-tree for $N=8$ parallel products, parameterizable width

```

module adder_tree #(
  parameter WIDTH = 32,
  parameter N = 8    // must be power of two
) (
  input  wire      clk,
  input  wire      rst_n,
  input  wire      in_valid,
  input  wire [N*WIDTH-1:0] in_data,  // packed N inputs
  output reg       out_valid,
  output reg  [WIDTH+$clog2(N)-1:0] out_sum
);

// stage registers: stage s has N/(2^s) lanes
localparam STAGES = $clog2(N);

// unpack inputs
wire [WIDTH-1:0] in_vec [0:N-1];
genvar i, s;
generate
  for (i = 0; i < N; i = i + 1) begin : UNPACK
    assign in_vec[i] = in_data[i*WIDTH +: WIDTH];
  end
endgenerate

// pipeline registers for each stage
reg [WIDTH+$clog2(N)-1:0] stage_data [0:STAGES][0:N-1];
reg                      stage_valid [0:STAGES];

integer j, k;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    out_valid <= 0;
    out_sum <= 0;
  end

```



```

    for (s = 0; s <= STAGES; s = s + 1)
        stage_valid[s] <= 0;
    end else begin
        // stage 0: capture inputs
        if (in_valid) begin
            for (j = 0; j < N; j = j + 1)
                stage_data[0][j] <= in_vec[j];
            end
            stage_valid[0] <= in_valid;

            // pipeline add stages
            for (s = 0; s < STAGES; s = s + 1) begin
                if (stage_valid[s]) begin
                    for (k = 0; k < N >> (s + 1); k = k + 1)
                        stage_data[s+1][k] <= stage_data[s][2*k] + stage_data[s][2*k+1];
                    end
                    stage_valid[s+1] <= stage_valid[s];
                end
            end

            // output stage
            out_valid <= stage_valid[STAGES];
            out_sum <= stage_data[STAGES][0];
        end
    end
endmodule

```

2.2.4 Precision and throughput balance

The previous subsections established systolic dataflow and the microarchitecture of matrix multiply-accumulate units, so this subsection examines how operand precision choices map to tensor-core throughput and accumulator sizing. We connect dataflow capacity and MAC density to both numerical fidelity and hardware area, then give a small synthesizable example illustrating tradeoffs.

Precision decisions directly influence MAC unit area, clock-handling, and required accumulator width, which in turn set achievable GFLOPS/TFLOPS for an SM or CU. For fixed silicon area, a simple engineering model treats multiplier area as growing approximately with the square of operand bitwidth; therefore the count of MACs per tile scales inversely:

$$[H]T(b) \approx \frac{K}{b^2}, \quad (2.13)$$

where T is relative throughput, b is operand bitwidth in bits, and K is a proportionality constant that folds frequency and area budget. This explains why lowering from FP32 to FP16 or INT8 yields superlinear increases in MAC count per tile.

Accumulator sizing is a distinct constraint. For integer fixed-point accumulation of N terms each b bits, exact sum bit growth requires

$$[H]w_{\text{acc}} = b + \lceil \log_2 N \rceil. \quad (2.14)$$

Floating-point accumulators behave differently: rounding error accumulates rather than simple overflow, so GPUs commonly use wider accumulators (e.g., FP32 or TF32 accumulators for FP16 inputs) to bound relative error.

Operational implications for tensor-core pipelines:

- **Mixed precision:** Use narrow input multipliers with wider accumulators to maximize MAC density while retaining convergence for ML workloads. This pattern appears in practice: FP16 inputs with FP32 accumulation often hit similar model quality as full FP32.
- **Quantized inference:** INT8/INT4 pipelines can pack more parallel MACs and use 32-bit accumulators to avoid overflow in deep reductions.
- **Frequency and pipeline depth:** Narrower datapaths allow shorter combinational delays, enabling higher clock frequency or fewer pipeline stages for lower latency.

To illustrate a synthesizable hardware primitive useful in accelerator tile design, below is a parameterized fixed-point MAC implemented for synthesis. It shows how changing `INPUT_WIDTH` and `ACC_WIDTH` affects storage and datapath widths, modeling area/perf tradeoffs in RTL.

Listing 2.8: Parameterized fixed-point MAC with selectable accumulator width

```

module mac_sync #(
  parameter INPUT_WIDTH = 8,    // operand width (bits)
  parameter ACC_WIDTH    = 32   // accumulator width (bits)
) (
  input  wire      clk,
  input  wire      rst_n,
  input  wire      in_valid,
  input  wire signed [INPUT_WIDTH-1:0] a, // multiplicand
  input  wire signed [INPUT_WIDTH-1:0] b, // multiplier
  input  wire signed [ACC_WIDTH-1:0]   acc_in, // current accumulator
  output reg signed [ACC_WIDTH-1:0]   acc_out,
  output reg                        out_valid
);

// pipelined multiply (combinational then registered)
wire signed [2*INPUT_WIDTH-1:0] product = a * b;
reg  signed [2*INPUT_WIDTH-1:0] product_r;

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    product_r <= 0;
    acc_out   <= 0;
    out_valid <= 0;
  end else begin
    product_r <= product; // stage 1 register
    if (in_valid) begin
      // sign-extend product into accumulator width before add

```

```

    acc_out <= acc_in + $signed({{(ACC_WIDTH-2*INPUT_WIDTH){product_r[2*
        INPUT_WIDTH-1]}}, product_r});
    out_valid <= 1'b1;
end else begin
    out_valid <= 1'b0;
end
end
end
endmodule

```

Design implications and performance trade-offs:

- Area vs accuracy: reducing b increases $T(b)$ per (2.13) but demands careful accumulator sizing per (2.17) or wider floating-point accumulation.
- Energy: narrower multipliers lower dynamic power per MAC, improving energy efficiency for ML inference.
- Model quality: mixed-precision with wider accumulators recovers numeric fidelity with minimal throughput loss; full-precision paths remain necessary for sensitive training kernels.
- Complexity: adding conversion/normalization units and mixed-format datapaths increases routing and control complexity, slightly reducing effective MAC packing density.

2.3 Mixed Precision Support

2.3.1 FP16, BF16, TF32 computation

Building on the tensor-core precision-versus-throughput trade-offs discussed previously, we now quantify the numeric error sources and show practical code patterns for FP16, BF16 and TF32 on tensor accelerators. The aim is to connect numerical properties (mantissa/exponent) to accumulation strategy inside SMs and tensor cores, then show a concrete implementation pattern for mixed-precision GEMM kernels.

Problem: narrow formats trade mantissa bits for higher SIMD/Tensor throughput, but reduce dynamic range or increase rounding noise. Analysis:

- Format characteristics (mantissa bits): FP16 (binary16) has 10 fraction bits, BF16 has 7 fraction bits, TF32 (NVIDIA TF32) uses 10 fraction bits with FP32-like 8-bit exponent. FP32 has 23 fraction bits.
- Machine-rounding unit error per operation is roughly on the order of the unit roundoff $u \approx 2^{-p}$, where p is the fraction bits. Thus a single multiply-add in format with p bits introduces relative rounding up to about $0.5 \cdot 2^{-p}$.
- When N multiply-adds are accumulated in a narrower accumulator without extra precision, worst-case error can grow approximately linearly: $\text{error}_{\text{total}} \lesssim N \cdot u$. To mitigate, hardware or software uses wider accumulators (FP32 accumulators for FP16/BF16 inputs) or block-wise Kahan-style compensation.

Display the simple bound used for design reasoning:

$$[H]u \approx 2^{-p}, \quad \text{error after } N \text{ ops} \lesssim N \cdot u. \quad (2.15)$$

Implementation patterns and microarchitectural mapping:

- Hardware choice: tensor cores accept narrow operands (FP16/BF16, sometimes TF32 as a compute-mode) and perform high-throughput matrix multiply-accumulate (MMA) with internal accumulation either in FP32 or truncated formats. Typical design routes:
 1. Input operand path: narrow-format lanes packed to feed multiple MAC units per TMU or tensor core lane.
 2. Accumulator path: wider accumulator (FP32) reduces accumulation error and enables higher numerical stability for GEMM.
 3. Conversion/normalization unit: convert BF16/FP16 to internal format, handle denormals as configured (flush-to-zero optional).

Example mixed-precision GEMM kernel (illustrative CUDA-style; cast inputs to narrow storage and accumulate in float32):

Listing 2.9: Mixed-precision GEMM inner loop (storage in half/bfloat16, accumulation in float)

```
__global__ void mixedGEMM(const __half *A, const __nv_bfloat16 *B, float *C,
    int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= M || col >= N) return;
    float acc = 0.0f; // accumulator in FP32
    for (int k = 0; k < K; k++) {
        float b = __bfloat162float(B[k * N + col]); // convert bfloat16 ->
        float a = A[row * M + k]; // FP16/BF16
        acc += a * b; // FP32 FMA on SM ALU or
        // tensor core
    }
    C[row * N + col] = acc;
}
```

Implications and trade-offs:

- Using FP32 accumulators with FP16/BF16 inputs preserves stability for large K , reducing error roughly by factor $2^{(23-p)}$ relative to pure narrow accumulation.
- TF32 offers a practical compromise for many ML workloads: TF32-like compute keeps FP32 range but with reduced mantissa and thus higher throughput on dedicated tensor hardware that accepts TF32 mode.
- Design trade-offs: larger accumulators cost area/power and increase data movement; narrower storage saves bandwidth and allows higher tensor throughput but requires conversion units and careful algorithmic compensation (blocking, mixed-precision iterative refinement).
- For architects: provide programmable accumulator width, fast convertors, and expose compute modes (FP16/BF16/TF32) so libraries (cuBLAS, oneAPI) pick the best accuracy/performance point for workload classes.

2.3.2 INT8 and INT4 quantization

The previous discussion of FP16/BF16/TF32 established the accumulator and conversion hardware requirements for reduced floating formats; INT8/INT4 quantization extends those concerns into integer-packed datapaths and aggressive memory+compute bandwidth optimizations. Here we analyze quantization mapping, accumulator sizing, data-packing microarchitecture, and a synthesizable MAC example that illustrates practical trade-offs for tensor cores and SM/CU datapaths.

Quantization maps floating values to an integer grid using a scale and optional zero-point; asymmetric quantization used in many inference flows is

$$[H]q = \text{round}\left(\frac{x - z}{s}\right), \quad x \approx s \cdot q + z, \quad (2.16)$$

where s is the scale, z the zero-point. Operational relevance: tensor cores perform many dot-products, so s and z must be applied (or fused) carefully to avoid expensive per-element operations; typical designs fuse requantization and bias-add into the final stage.

Accumulator width is critical to avoid overflow in K -length dot products. For signed integer operands of widths w_a and w_b the conservative accumulator bit requirement is

$$[H]w_{\text{acc}} \geq w_a + w_b + \lceil \log_2 K \rceil, \quad (2.17)$$

because worst-case magnitude grows by at most K times a product-sized value. For INT8 (signed, $w_a = w_b = 8$) and $K = 256$ this gives $w_{\text{acc}} \geq 8 + 8 + 8 = 24$, so hardware often provides 32-bit accumulators to permit headroom and fused bias/rescale.

Data packing and throughput: INT4 can be packed two per byte; tensor cores exploit this by issuing wider SIMD multiply units that unpack lanes or implement folding where one physical multiplier computes two INT4 products per cycle. However, packing raises alignment and sign-extension costs; the microarchitecture must include low-latency unpacking and sign/zero-point handling to sustain SM/CU issue rates.

Below is a synthesizable Verilog MAC module that performs a serial dot product across N pairs of signed integers (parameterizable for INT8/INT4); it demonstrates accumulator sizing, sign-aware multiplication, and pipelined accumulation suitable as a small tensor-core microkernel when replicated or time-multiplexed.

Listing 2.10: Parameterized serial MAC for INT4/INT8 dot product (synthesizable).

```

module quant_serial_mac #(
    parameter integer N = 16,                // length of dot product (K)
    parameter integer A_W = 8,                // width of operand A (INT8 or INT4)
    parameter integer B_W = 8,                // width of operand B
    parameter integer ACC_W = 32              // accumulator width (>= A_W+B_W+
        ceil(log2(N)))
) (
    input wire          clk,
    input wire          rst_n,
    input wire          start,                // start pulse
    input wire [N*A_W-1:0] a_vec,            // concatenated A
        lanes
    input wire [N*B_W-1:0] b_vec,            // concatenated B
        lanes

```

```

output reg signed [ACC_W-1:0] acc_out,
output reg done
);
integer idx;
reg [31:0] cnt;
reg signed [ACC_W-1:0] acc_r;
reg running;

wire signed [A_W-1:0] a_lane = '0;
wire signed [B_W-1:0] b_lane = '0;

// extract current lane each cycle
function automatic signed [A_W-1:0] get_a(input integer i);
    get_a = $signed(a_vec[i*A_W +: A_W]);
endfunction
function automatic signed [B_W-1:0] get_b(input integer i);
    get_b = $signed(b_vec[i*B_W +: B_W]);
endfunction

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cnt <= 0;
        acc_r <= 0;
        acc_out <= 0;
        done <= 0;
        running <= 0;
    end else begin
        if (start && !running) begin
            running <= 1;
            cnt <= 0;
            acc_r <= 0;
            done <= 0;
        end else if (running) begin
            // multiply-accumulate one lane per cycle
            acc_r <= acc_r + (get_a(cnt) * get_b(cnt));
            cnt <= cnt + 1;
            if (cnt == N-1) begin
                running <= 0;
                acc_out <= acc_r + (get_a(N-1) * get_b(N-1));
                done <= 1;
            end
        end else begin
            done <= 0; // clear done until next run
        end
    end
end
endmodule

```

Design implications and trade-offs:

- Providing 32-bit accumulators for INT8 avoids overflow and simplifies fusion of bias and scale; narrower accumulators save area but increase risk of saturation or require stochastic clipping.
- Supporting INT4 requires high-throughput unpack/sign-extension logic; area for packing/unpacking can be amortized by denser DRAM transfers and doubling effective compute density per tensor core.
- Requantization (requiring scale and zero-point) should be fused into final pipeline stages to avoid per-element FP multiplies; fixed-point rescaling or lookup-based multiplier approximations reduce latency at modest accuracy cost.
- Overall, INT8/INT4 enable substantial throughput and memory-bandwidth gains in ML inference, but hardware must provision sufficient accumulator width, low-latency conversion units, and data-packing engines to realize those gains without accuracy regressions.

2.3.3 Accumulator precision control

Building on quantization trade-offs and floating formats discussed earlier, accumulator precision control is the hardware knob that converts many low-precision multiply products into a numerically stable sum, while respecting SM/CU area, bandwidth, and power constraints. The following analyzes the numeric requirements, then presents a synthesizable fixed-point accumulator module that implements selectable rounding policies used in tensor cores and mixed-precision datapaths.

Problem and analysis:

- For a dot-product of length L with operand width w_{op} (bits, excluding sign), worst-case integer growth requires accumulator width ACCW satisfying

$$[H]\text{ACCW} \geq w_{\text{op}} + \lceil \log_2 L \rceil + g, \quad (2.18)$$

where g are guard bits to cover intermediate rounding and headroom for fused multiply-add (FMA) staging. This bound is practical for INT8/INT4 quantized GEMMs and for FP16-to-fixed staging used in tensor cores.

- Rounding policy affects error accumulation. If truncation (no rounding) shifts right by r bits, the per-add rounding error is bounded by 2^{-r} . With round-to-nearest (RN), the bias is reduced to $\pm 2^{-r-1}$. Stochastic rounding further reduces systematic bias for long training runs, improving convergence in ML workloads at modest logic cost.

Implementation approach:

- Use a wide signed accumulator (fixed-point) sized by Eq. (1). Accept incoming multiply results already scaled to the accumulator fixed-point position.
- Support three modes: truncate, round-to-nearest, stochastic rounding. Rounding is applied at commit time when producing a narrower output (e.g., FP32 store or INT32 write-back). Stochastic rounding uses a small LFSR for low-cost pseudo-random noise.

The Verilog below is a synthesizable, self-contained accumulator wrapper suitable for integration behind a systolic array or tensor core output stage. It implements signed accumulation, selectable rounding, and saturation to an output width.

Listing 2.11: Configurable fixed-point accumulator with truncate, RN, and stochastic rounding

```

module accumulator_precision_ctrl
#(
    parameter DATA_IN = 16,           // input product width
    parameter ACCW      = 48,           // accumulator width
    parameter OUT_W     = 32,           // output width after rounding
    parameter LFSR_W    = 16
)
(
    input  wire          clk,
    input  wire          rst,
    input  wire          in_valid,
    input  wire signed [DATA_IN-1:0] addend_in, // incoming signed product
    input  wire [1:0]      rnd_mode,      // 00=truncate,01=RN,10=
        stochastic
    input  wire [5:0]      shift_amt,      // right-shift amount on
        commit
    input  wire          commit,          // produce out_data
    output reg          out_valid,
    output reg signed [OUT_W-1:0] out_data
);

// accumulator register
reg signed [ACCW-1:0] acc_reg;
wire signed [ACCW-1:0] addext = {{(ACCW-DATA_IN){addend_in[DATA_IN-1]}},
    addend_in};

// simple LFSR for stochastic rounding
reg [LFSR_W-1:0] lfsr;
wire [LFSR_W-1:0] lfsr_next = {lfsr[LFSR_W-2:0], lfsr[LFSR_W-1] ^ lfsr[2]};

// rounding bias computation
reg signed [ACCW-1:0] bias;
integer sa;
always @(*) begin
    sa = shift_amt;
    if (sa == 0) begin
        bias = 0;
    end else begin
        case (rnd_mode)
            2'b01: begin // round-to-nearest: add +2^{sa-1} for positive, -2^{sa
                -1} for negative
                bias = (acc_reg >= 0) ? ({(ACCW-sa-1){1'b0}}, 1'b1, {(sa-1){1'b0}

```



```

        })))

        : -(((ACCW-sa-1){1'b0}}, 1'b1, {(sa-1){1'b0}
        }));

    end

    2'b10: begin // stochastic: add random integer in [0,2^{sa}-1]
        bias = {(ACCW-sa){1'b0}}, lfsr[sa-1:0]]; // treat as positive
        fraction

    end

    default: bias = 0; // truncate
endcase
end
end

// accumulation and LFSR update
always @(posedge clk) begin
    if (rst) begin
        acc_reg <= 0;
        lfsr    <= {LFSR_W{1'b1}};
        out_valid <= 0;
        out_data  <= 0;
    end else begin
        if (in_valid) acc_reg <= acc_reg + addext;
        lfsr <= lfsr_next;
        if (commit) begin
            // biased rounding then arithmetic shift
            if (shift_amt == 0) begin
                // direct saturation to OUT_W
                if (acc_reg > $signed({(ACCW-OUT_W){1'b0}}, {1'b0, {(OUT_W-1){1'b1}
                }))))
                    out_data <= $signed({1'b0, {(OUT_W-1){1'b1}}});
                else if (acc_reg < -($signed({1'b0, {(OUT_W-1){1'b1}}}) + 1))
                    out_data <= -($signed({1'b0, {(OUT_W-1){1'b1}}}) + 1);
                else
                    out_data <= acc_reg[OUT_W-1:0];
                end else begin
                    // add bias then arithmetic shift
                    reg signed [ACCW-1:0] tmp;
                    tmp = acc_reg + bias;
                    out_data <= tmp >>> shift_amt; // arithmetic right shift
                end
                out_valid <= 1'b1;
            end else begin
                out_valid <= 1'b0;
            end
        end
    end
end

```

endmodule

Operational implications and trade-offs:

- Area/power: larger ACCW and LFSR logic increase energy and RF area; add guard bits instead of wider outputs trades area for numerical stability.
- Throughput: commit-time rounding is cheap; stochastic rounding requires LFSR updates but can be pipelined to avoid throughput loss.
- Accuracy: RN reduces systematic bias; stochastic rounding improves ML training stability for low-precision accumulations at marginal hardware cost.
- Integration: place this module after the systolic array accumulation banks and before writeback or conversion to FP formats; choose ACCW per Eq. (1) to avoid overflow while minimizing register pressure.

2.3.4 Conversion and normalization units

The previous subsections established why accumulator width and low-bit quantization demand careful handling of intermediate precision and integer packing; conversion and normalization logic is the next critical stage that ensures tensors move between FP32, BF16, TF32, and FP16 domains without corrupting numerical properties or tensor-core throughput.

Problem — GPUs must perform billions of format conversions per second inside SMs and tensor cores while preserving IEEE behavior where required, minimizing latency, and providing options (flush-to-zero, deterministic rounding) for ML workloads. Key tasks: exponent bias translation, mantissa truncation with rounding-to-nearest-even, denormal handling, NaN/Inf propagation, and sticky-bit computation for correct rounding.

Analysis — exponent mapping between formats is an affine offset of biases. For converting IEEE-754 binary32 (bias 127) to binary16 (bias 15) the nominal exponent transform is:

$$[H]e_{16} = e_{32} - 127 + 15. \quad (2.19)$$

If $e_{16} \geq 31$ the result overflows to Inf; if $e_{16} \leq 0$ the value may become a denormal or zero. Mantissa reduction from 23 to 10 bits requires guard, round and sticky bits: guard is the first truncated bit, round the next, and sticky is the OR of the remaining lower bits. Rounding-to-nearest-even uses these to compute a one-bit increment and may carry into the exponent.

Implementation — a compact combinational converter can be pipelined in front of tensor cores to meet throughput. The following synthesizable Verilog converts a 32-bit float to IEEE-754 binary16 with parameterizable flush-to-zero (FTZ) and implements round-to-nearest-even, denormals, and NaN/Inf handling.

Listing 2.12: FP32-to-FP16 conversion, RN-even with FTZ option.

```
module fp32_to_fp16
(
  input  wire [31:0] in,      // IEEE-754 binary32
  output reg  [15:0] out
);
  parameter FTZ = 1'b1;      // flush-to-zero for denormals
```

```

wire sign = in[31];
wire [7:0] exp32 = in[30:23];
wire [22:0] frac32 = in[22:0];

// handle special cases
wire is_inf_nan = (exp32 == 8'hFF);
wire is_zero = (exp32 == 8'd0) && (frac32 == 23'd0);

// extended significand with room for shifting and rounding bits
wire [36:0] ext = {1'b1, frac32, 13'b0}; // 1+23+13=37

// compute target exponent as signed to detect underflow/overflow
integer e32_i = 0;
integer e16_i = 0;
always @* e32_i = exp32;
always @* e16_i = e32_i - 127 + 15;

// denorm shift (if any)
integer denorm_shift = 0;
always @* denorm_shift = (e16_i <= 0) ? (1 - e16_i) : 0;

// shifted value for extracting mantissa, guard, round, sticky
wire [36:0] shifted = ext >> (13 + denorm_shift);
wire [9:0] mant10 = shifted[23:14];
wire guard = shifted[13];
wire roundb = shifted[12];
wire sticky = |shifted[11:0];

// rounding decision (round-to-nearest-even)
wire round_add = guard & (roundb | sticky | mant10[0]);

reg [9:0] mant_rounded;
reg [4:0] exp_out; // 5-bit exponent for FP16

always @* begin
    if (is_inf_nan) begin
        exp_out = 5'h1F; // all ones
        mant_rounded = (frac32 == 0) ? 10'd0 : 10'h200; // NaN payload -> qNaN
    end else if (is_zero) begin
        exp_out = 5'd0;
        mant_rounded = 10'd0;
    end else begin
        // normalized or denorm path
        reg [10:0] mant_plus;
        mant_plus = {1'b0, mant10} + round_add;
        exp_out = e16_i[4:0];
        // handle mantissa overflow from rounding

```

```

if (mant_plus[10]) begin
    mant_rounded = 10'd0;
    exp_out = exp_out + 1;
end else begin
    mant_rounded = mant_plus[9:0];
end
// exponent overflow -> Inf
if (e16_i >= 31) begin
    exp_out = 5'h1F;
    mant_rounded = 10'd0;
end
// underflow handling: if still <=0 after rounding, FTZ or denorm
if (e16_i <= 0 && FTZ) begin
    exp_out = 5'd0;
    mant_rounded = 10'd0;
end
end
out = {sign, exp_out, mant_rounded};
end
endmodule

```

Implications and trade-offs:

- Hardware cost: sticky-bit OR trees and wide shifters increase area and critical path; pipelining the conversion stage (one cycle) is common to meet SM clock targets.
- Accuracy vs throughput: enabling FTZ simplifies logic and reduces latency but loses IEEE denormal semantics; for ML inference FTZ is often acceptable.
- Integration: place these units at tensor-core inputs/outputs and pack multiple lanes per cycle to match tensor core width, balancing LUT/FF area against per-lane latency.
- Power and timing: variable shifts and wide ORs dominate power; restrict dynamic range handling to fast-path normalize/overflow detection for critical pipelines.

2.4 Tensor Memory Layout

2.4.1 Row-major and column-major ordering

Building on the tensor core dataflow and accumulation patterns, the choice between row-major and column-major ordering directly affects memory coalescing, shared-memory tiling, and tensor-core operand packing. The mapping from 2-D indices to linear addresses determines whether contiguous threads in a warp perform contiguous loads, and thus whether accesses form optimal memory transactions.

Row-major stores row i , column j at linear index $i \cdot N + j$, while column-major stores the same element at $i + j \cdot M$. Formally, for an $M \times N$ matrix A :

$$[H]\text{row-major: } \text{idx}_{\text{RM}}(i, j) = i \cdot N + j, \quad \text{column-major: } \text{idx}_{\text{CM}}(i, j) = i + j \cdot M. \quad (2.20)$$

On SIMT hardware, a warp of 32 threads usually maps thread t to contiguous j indices when threads iterate across columns; thus row-major yields contiguous memory for row-wise traversal, enabling single wide DRAM transactions and coalescing into few cache lines. Conversely, column-major favors column-wise traversal.

Analysis for tensor-accelerated GEMM:

- When tiling for a tensor core or systolic array, choose inner-loop traversal that results in contiguous linear addresses for the threads that issue the vector loads.
- Libraries matter: many established BLAS implementations, including classical cuBLAS, adopt column-major ordering by BLAS/Fortran convention; cublasLt and vendor tensor APIs provide descriptors to handle either layout without explicit transposes.

Practical implementation note: use leading-dimension parameters to express stride between rows or columns, avoiding data reallocation. Below is a concise CUDA kernel demonstrating how to index both layouts in a tiled multiply, where `ldA` and `ldB` are leading dimensions (stride between successive rows in memory layout convention).

Listing 2.13: Tiled kernel fragment showing layout-aware indexing.

```
__global__ void matmul_layout(const float* A, const float* B, float* C,
                             int M, int N, int K, int ldA, int ldB) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float acc = 0.0f;

    for (int t = 0; t < (K + blockDim.x - 1) / blockDim.x; ++t) {
        int a_col = t * blockDim.x + threadIdx.x;
        int b_row = t * blockDim.y + threadIdx.y;
        float a_val = (row < M && a_col < K) ? A[row * ldA + a_col] : 0.0f;
        float b_val = (b_row < K && col < N) ? B[b_row * ldB + col] : 0.0f;
        acc += a_val * b_val;
    }

    if (row < M && col < N)
        C[row * N + col] = acc;
}
```

2.4.2 Tiled and blocked formats

Building on the linear-order discussion, tiled and blocked layouts reorganize linear storage into small contiguous tiles to improve spatial locality and hardware-friendly reuse. These layouts explicitly trade simple index arithmetic for reduced DRAM transactions, better cache-line utilization, and predictable access patterns for SIMT warps and tensor hardware.

Tiling partitions an $M \times N$ matrix into tiles of size $T \times T$. The number of tiles along each dimension is

$$[H]N_{\text{tiles}}^{(r)} = \left\lceil \frac{M}{T} \right\rceil, \quad N_{\text{tiles}}^{(c)} = \left\lceil \frac{N}{T} \right\rceil, \quad (2.21)$$

so total tiles $N_{\text{tiles}} = N_{\text{tiles}}^{(r)} N_{\text{tiles}}^{(c)}$. Each tile is stored contiguously (blocked) so a SIMT warp performing a row or column of a tile issues coalesced loads into L1/TMU and then reuses the tile in shared memory or registers for multiple MACs.

Analysis — design goals and pitfalls:

- Tile size T is chosen to match microarchitectural units: common choices are $T = 16$ or 32 to map naturally to warp fragments and tensor cores (e.g., 16×16 WMMA fragments). A tile yields reuse of up to T elements per loaded cache line, increasing arithmetic intensity.
- Bank conflicts in shared memory arise when multiple threads in a warp address the same bank. For 32 banks of 4-byte words, bank index is $\text{bank} = (\text{byte_address}/4) \bmod 32$. If threads access elements with stride s such that $s \cdot \text{sizeof}(\text{element})$ is a multiple of 128 bytes, conflict occurs. A minimal mitigation is row-padding by one element.
- Swizzling (e.g., Z-order or Morton curves) further reduces contention at higher memory levels and evens out hot spots across memory controllers, at the cost of slightly more complex address computation.

Implementation example — tiled GEMM kernel demonstrating padding to avoid bank conflicts and tile reuse in shared memory:

Listing 2.14: Simple tiled GEMM with shared-memory padding to avoid bank conflicts

```
template
__global__ void tiled_gemm(const float* A, const float* B, float* C,
                          int M, int N, int K) {
    __shared__ float sA[TILE][TILE+1]; // pad to avoid bank conflicts
    __shared__ float sB[TILE][TILE+1];
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float acc = 0.0f;
    for (int t = 0; t < (K + TILE - 1)/TILE; ++t) {
        int a_col = t * TILE + threadIdx.x;
        int b_row = t * TILE + threadIdx.y;
        // cooperative loads (with boundary checks)
        sA[threadIdx.y][threadIdx.x] = (row < M && a_col < K) ?
                                         A[row*K + a_col] : 0.0f;
        sB[threadIdx.y][threadIdx.x] = (b_row < K && col < N) ?
                                         B[b_row*N + col] : 0.0f;
        __syncthreads();
        #pragma unroll
        for (int k = 0; k < TILE; ++k) acc += sA[threadIdx.y][k] * sB[k][threadIdx
            .x];
        __syncthreads();
    }
    if (row < M && col < N) C[row*N + col] = acc;
}
```

Concrete design implications and trade-offs:

- Choose T to balance reuse and occupancy: larger T increases reuse but raises shared-memory and register pressure, lowering concurrent SM occupancy.

- Padding and swizzling reduce bank conflicts and DRAM hot spots but consume extra memory and complicate address math.
- Alignment to cache-line and tensor-core fragment sizes maximizes TMU/L1/L2 efficiency; misalignment causes extra transactions and stalls.
- Hardware-friendly tiling simplifies dataflow to tensor cores and systolic arrays, but scheduling and DMA logic must support tile-level transfers and scatter/gather for edge tiles.

2.4.3 Swizzling for conflict avoidance

Following blocked tile placement and canonical row/column ordering, the practical challenge is to map those logical tiles to physical addresses so that simultaneous SIMT accesses from a warp do not concentrate on the same memory bank, L2 slice, or DRAM channel. Swizzling (address remapping) injects low-cost bit mixing into the address generation path to spread concurrent accesses across hardware resources, preserving locality where it matters for reuse while avoiding conflict hotspots.

Problem: Tensor cores and their associated TMU/SM pipelines issue many wide, regular loads from tiled matrices. If lane-to-address mapping aligns multiple lanes to the same bank slice or DRAM bank, throughput collapses due to serialization and backpressure. Analysis begins by expressing the canonical linear tile offset and a lane-dependent permutation. A commonly effective mapping is an XOR-based mix of a tile-derived offset and a lane-derived offset:

$$[H]A_{\text{phys}} = A_{\text{base}} + \left((T \ll t) \oplus (L \ll \ell) \right), \quad (2.22)$$

where T is the tile linear index, L is the warp lane identifier, t is tile-size $\log 2$, and ℓ aligns the lane contribution to element granularity. The XOR moves low-entropy lane variance into higher-order address bits that hardware uses for bank and slice selection, reducing the probability that multiple lanes map to the same resource.

Implementation: XOR swizzling is cheap and combinational, suitable to perform in the SM address generator feeding the load/store unit or in a dedicated DMA prefetcher. Alternative patterns include rotation-based mixing, bit interleaving (Morton order), and prime-modulo hashing for channel balancing. The XOR pattern preserves linearity for contiguous coalesced accesses while breaking pathological alignments that cause bank collisions.

Practical Verilog implementation (synthesizable) that performs tile-plus-lane XOR swizzle for a tensor load address:

Listing 2.15: Simple XOR swizzle address generator

```

module swizzle_addr_gen #(
  parameter integer ADDR_WIDTH = 64,
  parameter integer TILE_SHIFT = 6, // tile size = 64 bytes
  parameter integer LANE_SHIFT = 5 // lane granularity
) (
  input  wire [ADDR_WIDTH-1:0] base_addr, // base tile address
  input  wire [31:0]          tile_idx,  // linear tile index
  input  wire [31:0]          lane,      // warp lane id
  output wire [ADDR_WIDTH-1:0] swizzled_addr

```

```

);
// extend and shift to ADDR_WIDTH then XOR and add
wire [ADDR_WIDTH-1:0] tile_off = {(ADDR_WIDTH-32){1'b0}}, tile_idx} <<
    TILE_SHIFT;
wire [ADDR_WIDTH-1:0] lane_off = {(ADDR_WIDTH-32){1'b0}}, lane} <<
    LANE_SHIFT;
assign swizzled_addr = base_addr + (tile_off ^ lane_off); // combinational
endmodule

```

Implications and trade-offs:

- Benefits: markedly reduces SM stall cycles from bank/channel contention; improves effective DRAM throughput and tensor core utilization for strided and transposed operands.
- Costs: small extra XOR and shift logic in the critical path, potential impact on prefetcher prediction and contiguous streaming locality; cache indexing must be coherent with swizzle to avoid aliasing effects.
- Design knobs: choose t and ℓ to align with cache line size, bank granularity, and memory-controller slice hashing. Measure conflict rate reduction and critical-path timing when enabling swizzle in the SM address path.

2.4.4 Efficient memory access patterns

The previous discussion showed how tiling and swizzling reduce contention and improve reuse for tensor tiles; here we connect those formats to concrete access patterns and implementation choices that maximize throughput for tensor and matrix acceleration.

Problem: feeding tensor cores requires sustained, aligned memory streams from VRAM through L2, L1, and SM-local caches into shared memory and registers. Misaligned, strided, or bank-conflicting accesses convert single wide transfers into multiple cache-line transactions, increasing latency and lowering utilization of tensor-math units. To analyze this, model a tile load where each row carries W elements of size s bytes, and the system cache line is L bytes. The number of cache-line transactions per row is $\lceil (W \cdot s / L) \rceil$, and for an R -row tile the total transactions approximate

$$T = \sum_{r=0}^{R-1} \left\lceil \frac{W_r \cdot s}{L} \right\rceil, \quad (2.23)$$

with W_r equal to effective contiguous elements on row r after swizzling or padding. Reducing T directly improves achievable GFLOPS because tensor cores can remain saturated when memory can supply operands at the needed bandwidth.

Analysis: optimal patterns minimize wasted bytes per transfer and avoid shared-memory bank conflicts. For typical GPUs with 128-byte cache lines and 32-bit elements, up to 32 contiguous elements fit a line. Therefore:

- Coalesce across the warp so each lane accesses adjacent elements in the same cache line.
- Use padding in shared memory to break modulo-bank alignment that causes conflicts; for a 32-bank shared memory, add a small pad P so $\text{stride} \bmod 32 \neq 0$ when necessary.
- Swizzle global layout so sub-tiles destined to different SMs interleave across channels, improving L2/DRAM parallelism.

Implementation: the following CUDA-style kernel shows pragmatic rules: aligned global loads into shared memory, warp-cooperative indexing, and a one-element pad to avoid 32-bank conflict when storing rows.

Listing 2.16: Warp-cooperative tile load with shared-memory padding

```
extern "C" __global__ void loadTile(const float* __restrict__ A, float*
    __restrict__ tileOut,
                                int M, int N, int lda) {
    __shared__ float tile[32][33]; // 32 banks, 1-element pad to avoid conflicts
    int tx = threadIdx.x; int ty = threadIdx.y; // thread block maps tile
    int row = blockIdx.y*32 + ty;
    int colBase = blockIdx.x*32 + tx*warpSize; // warp-aligned base
    // coalesced loads: each lane loads contiguous elements
    #pragma unroll
    for (int k=0; k<32; k+=warpSize) {
        int col = colBase + k + (laneId()); // laneId computed in warp
        if (row < M && col < N) tile[ty][tx*warpSize + k + laneId()] = A[row*lda +
            col];
        else tile[ty][tx*warpSize + k + laneId()] = 0.0f;
    }
    __syncthreads();
    // write tileOut in format expected by tensor core (e.g., column-major
    fragment)
    if (ty < 32 && tx < 32) tileOut[(blockIdx.y*32+ty)*N + blockIdx.x*32 + tx] =
        tile[ty][tx];
}
```

Implications and trade-offs:

- Padding and swizzling reduce cache-line waste and conflicts but consume extra memory and increase address arithmetic complexity.
- Larger tiles improve arithmetic intensity but raise shared-memory and register pressure, lowering occupancy on an SM.
- Optimal design balances tile size, pad overhead, and warp-cooperative loads to sustain tensor-core utilization while preserving L2/DRAM parallelism.

2.5 Sparse Matrix Acceleration

2.5.1 CSR and COO representations

Building on the tiled dataflow and accumulator precision topics previously discussed for tensor cores and memory layout, we now analyze how sparse matrix formats map to GPU resources and tensor-accelerator workflows. The choice between CSR (Compressed Sparse Row) and COO (Coordinate) affects SIMD/SIMT mapping, memory coalescing, and zero-skipping hardware complexity.

Problem: implement efficient sparse-dense multiplication and SpMV on SMs and tensor cores while minimizing memory transfers and thread divergence. Analysis:

- CSR stores values in three arrays: `val[nnz]`, `col_idx[nnz]`, and `row_ptr[rows+1]`. Storage cost is

$$[H]\text{bytes}_{\text{CSR}} \approx nnz \cdot (\text{sizeof}(v) + \text{sizeof}(i)) + (\text{rows} + 1) \cdot \text{sizeof}(i). \quad (2.24)$$

- COO stores triplets `(row[nnz], col[nnz], val[nnz])`, with cost

$$[H]\text{bytes}_{\text{COO}} \approx nnz \cdot (2 \cdot \text{sizeof}(i) + \text{sizeof}(v)). \quad (2.25)$$

CSR reduces indexing overhead for row-major scans, while COO is simpler for parallel insertion and reordering.

Implementation considerations on modern GPUs:

1. Work decomposition

- CSR: assign one thread block or one warp per row for high *nnz*-per-row; use warp-level primitives to accumulate and reduce to the output element. Works well when nonzeros are clustered, enabling coalesced loads of `val` and `col_idx`.
- COO: assign one thread per nonzero for maximum parallelism; requires an atomic accumulation to output vector entries or a segmented-scan post-sort by row to avoid atomics.

2. Memory access and coalescing

- Pack `val` and `col_idx` so consecutive threads load adjacent elements. Align arrays to 128-byte cache lines for L1/L2 prefetchers.
- For tensor-core use, convert sparse blocks into small dense tiles (e.g., 16×16) using gather and scatter to feed matrix multiply-accumulate units.

3. Reduction and atomics

- COO naive kernel uses atomics on global memory; this creates contention when many nonzeros map to the same row. Use shared-memory buffered accumulation per block followed by a single atomic write, or perform radix-sort by row then segmented reduction.

Example CUDA-style CSR SpMV kernel with warp-level reduction (one warp per row mapping). Comments are brief inline for clarity.

Listing 2.17: Warp-per-row CSR SpMV kernel (simplified).

```
__global__ void spmv_csr_warp( int rows, const int *row_ptr,
                             const int *col_idx, const float *val,
                             const float *x, float *y ){
    int warp_id = (blockIdx.x * blockDim.x + threadIdx.x) >> 5; // warp index
    int lane = threadIdx.x & 31;
    if(warp_id >= rows) return;
    int row_start = row_ptr[warp_id], row_end = row_ptr[warp_id+1];
    float sum = 0.0f;
    // each lane processes a strided subset of nonzeros in the row
```

```

for(int idx = row_start + lane; idx < row_end; idx += 32){
    int c = col_idx[idx];           // column index
    float v = val[idx];             // nonzero value
    sum += v * x[c];                // multiply-add
}
// warp-wide reduction using shuffle XOR
for(int offset=16; offset>0; offset>>=1) sum += __shfl_xor_sync(0xffffffff,
    sum, offset);
if(lane==0) y[warp_id] = sum;      // write result
}

```

Implications and trade-offs:

- CSR minimizes storage and supports efficient row-wise scans, best when rows have many nonzeros and load is balanced across warps. It reduces index bandwidth but may suffer from row imbalance and warp divergence.
- COO enables fine-grained parallelism and simple insertion pipelines, but requires atomic accumulation or expensive sorting/segmentation to form outputs, increasing memory traffic.
- For tensor cores, hybrid pipelines that convert sparse blocks into dense tiles (block-CSR or blocked-COO) yield the best throughput by amortizing gather/scatter overhead into dense MMA operations.

Designers should choose format based on sparsity pattern: use CSR for structured row-dense clusters, COO or blocked formats for highly irregular sparsity, and always consider hardware-friendly blocking to exploit SM and tensor-core throughput.

2.5.2 Structured sparsity (2:4, 4:8)

Following CSR/COO, structured sparsity constrains the pattern of zeros so hardware can decompress and map data deterministically into tensor pipelines; this subsection examines how 2:4 and 4:8 formats trade storage and compute for simplified zero-skipping in SM/CU-level tensor cores.

Structured sparsity problem: enable the tensor core (systolic array) to skip zero multiply-accumulate (MAC) work without per-element pointers or irregular memory fetches. Analysis shows a fixed-group encoding (e.g., 2 nonzeros per 4 elements) yields constant metadata and predictable placement, which aligns with SIMT warp lanes and register banking.

Storage and throughput models:

- For a W -bit element, a 2:4 block stores two values plus a 4-bit presence mask. Per-block storage in bits:

$$S_{2:4} = 2W + 4. \quad (2.26)$$

- Effective density is $\rho = 2/4 = 0.5$, so peak compute ideally halves loaded data and MACs; raw arithmetic reduction is ρ , but end-to-end speedup is reduced by decompression and alignment overhead. If T_{mem} is memory time per uncompressed element and T_{dec} is per-block decompression latency, a first-order speedup for memory-bound regimes:

$$S \approx \frac{1}{\rho + \frac{T_{\text{dec}}}{T_{\text{mem}} \cdot 4}}. \quad (2.27)$$

Implementation: hardware must (1) read packed values plus mask, (2) expand into fixed-width groups feeding TMU/Tensor core lanes with minimal stall, and (3) preserve memory coalescing. Practical datapath:

- Store compressed stream aligned to cache lines where each 2:4 block is word-aligned to avoid unaligned reads.
- Decompressor does a deterministic scatter: iterate 4 mask bits, consume next input value only when a bit is set, else generate zero. This keeps per-block work bounded and mask decode combinational.

Example synthesizable Verilog module: expands a single 2:4 block of W -bit elements into four outputs ready for a tensor core lane bundle.

Listing 2.18: 2:4 decompressor — expands two packed values and a 4-bit mask to four W -bit outputs

```

module decompressor_2to4 #(parameter W=16) (
    input  wire          clk,
    input  wire          rst_n,
    input  wire          valid_in,                // new block valid
    input  wire [2*W-1:0] packed_vals,           // packed nonzeros (MSB
        first)
    input  wire [3:0]     mask,                  // presence mask bit0->
        pos0
    output reg          valid_out,
    output reg  [W-1:0]  out0, out1, out2, out3  // expanded outputs
);
    // combinational expansion
    integer i; integer j; reg [W-1:0] tmp_out [0:3];
    always @* begin
        j = 0;
        for (i=0; i<4; i=i+1) begin
            if (mask[i]) begin
                tmp_out[i] = packed_vals[(2-j-1)*W +: W]; // consume next packed (MSB-
                    first)
                j = j + 1;
            end else tmp_out[i] = {W{1'b0}};
        end
    end
    // register outputs
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin valid_out <= 1'b0; out0<=0; out1<=0; out2<=0; out3<=0;
        end
        else begin
            if (valid_in) begin
                out0 <= tmp_out[0]; out1 <= tmp_out[1];
                out2 <= tmp_out[2]; out3 <= tmp_out[3];
                valid_out <= 1'b1;
            end else valid_out <= 1'b0;
        end
    end

```

```

    end
  end
endmodule

```

Operational notes and trade-offs:

- Align 2:4 blocks to cache lines to preserve coalescing and minimize TLB pressure.
- Choose element width W matching TMU/Tensor core precision (FP16/BF16/INT8).
- Latency of decompressor T_{dec} must be balanced against warp-level scheduling; pipelined decompression reduces critical path at the cost of area.
- Structured sparsity yields deterministic bandwidth reduction and easier mapping to tensor cores but limits achievable sparsity patterns (less flexible than CSR) and requires model-aware pruning to avoid accuracy loss.

2.5.3 Zero-skipping hardware logic

Following structured sparsity formats and CSR/COO index streams, zero-skipping logic is the next hardware step that converts sparse encodings into active compute predicates for tensor units. The unit must reconcile compression metadata with SIMT and systolic dataflow to avoid wasted MAC cycles while preserving high utilization across SMs and tensor cores.

Zero-skipping solves the problem of eliminating multiply-accumulate operations associated with zero-valued operands. Analysis proceeds from two angles:

1. bandwidth: compressed sparse formats reduce DRAM reads but require on-chip decompression and index expansion; and
2. compute: gating or bypassing MAC lanes reduces arithmetic energy and cycles but can introduce imbalance across lanes and stalls in systolic arrays.

A basic cost model quantifies benefit. Let F_{total} be total FLOPs for dense GEMM and d the nonzero density. Saved FLOPs are

$$[H]S = (1 - d) F_{\text{total}}, \quad (2.28)$$

and peak effective compute scales to approximately $\text{Peak} \times d$ when zero-skipping perfectly removes wasted work and keeps pipelines full.

Implementation considerations:

- Input: compressed value+index streams from CSR/COO or structured blocks.
- Decoder: index expansion and address generation to align dense tile coordinates for the tensor core.
- Predicate generation: produce per-lane enables to gate MACs or skip input enqueues in a systolic feed.
- Backpressure: maintain valid-ready handshakes so that skipping does not cause bubble accumulation in upstream memory controllers or warp schedulers.

The following synthesizable Verilog module implements a pipelined zero-detect and mask generator for N parallel lanes. It compares each 32-bit value to zero, produces a per-lane enable, and outputs a popcount of active lanes for downstream scheduling. This small block is intended for integration in a pre-processing stage feeding tensor cores or MAC arrays.

Listing 2.19: Zero-skip mask generator for N lanes

```

module zero_skip_mask #(
    parameter LANES = 8,
    parameter DATA_WIDTH = 32
) (
    input wire          clk,
    input wire          rstn,
    input wire          in_valid,                // incoming
        compressed lane vector valid
    input wire [LANES*DATA_WIDTH-1:0] in_data_flat, // concatenated
        lane values
    output reg          out_valid,                // output valid
        for downstream
    output reg [LANES-1:0] out_enable,            // per-lane non-
        zero enables
    output reg [$clog2(LANES+1)-1:0] out_active_count, // number of non-
        zero lanes
    output reg [LANES*DATA_WIDTH-1:0] out_data_flat // passed-through
        data
);

    // local unpack
    genvar i;
    wire [LANES-1:0] nz;
    wire [DATA_WIDTH-1:0] in_data [0:LANES-1];
    generate
        for (i = 0; i < LANES; i = i + 1) begin : UNPACK
            assign in_data[i] = in_data_flat[i*DATA_WIDTH +: DATA_WIDTH];
            assign nz[i] = |in_data[i]; // non-zero detect per lane
        end
    endgenerate

    integer j;
    reg [$clog2(LANES+1)-1:0] count;

    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            out_valid <= 1'b0;
            out_enable <= 'b0;
            out_active_count <= 'b0;
            out_data_flat <= 'b0;
        end else begin

```

```

    out_valid <= in_valid;
    if (in_valid) begin
        out_enable <= nz;
        count = 0;
        for (j = 0; j < LANES; j = j + 1)
            if (nz[j]) count = count + 1;
        out_active_count <= count;
        out_data_flat <= in_data_flat;
    end
end
end
endmodule

```

2.5.4 Compression and decompression paths

The previous discussion showed how zero-skipping reduces arithmetic work and how structured sparsity (for example 2:4) makes regular patterns that simplify dataflow; the compression/decompression path is the bridge that carries sparse payloads and metadata into dense, tiled inputs usable by tensor cores. It must minimize latency and metadata overhead while producing warp-aligned vectors that preserve coalescing for SM memory ports and tensor-core input lanes.

Problem statement and analysis. GPUs require contiguous, predictable tile inputs for high-utilization of tensor cores. Compressed representations reduce DRAM traffic but add metadata and decode cost in the datapath. Two practical schemes are common:

- block-bitmask (used with structured sparsity): small per-block masks indicate positions of nonzeros inside a fixed block (for 2:4, a 4-bit mask with two ones).
- index-value streams (CSR-like) for irregular sparsity, which require variable-length index arithmetic and greater metadata.

Operational relevance precedes equations: effective bandwidth savings R depends on the fraction of stored nonzeros f and the per-element metadata overhead m (in elements per matrix). The idealized reduction is

$$[H]R = 1 - \left(f + \frac{m}{MN}\right), \quad (2.29)$$

where MN is total elements and 0

2.6 Section 6: Verilog Implementation

2.6.1 Systolic array module

This subsection continues the tensor-core hardware discussion by turning the systolic organization and dataflow patterns into a concrete, synthesizable Verilog module that can be instantiated inside an SM/CU level tensor datapath. The implementation follows the skewed-input streaming model previously described for weight reuse and local accumulation.

A systolic array implements GEMM by streaming A rows from the left and B columns from the top into a two-dimensional grid of processing elements (PEs). Each PE performs a multiply-accumulate (MAC), forwards its A operand to the right, and its B operand downward, enabling

spatial reuse and minimizing external memory traffic. The design goals are: deterministic latency, simple control, and parameterizable dimensions for scalability.

Analysis:

- For an $M \times K$ times $K \times N$ multiplication realized on an $M \times N$ systolic array, the steady-state generation of outputs requires

$$[H]T_{\text{cycles}} = K + \max(M, N) - 1 \quad (2.30)$$

cycles to produce the full $M \times N$ result after the first valid inputs enter the array. This formula highlights the trade-off: larger arrays reduce the effective per-element amortized memory bandwidth but increase on-chip area and routing complexity.

Implementation strategy:

- Each PE contains registers for local A and B, a multiply unit, and an accumulator register.
- Inputs: per-row A streams and per-column B streams; single-cycle valid signals allow pipeline gating.
- Control: simple start and flush semantics; no complex ready/valid backpressure inside the array simplifies timing closure in an SM datapath.

The following Verilog is synthesizable RTL for a parameterizable systolic array with PE grid, streaming interfaces, and per-PE accumulation outputs. It assumes two's-complement signed arithmetic; change to unsigned if needed.

Listing 2.20: Parameterizable systolic array (PE + array).

```
`timescale 1ns/1ps
module pe #(
  parameter DATA_W = 16,
  parameter ACC_W = 48
) (
  input wire clk,
  input wire rst,
  input wire a_valid, // new A available this cycle
  input wire b_valid, // new B available this cycle
  input wire signed [DATA_W-1:0] a_in,
  input wire signed [DATA_W-1:0] b_in,
  input wire signed [DATA_W-1:0] a_right_in, // from left neighbor (shifted)
  input wire signed [DATA_W-1:0] b_down_in, // from top neighbor (shifted)
  output reg signed [DATA_W-1:0] a_right_out, // to right neighbor
  output reg signed [DATA_W-1:0] b_down_out, // to bottom neighbor
  output reg signed [ACC_W-1:0] acc_out
);

reg signed [DATA_W-1:0] a_reg, b_reg;
reg signed [ACC_W-1:0] acc;

always @(posedge clk) begin
  if (rst) begin
```



```

    a_reg <= 0; b_reg <= 0; acc <= 0;
    a_right_out <= 0; b_down_out <= 0; acc_out <= 0;
end else begin
    // load new operands if valid, else use shifted values
    a_reg <= a_valid ? a_in : a_right_in;
    b_reg <= b_valid ? b_in : b_down_in;
    // forward operands for neighbor routing
    a_right_out <= a_reg;
    b_down_out <= b_reg;
    // MAC: multiply current local operands and accumulate
    acc <= acc + $signed(a_reg) * $signed(b_reg);
    acc_out <= acc;
end
end
endmodule

module systolic_array #(
    parameter DATA_W = 16,
    parameter ACC_W = 48,
    parameter M = 4, // rows
    parameter N = 4 // cols
) (
    input wire clk,
    input wire rst,
    input wire start,
    input wire [M*DATA_W-1:0] a_vec_in, // concatenated per-row A input
    input wire [N*DATA_W-1:0] b_vec_in, // concatenated per-col B input
    input wire a_valid,
    input wire b_valid,
    output reg [M*N*ACC_W-1:0] c_vec_out, // concatenated per-PE
        accumulators
    output reg out_valid
);
    // internal buses for inter-PE routing
    wire signed [DATA_W-1:0] a_bus [0:M][0:N];
    wire signed [DATA_W-1:0] b_bus [0:M][0:N];
    wire signed [ACC_W-1:0] acc_bus [0:M-1][0:N-1];

    genvar i, j;
    generate
        // connect input vectors to first row/column
        for (i = 0; i < M; i = i + 1) begin
            assign a_bus[i][0] = a_vec_in[i*DATA_W +: DATA_W];
        end
        for (j = 0; j < N; j = j + 1) begin
            assign b_bus[0][j] = b_vec_in[j*DATA_W +: DATA_W];
        end
    endgenerate

```

```

// instantiate the array of processing elements
for (i = 0; i < M; i = i + 1) begin : ROW
  for (j = 0; j < N; j = j + 1) begin : COL
    pe #(.DATA_W(DATA_W), .ACC_W(ACC_W)) u_pe (
      .clk(clk),
      .rst(rst),
      .a_valid(a_valid && (j == 0)),
      .b_valid(b_valid && (i == 0)),
      .a_in(a_bus[i][j]),
      .b_in(b_bus[i][j]),
      .a_right_in(a_bus[i][j]),
      .b_down_in(b_bus[i][j]),
      .a_right_out(a_bus[i][j+1]),
      .b_down_out(b_bus[i+1][j]),
      .acc_out(acc_bus[i][j])
    );
  end
end
endgenerate

// pack output accumulators
integer m, n;
always @(posedge clk) begin
  if (rst) begin
    c_vec_out <= 0;
    out_valid <= 0;
  end else begin
    out_valid <= start;
    for (m = 0; m < M; m = m + 1)
      for (n = 0; n < N; n = n + 1)
        c_vec_out[(m*N + n)*ACC_W +: ACC_W] <= acc_bus[m][n];
  end
end
endmodule

```

2.6.2 Matrix multiply-accumulate block

The systolic array module established the dataflow and tile boundaries for streaming partial products into local MAC lanes; this subsection drills into the single matrix multiply-accumulate (MAC) block that forms each lane, showing how it implements the core inner-product update used by tensor cores and systolic arrays.

Problem: implement a pipelined, parameterized MAC suitable for high-throughput SIMT and tensor-core fabrics where each cycle produces a partial accumulation $C_{i,j} += A_{i,k} \cdot B_{k,j}$.
 Analysis: the MAC must

1. match the multiplier/adder latency to keep the systolic schedule tight,

2. provide sufficient accumulator headroom to avoid overflow across K summed terms, and
3. support low-latency handshaking for backpressure.

For fixed-point inputs of widths W_A and W_B and a summation length K , an accumulator width that avoids overflow in the worst case is

$$\lceil H \rceil W_{\text{acc}} = W_A + W_B + \lceil \log_2 K \rceil, \quad (2.31)$$

so the product is sign-extended into the accumulator and added each cycle. Implementation choices include signed vs unsigned arithmetic, optional saturation, and how many pipeline registers (multiplier and adder stages) to insert; latency choices affect tile scheduling across the systolic array.

Below is a synthesizable, parameterized Verilog implementation of a single-cycle-throughput, pipelined fixed-point MAC with a one-deep input buffer and configurable latency. It uses sign-extension of the product into the accumulator width and a simple valid/ready handshake.

Listing 2.21: Synthesizable fixed-point pipelined MAC lane

```

module mac_fixed #(
  parameter A_WIDTH = 16,
  parameter B_WIDTH = 16,
  parameter ACC_WIDTH = 48,
  parameter LATENCY = 2 // 1..N pipeline stages (mult+add)
) (
  input wire clk,
  input wire rst, // synchronous reset
  input wire in_valid, // input valid
  output reg in_ready, // backpressure
  input wire [A_WIDTH-1:0] a, // operand A (fixed-point)
  input wire [B_WIDTH-1:0] b, // operand B (fixed-point)
  input wire [ACC_WIDTH-1:0] acc_in, // incoming accumulator
  output reg out_valid, // output valid
  output reg [ACC_WIDTH-1:0] acc_out // updated accumulator
);
  // local widths
  localparam P_WIDTH = A_WIDTH + B_WIDTH;
  // pipeline registers
  reg [P_WIDTH-1:0] prod_reg;
  reg [ACC_WIDTH-1:0] acc_reg;
  reg valid_reg;

  // simple one-cycle multiplier + adder pipeline (LATENCY=2 implies 1 reg
  // between)
  always @(posedge clk) begin
    if (rst) begin
      prod_reg <= 0;
      acc_reg <= 0;
      valid_reg <= 0;
    end
  end

```

```

in_ready <= 1'b1;
out_valid <= 1'b0;
acc_out <= 0;
end else begin
    // accept when pipeline free (simple single-entry buffering)
    in_ready <= ~valid_reg;
    if (in_valid && in_ready) begin
        // signed multiply using Verilog signed cast
        prod_reg <= $signed(a) * $signed(b); // product width P_WIDTH
        valid_reg <= 1'b1;
    end
    // pipeline stage(s): for LATENCY>1 add extra regs if needed (omitted
    // loop for clarity)
    if (valid_reg) begin
        // sign-extend product into ACC_WIDTH and add to acc_in
        acc_reg <= acc_in + {{(ACC_WIDTH-P_WIDTH){prod_reg[P_WIDTH-1]}},
        prod_reg};
        out_valid <= 1'b1;
        acc_out <= acc_reg;
        valid_reg <= 1'b0;
    end else begin
        out_valid <= 1'b0;
    end
end
end
endmodule

```

Implementation notes and integration:

- Use `LATENCY` to tune pipeline depth; increase registers to meet timing on deep arrays.
- Use sign-extension as shown to preserve two's-complement semantics; convert to unsigned if operands are nonnegative.
- For mixed-precision (FP16/BF16), replace fixed-point multiply-add with an FMA IP and keep the same handshake.

Design implications and performance trade-offs:

- Wider accumulators (Equation 2.31) increase area and power but avoid costly renormalization or spill to higher precision.
- Deeper pipelining raises achievable clock frequency but increases systolic schedule latency and requires larger buffering between MACs.
- A single-cycle-throughput MAC enables one partial product per cycle per lane, so an $N \times M$ array yields $N \cdot M$ multiplies per cycle, directly scaling GFLOPS with clock.
- Choice between saturation, rounding, and higher-precision accumulation trades arithmetic correctness for area and power, critical for ML inference vs training.

2.6.3 Data distribution network

The MAC accumulation behavior and the systolic timing constraints described earlier impose strict requirements on how tiles and operand streams are delivered to the tensor array; the distribution network must therefore guarantee ordered, low-latency delivery with backpressure support and per-output queuing to match the SM-level scheduling.

Problem and analysis:

- Goal: deliver packed operand packets from shared on-chip buffers or L2 memory to multiple tensor cores or PEs with minimal arbitration stalls, preserving alignment for blocked GEMM tiles and supporting valid-ready backpressure used across SMs.
- Constraints: simultaneously arriving tile streams can target the same systolic column; per-output bandwidth must meet the array's consumption rate; contention must be resolved without deadlock.
- Key parameters: `IN_PORTS`, `OUT_PORTS`, `DATA_WIDTH`, `DEST_WIDTH`, `FIFO_DEPTH`. Sustaining a systolic array running at frequency f_{clk} with P active lanes of width W requires input bandwidth

$$[H]B_{\min} = P \cdot W \cdot f_{\text{clk}} \quad (2.32)$$

and the DDN must provide matching aggregate throughput after accounting for arbitration inefficiency η ($0 < \eta \leq 1$): available $B \geq \eta \cdot B_{\min}$.

Implementation approach:

1. Packet format packs a DEST field plus payload: {dest, payload}.
2. Per-output FIFO (circular buffer) isolates contention and decouples producer/consumer rates; FIFO depth trades latency tolerance vs area.
3. Per-cycle arbitration: for each output select at most one input (fixed priority or round-robin) to avoid multi-write hazards into a single FIFO. This keeps control logic simple and timing-friendly.
4. Valid-ready handshake on each input and output port provides backpressure propagation to SMs and upstream DMA engines.

Verilog implementation (synthesizable): a parameterized router with input flattened buses and per-output FIFOs. Grants one producer per output per cycle (fixed-priority). Inline comments are brief.

Listing 2.22: Parameterizable data-distribution router with per-output FIFOs and arbitration

```

module ddn_router #(
  parameter IN_PORTS = 4,
  parameter OUT_PORTS = 2,
  parameter DATA_WIDTH = 32,
  parameter DEST_WIDTH = 1,
  parameter FIFO_ADDR = 3 // depth = 2^FIFO_ADDR
) (
  input wire                                clk,

```

```

input wire                                rst,
input wire [(IN_PORTS*(DEST_WIDTH+DATA_WIDTH))-1:0] in_bus, // packed
        packets
input wire [IN_PORTS-1:0]                in_valid,
output reg  [IN_PORTS-1:0]                in_ready,
output reg  [OUT_PORTS-1:0]               out_valid,
input wire  [OUT_PORTS-1:0]               out_ready,
output reg  [(OUT_PORTS*DATA_WIDTH)-1:0] out_bus
);

localparam DEPTH = (1<0) begin
    out_bus[(j*DATA_WIDTH)+:DATA_WIDTH] <= fifo_mem[j][rd_ptr[j]];
    if (out_ready[j]) begin
        rd_ptr[j] <= rd_ptr[j] + 1;
        count[j] <= count[j] - 1;
        out_valid[j] <= 1;
    end else begin
        out_valid[j] <= 1;
    end
    end else begin
        out_valid[j] <= 0;
    end
end
end
end
endmodule

```

Design implications and trade-offs:

- Single-grant-per-output simplifies timing but limits burst acceptance when many producers target one column; moving to multi-write multi-bank FIFOs or a true crossbar raises area and timing cost.
- FIFO depth trades buffering against area and latency; deeper FIFOs tolerate bursty DMA transfers to saturate tensor cores but increase power and timing load on read/write paths.
- Arbitration policy (fixed-priority vs round-robin) affects tail latency fairness; implement round-robin to reduce starvation at modest control complexity.

2.6.4 Tensor core testbench

The testbench builds directly on the data distribution network's input ordering and the matrix multiply-accumulate (MMAC) block's pipeline latency, exercising tile feeding, handshake backpressure, and accumulation correctness across precisions. It must validate functional correctness, timing under realistic stalls, and throughput accounting for pipeline depth.

Problem statement and analysis:

- Verify GEMM correctness for tiled inputs with accumulation: for a tile size $M \times N$ with inner dimension K , the number of floating-point operations for one tile multiply-

accumulate is

$$[H]\text{FLOP}_{\text{tile}} = 2 \cdot M \cdot N \cdot K, \quad (2.33)$$

and sustained GFLOPS equals $\text{FLOP}_{\text{tile}}$ divided by elapsed seconds. The testbench must therefore (a) drive tile streams in the order the data distribution network produces, (b) model backpressure by toggling `ready`, and (c) compute a golden reference for each output to assert equality within rounding tolerances for FP16/BF16 simulations.

- To stress pipeline control, inject random stalls and misaligned start times; verify the MMAC accumulation across multiple cycles and ensure no data hazards when new tiles arrive before previous results are read.

Implementation summary:

- Use a self-checking harness that feeds configurable tiles, models handshake signals, and computes a reference result using the same arithmetic path used in the DUT precision.
- Measure achieved throughput by counting completed tile outputs and elapsed cycles; compute GFLOPS from Equation (1).

The following synthesizable Verilog provides a compact, production-ready DUT for a 2×2 tile tensor core and a self-checking testbench that:

1. Generates a clock and reset,
2. Streams tiles with random stalls,
3. Computes golden reference,
4. Compares DUT outputs and increments an error counter.

Listing 2.23: Synthesizable 2x2 tensor-core DUT with self-checking testbench

```
`timescale 1ns/1ps
module tensor_core_2x2 (
    input wire      clk,
    input wire      rst_n,
    input wire      in_valid,
    output wire     in_ready,
    input wire signed [15:0] a00,a01,a10,a11, // FP16 encoded as 16-bit words
    input wire signed [15:0] b00,b01,b10,b11,
    output reg      out_valid,
    input wire      out_ready,
    output reg signed [31:0] c00,c01,c10,c11 // 32-bit accumulator
);
    // Simple pipeline stage: latch inputs, compute integer multiply-
    accumulate
    reg      stage_valid;
    reg signed [15:0] ra00,ra01,ra10,ra11;
    reg signed [15:0] rb00,rb01,rb10,rb11;
    assign in_ready = !stage_valid;
```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        stage_valid <= 1'b0;
        out_valid <= 1'b0;
        c00 <= 0; c01 <= 0; c10 <= 0; c11 <= 0;
    end else begin
        // input latch when ready
        if (in_valid && in_ready) begin
            stage_valid <= 1'b1;
            ra00<=a00; ra01<=a01; ra10<=a10; ra11<=a11;
            rb00<=b00; rb01<=b01; rb10<=b10; rb11<=b11;
        end
        // computation completes next cycle
        if (stage_valid) begin
            // integer multiply accumulate as proxy for FP fused op
            c00 <= c00 + ra00*rb00 + ra01*rb10;
            c01 <= c01 + ra00*rb01 + ra01*rb11;
            c10 <= c10 + ra10*rb00 + ra11*rb10;
            c11 <= c11 + ra10*rb01 + ra11*rb11;
            out_valid <= 1'b1;
            stage_valid <= 1'b0;
        end else if (out_valid && out_ready) begin
            out_valid <= 1'b0;
        end
    end
end
endmodule

module tb_tensor_core;
    reg clk = 0;
    reg rst_n = 0;
    always #5 clk = ~clk; // 100MHz clock for simulation

    // DUT signals
    reg in_valid;
    wire in_ready;
    reg signed [15:0] a00,a01,a10,a11,b00,b01,b10,b11;
    wire out_valid;
    reg out_ready;
    wire signed [31:0] c00,c01,c10,c11;

    tensor_core_2x2 DUT(
        .clk(clk), .rst_n(rst_n),
        .in_valid(in_valid), .in_ready(in_ready),
        .a00(a00), .a01(a01), .a10(a10), .a11(a11),
        .b00(b00), .b01(b01), .b10(b10), .b11(b11),

```



```

        .out_valid(out_valid), .out_ready(out_ready),
        .c00(c00), .c01(c01), .c10(c10), .c11(c11)
    );

    integer cycle_count = 0;
    integer errors = 0;
    integer completed = 0;
    // golden reference accumulators
    integer g00=0,g01=0,g10=0,g11=0;

    // deterministic stimulus FSM
    initial begin
        rst_n = 0; in_valid = 0; out_ready = 0;
        repeat (5) @(posedge clk);
        rst_n = 1;
        // feed 16 tiles with random stall injection
        repeat (16) begin
            // generate small integer operands to avoid overflow when
            // multiplied
            a00 = $urandom_range(-8,8); a01 = $urandom_range(-8,8);
            a10 = $urandom_range(-8,8); a11 = $urandom_range(-8,8);
            b00 = $urandom_range(-8,8); b01 = $urandom_range(-8,8);
            b10 = $urandom_range(-8,8); b11 = $urandom_range(-8,8);
            // push tile when DUT ready
            @(posedge clk);
            while (!in_ready) @(posedge clk);
            in_valid = 1;
            @(posedge clk);
            in_valid = 0;
            // update golden reference immediately for this simple pipeline
            g00 = g00 + a00*b00 + a01*b10;
            g01 = g01 + a00*b01 + a01*b11;
            g10 = g10 + a10*b00 + a11*b10;
            g11 = g11 + a10*b01 + a11*b11;
            // randomly allow backpressure on the read side
            out_ready = $urandom_range(0,1);
            // allow some cycles between tiles
            repeat ($urandom_range(0,2)) @(posedge clk);
        end
        // wait for remaining outputs
        repeat (10) @(posedge clk);
        $display("Completed: %0d, Errors: %0d, Cycles: %0d", completed, errors
            , cycle_count);
        $finish;
    end

    // monitor outputs and compare to golden

```

```

always @(posedge clk) begin
    cycle_count <= cycle_count + 1;
    if (out_valid && out_ready) begin
        completed = completed + 1;
        if (c00 !== g00) errors = errors + 1;
        if (c01 !== g01) errors = errors + 1;
        if (c10 !== g10) errors = errors + 1;
        if (c11 !== g11) errors = errors + 1;
    end
end
endmodule

```

Design implications and trade-offs:

- A self-checking harness must exercise backpressure and misalignment to expose pipeline hazards; increased test complexity improves defect coverage but lengthens regression time.
- Measuring throughput via completed tiles per cycle and Equation (1) ties functional correctness to performance metrics; verification should include precision-aware reference models to capture rounding-induced divergence.
- For larger systolic arrays, scale the testbench to cover tiling permutations and edge tiles; this increases stimulus bandwidth and demands more sophisticated golden models, trading simulation cost for higher confidence.

Chapter 3

Ray Tracing Hardware

3.1 Ray Tracing Fundamentals

3.1.1 Ray representation

Building on earlier treatment of shader-generated rays and SIMT execution constraints in shader cores, this subsection formalizes the per-ray state, storage layout, and compute precomputation that RT cores expect for high-throughput traversal and intersection.

A ray is the parametric line used by traversal and intersection units; operationally we maintain its origin, direction, valid parameter interval, and a small control payload. Mathematically:

$$[H]\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \in [t_{\min}, t_{\max}], \quad (3.1)$$

where $\mathbf{o}$ is the origin, $\mathbf{d}$ the direction, and $t_{\min}, t_{\max}$ bound the query interval. Intersection units evaluate $\mathbf{r}(t)$ against primitives (triangles, AABBs) and update $t_{\max}$ when a nearer hit is found.

Practical GPU-friendly representation and precomputation:

- Store origin and direction as three-component floats per ray. Normalizing direction removes the need for magnitude compensation but costs an expensive $\sqrt{}$ on generation; many RT cores accept non-normalized directions and rely on reciprocal-direction optimization.
- Precompute reciprocal direction and per-component sign bits to convert divisions into multiplies and branchless slab tests. In text, these fields are `dir_rcp` and `dir_sign`.
- Keep the ray interval as `tmin` and `tmax` (floats), and a small 32-bit payload pointer or register index for shader-returning.

Slab intersection uses the reciprocal to compute interval overlaps without division:

$$[H]t_{x1} = (x_{\min} - o_x) \cdot d_x^{-1}, \quad t_{x2} = (x_{\max} - o_x) \cdot d_x^{-1}, \quad (3.2)$$

and analogous for y, z . Using precomputed `dir_rcp` avoids per-box divisions during traversal. Memory layout and SIMD/SIMT considerations:

- Structure-of-arrays (SoA) groups all origins and directions per component, improving coalesced loads and enabling wide-vector arithmetic in traversal and intersection units.

- Align and pad per-ray state to 64 bytes when possible to match cache line and L2 bank width, minimizing partial-line reads.

Example SoA storage and ray init (C++/CUDA style):

Listing 3.1: SoA ray layout with precomputed reciprocals

```
struct RaySoA { // arrays for N rays; N is wavefront size
    float* ox; float* oy; float* oz;           // origin arrays
    float* dx; float* dy; float* dz;         // direction arrays
    float* dir_rcpx; float* dir_rcpy; float* dir_rcpz; // reciprocals
    uint8_t* dir_sign;                       // 3-bit sign per ray
    float* tmin; float* tmax;               // interval arrays
    uint32_t* payload;                      // shader payload index
};

// Example init inner loop (host or kernel): compute reciprocals and sign
// for (i = 0; i < N; ++i) { dir_rcpx[i] = 1.0f / dx[i]; dir_sign[i] = dx[i] <
    0; ... }
```

Design implications and trade-offs:

- Precomputing reciprocals trades extra storage (three floats) for removal of divisions in hot loops, improving traversal throughput and reducing latency on RT cores.
- SoA increases memory locality for componentwise SIMD ops but can complicate per-ray scatter writes for payloads.
- FP16 compression halves bandwidth and memory but increases miss/precision risk on long rays; use selectively (primary rays, coherent packets).
- Aligning ray records to cache-line sizes reduces load fragmentation but raises memory footprint; choose alignment to match L1/L2 bank widths for best balance.

3.1.2 Intersection with primitives

Building on the parametric ray form introduced earlier, we now examine exact intersection tests used by traversal and RT cores to decide hits against axis-aligned bounding boxes (AABB) and triangles. These primitives dominate both BVH traversal branching and RT-core datapath throughput, so correctness and throughput drive architectural choices.

Problem statement and analysis: Given ray $r(t) = o + td$ with origin o and direction d , we need robust, branch-light tests for:

- AABB: fast rejection to guide BVH traversal.
- Triangle: accurate hit point, normal, and barycentric coordinates for shading and secondary rays.

Ray-AABB slab test: compute entry and exit t for each axis, then intersect intervals. Numerical safety requires handling zero-direction components.

$$[H]t_{\min} = \max_{i \in \{x,y,z\}} \min \left(\frac{b_{\min,i} - o_i}{d_i}, \frac{b_{\max,i} - o_i}{d_i} \right), \quad t_{\max} = \min_{i \in \{x,y,z\}} \max \left(\frac{b_{\min,i} - o_i}{d_i}, \frac{b_{\max,i} - o_i}{d_i} \right). \quad (3.3)$$

Intersection exists if $t_{\max} \geq \max(t_{\min}, 0)$.

Triangle intersection (Möller–Trumbore): solve for barycentrics (u, v) and t via cross and dot products. This is branch-lean and maps well to vector pipelines and RT cores:

$$[H]pvec = d \times e_2, \quad \det = e_1 \cdot pvec, \quad t = \frac{(e_2 \cdot qvec)}{\det}, \quad u = \frac{(tvec \cdot pvec)}{\det}, \quad v = \frac{(d \cdot qvec)}{\det}, \quad (3.4)$$

where $e_1 = v_1 - v_0$, $e_2 = v_2 - v_0$, $tvec = o - v_0$, $qvec = tvec \times e_1$.

Implementation example: a SIMD-friendly C++ routine suitable for shader or CPU-side validation; the code avoids branches where possible and returns barycentrics for shading.

Listing 3.2: Möller–Trumbore triangle intersection (float)

```
struct Hit { bool hit; float t,u,v; };
Hit intersectTriangle(const Vec3& o, const Vec3& d,
                    const Vec3& v0, const Vec3& v1, const Vec3& v2,
                    bool cullBackface=false) {
    Vec3 e1 = v1 - v0; Vec3 e2 = v2 - v0;
    Vec3 pvec = cross(d, e2);
    float det = dot(e1, pvec);
    if (cullBackface) { if (det < 1e-8f) return {false,0,0,0}; }
    else { if (fabs(det) < 1e-8f) return {false,0,0,0}; }
    float invDet = 1.0f / det;
    Vec3 tvec = o - v0;
    float u = dot(tvec, pvec) * invDet;
    if (u < 0.0f || u > 1.0f) return {false,0,0,0};
    Vec3 qvec = cross(tvec, e1);
    float v = dot(d, qvec) * invDet;
    if (v < 0.0f || u + v > 1.0f) return {false,0,0,0};
    float t = dot(e2, qvec) * invDet;
    if (t <= 0.0f) return {false,0,0,0};
    return {true, t, u, v};
}
```

Operational relevance and implications:

- SIMT/GPU: divergence arises when rays in a warp take different traversal paths; prefer packet/batched coherent rays to keep RT core utilization high.
- RT core mapping: triangle unit must support high-throughput cross/dot ALUs and low-latency reciprocal for *invDet*; AABB unit emphasizes compares and reciprocal reuse.
- Numerical robustness: use watertight edge tests or double precision accumulation for large scenes to avoid self-intersections.

Concrete design implications and trade-offs:

- Favor branchless algebraic formulations to minimize SIMT divergence and maximize SM/RT-core pipelines.
- Hardware should provide fast reciprocal and fused cross/dot primitives to accelerate Möller–Trumbore.

- AABB speed reduces BVH traversal cost; invest in specialized comparators and precomputed *invDir* per ray.
- Accuracy vs throughput: higher precision reduces artifacts but increases area and latency; choose per-application balance (graphics vs photoreal offline rendering).

3.1.3 BVH construction principles

Building a high-performance BVH follows directly from the ray parameterization and primitive intersection costs: the ray representation sets numerical stability and packetization choices, while per-primitive intersection cost defines the optimization objective for node splits. The BVH construction stage therefore balances build throughput (for dynamic scenes) against traversal throughput (for RT cores and SM-level path tracing).

Problem: minimize traversal cost for a given scene while respecting GPU constraints (memory, SIMT divergence, SM-local shared memory). Analysis centers on the Surface Area Heuristic (SAH), which models expected ray traversal cost using geometric probabilities and per-operation costs. Operationally, we optimize splits to minimize the expected cost of visiting a parent node P split into left (L) and right (R) children:

$$[H]C(P \rightarrow L, R) = C_{\text{trav}} + \frac{SA(L)}{SA(P)} N_L C_{\text{isec}} + \frac{SA(R)}{SA(P)} N_R C_{\text{isec}} \quad (3.5)$$

where C_{trav} is the traversal cost (node box test), C_{isec} is per-primitive intersection cost, $SA(\cdot)$ are surface areas, and N_L, N_R are primitive counts. Minimizing (1) across axes and candidate split positions gives the highest-quality binary BVH under SAH.

Implementation: exact SAH requires evaluating many candidate splits; on modern GPUs we use approximate binning to trade quality for parallelism. A scalable GPU builder pipeline:

1. Compute centroid bounds and assign primitives to B bins per axis in parallel (one thread per primitive or warp-cooperative).
2. Atomically accumulate per-bin AABBs and counts into shared or global buffers, using warp-reduction to limit contention.
3. Evaluate prefix/suffix combinations of bin aggregates to compute SAH for $B - 1$ split positions, choosing the minimum.
4. Recurse (stack or parallel wavefront) until leaf criteria (max leaf size or SAH benefit threshold).

A concise CUDA-style kernel for per-node binning and reduction (illustrative, production kernels include more warp-level tuning):

Listing 3.3: Parallel binning kernel for SAH-based split (illustrative)

```
__global__ void binPrimitives(const Primitive *prims, int n, Bin *bins, AABB
    centroidBounds) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    // compute bin index from centroid (simple linear mapping)
```

```

float t = (prims[i].centroid.x - centroidBounds.min.x) / centroidBounds.
    extent.x;
int b = min(max(int(t * NUM_BINS), 0), NUM_BINS-1);
// atomic updates to bin AABB and count (reduced with warp-level ops in
    tuned code)
atomicMin(&bins[b].aabb.min.x, prims[i].aabb.min.x);
atomicMax(&bins[b].aabb.max.x, prims[i].aabb.max.x);
atomicAdd(&bins[b].count, 1);
}

```

Practical choices and trade-offs:

- LBVH (Morton codes + radix sort) yields low-latency builds suitable for dynamic scenes but produces higher overlap and worse traversal cost than SAH.
- SAH with spatial splits minimizes overlap and traversal count, but requires more memory and build time; suitable for offline or RT-core-accelerated hybrid builds.
- On GPUs, minimize SIMT divergence during binning/traversal, use warp-cooperative reductions, and place node arrays to favor RT core cache lines.
- Branching factor: 4-ary or wide nodes reduce tree depth and traversal overhead but increase node test complexity; choose based on RT core box-test throughput versus memory footprint.

Design implications: choose LBVH for rapid refits and interactive updates, SAH-based with spatial splits for final-frame quality; architect build kernels to trade shared memory and atomic contention against SM occupancy to maximize overall frame throughput.

3.1.4 Traversal algorithms

Continuing from BVH construction choices and the numeric robustness of triangle intersections, traversal algorithms determine how rays are mapped onto that acceleration structure and ultimately dominate ray-tracing throughput when intersection cost is amortized. The next paragraphs analyze traversal trade-offs, present a compact warp-friendly implementation pattern, and finish with hardware implications.

Traversal problem: minimize node visits and divergence while maximizing throughput on SIMT hardware or RT cores. Analysis starts with the basic cost model: ray traversal time is dominated by node tests and primitive intersections. A simple linear model is

$$[H]T_{\text{ray}} \approx N_{\text{node}} \cdot t_{\text{node}} + N_{\text{tri}} \cdot t_{\text{tri}}, \quad (3.6)$$

where N_{node} and N_{tri} are expected node and triangle visits respectively, and $t_{\text{node}}, t_{\text{tri}}$ are per-test latencies. Reducing N_{node} via tighter BVH heuristics or reordering nodes in memory reduces both memory traffic and branch divergence in SIMT warps.

Key traversal strategies and their GPU relevance:

- Stack-based depth-first traversal: simple and works on RT cores; stack depth typically bounded (average < 64), making small hardware stacks practical.

- Stackless traversal (ropes): eliminates per-ray stacks, reducing register pressure; increases node pointer arithmetic and memory indirections.
- Packet traversal and ray sorting: group coherent rays into packets to amortize BVH node fetches and reduce divergence; effective when primary rays are coherent but less so for secondary rays in path tracing.
- Breadth-first or hybrid schemes: expose parallelism across rays for hardware traversal engines and tensor-like SIMD units.

Implementation (warp-friendly per-thread stack traversal). This C++/CUDA-style snippet shows a compact per-ray loop suitable for persistent-thread scheduling on an SM. It minimizes branches and keeps a small explicit stack per thread.

Listing 3.4: Warp-friendly stack-based BVH traversal (per-thread)

```

struct Ray { float3 o, d, invd; }; // ray origin, direction, inverse dir
struct Node { float3 min, max; int left, right; int triOffset, triCount; };

__device__ bool intersect_aabb(const Ray& r, const Node& n, float& tmin) {
    // branch-minimized slab test; returns hit distance in tmin
    float t0 = (n.min.x - r.o.x) * r.invd.x;
    float t1 = (n.max.x - r.o.x) * r.invd.x;
    float tmin_x = fminf(t0,t1), tmax_x = fmaxf(t0,t1);
    // repeat for y,z and combine (omitted for brevity)
    // final tmin assignment and test against tmax
    return /*true if intersects*/ true;
}

__global__ void traverse_kernel(const Node* bvh, const Triangle* tris, Ray*
    rays, Hit* hits, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= N) return;
    Ray r = rays[idx];
    int stack[64]; int sp = 0; stack[sp++] = 0; // root index
    float tmin;
    while (sp) {
        int nodeId = stack[--sp];
        const Node& node = bvh[nodeId];
        if (!intersect_aabb(r,node,tmin)) continue; // reject
        if (node.triCount) {
            // test triangles (triangle intersection code omitted)
        } else {
            // push children; push order can be near-first for early-out
            stack[sp++] = node.right; stack[sp++] = node.left;
        }
    }
    hits[idx] = /*closest hit record*/ {};
}

```


Design implications and trade-offs:

- Hardware RT cores implement parallel node tests and prioritized traversal to reduce t_{node} , making stack-based DFS with small hardware stacks attractive.
- Packetization reduces memory bandwidth but increases algorithmic complexity for incoherent rays; sort cost must be weighed.
- Stackless methods lower register usage but increase pointer-chasing, harming performance when L1/L2 latency dominates.
- Optimizing node layout for spatial locality (GPU-friendly memory layout) reduces N_{node} effective cost by improving cache hit rates, directly improving throughput on SMs and RT cores.

3.2 RT Core Architecture

3.2.1 Ray-box intersection logic

This subsection builds directly on the ray parameterization and BVH traversal strategies introduced earlier, using the ray's origin and direction to drive efficient early-rejection logic inside the RT core. The slab method is the canonical, hardware-friendly formulation for axis-aligned bounding box (AABB) tests and maps well to SIMD/SIMT pipelines and fixed-latency units inside an RT accelerator.

The problem: determine whether a ray $r(t) = o + td$ intersects an AABB defined by bounds $b_{\min}$ and $b_{\max}$, with minimal latency and predictable control flow for hardware (warp/packet coherence and RT core pipelines). Analysis shows the slab method reduces the 3D test to six scalar interval tests and two reductions (max of enters, min of exits). For each axis compute

$$[H]t_1 = (b_{\min} - o) \cdot d^{-1}, \quad t_2 = (b_{\max} - o) \cdot d^{-1}, \quad (3.7)$$

then $t_{\min_axis} = \min(t_1, t_2)$, $t_{\max_axis} = \max(t_1, t_2)$. Let $t_{\text{enter}} = \max(t_{\min_x}, t_{\min_y}, t_{\min_z})$ and $t_{\text{exit}} = \min(t_{\max_x}, t_{\max_y}, t_{\max_z})$. The ray hits the box if $t_{\text{enter}} \leq t_{\text{exit}}$ and $t_{\text{exit}} \geq 0$.

Operational concerns and microarchitectural choices:

- Precompute and cache the reciprocal direction d^{-1} per ray in register files or lane-local storage; use a reciprocal approximation (Newton–Raphson) to save cycles in RT cores, with one refinement step to meet IEEE float error bounds.
- Handle parallel rays (d component ≈ 0) by detecting large-magnitude reciprocals (saturated fixed-point/infinite): treat axis as passing-through if both t_1, t_2 become $\pm\infty$, then use comparisons that avoid NaN propagation.
- Use fixed-point arithmetic in the intersection pipeline to minimize area and determinism—Q16.16 is a common tradeoff between precision and hardware cost for world-space extents; floating-point units yield simpler correctness but larger area and latency.
- Reduce divergence by grouping rays with similar direction signs; sign determines whether t_1 or t_2 is nearer, enabling simplified selection logic.

Implementation (synthesizable Verilog, fixed-point Q16.16): a single-cycle pipelined slab test for one box per ray lane. Inputs are signed 32-bit fixed-point values; outputs indicate hit and computed near/far param (Q16.16).

Listing 3.5: Fixed-point slab-based ray-AABB test (Q16.16)

```

module ray_box_slab(
  input  wire      clk,
  input  wire      rst,
  input  wire      valid_in,
  input  wire signed [31:0] ox, oy, oz,          // Q16.16 origin
  input  wire signed [31:0] invdx, invdy, invdz, // Q16.16 reciprocal dir
  input  wire signed [31:0] bxmin, bymin, bzmin,
  input  wire signed [31:0] bxmax, bymax, bzmax,
  output reg      hit,
  output reg signed [31:0] t_near, t_far,
  output reg      ready
);
  // Multiply Q16.16 * Q16.16 -> Q32.32, then shift >>16 to get Q16.16
  wire signed [63:0] t1x = ($signed(bxmin) - $signed(ox)) * $signed(invdx);
  wire signed [63:0] t2x = ($signed(bxmax) - $signed(ox)) * $signed(invdx);
  wire signed [63:0] t1y = ($signed(bxmin) - $signed(oy)) * $signed(invdy);
  wire signed [63:0] t2y = ($signed(bymax) - $signed(oy)) * $signed(invdy);
  wire signed [63:0] t1z = ($signed(bzmin) - $signed(oz)) * $signed(invdz);
  wire signed [63:0] t2z = ($signed(bzmax) - $signed(oz)) * $signed(invdz);

  // reduce to Q16.16
  wire signed [31:0] tx1 = t1x[47:16], tx2 = t2x[47:16];
  wire signed [31:0] ty1 = t1y[47:16], ty2 = t2y[47:16];
  wire signed [31:0] tz1 = t1z[47:16], tz2 = t2z[47:16];

  wire signed [31:0] tminx = (tx1 < tx2) ? tx1 : tx2;
  wire signed [31:0] tmaxx = (tx1 > tx2) ? tx1 : tx2;
  wire signed [31:0] tminy = (ty1 < ty2) ? ty1 : ty2;
  wire signed [31:0] tmaxy = (ty1 > ty2) ? ty1 : ty2;
  wire signed [31:0] tminz = (tz1 < tz2) ? tz1 : tz2;
  wire signed [31:0] tmaxz = (tz1 > tz2) ? tz1 : tz2;

  wire signed [31:0] enter = (tminx > tminy) ? ((tminx > tminz) ? tminx :
    tminz) : ((tminy > tminz) ? tminy : tminz);
  wire signed [31:0] exitv = (tmaxx < tmaxy) ? ((tmaxx < tmaxz) ? tmaxx :
    tmaxz) : ((tmaxy < tmaxz) ? tmaxy : tmaxz);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      hit <= 1'b0; ready <= 1'b0; t_near <= 32'b0; t_far <= 32'b0;
    end else begin
      ready <= valid_in;
    end
  end

```

```

    if (valid_in) begin
        hit <= (enter <= exitv) && (exitv >= 32'b0); // exit >= 0
        t_near <= enter;
        t_far <= exitv;
    end
end
end
endmodule

```

Design implications and trade-offs:

- Fixed-point pipelines cut area and deterministic latency but require careful scaling and may need occasional double-precision for large scenes; floating-point reduces precision headaches at the cost of area and power.
- Reciprocal approximation reduces latency per ray but increases per-ray register footprint; hardware RT cores often amortize this by sharing reciprocal storage across packet lanes.
- Branchless min/max and saturation logic favor SIMD execution and reduce warp divergence, improving throughput in coherent workloads but offering limited gains for incoherent path tracing.

3.2.2 Ray-triangle intersection unit

Following the ray-box intersection logic, the ray-triangle intersection unit implements the primitive-level test used after BVH traversal identifies candidate triangles. This unit must deliver one-cycle-per-stage throughput when pipelined, apply watertight and backface culling options, and fit into RT core datapaths that feed shading payloads to SMs.

Problem statement and operational relevance: the unit computes the intersection between a parametric ray $r(t) = o + td$ and a triangle (v_0, v_1, v_2) . It must return hit distance t , barycentric coordinates (u, v) , and a hit mask, while minimizing latency and numerical error that cause missed or duplicate hits in coherent packet traversal. The Möller–Trumbore algorithm is compact and well-suited for hardware pipelines; the core algebra is:

$$\begin{aligned}
 \mathbf{e}_1 &= \mathbf{v}_1 - \mathbf{v}_0, & \mathbf{e}_2 &= \mathbf{v}_2 - \mathbf{v}_0, \\
 \mathbf{p} &= \mathbf{d} \times \mathbf{e}_2, & \det &= \mathbf{e}_1 \cdot \mathbf{p}, \\
 [H] \quad \mathbf{t} &= \mathbf{o} - \mathbf{v}_0, & u &= (\mathbf{t} \cdot \mathbf{p}) / \det, \\
 \mathbf{q} &= \mathbf{t} \times \mathbf{e}_1, & v &= (\mathbf{d} \cdot \mathbf{q}) / \det, \\
 t &= (\mathbf{e}_2 \cdot \mathbf{q}) / \det.
 \end{aligned} \tag{3.8}$$

Hardware needs to test $|\det| > \epsilon$ to avoid near-degenerate triangles and then check $u \in [0, 1]$, $v \in [0, 1]$, and $u + v \leq 1$.

Analysis and implementation strategy: to keep the design synthesizable and timing-friendly, implement arithmetic in 32-bit fixed-point Q2.30 (two integer bits, 30 fractional bits). Fixed-point avoids costly IEEE-754 IP for this unit, and preserves predictable latency for deeply pipelined RT cores. The unit supports optional backface culling by sign-checking $\det$. It exposes a valid-ready handshake for integration with BVH traversal and can operate on ray packets by instantiating one unit per lane within an RT core.

Example synthesizable Verilog: a pipelined fixed-point Möller–Trumbore module with valid-ready interface and configurable culling.

Listing 3.6: Fixed-point pipelined ray-triangle intersection (Q2.30)

```

module rt_tri_intf #(parameter W=32, F=30) (
    input wire          clk,
    input wire          rstn,
    // input ray (origin, direction)
    input wire          in_valid,
    input wire [W-1:0]  ox, oy, oz,
    input wire [W-1:0]  dx, dy, dz,
    // triangle vertices
    input wire [W-1:0]  v0x, v0y, v0z,
    input wire [W-1:0]  v1x, v1y, v1z,
    input wire [W-1:0]  v2x, v2y, v2z,
    input wire          backface_cull,
    output reg          out_valid,
    output reg [W-1:0]  tout, uout, vout,
    output reg          hit
);

// Stage 1: edge vectors
wire signed [W-1:0] e1x = v1x - v0x;
wire signed [W-1:0] e1y = v1y - v0y;
wire signed [W-1:0] e1z = v1z - v0z;
wire signed [W-1:0] e2x = v2x - v0x;
wire signed [W-1:0] e2y = v2y - v0y;
wire signed [W-1:0] e2z = v2z - v0z;

// Stage 2: p = d x e2 (fixed-point mults)
wire signed [W-1:0] px = (dy*e2z - dz*e2y) >>> F;
wire signed [W-1:0] py = (dz*e2x - dx*e2z) >>> F;
wire signed [W-1:0] pz = (dx*e2y - dy*e2x) >>> F;
// det = e1 . p
wire signed [W-1:0] det = (e1x*px + e1y*py + e1z*pz) >>> F;
// tvec = o - v0
wire signed [W-1:0] tx = ox - v0x;
wire signed [W-1:0] ty = oy - v0y;
wire signed [W-1:0] tz = oz - v0z;
// u numerator
wire signed [W-1:0] un = (tx*px + ty*py + tz*pz) >>> F;
// q = t x e1
wire signed [W-1:0] qx = (ty*e1z - tz*e1y) >>> F;
wire signed [W-1:0] qy = (tz*e1x - tx*e1z) >>> F;
wire signed [W-1:0] qz = (tx*e1y - ty*e1x) >>> F;
// v numerator and t numerator
wire signed [W-1:0] vn = (dx*qx + dy*qy + dz*qz) >>> F;
wire signed [W-1:0] tn = (e2x*qx + e2y*qy + e2z*qz) >>> F;
// simple inverse approximation: use reciprocal via Newton iteration
externally.
// Here compute hit tests without explicit division by comparing scaled
ranges.

```

```

always @(posedge clk or negedge rstn) begin
  if (!rstn) begin out_valid<=0; hit<=0; end
  else begin
    out_valid <= in_valid; // one-cycle flow model for example
    // culling and epsilon check
    if (in_valid) begin
      if (det == 0) begin hit<=0; end
      else begin
        // backface cull
        if (backface_cull && det < 0) hit<=0;
        else begin
          // compare u = un/det in [0,1] via sign and magnitude cross-
          multiplication
          if ((un < 0) || (un > det)) hit<=0;
          else if ((vn < 0) || (un + vn > det)) hit<=0;
          else if (tn <= 0) hit<=0;
          else begin
            hit <= 1;
            // approximate outputs (fixed-point division omitted for brevity
            )
            tout <= tn; uout <= un; vout <= vn;
          end
        end
      end
    end else hit<=0;
  end
end
endmodule

```

Implications and trade-offs:

- Fixed-point arithmetic reduces area and gives deterministic latency, but requires careful quantization to avoid miss rates for thin triangles.
- A reciprocal/divide unit improves numerical correctness at the cost of cycles or area; pipelining mitigates latency.
- Instantiating one unit per ray-lane in an RT core scales linearly in area but maximizes throughput for coherent packets; sharing a unit across lanes saves area but increases traversal stalls.

3.2.3 BVH traversal engine

The traversal engine consumes the outputs of the ray-box and ray-triangle units to orchestrate node evaluation and scheduling of primitive intersection tests; it therefore sits immediately after the ray-box intersection stage and feeds the triangle unit when leaf nodes qualify. This subsection describes the traversal problem, analyzes ordering and memory considerations, and presents an RTL-friendly FSM for a BRAM-backed stacked traversal that interoperates with the previously defined intersection units.

BVH traversal problem and goals: given a ray with entry and exit parameters, find the nearest primitive hit while minimizing node fetches and memory bandwidth. Key correctness criterion is evaluating children in near-to-far order so that early hit tests can prune farther subtrees. For an axis-aligned bounding box (AABB) the entry and exit parameters along each axis produce:

$$[H]t_{\text{enter}} = \max\{t_{x,\min}, t_{y,\min}, t_{z,\min}\}, \quad t_{\text{exit}} = \min\{t_{x,\max}, t_{y,\max}, t_{z,\max}\}. \quad (3.9)$$

A node is missed if $t_{\text{enter}} > t_{\text{exit}}$ or $t_{\text{exit}} < 0$.

3.2.4 Hit and miss determination

The BVH traversal stage hands a stream of candidate triangles to the intersection stage, and the triangle unit returns precise intersection parameters for each candidate; the hit/miss logic must therefore decide correctness and perform nearest-hit reduction or any-hit early-out based on those parameters. This subsection formalizes the decision criteria, outlines a small hardware datapath suitable for RT cores, and explains microarchitectural trade-offs relevant to SM/RT core integration.

Problem: given a candidate intersection with signed distance t , barycentric coordinates (u, v) , and hit predicate from the triangle unit, determine whether the ray records a hit or misses. For correctness, the unit must enforce:

- ray bounds: $t_{\min} \leq t \leq t_{\max}$, where $t_{\min}$ often encodes self-intersection bias;
- barycentric validity: $u \geq 0$, $v \geq 0$, and $u + v \leq 1$ (usually prechecked by the triangle unit);
- instance, geometry, and visibility masks: bitwise tests between ray and primitive masks.

Analysis: define a hit condition

$$[H]\text{hit} \iff (t_{\min} \leq t \leq t_{\max}) \wedge \text{mask_pass} \wedge \text{tri_valid}, \quad (3.10)$$

where mask_pass is a boolean from $(\text{ray_mask} \& \text{prim_mask}) \neq 0$. For closest-hit rays the unit must maintain the current nearest t_{best} and update only when $t < t_{\text{best}}$. For any-hit (shadow) rays the first valid hit can conclude traversal and assert early-out to save traversal traffic and memory bandwidth.

Implementation: a small, single-cycle combinational comparator plus a register holding t_{best} supports nearest-hit reduction; an ANY/HIT mode input gates an early-out valid. The Verilog below is synthesizable and implements a valid-ready handshake, nearest-hit update, and any-hit early-out for a single-lane RT core datapath.

Listing 3.7: Hit/miss determination unit (single-lane) with valid-ready interface

```
module hit_miss_unit #(
    parameter FP_WIDTH = 32
) (
    input wire          clk,
    input wire          rst_n,
    // input candidate from triangle unit
    input wire          in_valid,
```

```

    output wire          in_ready,
    input  wire [FP_WIDTH-1:0] in_t,
    input  wire [31:0]      in_prim_mask,
    input  wire          in_tri_valid, // barycentrics already checked
    // ray state
    input  wire [FP_WIDTH-1:0] ray_tmin,
    input  wire [FP_WIDTH-1:0] ray_tmax,
    input  wire [31:0]      ray_mask,
    input  wire          mode_any_hit, // 1 = any-hit (shadow), 0 =
        closest
    // outputs
    output reg          out_hit_valid,
    output reg [FP_WIDTH-1:0] out_hit_t,
    output reg [31:0]      out_hit_prim_mask,
    input  wire          out_ready
);

assign in_ready = 1'b1; // simple pipeline stage: always ready

wire mask_pass = ~(ray_mask & in_prim_mask); // bitwise AND non-zero
wire t_in_range;
assign t_in_range = (in_t >= ray_tmin) && (in_t <= ray_tmax);

reg [FP_WIDTH-1:0] t_best;
reg                found_any;

// combinational hit detection
wire candidate_hit = in_valid && in_tri_valid && mask_pass && t_in_range;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        out_hit_valid    <= 1'b0;
        out_hit_t        <= {FP_WIDTH{1'b0}};
        out_hit_prim_mask <= 32'b0;
        t_best           <= {FP_WIDTH{1'b1}}; // init to max value
        found_any        <= 1'b0;
    end else begin
        out_hit_valid <= 1'b0;
        if (candidate_hit) begin
            if (mode_any_hit) begin
                // any-hit: output immediately and mark found
                out_hit_valid    <= 1'b1;
                out_hit_t        <= in_t;
                out_hit_prim_mask <= in_prim_mask;
                found_any        <= 1'b1;
            end else begin
                // closest-hit: compare and update

```

```

    if (in_t < t_best) begin
        t_best          <= in_t;
        out_hit_valid    <= out_ready ? 1'b1 : 1'b0; // flow control
        out_hit_t        <= in_t;
        out_hit_prim_mask <= in_prim_mask;
    end
end
end
// optional: clear found_any when traversal restarts externally
end
end
endmodule

```

Implications and trade-offs:

- Early-out (ANY/HIT) reduces BVH memory traffic and increases throughput for shadow rays but complicates coherent traversal batching.
- Maintaining t_{best} in-register is low-latency but scales poorly for wide-wavefront reductions; cross-lane reductions require additional tree-reduction logic in the SM/RT core.
- Mask checks are cheap bitwise ops but widen the datapath when supporting many visibility layers; choose mask width to balance scene layering versus area.
- Design must balance single-cycle comparators (low latency) against pipelined multi-stage units to meet RT core frequency targets.

3.3 Acceleration Structures

3.3.1 BVH node hierarchy

The previous discussion of ray-box intersection primitives and the traversal engine motivates a careful look at how nodes are represented and laid out in memory, because layout directly affects traversal throughput on SIMT hardware and RT cores. This subsection analyzes the node-level encoding choices engineers use to balance footprint, traversal latency, and memory-bandwidth efficiency.

A BVH node must encode AABB extents plus connectivity or primitive lists. Two common node classes exist: interior nodes store two child references and a split axis; leaf nodes store a primitive base index and count. For GPU hardware, designers favor fixed-size node words aligned to cachelines to allow coalesced loads across warps. A typical design packs two quantized 3D boxes and a 32-bit union field into a 128-bit node, enabling one 128-bit read to supply all data required by an RT core for ray-box tests and branching.

Quantization reduces storage for bounding coordinates by mapping floating ranges to integers. If each coordinate is stored in b bits, the quantization scale for a node with world-space bounds $[L, R]$ is

$$[H]S = \frac{R - L}{2^b - 1}, \quad (3.11)$$

and reconstruction uses $x = L + q \cdot S$. With $b = 16$, per-coordinate error is bounded by $S/2$, which is negligible for large scenes but must be evaluated for thin geometry.

Memory footprint calculation informs capacity planning: with 128 bits per node, N nodes require $16N$ bytes. For example, a 50M-node BVH consumes 800 MB, so reducing node size or using compressed leaf lists affects VRAM and L2 pressure directly.

Traversal performance depends on:

- alignment to hardware cacheline size (e.g., 64 bytes) to avoid cross-line accesses;
- placing likely-hit child first (pointer ordering) to increase early termination and reduce divergence among threads in a warp;
- using relative offsets instead of full pointers to save bits and improve locality.

A practical, synthesizable packing module for node construction follows. It encodes quantized min/max as 16-bit values per axis and a 32-bit union field; interior nodes use child offsets and axis, leaves use primitive index and count.

Listing 3.8: BVH node packer: 128-bit node word; union field encodes leaf or interior

```
module bvh_node_packer(
    input wire      is_leaf,                // 1=leaf, 0=interior
    input wire [1:0] axis,                  // split axis for interior
    input wire [14:0] child0_off, child1_off, // relative offsets for interior
    input wire [19:0] prim_index,           // base primitive index for leaf
    input wire [11:0] prim_count,           // primitive count for leaf
    input wire [15:0] min_x, min_y, min_z,   // quantized bbox min
    input wire [15:0] max_x, max_y, max_z,   // quantized bbox max
    output wire [127:0] node_word            // packed node output
);
    wire [31:0] union_field;
    // interior: [31]=0 reserved, [30:15]=child0, [14:0]=child1 with axis in top
    // bits
    // leaf:    [31:12]=prim_index, [11:0]=prim_count
    assign union_field = is_leaf ?
        {prim_index, prim_count} :
        {1'b0, axis, child0_off, child1_off}; // packs axis and two 15-bit offsets
    assign node_word = {max_z, max_y, max_x, min_z, min_y, min_x, union_field};
endmodule
```

Design implications and trade-offs:

- Smaller nodes reduce bandwidth but increase CPU/GPU decode cost and quantization error; larger nodes allow per-node prefetching and fewer pointer jumps.
- Using relative offsets saves pointer bits and improves node locality but limits maximum BVH size per memory region.
- Ordering children by likely hit probability reduces SIMT divergence and average traversal steps, improving RT core utilization at a small build-time cost.

3.3.2 Memory layout optimization

The node-hierarchy discussion showed how BVH topology determines traversal paths; memory layout optimization converts that topology into real-world latency and bandwidth behavior on SMs and RT cores. Optimizing layout reduces SIMT divergence from pointer-chasing and improves L1/L2 hit rates for coherent ray batches, directly lowering traversal latency.

Problem statement and analysis: ray traversal cost is often dominated by scattered loads of inner-node AABBs and leaf primitive lists. For a given BVH, expected memory accesses depend on visit probabilities and node size; modeling this exposes optimization targets before code changes. Operationally, the goal is to minimize expected cache lines touched per ray:

$$[H]E[A] \approx \sum_{n \in \text{nodes}} p_n \cdot \frac{s_n}{L}, \quad (3.12)$$

where p_n is the probability a ray visits node n , s_n is the byte size of node n , and L is cache line size. Reducing s_n , increasing locality (reducing p_n for distant nodes), or aligning multiple frequently-visited nodes to the same cache line lowers $E[A]$.

Practical layout strategies and their trade-offs:

- Contiguous subtree packing (preorder DFS flattening): place each subtree in contiguous memory to favor coherent rays traversing the same region; reduces pointer chasing for primary rays.
- Breadth-first (level-order) layout: improves simultaneous access to nodes at the same depth, useful when coherent wavefronts examine similar-tree depths across warps.
- Morton (Z-order) sorting of primitives before build: improves spatial locality of leaves, benefiting incoherent secondary rays.
- Node packing and SoA (structure-of-arrays): separate child indices, AABB min/max, and metadata into aligned arrays to allow the RT core to fetch only required fields and enable vectorized loads.
- Quantized bounds and pointer compression: reduce s_n , trading precision or build complexity for fewer cache lines touched.

Implementation: below is a production-ready C++ routine to flatten a pointer-based BVH into a contiguous array in preorder (DFS), producing an index map used by traversal units. This improves locality for depth-first traversals commonly used in hardware BVH engines.

Listing 3.9: Preorder BVH flattening for contiguous subtree layout

```
#include
struct Node { float bmin[3], bmax[3]; int left, right; int firstPrim,
    primCount; };
void flattenDFS(const Node* root, std::vector& flat) {
    flat.reserve(1024); // pre-reserve to reduce reallocations
    std::function dfs = [&](const Node* n)->int{
        int idx = flat.size();
        flat.push_back(*n); // placeholder copy; indices fixed after recursion
        if (n->left >= 0) { // internal node
```

```

    int leftIdx = dfs(&root[n->left]); // flatten left subtree
    int rightIdx = dfs(&root[n->right]); // flatten right subtree
    flat[idx].left = leftIdx; flat[idx].right = rightIdx; // update children
}
return idx;
};
dfs(root);
}

```

Implications and trade-offs:

- Preorder flattening minimizes scattered loads for coherent rays, but may increase build-time reordering cost and reduce performance for certain incoherent workloads.
- SoA and node compression save bandwidth, but require traversal logic changes in RT cores and careful alignment to L and HBM transaction widths (e.g., 128 bytes).
- Level-order layouts help warp-synchronous traversal patterns; subtree packing helps per-ray stackless engines.

Designers must choose layouts that balance build complexity, memory footprint, and the expected ray coherence patterns of target workloads.

3.3.3 Update and refit mechanisms

Following memory-layout optimization and the BVH node hierarchy discussion, we now examine how hardware and algorithms maintain correctness when scene geometry moves or deforms. Update and refit mechanisms must exploit GPU parallelism while minimizing memory traffic and synchronization inside the RT core and SM clusters.

Problem: dynamic scenes require BVH updates without full rebuilds to keep RT throughput high. Two common operations exist:

1. Refit — recompute ancestor axis-aligned bounding boxes (AABBs) after leaf geometry changes.
2. Partial rebuild — locally restructure poorly fitting subtrees when refit quality degrades.

Analysis: a naive serial refit visits every ancestor of every changed leaf; for m changed leaves in a BVH of N leaves and height $h \approx O(\log N)$ the sequential work is

$$[H]W_{\text{seq}} = O(mh) \approx O(m \log N). \quad (3.13)$$

On GPUs, use level-synchronous parallelism: recompute all leaf AABBs in one kernel, then perform upward reductions by tree level where many parent updates are independent and map well to SIMT lanes. Parallel time T with P workers approximates

$$[H]T \approx \sum_{\ell=0}^{h-1} \left\lceil \frac{n_\ell}{P} \right\rceil \tau_\ell + T_{\text{sync}}, \quad (3.14)$$

where n_ℓ nodes at level ℓ , τ_ℓ per-node combine latency (memory read + union), and T_{sync} is level barrier overhead. Real implementations reduce τ by packing node data for coalesced L1/L2 reads and using shared memory for per-block reductions.

Implementation: level-synchronous refit fits RT core design points:

- Kernel A computes leaf AABBs from triangle positions using SM vector ALUs; writes to compact node buffer aligned for texture/cache lines.
- Kernel B runs per-level parent merges from leaves upward; branchless union code reduces divergence.
- If a parent's surface area heuristic (SAH) increases beyond threshold, flag subtree for partial rebuild offloaded to an asynchronous rebuild worker or a lower-priority SM.

Example GPU compute kernels (level-synchronous leaf recompute and internal merge):

Listing 3.10: Level-synchronous BVH refit on GPU

```
extern "C" __global__ void refitLeaves(AABB* leafAabbs, const Tri* tris, int
    leafCount){
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i>=leafCount) return;
    // recompute leaf AABB from triangle vertices
    Tri t = tris[i]; // triangle assigned to leaf i
    float3 v0=t.v0, v1=t.v1, v2=t.v2;
    AABB a; a.min = fminf(fminf(v0,v1), v2); a.max = fmaxf(fmaxf(v0,v1), v2);
    leafAabbs[i] = a; // write updated leaf bbox
}

__global__ void refitInternal(AABB* nodes, int levelStart, int nodeCount){
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i>=nodeCount) return;
    int idx = levelStart + i;
    AABB left = nodes[2*i + levelStart]; // layout-dependent index calc
    AABB right = nodes[2*i+1 + levelStart];
    nodes[idx] = unionAABB(left,right); // parent bbox = union of children
}
```

Implications and trade-offs:

- Refit is cheap when motion is small and geometry stays within leaf AABBs; often $O(m \log N)$ work dominated by memory bandwidth.
- Partial rebuilds restore quality but cost more compute and memory writes; trigger heuristics must balance visual correctness, RT core utilization, and latency.
- Hardware-friendly layout (breadth-first arrays, node packing) enables coalesced reads during upward passes and lower τ in equation (2).
- Design choices: prioritize frequent light-weight refits on the fast path inside RT core; offload expensive rebuilds to background threads or lower-priority SMs to maintain steady ray throughput.

3.3.4 Build-time vs runtime tradeoffs

The prior discussion of refit and partial-update strategies established that cheap refits preserve BVH topology while memory-layout choices determine traversal locality and cache behavior.

Those constraints drive whether a system prefers expensive high-quality builds or frequent lightweight updates on the GPU or host.

Problem statement and analysis. Real-time ray tracing systems must minimize frame latency while supporting dynamic scenes. Let T_{build} denote the time to (re)build an acceleration structure and T_{refit} the time to refit nodes; let R be rays traced per frame and T_{trav} the average per-ray traversal cost. The total time per frame when rebuilding every frame is

$$[H]T_{\text{frame}} = T_{\text{build}} + R \cdot T_{\text{trav}}. \quad (3.15)$$

If the application instead refits or incrementally updates, replace T_{build} by T_{refit} (usually $T_{\text{refit}} \ll T_{\text{build}}$) but accept a higher T_{trav} because node quality is lower. The break-even ray count R^* solving $T_{\text{build}} + R^*T_{\text{trav}}^{(\text{high})} = T_{\text{refit}} + R^*T_{\text{trav}}^{(\text{low})}$ quantifies when full rebuilds amortize.

Key axes and hardware relevance.

- Build-time optimizations: spatial splits, SAH (surface area heuristic) sorting, and high branching-factor nodes reduce T_{trav} but increase T_{build} and memory consumption. These algorithms are compute-intensive and map to SMs and compute pipelines; GPU builders leverage parallel radix sorts and fast prefix sums on SMs, or offload heavy index work to specialized tensor-like units for batch reductions.
- Runtime cost: traversal latency depends on node packing, cache line utilization, and RT core traversal primitives. Memory layout optimizations (interleaved node arrays, packed AABB floats, compressed child indices) reduce L1/L2 misses on SMs and RT cores, lowering T_{trav} .
- Dynamic strategies: two-level BVHs (per-instance top-level refitable structures), lazy rebuilds, and hybrid refit+local-rebuild schemes trade minimal T_{refit} against localized T_{build} .

Implementation sketch (control policy for rebuild vs refit). The following synthesizable Verilog FSM chooses rebuild/refit based on scene_change rate and on-going ray load; it outputs start signals for GPU kernel dispatch or host rebuild DMA. Brief inline comments explain signals.

Listing 3.11: BVH build/refit policy FSM (synthesizable).

```

module bvh_policy (
    input  clk,                // system clock
    input  rst_n,             // active-low reset
    input  scene_change,      // flag when geometry change detected
    input  high_ray_load,     // heuristic: many rays in flight
    output reg start_rebuild, // trigger full GPU/host rebuild
    output reg start_refit,  // trigger refit/update kernel
    output reg busy          // busy while scheduling
);
    // simple counter to measure frequency of scene changes
    reg [7:0] change_count;
    parameter THRESH=8'd4;    // threshold for rebuild
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin

```

```

change_count<=8'd0; busy<=1'b0;
start_rebuild<=1'b0; start_refit<=1'b0;
end else begin
    // clear pulses
    start_rebuild<=1'b0; start_refit<=1'b0;
    if (scene_change) change_count<=change_count+1'b1;
    // decision logic: rebuild if many changes and ray load high
    if (change_count>=THRESH && high_ray_load && !busy) begin
        start_rebuild<=1'b1; busy<=1'b1; change_count<=8'd0;
    end else if (scene_change && !busy) begin
        start_refit<=1'b1; busy<=1'b1;
    end else if (busy) begin
        // consumer clears busy when the kernel completes (not shown)
        busy<=busy;
    end
end
end
endmodule

```

Implications and performance trade-offs.

- Rebuilds using SAH produce fewer traversal steps, lowering T_{trav} but expanding T_{build} and temporary memory; optimal for static or slowly changing scenes where R is large.
- Refit and lazy-local rebuilds minimize build stalls, suit highly dynamic content, but increase traversal misses and per-ray arithmetic, shifting the bottleneck from compute to memory subsystem.
- Memory layout choices (node packing, cache-line alignment, node prefetching) significantly alter the slope $dT_{\text{trav}}/d(\text{node_quality})$, so co-design of builder and traversal memory format is essential.
- Hardware support: including GPU-accelerated builders on SMs and RT-core friendly node encodings permits more frequent high-quality rebuilds within thermal and latency budgets.

3.4 Ray Coherence and Sorting

3.4.1 Coherent Ray Batching

Building on the BVH traversal engine and ray–triangle intersection unit discussed earlier, coherent ray batching formalizes grouping strategies that trade scheduling latency for traversal and memory-efficiency gains. The core problem is minimizing redundant BVH node visits and texture/vertex fetches when launching millions of primary and secondary rays under the SIMT execution model.

Problem. Independent rays from nearby pixels or similar directions produce overlapping BVH node accesses but cause divergent traversal paths when dispatched naïvely by warps. This increases L2/RT-core memory pressure and lowers SM utilization.

Analysis. Define a direction coherence metric for a batch of N normalized ray directions $\mathbf{d}_i$:

$$[H]C = \frac{2}{N(N-1)} \sum_{i < j} \mathbf{d}_i \cdot \mathbf{d}_j \quad (3.16)$$

C ranges from 0 (completely incoherent directions) to 1 (perfectly aligned directions). High C indicates that batching those rays together will reduce redundant BVH and cache activity.

Listing 3.12: Warp-level direction coherence compaction.

```
__global__ void direction_coherence_kernel(Ray *rays, int N) {
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    if (gid >= N) return;

    // Quantize direction into discrete bins
    uint8_t b = quantize_dir(rays[gid].dir);

    // Warp-wide mask (example: all threads compare their bins)
    unsigned mask = __ballot_sync(0xffffffff, b == b);

    // Compute local offset within warp (prefix sum for compaction)
    unsigned lane = threadIdx.x & 31;
    unsigned pre = __popc(mask & ((1u << lane) - 1));

    // Store results or perform per-bin accumulation...
}
```

3.4.2 Ray binning and bucketing

Building on coherent ray batching (grouping rays to exploit shared BVH traversal and reduce divergence), binning and bucketing are practical low-latency reordering techniques that trade inexpensive local buffering and lightweight quantization for large reductions in RT core and cache pressure.

The problem: millions of rays per frame arrive with little coherence; naive traversal causes redundant BVH node fetches, low SIMD/SIMT utilization on SMs, and poor texture/cache locality for triangle data. Binning partitions rays so that rays processed together are likely to traverse the same high-level BVH nodes or target nearby triangles, increasing cache hit rate and improving RT core throughput.

Analysis (operational relevance first). Useful bin types:

- Screen-space tile bins (fixed-size tiles): group rays by pixel tile to exploit spatial locality in primary rays and coherent secondary-ray distributions for shaders.
- Direction-space buckets (quantized solid angles): effective for glossy/importance-sampled secondary rays where direction clusters matter.
- Morton or world-space bins (spatial hashing): best when geometry-induced coherence dominates (e.g., localized light transport).

A compact cost model helps choose number of buckets B . Let N be rays, s cost coefficient for sorting work per ray, a overhead per bucket (setup, BVH root visits), and b per-ray traversal cost that does not change with binning. If we approximate uniform bucket occupancy $n = N/B$ and assume sorting cost scales like $sN \log_2(N/B)$, total cost is

$$[H]C(B) = sN \log_2\left(\frac{N}{B}\right) + aB + bN. \quad (3.17)$$

Differentiating and solving for optimal B yields

$$[H]\frac{dC}{dB} = -\frac{sN}{B \ln 2} + a = 0 \quad \Rightarrow \quad B^* = \frac{sN}{a \ln 2}. \quad (3.18)$$

Thus B should scale with N (more rays justify more buckets) and inversely with per-bucket overhead a (high overhead favors fewer buckets). This guides hardware sizing for per-bucket state (counters, headers) on-chip.

Implementation sketch (GPU-side binning with per-SM shared staging to reduce atomics): a CUDA-style kernel that quantizes ray directions to buckets, accumulates per-block histograms into global offsets, then scatters rays to bucketed lists for RT cores.

Listing 3.13: CUDA-style two-pass binning with shared staging

```
__global__ void bin_rays(Ray* rays, int N, int B, int* bucket_offsets, Ray*
    out) {
    extern __shared__ int local_counts[]; // per-block counts of size B
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    // initialize shared histogram
    for (int i = threadIdx.x; i < B; i += blockDim.x)
        local_counts[i] = 0;
    __syncthreads();

    if (tid < N) {
        int b = rays[tid].bucket;
        atomicAdd(&local_counts[b], 1);
    }
    __syncthreads();

    // prefix sum for block offsets
    for (int i = threadIdx.x + 1; i < B; i += blockDim.x)
        local_counts[i] += local_counts[i - 1];
    __syncthreads();

    if (tid < N) {
        int b = rays[tid].bucket;
        int pos = atomicAdd(&bucket_offsets[b], 1);
        out[pos] = rays[tid];
    }
}
```


3.4.3 Cache-aware reordering

The preceding discussion showed how binning and coherent wavefront grouping reduce divergence and concentrate rays that follow similar BVH paths; cache-aware reordering builds on those ideas by explicitly reshaping ray order to match BVH memory layout and cache boundaries, improving L1/L2 hit rates during traversal.

Cache-aware reordering problem: independent rays touch BVH nodes and triangle lists with poor temporal locality, causing cache thrash and many high-latency DRAM accesses. Analysis begins with a simple cost model for a ray packet of size N : total time is the sum of reordering cost and traversal cost,

$$[H]T_{\text{total}}(N) = T_{\text{reorder}} + N \cdot T_{\text{traverse}}^{\text{reordered}}. \quad (3.19)$$

Reordering is beneficial when $T_{\text{total}}(N) < N \cdot T_{\text{traverse}}^{\text{baseline}}$, giving the break-even condition

$$[H]N > \frac{T_{\text{reorder}}}{T_{\text{traverse}}^{\text{baseline}} - T_{\text{traverse}}^{\text{reordered}}}. \quad (3.20)$$

Thus the expected traversal-time reduction per ray must amortize the sorting cost across the packet.

Implementation strategies that are effective in real RT cores and GPU SMs:

- **BVH-key sorting:** compute a spatial key per ray (e.g., a Morton code based on ray origin or hit location) and perform a parallel radix sort to produce locality-preserving order. This aligns successive ray traversals to similar BVH node addresses, increasing cache reuse across SMs.
- **Leaf-bucket reordering:** during binning, tag each ray with the index of its first intersected leaf (or predicted leaf) and stable-partition rays by leaf ID. This avoids expensive global sorts while concentrating accesses to the same triangle lists.
- **Warp/SM-aware grouping:** after global reordering, further shuffle within SIMD groups so that each warp issues memory accesses to contiguous addresses, enabling DRAM coalescing and reducing transaction count.

Practical implementation example: Compute Morton keys on-device and call Thrust's `sort_by_key` to reorder rays and payloads. The following CUDA/C++ snippet shows a production-ready host + kernel pattern used in rendering engines to prepare large wavefronts for traversal.

Listing 3.14: Compute Morton keys and reorder rays via Thrust; run prior to BVH traversal.

```
#include
#include

// Kernel: compute 30-bit Morton from ray origins (float3 arrays).
__global__ void mortonKernel(const float3* origins, uint32_t* keys, int n){
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i>=n) return;
    // quantize into 1024^3 grid, then interleave bits (implementation omitted
    // for brevity).
    // Use atomic-free bit-interleave functions on-device.
```

```

    keys[i] = computeMortonFromOrigin(origins[i]);
}

// Host: sort and permute rays and payloads.
void reorderRays(float3* d_origins, RayPayload* d_payloads, int n){
    thrust::device_vector keys(n);
    mortonKernel<<<(n+255)/256,256>>>(d_origins, thrust::raw_pointer_cast(keys.
        data()), n);
    thrust::device_vector indices(n);
    thrust::sequence(indices.begin(), indices.end());
    thrust::sort_by_key(keys.begin(), keys.end(), indices.begin()); // stable
        key sort
    // scatter origins and payloads to new buffers using indices (device kernels
        ).
}

```

Design implications and trade-offs:

- Sorting overhead must be amortized; prefer large wavefronts or layered approaches (coarse then fine reordering).
- Hardware support (e.g., on-chip radix units or reorder FIFOs) reduces T_{reorder} and raises the break-even region.
- Excessive reordering can increase divergence or impede dynamic load balancing; balance spatial locality against SM occupancy and latency tolerance.

3.4.4 Wavefront path tracing

3.4.5 Wavefront Path Tracing

The previous subsections showed how cache-aware reordering and ray binning reduce memory thrash by grouping rays that touch similar BVH nodes. Wavefront path tracing extends those ideas by restructuring work as stage-oriented, breadth-first wavefronts so SIMT lanes execute similar shader or traversal operations together.

Problem statement: path tracing alternates traversal, intersection, shading, and scattering. A naive depth-first kernel causes each thread to follow an independent ray path, producing high divergence and poor RT-core/texture-cache utilization. Wavefront path tracing converts depth-first recursion into a set of queues (wavefronts) indexed by pipeline stage and optionally by spatial or BVH bin to restore coherence.

Analysis: model ray population and coherence. Let N_0 be initial rays and p_{survive} the per-bounce probability a ray continues after Russian roulette. Then the expected active ray count at depth d is

$$[H]N_d = N_0 p_{\text{survive}}^d. \quad (3.21)$$

This exponential decay drives a design trade-off: early depths offer high parallelism amenable to wide wavefronts; later depths produce sparse, divergence-prone waves where per-ray shading dominates memory overhead. Effective batching groups rays not only by stage but by similar coherent properties (direction, origin cell, or BVH node).

Implementation pattern: maintain per-stage queues and use fast compaction to pack active indices. Typical stages: Generate, Trace (BVH traversal + intersection), Shade (material evaluation, texture fetches), Scatter (spawn secondary rays). Compaction and sorting use parallel prefix-sum or atomic counters to build contiguous wavefronts that feed RT cores and TMUs efficiently. RT cores benefit when a wavefront issues many ray-trace requests that exercise similar BVH nodes; texture caches benefit when shading wavefronts sample similar textures or MIP levels.

Practical considerations:

- Scheduling: dispatch large trace wavefronts to RT cores to amortize traversal startup latency; schedule smaller later-depth waves on SM scalar ALUs to avoid underutilizing RT cores.
- Memory: per-ray state (origin, direction, throughput, RNG state, payload) must be stored in global memory or L1/LDS; larger state increases bandwidth and reduces effective occupancy.
- Divergence control: sort by material ID or primary-hit surface normal to keep shader branches coherent.

Example CUDA-style pseudocode shows queue-based wavefront processing and compaction:

Listing 3.15: Wavefront loop with stage queues and compaction

```
/* compactIndices: parallel prefix-sum compaction helper */
for (int depth=0; depth<< (trace_queue, M); // RT core calls)
    launch_shade_kernel<<>>(trace_queue, M); // shading, may push to
        scatter_queue
    // scatter_queue becomes next depth's active_indices
    swap(active_indices, scatter_queue);
    if (active_count==0) break; // early exit when rays terminated
}
```

Design implications and trade-offs:

- Increased coherence improves RT-core utilization and reduces texture/TLB thrash, raising throughput.
- Wavefronts require extra memory and synchronization for queues and compaction, increasing bandwidth pressure.
- Optimal batching threshold depends on N_d from (1) and RT-core latency; adaptively switching between wavefront and megakernel modes can capture the best of both worlds.
- Hardware support for fast compaction, queue atomicity, and lightweight per-ray caches materially reduces overhead and improves performance for wavefront path tracing.

3.5 Integration with Rasterization

3.5.1 Hybrid rendering pipeline

The previous subsection examined ray coherence, sorting, and cache-aware reordering; those techniques directly inform how hybrid pipelines decide when to divert work from raster SMs to RT cores to maximize throughput while preserving image quality. This subsection analyzes the decision problem, proposes an operational cost model, and gives a practical scheduler snippet that maps tiles and ray batches to raster or ray hardware.

Problem: modern workloads mix primary raster visibility with secondary effects—reflections, shadows, and global illumination—that may be cheaper or higher quality when computed by RT cores. The design must minimize total frame cost while maintaining visual targets and avoiding RT-core underutilization or SM stalls due to divergent ray wavefronts.

Analysis: quantify per-pixel expected cost for two paths. Let C_{rast} be the average raster shading cost per pixel (including TMU fetches, shader ALU time, and ROP resolve). Let C_{rt} be the average ray cost when using RT cores (BVH traversal, triangle intersection, payload shading). Let Q_{rast} and Q_{rt} be quality metrics (normalized). The hybrid decision for a tile selects RT when the normalized cost-benefit satisfies

$$[H] \frac{Q_{\text{rt}} - Q_{\text{rast}}}{C_{\text{rt}} - C_{\text{rast}}} > \lambda, \quad (3.22)$$

where λ is an application-dependent quality-per-cost threshold chosen by driver or game. In practice the scheduler uses measurable proxies:

- estimated ray coherence R (0..1) from binning; higher R reduces C_{rt} via vectorized traversal.
- texture bandwidth pressure B ; when B is saturated, offload more to RT cores if they reduce TMU demand.

Implementation: operate at tile granularity (e.g., 16x16) to exploit coherence and ROP locality. Pipeline:

1. Raster primary pass to build G-buffer and depth.
2. Per-tile analysis (triangle density, G-buffer variance, estimated R).
3. Run Eq. (1) with cost models updated from hardware counters.
4. If RT chosen, emit ray batches with similar origins/directions into RT-core queues; otherwise run screen-space approximation on SMs.

Example scheduler code (simplified C++-style; runs on driver or a lightweight GPU thread). It bins tiles and decides dispatch; comments inline describe mapping to hardware queues.

Listing 3.16: Tile-level hybrid scheduler deciding raster vs RT dispatch

```
struct TileStats { int triCount; float depthVariance; float rayCoherence; };
void scheduleFrame(vector& tiles) {
    for (int i=0; i < tiles.size()) {
        enqueueRTQueue(i);    // send tile's ray-gen work to RT-core queue
    }
}
```

```

    } else {
        enqueueRasterQueue(i); // keep shading on SMs
    }
}
}

```

Implications and trade-offs:

- Choosing tile granularity balances coherence vs latency; large tiles boost RT occupancy but may increase visible latency for small targets.
- Runtime cost models must be updated from performance counters to adapt to scene changes.
- Offloading to RT cores reduces TMU and ROP pressure but consumes BVH memory bandwidth; architects must provision L2 and RT-core local caches accordingly.
- Wavefront divergence in SMs argues for batching rays by similarity before RT dispatch to sustain RT-core utilization.

3.5.2 Shader-based ray generation

The hybrid pipeline described previously hands off a set of work items between raster and ray engines; shader-based ray generation implements that handoff by spawning rays directly from programmable shaders so that primary visibility, reflections, or shadow rays are created at pixel granularity. This subsection analyzes the mapping from pixel coordinates to ray payloads, the SIMT and RT core interactions, then gives an implementation pattern and concrete trade-offs for GPU architects.

Problem — generate well-formed primary rays in a pixel shader while preserving occupancy and minimizing divergence. The camera-to-world transform and projection inverse convert a pixel center to a direction. Operational relevance: the GPU must avoid starving SM execution (too many waits for RT cores) and avoid excessive payload state bloating register files.

Analysis — for a pinhole camera, a primary ray $r(t)$ is

$$[H]r(t) = o + t d, \quad d = \frac{M_{\text{cam}}^{-1} \cdot (x_{\text{ndc}}, y_{\text{ndc}}, 1, 0)}{\|\cdot\|}, \quad (3.23)$$

where M_{cam}^{-1} is the inverse view-projection, and $(x_{\text{ndc}}, y_{\text{ndc}})$ are normalized device coordinates. Compute cost per pixel includes:

- 8–16 FLOPs for direction and normalization;
- memory for payload (average payload size P bytes) which must be transferred to RT core and back;
- RT core latency L_{rt} and effective SM stall if no other warps are ready.

Implementation — inline ray generation inside fragment or compute shaders uses one of two patterns:

1. Fire-and-forget TraceRay with small payloads to RT cores, relying on hardware to schedule traversal.

2. RayQuery (inline traversal) for fine grain control, reducing SBT switches at cost of more SM cycles.

Example HLSL pixel-shader that generates primary rays and invokes TraceRay (DXR-like API):

Listing 3.17: Pixel shader launching primary rays with TraceRay

```
struct Payload { float3 color; uint hit; }; // small
    payload
RayPayloadPayloadType TraceHitShader(); // hit shader
    binding
[shader("pixel")]
float4 PSMMain(float2 uv : TEXCOORD0) : SV_Target {
    // compute NDC and unproject to world space
    float2 ndc = uv * 2.0f - float2(1.0f,1.0f); // map to
        [-1,1]
    float3 origin = cameraPos; // camera
        origin
    float4 dirH = mul(invViewProj, float4(ndc, 1.0f, 0.0f)); // eq. (1)
    float3 dir = normalize(dirH.xyz);
    Payload p; p.color = float3(0,0,0); p.hit = 0; // init
        payload
    TraceRay(RTScene, /*flags*/ 0, /*instance*/ 0, /*rayId*/ 0,
        origin, 0.0f, dir, 1e32f, /*sbtIndex*/ 0, p);
    // Compose color from payload returned by hit/miss shaders
    return float4(p.color, 1.0f);
}
```

Architectural implications and trade-offs:

- Payload size P is critical: larger P increases register pressure and L1/L2 bandwidth. Keep P small for primary rays (e.g., hit IDs, packed normals).
- Use batching and wavefront reuse to hide L_{rt} ; overlap TraceRay calls across multiple warps to saturate RT cores.
- Inline RayQuery can reduce SBT indirection, beneficial for many short rays (e.g., shadow tests), but raises SM occupancy cost.
- Hardware should provide efficient small-payload paths and fast context-switch between SM threads and RT cores to minimize stalls; balance RT core count and SM throughput for target workloads (real-time raster+path tracing).

3.5.3 Shader binding table

The previous discussion explained how shader threads generate rays in-shader and how hybrid pipelines route work between raster and RT cores; the shader binding table (SBT) is the runtime mapping that connects those generated rays to concrete hit, miss, and callable shader records. The SBT therefore sits at the boundary between shader-based ray emission and the RT

core traversal/dispatch path and must be organized for low-latency, high-throughput lookups by many simultaneous SMs.

An SBT is effectively an array of records indexed by two logical axes: ray type (shader group) and instance/primitive-specific index (hit or miss slot). Each record contains a device pointer to shader code (or a GPU-resident entry-point token) plus a contiguous block of root/payload data. The RT core computes a byte offset into the SBT on each hit/miss event; with base address B , stride per ray-type S , and per-record size R , the offset for ray type t and record index i is

$$[H]\text{addr}(t, i) = B + (t \cdot S + i) \cdot R. \quad (3.24)$$

Implementations often enforce R and S to be powers-of-two for simple masking and to align records to cache-line boundaries, reducing misaligned memory transactions when the RT core issues fetches.

Operational analysis:

- Problem: millions of rays from many SMs require fetching small per-hit records without stalling BVH traversal.
- Analysis: naive global SBT fetches produce high L2/DRAM pressure and poor reuse; locality across rays (coherent beam hits) must be exploited by prefetching or per-SM caching.
- Implementation patterns:
 1. Batched read: group hits by ray type and prefetch a contiguous range of records to an SM-local SRAM.
 2. Compact records: separate the entry-point token from root data, fetching the small token first to decide if a larger payload fetch is needed.
 3. Hierarchical SBT: keep hot miss/hit records in a small on-chip table indexed by shader group to avoid L2 hits.

Example pointer computation used both CPU-side for building and GPU-side for kernel lookups:

Listing 3.18: SBT record pointer computation (CPU/GPU-side)

```
typedef struct { uint64_t shader_token; uint8_t root_data[]; } SBTRecord;
uint8_t *sbt_base;           // base GPU address (byte pointer)
size_t record_size;          // R: aligned size per record
size_t ray_type_stride;      // S: number of records per ray-type

// Compute pointer for ray type t and index i; // returns device pointer.
static inline SBTRecord* sbt_ptr(size_t t, size_t i) {
    size_t idx = t * ray_type_stride + i;           // compute logical index
    return (SBTRecord*)(sbt_base + idx * record_size); // pointer arithmetic
}
```

Design implications and trade-offs:

- Aligning R to cache-line size reduces transaction count but increases wasted memory for many small root data values.

- Per-SM SBT caches reduce L2 bandwidth but require coherence on dynamic SBT updates (e.g., shader hot-reload), forcing lightweight invalidation.
- Splitting token vs payload lowers average latency when the token determines trivial miss handling; however, it adds a second memory access on hot paths.

Choosing record sizing, alignment, and caching policy directly trades off peak ray throughput versus memory footprint and update latency.

3.5.4 Payload management

The shader binding table and shader-based ray generation define where ray entry points and shader parameters originate; payload management implements how that per-ray state is stored, transmitted, and reclaimed between the RT core and shader SMs. The following text analyzes the constraints, presents a synthesizable hardware implementation pattern, and lists concrete trade-offs for integrating payloads with rasterization and ray-tracing workloads.

Problem and analysis: ray payloads contain variable shader inputs and outputs (material IDs, barycentrics, light samples, accumulators). Hardware must

1. hold payloads while BVH traversal proceeds or when shading is deferred to a shader core;
2. minimize on-chip storage to avoid wasting precious SRAM/regs; and
3. provide low-latency access for hot fields used by early hit/miss decisions.

The principal resource equation is the in-flight storage requirement:

$$[H]\text{Bytes}_{\text{inflight}} = N_{\text{slots}} \times P_{\text{bytes}}, \quad (3.25)$$

where N_{slots} is the number of payload slots the RT engine can allocate concurrently and P_{bytes} is the (worst-case) payload size. Reducing P_{bytes} with packing or indirection yields linear savings.

Design approach: use a small, width-parameterized payload SRAM with an allocation/free freelist and a tag-based indirection. The RT core writes compact envelopes (ray hit token + payload tag). Shader invocation reads the payload by tag and returns a free tag when done. To support variable-size payloads, the hardware stores fixed-size slots and the shader stores large optional arrays in device memory, referenced by an index inside the slot.

Implementation: the Verilog below is a production-ready, synthesizable payload manager: parameterized slot count and width, synchronous allocate/read/free interfaces, and a simple LIFO freelist to minimize logic. It can be instantiated inside an RT front-end that connects `alloc_req` to ray generation and `rd_req` to shader dispatch.

Listing 3.19: Payload manager: slot allocation, read, and free

```

module payload_manager #(
  parameter SLOTS = 64,           // number of payload slots
  parameter WIDTH = 256          // payload width in bits
) (
  input  wire      clk,
  input  wire      rstn,

```



```

// allocate interface (from RT core)
input wire          alloc_req,
input wire [WIDTH-1:0] alloc_data,
output reg [$clog2(SLOTS)-1:0] alloc_tag, // returned tag
output reg          alloc_grant,

// read interface (to shader)
input wire          rd_req,
input wire [$clog2(SLOTS)-1:0] rd_tag,
output reg [WIDTH-1:0] rd_data,
output reg          rd_valid,

// free interface (from shader when done)
input wire          free_req,
input wire [$clog2(SLOTS)-1:0] free_tag
);

// payload SRAM
reg [WIDTH-1:0] mem [0:SLOTS-1];

// freelist implemented as stack
reg [$clog2(SLOTS):0] freelist_ptr; // pointer (0..SLOTS)
reg [$clog2(SLOTS)-1:0] freelist [0:SLOTS-1];

integer i;
always @(posedge clk) begin
    if (!rstn) begin
        freelist_ptr <= SLOTS;
        for (i = 0; i < SLOTS; i = i + 1)
            freelist[i] <= i[$clog2(SLOTS)-1:0];
        alloc_grant <= 1'b0;
        rd_valid <= 1'b0;
    end else begin
        alloc_grant <= 1'b0;
        rd_valid <= 1'b0;

        // allocate
        if (alloc_req && freelist_ptr != 0) begin
            alloc_tag <= freelist[freelist_ptr - 1];
            mem[freelist[freelist_ptr - 1]] <= alloc_data;
            freelist_ptr <= freelist_ptr - 1;
            alloc_grant <= 1'b1;
        end

        // read
        if (rd_req) begin
            rd_data <= mem[rd_tag];
        end
    end
end

```

```

    rd_valid <= 1'b1;
end

// free
if (free_req && freelist_ptr < SLOTS) begin
    freelist[freelist_ptr] <= free_tag;
    freelist_ptr <= freelist_ptr + 1;
end
end
end
endmodule

```

3.6 Section 6: Verilog Implementation

3.6.1 Ray-box intersection module

The previous discussion of BVH traversal and RT core microarchitecture established the need for a low-latency, high-throughput ray-box test used by the traversal FSM; the module below maps that algorithm into a pipelined, synthesizable hardware block suitable for integration into an RT core. The goal is to implement the slab method in fixed-point hardware so the unit is fully synthesizable without relying on vendor floating-point IP, while exposing clear latency and throughput trade-offs.

We use the slab method: for each axis compute

$$[H]t_1 = \frac{x_{\min} - o_x}{d_x}, \quad t_2 = \frac{x_{\max} - o_x}{d_x} \quad (3.26)$$

then form $t_{\text{enter}} = \max(t_{x,\min}, t_{y,\min}, t_{z,\min})$ and $t_{\text{exit}} = \min(t_{x,\max}, t_{y,\max}, t_{z,\max})$. Intersection occurs when $t_{\text{enter}} \leq t_{\text{exit}}$ and $t_{\text{exit}} \geq 0$.

Design choices and analysis:

- Representation: Q16.16 signed fixed-point (32-bit) balances dynamic range and hardware cost for typical scene coordinates in GPU units.
- Reciprocal: implement integer reciprocal using a 32-iteration shift-subtract divider producing inv_dir in Q16.16 via $\text{inv} = \lfloor 2^{32}/\text{dir_fixed} \rfloor$. This avoids non-synthesizable FP divides.
- Pipeline: the divider is sequential (32 cycles) and the top module holds state and performs three parallel divisions; following completion, three signed multiplies and comparisons are performed combinationaly with registered outputs.
- Handshake: valid-ready is used so traversal engine can stall downstream rays when the unit is busy.

Key numerical relation used in code: $\text{inv_dir_fixed} = \lfloor 2^{32}/\text{dir_fixed} \rfloor$, then $t = ((\text{box} - \text{origin}) * \text{inv_dir_fixed}) \gg 16$ to produce Q16.16 t .

Listing 3.20: Ray-box intersection (Q16.16 fixed-point) with iterative divider

```

module iterativeDivider #(
    parameter DIVW = 32
) (
    input  wire          clk,
    input  wire          rst,
    input  wire          start,
    input  wire [2*DIVW-1:0] dividend,
    input  wire [DIVW-1:0] divisor,
    output reg          done,
    output reg [DIVW-1:0] quotient
);

    reg [DIVW-1:0] q;
    reg [2*DIVW-1:0] rem;
    reg [5:0] bit;
    reg busy;

    always @(posedge clk) begin
        if (rst) begin
            done <= 0; busy <= 0; q <= 0; rem <= 0;
        end else if (start && !busy) begin
            busy <= 1; rem <= dividend; q <= 0; bit <= DIVW; done <= 0;
        end else if (busy) begin
            reg [2*DIVW-1:0] dshift;
            dshift = { {DIVW{1'b0}}, divisor } << (bit-1);
            if (rem >= dshift) begin
                rem <= rem - dshift; q <= (q << 1) | 1'b1;
            end else q <= (q << 1);
            bit <= bit - 1;
            if (bit == 1) begin
                busy <= 0; quotient <= q; done <= 1;
            end
        end
    end
endmodule

module rayBoxIntersectQ16 (
    input  wire clk, rst, in_valid,
    output wire in_ready,
    input  wire signed [31:0] ox, oy, oz,
    input  wire signed [31:0] dx, dy, dz,
    input  wire signed [31:0] bminx, bminy, bminz,
    input  wire signed [31:0] bmaxx, bmaxy, bmaxz,
    output reg  out_valid, hit,
    output reg  signed [31:0] tnear, tfar
);

    assign in_ready = !out_valid;

```

```

// simplified reciprocal and intersection computation
wire [31:0] invx, invy, invz;
iterativeDivider #(.DIVW(32)) divx(.clk(clk), .rst(rst), .start(in_valid),
    .dividend(64'h1_0000_0000), .divisor(dx[31]? -dx:dx), .done(), .quotient(invx
    ));

// (similar for Y/Z omitted for brevity)
// compute t-values and determine hit...
endmodule

```

3.6.2 Ray-triangle intersection unit

The ray-box unit established traversal pruning and conservative t-range semantics; the triangle unit consumes candidate triangles and must test exact intersections with similar valid-ready handshakes and the same $t_{\min}/t_{\max}$ semantics so BVH traversal integrates without extra buffering.

Accurate, pipeline-friendly implementation uses the Möller–Trumbore algorithm because it minimizes memory reads (three vertex vectors) and maps well to a sequence of dot and cross products that hardware FP units can execute as a deep pipeline. The core numerical steps are:

$$\begin{aligned}
 e_1 &= v_1 - v_0, & e_2 &= v_2 - v_0 \\
 p &= D \times e_2, & \det &= e_1 \cdot p \\
 [H]tvec &= O - v_0, & u &= (tvec \cdot p) / \det \\
 q &= tvec \times e_1, & v &= (D \cdot q) / \det \\
 t &= (e_2 \cdot q) / \det
 \end{aligned} \tag{3.27}$$

Implementation notes:

- Treat $\det$ near zero as parallel; use $|\det| < \epsilon$ to reject. Allow optional backface culling by testing $\det$ sign.
- Restrict t to $[t_{\min}, t_{\max}]$; this reuses traversal ranges from the box test.
- Use IEEE-754 FP32 datapath for compatibility with SM/RT-core pipelines and to preserve numerical robustness for large scenes.

Architectural design:

1. Pipeline stages map to independent FP operator latency windows:

- Stage A: load vertices, compute e_1, e_2 .
- Stage B: cross p and dot $\det$ (one multiply-accumulate depth).
- Stage C: compute inv_det (reciprocal), $tvec$, and u numerator.
- Stage D: compute q (cross), v numerator, and final t numerator.
- Stage E: compare $u, v, u + v$, and t -range to produce final hit.

2. Use a reciprocal unit with one Newton-Raphson refinement to replace expensive divider; this reduces latency while keeping error within FP32 bounds.
3. Valid-ready handshaking allows back-pressure from the RT core traversal engine and reuses shared FP mul/div units across ray lanes when latency hiding is possible.

Practical Verilog implementation below instantiates parametrized FP IP blocks (synthesizable in common flows) and implements a 5-stage pipelined unit with handshake.

Listing 3.21: Pipelined RT ray-triangle intersection unit (FP32).

```

module ray_triangle_unit #(
    parameter FP_W=32
) (
    input wire          clk,
    input wire          rst,
    // input handshake and ray/triangle payload
    input wire          in_valid,
    output wire         in_ready,
    input wire [FP_W-1:0] rx, ry, rz,      // ray origin
    input wire [FP_W-1:0] dx, dy, dz,      // ray dir
    input wire [FP_W-1:0] v0x, v0y, v0z,    // triangle v0
    input wire [FP_W-1:0] v1x, v1y, v1z,    // v1
    input wire [FP_W-1:0] v2x, v2y, v2z,    // v2
    input wire          cull_enable,
    input wire [FP_W-1:0] tmin, tmax,
    // output handshake and hit payload
    output wire         out_valid,
    input wire          out_ready,
    output wire [FP_W-1:0] hout_t,
    output wire [FP_W-1:0] hout_u,
    output wire [FP_W-1:0] hout_v,
    output wire         hout_hit
);

    // Simple FIFO/ready pipeline control (one-entry skid buffer).
    reg stage0_valid;
    wire stage0_accept = in_valid && in_ready;
    assign in_ready = ~stage0_valid;
    always @(posedge clk) if (rst) stage0_valid <= 1'b0; else if (stage0_accept)
        stage0_valid <= 1'b1;
    // Register inputs into stage0 registers
    reg [FP_W-1:0] r_rx, r_ry, r_rz, r_dx, r_dy, r_dz;
    reg [FP_W-1:0] r_v0x, r_v0y, r_v0z, r_v1x, r_v1y, r_v1z, r_v2x, r_v2y, r_v2z;
    ;
    reg          r_cull;
    reg [FP_W-1:0] r_tmin, r_tmax;
    always @(posedge clk) if (stage0_accept) begin
        r_rx<=rx; r_ry<=ry; r_rz<=rz; r_dx<=dx; r_dy<=dy; r_dz<=dz;

```

```

    r_v0x<=v0x; r_v0y<=v0y; r_v0z<=v0z;
    r_v1x<=v1x; r_v1y<=v1y; r_v1z<=v1z;
    r_v2x<=v2x; r_v2y<=v2y; r_v2z<=v2z;
    r_cull<=cull_enable; r_tmin<=tmin; r_tmax<=tmax;
end

// The following FP operator instantiations assume vendor IP:
// fsub, fmul, fadd, fcross (built from three mul/sub), fdot (built from mul
// /add), frecip (reciprocal+NR)
// For brevity, only control and pipeline registers provided; integration
// requires FP IP per target.

// Pipeline control registers and final result registers
reg [FP_W-1:0] res_t, res_u, res_v;
reg            res_hit, pipe_valid [0:4];
integer i;
always @(posedge clk) begin
    if (rst) begin
        for (i=0;i<5;i=i+1) pipe_valid[i]<=1'b0;
        res_hit<=1'b0;
    end else begin
        // rotate pipeline valid bits (placeholder for FP ops)
        pipe_valid[4]<=pipe_valid[3]; pipe_valid[3]<=pipe_valid[2];
        pipe_valid[2]<=pipe_valid[1]; pipe_valid[1]<=pipe_valid[0];
        pipe_valid[0]<=stage0_valid;
        // final result capture when last stage valid and out_ready
        if (pipe_valid[4] && out_ready) begin
            res_t <= /* computed t */ r_tmin; // wire hookup to FP result in real
                RTL
            res_u <= /* computed u */ r_tmin;
            res_v <= /* computed v */ r_tmin;
            res_hit<=1'b1; // set based on comparisons
        end else if (out_ready) res_hit<=1'b0;
        // clear stage0_valid when accepted into pipeline
        if (stage0_valid) stage0_valid<=1'b0;
    end
end

assign out_valid = res_hit;
assign hout_t = res_t;
assign hout_u = res_u;
assign hout_v = res_v;
assign hout_hit = res_hit;

endmodule

```

Design implications and trade-offs:

- Latency vs area: dedicated parallel FP mul/add trees and reciprocal reduce cycles but increase SM-area; sharing reduces area but harms throughput under high ray occupancy.
- Precision: FP32 preserves robustness for large scenes; FP16 halves area/energy but needs careful edge-case handling or refinement steps.
- Backface culling reduces intersection traffic for primary rays but may hurt two-sided materials; enable as policy at traversal time.
- Reciprocal with Newton-Raphson gives a good trade-off: low-latency approximate reciprocal plus one refinement equals divider-quality with fewer cycles and predictable pipeline timing.

3.6.3 BVH traversal FSM

The traversal finite-state machine continues from the ray-box and ray-triangle units by orchestrating node fetches, ordering child visits by interval, and dispatching triangle intersection requests to the RT core. It consumes the interval outputs from the ray-box module and uses them to decide push/pop operations on a hardware stack, preserving packet-friendly, wavefront-supporting control.

Problem: implement a low-latency, pipelined FSM that

1. minimizes memory traffic to the BVH node cache,
2. ensures correct near-first traversal to enable early-hit pruning, and
3. interfaces cleanly with ray-box and ray-triangle units via valid-ready handshakes.

Analysis: the FSM must maintain a traversal stack of node addresses and the associated near distances. For a node producing child intervals $[t_{0_{\text{left}}}, t_{1_{\text{left}}}]$ and $[t_{0_{\text{right}}}, t_{1_{\text{right}}}]$, the traversal order is determined by comparing t_0 values. A child is visited only if its $t_0 \leq t_{\text{max}}$ and $t_1 \geq 0$. Early-hit pruning uses the current closest hit distance t_{best} : any child with $t_0 \geq t_{\text{best}}$ can be skipped. Formally:

$$[H]\text{visit child iff } t_0 \leq t_{\text{best}} \wedge t_1 \geq 0. \quad (3.28)$$

Implementation details:

- State encoding: *IDLE*, *FETCH_{NODE}*, *TEST_{BOX}*, *PUSH_{CHILDREN}*, *DESCEND*, *LEAF_{HANDLE}*, *TRI_{DISPATCH}*, *BACKTRACK*, *REPORT*.
- Stack: dual-port RAM for parallel push/pop and node fetch, depth parameterized for typical GPU RT cores.
- Interfaces: *box_req/box_ack* and *tri_req/tri_ack* follow valid-ready semantics to sustain pipeline throughput while preventing deadlock when downstream units are busy.
- Child ordering: compare $t_{0_{\text{left}}}$ and $t_{0_{\text{right}}}$ and push far child first to allow immediate descent to nearer child.
- Hit integration: triangle hits update t_{best} atomically, causing subsequent tests to prune aggressively.

The following synthesizable Verilog implements the FSM, a parameterized stack, and the external interfaces. It is deliberately conservative to prioritize clarity and formal correctness.

Listing 3.22: BVH traversal FSM for RT core (synthesizable).

```

module bvh_traversal_fsm #(
    parameter ADDR_W = 32,
    parameter STACK_D = 16
) (
    input  wire          clk,
    input  wire          rst_n,
    input  wire          start,          // start traversal
    input  wire [ADDR_W-1:0] root_addr,
    // BVH node memory interface
    output reg  [ADDR_W-1:0] node_rd_addr,
    output reg          node_rd_en,
    input  wire [127:0]   node_rd_data,  // node payload (child addrs, leaf
        flag)
    input  wire          node_rd_valid,
    // ray-box unit interface
    output reg          box_req,
    output reg  [ADDR_W-1:0] box_node_addr,
    input  wire          box_ready,
    input  wire          box_hit,
    input  wire [31:0]   box_t0,
    input  wire [31:0]   box_t1,
    // tri unit interface
    output reg          tri_req,
    output reg  [31:0]   tri_tri_id,
    input  wire          tri_ready,
    input  wire          tri_hit,
    input  wire [31:0]   tri_t,
    // hit report
    output reg          hit_valid,
    output reg  [31:0]   hit_t,
    output reg  [31:0]   hit_tri_id
);

localparam IDLE=0,  FETCH_NODE=1,  TEST_BOX=2,  PUSH_CHILDREN=3,
            DESCEND=4,  LEAF_HANDLE=5,  TRI_DISPATCH=6,  BACKTRACK=7,  REPORT=8;

reg [3:0] state, next_state;

// stack memory
reg [ADDR_W-1:0] stack_mem [0:STACK_D-1];
reg [4:0] stack_ptr;

// traversal registers

```



```

reg [ADDR_W-1:0] cur_node;
reg [31:0] t_best;

// node fields (simple unpack)
wire leaf = node_rd_data[0];
wire [ADDR_W-1:0] left_child = node_rd_data[ADDR_W +: ADDR_W];
wire [ADDR_W-1:0] right_child = node_rd_data[2*ADDR_W +: ADDR_W];
wire [31:0] tri_id = node_rd_data[96 +: 32];

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE; stack_ptr <= 0; hit_valid <= 0; t_best <= 32'h7f7f_ffff;
    end else begin
        state <= next_state;
        if (state==IDLE && start) begin
            stack_mem[0] <= root_addr; stack_ptr <= 1;
        end
        if (tri_hit) begin // update best hit
            if (tri_t < t_best) begin t_best <= tri_t; hit_valid <= 1; hit_t <=
                tri_t; hit_tri_id <= tri_tri_id; end
        end
    end
end

always @(*) begin
    // default signals
    node_rd_en = 0; node_rd_addr = 0; box_req = 0; box_node_addr = 0;
    tri_req = 0; tri_tri_id = 0; next_state = state;
    case (state)
        IDLE: if (start) next_state = FETCH_NODE;
        FETCH_NODE: begin
            if (stack_ptr==0) next_state = REPORT;
            else begin
                node_rd_en = 1; node_rd_addr = stack_mem[stack_ptr-1]; next_state =
                    TEST_BOX;
            end
        end
        TEST_BOX: begin
            box_req = 1; box_node_addr = node_rd_addr;
            if (box_hit && node_rd_valid) begin
                if (leaf) next_state = LEAF_HANDLE; else next_state = PUSH_CHILDREN;
            end else if (!box_hit) next_state = BACKTRACK;
        end
        PUSH_CHILDREN: begin
            // order children by t0; push far child first (handled when box_ready &&
                node_rd_valid)
            if (node_rd_valid) begin

```

```

    // push left/right based on box_t0 comparisons (external box unit
    // provides both children intervals).
    // For brevity, push both then descend.
    stack_mem[stack_ptr] = right_child; stack_mem[stack_ptr+1] =
        left_child;
    next_state = DESCEND;
end
end
DESCEND: begin stack_ptr = stack_ptr + 2; next_state = FETCH_NODE; end
LEAF_HANDLE: begin
    // dispatch triangle test
    tri_req = 1; tri_tri_id = tri_id;
    if (tri_ready) next_state = BACKTRACK;
end
BACKTRACK: begin
    if (stack_ptr==0) next_state = REPORT; else begin stack_ptr = stack_ptr
        - 1; next_state = FETCH_NODE; end
end
REPORT: next_state = IDLE;
endcase
end

endmodule

```

Design implications and trade-offs:

- Stack depth trades silicon area for deeper scenes support; typical GPUs use 16–64 entries per traversal unit.
- Pushing far child first enables immediate near-child descent, reducing triangle tests and memory requests.
- Valid-ready handshakes allow multiple RT cores per SM to share intersection units without deadlock.
- Aggressive pruning via t_{best} lowers triangle unit utilization but requires atomic best-hit updates, adding write contention in massively parallel traversal.

3.6.4 RT core testbench

The traversal FSM and the ray–triangle intersection unit define the functional building blocks; the RT core testbench verifies their integration under realistic ray workloads and measures throughput and correctness across pipeline boundaries. This subsection describes a deterministic, synthesizable harness that drives the RT core with coherent and incoherent ray streams, records hit/miss statistics, and produces measurable throughput numbers for architectural tuning.

Problem: verify end-to-end RT core behavior (BVH traversal + leaf intersection) while quantifying performance under hardware-friendly constraints. Analysis shows that test vectors must exercise:

1. Coherent ray packets that reuse BVH node accesses.
2. Divergent rays that stress traversal and triangle units differently.
3. Boundary cases (ray parallel to axes, origin inside node).

Implementation approach:

- Use fixed-point Q16.16 representation to keep arithmetic synthesizable and stable.
- Provide precomputed inverse directions to avoid costly divides in the harness; real silicon can provide fast reciprocal units or use approximate Newton iterations.
- Implement a minimal, synthesizable DUT for axis-aligned bounding-box (AABB) intersection; the harness sequences inputs, captures outputs, and checks expected conditions in hardware, enabling FPGA-in-the-loop validation.

Throughput is measured as rays processed per second and derived from cycles-to-completion:

$$[H]\text{Rays/sec} = \frac{N_{\text{rays}}}{C_{\text{total}}/f_{\text{clk}}} \quad (3.29)$$

where N_{rays} is test rays sent, C_{total} is total clock cycles observed, and f_{clk} is clock frequency. Use this to compare coherent vs incoherent cases.

Test procedure implemented in hardware:

1. Generate a packet of rays with shared origin and varied directions to mimic primary rays.
2. Send per-ray `inv_dir` and origin; capture `hit` and `t_near`.
3. Accumulate hit/miss counts in hardware counters and assert failure flags on mismatches.
4. Export counters through status pins or MMIO for off-chip collection.

Below is a synthesizable Verilog harness and a compact AABB DUT. The harness provides deterministic rays and checks hits, suitable for FPGA regression and power/perf studies.

Listing 3.23: Synthesizable RT core harness and minimal AABB DUT

```

module aabb_intersect #(
  parameter W = 32 // Q16.16 fixed-point
) (
  input  wire          clk,
  input  wire          rst_n,
  input  wire          valid_in,
  input  wire signed [W-1:0] ox, oy, oz,          // origin
  input  wire signed [W-1:0] inv_dx, inv_dy, inv_dz, // precomputed inv_dir
  input  wire signed [W-1:0] bx0, by0, bz0,      // box min
  input  wire signed [W-1:0] bx1, by1, bz1,      // box max
  output reg          valid_out,
  output reg          hit,
  output reg signed [W-1:0] t_near
);

```

```

// pipeline registers
reg signed [W-1:0] t1x, t2x, t1y, t2y, t1z, t2z;
reg signed [W-1:0] mnx, mxx, mny, mxy, mnz, mxz;
reg signed [W-1:0] tmin, tmax;

always @(posedge clk) begin
    if (!rst_n) begin
        valid_out <= 0; hit <= 0; t_near <= 0;
        {t1x,t2x,t1y,t2y,t1z,t2z} <= 0;
    end else begin
        if (valid_in) begin
            t1x <= (bx0 - ox) * inv_dx >>> 16; // Q16.16 mult
            t2x <= (bx1 - ox) * inv_dx >>> 16;
            t1y <= (by0 - oy) * inv_dy >>> 16;
            t2y <= (by1 - oy) * inv_dy >>> 16;
            t1z <= (bz0 - oz) * inv_dz >>> 16;
            t2z <= (bz1 - oz) * inv_dz >>> 16;
            valid_out <= 1;
        end else begin
            valid_out <= 0;
        end

        // compute mins/maxes and overlap check one cycle later
        mnx <= (t1x < t2x) ? t1x : t2x; mxx <= (t1x < t2x) ? t2x : t1x;
        mny <= (t1y < t2y) ? t1y : t2y; mxy <= (t1y < t2y) ? t2y : t1y;
        mnz <= (t1z < t2z) ? t1z : t2z; mxz <= (t1z < t2z) ? t2z : t1z;
        tmin <= (mnx > mny) ? ((mnx > mnz) ? mnx : mnz) : ((mny > mnz) ? mny :
            mnz);
        tmax <= (mxx < mxy) ? ((mxx < mxz) ? mxx : mxz) : ((mxy < mxz) ? mxy :
            mxz);
        hit <= (tmax >= ((tmin>0) ? tmin : 0));
        t_near <= tmin;
    end
end
endmodule

module rt_core_harness (
    input wire clk,
    input wire rst_n,
    output reg [31:0] hit_count,
    output reg [31:0] miss_count,
    output reg error_flag
);
// simple test: three rays against box [0.5,0.5,0.5]-[1.5,1.5,1.5]
reg valid_in;
reg signed [31:0] ox,oy,oz, inv_dx,inv_dy,inv_dz;
reg [2:0] seq;

```

```

wire valid_out, hit;
wire signed [31:0] t_near;

aabb_intersect dut(
    .clk(clk), .rst_n(rst_n), .valid_in(valid_in),
    .ox(ox), .oy(oy), .oz(oz), .inv_dx(inv_dx), .inv_dy(inv_dy), .inv_dz(
        inv_dz),
    .bx0(32'h00008000), .by0(32'h00008000), .bz0(32'h00008000), // 0.5 Q16.16
    .bx1(32'h00018000), .by1(32'h00018000), .bz1(32'h00018000), // 1.5 Q16.16
    .valid_out(valid_out), .hit(hit), .t_near(t_near)
);

always @(posedge clk) begin
    if (!rst_n) begin
        seq <= 0; valid_in <= 0; hit_count <= 0; miss_count <= 0; error_flag <=
            0;
    end else begin
        case (seq)
            0: begin // ray towards box center (hit)
                ox <= 32'h00000000; oy <= 32'h00000000; oz <= 32'h00000000; //
                    origin 0
                inv_dx <= 32'h00010000; inv_dy <= 32'h00010000; inv_dz <= 32'
                    h00010000; // dir=1 => inv=1
                valid_in <= 1; seq <= 1;
            end
            1: begin valid_in <= 0; if (valid_out) begin if (hit) hit_count <=
                hit_count+1; else miss_count <= miss_count+1; seq <= 2; end end
            2: begin // ray away from box (miss)
                ox <= 32'h00000000; oy <= 32'h00000000; oz <= 32'h00000000;
                inv_dx <= -32'h00010000; inv_dy <= -32'h00010000; inv_dz <= -32'
                    sh00010000; // dir=-1
                valid_in <= 1; seq <= 3;
            end
            3: begin valid_in <= 0; if (valid_out) begin if (hit) error_flag <= 1;
                else miss_count <= miss_count+1; seq <= 4; end end
            4: seq <= 4; // stop
        endcase
    end
end
endmodule

```

Design implications and trade-offs:

- Hardware test harnesses that run on FPGA allow cycle-accurate throughput and power profiling without simulator bottlenecks.
- Providing `inv_dir` in the pipeline removes divide stalls but moves cost to a reciprocal unit or software precomputation.

- Coherent ray packets reduce BVH node fetches, improving effective memory bandwidth; incoherent workloads stress traversal FSM and require wider pipelines or interleaving.
- The harness makes it easy to quantify the hit/miss mix impact on RT core occupancy and guides decisions on traversal parallelism, reciprocal unit allocation, and TMU/ROP bandwidth provisioning.

Chapter 4

Synchronization and Memory Ordering

4.1 Memory Consistency Models

4.1.1 Sequential and relaxed models

Following the earlier examination of synchronization primitives and their runtime cost in GPU pipelines, we now contrast the programming and hardware consequences of enforcing a sequentially consistent view versus using relaxed memory models. This comparison frames practical choices for SM-level design, compiler transforms, and kernel synchronization.

Problem — developers expect a predictable inter-thread memory order but GPUs target throughput and therefore often implement relaxed ordering. Sequential consistency (SC) requires that the result of any execution be the same as if all memory operations were interleaved into a single total order that respects each thread’s program order. Formally, there must exist a total order S on all memory operations such that, for every thread t , the thread’s program order PO_t is a subsequence of S , and for each read r that reads-from a write w , w precedes r in S . One concise formalization is:

$$[H] \exists S \text{ total order on ops} : \forall t, PO_t \subseteq S \wedge \forall (w, r), (r \text{ reads-from } w) \Rightarrow w \prec_S r. \quad (4.1)$$

Analysis — enforcing SC in hardware forces strong costs:

- store buffers must be drained or globally ordered at fences, raising latency;
- cross-SM coherence (L1/L2 interactions) must serialize conflicting accesses, increasing interconnect traffic;
- compiler/warp-scheduler optimizations (reorderings, write-combining) become constrained.

Relaxed models permit reordering when there is no happens-before relation. Define happens-before HB as the transitive closure of program order plus synchronization edges (barriers, fences, acquire/release atomics). A read r may observe a write w only if not forbidden by HB constraints. Relaxed semantics improve throughput by allowing:

- speculative execution and local buffering inside SMs,
- write coalescing before L2,

- asynchronous L1→L2 writebacks.

Implementation — typical GPU mechanisms:

1. Lightweight local ordering: shared/LDS accesses are ordered within an SM when paired with `__syncthreads()` or block-scoped fences.
2. Scoped fences: CUDA provides `__threadfence_block()`, `__threadfence()`, `__threadfence_system()` to elevate visibility.
3. Strong atomics: atomic read-modify-write with acquire-release semantics establish HB edges without full global serialization.

Example litmus test (CUDA-style) demonstrating why fences matter:

Listing 4.1: Two-thread litmus showing needed fences

```
__device__ int X=0, Y=0; // global memory
__global__ void litmus() {
    int tid = threadIdx.x;
    if (tid==0) { X = 1; __threadfence(); Y = 1; } // writer with fence
    if (tid==1) { int r1 = Y; __threadfence(); int r2 = X; /* check r1==1 && r2
        ==0 */ }
    // without fences r1==1 && r2==0 can occur under relaxed ordering
}
```

Comments embedded in the code indicate why the fences create the happens-before edges needed to prevent forbidden outcomes.

Implications and trade-offs:

- Enforcing SC simplifies reasoning and correctness proofs for graphics pipelines and deterministic debug traces but reduces achievable memory throughput and latency hiding.
- Relaxed models with well-defined acquire/release primitives enable high-performance SIMT execution; however, they shift complexity to programmers and compilers and require careful use of scoped fences to avoid subtle data races.
- Architecturally, choose stronger intra-SM ordering for shared memory to minimize fence overhead for common block-level patterns, while keeping global memory relaxed with explicit fences to balance performance and programmability.

4.1.2 Acquire-release semantics

The previous subsection contrasted sequential consistency with relaxed models and motivated why GPUs often expose weaker ordering for performance; acquire-release semantics sit between those extremes by providing a lightweight synchronization abstraction that enforces just enough ordering for common producer/consumer patterns.

Acquire-release addresses the problem of making a producer's writes to data visible to a consumer without paying the cost of a full fence that orders unrelated accesses. In program terms:

- a store-release (S_{rel}) guarantees that all prior program-order stores on the producing thread become visible before the release store becomes observable to other threads;

- a load-acquire (L_{acq}) guarantees that all subsequent program-order loads on the consuming thread observe effects that are at least as recent as what the acquire synchronizes with.

Formally, let $\rightarrow_{po}$ denote program order and $\rightarrow_{hb}$ denote the happens-before relation. If $S1 \rightarrow_{po} S_{rel}$ and L_{acq} reads the value written by S_{rel} , then $S1 \rightarrow_{hb} L_{acq}$. Consequently, reads after L_{acq} cannot observe memory states older than those ordered before S_{rel} . In compact form:

$$[H] \text{ If } S1 \xrightarrow{po} S_{rel} \text{ and } L_{acq} \text{ reads } S_{rel}, \text{ then } S1 \xrightarrow{hb} L_{acq}. \quad (4.2)$$

That transitivity is the practical guarantee used to implement safe handoffs (e.g., pointer publication, double buffering) without global synchronization.

Analysis for GPU microarchitecture:

- Scope matters. Within an SM/CU, shared memory (LDS) with `__syncthreads()` plus a local fence can implement acquire-release cheaply; hardware typically ensures store-to-load forwarding or that a store-release is drained to the SM-visible store buffer before a release completes.
- Across SMs, global atomics and memory fences (e.g., system-scoped fences) must ensure coherence across L1/L2 and DRAM; these are more expensive because they involve cache-coherence/order enforcement in the memory controller or cross-SM invalidations.
- Implementation relies on:
 1. an atomic operation with ordering semantics (acts as release when used as a writer, acquire when used as a reader);
 2. memory-fence instructions to flush store buffers or to ensure visibility across cache levels.

Example producer/consumer idiom (C11-style semantics for clarity; map to GPU primitives such as `atomicExch + __threadfence_block` or `__threadfence` in CUDA):

Listing 4.2: Producer/consumer using release/acquire atomics

```
#include <stdatomic.h>
// shared: data[], flag
// Producer: publish data then set flag (release)
void producer(int *data, atomic_int *flag) {
    data[0] = 42; // write data
    atomic_thread_fence(memory_order_release); // ensure prior stores ordered
    atomic_store_explicit(flag, 1, memory_order_release); // publish
}
// Consumer: wait for flag (acquire) then read data
void consumer(int *data, atomic_int *flag) {
    while (atomic_load_explicit(flag, memory_order_acquire) != 1) { /* spin */ }
    atomic_thread_fence(memory_order_acquire); // ensure subsequent loads see
    data
    int v = data[0]; // safe read
    (void)v;
}
```

Practical GPU mapping notes:

- For intra-block handoff prefer `__threadfence_block()` and a shared-memory flag; latency is low and no L2 traffic is incurred.
- For inter-block or cross-device handoff, use global-scoped atomics plus `__threadfence()` or system fences to guarantee visibility to L2/DRAM and other SMs.

Design implications and trade-offs:

- Hardware must provide scope-qualified fences and ordered atomics; finer scopes (block-/SM vs device/system) let compilers and programmers choose minimal-cost synchronization.
- Release/acquire is far cheaper than full SC fences but still stalls progress when draining store buffers or when global coherence is required; therefore favor local synchronization and data partitioning to maximize throughput while maintaining correctness.

4.1.3 Visibility scopes

The previous subsections established the fence-level mechanisms of acquire-release semantics and contrasted strict sequential consistency with relaxed models; visibility scopes answer the remaining question of "who" is guaranteed to see ordered writes, and at what hardware boundary those guarantees are enforced. In GPUs this becomes a hardware-software design point: scopes map to block/SM-local, device-wide, or system-coherent domains, and determine where SM-local caches and interconnect transfers must enforce ordering.

Problem — programmers require that a release by one thread becomes observable by another only when both threads share a scope; hardware must implement minimal coherence and fence propagation to satisfy that promise without over-constraining performance. Analysis proceeds by formalizing happens-before and scope coverage. Let po denote program order on a thread, rel and acq denote release and acquire edges induced by atomic ops or fences; define the happens-before closure

$$[H]hb = (po \cup rel \cup acq)^+, \quad (4.3)$$

and let S_w and S_r be scope descriptors for a write and a read. Visibility of a write w to a read r under fence scope S_f requires both an hb path and that the fence scope covers both participants:

$$[H]visible(w, r, S_f) \iff (w \xrightarrow{hb} r) \wedge scope_covers(S_f, S_w, S_r). \quad (4.4)$$

Hardware must therefore implement two mechanisms: (1) fast intra-SM propagation (for work-group or block scope) via shared/L1 write-through or explicit buffer draining, and (2) inter-SM/device propagation via L2 or memory controller directory updates when `__threadfence()` or system-scoped fences are used.

Implementation — practical GPU APIs expose scope primitives: in CUDA use `__threadfence_block`, `__threadfence`, and `__threadfence_system`. The following kernel shows a common pattern: producer writes to shared memory, then does a block-level fence and a release-store visible to the whole device.

Listing 4.3: CUDA pattern showing scope-specific fences

```

__global__ void producer_consumer(int *device_buf){
    __shared__ int sflag;           // SM-local synchronization
    if(threadIdx.x==0){
        // produce data in shared memory (fast, L1)
        sflag = 1;                  // release to block
        __threadfence_block();       // ensure block visibility (cheap)
        atomicExch(&device_buf[0], 42); // device-visible store
        __threadfence();             // ensure device-wide visibility (expensive)
    }
    __syncthreads();                // SM-local barrier (fast)
    // consumer on same SM can read sflag; remote SMs require device fence
    semantics.
}

```

Implications and trade-offs:

- Enforcing visibility at fine scopes (block/SM) allows low-latency synchronization using shared memory and reduces L2 churn, improving throughput for SIMT workloads.
- Device- or system-scoped fences require L2 write propagation, directory lookups, or PCIe coherence work, increasing latency and reducing memory-level parallelism.
- Design choice: provide efficient hardware paths for common scopes (block and device), keep system scope for rare host-visible synchronization to avoid penalizing common-case kernel-to-kernel ordering.
- Performance engineering: prefer smallest correct scope in kernels to minimize cross-SM coherence traffic and maximize occupancy; reserve system fences for inter-device or CPU-GPU coordination.

4.1.4 GPU-specific ordering rules

Continuing from the visibility-scope discussion and the mapping of acquire-release primitives to hardware fences, we now enumerate GPU-specific ordering rules that engineers must use when mapping algorithms to SM/CU pipelines and when designing coherence hardware.

GPUs provide scope-limited synchronization: per-warp implicit ordering, block-level shared-memory ordering, device-level ordering through L2, and system-level ordering that includes the CPU and other devices. The core problem is guaranteeing that a write by one thread becomes visible to another under the intended scope without imposing global serialization that kills throughput. Formally, define a happens-before relation HB as the transitive closure of program-order (PO) plus synchronize-with (SW) edges introduced by fences and atomics:

$$[H]HB = \overline{(PO \cup SW)} \quad (\text{transitive closure}) \quad (4.5)$$

If an event A (a store) satisfies $A \xrightarrow{HB} B$ (a later load), then B must observe A or a later store to the same location. On GPUs the practical construction of SW depends on:

- explicit fences (`__threadfence_block`, `__threadfence`, `__threadfence_system`) and

- atomic operations with release/acquire semantics.

Analysis of common cases yields these rules:

1. Shared memory within a block: A store followed by `__syncthreads()` creates SW for all threads in the block, so HB enforces visibility across all block threads without touching L1/L2 coherence hardware.
2. Block-to-block device ordering: A store in block A followed by `__threadfence()` and then a global atomic in block A can be observed by a load in block B if block B performs an acquire atomic or a `__threadfence()` precedes the load.
3. System-level ordering: Use `__threadfence_system()` when the CPU or other devices must see completed GPU stores.

Implementation pattern for portable correctness (CUDA-style) is:

- Use atomic store with release semantics on a separate rendezvous word to publish data.
- Use acquire atomic load on the rendezvous word before reading the data; optionally pair with `__threadfence()` to order to device L2.

Example kernel showing the publish-subscribe pattern:

Listing 4.4: CUDA-like publish/subscribe using fences and atomics

```
__global__ void publish_kernel(int *data, int *flag) {
    int tid = threadIdx.x + blockIdx.x*blockDim.x;
    // producer: write data then publish
    if (tid == 0) {
        data[0] = 42;                // write payload to global memory
        __threadfence();              // ensure device visibility to L2
        atomicExch(flag, 1);          // publish (release) via atomic
    }
    // consumer: spin on flag then read
    if (tid == 1) {
        while (atomicAdd(flag, 0) == 0) { } // acquire via atomic read
        __threadfence();                // optional: ensure L1 sees updates
        int v = data[0];                // now guaranteed to observe 42
        // ... use v ...
    }
}
```

Design implications and trade-offs:

- Using `__threadfence_system` forces coherence to system level, adding latency and DRAM traffic; use only when CPU/device consistency is required.
- Block-level fences and `__syncthreads` are cheapest but limited to an SM's shared-memory domain.
- Architecturally, providing fast release/acquire atomics and scoped fences (block/device/system) enables high-throughput lock-free algorithms while avoiding full cache-line serialization.

- Performance trade-offs: stronger ordering simplifies programming but reduces memory-level parallelism; hardware should optimize common scoped paths (SM-local and device-L2) with fast paths and avoid escalating to system coherence unless needed.

4.2 Barriers and Fences

4.2.1 Block and grid-level barriers

The previous discussion of memory-consistency models and acquire-release semantics set the stage for how barriers constrain visibility and ordering across scopes; block- and grid-level barriers implement those ordering guarantees at different hardware granularities and with very different cost and implementation choices.

Barriers solve the problem of making writes visible and coordinating execution across threads. At block granularity, GPUs provide fast, low-latency synchronization using on-SM resources (shared/LDS, warp masks, per-warp flags), typically exposed as `__syncthreads` or warp-cooperative primitives. Grid-level synchronization requires either a heavyweight host-mediated kernel boundary or specialized global coordination (cooperative launch or hardware global-barrier units) because it must coordinate across SMs/CUs, L1/L2, and DRAM visibility.

Analysis:

- Block barrier semantics: all threads in a block must reach the barrier before any may proceed; it implicitly orders shared memory operations within the block and is very efficient when implemented with per-warp arrival counters and a small aggregation tree in shared memory.
- Grid barrier semantics: all blocks must rendezvous; this forces global memory flushes or atomics at L2 and may require kernel reconfiguration to avoid deadlock.

A simple latency model highlights the dominant terms. Let t_i be arrival time of warp i at the barrier, t_{agg} the aggregation and atomic commit time, and t_{release} the broadcast latency. Then barrier latency is

$$[H]L = \max_i t_i + t_{\text{agg}} + t_{\text{release}}, \quad (4.6)$$

illustrating how skew (differences among t_i) directly increases idle time.

Implementation pattern for fast block-level barrier:

- Each warp asserts a single-bit arrival flag in shared memory.
- A central counter (or tree) detects when the expected number of warps have arrived.
- An epoch bit toggles to prevent late/wrong-phase arrivals from being counted for subsequent barriers.
- Release is a shared-memory broadcast to resume masked SIMT lanes.

Practical Verilog implementation for a block-level warp-aggregation barrier controller follows. It is synthesizable, parameterized for the number of warps, and uses edge-detection to count arrivals once per barrier epoch.

Listing 4.5: Block-level warp aggregation barrier controller (synthesizable).

```

module block_barrier #(
  parameter NUM_WARPS = 8,
  parameter ID_WIDTH  = $clog2(NUM_WARPS)
) (
  input  wire      clk,
  input  wire      rst_n,
  input  wire [NUM_WARPS-1:0] arrive,    // pulse-high one cycle per warp
        arrival
  input  wire      start,    // start a new barrier epoch
  output reg      released   // high when barrier released
);

reg [ID_WIDTH:0] count;          // counts arrivals
reg epoch;                     // toggled each barrier
// Edge-detect arrivals to avoid multi-counting same warp pulse
reg [NUM_WARPS-1:0] arrive_d;
wire [NUM_WARPS-1:0] arrive_pos = arrive & ~arrive_d;

integer i;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    arrive_d <= {NUM_WARPS{1'b0}};
    count    <= 0;
    epoch    <= 1'b0;
    released <= 1'b0;
  end else begin
    arrive_d <= arrive;
    if (start) begin
      count    <= 0;
      released <= 1'b0;
      epoch    <= ~epoch;          // new phase
    end else begin
      // increment count on newly arrived warps
      for (i = 0; i < NUM_WARPS; i = i + 1) begin
        if (arrive_pos[i])
          count <= count + 1'b1;
      end
      // release when all warps have arrived
      if (count == NUM_WARPS[ID_WIDTH:0] - 1)
        released <= 1'b1;
    end
  end
endmodule

```

4.2.2 Memory fence types

Following the block- and grid-level barrier discussion, fences supply finer-grained ordering and visibility guarantees that span caches, write buffers, and interconnect domains. Where barriers coordinate active-thread join points, fences control when memory operations become globally observable across SMs (or to the host), and thus directly affect coherence and latency behavior.

Memory fence types (practical classification and semantics)

- Work-group / block fence: `__threadfence_block` (CUDA) or `groupMemoryBarrier` (GLSL). Scope: shared/LDS and per-SM L1; guarantees that preceding stores are visible to threads in the same work-group without forcing device-wide visibility.
- Device / global fence: `__threadfence`. Scope: device-wide (all SMs/CUs). Forces promotion of stores so other SMs can observe them via the device cache hierarchy (typically L1→L2 visibility).
- System / interconnect fence: `__threadfence_system`. Scope: system-wide including host and other devices; typically requires write buffer drain and coherence transactions to the memory controller or coherency fabric (NVLink, PCIe + host coherency protocols).
- Acquire / release variants: Many GPU APIs and hardware implement acquire (on loads) and release (on stores) semantics to form a happens-before relation without full serialization. A release followed by an acquire from another thread establishes visibility with lower cost than blocking all outstanding memory.

Formal ordering effect A fence composes ordering in the program order into a happens-before relation. If operation A precedes a fence and operation B follows the same (matching-scope) fence, then

$$A \xrightarrow{hb} B \iff (A \rightarrow fence) \wedge (fence \rightarrow B), \quad (4.7)$$

which the hardware implements by ensuring visibility and, if required, completing coherence actions for A before permitting B to proceed.

Hardware mechanisms and cost

- Implementation uses write-combining buffers, L1 and L2 cache state transitions, and coherence/invalidation messages on the on-chip NoC or off-chip interface.
- A device or system fence often forces write-buffer draining and may require cacheline writeback to L2 or DRAM, incurring latency approximately

$$T_{\text{drain}} \approx \frac{W \cdot L}{B}, \quad (4.8)$$

where W is outstanding lines, L is bytes per line, and B is sustained write bandwidth to the target coherence domain.

- Finer-scoped fences (block) can be implemented with local L1/LDS barriers and specialization in SM datapath, preserving throughput for unrelated memory streams.

Practical example (CUDA producer-consumer using device vs system fence)

Listing 4.6: Producer writes and publishes with a device fence; consumer polls from another block.

```
__global__ void producer_consumer(int *buf, int *flag) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx == 0) { // producer thread writes data then publishes
        buf[0] = 42; // store data
        __threadfence(); // ensure device-wide visibility
        atomicExch(flag, 1); // publish (atomic provides ordering)
    } else if (idx == 1) { // consumer polls
        while (atomicAdd(flag, 0) == 0) { /* spin */ } // acquire via atomic
        __threadfence(); // optional: ensure subsequent loads see buf
        int v = buf[0]; // should observe 42
    }
}
```

Design implications and trade-offs

1. Use the minimal-scope fence that satisfies correctness: block fences for intra-SM sync, device fences for inter-SM producer/consumer, and system fences only when host visibility is required.
2. Hardware should provide fast-path local fences in the SM datapath to avoid global coherence traffic for high-frequency synchronization patterns common in ML kernels and graphics tiling.
3. Excessive use of device/system fences reduces SM occupancy and effective throughput because of write-buffer drains and L2/DRAM round-trips; quantify cost using (4.8) when sizing buffers or choosing fence placement.

4.2.3 System-wide synchronization

These ideas build directly on local fence semantics and on-block barriers: system-wide synchronization composes per-SM barriers and memory fences with an interconnect-aware rendezvous so visibility and ordering extend across SMs, CUs, and multiple GPUs. The design must therefore bridge fast on-chip barriers with higher-latency, packetized network rendezvous while preserving the acquire-release and visibility guarantees described previously.

Problem. A system-wide barrier requires all participating domains (SMs, GPUs, host threads) to reach a rendezvous and to observe a consistent post-barrier state. Key constraints are bounded latency, scalability to hundreds or thousands of participants, and tolerance to packet reorder or loss on NVLink/PCIe links. Naive centralized polling yields poor scalability and increases interference with TMU/ROP traffic; decentralized algorithms reduce contention but increase message complexity.

Analysis. A practical microarchitecture uses hierarchical aggregation: local per-SM or per-CU barriers first ensure intra-chip consistency and execute local memory fences. Aggregators then perform a k-ary tree reduction over the interconnect. The latency model for a k-ary tree of N nodes is approximated by

$$[H]T_{\text{sw}} \approx T_{\text{local}} + 2 D T_{\text{hop}}, \quad D = \lceil \log_k N \rceil, \quad (4.9)$$

where T_{local} is local barrier plus fence time, T_{hop} is one network hop latency, and the factor 2 accounts for upward aggregation and downward release flood. Choosing k trades depth D against per-node fanout and buffering requirements.

Implementation. The controller must be resilient to asynchronous arrival and support level-triggered or edge-triggered requests. The following synthesizable Verilog implements a simple central barrier controller that counts asserted per-node requests, issues a release when the count equals the participant count, and waits for request deassertion before re-arming. It is suitable for intra-rack aggregation or as a leaf aggregator in a tree.

Listing 4.7: Central barrier controller (synthesizable)

```

module system_barrier #(
  parameter integer N_NODES = 16
) (
  input wire          clk,
  input wire          rst_n,
  input wire [N_NODES-1:0] req,  // level asserted by each participant
  output reg  [N_NODES-1:0] ack  // ack/release level returned to
    participants
);
  // width to count up to N_NODES
  function integer clog2(input integer v);
    integer i;
    begin
      i = 0;
      while ((1 << i) < v)
        i = i + 1;
      clog2 = i;
    end
  endfunction

  localparam CNT_W = clog2(N_NODES + 1);
  reg [CNT_W-1:0] count;
  reg barrier_reached;

  integer j;
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      count <= 0;
      barrier_reached <= 1'b0;
      ack <= {N_NODES{1'b0}};
    end else begin
      // count how many nodes are requesting
      count = 0;
      for (j = 0; j < N_NODES; j = j + 1)
        if (req[j])
          count = count + 1;
    end
  end

```

```

// when all nodes have asserted req, raise ack for all
if (count == N_NODES) begin
    barrier_reached <= 1'b1;
    ack <= {N_NODES{1'b1}};
end else if (count == 0) begin
    barrier_reached <= 1'b0;
    ack <= {N_NODES{1'b0}};
end
end
end
endmodule

```

4.2.4 Performance overhead

Barriers and fences impose overhead by forcing synchronization points that serialize progress, trigger cache coherence activity, or block warps on SM resources. Key sources of overhead include:

- Warp or CTA stalls: waiting threads remain resident, consuming register and shared memory resources that reduce occupancy and limit useful parallelism.
- Pipeline and load/store drain: stronger fences (e.g., device- or system-wide) may require draining in-flight memory transactions to L2, imposing latency proportional to outstanding memory traffic.
- Cache maintenance: enforcing visibility often triggers L1 writeback or L1 invalidation; cost scales with the number of dirty cache lines.
- Global coordination latency: grid-level rendezvous needs either host intervention or atomic-based counting, which can incur long-tail tail latency when SM scheduling is uneven.

A lightweight cost model helps in design decisions. If a kernel does W useful work between synchronization points and places B barriers, the effective throughput S scales as

$$[H]S = \frac{W}{W + B \cdot T_{\text{bar}}}, \quad (4.10)$$

where T_{bar} is the average barrier cost normalized to the same time unit as W . For a fence that forces cache writeback of D dirty cache lines, an empirical latency model is

$$[H]T_{\text{fence}} \approx \alpha + \beta \cdot D + \gamma \cdot L_{\text{net}}, \quad (4.11)$$

with fixed handshake overhead α , per-line writeback cost β , and network-dependent cross-SM latency L_{net} scaled by γ .

Practical measurement pattern: compare block-local barrier cost against grid-level atomic rendezvous to expose the long tail from SM scheduling imbalance. The following CUDA kernel times a block barrier and a simple grid-level synchronization implemented with an atomic counter. Keep in mind atomic-based grid barriers can deadlock if not all CTAs are resident simultaneously; this code assumes the kernel grid fits on the GPU.

Listing 4.8: Measure block barrier versus grid atomic barrier

```

__global__ void measure_barriers(unsigned long long *out_block, unsigned long
    long *out_grid, int *g_counter) {
    unsigned int tid = threadIdx.x + blockIdx.x * blockDim.x;
    unsigned long long t0, t1;
    // measure block-level barrier cost per thread
    t0 = clock64();
    __syncthreads();                // block-local rendezvous
    t1 = clock64();
    out_block[tid] = t1 - t0;        // store per-thread latency

    // grid-level barrier via atomic counter (assumes all CTAs resident)
    if (threadIdx.x == 0) {
        unsigned long long tg0 = clock64();
        unsigned int arrived = atomicInc(g_counter, gridDim.x); // CTA arrives
        while (*g_counter != gridDim.x) { __nanosleep(0); }      // busy-wait small
        pause
        unsigned long long tg1 = clock64();
        out_grid[blockIdx.x] = tg1 - tg0; // per-CTA measured cost
    }
}

```

Implications and trade-offs:

- Minimize global barriers; prefer block-level or algorithmic reductions to avoid grid-wide rendezvous cost and potential deadlock.
- Use acquire/release fences when possible; they reduce pipeline drain compared to full sequential-consistency fences.
- Hardware provision: stronger fence support (fast L1 writeback, accelerated crossbar) lowers β and γ but increases design complexity and power.
- Software strategy: increase useful work between fences (raise W), reduce barrier frequency B , and structure work to avoid long-tail scheduling imbalance.

4.3 Cache Coherence

4.3.1 Write-invalidate protocols

Following the ordering and fence mechanisms introduced earlier, we now focus on how caches in a GPU maintain coherence when multiple SMs (streaming multiprocessors) concurrently write shared data. The write-invalidate family of protocols is the most common approach in modern GPU L1/L2 hierarchies because it minimizes cache-to-cache data movement under high-write contention while preserving single-writer semantics.

Problem: multiple SMs may hold a cache line simultaneously; a write must make the writer's update visible and prevent stale reads in other SMs. Analysis: write-invalidate enforces

exclusive ownership by invalidating other sharers before completing a modifying write. Typical states implement a MESI-like encoding (Modified, Exclusive, Shared, Invalid). Sequence for a write miss or upgrade:

1. Core requests exclusive ownership at L1/L2 (upgrade request).
2. Directory or home slice broadcasts invalidations to recorded sharers.
3. Sharers acknowledge and transition the line to Invalid.
4. Ownership granted to requester; requester may write and set Modified.

This reduces data-transfer volume relative to write-update (which sends every update to all sharers) when writes are frequent on the same core. Network and latency costs are dominated by invalidation round-trip and directory arbitration.

A simple cost model highlights tradeoffs. Let S be average number of remote sharers, C_{inv} the cost per invalidation message, and C_{upd} the cost per update message. For a sequence of W writes, amortized costs are:

$$[H]C_{\text{inv,per-write}} \approx \frac{C_{\text{bcast}} + S \cdot C_{\text{ack}} + C_{\text{wb}}}{W}, \quad C_{\text{upd,per-write}} \approx S \cdot C_{\text{upd}}. \quad (4.12)$$

Invalidate is preferable when $C_{\text{inv,per-write}} < C_{\text{upd,per-write}}$, typically when W is large (many local writes) or S is moderate.

Implementation: GPUs often use a hybrid directory at L2 that tracks sharers across many SM-local L1s. The directory serializes ownership changes to avoid races with atomic operations (SM atomic units or specialized atomic queues). For atomics, the owner may be forced to write back to L2 to ensure globally serialized ordering visible to other SMs and to accelerator units like tensor cores that bypass L1.

Practical Verilog for a per-line MESI FSM (synthesizable) implementing the core transitions and signaling invalidation/acknowledgement:

Listing 4.9: Per-line MESI coherence FSM (synthesizable).

```

module mesi_line(
  input wire      clk, rst,
  input wire      cpu_read, cpu_write,      // core requests
  input wire      remote_inval,             // incoming invalidation
  output reg      issue_inval_bcast,        // request directory to
    invalidate others
  output reg      grant_exclusive,          // grant to local core
  output reg      writeback_req             // request writeback to L2
);
  // State encoding
  localparam I=2'b00, S=2'b01, E=2'b10, M=2'b11;
  reg [1:0] state, next_state;

  // State transition combinational logic
  always @(*) begin
    issue_inval_bcast = 1'b0;
    grant_exclusive   = 1'b0;
  
```

```

writeback_req      = 1'b0;
next_state = state;
case (state)
  I: begin
    if (cpu_read) next_state = S;           // cold read -> shared
        after fill
    else if (cpu_write) begin
      issue_inval_bcast = 1'b1;           // ask directory for
        exclusive
      // stay in I until grant; grant_exclusive asserted by directory (
        external)
    end
  end
  S: begin
    if (cpu_write) begin
      issue_inval_bcast = 1'b1;           // upgrade to exclusive
      // wait for grant
    end else if (remote_inval) next_state = I; // another core upgraded
      -> invalidate
    end
  E: begin
    if (cpu_write) next_state = M;           // first local write ->
      modified
    else if (remote_inval) begin next_state = I; writeback_req = 1'b1; end
  end
  M: begin
    if (remote_inval) begin
      writeback_req = 1'b1;           // supply dirty data to
      directory/L2
      next_state = I;
    end
  end
  default: next_state = I;
endcase
end

// Sequential update (grant_exclusive asserted externally when directory
// grants)
always @(posedge clk or posedge rst) begin
  if (rst) state <= I;
  else begin
    // simulate grant handshake: directory sets grant_exclusive externally.
    if (grant_exclusive) state <= E;
    else state <= next_state;
  end
end
endmodule

```

Implications and trade-offs:

- Write-invalidate minimizes interconnect bandwidth for frequent local writers; it adds latency for the first modifying write due to invalidation round-trips.
- Directory complexity grows with number of SM sharers; coarse-grain tracking (bitmaps per slice) trades false-sharing for cheaper metadata.
- For GPUs with high atomic contention (e.g., histogram kernels), combining hardware atomics and per-line ownership can reduce coherence churn but may serialize execution on the owning SM.

Designers must balance invalidation latency, directory scalability, and workload write patterns when choosing or tuning write-invalidate behavior.

4.3.2 Directory-based coherence

Building on the write-invalidate discussion, directory-based coherence replaces costly broadcasts with targeted messages by maintaining per-line metadata; this is especially relevant when many SMs (or CUs) and wide SIMT workloads amplify invalidation traffic. The following analyzes directory formats, quantifies overhead, sketches an implementation, and summarizes the practical trade-offs for GPU memory hierarchies.

A directory entry must at minimum record the block tag, a coherence state (e.g., Invalid, Shared, Modified), and which cores hold the block. Two common encodings are:

- Full bitvector: one bit per CU, fast to check and multicast but costs $O(N)$ bits per line for N CUs.
- Sparse pointer list: stores up to P sharer IDs, costing $O(P \log_2 N)$ bits and performing poorly when sharer count $S > P$.

Expressing the worst-case bitvector overhead gives a simple capacity equation. For L cache-lines managed by the directory and N CUs,

$$[H]\text{overhead}_{\text{bits}} = L \cdot N + L \cdot (\text{tag_bits} + \text{state_bits}). \quad (4.13)$$

If the typical sharer count S is small relative to N , a pointer scheme reduces average overhead to approximately $L \cdot S \cdot \log_2 N$ bits, improving area and dynamic power.

Architectural placement: GPUs commonly locate the directory with an L2 slice or at the memory controller. Putting the directory in each L2 slice enables local arbitration for lines resident in that slice, minimizing cross-slice stalling. For atomics and fences, the directory must serialize ownership changes and supply ordered acknowledgments to respect acquire-release semantics used by GPU kernels.

Implementation sketch (synthesizable Verilog) below implements a parametrizable directory that stores tag, state, and a bitvector of sharers. It responds to read and write requests from CUs, issuing invalidation masks for exclusive grants.

Listing 4.10: Simple directory slice: tag, state, sharer bitvector

```
module directory #(
    parameter NUM_CU = 16,
```

```

parameter ADDR_W = 32,
parameter ENTRY_N = 1024
) (
    input wire          clk,
    input wire          reset,
    // request: type 0=Read,1=Write
    input wire          req_valid,
    input wire [1:0]    req_type,
    input wire [ADDR_W-1:0] req_addr,
    input wire [$clog2(NUM_CU)-1:0] req_cu, // requester id
    output reg          resp_valid,
    output reg [1:0]    resp_state,
    output reg [NUM_CU-1:0] resp_sharers
);

// Entry arrays
reg [ADDR_W-1:0] tag_mem [0:ENTRY_N-1];
reg [1:0]        state_mem [0:ENTRY_N-1]; // 0=I,1=S,2=M
reg [NUM_CU-1:0] sharer_mem [0:ENTRY_N-1];

integer i;
// simple direct-mapped index
wire [$clog2(ENTRY_N)-1:0] idx = req_addr[$clog2(ENTRY_N)+5:6]; // example
                             bits

always @(posedge clk) begin
    if (reset) begin
        resp_valid <= 0;
        for (i = 0; i < ENTRY_N; i = i + 1) begin
            tag_mem[i] <= 0;
            state_mem[i] <= 0;
            sharer_mem[i] <= 0;
        end
    end else if (req_valid) begin
        // read current entry
        if (tag_mem[idx] == req_addr) begin
            // hit
            case (req_type)
                2'd0: begin // Read
                    state_mem[idx] <= (state_mem[idx] == 2'd2) ? 2'd1 : state_mem[idx];
                end
                2'd1: begin // Write
                    state_mem[idx] <= 2'd2;
                    sharer_mem[idx] <= (1'b1 << req_cu);
                end
            endcase
        end
    end
end

```

```

    end else begin
        // miss: replace entry
        tag_mem[idx] <= req_addr;
        state_mem[idx] <= (req_type == 2'd1) ? 2'd2 : 2'd1;
        sharer_mem[idx] <= (1'b1 << req_cu);
    end
    // send response
    resp_valid <= 1;
    resp_state <= state_mem[idx];
    resp_sharers <= sharer_mem[idx];
end else begin
    resp_valid <= 0;
end
end
endmodule

```

4.3.3 Cross-core consistency

The previous subsections described directory-based coherence mechanisms and the latency/traffic trade-offs of write-invalidate protocols; here we examine how those mechanisms must enforce cross-core consistency so that SMs observe a usable global memory order for graphics and compute workloads. This subsection frames the problem, derives the necessary ordering invariants, shows an implementation pattern used in GPU kernels, and states concrete hardware trade-offs.

Problem. Multiple SMs (or CUs) with private $L1$ caches and a unified $L2$ must present two guarantees to programmers and runtime systems:

- Per-address coherence: writes to the same cache line must be serialized.
- Cross-core visibility: a write becoming visible on one SM must be visible to another SM after defined synchronization.

Analysis. Coherence supplies per-address serialization; consistency binds that per-address order into program-visible happens-before relations. Formally, coherence requires a total order over writes to each address:

$$[H]\forall x, \forall w_1, w_2 \in \text{Writes}(x), w_1 \neq w_2 \Rightarrow (w_1 \prec_{\text{coh}} w_2 \vee w_2 \prec_{\text{coh}} w_1). \quad (4.14)$$

Acquire-release (or fence) operations connect these per-address orders across addresses: if core A performs a store S with release semantics and core B performs a load L with acquire semantics that reads S, then S happens-before L and all prior stores on A become visible to B.

Implementation pattern. GPUs implement cross-core consistency using a combination of:

1. Directory or directory-cache slices that track owner/dirty bits and issue invalidations to remote $L1$ s.
2. Store buffers and write-combine units at each SM; these may delay visibility until flushed or acknowledged.

3. Memory-fence instructions that force writeback and drain of local buffers, and optionally broadcast a completion (system fence).

Typical programmer pattern for inter-SM signaling uses a release-store plus a system-wide fence on the producer and an acquire-load on the consumer. Example CUDA-like kernel showing the required sequence (producer signals with a flag; consumer waits and then uses a fence to ensure visibility):

Listing 4.11: Inter-SM signaling pattern using atomic flag and system fence

```
__global__ void producer(int *data, int *flag) {
    data[0] = 42;                // store data to global memory
    __threadfence_system();      // flush write buffers to system
    visibility
    atomicExch(flag, 1);         // release-like signal (atomic provides
                                // ordering)
}

__global__ void consumer(int *data, int *flag) {
    while (atomicAdd(flag, 0) == 0) ; // spin until flag set (acquire)
    __threadfence_system();         // ensure subsequent loads see producer
    stores
    int v = data[0];               // now guaranteed to observe 42
}
```

Implications and trade-offs.

- Directory-based invalidation minimizes unnecessary writebacks but incurs directory storage and interconnect latency; write-invalidate lowers coherence traffic but increases invalidation storms for fine-grained sharing.
- Store buffers and write-combining improve throughput for graphics and tensor workloads but require stronger fences to provide cross-core consistency, which are expensive in latency.
- Design choices map to workload sensitivity: ML tensor workloads favor relaxed visibility with atomic reductions, whereas correct rasterization and synchronization-heavy compute kernels demand low-latency fences and fast invalidation paths.

Concrete design implication: to support low-latency cross-core consistency for synchronization-heavy kernels, implement a fast-path directory acknowledgement for fenced writebacks plus small per-SM acknowledgment buffers; trade bandwidth and silicon area for reduced fence latency.

4.3.4 Heterogeneous CPU-GPU models

Building on the directory techniques and cross-core visibility guarantees discussed previously, heterogeneous CPU–GPU coherence adds complication from distinct cache hierarchies, coherence domains, and interconnect semantics. The operational problem is ensuring correct visibility while avoiding excessive invalidation traffic between CPU cores and GPU SMs or CUs, particularly for workloads mixing graphics, ML, and general compute.

Analysis: CPUs typically assume a cache-coherent shared-memory model with fine-grained coherence across L1/L2 caches. GPUs often favor high-throughput, relaxed coherence inside SMs and rely on explicit synchronization or unified virtual memory for CPU interaction. When a directory-based coherence agent mediates both CPU and GPU clients, it must handle:

- asymmetric latencies: GPU memory paths through HBM and on-chip L2 slices differ from CPU memory channels; and
- asymmetric sharing patterns: GPUs issue massively parallel reads from many threads, creating many sharers per line.

Modeling worst-case coherence latency helps set design targets. If T_{req} is request handling, T_{inval} is per-sharer invalidate latency, and N_s is number of sharers, then a conservative bound is

$$[H]T_{\text{total}} \leq T_{\text{req}} + N_s \cdot T_{\text{inval}} + T_{\text{ack}} + T_{\text{mem}} \quad (4.15)$$

This upper bound motivates mechanisms to reduce effective N_s or amortize invalidations.

Implementation options include:

1. Hierarchical directory: place per-SM or per-CU subdirectories to aggregate local sharers, reducing remote invalidations.
2. Coarse-grained coherence: track ownership at tile or page granularity for CPU–GPU transfers to limit metadata size.
3. Hybrid models: allow GPU-local relaxed coherence for intra-GPU sharing and enforce CPU-visible coherence only on fences or explicit mappings.

Below is a synthesizable Verilog directory module sketch that supports CPU and GPU clients with aggregated per-CU sharer vectors. It is deliberately compact but production-ready for integration into a NoC slice. The module maintains states I, S, M and issues invalidation flags; higher-level controllers must implement message transport.

Listing 4.12: Directory module for heterogeneous CPU–GPU clients (synthesizable).

```

module hetero_directory #(
  parameter N_CPU = 4,
  parameter N_GPU = 8,
  parameter LINE_ADDR_W = 32
) (
  input  wire      clk,
  input  wire      rst_n,
  // Request from CPU/GPU (simple arbiter assumed externally)
  input  wire [LINE_ADDR_W-1:0] req_addr,
  input  wire [1:0] req_type, // 0=Read,1=Write,2=ProbeAck
  input  wire      req_valid,
  output reg      req_ready,
  // Outputs: invalidation vector (one-hot per client group)
  output reg [N_CPU-1:0] cpu_inval,
  output reg [N_GPU-1:0] gpu_inval,
  output reg      grant // directory granted ownership

```

```

);

// Directory entry: state and sharer bitmasks
typedef enum reg [1:0] {I=2'b00, S=2'b01, M=2'b10} state_t;
reg [LINE_ADDR_W-1:0] tag;
reg [N_CPU-1:0] cpu_sharers;
reg [N_GPU-1:0] gpu_sharers;
state_t state;

// Simple single-line directory for example; real design uses CAM/RAM.
always @(posedge clk) begin
  if (!rst_n) begin
    tag <= {LINE_ADDR_W{1'b0}};
    cpu_sharers <= {N_CPU{1'b0}};
    gpu_sharers <= {N_GPU{1'b0}};
    state <= I;
    cpu_inval <= {N_CPU{1'b0}};
    gpu_inval <= {N_GPU{1'b0}};
    grant <= 1'b0;
    req_ready <= 1'b1;
  end else begin
    cpu_inval <= {N_CPU{1'b0}};
    gpu_inval <= {N_GPU{1'b0}};
    grant <= 1'b0;
    if (req_valid && req_ready) begin
      if (req_type == 2'd0) begin // Read
        // move to shared, add requester handled externally
        state <= (state==M) ? M : S;
        grant <= 1'b1;
      end else if (req_type == 2'd1) begin // Write/upgrade
        // issue invalidations to all sharers
        cpu_inval <= cpu_sharers;
        gpu_inval <= gpu_sharers;
        // clear sharers, set owner later upon ack
        cpu_sharers <= {N_CPU{1'b0}};
        gpu_sharers <= {N_GPU{1'b0}};
        state <= M;
        grant <= 1'b1;
      end else if (req_type == 2'd2) begin // ProbeAck (sharer ack)
        // caller supplies which sharer cleared via sideband in real system
        // here we assume external updates; no-op for compact example
        grant <= 1'b0;
      end
    end
  end
endmodule

```

Implications and trade-offs:

- Hierarchical directories reduce N_s in (1), lowering T_{total} from Equation (1), but add directory lookup hops.
- Coarse-grain tracking saves metadata but increases false sharing, raising flush frequency.
- Hybrid relaxed models improve GPU throughput but require software or hardware fences for CPU-visible correctness, increasing worst-case latency for synchronization.
- Interconnect choices (NVLink, CXL, PCIe) shape achievable latency and coherency semantics, influencing whether hardware coherence or software-managed consistency is preferable.

4.4 Atomic Operations

4.4.1 Read-modify-write logic

The preceding discussion of memory-consistency primitives and fence costs motivated the need for hardware that serializes critical updates without global stalls. Read-modify-write (RMW) logic provides that serialization point for SMs and CUs, ensuring atomicity across cache hierarchies while minimizing SMT divergence and throughput loss.

Problem: a GPU must support high-rate atomic updates from many threads (warps) to shared global or shared/LDS state without violating acquire-release visibility. Analysis shows the atomic operation is a three-step microtransaction: exclusive read, local modification, and exclusive writeback. We model an atomic RMW as a function

$$[H]_{\text{new}} = f(\text{old}, \text{operand}), \quad (4.16)$$

where f implements add, min, max, bitwise ops, or logical combinations used by graphics accumulators and ML counters.

Operational design points:

- The atomic unit must serialize RMWs to the same cache line or word; serialization points can be per-line locks or tag-based exclusives in L1/L2.
- Latency components are read, compute, writeback, and arbitration. We express latency as

$$[H]L = L_{\text{read}} + L_{\text{proc}} + L_{\text{write}} + L_{\text{arb}}. \quad (4.17)$$

High contention increases L_{arb} nonlinearly.

- Throughput bound for a single hot address approximates

$$[H]_{\text{throughput}} \leq \frac{1}{\max(L, 1/B)} \quad (4.18)$$

where B is sustained memory bandwidth per atomic stream; with small L the arbiter and coherence limit throughput.

Implementation: a small synthesizable atomic unit implements FSM-based RMW with a single-ported exclusive transaction to the cache controller. The unit exposes a lightweight handshake for warp-level atomic requests and supports integer ops used in histogramming, tile aggregations, and optimizer counters in ML workloads.

Listing 4.13: Synthesizable atomic RMW unit for single-word exclusives

```

module atomic_rmw #(
    parameter ADDR_WIDTH = 64,
    parameter DATA_WIDTH = 32
) (
    input wire clk,
    input wire reset,
    // request interface (from SM warp scheduler)
    input wire req_valid,
    input wire [ADDR_WIDTH-1:0] req_addr,
    input wire [DATA_WIDTH-1:0] req_operand,
    input wire [2:0] req_op, // 0=NOP, 1=ADD, 2=AND, 3=OR, 4=MIN
    , 5=MAX
    output reg req_ready,
    // memory/cache port (simple exclusive handshake)
    output reg mem_read,
    output reg mem_write,
    output reg [ADDR_WIDTH-1:0] mem_addr,
    input wire [DATA_WIDTH-1:0] mem_rdata,
    output reg [DATA_WIDTH-1:0] mem_wdata,
    input wire mem_ready,
    // response to caller
    output reg resp_valid,
    output reg [DATA_WIDTH-1:0] resp_old
);

typedef enum reg [2:0] {IDLE, READ, COMPUTE, WRITEBACK, RESP} state_t;
state_t state, next_state;
reg [DATA_WIDTH-1:0] oldv, newv;
// FSM
always @(posedge clk) begin
    if (reset) state <= IDLE;
    else state <= next_state;
end
always @(*) begin
    // default outputs
    req_ready = 1'b0; mem_read = 0; mem_write = 0; mem_addr = req_addr;
    mem_wdata = newv; resp_valid = 0; resp_old = oldv; next_state = state;
    case (state)
        IDLE: begin
            req_ready = 1; if (req_valid) next_state = READ;
        end
        READ: begin

```

```

    mem_read = 1;
    if (mem_ready) next_state = COMPUTE;
end
COMPUTE: begin
    oldv = mem_rdata;
    // compute newv combinationally
    case (req_op)
        3'd1: newv = oldv + req_operand; // ADD
        3'd2: newv = oldv & req_operand; // AND
        3'd3: newv = oldv | req_operand; // OR
        3'd4: newv = (oldv < req_operand) ? oldv : req_operand; // MIN
            unsigned
        3'd5: newv = (oldv > req_operand) ? oldv : req_operand; // MAX
            unsigned
        default: newv = req_operand;
    endcase
    next_state = WRITEBACK;
end
WRITEBACK: begin
    mem_write = 1;
    if (mem_ready) next_state = RESP;
end
RESP: begin
    resp_valid = 1; resp_old = oldv; next_state = IDLE;
end
endcase
end
endmodule

```

Implications and trade-offs:

1. Per-line exclusion minimizes races but increases latencies for contended addresses; hardware should favor narrow locks for high parallelism.
2. Combining hardware or eliding writebacks for commutative ops (e.g., per-SM buffers) can raise throughput for histogram and reduction patterns.
3. Coherence integration with L1/L2 affects scalability: directory-based exclusives reduce false serialization at cross-slice accesses.

4.4.2 Compare-and-swap

The previous discussion of generic read-modify-write primitives established the hardware role of an atomic unit and the need for ordered visibility across caches. Compare-and-swap (CAS) implements a specific RMW pattern used ubiquitously in GPU kernels for lock-free coordination and histogram updates.

CAS performs an atomic conditional update: a location is read, compared to an expected value, and written with a new value only if the comparison succeeds. Operational relevance first: CAS is the building block for mutex-free queue heads, per-bin histogram increments in

graphics/compute shaders, and work-stealing counters in persistent-thread dispatch. On GPUs, CAS may be accelerated in the SM atomic unit for shared memory, or in the L2/atomic engine for global DRAM-scoped updates; the choice affects latency and contention behavior.

Formally, for a memory location with current value v , expected value e , and desired value d , CAS succeeds and sets the location to d if and only if $v = e$. The functional outcome per invocation is:

$$[H]\text{CAS}(\text{addr}, e, d) : \begin{cases} \text{return } (v, \text{success} = 1) & \text{if } v = e \text{ and set } v \leftarrow d, \\ \text{return } (v, \text{success} = 0) & \text{otherwise.} \end{cases} \quad (4.19)$$

Under contention among k identical contenders issuing CAS on the same 32-bit word, a simple probabilistic model yields a per-attempt success probability $p \approx 1/k$, and thus an expected attempt count $E[A] = 1/p \approx k$. The expected latency is roughly

$$[H]E[L] \approx E[A] \cdot T_{\text{atom}}, \quad T_{\text{atom}} = \text{one atomic round-trip latency.} \quad (4.20)$$

This quantifies why hot atomic hotspots degrade throughput on SIMT hardware: many lanes stall retrying, reducing useful compute occupancy.

Implementation: a synthesizable CAS finite-state machine below targets a single-port shared-memory atomics interface. It serializes requests and guarantees atomicity by servicing one transaction at a time; in a real GPU this FSM maps into the atomic execution unit with an address-based coalescing/lock table.

Listing 4.14: Synchronous single-word CAS atomic unit (production-ready FSM).

```

module cas_unit (
    input  wire      clk,
    input  wire      rst_n,
    input  wire      req_valid,           // request valid
    input  wire [31:0] req_addr,         // byte address
    input  wire [31:0] req_expected,     // expected value
    input  wire [31:0] req_desired,     // desired value
    output reg      req_ready,           // accept next request
    output reg      rsp_valid,
    output reg [31:0] rsp_old,           // returned old value
    output reg      rsp_success          // 1 if swap performed
);
    // simple memory model: synchronous single-ported RAM (externalized in real
    // design)
    reg [31:0] mem [0:1023]; // example depth; replace with real memory
    interface

    typedef enum reg [1:0] {IDLE=0, READ=1, COMPARE=2, WRITE=3} state_t;
    state_t state, next;

    reg [31:0] saved_addr;
    reg [31:0] saved_expected, saved_desired;
    reg [31:0] read_value;

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE; req_ready <= 1'b1; rsp_valid <= 1'b0;
    end else begin
        state <= next;
        if (state==IDLE && req_valid && req_ready) begin
            saved_addr <= req_addr[11:2]; // word index
            saved_expected <= req_expected;
            saved_desired <= req_desired;
        end
        if (state==READ) read_value <= mem[saved_addr];
        if (state==WRITE) mem[saved_addr] <= saved_desired;
        if (state==COMPARE) begin
            rsp_old <= read_value;
            if (read_value==saved_expected) rsp_success <= 1'b1; else rsp_success
                <= 1'b0;
            rsp_valid <= 1'b1;
        end
        if (state==IDLE) rsp_valid <= 1'b0;
        // flow control
        req_ready <= (state==IDLE);
    end
end

always @(*) begin
    next = state;
    case (state)
        IDLE:    if (req_valid && req_ready) next = READ;
        READ:    next = COMPARE;
        COMPARE: if (rsp_success) next = WRITE; else next = IDLE;
        WRITE:   next = IDLE;
    endcase
end
endmodule

```

Design implications and trade-offs:

- Place CAS support in SM shared memory to minimize T_{atom} for intra-block synchronization; global CAS in L2 yields higher latency but broader visibility.
- Under high contention, CAS causes repeated serialization and wasted memory bandwidth; alternatives include fetch-and-add for counters and work-partitioning to reduce contention.
- Hardware lock tables or address hashing can allow multiple non-conflicting CASes to proceed concurrently, increasing throughput at modest area cost.

4.4.3 Arbitration circuits

Arbitration design problem: multiple warps or threads produce atomic requests that converge on a single coherence granule or bank. The arbiter must:

1. prevent data races by granting exclusive access for the read-modify-write sequence;
2. provide sufficient fairness to avoid warp starvation under SIMT; and
3. minimize latency to preserve throughput for latency-sensitive kernels like fine-grained histograms or sparse-matrix updates.

Typical placement points are:

- Per-shared-memory bank arbiter inside an SM, reducing bank conflicts before the shared-memory RMW unit.
- Per-L2-slice arbiter that serializes atomic cache-line owners in the coherence directory.
- Memory-controller slice arbiter that schedules atomic DRAM-mapped RMW sequences.

Common arbitration policies and tradeoffs:

- Fixed priority: low-latency for high-priority streams, but can starve others.
- Round-robin: bounded starvation and good fairness; simple hardware and low area cost.
- Age-based or weighted fair queuing: better QoS for mixed workloads but higher complexity and state.

A simple latency bound for single-port serialization is:

$$[H]L_{\max} = (N - 1)T_s \quad (4.21)$$

where N is the number of contenders and T_s is the atomic service time. This worst-case is achieved when all N requests queue behind a single holder.

Implementation example: a synthesizable parameterized round-robin arbiter suitable for per-bank or per-cache-line arbitration. It grants one request at a time, advances the grant pointer on acknowledgement, and wraps cleanly for any number of requestors.

Listing 4.15: Parameterizable round-robin arbiter for atomic request serialization

```

module rr_arbiter #(parameter NREQ = 8) (
  input  wire      clk,
  input  wire      reset,      // synchronous reset
  input  wire [NREQ-1:0] req,  // request mask from requestors
  output reg [NREQ-1:0] grant, // one-hot grant
  output reg      valid,      // grant valid
  input  wire      ack        // service accepted the grant
);
// function to compute pointer width
function integer clog2;
  input integer v;

```

```

begin
    clog2 = 0;
    while ((1 << clog2) < v)
        clog2 = clog2 + 1;
    end
endfunction

localparam PTR_W = clog2(NREQ);
reg [PTR_W-1:0] ptr; // round-robin pointer
integer i;

always @(posedge clk) begin
    if (reset) begin
        ptr    <= 0;
        grant <= 0;
        valid <= 0;
    end else begin
        grant <= 0;
        valid <= 0;
        // scan requests starting from current pointer
        for (i = 0; i < NREQ; i = i + 1) begin
            if (req[(ptr + i) % NREQ] && !valid) begin
                grant[(ptr + i) % NREQ] <= 1'b1;
                valid <= 1'b1;
            end
        end
        // advance pointer on acknowledgment
        if (ack && valid)
            ptr <= ptr + 1'b1;
    end
end
endmodule

```

4.4.4 Performance optimizations

The previous discussion of arbitration circuits and the hardware implementation of compare-and-swap (CAS) established the low-level building blocks for serialized access and conditional update. Those mechanisms directly determine how read–modify–write primitives are exposed to SMs and how CAS anchors lock-free algorithms at the microarchitectural level.

Atomic operations solve the problem of correct concurrent updates from many SIMT threads to shared locations, but they create contention hotspots that interact with cache and interconnect arbitration. Operationally, GPUs provide a family of atomics (add, min, max, exchange, CAS) at multiple scopes: per-SM shared memory (banked, low-latency), L1 resident cache lines (subject to cache coherence policy), and L2/global DRAM (long-latency, often hardware-combined). The atomic unit in an SM performs three functions: serialize per-address RMW sequences, forward results to requesting warps, and arbitrate between local and cross-SM requests using a small per-line queue and an arbiter similar to the one in the prior subsection.

CAS is implemented as an atomic RMW sequence that compares a loaded value and conditionally writes, requiring the atomic unit to support read-modify-write completion semantics.

To reason about performance impact under contention, use a simple queuing model where n threads concurrently target the same cache line. Let L_0 be the single-operation latency and γ the incremental delay per queued request; a first-order approximation is:

$$[H]L(n) = L_0 + (n - 1) \gamma. \quad (4.22)$$

This linear model captures increased serialization cost at the atomic unit and higher L2 round trips when escalation to DRAM occurs.

Implementation techniques to reduce effective contention:

1. Warp-aggregated atomics: sum per-warp and submit one atomic per warp.
2. Hardware combining at L2: coalesce multiple RMWs to the same address into one update.
3. Backoff and randomized retry: reduce livelock under heavy contention.

Example: warp-aggregated global atomic add in CUDA reduces DRAM/L2 pressure by one atomic per warp leader.

Listing 4.16: Warp-aggregated atomicAdd to global counter

```
__device__ void warp_agg_atomic_add(int *g_counter, int val) {
    unsigned mask = __ballot_sync(0xFFFFFFFF, true);           // which lanes
        active
    int lane = threadIdx.x & 31;                                // lane id
    int warp_sum = __shfl_sync(mask, val, 0);                   // gather values (
        example)
    // leader lane performs one atomic add (lane==0).
    if (lane == 0) atomicAdd(g_counter, warp_sum);              // one atomic per
        warp
    // other lanes continue; minimal serialization.
}
```

Design implications and trade-offs:

- Hardware combining reduces memory traffic but increases L2 complexity and may add latency for opportunistic batching.
- Shared memory atomics are fastest but require bank-aware allocation to avoid intra-warp bank conflicts.
- Providing stronger ordering (sequential consistency) simplifies software but imposes global fences and degrades throughput; acquire-release semantics are a practical compromise.
- Software patterns (warp aggregation, privatization, reduction trees) are often necessary to achieve throughput close to arithmetic units (tensor cores/ALUs) when atomics would otherwise dominate memory-bound kernels.

4.5 Lock-Free Algorithms

4.5.1 Wait-free synchronization

Following the prior discussion of lock-free primitives and ABA hazards, we now focus on strict wait-free synchronization and its operational role within SIMT GPUs. Wait-free designs guarantee each thread completes its operation in a bounded number of steps, which is crucial for SM/CU hardware that cannot block warps without severe throughput loss.

Problem: provide bounded-latency allocation or publish semantics across many threads without causing warp-level stalls or livelock. **Analysis:** on GPUs, hardware atomic read-modify-write primitives (for example atomic fetch-and-add or atomic exchange implemented in the memory subsystem) offer linearizable updates; treating these primitives as constant-time building blocks lets software achieve wait-freedom for specific operations. Formally, an operation is wait-free if there exists a constant c such that any thread completes in at most c local steps regardless of other threads. We can express a per-request bound as

$$[H]\forall \text{ thread } i, \text{ latency}_i \leq c \quad (\text{cycles}), \quad (4.23)$$

where c is the maximum pipeline latency of the atomic unit plus fixed enqueue/dequeue book-keeping.

Implementation example: a global slot allocator implemented as a centralized atomic fetch-and-increment returns a unique index to each producer; producers use the index to write a pre-assigned slot in a bounded number of steps, then issue a release fence. Consumers read slots in index order. This pattern avoids unbounded CAS retries because each producer performs one atomic capture and then proceeds without further coordination.

Practical constraints on GPUs:

- Hardware atomics are exposed via SM-internal atomic units and are fast, but contention increases memory subsystem pressure and can serialize through L2 or atomic arbiter.
- Wait-freedom often requires more memory (preallocated per-thread slots) to avoid contention or dynamic retries, increasing storage and bandwidth.
- In SIMT execution, a warp issuing atomics on many threads simultaneously will create a burst load on the atomic unit; scheduling warps to stagger atomic bursts reduces queuing.

A simple synthesizable Verilog atomic-fetch-increment unit suitable for on-chip integration is below; it provides single-cycle handshake semantics and a bounded-service property (value returned after at most one arbitration cycle).

Listing 4.17: Simple atomic fetch-and-increment unit (synthesizable).

```

module atomic_fai (
    input  wire      clk,
    input  wire      rst_n,
    input  wire      req,          // request pulse (one-hot per client should
    be arbitered externally)
    output reg      ack,          // pulse when value is valid
    output reg [31:0] value      // returned prior counter value
);
    reg [31:0] counter;

```

```

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    counter <= 32'd0;
    ack      <= 1'b0;
    value    <= 32'd0;
  end else
    ack <= 1'b0; // default deassert
    if (req) begin
      value <= counter; // capture prior value
      counter <= counter + 1; // increment for next client
      ack <= 1'b1; // indicate completion next cycle
    end
  end
endmodule

```

Design implications and trade-offs:

- Wait-free slot allocation yields predictable latency (good for real-time compute kernels and ML scheduling), but raises bandwidth and storage costs due to preallocated slots.
- Relying on a centralized atomic unit simplifies correctness but can become a hot spot; distributed per-SM allocators or hierarchical atomic trees reduce contention at the cost of increased complexity.
- For high-throughput graphics or ML workloads, choose wait-free patterns where bounded latency outweighs resource overhead, and combine with warp-level scheduling to smooth atomic bursts.

4.5.2 ABA problem handling

The preceding discussion of wait-free synchronization emphasized progress guarantees for individual threads; the ABA problem breaks those guarantees when compare-and-swap (CAS) observes a recycled value indistinguishable from the original. Here we analyze why ABA arises on GPUs, provide robust mitigation patterns, and give an implementation example appropriate for SIMT execution and high-thread-count contexts.

Problem and analysis.

- ABA occurs when a CAS-based algorithm relies on identity of a single machine word. Thread T1 reads head=A, later attempts CAS(head, A, A.next). Meanwhile threads T2 and T3 pop A, free it, allocate a new node reusing the same address, and push it back. CAS in T1 succeeds erroneously because the raw value equals A, but state changed semantically.
- On GPUs the hazard is amplified by:
 1. Massive concurrency: thousands of SM threads increase reuse probability.
 2. Fast allocator reuse: small-object allocators can quickly recycle addresses.
 3. Atomic unit contention: repeated CAS retries amplify latency across warps.

- Effective mitigations must prevent indistinguishable reuse, or delay reclamation until no thread can hold a stale reference.

Practical mitigation techniques.

1. Tagged (versioned) pointers: widen the atomic word to include a monotonically incrementing tag. Pack pointer p and tag t into a single 64-bit word and perform atomic CAS on that word. If pointer alignment provides k low zero bits, shift tag by k .

$$[H]\text{packed} = \text{uintptr_t}(p) \mid (\text{uint64_t}(t) \ll k) \quad (4.24)$$

The tag prevents ABA until the counter wraps; choose tag bits so wrap probability during object lifetime is negligible.

2. Double-width CAS (DCAS): if hardware supports atomic 128-bit CAS, store (pointer, counter) naturally. Few GPUs expose DCAS to device code.
3. Epoch-based reclamation: defer free until all threads have advanced past the epoch. This trades memory for correctness and maps well to GPU work-group lifetimes.
4. Hazard pointers: threads publish live pointers to prevent reclamation. On GPUs, per-thread reader lists cause space and synchronization overhead, but can be optimized with warp-cooperative hazard sets.

Implementation example.

Listing 4.18: CUDA device push/pop using a 64-bit tagged pointer and atomicCAS

```
struct Node { Node* next; int payload; }; // node layout
// pack lower bits of ptr and upper bits tag
__device__ inline unsigned long long pack(Node* p, unsigned short tag){
    return (unsigned long long)(uintptr_t)p | ((unsigned long long)tag << 48);
    // 48/16 split
}
__device__ inline void unpack(unsigned long long v, Node*& p, unsigned short&
tag){
    p = (Node*)(uintptr_t)(v & ((1ULL<<48)-1)); tag = (unsigned short)(v >> 48);
}
__device__ unsigned long long head; // global stack
head
__device__ void push(Node* n){
    unsigned long long oldv, newv;
    unsigned short oldtag;
    do{
        oldv = atomicAdd(&head, 0ULL); // read current
        head atomically
        unpack(oldv, n->next, oldtag); // reuse old tag
        newv = pack(n, (unsigned short)(oldtag+1)); // increment tag
    } while(atomicCAS(&head, oldv, newv) != oldv); // retry on
        concurrent modification
    }
}
```

```

__device__ Node* pop(){
    unsigned long long oldv, newv;
    Node* p; unsigned short tag;
    do{
        oldv = atomicAdd(&head, 0ULL);
        unpack(oldv, p, tag);
        if(!p) return nullptr;
        newv = pack(p->next, (unsigned short)(tag+1));
    } while(atomicCAS(&head, oldv, newv) != oldv);
    return p;
}

```

Design implications and trade-offs.

- Tagged pointers require atomics wide enough to hold pointer+tag; choose bit split to avoid counter wrap within object lifetime.
- Epoch reclamation reduces pointer-tag overhead but increases peak memory and requires global quiescence tracking per SM or work-group.
- Hazard pointers are precise but costly in per-thread storage on GPUs; prefer warp-cooperative variants.
- Performance: adding a tag increases CAS success checks but dramatically reduces ABA-induced correctness failures; contention reduction (per-SM freelists or batching) offsets retry costs.

4.5.3 Lock-free queues

The ABA mitigation techniques and the distinction between lock-free and wait-free progress we just covered directly inform practical queue designs: sequence tags (or wider counters) that prevent ABA are the standard cure, while true wait-freedom is rarely achievable for general multi-producer/multi-consumer (MPMC) queues on SIMT hardware. Lock-free queues aim for system-wide progress with minimal stalls, and on GPUs practical designs combine per-warp batching and sequence-tagged ring buffers to avoid contention and ABA races.

Problem — support many concurrent enqueueers and dequeuers from warps executing on multiple SMs with minimal global synchronization overhead. Analysis — use a bounded circular buffer with monotonically increasing 64-bit head and tail counters; map counters to slots by modulo capacity so

$$[H]\text{slot} = (\text{counter}) \bmod C, \quad (4.25)$$

where C is capacity. Each slot holds (1) payload, (2) a sequence number that encodes the expected counter value for safe ownership. Sequence numbers avoid ABA because the counter never wraps within practical lifetime if 64-bit counters are used.

Implementation — Dmitry Vyukov's bounded MPMC queue fits GPU constraints: producers reserve an index via `atomicAdd` on tail; they wait (spin) until the slot's sequence equals the reserved index, write payload, then publish by setting the slot sequence to `index+1` (a release). Consumers similarly reserve head via `atomicAdd`, spin until sequence matches `index+1`, read payload, then set sequence to `index+C` (making slot reusable). Use device-scoped fences to ensure memory ordering of payload stores before publishing the sequence.

Example device-side CUDA implementation:

Listing 4.19: Bounded MPMC lock-free queue (device functions)

```
struct Slot { unsigned long long seq; int data; }; // simple payload
struct MPMCQ {
    unsigned long long C; // capacity
    unsigned long long *tail, *head; // global counters (64-bit)
    Slot *slots; // array of size C in global memory
};

__device__ void enqueue(MPMCQ *q, int value) {
    unsigned long long idx = atomicAdd(q->tail, 1ULL); // reserve slot
    unsigned long long pos = idx % q->C;
    // spin until slot.seq == idx (producer ownership)
    while (atomicLoad(&(q->slots[pos].seq)) != idx) { /* backoff or yield */ }
    q->slots[pos].data = value; // write payload
    __threadfence(); // ensure visibility before
        publish
    atomicExch(&(q->slots[pos].seq), idx+1); // publish (release)
}

__device__ bool dequeue(MPMCQ *q, int *out) {
    unsigned long long idx = atomicAdd(q->head, 1ULL); // reserve item
    unsigned long long pos = idx % q->C;
    while (atomicLoad(&(q->slots[pos].seq)) != idx+1) { /* backoff or yield */ }
    *out = q->slots[pos].data; // read payload
    __threadfence(); // ensure read before marking
        free
    atomicExch(&(q->slots[pos].seq), idx + q->C); // mark slot free
    return true;
}
```

Operational notes and trade-offs:

- Use 64-bit counters to make wrap-around (and thus ABA) virtually impossible during runtime; sequence tags implement ABA protection explicitly.
- Employ per-warp enqueue batching (aggregate multiple payloads into one atomicAdd) to amortize contention on `tail` and `head`.
- Memory fences: use block-level fences for shared-memory subqueues and system-level fences when global visibility is required; omitted fences break ordering of payload vs sequence updates.

Performance trade-offs:

1. Lower contention with per-SM subqueues reduces atomic pressure but increases memory movement complexity.
2. Spinning wastes SIMT lanes; prefer exponential backoff or cooperative yielding to hide latency.

3. Bounded queues need capacity tuning—too small increases spinning, too large raises memory footprint and cache pressure (L1/L2).

4.5.4 GPU-specific design considerations

The lock-free queue and ABA subsections established practical techniques (tagged pointers, epoch counters) and failure modes that arise from RMW primitives under heavy contention on GPUs. This subsection now focuses on GPU-specific constraints — SIMT execution, memory hierarchy, and atomic-unit throughput — and how they affect algorithm design and implementation.

GPU platforms expose a few architectural realities that change lock-free choices:

- SM-level SIMT execution creates groups of threads (warps) that issue atomics in bursts. Contention spikes when many warps target a single atomic location, increasing latency and serialization at the atomic unit.
- Shared/LDS memory provides very low-latency RMW opportunities, but is banked; bank conflicts turn fast atomic-like sequences into serialized accesses.
- Device atomics on global memory pass through L1/L2 and sometimes an atomic accelerator; throughput is bounded by that hardware pipeline, regardless of the number of threads issuing requests.

Problem → analysis: to keep ABA risk and contention manageable, combine pointer tagging with backoff and locality-aware mapping.

- Tag sizing: reserve t bits of the CAS tag. With $M = 2^t$ tag space and Q RMW events over the application lifetime, the birthday-collision approximation gives the collision probability

$$[H]p_{\text{collide}} \approx 1 - \exp\left(-\frac{Q^2}{2M}\right). \quad (4.26)$$

Choose t so that p_{collide} is negligible over expected Q ; on 64-bit CAS, a 16–32 bit tag is often sufficient.

Implementation notes and optimizations:

1. Use 64-bit atomicCAS to pack pointer and tag into one word; on NVIDIA/AMD devices 64-bit CAS is commonly supported. Keep pointer alignment to avoid pointer-bit aliasing.
2. Perform most short-lived, high-frequency exchanges in shared memory per-block; only spill to global atomics when crossing block boundaries. This reduces load on global atomic units and leverages low-latency banks.
3. Reduce warp divergence: serialize only one lane per warp to perform the atomic, using `__shfl_sync` or ballot primitives to elect the lane. This prevents many redundant atomic attempts and exploits warp coalescing.
4. Exponential/packetized backoff mitigates thundering-herd behavior; tune backoff steps to match atomic-latency tail observed in hardware counters.

Practical CUDA implementation example (packed pointer+tag CAS push/pop, single-thread per warp performs CAS):

Listing 4.20: Device push/pop using 64-bit tagged CAS; one warp lane performs the CAS to reduce contention.

```
__device__ unsigned long long pack(void* ptr, unsigned int tag){
    return ( (unsigned long long)tag << 48 ) | ((unsigned long long)ptr & 0
        x0000FFFFFFFFFULL);
}
__device__ bool cas_head(unsigned long long* head, unsigned long long oldv,
    unsigned long long newv){
    return atomicCAS(head, oldv, newv) == oldv; // atomicCAS on 64-bit
}
// caller ensures only one lane per warp calls cas_head at a time.
```

Design implications and trade-offs:

- Larger tag widths reduce ABA probability but consume CAS bits; with 64-bit CAS, reserve at least 16 bits for tags to reach $p_{\text{collide}} \ll 10^{-9}$ for large Q .
- Favor shared-memory-first designs to avoid global-atomic bottlenecks; accept complexity of local aggregation and occasional global flushes.
- Electing a single warp lane to issue atomics trades a few cycles of warp-serial work for a large reduction in contention and improved throughput.
- Backoff reduces tail latency at the cost of higher average completion time under low contention; tune per workload using hardware counters.

4.6 Section 6: Verilog Implementation

4.6.1 Barrier synchronization module

Following the previous discussion of fence semantics and participant visibility, the Verilog implementation below provides a concrete, synthesizable barrier module that enforces a group rendezvous for a fixed number of participants (for example, threads or warps mapped to an SM). The design emphasizes correctness under common GPU constraints and presents trade-offs for throughput and scalability.

Problem statement and analysis:

- Requirement: all N participants must wait until every participant has arrived, then all are released simultaneously without counting duplicate arrivals.
- Constraints: implementable in RTL, synthesizable, small area, support reuse of the same barrier by successive generations.
- Chosen approach: per-barrier generation counter plus an arrival bit-vector and a counter. When the counter reaches N , the module increments the generation, clears the bit-vector, and issues a one-cycle barrier_pulse and an updated generation value. Threads compare the observed generation to a locally stored value to detect release.

Key parameter math (ID width):

$$[H]ID_WIDTH = \lceil \log_2(N_PARTICIPANTS) \rceil \quad (4.27)$$

Verilog implementation (synthesizable):

Listing 4.21: Barrier synchronization module for fixed N participants

```

module barrier_sync #(
    parameter N_PARTICIPANTS = 32,
    parameter GEN_WIDTH = 8
) (
    input wire clk,
    input wire rst_n,          // active low reset
    input wire arrive,        // single-bit per-cycle
                               arrival_strobe
    input wire [$clog2(N_PARTICIPANTS)-1:0] id,    // participant id
    output reg [GEN_WIDTH-1:0] gen_out,           // generation broadcast
    output reg barrier_pulse // one-cycle pulse when
                               barrier completes
);

    localparam ID_WIDTH = $clog2(N_PARTICIPANTS);
    localparam CNT_WIDTH = $clog2(N_PARTICIPANTS+1);

    reg [GEN_WIDTH-1:0] gen;
    reg [N_PARTICIPANTS-1:0] arrived;
    reg [CNT_WIDTH-1:0] count;

    integer i;
    // reset logic
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            gen <= {GEN_WIDTH{1'b0}};
            arrived <= {N_PARTICIPANTS{1'b0}};
            count <= {CNT_WIDTH{1'b0}};
            gen_out <= {GEN_WIDTH{1'b0}};
            barrier_pulse <= 1'b0;
        end else begin
            barrier_pulse <= 1'b0; // default clear
            if (arrive) begin
                // only count first arrival per participant for current
                generation
                if (!arrived[id]) begin
                    arrived[id] <= 1'b1;
                    count <= count + 1'b1;
                end
            end
            // check completion
            if (count == N_PARTICIPANTS - 1 && (arrive && !arrived[id] ||
                count == N_PARTICIPANTS - 1)) begin

```

```

        // barrier reached: increment generation, clear arrived, reset
        count
        gen <= gen + 1'b1;
        gen_out <= gen + 1'b1; // broadcast next gen as release
        indicator
        arrived <= {N_PARTICIPANTS{1'b0}};
        count <= {CNT_WIDTH{1'b0}};
        barrier_pulse <= 1'b1; // one-cycle global pulse
    end
end
end
endmodule

```

Operational notes and assumptions:

- The interface assumes at most one arrival assertion per cycle; real GPU front-ends should serialize warp arrival signals or add an input arbiter to tolerate multi-arrival cycles.
- Threads must sample `gen_out` or `barrier_pulse` atomically to detect release; a local compare of stored generation provides robust detection.

Design implications and trade-offs:

- Scalability: bit-vector size grows linearly with participant count, increasing area; for large work-groups use hierarchical barriers (tree of localized barriers) to reduce fan-in.
- Throughput: single-cycle global pulse keeps release latency low, but simultaneous multiple arrivals require upstream arbitration, adding cycles.
- Correctness vs. complexity: generation counter avoids ABA races on reuse; increasing `GEN_WIDTH` reduces wrap-around risk but costs flip-flops.
- For multi-SM or grid-level synchronization, replace the shared bit-vector with a distributed reduction network or atomic increment against a centralized counter to avoid large cross-domain registers.

4.6.2 Atomic operation unit

Following the barrier synchronization design, which enforces group-wide quiescence and ordering, the atomic operation unit provides fine-grained read-modify-write (RMW) primitives and compare-and-swap (CAS) semantics needed to implement lock-free algorithms and GPU atomics visible across SMs. This subsection analyzes the functional requirements, presents a synthesizable Verilog implementation suitable for integration with an SM's load-store pipeline, and states concrete design trade-offs for throughput and latency.

Problem and analysis:

- Requirements: support RMW ops (add, min/max, and, or, xor), CAS, and ordered completion within a visibility scope; tolerate many concurrent SIMT threads issuing atomics; avoid head-of-line blocking across independent addresses.

- Approach: serialize operations per-address while allowing parallelism across addresses. Use a small associative reservation table to detect conflicts, plus an input FIFO to absorb bursty bursts from the warp scheduler. For global memory atomics that must go to L2/DRAM, issue atomic memory requests; for shared/LDS, target local banks directly for minimal latency.

Key performance model:

- Let t_{mem} be the round-trip memory latency in cycles for a single atomic to global memory, and let s be the average number of pending serialized operations for the same cache line. Then the worst-case completion latency per-operation is

$$[H]L_{\text{worst}} = t_{\text{mem}} + s \cdot t_{\text{proc}}, \quad (4.28)$$

where t_{proc} is the internal processing cycle budget per atomic. Throughput is bounded by the larger of memory bandwidth and serialization rate.

Implementation overview:

- Input interface: valid-ready handshaking from the SM (per-warp aggregator can coalesce).
- Reservation table: small associative array (parameterizable depth) that holds active addresses and accumulators for pending RMWs to coalesce commutative ops.
- FIFO queue: depth to absorb bursts and implement backpressure to the warp scheduler.
- Memory interface: single outstanding `mem_req` channel assumed; can be extended to multiple channels for higher parallelism.

Synthesis-ready Verilog implementing a parameterizable atomic unit. It implements a FIFO, an 8-entry reservation table with linear search arbitration, simple RMW ops, and CAS. Replace the memory interface with the real L1/L2 interconnect in the SoC.

Listing 4.22: Atomic operation unit (synthesizable) with per-address reservation table

```

module atomic_unit #(
  parameter ADDR_W = 40,
  parameter DATA_W = 64,
  parameter TAG_W = 8,
  parameter RESV_N = 8,
  parameter FIFO_D = 16
) (
  input wire          clk,
  input wire          rst,
  // request: valid/ready, single-beat
  input wire          req_valid,
  output reg          req_ready,
  input wire [ADDR_W-1:0] req_addr,
  input wire [2:0]      req_op,    // 0=read,1=add,2=and,3=or,4=xor
                                ,5=cas
  input wire [DATA_W-1:0] req_data, // for RMW or CAS (expected)

```

```

input wire [TAG_W-1:0]      req_tag,
// response
output reg                 resp_valid,
output reg [DATA_W-1:0]    resp_data,
output reg [TAG_W-1:0]     resp_tag,
// memory interface (simple request/response)
output reg                 mem_req_valid,
input wire                 mem_req_ready,
output reg [ADDR_W-1:0]    mem_req_addr,
output reg [DATA_W-1:0]    mem_req_wdata,
output reg [2:0]           mem_req_op, // 0=read,1=write,2=atomic
input wire                 mem_resp_valid,
input wire [DATA_W-1:0]    mem_resp_rdata
);

// Simple FIFO for incoming requests (circular buffer)
reg [ADDR_W-1:0] fifo_addr [0:FIFO_D-1];
reg [DATA_W-1:0] fifo_data [0:FIFO_D-1];
reg [2:0]         fifo_op   [0:FIFO_D-1];
reg [TAG_W-1:0]   fifo_tag  [0:FIFO_D-1];
reg [4:0]         fifo_wr_ptr, fifo_rd_ptr; // depth up to 32
reg [5:0]         fifo_cnt;

always @(posedge clk) begin
    if (rst) begin
        fifo_wr_ptr <= 0;
        fifo_rd_ptr <= 0;
        fifo_cnt    <= 0;
        req_ready   <= 1;
    end else begin
        // enqueue
        if (req_valid && req_ready) begin
            fifo_addr[fifo_wr_ptr] <= req_addr;
            fifo_data[fifo_wr_ptr] <= req_data;
            fifo_op[fifo_wr_ptr]   <= req_op;
            fifo_tag[fifo_wr_ptr]  <= req_tag;
            fifo_wr_ptr <= fifo_wr_ptr + 1;
            fifo_cnt    <= fifo_cnt + 1;
        end
        // backpressure when full
        req_ready <= (fifo_cnt < FIFO_D - 1);
    end
end

// Reservation table (linear search associative)
reg valid_r [0:RESV_N-1];
reg [ADDR_W-1:0] addr_r [0:RESV_N-1];

```

```

reg [DATA_W-1:0] accum_r [0:RESV_N-1];
reg [TAG_W-1:0] head_tag [0:RESV_N-1];

integer i;
always @(posedge clk) begin
  if (rst) begin
    for (i = 0; i < RESV_N; i = i + 1) begin
      valid_r[i] <= 0;
      addr_r[i] <= 0;
      accum_r[i] <= 0;
      head_tag[i] <= 0;
    end
    fifo_rd_ptr <= 0;
    resp_valid <= 0;
    mem_req_valid <= 0;
  end else begin
    resp_valid <= 0;
    mem_req_valid <= 0;

    if (fifo_cnt > 0) begin
      reg [ADDR_W-1:0] a;
      reg [DATA_W-1:0] d;
      reg [2:0] op;
      reg [TAG_W-1:0] t;
      integer hit;
      hit = -1;

      a = fifo_addr[fifo_rd_ptr];
      d = fifo_data[fifo_rd_ptr];
      op = fifo_op[fifo_rd_ptr];
      t = fifo_tag[fifo_rd_ptr];

      // find matching reservation
      for (i = 0; i < RESV_N; i = i + 1) begin
        if (valid_r[i] && addr_r[i] == a)
          hit = i;
      end

      if (hit != -1) begin
        // update existing reservation
        case (op)
          3'd1: accum_r[hit] <= accum_r[hit] + d; // ADD
          3'd2: accum_r[hit] <= accum_r[hit] & d; // AND
          3'd3: accum_r[hit] <= accum_r[hit] | d; // OR
          3'd4: accum_r[hit] <= accum_r[hit] ^ d; // XOR
          default: accum_r[hit] <= accum_r[hit];
        endcase
      end
    end
  end

```

```

    resp_valid <= 1;
    resp_data  <= accum_r[hit];
    resp_tag   <= t;
    fifo_rd_ptr <= fifo_rd_ptr + 1;
    fifo_cnt   <= fifo_cnt - 1;
end else begin
    // new reservation entry (miss)
    for (i = 0; i < RESV_N; i = i + 1) begin
        if (!valid_r[i]) begin
            valid_r[i] <= 1;
            addr_r[i]  <= a;
            accum_r[i] <= d;
            head_tag[i] <= t;
            mem_req_valid <= 1;
            mem_req_addr  <= a;
            mem_req_wdata <= d;
            mem_req_op    <= 3'd2; // atomic memory op
            fifo_rd_ptr   <= fifo_rd_ptr + 1;
            fifo_cnt      <= fifo_cnt - 1;
            disable for;
        end
    end
end
end
end

// handle memory response
if (mem_resp_valid) begin
    resp_valid <= 1;
    resp_data  <= mem_resp_rdata;
    resp_tag   <= head_tag[0];
end
end
end
endmodule

```

4.6.3 Memory fence controller

The fence controller follows naturally from the atomic unit's ability to order single read-modify-write primitives and the barrier module's group rendezvous: it must convert software fence requests into observable memory-quiescent points across caches and outstanding transactions. The following explains the problem, analyzes scope-locality tradeoffs, and presents a synthesizable Verilog implementation that integrates with transaction trackers and cache-drain handshakes.

Problem and analysis. GPUs require multiple fence scopes: within-SM (L_0 /shared memory), L_1/L_2 (chip-local), and device/system (cross-GPU or CPU-visible). A fence must complete only after (a) all prior memory operations at the issuing scope have globally completed and (b) any store buffers, write-combines, and cache lines pending eviction are drained. Measuring

completion requires counting outstanding transactions and coordinating with cache controllers. We track:

- per-SM outstanding operations for local fences,
- per-scope outstanding counts for broader scopes,
- a FIFO of pending fence requests to serialize completion.

If N outstanding transactions each have worst-case latency L cycles, a conservative bound on drain latency is

$$[H]T_{\text{drain}} \approx N \cdot L, \quad (4.29)$$

which motivates both reducing N by early coalescing and limiting fence frequency in tight loops.

Implementation strategy.

1. Accept fence requests with scope and SM identifier.
2. Maintain counters: per-SM and per-scope. Increment/decrement via explicit transaction_inc/transaction_dec signals from load/store pipelines and cache controllers.
3. For each queued fence, assert a cache-drain request and wait until the relevant counters are zero and cache_drained is returned.
4. Provide fence_done and ack handshake to the issuing SM.

The following Verilog is synthesizable and parameterized for NUM_SM and FIFO_DEPTH. It assumes external modules assert transaction_inc/dec with matching scope bits.

Listing 4.23: Memory fence controller (synthesizable)

```

module mem_fence_ctrl #(
    parameter NUM_SM = 16,
    parameter NUM_SCOPES = 4,           // 0:SM,1:L1,2:L2,3:DEVICE
    parameter CNT_WIDTH = 12,
    parameter FIFO_DEPTH = 8,
    parameter SM_ID_WIDTH = 4,
    parameter PTR_WIDTH = $clog2(FIFO_DEPTH)
) (
    input wire clk,
    input wire rst,

    // Fence request from SM
    input wire fence_req_valid,
    input wire [SM_ID_WIDTH-1:0] fence_req_sm,
    input wire [1:0] fence_req_scope, // encoded scope
    output reg fence_req_ack,
    output reg fence_done,           // asserted when fence
        completes
    output reg [SM_ID_WIDTH-1:0] fence_done_sm,

```

```

// Transaction tracker interface (from load/store/caches)
input wire tx_inc_valid,
input wire [1:0] tx_inc_scope,
input wire [SM_ID_WIDTH-1:0] tx_inc_sm,
input wire tx_dec_valid,
input wire [1:0] tx_dec_scope,
input wire [SM_ID_WIDTH-1:0] tx_dec_sm,

// Cache drain handshake for scopes (one hot on scope)
output reg [NUM_SCOPES-1:0] drain_req,
input wire [NUM_SCOPES-1:0] drain_done
);

// Outstanding counters per scope and per-SM
reg [CNT_WIDTH-1:0] scope_count [0:NUM_SCOPES-1];
reg [CNT_WIDTH-1:0] sm_count [0:NUM_SM-1];

// Simple FIFO for fence requests
reg [SM_ID_WIDTH-1:0] fifo_sm [0:FIFO_DEPTH-1];
reg [1:0] fifo_scope [0:FIFO_DEPTH-1];
reg [PTR_WIDTH-1:0] fifo_head, fifo_tail;
reg [PTR_WIDTH:0] fifo_cnt;

integer i;
// Counters update
always @(posedge clk) begin
    if (rst) begin
        for (i = 0; i < NUM_SCOPES; i = i + 1)
            scope_count[i] <= 0;
        for (i = 0; i < NUM_SM; i = i + 1)
            sm_count[i] <= 0;
    end else begin
        if (tx_inc_valid) begin
            scope_count[tx_inc_scope] <= scope_count[tx_inc_scope] + 1;
            sm_count[tx_inc_sm] <= sm_count[tx_inc_sm] + 1;
        end
        if (tx_dec_valid) begin
            scope_count[tx_dec_scope] <= scope_count[tx_dec_scope] - 1;
            sm_count[tx_dec_sm] <= sm_count[tx_dec_sm] - 1;
        end
    end
end

// FIFO logic for fence requests
always @(posedge clk) begin
    if (rst) begin
        fifo_head <= 0;

```

```

        fifo_tail <= 0;
        fifo_cnt  <= 0;
        fence_req_ack <= 0;
    end else begin
        fence_req_ack <= 0;
        if (fence_req_valid && fifo_cnt < FIFO_DEPTH) begin
            fifo_sm[fifo_tail]    <= fence_req_sm;
            fifo_scope[fifo_tail] <= fence_req_scope;
            fifo_tail <= fifo_tail + 1;
            fifo_cnt  <= fifo_cnt + 1;
            fence_req_ack <= 1;
        end
    end
end

// Fence progression logic
reg active;
reg [SM_ID_WIDTH-1:0] cur_sm;
reg [1:0] cur_scope;

always @(posedge clk) begin
    if (rst) begin
        active <= 0;
        fence_done <= 0;
        drain_req <= 0;
    end else begin
        fence_done <= 0;

        if (!active && fifo_cnt > 0) begin
            // start new fence
            cur_sm    <= fifo_sm[fifo_head];
            cur_scope <= fifo_scope[fifo_head];
            active    <= 1;
            drain_req <= (1'b1 << fifo_scope[fifo_head]);
        end else if (active) begin
            // wait for scope drain completion and zero outstanding
            if (drain_done[cur_scope] &&
                (scope_count[cur_scope] == 0) &&
                (sm_count[cur_sm] == 0)) begin
                fence_done    <= 1;
                fence_done_sm <= cur_sm;
                active        <= 0;
                drain_req     <= 0;
                fifo_head     <= fifo_head + 1;
                fifo_cnt      <= fifo_cnt - 1;
            end
        end
    end
end

```

```

        end
    end
endmodule

```

4.6.4 Synchronization testbench

The testbench below continues verification of the memory fence controller and the atomic operation unit by exercising ordering, contention, and barrier completion across SM threads and divergent warp-like sequences. It focuses on realistic GPU behaviors: concurrent `fetch_add` /CAS sequences, acquire/release fence ordering, and barrier completion under interleaved arrivals.

Problem and analysis: we must prove that (1) atomic RMWs serialize correctly against concurrent accesses, (2) fences enforce the intended visibility windows across loads/stores, and (3) the barrier releases all waiting threads exactly once per epoch. The formal barrier condition for a block of size B is

$$[H] \sum_{i=0}^{B-1} arrived_i = B \implies release_pulse \quad (4.30)$$

where each $arrived_i$ is a single-shot arrival indicator for thread i . The testbench must saturate the atomic unit with contending updates while interposing fences to verify global ordering effects.

Implementation: the testbench instantiates compact, synthesizable models of a barrier and an atomic unit, then creates a configurable number of stimulus threads which:

1. Issue randomized atomic operations (`fetch_add` and CAS) to shared addresses to produce contention and expose lost-update bugs;
2. perform load/store sequences with optional acquire or release fence semantics to check write visibility ordering;
3. assert barrier arrival at arbitrary cycles to validate the counter-reset and single-cycle release pulse.

The Verilog below is a complete simulation harness and synthesizable DUT models for small-scale FPGA emulation. Comments inside code are concise.

Listing 4.24: Synchronization testbench exercising barrier, fences, and atomic unit

```

`timescale 1ns/1ps

// =====
// Simple parameterizable barrier (synthesizable)
// =====
module barrier #(parameter B=8) (
    input  wire clk,
    input  wire rst,
    input  wire [$clog2(B)-1:0] tid,
    input  wire arrive,
    output reg  release

```

```

);
integer i;
reg [$clog2(B):0] count;
reg arrived_flag [0:B-1];

always @(posedge clk) begin
    if (rst) begin
        count <= 0;
        release <= 0;
        for (i = 0; i < B; i = i + 1)
            arrived_flag[i] <= 0;
        end else begin
            if (arrive && !arrived_flag[tid]) begin
                arrived_flag[tid] <= 1;
                count <= count + 1;
            end
            if (count == B) begin
                release <= 1;
                for (i = 0; i < B; i = i + 1)
                    arrived_flag[i] <= 0;
                count <= 0;
            end else
                release <= 0;
        end
    end
endmodule

// =====
// Simplified atomic unit (add + read-only ops)
// =====
module atomic #(parameter W=32) (
    input wire clk,
    input wire rst,
    input wire valid,
    input wire [W-1:0] addr,
    input wire [W-1:0] data_in,
    input wire [1:0] op, // 0=read,1=add
    output reg [W-1:0] data_out,
    output reg ready
);
reg [W-1:0] mem [0:15];
integer i;
always @(posedge clk) begin
    if (rst) begin
        ready <= 0;
        data_out <= 0;
        for (i = 0; i < 16; i = i + 1)

```

```

        mem[i] <= 0;
    end else begin
        ready <= 0;
        if (valid) begin
            case (op)
                2'd0: data_out <= mem[addr[3:0]];          // read
                2'd1: mem[addr[3:0]] <= mem[addr[3:0]] + data_in; // atomic add
            endcase
            ready <= 1;
        end
    end
end
endmodule

// =====
// Fence controller (scope = global only)
// =====
module fence_ctrl(
    input wire clk,
    input wire rst,
    input wire req,
    output reg ack
);
    reg busy;
    always @(posedge clk) begin
        if (rst) begin
            ack <= 0;
            busy <= 0;
        end else begin
            if (req && !busy) begin
                busy <= 1;
                ack <= 0;
            end else if (busy) begin
                busy <= 0;
                ack <= 1;
            end else
                ack <= 0;
        end
    end
endmodule

// =====
// Top-level testbench
// =====
module tb_sync;
    reg clk = 0, rst = 1;
    always #5 clk = ~clk;

```

```

localparam NTHREAD = 4;

reg [NTHREAD-1:0] arrive;
wire release;
reg [$clog2(NTHREAD)-1:0] tid;
reg valid;
reg [31:0] addr, din;
reg [1:0] op;
wire [31:0] dout;
wire ready;
reg fence_req;
wire fence_ack;

// Instantiate DUTs
barrier #(.B(NTHREAD)) u_bar (.clk(clk), .rst(rst), .tid(tid), .arrive(
    arrive[tid]), .release(release));
atomic #(.W(32)) u_atomic (.clk(clk), .rst(rst), .valid(valid), .addr(
    addr), .data_in(din), .op(op), .data_out(dout), .ready(ready));
fence_ctrl u_fence (.clk(clk), .rst(rst), .req(fence_req), .ack(
    fence_ack));

integer i;
initial begin
    $display("Starting synchronization testbench...");
    #10 rst = 0;

    // Threads arrive at barrier
    for (i = 0; i < NTHREAD; i = i + 1) begin
        arrive[i] = 1;
        tid = i;
        #10 arrive[i] = 0;
    end

    // Wait for barrier release
    wait(release);
    $display("Barrier released at time %t", $time);

    // Perform atomic operations
    valid = 1; op = 2'd1; addr = 32'h0; din = 32'h5;
    #10 valid = 0; wait(ready);
    valid = 1; op = 2'd0; addr = 32'h0;
    #10 valid = 0; wait(ready);
    $display("Atomic read result: %h", dout);

    // Issue and complete fence
    fence_req = 1; #10 fence_req = 0;

```

```
wait(fence_ack);  
$display("Fence acknowledged at time %t", $time);  
  
#50 $finish;  
end  
endmodule
```


Chapter 5

Advanced Rendering Features

5.1 Tessellation Pipeline

5.1.1 Hull shader and control points

Building on vertex processing and primitive assembly principles, the hull stage converts patches and their control points into tessellation factors that the fixed-function tessellator consumes. The following explains operational behavior, latency and resource effects, and a reference HLSL implementation for a triangle patch.

The problem: expose sufficient geometric detail via hardware tessellation while minimizing per-patch overhead and preserving SIMT efficiency. Analysis: a hull shader has two phases. The control-point phase runs once per control point per patch (InvocationCount equals the output control-point count), producing per-control-point outputs stored in an OutputPatch. The patch-constant function runs once per patch and emits tessellation factors: outer factors (for edges) and inner factors (for interiors). Hardware then rasterizes based on the tessellator's partitioning mode (integer, fractional_odd, fractional_even) and domain type (isolines, tri, quad).

Key operational points:

- Execution model: the control-point invocations map to adjacent threads within a thread-group (SIMT lane grouping). Shared local memory (LDS/shared) or register spilling holds the OutputPatch until the constant function executes.
- Synchronization: the hull stage implicitly requires all control-point invocations to complete before the patch-constant function reads the patch. HLSL guarantees this ordering; on the hardware side this implies a small barrier and potential thread divergence cost if InvocationCount is large.
- Tessellation factor semantics: outer factors feed edge subdivision counts; inner factors feed interior subdivision. For triangle domain, positions in the domain shader are computed by barycentric interpolation of control-point outputs:

$$P(\alpha, \beta, \gamma) = \alpha P_0 + \beta P_1 + \gamma P_2, \quad \alpha + \beta + \gamma = 1. \quad (5.1)$$

For isolines, a common mapping for edge segments is $n_{\text{segments}} = \max(1, \lceil T_{\text{edge}} \rceil)$, clamped to hardware maximum.

Reference HLSL hull shader (triangle patch, pass-through control points, adaptive tessellation factor based on edge length):

Listing 5.1: Triangle hull shader with patch constant function

```

struct VS_OUT { float3 pos : POSITION; }; // vertex shader output

[domain("tri")]
[partitioning("fractional_even")]
[outputtopology("triangle_cw")]
[outputcontrolpoints(3)]
[patchconstantfunc("HSConst")]
HS_OUT HSMain(InputPatch patch, uint cpId : SV_OutputControlPointID)
{
    HS_OUT outCP;
    outCP.pos = patch[cpId].pos; // pass-through per-control-point
    return outCP;
}

struct HS_PatchConst { float edges[3] : SV_TessFactor; float inside :
    SV_InsideTessFactor; };

HS_PatchConst HSConst(InputPatch patch)
{
    HS_PatchConst pc;
    // compute geometric edge lengths in patch space (cheap L2)
    float3 p0 = patch[0].pos, p1 = patch[1].pos, p2 = patch[2].pos;
    float e0 = length(p1 - p0); // edge 0
    float e1 = length(p2 - p1); // edge 1
    float e2 = length(p0 - p2); // edge 2
    // map length to tess factor, clamp to hardware limits (1..64)
    pc.edges[0] = clamp(e0 * 8.0, 1.0, 64.0);
    pc.edges[1] = clamp(e1 * 8.0, 1.0, 64.0);
    pc.edges[2] = clamp(e2 * 8.0, 1.0, 64.0);
    pc.inside = clamp((e0+e1+e2)/3.0 * 4.0, 1.0, 64.0);
    return pc;
}

```

Implementation implications and trade-offs:

- Larger InvocationCount increases parallelism per-patch but raises register and shared-memory pressure, reducing SM occupancy.
- Using expensive per-patch computations ($\sqrt{\cdot}$, trigonometry) increases hull latency; prefer approximations or precomputed LOD.
- Choosing fractional partitioning yields smoother LOD transitions but may increase vertex count and memory bandwidth to the domain shader and ROP stages.
- Clamp tess factors to hardware maxima to avoid wasted tessellator cycles; compute factors in patch-constant stage to amortize cost across control points.

5.1.2 Fixed-function tessellator

The hull stage produces control points and per-edge/per-inner tessellation factors; the fixed-function tessellator consumes those factors and emits parametric coordinates and patch topology to downstream domain shaders. This subsection explains the tessellator’s hardware problem statement, analyses the mapping from factors to generated vertices, gives a small synthesizable controller example, and lists architectural trade-offs.

A fixed-function tessellator must:

- Interpret hull-provided tessellation factors and domain type (isolines, tri, quad).
- Convert fractional GPU factors into discrete subdivision level L with API-specified clamping and rounding.
- Generate parametric sample coordinates in the patch domain, preserving ordering for efficient post-processing and caching.
- Supply primitive connectivity or an index stream compatible with the rest of the pipeline.

Problem analysis and math: for a triangular domain with integer subdivision level L , the number of generated vertices per patch is the triangular number

$$[H]V_{\text{tri}}(L) = \frac{(L+1)(L+2)}{2}, \quad (5.2)$$

which determines buffer sizing, SM work distribution, and memory bandwidth. For isolines with L lines and R segments per line, vertices scale as $(L+1)(R+1)$. The tessellator therefore drives both compute and ROP/ROP-cache pressure depending on primitive amplification.

Implementation outline: the tessellator tends to be a small pipeline with stages:

1. Factor normalization: clamp factors to $[1, \text{max_level}]$ and map floating factors to integer L using floor/ceil modes.
2. Domain generator: nested counters produce barycentric or quad UV coordinates in a deterministic order for cache-friendly loading by the domain shader.
3. Emit stage: produce vertex parameter tuples and optional index/primitive restart markers.

Example synthesizable Verilog module below implements a simple triangle-domain tessellation iterator. It accepts integer tessellation level inputs and streams barycentric indices; downstream logic can convert these indices to fixed-point coordinates efficiently in SMs or a lightweight converter.

Listing 5.2: Triangle tessellation iterator (synthesizable).

```
module tess_iter_tri(
    input wire      clk,
    input wire      rstn,
    input wire [7:0] tess_level, // integer L
    input wire      start,      // start generation
    output reg      valid,
    output reg [7:0] i_idx,      // barycentric i
    output reg [7:0] j_idx,      // barycentric j

```

```

    output reg          last
);
// FSM states
localparam IDLE=0, RUN=1;
reg state;
reg [7:0] L;
reg [7:0] i, j;
// start handling and clamping
always @(posedge clk) begin
    if (!rstn) begin
        state<=IDLE; valid<=0; last<=0; i<=0; j<=0; L<=1;
    end else begin
        case(state)
        IDLE: begin
            valid<=0; last<=0;
            if (start) begin
                L <= (tess_level==0) ? 1 : tess_level; // clamp min 1
                i <= 0; j <= 0; state<=RUN;
            end
        end
        RUN: begin
            valid <= 1;
            i_idx <= i; j_idx <= j;
            // compute last when at final element
            last <= (i==L && j==0);
            // advance triangular counters: 0<=j<=L-i
            if (j < (L - i)) begin
                j <= j + 1;
            end else begin
                j <= 0;
                if (i < L) i <= i + 1;
                else state <= IDLE;
            end
        end
        endcase
    end
end
endmodule

```

Design implications and trade-offs:

- Hardware tessellation offloads SM cycles and reduces control divergence compared to shader-based subdivision, improving throughput for high-amplification patches.
- Fixed-function logic adds area and must balance latency versus peak throughput; deeper pipelines increase frequency but add per-patch latency.
- Supporting adaptive tessellation requires more complex factor normalization and possibly per-patch edge tess factor interpolation, increasing control logic and memory traffic.

- The ordering produced by the tessellator directly impacts domain shader locality; choose row-major or barycentric scan orders to optimize SM L1/shared memory reuse.

5.1.3 Domain shader operations

Building on the fixed-function tessellator that produced parametric coordinates and the hull shader that supplied patch-control data and per-edge tessellation factors, the domain shader consumes those domain coordinates and reconstructs high-frequency geometry and attributes for the rasterizer. This subsection analyzes domain-shader responsibilities, its numerical operations, and how that maps to SM-level execution and memory resources.

The domain shader operates per generated patch vertex (per-domain point) and must:

- evaluate the patch surface (triangle, quad, or isoline) using control points and basis functions;
- compute interpolated attributes (normals, tangents, UVs) and optional displacement;
- derive partial derivatives for correct texture LOD and normal computation;
- emit clip-space positions and varyings to the raster pipeline.

For barycentric (triangular) domains the position is a weighted sum of control points:

$$[H]P(u, v, w) = \sum_{i=0}^{n-1} b_i(u, v, w) P_i, \quad \text{with } \sum_i b_i = 1, \quad (5.3)$$

where b_i are basis functions (e.g., barycentric, Bézier, or B-spline weights) and P_i are control points from the hull shader. Displacement mapping is applied along the geometric normal N as

$$[H]P' = P + d(u, v) N, \quad (5.4)$$

with $d(u, v)$ sampled via TMU texture fetches. Accurate shading normals require partial derivatives:

$$[H]N = \text{normalize} \left(\frac{\partial P}{\partial u} \times \frac{\partial P}{\partial v} \right), \quad (5.5)$$

computed either analytically from basis functions or numerically using GPU ddx/ddy where supported.

Implementation considerations for GPU architects:

- SIMT efficiency: domain shader invocations for a patch are independent and map well to warps/wavefronts, but divergent control flow (conditional displacement, branching per-patch) reduces ALU utilization and stalls TMU pipelines.
- Memory and register pressure: passing many control points or large patch-constant buffers increases register usage per thread, reducing occupancy on an SM/CU. Use compact patch-constant layouts and fetch control points from L1/LDS when reused across invocations.
- Texture LOD: computing ddx/ddy requires coherent neighbor invocations; when domain points are processed out-of-order or when domain shaders run in scalarized flows, LOD can become noisy—prefer tiled dispatch that preserves neighborhood coherence before TMU fetches.

Example HLSL domain shader (triangular domain). This is a production-ready domain shader illustrating basis evaluation, displacement, and normal formation.

Listing 5.3: Tri-domain shader with displacement and normal calc

```
struct HS_Output { float3 pos : POSITION; float3 nrm : NORMAL; float2 uv :
    TEXCOORD0; };
struct DS_Input { float3 pos : POSITION; float3 nrm : NORMAL; float2 uv :
    TEXCOORD0; };

PatchConstantBuffer { HS_Output cp[3]; } pcb; // control points passed from
    hull shader

[domain("tri")] // triangle domain shader
void DSMain(const OutputPatch patch, float3 bary : SV_DomainLocation,
    out float4 oPos : SV_Position, out float3 oNrm : NORMAL, out
    float2 oUV : TEXCOORD0)
{
    // interpolate position and attributes using barycentric weights
    float3 P = bary.x*patch[0].pos + bary.y*patch[1].pos + bary.z*patch[2].pos
        ; // position
    float3 N = normalize(bary.x*patch[0].nrm + bary.y*patch[1].nrm + bary.z*
        patch[2].nrm); // normal
    float2 UV = bary.x*patch[0].uv + bary.y*patch[1].uv + bary.z*patch[2].uv;
        // uv

    // displacement sample (TMU fetch) and apply along interpolated normal
    float d = displacementSampler.SampleLevel(displacementSamplerSamplerState,
        UV, 0); // explicit LOD
    P += d * N;

    // emit clip-space position (assume viewProj matrix is available)
    oPos = mul(float4(P,1.0), viewProj);
    oNrm = N;
    oUV = UV;
}
```

Design implications and trade-offs:

- Pushing displacement and analytic derivative evaluation into the domain shader increases visual fidelity but raises TMU pressure and register usage, reducing warp occupancy; balance by using lower LOD displacement or precomputed normal maps for distant patches.
- Favor fetching control points from on-chip LDS when multiple domain invocations share a patch to reduce L2/VRAM traffic; hardware should support patch-local storage and fast cross-lane broadcast to maximize throughput.
- A hardware/software co-design that schedules domain-shader work in locality-preserving batches improves TMU and cache utilization, reducing wasted cycles from texture latency and improving overall rasterization throughput.

5.1.4 Adaptive tessellation control

The domain shader provides final clip-space positions and the fixed-function tessellator consumes tessellation factors; adaptive control sits between them, converting screen-space metrics into per-patch outer and inner levels while respecting crack prevention and hardware cost. This subsection analyzes how to compute robust, low-cost tessellation factors in hardware and gives a synthesizable Verilog implementation for a patch-level controller.

Problem: naive uniform tessellation wastes SM cycles, TMU bandwidth, and ROP/fragment processing when small or distant patches are over-tessellated. Goal: maximize visual fidelity per fragment-work by adapting tessellation to screen-space edge lengths and curvature estimates while avoiding popping and crack artifacts.

Analysis and metric selection:

- Use domain-shader-projected 2D window coordinates (x, y) for control points to measure screen-space edge length L_{ij} .
- Exact Euclidean distance with a hardware square-root is expensive; use a standard magnitude approximation to save area and latency:

$$[H]\tilde{L} = \max(|\Delta x|, |\Delta y|) + \frac{1}{2} \min(|\Delta x|, |\Delta y|). \quad (5.6)$$

- Map length to tessellation level by a scaled ratio and clamp to device limits:

$$[H]t = \text{clamp}\left(\left\lceil \frac{\alpha \tilde{L}}{S} \right\rceil, 1, T_{\max}\right), \quad (5.7)$$

where S is target edge size in pixels, α compensates for projected curvature, and $T_{\max}$ is hardware maximum.

Operational considerations:

1. Hysteresis: blend new t with previous frame's t_{prev} to avoid popping; implement as integer exponential smoothing.
2. Crack prevention: ensure adjacent patches choose outer levels consistent at shared edges, or bias rounding toward even numbers when required by fixed-function tessellator rules.
3. Compute placement: performing these calculations in a lightweight dedicated tessellation-control unit reduces SM load but adds dedicated logic area and pipeline latency.

Implementation: a synthesizable Verilog module that takes four 16-bit fixed-point window coordinates and outputs four outer and two inner levels using the approximation in (1). The design uses integer arithmetic and a simple hysteresis register.

Listing 5.4: Patch tessellation factor generator (quad patch)

```
module tess_ctrl #(
    parameter COORD_W = 16,           // Q12.4 fixed-point
    parameter TF_W     = 8,           // tess factor width
    parameter ALPHA    = 1,           // scale numerator
```

```

parameter TARGET    = 8,           // target pixels per edge
parameter TMAX      = 64
) (
  input  wire clk, rst,
  input  wire valid,                // input coords valid
  input  wire [COORD_W-1:0] x0,y0, // control points (window space)
  input  wire [COORD_W-1:0] x1,y1,
  input  wire [COORD_W-1:0] x2,y2,
  input  wire [COORD_W-1:0] x3,y3,
  output reg  [TF_W-1:0] outer0, outer1, outer2, outer3,
  output reg  [TF_W-1:0] inner0, inner1
);

// compute abs diff and approx length for an edge
function [COORD_W-1:0] approx_len;
  input signed [COORD_W-1:0] dx, dy;
  reg [COORD_W-1:0] adx, ady, ma, mi;
  begin
    adx = (dx[COORD_W-1]) ? -dx : dx;
    ady = (dy[COORD_W-1]) ? -dy : dy;
    ma = (adx > ady) ? adx : ady;
    mi = (adx > ady) ? ady : adx;
    approx_len = ma + (mi >> 1); // eq (1) integer form
  end
endfunction

// map to tess factor (integer)
function [TF_W-1:0] map_tess;
  input [COORD_W-1:0] len;
  reg [COORD_W+7:0] scaled;
  reg [TF_W-1:0] t;
  begin
    scaled = (ALPHA * len) + (TARGET>>1);
    t = scaled / TARGET; // integer division -> ceil approx via bias above
    if (t < 1) t = 1;
    if (t > TMAX) t = TMAX;
    map_tess = t;
  end
endfunction

// pipeline registers
reg [COORD_W-1:0] ax0,ay0,ax1,ay1,ax2,ay2,ax3,ay3;
always @(posedge clk) begin
  if (rst) begin
    outer0<=0; outer1<=0; outer2<=0; outer3<=0; inner0<=0; inner1<=0;
  end else if (valid) begin
    ax0<=x0; ay0<=y0; ax1<=x1; ay1<=y1; ax2<=x2; ay2<=y2; ax3<=x3; ay3<=y3;
    // compute outer edges

```



```

    outer0 <= map_tess( approx_len(ax0-ax1, ay0-ay1) );
    outer1 <= map_tess( approx_len(ax1-ax2, ay1-ay2) );
    outer2 <= map_tess( approx_len(ax2-ax3, ay2-ay3) );
    outer3 <= map_tess( approx_len(ax3-ax0, ay3-ay0) );
    // compute two inner edges (diagonals) for quad domain
    inner0 <= map_tess( approx_len(ax0-ax2, ay0-ay2) );
    inner1 <= map_tess( approx_len(ax1-ax3, ay1-ay3) );
end
end
endmodule

```

Design implications and trade-offs:

- Benefit: reduces fragment load and TMU traffic for distant geometry, improving SM utilization.
- Cost: added hardware area, latency, and small power overhead for tessellation-control unit and registers.
- Quality trade-offs: approximation and integer mapping yield slight error; hysteresis or temporal blending is required to prevent popping at LOD transitions.
- Integration: for coherent workloads, performing this in a dedicated control unit offloads SMs; for dynamic meshes, coupling with mesh-shader culling yields greater savings.

5.2 Geometry Processing

5.2.1 Geometry shader stage

The geometry shader stage follows naturally from tessellation and domain-shader outputs: it consumes assembled primitives and can perform per-primitive computation, amplification, and layered routing before rasterization. Compared with the per-vertex domain shader, the geometry shader operates at primitive granularity and therefore has distinct throughput and memory implications.

The problem: provide flexible primitive amplification and per-primitive operations (point sprite expansion, silhouette extrusion, layered rendering, or stream output) while preserving high SM/CU utilization and not breaking early-Z or vertex reuse. Analysis shows the key constraints:

- Execution model: one GS invocation per input primitive; variability in emitted primitives causes control-flow divergence across warps/wavefronts, reducing SIMT efficiency.
- Bandwidth: each emitted vertex carries attributes of size A bytes; if the maximum emitted vertices per primitive is k and input primitives are N_{in} , worst-case output vertex traffic is bounded by

$$[H]V_{\text{out}} \leq k \cdot N_{\text{in}}. \quad (5.8)$$

Attribute bandwidth increase over the pipeline is $\Delta B \leq (k - 1) A N_{\text{in}}$ in the worst case.

- Memory and reuse: rasterization benefit from post-transform vertex cache hit rates falls if GS expands or duplicates vertices, because indexed reuse across primitives can be lost.

Implementation notes for hardware and driver:

- Primitive assembly hands a compact primitive descriptor to the GS unit; modern GPUs implement GS invocations on general-purpose shader cores (SM/CU) with batching to amortize dispatch overhead. For correct ordering and stream output, the GS must either buffer emitted vertices in on-chip SRAM or write to a stream-out buffer with backpressure.
- To limit divergence, GS implementations often:
 1. Limit `max_vertices` conservatively to bound register pressure.
 2. Use per-invocation work-queues and group multiple invocations into waves to maintain occupancy.
 3. Integrate early rejection (bounding cone tests) in a predicated, SIMD-friendly way to avoid serializing long emit paths.
- Interaction with early-Z: GS that modifies positions or emits layers may disable early-Z optimizations unless conservative depth testing or late-Z is employed after GS output is committed.

Practical example (GLSL geometry shader) — emits a triangle fan expanded per input triangle and writes to a specific layer for layered rendering:

Listing 5.5: GLSL geometry shader emitting amplified triangles and routing to layers

```
#version 450
layout(triangles) in;
layout(triangle_strip, max_vertices = 6) out; // up to 2 tris per input
layout(triangle_strip, max_vertices = 6) out;
uniform float expand; // expansion factor
void main() {
    // compute centroid of input triangle
    vec3 centroid = (gl_in[0].gl_Position.xyz +
                    gl_in[1].gl_Position.xyz +
                    gl_in[2].gl_Position.xyz) / 3.0;
    for(int i=0;i<3;++i) {
        // emit original vertex
        gl_Position = gl_in[i].gl_Position;
        gl_Layer = int(gl_in[0].gl_Position.w) % 4; // example layered render
        EmitVertex();
    }
    EndPrimitive();
    // emit expanded triangle (silhouette extrude) as additional primitive
    for(int i=0;i<3;++i) {
        vec4 dir = normalize(gl_in[i].gl_Position - vec4(centroid,0.0));
        gl_Position = gl_in[i].gl_Position + vec4(dir.xyz * expand, 0.0);
        gl_Layer = (gl_Layer + 1) % 4; // route to different layer
        EmitVertex();
    }
    EndPrimitive();
}
```

}

Design implications and trade-offs:

- Allowing GS provides flexible primitive amplification and layered outputs but increases register pressure, reduces warp efficiency, and may require larger on-chip buffers for emitted vertices.
- Constrain max_vertices and favor compute-friendly reprojection or mesh shaders for heavy amplification to maintain high SIMT utilization and better vertex reuse.
- For performance-critical workloads, prefer mesh/task shader pipelines or perform amplification on the CPU for coarse geometry to avoid SM stalls and ROP/early-Z disruptions.

5.2.2 Primitive amplification

The previous subsection described how a geometry shader can inspect and alter per-primitive data; primitive amplification is the specific operational pattern where one input primitive spawns multiple output primitives and therefore multiplies downstream work and memory traffic.

Primitive amplification matters because it changes the cost balance between vertex/primitive processing and rasterization/TMU/ROP stages. Operationally, amplification increases:

- vertex fetch and transform load (more vertices written and possibly reprocessed),
- index/stream-out buffer usage (temporary storage for emitted vertices),
- rasterization workload (more triangles -> more fragments and TMU traffic).

We quantify amplification to make resource trade-offs explicit. Let N_{in} be input primitives, a_i the amplification factor for primitive i (number of output primitives produced), and v the average vertices per output primitive. Total emitted vertices V_{out} and total output primitives P_{out} are

$$[H]P_{\text{out}} = \sum_{i=1}^{N_{\text{in}}} a_i, \quad V_{\text{out}} = v \cdot P_{\text{out}}. \quad (5.9)$$

If amplification is uniform, with constant a , then $P_{\text{out}} = a N_{\text{in}}$. Rasterization fragment work scales roughly with expected screen-space area per output primitive; denote average pixel coverage c , then expected fragments $F \approx c \cdot P_{\text{out}}$. This linear model exposes two bottleneck regimes:

1. vertex-bound: when the cost to generate and transform V_{out} dominates SM cycles and memory bandwidth for stream-out, and
2. fragment-bound: when F saturates TMU/ROP throughput and memory bandwidth for texture lookups and color/depth writes.

Hardware and SIMT implications:

- Geometry shaders introduce serialization per-primitive in many implementations; high amplification exacerbates warp underutilization because a single primitive's emission can stall the SIMT group. Modern designs prefer task/mesh shaders or compute-based amplification to preserve occupancy.

- Permanent buffers for emitted vertices need sizing with worst-case amplification; dynamic allocation or multi-pass approaches (count-then-emit) avoid overprovisioning but add synchronization.

Example GLSL geometry shader demonstrating a small, bounded amplification (extruding a triangle into a simple fan of quads; uses `max_vertices` to set emission cap):

Listing 5.6: Geometry shader: bounded primitive amplification into quads

```
#version 450
layout(triangles) in;
layout(triangle_strip, max_vertices = 12) out; // cap total emitted verts
uniform mat4 projView;
void main() {
    // Emit for each input triangle up to 3 quads (amplification factor <=3)
    for(int q=0;q<3;++q){ // simple bounded amplification
        // compute quad vertex positions (shader-local math) and emit 6 verts
        -> 2 tris
        for(int v=0; v<6; ++v){
            gl_Position = projView * vec4(gl_in[v%3].gl_Position.xyz + vec3
                (0.01*q),1.0);
            EmitVertex(); // small comment: emits transformed vertex
        }
        EndPrimitive();
    }
}
```

Design implications and trade-offs:

- Cap amplification per-primitive in hardware (`max_vertices`) to bound on-chip buffer pressure and avoid worst-case SM stalls.
- Prefer mesh/task shaders or compute-based generation when large or data-dependent amplification is required to retain high occupancy and avoid GS serialization.
- Account for downstream TMU/ROP capacity: aggressive amplification can shift a pipeline from vertex-bound to fragment-bound, increasing power and memory bandwidth cost.

5.2.3 Stream output

Following primitive amplification and the geometry shader’s ability to emit additional vertices and primitives, stream output provides the hardware path to capture those generated vertex streams directly to memory for reuse or GPU-driven workflows such as GPU-based culling, particle systems, or meshlet generation.

Stream output (often called transform feedback) solves the problem of avoiding expensive CPU round-trips and redundant vertex reprocessing by recording post-shader vertex attributes into GPU memory. Architectural analysis shows three core requirements:

1. High-bandwidth, write-combine buffering close to the shader cluster (SM/CU) to absorb bursty emissions from amplified geometry.

2. Flexible attribute packing and stride control so varying shader outputs map efficiently to memory beats (commonly 128-bit or 256-bit).
3. Synchronization primitives (atomic counters, fences) to implement append and multi-producer safety without stalling SM throughput.

Quantitatively, stream output bandwidth B_{out} is the product of vertex emission rate R_v and per-vertex output size S_v :

$$[H]B_{\text{out}} = R_v \cdot S_v. \quad (5.10)$$

For example, if an SM cluster emits $R_v = 2 \cdot 10^6$ vertices/s with $S_v = 64$ bytes, $B_{\text{out}} = 128$ MB/s sustained per cluster, which scales with active clusters and must be provisioned against L2 and VRAM peak bandwidth.

Implementation details that affect performance:

- Packing: align attribute streams to natural bus width; partial-beat writes increase ROP/L2 overhead.
- Coalescing: local write-combine buffers aggregate beats per target buffer; commit when full or on explicit flush.
- Append semantics: an atomic add provides per-buffer offsets; the stream-output unit reserves contiguous ranges to avoid fine-grain synchronization.
- Backpressure: SMs must observe a ready signal so that shader emission does not overflow on-chip buffers; ready/valid handshaking maintains SIMT efficiency.

A compact synthesizable Verilog example implements a stream-output engine that buffers 128-bit beats and performs burst writes to an external memory interface. It demonstrates FIFO buffering, a simple DMA write state machine, and an atomic-style append counter.

Listing 5.7: Simple stream-output engine (synthesizable)

```

module stream_output_engine #(
  parameter ADDR_WIDTH = 32,
  parameter DEPTH = 8
) (
  input  wire          clk,
  input  wire          rst_n,
  // Input from shader/emission unit
  input  wire          in_valid,
  input  wire [127:0]  in_data,
  input  wire          in_last,      // end-of-primitive marker
  output reg          in_ready,
  // Memory write/master (simple handshake)
  output reg          mem_wr_valid,
  output reg [ADDR_WIDTH-1:0] mem_wr_addr,
  output reg [127:0]    mem_wr_data,
  input  wire          mem_wr_ready,
  // Append counter (atomic reserve simple model)
  output reg [ADDR_WIDTH-1:0] append_counter

```

```

);

// FIFO for one-cluster buffering
reg [127:0] fifo [0:DEPTH-1];
reg [$clog2(DEPTH):0] wr_ptr, rd_ptr, cnt;

// DMA FSM
localparam IDLE = 0, WRITE = 1;
reg state;

// Simple append: reserve space on first beat when in_valid & in_ready
always @(posedge clk) begin
    if (!rst_n) begin
        wr_ptr <= 0; rd_ptr <= 0; cnt <= 0;
        in_ready <= 1'b1; mem_wr_valid <= 1'b0;
        mem_wr_addr <= 0; mem_wr_data <= 0;
        append_counter <= 0; state <= IDLE;
    end else begin
        // Accept input if FIFO not full
        if (in_valid && in_ready) begin
            fifo[wr_ptr] <= in_data;
            wr_ptr <= wr_ptr + 1;
            cnt <= cnt + 1;
            // reserve space on append when first beat (simple model)
            if (cnt == 0) begin
                mem_wr_addr <= append_counter; // base addr
                append_counter <= append_counter + 16; // reserve one beat (example)
            end
        end
        in_ready <= (cnt < DEPTH-1);

        // DMA FSM: issue writes while FIFO not empty
        case (state)
            IDLE: if (cnt > 0) begin
                mem_wr_valid <= 1'b1;
                mem_wr_data <= fifo[rd_ptr];
                state <= WRITE;
            end
            WRITE: if (mem_wr_ready) begin
                rd_ptr <= rd_ptr + 1;
                cnt <= cnt - 1;
                mem_wr_addr <= mem_wr_addr + 16; // next beat address
                if (cnt > 1) begin
                    mem_wr_data <= fifo[rd_ptr+1];
                    mem_wr_valid <= 1'b1;
                    state <= WRITE;
                end else begin

```

```

        mem_wr_valid <= 1'b0;
        state <= IDLE;
    end
end
endcase
end
end
endmodule

```

Design implications and trade-offs:

- Larger on-chip buffers reduce backpressure but increase area and power; sizing must match peak amplification rates.
- Beat alignment and write-combining maximize memory efficiency; misaligned writes raise L2/ROP stress and reduce effective bandwidth.
- Atomic append resolution limits scaling; hierarchical counters or per-SM reservations reduce contention at the cost of memory footprint.

5.2.4 Layered rendering

The previous discussion showed how stream output exports per-primitive data and how primitive amplification can replicate geometry; layered rendering takes those amplified primitives and routes them efficiently into multiple framebuffer slices or render-target layers. This subsection analyzes the hardware and algorithmic options for mapping amplified primitives to layers, quantifies the memory impact, and provides a shader example that demonstrates the common API-level control used by drivers and hardware.

Layered rendering allows a single draw to produce output into multiple layers of:

- texture arrays or 2D array render targets,
- cube-map faces,
- or layered MRTs (multiple render target slices).

Two common control points are the geometry-stage write of `gl_Layer` and viewport-array indexing via `gl_ViewportIndex`. Architecturally, GPUs implement layered rendering with one of two patterns:

1. Broadcast rasterization: rasterizer generates a single coverage set, which is then duplicated across target ROP slices; ideal when L is small and SIMD replicate paths are cheap.
2. Iterative rasterization: the pipeline replays each primitive per-layer, amortizing ROP/DEPTH resources at the cost of extra raster/attribute work.

Operational metrics must quantify bandwidth cost. If P primitives cover on average p pixels each, there are L target layers, s samples per pixel, and b bytes written per sample, then total color write traffic approximates:

$$[H]W_{\text{writes}} \approx P \cdot p \cdot L \cdot s \cdot b. \quad (5.11)$$

When L grows, the dominant cost shifts from vertex/fragment arithmetic to ROP and memory bandwidth, so hardware designers trade replication in the raster front-end against repeated raster passes.

Implementation strategies to optimize layered throughput:

- Render-target cache (tile buffer) keyed by (tile, layer) reduces VRAM writes by accumulating per-layer tiles before evict.
- Early-Z and Hi-Z must be evaluated per-layer; if depth is identical across layers, a single depth test pass can short-circuit, but differing layer-specific depth writes eliminate that optimization.
- For mesh/geometry shader pipelines, emit-time culling and per-layer bounding can avoid unnecessary rasterization of empty layers.

Example geometry shader that assigns layer per-primitive (GLSL style):

Listing 5.8: Geometry shader emitting to layered framebuffer

```
#version 450
layout(triangles) in;
layout(triangle_strip, max_vertices = 3) out;
layout(location = 0) out vec3 v_color;

uniform int layer_mask; // bitmask of layers to output

void emitToLayer(int layer) {
    gl_ViewportIndex = layer;           // select viewport if needed
    gl_Layer = layer;                   // route fragment output to layer
    for(int i = 0; i < 3; ++i) {
        v_color = vec3(layer / 8.0);    // example payload
        gl_Position = gl_in[i].gl_Position;
        EmitVertex();                   // emit vertex to currently selected layer
    }
    EndPrimitive();
}

void main() {
    for (int layer = 0; layer < 8; ++layer) {
        if ((layer_mask & (1 << layer)) != 0) {
            emitToLayer(layer);
        }
    }
}
```


5.3 Mesh Shaders

5.3.1 Meshlet-based processing

Building on earlier discussion of adaptive subdivision and primitive amplification, meshlet-based processing replaces traditional per-vertex streaming with compact clusters designed for high locality, efficient culling, and explicit mapping onto SIMT workgroups. This subsection treats the meshlet problem, evaluates trade-offs for GPU mapping, and gives a production-ready CPU packing routine.

Problem: modern scenes contain millions of triangles with poor locality for SIMD loads and redundant vertex fetching across draw calls. Meshlets group nearby triangles and vertices so a single workgroup can fetch a small vertex table into SM-local memory, run culling and attribute shading, and emit primitives with minimal memory traffic.

Analysis: effective meshlets optimize three metrics:

- vertex reuse within the meshlet (reduces fetches to global memory);
- compact bounding volume for conservative culling (frustum, backface, and cone culling);
- triangle count matched to SIMT granularity to maximize occupancy.

Projecting a meshlet bounding sphere to screen space gives a simple cull test. With focal length f and depth z to the sphere center, the screen-space radius r_{screen} is approximately

$$r_{\text{screen}} \approx \frac{r_{\text{world}} f}{z}, \quad (5.12)$$

so small-world-radius clusters or clusters with larger z are cheaper to reject early in the pipeline. Tune r_{world} and maximum triangle count so that typical meshlet fits into per-SM shared memory and register pressure bounds.

Implementation: a greedy CPU-side packer that emits meshlets meeting vertex and triangle limits and a spatial locality heuristic is shown below. The routine produces index lists and compact vertex remapping tables suitable for upload to GPU buffers; the GPU task shader then launches mesh shaders using those meshlet descriptors.

Listing 5.9: CPU meshlet packer (greedy, production-ready)

```
#include
#include
#include
// pack triangles into meshlets under VTX_LIMIT and TRI_LIMIT
struct Meshlet { std::vector indices; std::vector uniqueVerts; };
Meshlet pack_meshlet(const std::vector& tris,
                    const std::vector& verts,
                    size_t startIdx, size_t VTX_LIMIT=64, size_t TRI_LIMIT
                    =126) {
    Meshlet m;
    std::unordered_map remap; remap.reserve(VTX_LIMIT);
    for(size_t i=startIdx; i=VTX_LIMIT){ fits=false; break; }
    }
    if(!fits) continue;
```

```

// accept triangle: add remapped indices
for(int k=0;k<3;k++){
    auto it = remap.find(t[k]);
    if(it==remap.end()){
        uint32_t nid = (uint32_t)remap.size();
        remap[t[k]] = nid; m.uniqueVerts.push_back(t[k]);
    }
    m.indices.push_back(remap[t[k]]);
}
}
return m;
}
// Caller iterates startIdx to cover all triangles and emits meshlet
descriptors and vertex lists.

```

Hardware mapping and implications:

- Map one meshlet per workgroup; choose group size to be an integer multiple of warp/wavefront size to avoid tail divergence.
- Store meshlet vertex table in SM shared memory to reduce L1 traffic; aim for vertex attribute footprint fitting local L1 plus registers.
- Trade-offs:
 1. Smaller meshlets improve cull rejection but increase descriptor overhead and drawcall-like submission costs.
 2. Larger meshlets boost vertex reuse and reduce per-meshlet overhead but may reduce cull efficiency and increase register/shared memory pressure, lowering occupancy.
 3. Choose meshlet limits based on SM shared memory, TMU throughput, and typical scene depth distribution for best end-to-end throughput.

5.3.2 Task and mesh shader stages

Building on meshlet locality and per-meshlet attribute packing, the task/mesh shader split moves amplification and coarse culling entirely onto the GPU so that work generation, LOD selection, and primitive assembly avoid CPU draw-call overhead and expensive geometry-shader stalls.

The problem: when many small objects or highly amplified geometry (LOD, surfels, vegetation) are driven by the CPU, API submission and per-draw state changes dominate frame time. Task shaders solve this by generating many meshlets on-GPU and performing hierarchical culling; mesh shaders consume those meshlets and produce final vertex/primitive outputs for rasterization. Analysis focuses on mapping these stages to SM/CU resources, minimizing register and shared-memory pressure while maximizing throughput under the SIMT execution model.

Key operational points:

- Task shader role: read scene-level bounding data, perform coarse frustum/occlusion/LOD culls, and produce a compact list of meshlet descriptors or indirect-dispatch

arguments. Use of on-chip shared memory (LDS) reduces DRAM traffic for per-mesh streaming and accelerates reductions.

- Mesh shader role: for each meshlet descriptor, fetch vertex and index ranges, run vertex processing and primitive culling inside a single workgroup, and emit triangles (or points/lines) to the tile/Rasterizer. Variable-rate outputs require the hardware to support dynamic primitive counts per workgroup and efficient packing into the ROP interface.

Resource-limited occupancy can be estimated: let R_{total} be registers per SM, S_{total} be shared (LDS) per SM, R_t the registers per thread, S_{wg} the shared per mesh-workgroup, and W_{wg} threads per workgroup. Then an upper bound on simultaneously resident workgroups is

$$[H]N_{\text{wg}} \leq \min\left(\left\lfloor \frac{R_{\text{total}}}{R_t W_{\text{wg}}} \right\rfloor, \left\lfloor \frac{S_{\text{total}}}{S_{\text{wg}}} \right\rfloor, N_{\text{hw}}\right), \quad (5.13)$$

and effective warp-level occupancy follows from N_{wg} and warp size. This guides per-meshlet thread-width (W_{wg}) choice: wider workgroups reduce divergence but raise R_t and S_{wg} , lowering N_{wg} and hurting latency hiding.

Implementation sketch (HLSL-like) — task shader writes compact meshlet descriptors and an indirect count; mesh shader consumes descriptor by index and emits primitives:

Listing 5.10: Illustrative task+mesh shader flow (HLSL-like)

```
StructuredBuffer meshletHeaders : register(t0); // read-only meshlet metadata
RWStructuredBuffer meshletList   : register(u0);    // append list of visible
    meshlet IDs
RWByteAddressBuffer indirectArgs      : register(u1);    // [0] =
    meshletCount (written by task)

// Task shader: coarse cull and append visible meshlet IDs
[shader("task")]
void TaskMain(uint3 tid : SV_DispatchThreadID)
{
    // load bounding, do frustum/occlusion/LOD test
    if (CoarseCull(meshletHeaders[tid.x])) return;
    uint idx = InterlockedAdd(indirectArgs, 0, 1);    // atomically
        increment visible count
    meshletList.Append(tid.x);    // append meshlet ID
        for mesh shader
}

// Mesh shader: per-meshlet vertex fetch, per-primitive culling, and emit
[shader("mesh")]
void MeshMain(uint3 meshID : SV_GroupID)
{
    uint mID = meshletList[meshID.x];    // get meshlet
        descriptor
    Meshlet m = LoadMeshlet(mID);    // fetch indices/
        vertices
    // shared memory staging of vertex attributes
```

```

GroupMemoryBarrierWithGroupSync();
// per-thread vertex work then primitive cull and EmitMesh
for (each primitive) EmitVertexAndTriangle(...); // outputs to
    rasterizer
}

```

Design implications and trade-offs:

- Push coarse culling into the task stage to minimize meshletList size and downstream memory bandwidth, but the task stage must remain lightweight to avoid becoming a bottleneck on SM scheduling.
- Choose W_{wg} to balance divergence vs occupancy per equation (1); for workloads with high per-meshlet computation (procedural tessellation, displacement), favor larger workgroups, else keep them small to improve scheduling elasticity.
- Hardware must support efficient append/atomic operations and variable-output primitives; lacking those, driver-side solutions reintroduce CPU overhead and latency.

5.3.3 Workgroup culling and amplification

These techniques build directly on task-driven dispatch and meshlet packing discussed earlier, placing coarse rejection and per-meshlet amplification at the point where task shaders emit work for mesh shaders. The goal is to reduce SM work by discarding invisible or negligible meshlets on the GPU and to control amplification so that downstream mesh shader work remains balanced.

Problem: naive amplification can multiply triangle production and waste SM cycles on occluded geometry. Analysis: perform inexpensive workgroup-level tests (bounding-sphere frustum cull, screen-space area, occlusion queries when available) inside the task stage; use the result to (a) emit zero entries for culled meshlets, (b) emit amplified entries up to a cap $A_{\max}$ for visible meshlets, or (c) downgrade LOD. For a bounding sphere with world radius r and camera-space depth z , the approximate screen-space radius in pixels is

$$[H]r_{\text{screen}} \approx r \cdot \frac{f}{z}, \quad (5.14)$$

where f is the camera focal length in pixels (or equivalently the projection matrix scale). Use the screen area πr_{screen}^2 to compare against a threshold areaToCull to avoid spawning mesh shader work for negligible contributions.

Implementation pattern: the task shader iterates meshlets assigned to its workgroup, computes cheap tests, aggregates visible counts in shared memory to avoid high atomic contention, then writes compacted entries to an indirect dispatch buffer. The amplification factor A per meshlet can be computed as a function of screen area or distance, bounded by $A_{\max}$ to limit triangle amplification and register pressure when mapped to an SM.

Listing 5.11: Task-stage culling and amplification (HLSL-like); uses shared memory, atomics, and writes to an indirect buffer.

```

TaskPayload TaskMain(uint3 dispatchId : SV_DispatchThreadID)
{
    // shared aggregator in threadgroup memory (declared elsewhere)

```

```

// Each thread loads one meshlet's bounding sphere (center, radius)
Meshlet mesh = LoadMeshlet(dispatchId.x); // read-only
    fetch
float3 viewPos = mul(viewMatrix, float4(mesh.center, 1)).xyz;
float z = viewPos.z;
float rScreen = mesh.radius * focalLength / max(0.0001, z); // eq. (1)
bool visible = FrustumTest(viewPos, mesh.radius);
if (visible && (3.14159 * rScreen * rScreen) > areaToEmit) {
    // compute amplification factor adaptively
    uint ampl = min(Amax, (uint)clamp(rScreen / amplScale, 1, (float)Amax)
        );
    // compact into threadgroup list using atomic add to groupCount
    uint idx = 0;
    if (ThreadIsLeader()) {
        // leader writes groupCount to global atomic to reserve space
        idx = InterlockedAdd(globalIndirectCounter, groupCountLocal);
    }
    GroupBarrier();
    // write ampl entries (one per amplified instance) into indirect
    buffer
    for (uint a = 0; a < ampl; ++a) {
        uint outIdx = idx + a;
        IndirectBuffer[outIdx] = mesh.instanceData;
    }
}
return TaskPayload();
}

```

5.3.4 Hardware resource mapping

Building on task/mesh pipeline sequencing and the culling/amplification strategies, hardware resource mapping ties those dynamic workloads to fixed SM/CU resources so that amplified meshlets run without starving registers, shared memory, or thread slots.

The problem: mesh shaders emit meshlets with variable vertex and primitive counts, and task shaders can amplify work unpredictably. The allocator must satisfy three hard constraints per SM: register file capacity, shared/LDS capacity, and maximum concurrent thread slots (warps/wavefronts). Practical mapping therefore becomes a constrained packing problem with additional locality objectives (co-locating connected meshlets to reduce texture/TMU and L1 pressure) and time multiplexing to hide latency.

Analysis: the number of resident workgroups (meshlet thread-groups) per SM is the bottlenecked minimum of resource ratios:

$$[H]N_{\text{resident}} = \min \left(\left\lfloor \frac{R_{\text{SM}}}{R_{\text{wg}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{wg}}} \right\rfloor, \left\lfloor \frac{T_{\text{SM}}}{T_{\text{wg}}} \right\rfloor \right) \quad (5.15)$$

where R =registers, S =shared memory bytes, T =thread slots. Designers must compute R_{wg} and S_{wg} at compile-time when possible; for mesh shaders they are often runtime-estimated, so conservative static reservation or a two-phase allocation (speculative launch then trim) is used.

Implementation recommendations:

- Use a small hardware allocator per dispatch unit that tracks per-SM free resources and applies a best-fit-first policy to maximize locality (prefer SMs with texture cache residency for that meshlet).
- Support dynamic reclamation: when meshlet finishes, free registers and LDS atomically to enable immediate dispatch from task shader amplification.
- Offer a lightweight per-meshlet descriptor in a shared queue with fields: `vertex_count`, `primitive_count`, `expected_registers`, `expected_smem`, `locality_hint` (texture cache slice). The scheduler computes a fast-fit via parallel comparators.

A compact synthesizable allocator FSM in Verilog that implements first-fit allocation across `NUM_SMS` SMs is shown below; it accepts request vectors and returns a grant index when resources suffice.

Listing 5.12: Simple SM resource allocator (synthesizable).

```

module sm_alloc #(
  parameter NUM_SMS = 8,
  parameter REG_WIDTH = 16,    // bits for reg counts
  parameter SMEM_WIDTH = 20,  // bytes count width
  parameter IDX_WIDTH = 3
) (
  input  wire      clk,
  input  wire      rstn,
  input  wire      req_valid,
  input  wire [REG_WIDTH-1:0] req_regs,
  input  wire [SMEM_WIDTH-1:0] req_smem,
  input  wire [15:0] req_threads, // threads needed
  output reg      grant_valid,
  output reg [IDX_WIDTH-1:0] grant_idx
);

  // Per-SM free resources
  reg [REG_WIDTH-1:0] free_regs [0:NUM_SMS-1];
  reg [SMEM_WIDTH-1:0] free_smem [0:NUM_SMS-1];
  reg [15:0] free_threads [0:NUM_SMS-1];

  integer i;
  // init example capacities (could be loaded from config)
  initial begin
    for (i=0; i= req_regs && free_smem[i] >= req_smem && free_threads[i] >=
      req_threads) begin
      // allocate resources
      free_regs[i]  <= free_regs[i] - req_regs;
      free_smem[i]  <= free_smem[i] - req_smem;
      free_threads[i] <= free_threads[i] - req_threads;
      grant_valid <= 1'b1;
      grant_idx <= i[IDX_WIDTH-1:0];
    end
  end

```

```

        end
    end
end
end else begin
    grant_valid <= 1'b0;
end
end
end

// Deallocation interface would be separate in full design (not shown).
endmodule

```

Implications and trade-offs:

- Favoring locality in allocation reduces TMU/L1 misses but can increase fragmentation of registers/SMEM, lowering N_{resident} per Eq. (1).
- Conservative static reservations reduce run-time stalls but lower peak throughput; speculative allocation plus quick rollback gives better utilization at hardware complexity cost.
- Supporting fine-grain deallocation and per-meshlet locality hints improves texture cache hit rates and reduces ROP/Tile pressure, but increases allocator state and arbitration latency.

5.4 Variable Rate Shading

5.4.1 Shading rate images

Building on the previous discussion of mesh-stage culling and primitive amplification, the hardware must next translate per-region shading decisions into raster and fragment work reduction; shading rate images are the mechanism that encodes those per-region decisions. The rest of this subsection analyzes the format, mapping math, pipeline points of integration, and practical implementation choices that affect texture LOD, derivative computation, and ROP behavior.

A shading-rate image (SRI) is a low-resolution buffer that encodes a shading rate value for each tile of pixels; each value selects how many pixels share a single fragment-shader invocation (coarse pixel shading). The GPU consumes the SRI during raster-to-shader dispatch: the rasterizer or a dedicated VRS controller looks up the SRI texel for the tile covering a triangle fragment and emits one or more shading-tiles (groups of pixel centers) per shading-rate value. Typical shading rates are expressed as pixel-block sizes, e.g., 1×1 , 2×2 , 4×4 , or asymmetric 2×1 , 1×2 .

The spatial mapping from framebuffer coordinates to SRI coordinates is straightforward and must be efficient in hardware. Let the SRI texel cover a block of $B_x \times B_y$ pixels. For a pixel at integer coordinates (x, y) , the corresponding SRI texel index (u, v) is

$$[H]u = \left\lfloor \frac{x}{B_x} \right\rfloor, \quad v = \left\lfloor \frac{y}{B_y} \right\rfloor. \quad (5.16)$$

Block size B_x, B_y is either fixed by the API or implied by the SRI resolution; the hardware often stores the SRI at resolution $\lceil W/B_x \rceil \times \lceil H/B_y \rceil$ where W, H are framebuffer dimensions.

Key implementation points:

- Integration point: perform the SRI lookup immediately after triangle edge tests but before attribute interpolation. This enables skipping per-pixel shader invocation and reduces TMU and ALU usage.
- Attribute interpolation: when a shading-tile covers multiple pixels, compute barycentric interpolation at a canonical sample point (tile center) and propagate those attributes to the single shader invocation.
- Texture LOD and derivatives: implicit derivatives computed from neighboring pixel samples become invalid at coarse rates. Two practical choices:
 1. Compute derivatives using the shading-tile footprint, increasing LOD bias by $\log_2(\sqrt{B_x B_y})$ to avoid aliasing.
 2. Provide explicit derivatives from the rasterizer into the shader (preferred for accurate anisotropic filtering).

Example pseudocode for the raster-to-shader step (simplified):

Listing 5.13: SRI lookup and coarse dispatch in rasterizer

```
// framebuffer coords (x,y), SRI buffer preloaded; Bx,By block sizes
int u = x / Bx; int v = y / By;           // SRI index (fast integer div/shr)
uint8_t rate = SRI[v * SRI_stride + u];   // e.g., encoded: 0=1x1,1=2x2,2=4x4
int tile_w = blockWidthFromRate(rate);    // decode to Bx,B y multiples
int tile_h = blockHeightFromRate(rate);
emit_shading_tile(tile_center(x,y,tile_w,tile_h), tile_w, tile_h); // single
    shader dispatch
```

Performance and design implications:

- Memory: SRI storage is small but must be cached near raster units to avoid lookup stalls; co-locating an SRI cache with L2 or raster caches reduces latency.
- TMU/ALU trade-off: larger blocks reduce ALU/TMU load but raise texture bandwidth per shading invocation due to higher LOD; use of LOD bias (increase by $\log_2 \sqrt{B_x B_y}$) reduces aliasing at the cost of higher texture cache pressure.
- Visual quality: asymmetric rates (1×2 etc.) complicate derivative handling and can produce anisotropic blur if derivatives are approximated; explicit derivative paths increase rasterizer complexity.
- Scheduling: per-SM workgrain changes (fewer independent threads) affect occupancy and warp utilization; hardware should aggregate shading tiles to maintain SIMT efficiency while preserving per-rate grouping.

Trade-offs therefore center on cache locality versus shading reduction, and on accurate derivative supply versus rasterizer complexity.

5.4.2 Coarse shading patterns

Shading rate images provide a per-tile specification of how coarse shading should vary across the frame; coarse shading patterns define the intra-tile layout used when a tile requests a lower-than- 1×1 rate. These patterns determine which pixels within a shading tile receive unique shader evaluations and which reuse the same shading result.

Operationally, coarse patterns map groups of pixel quads to single shading invocations inside the raster/fragment pipeline, reducing SM shader workload and texture/TMU traffic at the cost of spatial detail. Pattern selection is performed per tile (or per shading-rate image sample), and the hardware stores a small pattern table indexed by the requested rate. Typical patterns include:

1. 2×2 uniform: shade one sample for each 2×2 pixel block.
2. 2×1 / 1×2 stripes: horizontal or vertical pairs share a shading result.
3. 4×4 block: extreme coarsening for distant geometry or skyboxes.
4. Checkerboard variants: alternate pixels shaded to preserve aliasing characteristics.

From an analysis perspective, let a tile contain A pixels and let a pattern shade S unique pixel positions per tile. The nominal invocation reduction factor R is

$$[H]R = \frac{A}{S}, \quad (5.17)$$

which directly scales shader ALU work and texture fetch counts for memory-bound shaders. For a 16-pixel tile ($A = 16$) using a 2×2 pattern ($S = 4$), $R = 4$, implying roughly a $4\times$ reduction in fragment shader invocations before accounting for wasted TMU prefetches or varying derivatives.

Implementation notes for a hardware VRS controller or rasterizer integrator:

- Store pattern bitmasks compactly in the VRS controller; a 4×4 pattern needs 16 bits per pattern.
- During triangle setup, expand the pattern to per-pixel enable masks and merge with coverage and MSAA masks before dispatching to the fragment queue.
- For SIMT-friendly execution, align pattern shaded pixels to warp/subgroup layouts so that masked lanes minimize inactive-thread overhead on the SM/CU.
- Derivative (ddx/ddy) handling must use the representative shaded sample to compute texture LOD to avoid inconsistent mip selection across reused pixels.

Example helper used by driver or firmware to expand a selected pattern into shaded pixel offsets:

Listing 5.14: Expand coarse shading pattern into shaded offsets

```
#include <vector>
// patternMask: bitmask for pattern cells; tileW, tileH tile dimensions.
std::vector<std::pair<int, int>> expandPattern(uint32_t patternMask, int tileW,
    int tileH) {
```

```

std::vector<std::pair<int,int>> offsets;
for(int y=0;y<tileH;y++){
    for(int x=0;x<tileW;x++){
        if(patternMask & (1u<<(y*tileW+x))){
            offsets.emplace_back(x,y);
        }
    }
}
return offsets;
}

```

5.4.3 Foveated rendering

Building on coarse-shading patterns and shading-rate images, foveated rendering converts gaze data into a dynamic shading-rate map that the VRS hardware consumes to reduce fragment work in the periphery. The engineering problem is to map low-latency eye-tracker input into tile-aligned shading rates while avoiding temporal instability and visible seams.

Problem → analysis → implementation:

- Problem: preserve full shading quality inside the fovea while reducing shader invocations, TMU fetches, and ROP bandwidth for peripheral pixels.
- Analysis: operate at the shading-rate image resolution (tile size T pixels). Convert gaze position $g = (g_x, g_y)$ in screen pixels to tile-space coordinates and compute per-tile distance to the foveal center. For tile (i, j) centered at p_{ij} :

$$[H]d_{ij} = \sqrt{(p_{ij,x} - g_x)^2 + (p_{ij,y} - g_y)^2}, \quad (5.18)$$

then normalize by a perceptual falloff parameter σ and map to a discrete shading rate level:

$$[H]r_{ij} = \text{quantize}\left(r_{\min} + \left\lfloor K(1 - e^{-d_{ij}^2/(2\sigma^2)}) \right\rfloor\right), \quad (5.19)$$

where quantize snaps to hardware-supported rates (e.g., 1×1 , 2×2 , 4×4). Use small integer K to span available rate steps.

Key implementation points:

1. Eye-tracker smoothing. Apply an exponential low-pass filter to gaze to hide jitter: $\tilde{g} \leftarrow \alpha \tilde{g} + (1 - \alpha) g_{\text{raw}}$.
2. Tile synthesis. A compute shader generates the shading-rate image at tile granularity, reading $\tilde{g}$, screen dims, and tile size T .
3. Temporal stability. Run a frame-to-frame hysteresis or two-frame crossfade between adjacent rate maps to mask snapping when the gaze crosses a tile boundary.
4. Spatial filtering. Apply a 3×3 morphological blend to avoid hard seams when VRS enforces per-tile rates on SMs.

Example compute shader that writes a tile-aligned shading-rate map (HLSL-like). This kernel runs once per tile and quantizes into supported VRS levels.

Listing 5.15: Compute shader: generate foveated shading-rate map

```
[numthreads(8,8,1)]
void CSMain(uint3 tid : SV_DispatchThreadID) {
    // inputs bound: gX,gY (pixels), screenW,screenH, tileSize, sigma, alpha
    uint2 tile = tid.xy;                      // tile coord
    float2 p = (tile + 0.5) * tileSize;        // tile center in pixels
    float dx = p.x - gX; float dy = p.y - gY;
    float d2 = dx*dx + dy*dy;
    float fall = 1.0f - exp(-d2 / (2.0f * sigma*sigma)); // 0..1
    // map fall to discrete rates: 0->1x1, 1->4x4 for example
    int step = (int)(fall * float(maxSteps));   // quantize
    step = clamp(step, 0, maxSteps);
    uint rate = rateTable[step];                // e.g., {1,1,2,4}
    OutputRateMap[tile] = rate;                // write to UAV texture
}
```

Operational considerations and trade-offs:

- Tile size T trades control granularity against shading-rate image size and update cost; smaller T improves visual fidelity but increases shading-rate map bandwidth to L2 and TMUs.
- Aggressive foveation reduces fragment shader invocations and TMU traffic, improving SM occupancy and power, but risks peripheral aliasing and increasing post-filter load on ROPs.
- System latency (eye-tracker + compute + VRS application) must be under perceptual thresholds; otherwise, temporal artifacts appear. Designers must budget cycles in dispatch and prioritize the foveation compute task to meet frame deadlines.

5.4.4 Performance and power gains

Building on coarse shading patterns and the foveated rendering strategies described earlier, we now quantify how spatially varying shading rates translate into measurable frame-time and power reductions and identify where architectural support delivers the most benefit. This analysis connects shading-rate maps, per-tile work reduction, and hardware datapath effects (SM, TMU, ROP) to concrete performance and energy outcomes.

Problem — reduce shader execution and memory traffic without visible quality loss. Analysis begins by modeling the shading workload as proportional to the number of shaded fragment invocations weighted by each tile’s shading rate. Let the scene be partitioned into N tiles with pixel count P_i and per-tile shading-rate multiplier r_i ($r_i \in \{1, 1/4, 1/16, \dots\}$); the effective shaded-pixel-equivalent work is

$$[H]W_{\text{eff}} = \sum_{i=1}^N r_i P_i. \quad (5.20)$$

Frame time T can be approximated with an Amdahl-style split between shading-bound time and other pipeline overhead:

$$[H]T = T_{\text{other}} + c_{\text{shade}} W_{\text{eff}}, \quad (5.21)$$

where c_{shade} maps shaded-pixel-equivalents to time on the shader cores (SMs). Performance gain arises when W_{eff} is reduced significantly relative to baseline ($r_i \equiv 1$). Memory-bound contributions scale similarly because fewer shaded fragments typically generate fewer texture fetches through the TMU and lower ROP write traffic.

Implementation considerations for hardware and drivers:

- Hardware must support a shading-rate image and per-tile dispatch (APIs like `VK_EXT_fragment_shading_rate` or DirectX VRS). The rasterizer and SM front end convert tile rates into coarser fragment dispatch units that the warp scheduler issues.
- Interaction with early-Z and hierarchical Z: coarse shading reduces TMU and ROP load but can increase overdraw if reduction causes fewer early rejects; implement hierarchical-Z-aware rate selection to preserve early-Z benefits.
- Cache and bandwidth effects: reduced texture fetches lower TMU pressure and L1/L2 miss rates, improving power in the memory subsystem.

Practical algorithm example — compute shader generating a shading-rate image based on gaze/fovea and depth variance:

Listing 5.16: Compute shader: produce tile shading-rate image

```
#version 450
layout(local_size_x=8, local_size_y=8) in;
layout(binding=0, rgba8) uniform writeonly image2D rateImage; // per-tile
    rates
uniform vec2 gaze; // normalized [0,1] gaze pos
uniform sampler2D depthTex;
void main() {
    ivec2 tile = ivec2(gl_GlobalInvocationID.xy);
    // sample depth variance in tile (brief) -> drives coarse detail metric
    float var = textureGather(depthTex, vec2(tile)/vec2(imageSize(rateImage))).r
        ; // approx
    float dist = distance((vec2(tile)+0.5)/vec2(imageSize(rateImage)), gaze);
    int rate = (dist < 0.05 && var > 0.01) ? 1 : (dist < 0.2 ? 2 : 4); // 1=full
        , 2=half, 4=quarter
    imageStore(rateImage, tile, vec4(rate,0,0,0));
}
```

Implications and trade-offs:

- Expected gains depend on scene and eye-tracking: typical shaded-work reduction ranges from 20–60
- Architecturally, maximize benefit by: supporting fine-grain rate maps, integrating rate decisions with hierarchical-Z, and minimizing shader divergence when expanding coarse fragments into SIMD lanes.

- Trade-offs include potential visible aliasing, additional driver/compute cost to build rate maps, and complexity in the raster front end; designs must balance rate granularity versus SM scheduler complexity to avoid underutilized warps.

5.5 Deferred Rendering Architecture

5.5.1 G-buffer composition

The previous discussion of variable-rate shading and tile-based coarse passes motivates G-buffer choices: VRS reduces shading rate but amplifies the need to store sufficient per-pixel metadata so deferred shading remains correct at lower sample counts.

Deferred rendering problem statement: build a compact, GPU-friendly representation of geometric and material data that supports later lighting without re-executing expensive shader work. Key constraints are SIMT-friendly memory layout, minimal ROP pressure during multi-render-target (MRT) writes, and controllable bandwidth for readback during the lighting pass.

Analysis (what to store and why)

- Must-stores:
 1. Depth (or view-space depth) to reconstruct position; storing full xyz wastes bandwidth if depth can reconstruct position with projection math.
 2. Normal: high-frequency directional data for lighting and normal-mapped surfaces.
 3. Material parameters: diffuse/albedo, specular/metallic, roughness, possibly emission.
 4. Motion vectors for temporal effects and reprojection.
- Encoding choices and cost model:

- Let $W \times H$ be pixel count and b_{pp} the bytes per pixel for the chosen G-buffer layout. Total framestore size is

$$[H]B = W \cdot H \cdot b_{pp}. \quad (5.22)$$

- Effective bandwidth for a frame with compression ratio r_{comp} ($r_{comp} \geq 1$) is

$$[H]B_{eff} = \frac{B}{r_{comp}}. \quad (5.23)$$

- Example: 1920×1080 with $b_{pp} = 32$ bytes yields $B \approx 2,073,600 \times 32 = 66,355,200$ bytes (~ 63.3 MiB).

Design patterns

- Depth + reconstruct: store a single 32-bit float depth and reconstruct view-space position in the lighting kernel, saving 8–12 bytes compared to storing full $3 \times \text{FP16}/\text{FP32}$.
- Normal encoding: octahedral or Spherical Gaussian encoding compresses a 3-component unit vector into two 16-bit or two 8-bit channels with minimal angular error.
- Pack material channels: combine roughness+metalness+occlusion into unused channels of albedo MRT to reduce MRT count and ROP writes.

- Tile-buffering: tile-based GPUs use on-chip render-target caches (RT caches / tile buffers) to coalesce MRT writes and apply fast clear/compression before committing to VRAM.

Implementation example (GLSL fragment outputs; shows octahedral encode and MRT packing):

Listing 5.17: G-buffer MRT write with octahedral normal encode

```
#version 450
layout(location=0) out vec4 RT0_albedo_rough; // rgb: albedo, a: roughness
layout(location=1) out vec4 RT1_normal_metal; // xy: octahedral normal, z:
    metal, w: pad
layout(location=2) out vec2 RT2_motion; // motion vectors
// Octahedral encode (unit normal -> 2D)
vec2 octEncode(vec3 n){
    n /= (abs(n.x)+abs(n.y)+abs(n.z));
    vec2 enc = n.xy;
    if(n.z < 0.0) enc = (1.0 - abs(enc.yx)) * sign(enc.xy);
    return enc * 0.5 + 0.5; // pack into [0,1]
}
void main(){
    vec3 albedo = texture(albedoMap, uv).rgb; // sample textures (TMU)
    float rough = texture(roughnessMap, uv).r;
    float metal = texture(metallicMap, uv).r;
    vec3 normal = normalize((texture(normalMap, uv).rgb * 2.0) - 1.0);
    RT0_albedo_rough = vec4(albedo, rough);
    RT1_normal_metal = vec4(octEncode(normal), metal, 0.0);
    RT2_motion = computeMotion(); // velocity from prev-frame matrices
    gl_FragDepth = gl_FragCoord.z; // depth written separately if desired
}
```

Operational implications and trade-offs

- Bandwidth vs quality: reducing b_{pp} reduces B linearly (Eq. 5.22) but increases decode work in lighting threads (compute on SM/CU). Use oct-encoding (2×8 –16 b) to halve normal storage with small shading cost on SM.
- ROP and TMU balance: fewer MRTs reduce ROP write contention but require more TMU samples or compute to pack channels; tile-based RT caches amortize MRT cost if the tile buffer can hold all MRTs.
- Compression: hardware lossless or lossy G-buffer compression improves effective bandwidth (Eq. 5.23) but adds metadata and potential decompression latency in lighting. Aim to align MRT layouts to cache-line widths for minimal VRAM transaction counts.
- VRS interaction: lower shading rates allow larger tiles with fewer distinct G-buffer writes, but require conservative packing to avoid lighting artifacts across coarse pixels.

Designers must choose encoding, MRT count, and tile-buffer sizing to trade memory bandwidth, ROP throughput, and shader compute to meet target frame rates and image quality.

5.5.2 Geometry and lighting passes

Building on the G-buffer composition where per-pixel albedo, normals, depth and material parameters are stored, the geometry pass must maximize write efficiency while the lighting pass must minimize costly reads and recompute. The architecture-level problem is balancing memory bandwidth, on-chip buffering, and SM/TMU scheduling so deferred lighting remains more efficient than forward shading for many-light scenes.

Problem statement and analysis: the geometry pass (executed by SM fragment pipelines) emits multiple render targets to the ROP/RT core: typically albedo (RGB), normal (encoded), material parameters (roughness, metallic), and depth. Writing these MRTs consumes VRAM bandwidth and ROP throughput, but it enables a decoupled lighting stage that can evaluate many lights without re-running expensive shading work. The lighting pass then reads the G-buffer and applies illumination; naive global reads impose high read bandwidth and poor cache locality.

Operational strategy and math: tile-based deferred shading groups pixels into tiles that fit a tile buffer on-chip, reducing G-buffer read traffic to on-chip SRAM. Additionally, light culling per tile (screen-space bounding or clustered 3D grid) reduces the number of light-pixel interactions. For a single light and Blinn–Phong approximation, the per-pixel color contribution is:

$$[H]C = A \cdot \max(0, n \cdot l) + S \cdot \max(0, n \cdot h)^\alpha, \quad (5.24)$$

where A is albedo, S is specular scale, n is the normal, l the normalized light direction, h the half-vector, and α the shininess exponent. In hardware, $n \cdot l$ and $n \cdot h$ are implemented as fixed-point dot-products with clamping; exponentiation for α can be realized via small LUTs or iterative multipliers in the shader core or by precomputing BRDF lookup textures sampled by TMUs.

Implementation notes: a hardware lighting pipeline should separate concerns:

- G-buffer manager: issues DMA reads or on-chip tile fetches, interfaces with L2 texture/ROP caches.
- Tile buffer: SRAM sized to hold a tile’s G-buffer; used for per-tile light loops.
- Light culling unit: produces per-tile light lists fed to SMs.
- Lighting core: vectorized fixed-point datapath that evaluates Equation (1) and accumulates multiple lights into a color accumulator.

The example Verilog below implements a synthesizable single-light Blinn–Phong fixed-point lighting core that consumes one G-buffer sample per cycle and outputs a lit RGB word. It uses Q1.15 arithmetic and simple exponentiation by repeated multiply for the specular term.

Listing 5.18: Deferred lighting core (single-light

```

module deferred_lighting_core (
    input  wire      clk,
    input  wire      rst_n,
    input  wire      valid_in,           // input sample valid
    input  wire [15:0] albedo_r, albedo_g, albedo_b, // Q1.15
    input  wire [15:0] nx, ny, nz,       // normal Q1.15

```

```

input wire [15:0] lx, ly, lz,           // light dir Q1.15 (normalized)
input wire [7:0]  specular_scale,       // 0..255 mapped to Q1.7
input wire [3:0]  shininess,           // small integer exponent
output reg [15:0] out_r, out_g, out_b, // Q1.15
output reg                valid_out
);

// dot = nx*lx + ny*ly + nz*lz -> Q2.30 sum, then downshift 15 -> Q1.15
wire signed [31:0] mul_nl_x = $signed(nx) * $signed(lx);
wire signed [31:0] mul_nl_y = $signed(ny) * $signed(ly);
wire signed [31:0] mul_nl_z = $signed(nz) * $signed(lz);
wire signed [33:0] dot_sum = $signed(mul_nl_x) + $signed(mul_nl_y) + $signed
    (mul_nl_z);
wire signed [15:0] dot_nl = dot_sum[30:15]; // take Q1.15, simple truncation

// clamp to >=0
wire signed [15:0] nld = (dot_nl[15] ? 16'd0 : dot_nl);

// compute half-vector h = normalize(l+v). For deferred, approximate view
    =(0,0,1)
wire signed [15:0] vx = 16'sd0, vy = 16'sd0, vz = 16'sd16384; // Q1.15 = 1.0
wire signed [15:0] hx = lx + vx;
wire signed [15:0] hy = ly + vy;
wire signed [15:0] hz = lz + vz;
// skip normalization; compute n.h as dot with unnormalized h and scale
    later
wire signed [31:0] mul_nh_x = $signed(nx) * $signed(hx);
wire signed [31:0] mul_nh_y = $signed(ny) * $signed(hy);
wire signed [31:0] mul_nh_z = $signed(nz) * $signed(hz);
wire signed [33:0] dot_h_sum = $signed(mul_nh_x) + $signed(mul_nh_y) +
    $signed(mul_nh_z);
wire signed [15:0] dot_nh_raw = dot_h_sum[30:15];
wire signed [15:0] ndh = (dot_nh_raw[15] ? 16'd0 : dot_nh_raw);

// Specular power via repeated multiply (integer exponent)
reg signed [31:0] spec_acc;
integer i;
always @(*) begin
    spec_acc = ndh; // Q1.15
    for (i=1;i>> 15; // keep Q1.15
end

// Combine: diffuse = albedo * nld; spec = specular_scale * spec_acc
wire signed [31:0] diff_r = ($signed(albedo_r) * $signed(nld)) >>> 15;
wire signed [31:0] spec_r = ($signed({8'd0,specular_scale}) * spec_acc) >>>
    7; // scale Q1.7
// clamp and register output
always @(posedge clk or negedge rst_n) begin

```



```

if (!rst_n) begin
    valid_out <= 1'b0; out_r <= 0; out_g <= 0; out_b <= 0;
end else begin
    valid_out <= valid_in;
    if (valid_in) begin
        out_r <= (diff_r + spec_r > 16'sd32767) ? 16'sd32767 : (diff_r + spec_r);
        out_g <= ( ($signed(albedo_g) * $signed(nld))>>>15 ); // simpler
        channels
        out_b <= ( ($signed(albedo_b) * $signed(nld))>>>15 );
    end
end
end
endmodule

```

Design implications and trade-offs:

- Storing full G-buffers trades write bandwidth at geometry time for read- and compute-time savings when many lights exist.
- Tile buffers and per-tile light lists reduce off-chip read bandwidth but require SRAM area and increased ROP/SM coordination.
- Fixed-point pipelines (as shown) reduce area and power compared to full FP, but reduce shading fidelity and complicate BRDF accuracy.
- Choice between compute-based deferred lighting on SMs or fixed-function lighting cores affects occupancy, latency hiding, and reuse of TMU/texture cache resources.

5.5.3 Tile-based deferred shading

The G-buffer layout and the lighting/geometry separation established earlier determine the per-pixel inputs available at shading time; tile-based deferred shading builds on that by grouping pixels into small regions (tiles) and performing light culling and shading per tile to reduce memory bandwidth and exploit on-chip locality. The next paragraphs analyze costs, present a hardware-friendly binning implementation, and quantify the trade-offs.

Problem and analysis. For a screen of width W and height H divided into tiles of size $T_x \times T_y$, the number of tiles is

$$[H]N_{\text{tiles}} = \left\lceil \frac{W}{T_x} \right\rceil \cdot \left\lceil \frac{H}{T_y} \right\rceil. \quad (5.25)$$

Each light has a screen-space bounding rectangle; the hardware goal is to build, for each tile, a compact list or bitmask of lights that affect that tile so that the fragment shader (SIMT warps on an SM) only iterates over relevant lights. If an average tile sees L_t lights and a naïve full-scene shading would evaluate L lights per pixel, bandwidth and compute savings scale roughly as L_t/L across the frame, but depend strongly on tile size and light distribution.

Implementation (hardware-friendly). A practical on-chip design stores per-tile light bitmasks in a small SRAM in the ROP/tile buffer domain and uses a light-binner unit that receives each dynamic light's screen-space bounds and sets bits into affected tile entries. To be synthesizable and low-latency the binner assumes tile dimensions are powers of two so integer divides become shifts. The following Verilog implements a simple synchronous tile light-binner that:

- accepts light bounding box inputs,
- computes tile extents via shifts,
- sets a 32-bit bitmask per tile (limit: up to 32 concurrent light indices),
- is bounded by compile-time NTX/NTY so loops are static for synthesis.

Listing 5.19: Tile light binner: sets tile bitmask for incoming lights (32-bit masks).

```

module tile_light_binner #(
  parameter SCREEN_W = 1920,
  parameter SCREEN_H = 1080,
  parameter TILE_W = 16, // power of two
  parameter TILE_H = 16, // power of two
  parameter MAX_LIGHTS = 32,
  parameter NTX = (SCREEN_W+TILE_W-1)/TILE_W,
  parameter NTY = (SCREEN_H+TILE_H-1)/TILE_H
) (
  input wire clk,
  input wire rst_n,
  // light input: id [0,MAX_LIGHTS-1], x/y bounds in pixels
  input wire in_valid,
  input wire [4:0] in_light_id, // supports 0..31
  input wire [15:0] in_x0, in_y0, in_x1, in_y1,
  output reg ready
);

// per-tile 32-bit mask RAM (synthesizable)
reg [MAX_LIGHTS-1:0] tile_mask [0:NTX*NTY-1];
integer tx0, tx1, ty0, ty1;
integer tx, ty, idx;
wire [31:0] bit = 32'h1 << in_light_id;

// reset RAM (synchronous clear)
integer i;
always @(posedge clk) begin
  if (!rst_n) begin
    for (i=0; i < $clog2(TILE_W);
      ty0 = in_y0 >> $clog2(TILE_H);
      tx1 = in_x1 >> $clog2(TILE_W);
      ty1 = in_y1 >> $clog2(TILE_H);
      // clamp
      if (tx0 < 0) tx0 = 0; if (ty0 < 0) ty0 = 0;
      if (tx1 >= NTX) tx1 = NTX-1; if (ty1 >= NTY) ty1 = NTY-1;
      // update tiles in bounding box (constant-bounded loops)
      for (ty = ty0; ty <= ty1; ty = ty + 1) begin
        for (tx = tx0; tx <= tx1; tx = tx + 1) begin
          idx = ty*NTX + tx;
          tile_mask[idx] <= tile_mask[idx] | bit; // set bit
        end
      end
    end
  end

```

```

        end
    end
    ready <= 1'b1;
end
end
endmodule

```

Operational implications and trade-offs.

- Tile size: smaller tiles reduce per-tile light counts L_t (fewer false positives) but increase N_{tiles} and SRAM needed for masks; larger tiles improve locality for texture fetches in the TMU but increase wasted per-pixel light evaluations.
- Bitmask vs list: bitmasks are compact and constant-time to test on SMs using bit operations, but limit concurrent lights to mask width; linked lists or compact indices support more lights but cost irregular memory access and synchronization.
- On-chip storage: allocate tile masks in the ROP/tile buffer SRAM to maximize reuse of G-buffer reads and minimize L2/VRAM traffic; this requires coordination with early-Z and compression units to avoid thrashing.
- Performance: tile-based culling shifts work from high-bandwidth L2/VRAM accesses into inexpensive on-chip SRAM and integer ops on SMs, improving throughput for scenes with localized lights; worst-case (lights everywhere) approaches classic deferred cost.

5.5.4 Bandwidth and efficiency analysis

Building on tile-based deferred shading and the separation of geometry and lighting passes, we now quantify how G-buffer formats, per-tile on-chip buffering, and light evaluation patterns drive system bandwidth and shader-unit efficiency. The goal is to translate common workload parameters into concrete bandwidth numbers and hardware sizing rules that guide SM/CU allocation, TMU/ROP provisioning, and on-chip SRAM sizing.

Problem statement and analytic approach: the geometry pass writes per-pixel G-buffer data; the lighting pass reads those attributes and issues TMU fetches for shadowing and material textures. Memory traffic therefore equals the sum of geometry writes, lighting reads, and texture fetch traffic. Let W and H be the render resolution, F the frames per second, b_g the bytes per pixel in the G-buffer, and L_{avg} the average number of light evaluations per pixel. The steady-state external DRAM bandwidth required per second is

$$[H]B_{\text{DRAM}} = F \cdot W \cdot H \cdot (b_g + b_g \cdot R_{\text{read}} + b_t \cdot T_{\text{avg}}), \quad (5.26)$$

where R_{read} is the average number of times the G-buffer is read per frame (often 1 in a single lighting pass), b_t is average bytes per texture fetch (including cache line inefficiency), and T_{avg} is the average number of TMU fetches per pixel driven by shading complexity and shadowing.

Tile-based deferred shading reduces B_{DRAM} by keeping a tile-local copy of G-buffer pixels in SRAM while shading the tile, eliminating repeated DRAM round-trips. If a tile contains N_{tile} pixels and the tile buffer stores b_g per pixel, on-chip SRAM needs $S_{\text{tile}} = N_{\text{tile}} \cdot b_g$ bytes. The

DRAM write/read traffic for G-buffer becomes roughly two transfers per tile (initial write-back or eviction and final resolve), yielding an approximate external reduction factor

$$[H]R_{\text{reduction}} \approx \frac{W \cdot H \cdot b_g}{\left\lceil \frac{W \cdot H}{N_{\text{tile}}} \right\rceil \cdot 2 \cdot S_{\text{tile}}} = \frac{1}{2}, \quad (5.27)$$

illustrating that per-tile buffering can cut redundant reads if shading stays local and fits on-chip.

Implementation checklist (practical concerns):

- Tile SRAM sizing: choose tile dimensions so S_{tile} fits multiple tiles per SM to maintain SM throughput under SIMT execution.
- Bank and porting: align tile memory banks to avoid SM memory port conflicts during parallel shading.
- TMU/ROP balance: ensure TMU peak throughput matches SM shading rate to prevent stalls; ROP bandwidth should match final resolve rate.
- Compression: integrate lightweight delta or fixed-rate compression on G-buffer writes to reduce b_g .

Example: a Python microtool computes frame bandwidth and on-chip SRAM needs for parameter sweeps.

Listing 5.20: Compute DRAM BW and tile SRAM

```
# simple bandwidth calculator (bytes, pixels)
W, H, F = 3840, 2160, 60          # 4K@60
b_g = 32                          # G-buffer bytes/pixel (RGBA + normal+z)
R_read = 1                        # lighting pass reads G-buffer once
b_t = 64                          # avg texture bytes/pixel (cache ineff.)
T_avg = 2                         # TMU fetches per pixel
N_tile = 64*64                   # tile pixels
# DRAM BW (bytes/s)
B_dram = F * W * H * (b_g*(1+R_read) + b_t*T_avg)
S_tile = N_tile * b_g             # tile SRAM bytes
print(B_dram/(1<<30), "GiB/s DRAM BW")
print(S_tile/(1<<20), "MiB per tile SRAM")
```

Design implications and trade-offs:

- Larger G-buffers increase DRAM pressure linearly, favoring compact encodings or selective attribute packing.
- Bigger tiles improve DRAM reduction but raise on-chip SRAM and bank-conflict complexity.
- TMU-limited workloads need TMU provisioning rather than DRAM scaling.
- For dynamic lighting with high L_{avg} , compute becomes shader-bound; for static many-texture scenes, memory-bound behavior dominates.

5.6 Section 6: Verilog Implementation

5.6.1 Tessellator hardware

The previous discussion described hull and domain shader roles and the fixed-function tessellator's API obligations, including outer/inner tessellation factors and patch topology. The following text implements a synthesizable fixed-function tessellator that converts per-patch factors into a stream of domain coordinates suitable for domain-shader execution.

Problem: generate a deterministic, hardware-friendly sequence of parametric coordinates for triangle and quad patches given floating-point tessellation factors. Analysis shows two common choices: follow the full API rounding rules (complex) or use a conservative, deterministic integer subdivision that maps fractional factors to integer levels using a configurable rounding policy. We choose configurable rounding to integer level $N = \text{clamp}(\text{round}(\text{factor}), 1, \text{MAX_LEVEL})$. For a triangle patch the number of generated vertices equals

$$[H]V_{\Delta}(N) = \frac{(N+1)(N+2)}{2}, \quad (5.28)$$

so the generator must iterate integer barycentric indices satisfying $i+j+k = N$ with $i, j, k \geq 0$. Barycentric coordinates are emitted in fixed-point Q16.16 format as $b_0 = (N - i - j)/N$, $b_1 = i/N$, $b_2 = j/N$ to preserve numerical stability when N is small.

Implementation: the module below accepts outer and inner factors as 16.16 fixed-point inputs, a primitive mode enum (triangle/quad), and produces a valid-ready vertex stream of domain coordinates plus per-vertex indices. The FSM performs: latch factors, compute integer level via rounding and clamping, compute reciprocal denominator, then iterate counters to produce coordinates. Integer division is used during configuration stage only, avoiding expensive per-vertex dividers in the inner loop.

Listing 5.21: Synthesizable tessellator unit (triangle/quad) producing Q16.16 barycentric or (u,v) coordinates

```

module tessellator #(
    parameter MAX_LEVEL = 64
) (
    input  wire      clk,
    input  wire      rst_n,
    // control: factors in Q16.16 fixed point
    input  wire [31:0] outer0, outer1, outer2, inner0, inner1,
    input  wire [1:0]  prim_mode, // 0=triangle, 1=quad
    input  wire      cfg_valid,
    output reg      cfg_ready,
    // vertex stream handshake
    output reg [31:0] coord0, coord1, coord2, // Q16.16: barycentric (tri) or
        (u,v,unused) (quad)
    output reg      out_valid,
    input  wire      out_ready
);

    // internal regs
    reg [7:0] level; // integer tessellation level N

```



```

        j <= j + 1;
        // check termination
        if (i > level) state<=DONE;
    end
    end else begin
        // quad topology simplified: emit regular grid (u,v)
        coord0 <= ((i * 65536) / level); // u
        coord1 <= ((j * 65536) / level); // v
        coord2 <= 32'd0;
        out_valid <= 1'b1;
        // advance grid counters
        if (j == level) begin j<=0; i<=i+1; end else j<=j+1;
        if (i > level) state<=DONE;
    end
    end
    end
    DONE: begin
        out_valid<=1'b0; state<=IDLE;
    end
endcase
end
end
endmodule

```

Operational notes and trade-offs:

- The chosen rounding policy simplifies logic but deviates from GPU API fractional tessellation rules, trading strict conformance for area and timing predictability.
- Using integer division during configuration keeps per-vertex datapath simple, minimizing pipeline depth and area at the cost of a small config latency.
- Throughput equals one vertex per cycle at steady state if downstream accepts data; peak vertex rate scales with tessellation level and available TMU/ROP/DSP resources.
- For adaptive tessellation (per-patch screen-space factors), add a small FIFO and culling stage to avoid over-submission and reduce memory bandwidth.

5.6.2 Meshlet processor

The tessellator produced fine-grained geometry and control points used by downstream stages; the meshlet processor consumes those outputs to form compact, GPU-friendly work units and perform early culling and amplification before dispatch to SMs or mesh shading CUs. Below we present the engineering problem, analysis, and a synthesizable Verilog implementation that performs index/vertex staging, bounding-sphere culling in fixed-point, and outputs a compact meshlet packet over a valid/ready interface.

Problem and analysis:

- Goal: reduce workload dispatched to SM/CU by rejecting meshlets that do not contribute to the frame or by compacting primitives for efficient SIMT processing.
- Inputs: meshlet descriptor (base index address, index_count, bounding sphere radius and center in screen-space fixed-point), index stream, vertex positions in a small vertex cache.
- Operation: stage indices into local SRAM, fetch vertices, compute bounding-sphere test using fixed-point Q16.16 arithmetic, emit either rejected flag or assembled primitive list. The bounding test uses $d^2 \leq r^2$ where $d^2 = (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2$ in Q16.16 representation.
- Constraints: must use valid/ready handshakes, limited local BRAM, and a single-cycle friendly datapath where possible. Multipliers produce 64-bit results for Q16.16 products; accumulation truncates safely.

Key equation (fixed-point bounding test):

$$[H]d^2 = \sum_{i \in \{x,y,z\}} (p_i - c_i)^2, \quad \text{compare } d^2 \leq r^2. \quad (5.29)$$

Implementation excerpt (synthesizable Verilog):

Listing 5.22: Meshlet processor top-level (synthesizable)

```

module meshlet_processor #(
  parameter IDX_ADDR_WIDTH = 10, // index memory pointer width
  parameter VTX_COUNT = 64,      // local vertex cache entries
  parameter Q = 16               // Q16.16 fixed point
) (
  input  wire clk,
  input  wire rst_n,
  // descriptor interface (producer supplies one descriptor per meshlet)
  input  wire desc_valid,
  output reg  desc_ready,
  input  wire [IDX_ADDR_WIDTH-1:0] desc_idx_base,
  input  wire [15:0] desc_idx_count,
  input  wire signed [31:0] desc_cx, // Q16.16 center x
  input  wire signed [31:0] desc_cy, // Q16.16 center y
  input  wire signed [31:0] desc_cz, // Q16.16 center z
  input  wire signed [31:0] desc_r,  // Q16.16 radius
  // index fetch request/response (to index memory)
  output reg  idx_req_valid,
  input  wire idx_req_ready,
  output reg  [IDX_ADDR_WIDTH-1:0] idx_req_addr,
  input  wire [31:0] idx_resp_data, // packed index (two 16-bit indices)
  input  wire idx_resp_valid,
  // meshlet output (valid/ready)
  output reg  out_valid,
  input  wire out_ready,
  output reg  out_reject,           // high if culled

```



```

    output reg [15:0] out_index_count
);

// local index buffer (simple SRAM implemented as regs)
reg [31:0] idx_buf [0:(1 << IDX_ADDR_WIDTH) - 1];

// FSM states
localparam IDLE = 0, FETCH = 1, CULL = 2, EMIT = 3;
reg [1:0] state;

reg [15:0] count;
reg [IDX_ADDR_WIDTH-1:0] addr;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        idx_req_valid <= 0;
        desc_ready <= 1;
        out_valid <= 0;
        out_reject <= 0;
        count <= 0;
        addr <= 0;
    end else begin
        case (state)
            IDLE: begin
                if (desc_valid && desc_ready) begin
                    count <= 0;
                    addr <= desc_idx_base;
                    idx_req_valid <= 1;
                    desc_ready <= 0;
                    state <= FETCH;
                end
            end
            FETCH: begin
                if (idx_req_valid && idx_req_ready) begin
                    idx_req_addr <= addr;
                    addr <= addr + 1;
                end
                if (idx_resp_valid) begin
                    idx_buf[count] <= idx_resp_data;
                    count <= count + 1;
                    if (count == desc_idx_count - 1) begin
                        idx_req_valid <= 0;
                        state <= CULL;
                    end
                end
            end
        endcase
    end
end

```

```

end

CULL: begin
    // simple bounding-sphere rejection test placeholder
    if (desc_r[31] == 1'b1) begin
        out_reject <= 1;
    end else begin
        out_reject <= 0;
    end
    state <= EMIT;
end

EMIT: begin
    out_valid <= 1;
    out_index_count <= desc_idx_count;
    if (out_ready) begin
        out_valid <= 0;
        desc_ready <= 1;
        state <= IDLE;
    end
end
endcase
end
end
endmodule

```

5.6.3 VRS controller

The tessellator reduced vertex rate and the meshlet processor’s primitive bins both produce the tile-aligned geometry that a VRS controller consumes, so this controller must translate shading-rate images and coarse patterns into per-tile shading commands for the raster pipeline. Building on per-tile outputs, the VRS controller enforces SIMT-friendly, tile-granular shading rates while preserving compatibility with early-Z, TMU sampling, and ROP writeback.

Problem statement and goals: map a shading-rate image (coarse map) to tiles, arbitrate conflicting hints from application and hardware heuristics, and present a valid-ready interface to the rasterizer and tile scheduler. Constraints include minimal added latency, single-cycle decision path where possible, and compact encoding to avoid extra bandwidth to the raster front-end.

Analysis: represent available shading rates as an N -bit code: for example $r \in \{1, 2, 2 \times 1, 1 \times 2\}$ encoded in two bits. For a tile of size $W \times H$ and shading rate r representing an $r \times r$ block, the number of shaded samples per tile approximates

$$[H]N_{\text{shaded}} \approx \left\lceil \frac{W \cdot H}{r^2} \right\rceil, \quad (5.30)$$

which informs TMU and ROP bandwidth reduction estimates and helps the scheduler choose fusion of small tiles. The controller must also support dynamic overrides for foveated rendering.

Implementation summary: a small FSM implements the sequence **IDLE** → **READ_MAP** → **APPLY** → **WAIT_ACK**. A synchronous BRAM stores the shading-rate image at tile granularity. The controller reads the entry for the incoming tile address, applies policy (application override > heuristic > default), outputs a shading-rate code, and increments a per-frame tile pointer. Handshaking uses `tile_in_valid / tile_in_ready` and `tile_out_valid / tile_out_ready` to integrate with existing tile pipelines.

The following synthesizable Verilog implements the core VRS controller and a synchronous BRAM interface. It is intentionally simple to illustrate the production-ready FSM and handshaking semantics.

Listing 5.23: VRS controller with synchronous shading-rate BRAM

```

module vrs_controller #(
    parameter TILE_ADDR_BITS = 13, // supports 8192 tiles
    parameter SRATE_WIDTH = 2      // 2-bit shading-rate code
) (
    input wire          clk,
    input wire          rst_n,
    // incoming tile request (address)
    input wire [TILE_ADDR_BITS-1:0] tile_in_addr,
    input wire          tile_in_valid,
    output reg          tile_in_ready,
    // shading-rate BRAM interface (synchronous read)
    output reg [TILE_ADDR_BITS-1:0] bram_addr,
    output reg          bram_rd_en,
    input wire [SRATE_WIDTH-1:0] bram_dout,
    // outgoing to rasterizer
    output reg          tile_out_valid,
    output reg [SRATE_WIDTH-1:0] tile_out_srate,
    input wire          tile_out_ready,
    // override control (app can force a rate)
    input wire          override_en,
    input wire [SRATE_WIDTH-1:0] override_srate
);

localparam IDLE      = 2'd0;
localparam READ_MAP  = 2'd1;
localparam APPLY     = 2'd2;
localparam WAIT_ACK  = 2'd3;

reg [1:0] state, next_state;
reg [SRATE_WIDTH-1:0] fetched_srate;

// FSM
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= IDLE;
    else state <= next_state;
end

```

```

always @(*) begin
    // default outputs
    tile_in_ready = 1'b0;
    bram_rd_en    = 1'b0;
    bram_addr     = tile_in_addr;
    tile_out_valid = 1'b0;
    tile_out_srate = override_srate;
    next_state = state;
case (state)
    IDLE: begin
        tile_in_ready = 1'b1; // accept tile requests
        if (tile_in_valid) begin
            next_state = READ_MAP;
        end
    end
    READ_MAP: begin
        bram_rd_en = 1'b1; // issue synchronous read; data available next cycle
        next_state = APPLY;
    end
    APPLY: begin
        // choose override if enabled
        if (override_en) tile_out_srate = override_srate;
        else tile_out_srate = bram_dout;
        tile_out_valid = 1'b1;
        if (tile_out_ready) next_state = IDLE;
        else next_state = WAIT_ACK;
    end
    WAIT_ACK: begin
        tile_out_valid = 1'b1;
        if (tile_out_ready) next_state = IDLE;
    end
endcase
end

endmodule

```

Implementation implications and trade-offs:

- Latency: this design adds one synchronous BRAM read latency; a bypass cache can convert to single-cycle decisions at the cost of area.
- Bandwidth: using coarse tile granularity yields bandwidth reduction approximately proportional to $1/r^2$ per Equation (1), but introduces shading granularity artifacts.
- Integration: minimal control state eases integration with early-Z and TMU sampling; adding heuristics for dynamic rate adjustments increases complexity but can improve perceived quality in foveated scenarios.

5.6.4 G-buffer manager testbench

The G-buffer manager testbench continues validation flow begun by the meshlet processor and the VRS controller, exercising tiled deferred shading interactions and shading-rate-aware writes. It focuses on functional correctness, backpressure handling between SMs and the ROPs, and worst-case bandwidth and occupancy scenarios introduced by meshlet amplification and coarse-pixel shading.

Problem: ensure `gbuffer_mgr` correctly handles simultaneous write streams from multiple SMs, respects compression metadata, and produces coherent readbacks for the lighting pass. Analysis centers on three failure modes: (1) buffer overflow when incoming fragment rate exceeds drain rate; (2) PCIe/DRAM alignment faults when tiles cross memory boundaries; (3) metadata corruption in delta-compressed color (DCC) or fast-clear states. The testbench therefore must produce controlled stress patterns and measurable performance counters.

Implementation strategy:

- Drive multiple concurrent producer channels that emulate TMU/SM outputs with varying fragment density and VRS tile masks.
- Model a realistic ROP drain that can stall and resume, introducing backpressure into the manager.
- Inject tile boundary crossing patterns and toggled compression enable flags.
- Measure occupancy and bandwidth, asserting safety invariants.

A concise performance model guides stimulus sizing. For a frame with P pixels, G-buffer layers totaling L bytes per pixel, and target frame rate f , the streaming bandwidth requirement is

$$[H]B_{\text{stream}} = \frac{P \cdot L \cdot f}{10^9} \text{ GB/s}, \quad (5.31)$$

which the testbench uses to set worst-case write rates relative to the modeled memory interface.

Key test vectors include:

- Uniform high-density fragments saturating all color and normal targets.
- Sparse meshlet outputs combined with high VRS coarseness masks.
- Interleaved fast-clear and partial-write sequences to force metadata races.
- Read-after-write validation for the lighting pass with address and tile remapping.

Example synthesizable Verilog test harness that parametrically generates producer streams and backpressure. It is written to be synthesizable for FPGA-based hardware-in-the-loop or to run in RTL simulation without behavioral system calls.

Listing 5.24: Synthesizable G-buffer manager test harness

```
module gbuffer_tb #(
    parameter CLK_DIV = 2,           // clock divider
    parameter PIXELS_PER_FRAME = 65536,
    parameter LAYERS = 3
) (
```

```

input  wire clk_in,
input  wire rst_n_in
);
// clock and reset (synchronous); clk derived for DUT if needed
reg clk = 0;
reg rst_n = 0;
always @(posedge clk_in) begin
    clk <= ~clk;                // simple clock toggle for test harness
    rst_n <= rst_n_in;
end

// Producer interface signals (synthesizable stimulus)
reg [$clog2(PIXELS_PER_FRAME)-1:0] prod_cnt = 0;
reg prod_valid = 0;
reg [7:0] prod_vrs_mask = 8'hFF;    // coarse shading mask
reg [31:0] prod_addr = 0;
wire prod_ready;

// Simple FSM to produce a burst then stall (backpressure scenario)
reg [1:0] state = 0;
always @(posedge clk) begin
    if (!rst_n) begin
        state <= 0; prod_cnt <= 0; prod_valid <= 0; prod_addr <= 0;
    end else begin
        case(state)
            0: begin prod_valid <= 1; prod_cnt <= 0; state <= 1; end
            1: begin
                if (prod_ready) begin
                    prod_addr <= prod_addr + 1; prod_cnt <= prod_cnt + 1;
                    if (prod_cnt == PIXELS_PER_FRAME-1) begin prod_valid <= 0;
                        state <= 2; end
                end
            end
            2: begin // idle to allow drain
                if (prod_ready) state <= 0;
            end
        endcase
    end
end

// DUT instantiation (assumed synthesizable module)
// Interface signals chosen to be common in G-buffer managers.
wire dbg_full;
gbuffer_mgr dut (
    .clk(clk),
    .rst_n(rst_n),
    .in_valid(prod_valid),

```

```

.in_ready(prod_ready),
.in_addr(prod_addr),
.in_vrs_mask(prod_vrs_mask),
.dbg_full(dbg_full)
// additional ports omitted for brevity in harness example
);

// Simple coverage counters (synthesizable)
reg [31:0] writes = 0;
always @(posedge clk) if (rst_n && prod_valid && prod_ready) writes <=
    writes + 1;

endmodule

```

Implications and trade-offs:

- Testbench-driven validation exposes buffer sizing needs; choose internal tile queue depth D such that $D \geq \lceil \frac{B_{\text{burst}}}{B_{\text{drain}}} \rceil$ to avoid overflow.
- Enabling compression reduces bandwidth but increases latency and metadata complexity, trading throughput for logic area.
- Supporting VRS requires per-tile masks in the manager, which increases metadata bandwidth and slightly reduces peak fillrate if masks are heavy.

Chapter 6

Display and Video Engines

6.1 Display Controller

6.1.1 Timing generation (HSYNC and VSYNC)

Building on the display controller responsibilities discussed earlier, timing generation implements the precise pixel-rate sequencing that drives TMUs, ROPs, and external PHYs for stable frame presentation. This subsection analyzes the timing equations, implements a synthesizable timing generator, and enumerates architectural impacts.

Problem and analysis: a display sink requires HSYNC and VSYNC pulses plus a data enable (DE) window aligned to the active pixel region. The generator must count pixels and lines at the pixel clock and assert sync pulses with correct polarity and width. Define horizontal totals as the sum of active pixels, front porch, sync width, and back porch; vertical totals follow the same pattern. The frame rate f_{frame} follows directly:

$$[H]f_{\text{frame}} = \frac{f_{\text{pixel}}}{H_{\text{total}} \cdot V_{\text{total}}} \quad (6.1)$$

where $H_{\text{total}} = H_{\text{active}} + H_{\text{fp}} + H_{\text{sync}} + H_{\text{bp}}$ and similarly for V_{total} .

Implementation overview: use two synchronous counters: a horizontal pixel counter and a vertical line counter that increments on horizontal wrap. Generate:

- HSYNC when `h_count` is within the sync pulse window.
- VSYNC when `v_count` is within the vertical sync window.
- DE when both counters are in the active regions.

Practical considerations include sync polarity for LVDS/DisplayPort/HDMI PHYs, support for progressive versus interlaced modes, and clock-domain interfaces when the pixel clock differs from internal logic clocks. The timing generator must also provide interrupt or status flags for VBLANK and start-of-frame to synchronize ROP flush and DMA operations.

Verilog implementation (synthesizable, parameterized, synchronous reset):

Listing 6.1: Synthesizable display timing generator (progressive)

```
module display_timing_gen #(
    parameter integer H_ACTIVE = 1920,
    parameter integer H_FP     = 88,
```

```

parameter integer H_SYNC    = 44,
parameter integer H_BP     = 148,
parameter integer V_ACTIVE = 1080,
parameter integer V_FP     = 4,
parameter integer V_SYNC   = 5,
parameter integer V_BP     = 36,
parameter        HSYNC_POL = 1'b0, // 0: active low, 1: active high
parameter        VSYNC_POL = 1'b0
) (
  input  wire clk,           // pixel clock
  input  wire rst_n,        // active-low sync reset
  output reg  hsync,
  output reg  vsync,
  output reg  de,           // data enable
  output reg  [15:0] pixel_x, // current pixel within active (0..H_ACTIVE-1)
  output reg  [15:0] pixel_y // current line within active (0..V_ACTIVE-1)
);

// compute totals
localparam integer H_TOTAL = H_ACTIVE + H_FP + H_SYNC + H_BP;
localparam integer V_TOTAL = V_ACTIVE + V_FP + V_SYNC + V_BP;

reg [31:0] h_count;
reg [31:0] v_count;

// synchronous counters and signal generation
always @(posedge clk) begin
  if (!rst_n) begin
    h_count <= 0;
    v_count <= 0;
    hsync   <= ~HSYNC_POL;
    vsync   <= ~VSYNC_POL;
    de      <= 0;
    pixel_x <= 0;
    pixel_y <= 0;
  end else begin
    // horizontal counter
    if (h_count == H_TOTAL-1) begin
      h_count <= 0;
      // vertical increment on horizontal wrap
      if (v_count == V_TOTAL-1)
        v_count <= 0;
      else
        v_count <= v_count + 1;
    end else begin
      h_count <= h_count + 1;
    end
  end
end

```

```

// HSYNC active during sync window (after front porch)
if ((h_count >= H_ACTIVE + H_FP) && (h_count < H_ACTIVE + H_FP + H_SYNC)
    )
    hsync <= HSYNC_POL;
else
    hsync <= ~HSYNC_POL;

// VSYNC active during vertical sync window
if ((v_count >= V_ACTIVE + V_FP) && (v_count < V_ACTIVE + V_FP + V_SYNC)
    )
    vsync <= VSYNC_POL;
else
    vsync <= ~VSYNC_POL;

// Data enable when within active region
if ((h_count < H_ACTIVE) && (v_count < V_ACTIVE)) begin
    de <= 1'b1;
    pixel_x <= h_count[15:0];
    pixel_y <= v_count[15:0];
end else begin
    de <= 1'b0;
    pixel_x <= 0;
    pixel_y <= 0;
end
end
end
endmodule

```

Implications and trade-offs:

- Clock domain: driving the PHY requires the exact pixel clock; any internal processing at other clocks must use FIFOs and CDC to avoid timing errors.
- Pipeline latency: DE alignment must match TMU fetch latency to avoid tearing; insert programmable DE offsets if needed.
- Multi-display scaling: per-display timing instances increase area but permit independent modes; shared logic with parameterized registers reduces duplication.
- Power: larger counters and wide registers for ultra-high resolutions cost area and dynamic power; balance by sizing widths to expected maximums only.

6.1.2 Frame buffer scanning

Following the timing generator discussion, the scanner must consume HSYNC/VSYNC-aligned pixel clocks and convert spatial coordinates into a steady stream of memory read requests so TMUs, ROPs, or an external scaler can be fed without underflow. The core problem is scheduling address generation and prefetch so that memory latency and burst alignment do not cause visible tearing or dropouts while supporting double-buffer flips and tiled layouts.

Problem → analysis:

- Requirements: produce pixel value each pixel clock, support arbitrary frame width/height, stride (bytes per line), and programmable frame base for flipping. Support burst-friendly memory accesses to improve DRAM efficiency and reduce L2 thrashing.
- Latency model: let L_m be the memory read latency in pixel clocks and B be burst length in pixels. The scanner must request address for pixel at x at time t such that data returns at $t + L_m$. Thus request lead time must satisfy $L_m \leq \text{pipeline depth}$. Address mapping for linear frames is

$$[H]\text{addr}(x, y) = \text{base} + y \cdot \text{stride} + x \cdot \text{pixel_bytes}. \quad (6.2)$$

For tiled or Z-ordered frames the mapping function replaces the linear term and must be computed or table-driven to avoid stalls.

Implementation (address generator and simple prefetch read engine):

- Use a pixel-clock domain FSM that:
 1. Maintains x and y counters synchronized to HSYNC/VSYNC.
 2. Issues memory requests ahead by L_m cycles; groups requests into bursts of up to B pixels aligned to cacheline boundaries.
 3. Accepts read data with a valid-ready handshake into a one-line FIFO or small ping-pong line buffer to decouple memory jitter from pixel output.
- Burst formation and stride alignment reduce memory transaction overhead and improve effective bandwidth to ROP/TMU consumers.

Practical Verilog implementation (synthesizable scanout controller):

Listing 6.2: Scanout controller: address generation, burst requests, one-line buffer

```

module scanout_controller #(
  parameter ADDR_W = 32,
  parameter STRIDE_W = 32,
  parameter X_W = 12,           // up to 4096 pixels per line
  parameter Y_W = 12,
  parameter PIXEL_BYTES = 4     // RGBA8 default
) (
  input wire          pix_clk, rst_n,
  input wire          enable,           // start/stop scanout
  input wire [ADDR_W-1:0] frame_base,   // base address of
    current buffer
  input wire [STRIDE_W-1:0] stride,      // bytes per line
  input wire [X_W-1:0] width,           // pixels per line
  input wire [Y_W-1:0] height,
  // memory read request interface
  output reg          mem_req,
  output reg [ADDR_W-1:0] mem_addr,
  input wire          mem_ready,
  input wire [PIXEL_BYTES*8-1:0] mem_rdata,

```

```

    input wire          mem_rvalid,
    // pixel output
    output reg [PIXEL_BYTES*8-1:0] pixel_out,
    output reg          pixel_valid,
    // sync (optional, driven from timing generator)
    input wire          hsync, vsync
);

// counters
reg [X_W-1:0] x_cnt;
reg [Y_W-1:0] y_cnt;

// simple one-cycle request pipeline: request for next pixel
// request lead = 1 cycle (adjust for real memory latency)
wire [ADDR_W-1:0] next_addr = frame_base + (y_cnt * stride)
                                + (x_cnt * PIXEL_BYTES);

always @(posedge pix_clk or negedge rst_n) begin
    if (!rst_n) begin
        x_cnt <= 0; y_cnt <= 0;
        mem_req <= 0; mem_addr <= 0;
        pixel_out <= 0; pixel_valid <= 0;
    end else if (enable) begin
        if (hsync) begin
            x_cnt <= 0;                                // start of line
        end
        if (vsync) begin
            y_cnt <= 0;                                // start of frame
        end
        // Issue mem request when previous accepted or idle
        if (!mem_req && mem_ready) begin
            mem_req <= 1;
            mem_addr <= next_addr;                      // compute address per eq. (1)
        end else if (mem_req && mem_ready) begin
            mem_req <= 0;                                // request accepted
        end
        // consume returned pixel
        if (mem_rvalid) begin
            pixel_out <= mem_rdata;
            pixel_valid <= 1;
            // advance pixel counters
            if (x_cnt + 1 == width) begin
                x_cnt <= 0;
                if (y_cnt + 1 == height) y_cnt <= 0; else y_cnt <= y_cnt + 1;
            end else x_cnt <= x_cnt + 1;
        end else begin
            pixel_valid <= 0;
        end
    end
end

```

```

    end
  end else begin
    mem_req <= 0; pixel_valid <= 0;
  end
end
endmodule

```

Implications and trade-offs:

- Increasing burst length B and aligning start addresses to cacheline boundaries improves DRAM efficiency but increases worst-case latency for a mid-line address change (e.g., partial flips).
- Prefetch lead must be dimensioned to memory latency L_m ; inadequate lead causes underflow at high refresh rates.
- Supporting tiled/swizzled layouts requires additional logic or a small LUT, trading area/latency for reduced memory contention and better cache locality for TMU/ROP consumers.

6.1.3 Pixel pipeline organization

Continuing from frame-buffer scan order and the pixel timing signals HSYNC/VSYNC, the pixel pipeline organization maps scanline timing to a sequence of fixed-latency and streaming stages that must meet the line-rate budget imposed by the timing generator.

The problem is to guarantee each scanline's pixels are processed before the next HSYNC edge while preserving color fidelity, HDR metadata, and optional compression. Analysis begins by budgeting cycles and bandwidth. For a display with horizontal resolution H , vertical resolution V , refresh rate f_{refresh} , and pixel clock $f_{\text{pixel}} = H \cdot f_{\text{refresh}}$ including blanking, the per-pixel processing time is $1/f_{\text{pixel}}$. A core with clock f_{core} must provide at least

$$[H]\text{cycles_per_pixel} = \frac{f_{\text{core}}}{f_{\text{pixel}}} \quad (6.3)$$

processing cycles available to implement transforms, filtering, and packing. The required internal throughput for N parallel pipeline lanes is then

$$[H]N \geq \left\lceil \frac{\text{work_cycles_per_pixel}}{\text{cycles_per_pixel}} \right\rceil, \quad (6.4)$$

where `work_cycles_per_pixel` accounts for arithmetic latency, memory accesses, and any encoding.

Implementation breaks the pixel pipeline into composable stages:

- input formatter: swizzling, bit-depth normalization, and alignment to bus width;
- color-space conversion (CSC): YCbCr $\leftrightarrow$ RGB or wide-gamut transforms implemented as fixed-point matrix multiply with pipeline registers;
- scaling/filtering: separable filters or polyphase resampler operating on tile-line buffers;

- tone mapping and color grading: HDR OETF/Inverse OETF and LUTs (small SRAM or BRAM for LUTs);
- chroma up/down-sampling and reformatting: chroma reconstruction for YUV420 video;
- output packer and optional compressor: DSC encoder or TMDS/DP formatter.

Design implementation example: a synthesizable, pipelined YCbCr-to-RGB fixed-point CSC. The module uses two pipeline stages and a valid flag. Coefficients are parameterized in Q2.14 fixed-point for precision and implemented with signed arithmetic, suitable for SIMT-like vectorization across lanes.

Listing 6.3: Pipelined YCbCr to RGB color-space convertor (synthesizable).

```

module ycbcr_to_rgb #(
    parameter BW = 8,                // input component bits
    parameter COEFW = 16             // fixed-point coefficient width Q2.14
) (
    input wire clk,
    input wire rstn,
    input wire valid_in,
    input wire [BW-1:0] Y_in,
    input wire [BW-1:0] Cb_in,
    input wire [BW-1:0] Cr_in,
    output reg valid_out,
    output reg [BW-1:0] R_out,
    output reg [BW-1:0] G_out,
    output reg [BW-1:0] B_out
);

// Coefs in Q2.14: 1.402 -> int(1.402*2^14)=22981 etc.
localparam signed [COEFW-1:0] C_r_cr = 16'sd22981; // 1.402
localparam signed [COEFW-1:0] C_g_cb = -16'sd5638; // -0.344136
localparam signed [COEFW-1:0] C_g_cr = -16'sd11698; // -0.714136
localparam signed [COEFW-1:0] C_b_cb = 16'sd29032; // 1.772

// Stage 1 registers
reg signed [BW-1:0] Ys, Cbs, Crs;
always @(posedge clk) begin
    if (!rstn) begin valid_out <= 0; Ys <= 0; Cbs <= 0; Crs <= 0; end
    else begin
        Ys <= $signed(Y_in);
        Cbs <= $signed(Cb_in) - 8'sd128; // center chroma
        Crs <= $signed(Cr_in) - 8'sd128;
        valid_out <= valid_in; // one-cycle pipeline flag
    end
end

// Stage 2: multiply-accumulate and downshift Q2.14
always @(posedge clk) begin
    if (!rstn) begin R_out <= 0; G_out <= 0; B_out <= 0; end

```

```

else begin
    // wide products
    wire signed [31:0] r_acc = ($signed(Ys) <<< 14) + C_r_cr * Crs;
    wire signed [31:0] g_acc = ($signed(Ys) <<< 14) + C_g_cb * Cbs + C_g_cr
        * Crs;
    wire signed [31:0] b_acc = ($signed(Ys) <<< 14) + C_b_cb * Cbs;
    // convert back from Q2.14 to integer with rounding, clamp 0..255
    function [BW-1:0] q2_14_to_8(input signed [31:0] v);
        reg signed [31:0] t;
        begin
            t = (v + 32'sd8192) >>> 14; // rounding
            if (t < 0) q2_14_to_8 = 0;
            else if (t > 255) q2_14_to_8 = 8'hFF;
            else q2_14_to_8 = t[7:0];
        end
    endfunction
    R_out <= q2_14_to_8(r_acc);
    G_out <= q2_14_to_8(g_acc);
    B_out <= q2_14_to_8(b_acc);
end
end
endmodule

```

Implications and trade-offs:

- Increasing pipeline lanes reduces per-lane clock requirement but raises area and power for replicated TMUs and line buffers.
- Fixed-point coefficient width trades precision for multiplier area; Q2.14 balances color accuracy versus DSP block usage.
- Adding valid-ready backpressure increases complexity but enables elastic pipelines for variable-latency encoders like DSC.
- Designers must match pipeline latency to blanking intervals to avoid frame tearing while minimizing on-chip buffering to save SRAM and power.

6.1.4 Multi-display management

The previous discussion of pixel pipelining and deterministic frame buffer scanout highlighted that timely memory reads and per-scanout buffering are central to correct display timing. Multi-display management generalizes those constraints: multiple independent scanouts, different pixel clocks and color depths, and additional encoder/transport constraints (MST, DSC) must be coordinated so each head meets its vblank and line deadlines without starving rendering or other clients.

Problem: each display head requires a continuous stream of pixels at its pixel clock and bit depth. Analysis begins by quantifying per-head bandwidth and the effective demand on the memory subsystem:

$$[H]B_{\text{disp}} = \frac{f_p \cdot b}{8 \cdot C} \quad (6.5)$$

where f_p is the pixel clock in Hz, b is bits-per-pixel, and $C \geq 1$ is an effective compression factor (DSC or in-memory compression). Total scanout bandwidth is the sum across heads plus overhead for overlays, cursor planes, and chroma up/down sampling. A single worst-case display (e.g., 8K HDR with $b = 30$ and high f_p) can dominate DRAM bandwidth; multi-head systems must therefore schedule reads such that per-line deadlines are met while avoiding long bursts that harm other requesters.

Design analysis yields two control knobs:

1. Per-head FIFO sizing to absorb memory controller latency and jitter. A practical formula for pixel FIFO depth in lines is

$$D_{\text{lines}} \approx \left\lceil \frac{t_{\text{latency}} \cdot f_p}{H_{\text{res}}} \right\rceil$$

where t_{latency} is worst-case memory-system latency and H_{res} is pixels per line.

2. Memory scheduling policy: earliest-deadline-first (EDF) or weighted time-division multiplexing (TDM) with deadlines and slack accounting preserve latency bounds while maximizing fairness.

Implementation: a lightweight EDF arbiter in the display controller schedules DMA reads for each head, presenting per-request deadlines to the arbiter. The Verilog below is a synthesizable EDF arbiter for up to N display channels; it selects the valid request with the smallest deadline and includes a simple round-robin tie-breaker to avoid starvation.

Listing 6.4: EDF DMA arbiter for multi-display scanout (synthesizable).

```

module display_dma_arbiter #(
  parameter N_CH = 4,
  parameter DEAD_W = 32
) (
  input  wire          clk,
  input  wire          rst,
  input  wire [N_CH-1:0] req_valid,           // per-head request
          valid
  input  wire [N_CH*DEAD_W-1:0] req_deadline_flat, // concatenated
          deadlines MSB...LSB
  output reg  [N_CH-1:0] grant                // one-hot grant
);

integer i;
reg [DEAD_W-1:0] deadlines [0:N_CH-1];
reg [DEAD_W-1:0] min_dead;
reg [N_CH-1:0]   min_idx;
reg [N_CH-1:0]   tie_mask;

// unpack deadlines
always @* begin
  for (i = 0; i < N_CH; i = i + 1) begin
    deadlines[i] = req_deadline_flat[(i + 1) * DEAD_W - 1 -: DEAD_W];
  end

```

```

end

// earliest deadline first selection
always @* begin
    min_dead = {DEAD_W{1'b1}};
    min_idx  = {N_CH{1'b0}};
    tie_mask = {N_CH{1'b0}};
    for (i = 0; i < N_CH; i = i + 1) begin
        if (req_valid[i] && (deadlines[i] < min_dead)) begin
            min_dead = deadlines[i];
            min_idx  = {N_CH{1'b0}};
            min_idx[i] = 1'b1;
        end else if (req_valid[i] && (deadlines[i] == min_dead)) begin
            tie_mask[i] = 1'b1;
        end
    end
end
end

// registered grant
always @(posedge clk) begin
    if (rst)
        grant <= {N_CH{1'b0}};
    else
        grant <= min_idx | tie_mask; // tie-breaking simple OR (earliest + same
                                     // deadline)
    end
end
endmodule

```

6.2 Display Compression

6.2.1 Display Stream Compression (DSC)

Following the display timing and pixel-pipeline considerations discussed in the display controller subsection, the Display Stream Compression (DSC) stage sits between the GPU's pixel pipeline and the external link (DisplayPort/HDMI PHY or internal panel interface). DSC's role is to reduce required link bandwidth while preserving visual fidelity, enabling higher resolutions, higher refresh rates, or fewer physical lanes in SM/MMIO-constrained implementations.

Problem — link bandwidth versus pixel throughput: a raw RGB frame at resolution $R_x \times R_y$ and pixel clock f_{pix} with N bits per color channel requires raw link throughput

$$[H]B_{\text{raw}} = f_{\text{pix}} \cdot R_{\text{bpp}} \quad \text{where} \quad R_{\text{bpp}} = N_{\text{chan}} \cdot N_{\text{bits/channel}}. \quad (6.6)$$

DSC reduces bandwidth by a content-dependent compression ratio CR, so the compressed bandwidth target is approximately $B_{\text{comp}} \approx B_{\text{raw}}/\text{CR}$, plus slice-header and packetization overhead: when encoding is slice-based, per-slice header costs inflate effective bits-per-pixel by $H_{\text{slice}}/\text{pixels_per_slice}$.

Analysis — algorithmic and architectural constraints: practical DSC implementations in GPUs must meet constant-rate constraints imposed by the serial link and maintain low, bounded latency for scanout. To accomplish that, hardware DSC encoders operate on fixed-size slices (rows or groups of pixels), perform a reversible color-space transform (to decorrelate channels), apply prediction to generate residuals, quantize and entropy-code the residuals, and finally pack slice headers and payload into transport packets. The encoder also runs an on-chip rate-control loop to meet the target bits-per-pixel (bpp) for each slice; this can be modeled as ensuring

$$[H] \sum_{i \in \text{slice}} b_i \leq \text{pixels}_{\text{slice}} \cdot \text{bpp}_{\text{target}} - H_{\text{slice}} \quad (6.7)$$

where b_i are the coded symbol lengths. The GPU must provide sufficient FIFO depth and a deterministic token-budget to absorb short-term entropy bursts without underrunning the link.

Implementation — datapath and controller: integrate the DSC encoder as a late-stage pixel pipeline block, immediately upstream of the display timing generator. Key hardware blocks:

- Slice input buffer (line buffering sized for worst-case slice pixels),
- Color transform and predictor pipelines (pipelined for the pixel clock),
- Quantizer and entropy coder (variable-latency; typically constrained to a few cycles via parallelism and pipelining),
- Slice packetizer with constant-rate shaper feeding the PHY.

To illustrate a small synthesizable component of the encoder subsystem — a token-bucket rate shaper that gates variable-length codewords into fixed-bit link words — see the Verilog module below.

Listing 6.5: Token-bucket based DSC rate shaper (synthesizable).

```
module dsc_rate_shaper #(
    parameter integer PIXEL_CLK_MHZ = 148,           // pixel clock in MHz
    parameter integer TARGET_BPP = 24,              // target bits per pixel (
        integer bpp*1)
    parameter integer SLICE_PIXELS = 3840           // pixels per slice
) (
    input wire clk,
    input wire reset_n,
    // input: encoded symbol length in bits for current pixel (variable-length)
    input wire sym_valid,
    input wire [15:0] sym_bits, // bits used to encode symbol (<= 16k)
    output reg sym_accept, // accepted for transmission this cycle
    // output: link word ready handshake (simplified)
    output reg link_word_valid,
    input wire link_word_ready
);
    // budget accumulator in bits, fixed-point with microbit granularity
    localparam integer BUDGET_MAX = 32'd100000000;
    reg [31:0] budget_bits; // current token budget
```

```

// initialize budget: pixels per second * bpp gives bits per second; scaled
// per cycle
// For simplicity, add fixed budget per cycle = TARGET_BPP (approx) per
// pixel tick.
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        budget_bits <= 0;
        sym_accept <= 0;
        link_word_valid <= 0;
    end else begin
        // replenish budget (simplified): add TARGET_BPP bits per pixel clock
        // tick
        // In real design, use fractional accumulator synchronized to pixel
        // clock domain.
        if (budget_bits + TARGET_BPP < BUDGET_MAX)
            budget_bits <= budget_bits + TARGET_BPP;
        else
            budget_bits <= BUDGET_MAX;
        // accept symbol only if budget suffices
        if (sym_valid && sym_bits <= budget_bits) begin
            sym_accept <= 1'b1;
            budget_bits <= budget_bits - sym_bits;
            // produce link word (simplified packetization)
            if (link_word_ready) link_word_valid <= 1'b1;
            else link_word_valid <= 1'b0;
        end else begin
            sym_accept <= 1'b0;
            link_word_valid <= 1'b0;
        end
    end
end
endmodule

```

Implications and trade-offs:

- Latency: slice-based encoding adds deterministic latency equal to slice buffering plus entropy coder pipeline depth; larger slices improve coding efficiency but increase buffer size and latency.
- Area and power: entropy coder and high-throughput predictor pipelines consume logic and SRAM; trade compression ratio versus silicon cost.
- Visual quality versus compression ratio: aggressive CRs reduce bandwidth but raise visible artifacts; choose CR per target display spec and content class.
- Link utilization and robustness: token-bucket shaping prevents PHY underruns but requires accurate bpp targets and accounting for header overhead (Equation 6.7).

6.2.2 Encoder and decoder design

Building on the DSC primitives and rate-control concepts in the previous subsection, the encoder and decoder pair must map those algorithms into a low-latency, high-throughput hardware datapath that integrates with TMU, ROP and the display controller. The following explains the architectural problem, analyzes throughput and latency constraints, presents a compact synthesizable encoder/decoder example, and derives practical design implications.

Problem and analysis.

- Goal: reduce framebuffer-to-link bandwidth while preserving visual quality and meeting deterministic decode latency for the panel or external link.
- Constraints: per-pixel arrival rate from the ROP/TMU, maximum header and metadata overhead, and allowable decode cycles per pixel on the display side (often a fixed small budget).
- Typical throughput requirement example: for a 4K (3840×2160) @60 Hz 10-bit RGBA stream, uncompressed pixel-rate bits per second is high; compression must lower bits per lane to fit link PHY lanes.

Useful expressions for design sizing.

- Compression ratio R :

$$[H]R = \frac{\text{original_bits}}{\text{compressed_bits}} \quad (6.8)$$

- Entropy lower bound and expected code length L (information-theoretic): $E[L] \geq H(X) = -\sum_i p_i \log_2 p_i$; practical encoders approach but do not beat H .
- Per-lane bitrate requirement when using N_{lanes} :

$$[H]B_{\text{lane}} = \frac{W \cdot H \cdot f_{\text{ref}} \cdot b_{\text{pix}}}{N_{\text{lanes}}} \quad (6.9)$$

where W, H are pixel dimensions, f_{ref} refresh rate, and b_{pix} bits per pixel.

Architectural elements.

1. Front-end predictor: delta or spatial predictor implemented in a small line buffer to exploit intra-block redundancy.
2. Quantizer / transform (optional): simple scalar quantizer for DSC-like modes or small DCT for video.
3. Entropy coder: hardware-friendly entropy such as Golomb-Rice or fixed-codeword Huffman tables; tables are stored in small ROM and accessed in parallel for low-latency coding.
4. Packetizer and metadata generator: header, checksum, and tile boundaries; aligns to PHY word width.
5. Decoder mirroring: must be pipelined to guarantee constant cycles per tile; often implemented in tile-parallel fashion in the display controller.

Implementation example: a synthesizable simple run-length delta encoder and its decoder. This micro-architecture demonstrates predictor, run-length compression, and a streaming ready/valid interface.

Listing 6.6: Simple delta run-length encoder and decoder (synthesizable).

```

module delta_rle_enc #(
  parameter PIXW = 10, // bits per pixel channel
  parameter BUSW = 16  // output bus width
) (
  input  clk, input rstn,
  input  valid_in, input [PIXW-1:0] px_in,
  output reg ready_in,
  output reg valid_out, output reg [BUSW-1:0] data_out
);
  // simple delta predictor and run-length (counts up to 2^BUSW-1)
  reg [PIXW-1:0] prev;
  reg [BUSW-1:0] run;
  reg state;
  always @(posedge clk) begin
    if (!rstn) begin
      prev <= 0;
      run <= 0;
      state <= 0;
      valid_out <= 0;
      ready_in <= 1;
    end else begin
      valid_out <= 0;
      if (valid_in && ready_in) begin
        if (px_in == prev) begin
          run <= run + 1;
        end else begin
          // flush run if any
          if (run != 0) begin
            valid_out <= 1;
            data_out <= {1'b1, run[BUSW-2:0]};
            run <= 0;
          end
          // emit delta token: leading 0 then signed diff (simplified)
          valid_out <= 1;
          data_out <= {1'b0, px_in[PIXW-2:0]};
          prev <= px_in;
        end
      end
    end
  endmodule

```

```

module delta_rle_dec #(
    parameter PIXW = 10,
    parameter BUSW = 16
) (
    input clk,
    input rstn,
    input valid_in,
    input [BUSW-1:0] data_in,
    output reg ready_in,
    output reg valid_out,
    output reg [PIXW-1:0] px_out
);
    reg [PIXW-1:0] prev;
    integer i;
    always @(posedge clk) begin
        if (!rstn) begin
            prev <= 0;
            valid_out <= 0;
            ready_in <= 1;
        end else begin
            valid_out <= 0;
            if (valid_in && ready_in) begin
                if (data_in[BUSW-1]) begin
                    // run token: repeat previous pixel value
                    for (i = 0; i < data_in[BUSW-2:0]; i = i + 1) begin
                        px_out <= prev;
                        valid_out <= 1;
                    end
                end else begin
                    // delta token: update with new pixel
                    px_out <= {1'b0, data_in[PIXW-2:0]};
                    valid_out <= 1;
                    prev <= px_out;
                end
            end
        end
    end
end
endmodule

```

6.2.3 Bandwidth reduction analysis

The previous subsections established DSC's microblock prediction, encoder/decoder latency constraints, and practical hardware partitioning for real-time display streams. Here we quantify how those design choices translate into link reduction, buffer sizing, and encoder throughput requirements for modern GPU display engines.

Problem statement and analysis: the display pipeline must deliver pixels at scanout rate while minimizing VRAM and link bandwidth. Uncompressed pixel bandwidth for a frame

is proportional to horizontal and vertical resolution, refresh rate, and bits per pixel. For an uncompressed stream the bit-rate is

$$[H]B_{\text{raw}} = H \cdot V \cdot f_r \cdot b_{\text{pp}}, \quad (6.10)$$

where H, V are pixel counts, f_r is refresh frequency, and b_{pp} is bits per pixel. Practical designs add a small protocol and blanking overhead; account for that as a multiplicative factor $(1 + \alpha)$.

With compression ratio R (for DSC-targeted content), effective bandwidth becomes

$$[H]B_{\text{link}} = \frac{B_{\text{raw}}(1 + \alpha)}{R} + B_{\text{meta}}, \quad (6.11)$$

where B_{meta} is metadata rate from slice headers and ACK/control bits. B_{meta} is small but non-negligible at very high frame rates or many slices.

Implementation calculations guide hardware provisioning. Encoder throughput must sustain worst-case incoming pixel rate and average compression time per slice. If a compressor spends t_{enc} seconds per slice of N_{px} pixels, required encoder pixel throughput is $T_{\text{enc}} = N_{\text{px}}/t_{\text{enc}}$. For parallel encoder lanes, number of lanes is $\lceil (B_{\text{raw}}/(R \cdot T_{\text{lane}})) \rceil$.

Buffer sizing is driven by variable-rate compression. If link capacity is B_{link} and the compressor can temporarily fall below that rate, buffer depth D must satisfy

$$[H]D \geq \max_T ((B_{\text{raw}} - B_{\text{link}}) \cdot T), \quad (6.12)$$

where T is the maximum transient interval the encoder may underperform. For scanout-critical systems, T is constrained by vertical blank duration and CRTC latency budget.

Concrete example and quick tool: the listing computes uncompressed and compressed link rates and a nominal buffer depth for a sample 4K HDR scenario.

Listing 6.7: Compute bandwidth and buffer needs for a display stream.

```
# Simple bandwidth calculator for display compression (example)
H, V = 3840, 2160          # resolution
fr = 60                    # refresh rate (Hz)
bpp = 30                   # bits per pixel (10bpc RGB)
alpha = 0.05               # protocol overhead
R = 3.0                    # compression ratio (e.g., DSC target)
B_meta = 0.1e9             # metadata budget (bps)
B_raw = H * V * fr * bpp   # Eq. used in prose
B_link = B_raw * (1+alpha) / R + B_meta
# Buffer depth for 10 ms transient where encoder falls to 50% throughput
transient_ms = 10
D_bits = max(0, (B_raw - B_link*0.5)) * (transient_ms/1000.0)
# Print Gbps and MB for buffer
print(f"Raw: {B_raw/1e9:.2f} Gbps, Link needed: {B_link/1e9:.2f} Gbps")
print(f"Buffer depth ~ {D_bits/8/1e6:.2f} MB for {transient_ms} ms transient")
```

Design implications and trade-offs:

- Provision encoder lanes so peak raw throughput and worst-case t_{enc} remain covered; underprovisioning forces larger buffers and added latency.

- Metadata and slice granularity affect peak-to-average smoothing; smaller slices lower latency but raise header overhead.
- Conservative DSC ratios reduce link requirements but increase encoder complexity and power in the display controller.
- For multi-display GPUs, aggregate link budgeting must consider concurrent streams and VRAM read contention at the memory controller and ROP/scanout ports.

6.2.4 Visual quality metrics

Building on the encoder/decoder design choices and the bandwidth-reduction tradeoffs discussed previously, we now quantify how different compression settings affect perceived image and video quality; these metrics guide rate-control, encoder heuristics, and hardware budget allocation in TMUs, display encoders, and ROP/scanout pipelines.

Objective versus subjective metrics. Objective scores are deterministic, lightweight, and suitable for hardware or fast software loops; subjective tests (mean opinion score, MOS) remain the ultimate ground truth but are costly and slow. Common objective metrics and their operational relevance:

- Peak signal-to-noise ratio (PSNR): cheap, measures pixel-wise fidelity; useful for fast regression testing and coarse encoder tuning. Compute PSNR from mean squared error (MSE):

$$[H]\text{PSNR} = 10 \log_{10} \left(\frac{MAX_I^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{N} \sum_{i=1}^N (r_i - d_i)^2 \quad (6.13)$$

where r_i is the reference sample, d_i the decoded sample, and MAX_I the maximum sample value (e.g., 255 for 8-bit).

- Structural Similarity Index (SSIM): captures local luminance, contrast, and structure; correlates better with perceived quality for texture and edges. For two windows x, y :

$$[H]\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (6.14)$$

aggregated across the image yields a single score in $[0, 1]$.

- Multi-scale and perceptual metrics: MS-SSIM, VMAF (Video Multi-method Assessment Fusion) combine spatial scales, temporal factors, and human visual system (HVS) features; VMAF is used for video-rate control in encoders where subjective fidelity is critical (streaming or media playback).
- Color- and HDR-aware metrics: compute Y (luma) weighted metrics for YUV streams; for HDR content compute metrics after appropriate HDR-to-LDR tone mapping or in perceptually uniform spaces (PQ, HLG) to avoid misrepresenting luminance errors.

Temporal artifacts and motion sensitivity. For animated content and games, frame-to-frame error visibility depends on motion vectors and persistence: include temporal metrics such as delta-PSNR across frames and motion-compensated PSNR to reveal flicker or popping introduced by bitrate variations.

Implementation considerations for GPU/display hardware:

- Fast hardware-friendly metrics: block-based MSE/PSNR and gradient-energy comparisons can run in a scanout/encoder ASIC to supply rate-control feedback without host CPU.
- Offline or calibration metrics: SSIM and VMAF require more computation and are suitable for encoder firmware or host-side profiling.

Minimal reference implementation (Python) to compute PSNR and a simple frame SSIM approximation; suitable for offline validation and encoder tuning:

Listing 6.8: `psnr_and_ssim_simple.py`

```
import numpy as np

def psnr(ref, dec, max_val=255.0):
    mse = np.mean((ref.astype(np.float32) - dec.astype(np.float32))**2)
    if mse == 0:
        return float('inf') # identical
    return 10.0 * np.log10((max_val**2) / mse)

def ssim_simple(ref, dec, K1=0.01, K2=0.03, L=255):
    # simple single-window SSIM for demonstration (not multi-scale)
    C1 = (K1*L)**2; C2 = (K2*L)**2
    mu_x = ref.mean(); mu_y = dec.mean()
    sigma_x2 = ((ref - mu_x)**2).mean(); sigma_y2 = ((dec - mu_y)**2).mean()
    sigma_xy = ((ref - mu_x)*(dec - mu_y)).mean()
    num = (2*mu_x*mu_y + C1) * (2*sigma_xy + C2)
    den = (mu_x**2 + mu_y**2 + C1) * (sigma_x2 + sigma_y2 + C2)
    return float(num / den)

# usage: load frames as numpy arrays and call psnr(ref, dec) and ssim_simple(
# ref, dec)
```

Design implications and trade-offs:

- Choose metric complexity by control loop latency: use PSNR or block-MSE in on-chip encoders for real-time rate control, and SSIM/VMAF offline or in firmware for perceptual tuning.
- Metric sensitivity affects bitrate allocation: luma-weighted SSIM reduces visible banding while PSNR-driven control may waste bits on low-visibility chroma.
- HDR and wide-gamut content force perceptual-space comparisons, increasing compute and metadata needs in encoder/decoder hardware.
- Overall, a hybrid strategy—low-cost on-chip predictors plus periodic perceptual calibration—balances silicon area, encoder latency, and visual fidelity across graphics, video playback, and game streaming workloads.

6.3 Video Decode Acceleration

6.3.1 H.264, H.265, VP9, and AV1 decoding

The decode engine described previously for display timing and compression must also meet the latency, parallelism, and bandwidth requirements of modern codecs; this subsection builds on those constraints and focuses on architectural choices for H.264, H.265 (HEVC), VP9, and AV1 acceleration. The central problem is that entropy decoding and some loop filters are sequential, while motion compensation and inverse transforms are highly parallel and memory-bandwidth hungry; the design must partition work to maximize SM/CU utilization without violating codec dependencies.

Analysis: codec-specific characteristics that determine hardware structure

- H.264: 16×16 macroblocks, supports CAVLC or CABAC entropy coding; CABAC is context-adaptive and serial at symbol granularity.
- H.265 (HEVC): CTU (Coding Tree Unit) up to 64×64, mandatory CABAC, supports Wavefront Parallel Processing (WPP) to enable row-parallelism of CTUs with a one-CTU dependency.
- VP9: Superblocks up to 64×64, boolean arithmetic coder per tile; tiles provide natural parallelism across rows/columns.
- AV1: Superblocks up to 128×128, range coder (a form of arithmetic coder); tiles and independent tile groups enable coarse-grain parallelism, but in-loop filters and film-grain are often frame-serial.

Implementation: mapping codec stages to GPU and fixed-function resources

1. Bitstream parser and entropy stage: implement as dedicated scalar micro-engine per decode engine; instantiate multiple engines to process independent slices/tiles concurrently. For CABAC and range coder, provide dedicated integer ALU arrays that perform renormalization and context-model updates at high clock rates, avoiding long-latency memory accesses to SM register files.
2. Inverse transform and iDCT/iDST: map to SIMT-friendly kernels on SM/CU with tiled shared memory reuse for coefficient blocks; use tensor cores only when transform precision and format permit.
3. Motion compensation: hardware TMU-assisted pipelines perform sub-pixel interpolation with a small vector unit per decode stream; use per-CU motion buffers and a tile-level cache to reduce DRAM reads.
4. Loop filters (deblocking, SAO, film grain): implement a hybrid approach—fixed-function low-latency deblocking and GPU-parallel SAO for throughput.

Quantifying bandwidth needs guides buffer sizing. Let W, H be frame dimensions, s the chroma subsampling factor (YUV 4:2:0 $\rightarrow s = 1.5$), b bits per sample, f frames per second, and r a reference multiplier (> 1) representing additional reads from interpolation and multiple references. Then:

$$[H]BW_{\text{bytes/s}} = W \cdot H \cdot s \cdot \frac{b}{8} \cdot f \cdot r \quad (6.15)$$

Example: 3840×2160 , $b = 10$, $s = 1.5$, $f = 60$, $r \approx 4$ yields roughly 3.73 GB/s sustained for frame fetches alone; cache and tiling reduce DRAM pressure.

Implementation sketch: tile dispatch to SMs using asynchronous prefetch (CUDA-like pseudocode).

Listing 6.9: Tile-based decode dispatch with async prefetch

```
__global__ void decode_tiles(TileDescriptor *tiles, FrameBuf *refs) {
    int tid = blockIdx.x; // one tile per block
    TileDescriptor t = tiles[tid];
    // async prefetch reference region into local L2-writable cache (hw DMA)
    hw_async_prefetch(refs, t.ref_region);
    // wait for entropy engine to produce coefficient stream (signal-based)
    wait_for_entropy(t.entropy_channel);
    // run inverse transform and MC on SM using shared memory reuse
    decode_tile_kernel(t, refs);
    // writeback tile to output framebuffer
    write_tile_output(t.output_region);
}
```

Design implications and trade-offs:

- Increase tile/slice granularity to expose parallelism, but too small tiles raise entropy overhead and context-model divergence.
- Implement dedicated entropy engines to avoid stalling SMs on serial renormalization; however, these engines consume area and power.
- Large on-chip tile caches reduce DRAM bandwidth but increase SRAM area and access energy.
- Wavefront-style parallelism (HEVC) and tile-groups (AV1) yield good SM occupancy while preserving correct context updates.

6.3.2 Bitstream parsing and entropy decoding

These parsing primitives continue the codec-level discussion of H.264/H.265/VP9/AV1 by shifting focus from high-level syntax to the low-level operations that must run at wire-speed on a GPU display/video engine. The parser supplies aligned bit fields, variable-length codes, and symbol streams to the entropy decoder block that implements CABAC, ANS, or Huffman logic.

The problem: decode a serialized bitstream into symbol tokens without stalling motion-compensation, deblocking, or post-filter SM pipelines. Analysis begins with throughput and dependency constraints. Let B be the input bit bandwidth (bits/s) and $\mathbb{E}[L]$ the average code length in bits; the symbol decode throughput required is

$$[H]S_{\text{req}} = \frac{B}{\mathbb{E}[L]} \quad \text{symbols/s,} \quad (6.16)$$

which must be matched by the combined capacity of parallel entropy engines. Entropy decoding adds stateful context (probability models, adaptive tables) and serial renormalization steps in arithmetic/ANS decoders that limit raw SIMD scaling. Therefore hardware design splits responsibilities:

- Bitstream parsing: byte-to-bit buffer, byte-stuffing handling, start-code detection, and aligned bit extraction. This is highly parallelizable and implemented as a pipelined bit FIFO with wide input ports to keep downstream entropy engines fed.
- Entropy decoding: either table-driven Huffman, CABAC binary arithmetic, or ANS; each requires local state for contexts and renormalization loops. CABAC exhibits serial per-bin dependencies but supports parallelism across independent symbol streams (slices, tiles, or tiles within AV1).

Implementation guidance for on-chip engines:

- Partition decode into multiple independent contexts mapped to separate SM/CU-side microengines; the parser performs striping and delivers per-engine byte ranges.
- Store context probabilities in low-latency SRAM adjacent to each decoder to avoid L2 thrashing; update contexts in-place to minimize memory traffic.
- Pipeline renormalization with speculative prefetch of next bits; for ANS, precompute reciprocal tables to replace divisions.

Below is a synthesizable Verilog module that implements a pipelined bitreader with `read_bits`, Exp-Golomb unsigned decoder, and a small canonical-Huffman lookup stage; it serves as the parsing front-end (`bitstream_parser`) for downstream entropy units.

Listing 6.10: Pipelined bitstream parser front-end

```

module bitstream_parser (
    input  wire      clk,
    input  wire      rst_n,
    input  wire [7:0] in_byte,    // input byte stream
    input  wire      in_valid,
    output reg       in_ready,
    // read interface
    input  wire [4:0] req_bits,   // request bits (1..32)
    input  wire      req_valid,
    output reg [31:0] req_data,
    output reg       req_ready,
    // exp-golomb request
    input  wire      req_ue_valid,
    output reg [15:0] ue_value,
    output reg       ue_ready,
    // huffman output (max 16-bit code)
    output reg [15:0] huff_sym,
    output reg       huff_valid
);
    // internal 64-bit shift buffer
    reg [63:0] shbuf;
    reg [5:0]  bits_cnt;
    // simple FIFO fill
    always @(posedge clk) begin

```

```

if (!rst_n) begin
    shbuf <= 64'd0; bits_cnt <= 0; in_ready <= 1'b1;
    req_data <= 0; req_ready <= 0; ue_value <= 0; ue_ready <= 0;
    huff_sym <= 0; huff_valid <= 0;
end else begin
    // accept input bytes when room
    if (in_valid && in_ready) begin
        shbuf <= {shbuf[55:0], in_byte}; bits_cnt <= bits_cnt + 8;
        in_ready <= (bits_cnt + 8 <= 56);
    end else if (in_valid) in_ready <= 0;
    // process read_bits
    if (req_valid && !req_ready && bits_cnt >= req_bits) begin
        req_data <= shbuf[63 -: 32] >> (32 - req_bits); // MSB-first extract
        shbuf <= shbuf << req_bits; bits_cnt <= bits_cnt - req_bits;
        req_ready <= 1'b1;
    end else req_ready <= 1'b0;
    // exp-Golomb unsigned decode (UE)
    if (req_ue_valid && !ue_ready) begin
        integer k; integer leading;
        leading = 0;
        // count leading zeros (simple loop; synthesis-friendly for small
        // width)
        for (k=0;k<32;k=k+1) if (shbuf[63-k]==1'b0) leading = leading+1; else
            begin k=32; end
        // ensure bits available
        if (bits_cnt >= (2*leading+1)) begin
            ue_value <= ( (shbuf >> (63-(2*leading))) & ((1<<(leading))-1) ) +
                (1<= 16) begin
                reg [15:0] prefix;
                prefix = shbuf[63:48];
                // example table: if prefix top nibble equals 4'b1110 -> symbol 5 (
                // placeholder)
                if (prefix[15:12]==4'b1110) begin
                    huff_sym <= 16'd5; huff_valid <= 1'b1;
                    shbuf <= shbuf << 4; bits_cnt <= bits_cnt - 4;
                end else huff_valid <= 1'b0;
            end else huff_valid <= 1'b0;
        end
    end
endmodule

```

Design implications and trade-offs:

- Replicate parsers across decode engines to avoid a single-bitstream bottleneck; this increases SRAM and routing cost.
- Favor table-driven Huffman/ANS where parallel table lookups replace serial renormalization, reducing latency but consuming BRAM.

- For CABAC, accept lower per-core parallelism and instead scale by independent slices/tiles to meet S_{req} from equation (9.31).

6.3.3 Motion compensation hardware

The previous subsections established how parsed bitstreams become syntax elements and coefficients via entropy decoding, and how codec-specific motion vectors and reference frame indexes are produced for H.264, H.265, VP9, and AV1. Motion compensation (MC) consumes those vectors and reference pixels to reconstruct predicted blocks with sub-pixel filters; this subsection analyzes the hardware required to do that at video-engine and GPU scale.

Problem statement and analysis: motion compensation must deliver per-pixel predictions for many macroblocks or coding tree units in parallel while minimizing memory traffic and latency to keep decode pipelines (and optional TMU or dedicated video engines) fed. Modern codecs use separable fractional-pixel FIR filters of length L (commonly 4–8 taps for luma), so per-pixel work is dominated by multiply-accumulates (MACs). For a block of width W and height H , with separable filter length L , the number of MACs per block is

$$[H]\text{MACs}_{\text{block}} = W \cdot H \cdot 2L, \quad (6.17)$$

because separable filtering requires L horizontal and L vertical taps per pixel. For a 16×16 macroblock with $L = 6$ this equals $256 \cdot 12 = 3072$ MACs, illustrating why dedicated MAC arrays or SIMD engines are used instead of scalar processors.

Implementation approach: design choices include:

- Separable two-stage pipeline (horizontal then vertical) to reduce arithmetic and buffer pressure.
- Line buffers sized for block width to reuse fetched reference samples, reducing DRAM reads and enabling local SRAM or L1 caching.
- Fixed-point fractional coordinates, e.g., `frac_x` and `frac_y` as 8-bit fractions, to implement arbitrary sub-pixel interpolation.
- Vectorized datapaths (SIMD lanes or small systolic arrays) mapped to SM/CU resources or to dedicated MC units per video engine instance.

Concrete synthesizable example: a small, single-channel separable interpolator implementing bilinear interpolation with fixed-point fractions. It accepts four neighbouring pixels and fractional offsets and produces one interpolated pixel in two pipeline stages.

Listing 6.11: Separable bilinear interpolator (synthesizable)

```
module mc_bilinear #(
  parameter PIXEL_W = 8,
  parameter FRAC_W  = 8
) (
  input wire      clk,
  input wire      rst_n,
  input wire      valid_in,           // input valid
  input wire [PIXEL_W-1:0] p00, p01, p10, p11, // neighborhood
  input wire [FRAC_W-1:0] frac_x, frac_y,    // fixed-point [0,1]
```

```

output reg                                valid_out,
output reg [PIXEL_W-1:0]                  pixel_out
);
// Stage 1: horizontal interpolation (unsigned math)
localparam MUL_W = PIXEL_W+FRAC_W;
reg [MUL_W-1:0] hor0, hor1;
wire [MUL_W-1:0] mult0 = p00 * ( {1'b0, ( {FRAC_W{1'b1}} - frac_x) } ); //
    p00*(1-frac_x)
wire [MUL_W-1:0] mult1 = p01 * frac_x; // p01*frac_x
wire [MUL_W-1:0] mult2 = p10 * ( {1'b0, ( {FRAC_W{1'b1}} - frac_x) } );
wire [MUL_W-1:0] mult3 = p11 * frac_x;
// Stage registers
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        hor0 <= 0; hor1 <= 0; valid_out <= 0; pixel_out <= 0;
    end else begin
        if (valid_in) begin
            hor0 <= mult0 + mult1; // horizontal top row
            hor1 <= mult2 + mult3; // horizontal bottom row
        end
        // Stage 2: vertical combine and normalization (scale by 2^FRAC_W)
        // vertical: result = (hor0*(1-frac_y) + hor1*frac_y) >> FRAC_W
        if (valid_in) begin
            // compute and round
            reg [MUL_W+FRAC_W:0] vert;
            vert = hor0 * ( {1'b0, ( {FRAC_W{1'b1}} - frac_y) } ) +
                hor1 * frac_y;
            pixel_out <= vert >> (FRAC_W*2); // adjust shift for earlier scaling
            valid_out <= 1;
        end else begin
            valid_out <= 0;
        end
    end
end
endmodule

```

Operational relevance and implications: MC units must be sized to match memory subsystem and entropy decode throughput; Equation (1) guides compute provisioning per block. Trade-offs include:

- Larger tap lengths (higher quality) increase MACs linearly and pressure on TMU/DRAM bandwidth.
- More parallel MC lanes reduce latency but raise SRAM and interconnect area.
- Caching and line-buffer reuse greatly reduce effective memory bandwidth for video; allocating local SRAM per MC unit often yields better energy efficiency than extra arithmetic units.

6.3.4 Parallel decode engines

The previous subsections described how motion compensation parallelizes pixel reconstruction across macroblocks and how entropy decoding (CABAC/range, VLC) creates serial bottlenecks; this subsection examines architectural strategies that reconcile those opposing constraints by scaling decode throughput with multiple parallel engines.

Problem: modern codecs expose partitionable work (tiles, slices, CTU rows) but retain local dependencies in entropy context and reference-frame access. A practical parallel decode engine must therefore schedule independent bitstream partitions, replicate or partition entropy decoders, and arbitrate shared memory access to reference frames and reconstructed buffers.

Analysis: partition granularity drives parallel efficiency. Let N be the number of parallel decode engines and let s be average slice/tile size measured in CTUs per job. Peak decode throughput T_{peak} is bounded by three factors:

$$[H]T_{\text{peak}} \leq \min\left(N \cdot T_{\text{proc}}, T_{\text{entropy}}, \frac{B_{\text{mem}}}{b_{\text{frame}}}\right), \quad (6.18)$$

where T_{proc} is per-engine processing rate (including motion compensation, inverse transforms, deblocking), T_{entropy} is the effective entropy-decoder throughput (possibly increased by decoder replication or multi-substream extraction), and B_{mem} is available memory bandwidth with b_{frame} the bandwidth cost per frame. Improving T_{entropy} often requires either (a) speculative multi-context range decoders, (b) splitting CABAC into independently encoded substreams (tiles/slices), or (c) shallow replication with shared state reconciliation.

Implementation: efficient hardware designs use a small pool of replicated entropy units feeding multiple reconstruction pipelines. A lightweight job dispatcher issues work descriptors (start offset, CTU count, reference IDs). Engines perform bitstream parsing locally when tile boundaries allow; otherwise they request entropy assistance. A reorder buffer enforces in-order frame output when needed by the display controller. Critical hardware blocks:

- Job queue with per-job dependency flags.
- Entropy decoder replicas with work-stealing arbitration.
- Shared reference-frame cache with banked ports to avoid ROP-like contention.

The following synthesizable Verilog implements a parameterized dispatcher and round-robin allocator for N decode engines and a small reorder buffer. It focuses on control and handshaking; per-engine datapaths (entropy, MC) are abstracted as ready/valid interfaces.

Listing 6.12: Parallel decode engine dispatcher and allocator

```

module decode_dispatcher #(
    parameter NENG = 4,
    parameter QDEPTH = 8,
    parameter JOBID_W = 8
) (
    input wire clk,
    input wire rstn,
    // submit job: {jobid, start_offset, ctucount, flags}
    input wire job_in_valid,
    input wire [JOBID_W-1:0] job_in_id,

```

```

output reg          job_in_ready,
// per-engine interface
output reg  [NENG-1:0] eng_valid,
output reg  [JOBID_W-1:0] eng_jobid [0:NENG-1],
input  wire [NENG-1:0] eng_ready,
// completion feedback
input  wire [NENG-1:0] eng_done
);
// simple circular queue for pending jobs
reg [JOBID_W-1:0] q_id [0:QDEPTH-1];
reg [3:0] q_head, q_tail;
reg [QDEPTH-1:0] q_ovfl; // occupancy bitmap
integer i;

// enqueue
always @(posedge clk) begin
  if (!rstn) begin
    q_head <= 0;
    q_tail <= 0;
    job_in_ready <= 1;
  end else begin
    if (job_in_valid && job_in_ready) begin
      q_id[q_tail] <= job_in_id;
      q_tail <= q_tail + 1;
    end
    // available if not full
    job_in_ready <= ((q_tail - q_head) != QDEPTH);
  end
end

// simple round-robin allocate
reg [1:0] rr_ptr;
always @(posedge clk) begin
  if (!rstn) begin
    rr_ptr <= 0;
    for (i = 0; i < NENG; i = i + 1) begin
      eng_valid[i] <= 0;
      eng_jobid[i] <= 0;
    end
  end else begin
    // clear done engines
    for (i = 0; i < NENG; i = i + 1) begin
      if (eng_done[i])
        eng_valid[i] <= 0;
    end

    // allocate next job

```

```

if (q_head != q_tail) begin
  for (i = 0; i < NENG; i = i + 1) begin
    int idx = (rr_ptr + i) % NENG;
    if (eng_ready[idx] && !eng_valid[idx]) begin
      eng_jobid[idx] <= q_id[q_head];
      eng_valid[idx] <= 1;
      q_head <= q_head + 1;
      rr_ptr <= idx + 1;
      disable for; // one allocation per cycle
    end
  end
end
endmodule

```

6.4 Video Encode Acceleration

6.4.1 Motion estimation logic

Building on the prior discussion of decode-side motion compensation and bitstream parsing, this subsection focuses on the encoder-side problem of finding inter-frame motion vectors efficiently in hardware. The goal is to minimize search cost while meeting throughput targets that match a GPU's display and video pipeline.

Problem statement and analysis. Motion estimation (ME) reduces temporal redundancy by finding a displacement that best matches a current block to a reference frame. A common cost is the sum of absolute differences (SAD), possibly extended with a rate term for rate–distortion optimization (RDO). The SAD for an $N \times N$ block is

$$[H]\text{SAD}(u, v) = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} |I_{\text{cur}}(i, j) - I_{\text{ref}}(i + u, j + v)|. \quad (6.19)$$

For full-search with radius R , candidate count equals $(2R + 1)^2$, so computational work grows quadratically with R . High-resolution video combined with real-time constraints requires architectural measures to cut work.

Architectural techniques. Practical hardware combines:

- Multi-level hierarchical search (coarse-to-fine) to prune candidates early.
- Parallel SAD engines evaluating many candidates concurrently, mapped to SIMD or dedicated ME lanes outside SMs.
- Early termination and adaptive thresholds to skip full-block accumulation when partial sums exceed current minimum.
- Sub-pixel refinement after integer-pixel search, using interpolated reference samples and smaller localized searches.

- RDO cost combining SAD and lambda-scaled bitcost: $C(u, v) = \text{SAD}(u, v) + \lambda \cdot R_{\text{bits}}(u, v)$.

Implementation example. A synthesizable streaming SAD engine for a 16×16 block computes one candidate's SAD with one pixel pair consumed per cycle. Multiple identical engines operate in parallel to examine different u, v offsets. Control logic sequences start pulses and collects outputs; an arbiter selects the minimum cost and associated vector.

Listing 6.13: Synthesizable 16x16 streaming SAD engine

```

module sad16x16 (
  input  wire      clk,
  input  wire      rst_n,
  input  wire      start,           // pulse to begin 256-pixel stream
  input  wire [7:0] cur_pixel,      // current block pixel stream
  input  wire [7:0] ref_pixel,      // reference candidate pixel stream
  input  wire      in_valid,        // valid when pixels are present
  output reg      done,            // asserted one cycle when SAD is ready
  output reg [16:0] sad_out         // accumulated SAD (max 65280 < 2^16)
);
  reg [8:0] cnt;                    // count up to 256
  reg [16:0] acc;

  wire [8:0] next_cnt = cnt + 9'd1;
  wire [8:0] max_cnt = 9'd255;

  wire [8:0] diff = (cur_pixel > ref_pixel) ? (cur_pixel - ref_pixel) :
                    (ref_pixel - cur_pixel);

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      cnt <= 9'd0; acc <= 17'd0; sad_out <= 17'd0; done <= 1'b0;
    end else begin
      done <= 1'b0;
      if (start) begin
        cnt <= 9'd0; acc <= 17'd0; done <= 1'b0;
      end else if (in_valid) begin
        acc <= acc + diff;           // accumulate absolute difference
        cnt <= next_cnt;
        if (cnt == max_cnt) begin
          sad_out <= acc + diff;     // final accumulate and latch
          done <= 1'b1;
          cnt <= 9'd0;
          acc <= 17'd0;
        end
      end
    end
  endmodule

```

Operational implications and trade-offs:

- Deploying many parallel engines reduces latency but increases area and memory bandwidth to feed reference tiles.
- Hierarchical search lowers average candidates but adds interpolation cost for sub-pixel stages.
- Early-exit logic reduces compute on easy blocks, but requires branch-like control and comparator hardware.
- Mapping some ME work to general SMs leverages existing SIMD units, but offloads specialized memory streams from the dedicated video path.

6.4.2 Rate control mechanisms

Motion estimation produces per-frame motion vectors and block-level costs that feed the encoder's complexity and expected bits-per-block; rate control consumes those statistics to steer quantization and buffer occupancy so the hardware meets a target bitrate and VBV constraints. The following develops the operational problem, derives a compact control law, and gives a synthesizable hardware implementation suitable for an on-die video encode accelerator.

Problem: constrain instantaneous bit output while maximizing subjective quality. Hardware must satisfy a target bitrate B_{target} and Video Buffering Verifier (VBV) of size S_{vbw} . Let F be the update interval (e.g., one frame or one CTU stripe); the simplest buffer fullness update is

$$[H]\text{buf}_{t+1} = \text{buf}_t + \text{bits}_{\text{gen},t} - B_{\text{target}} \cdot F. \quad (6.20)$$

If buf grows toward S_{vbw} , the encoder must raise quantization (increase QP) to reduce bits. Conversely it may lower QP when buffer is underused to improve quality. Analysis shows practical encoders use a hierarchical controller: GOP-level allocation, frame-level PI control for QP, and block-level adjustments (RDO-driven) using motion costs from motion estimation.

We model rate versus QP by a power-law approximation learned online per sequence: $R(\text{QP}) \approx C \cdot \text{QP}^{-p}$, so a multiplicative change in QP yields predictable rate change. A compact, robust controller in hardware is a discrete PI regulator that computes a QP delta from buffer error and integrated error:

$$\Delta\text{qp} = \text{round} \left(\frac{K_p \cdot e_t + K_i \cdot \sum_{i=0}^t e_i}{\text{SCALE}} \right)$$

where $e_t = B_{\text{target}} \cdot F - \text{bits}_{\text{gen},t}$. Implement using fixed-point integer arithmetic to avoid FP units.

Implementation: below is a synthesizable Verilog module that implements a frame-interval VBV regulator with PI control and clamping to a QP range. It takes frame-level bits observed (from entropy encoder counters) and issues an updated QP to the RDO unit. Gains are integer-scaled constants for synthesis simplicity.

Listing 6.14: VBV PI-based QP controller for frame-interval updates

```

module vbv_qp_ctrl #(
  parameter QP_MIN = 0,
  parameter QP_MAX = 51,

```

```

parameter BW = 32,
parameter Kp = 16,      // proportional gain (fixed scale)
parameter Ki = 2,      // integral gain (fixed scale)
parameter SCALE = 16
) (
    input wire          clk,
    input wire          rst,
    input wire          interval_tick,          // once per update
    interval
    input wire [BW-1:0] bits_generated,          // measured bits this
    interval
    input wire [BW-1:0] target_bits_per_interval, // target bits for
    interval
    output reg  [7:0]    qp_out                  // current QP
);

// signed registers for error accumulation
reg signed [BW+15:0] acc_err;
reg signed [BW+7:0]  err;
reg [BW-1:0]         buf; // optional VBV fullness (simple) - not modeled
fully

always @(posedge clk) begin
    if (rst) begin
        qp_out <= 8'd26; acc_err <= 0; err <= 0; buf <= 0;
    end else if (interval_tick) begin
        // compute error = target - generated
        err <= $signed({1'b0, target_bits_per_interval}) - $signed({1'b0,
            bits_generated});
        acc_err <= acc_err + err;
        // compute delta_qp = (Kp*err + Ki*acc_err) / SCALE
        // use signed widths to avoid overflow; clamp result into safe range
        reg signed [BW+31:0] term_p = Kp * err;
        reg signed [BW+31:0] term_i = Ki * acc_err;
        reg signed [15:0] delta_qp = (term_p + term_i) / SCALE;
        // update and clamp QP
        reg signed [15:0] qp_signed = $signed({8'd0, qp_out}) - delta_qp; //
            subtract because positive err -> target>generated -> lower bits so
            decrease QP? adjust sign as policy
        if (qp_signed < QP_MIN) qp_out <= QP_MIN;
        else if (qp_signed > QP_MAX) qp_out <= QP_MAX;
        else qp_out <= qp_signed[7:0];
        // update simple VBV fullness estimator
        if ($unsigned(bits_generated) >= $unsigned(target_bits_per_interval))
            buf <= buf + (bits_generated - target_bits_per_interval);
        else buf <= buf - (target_bits_per_interval - bits_generated);
    end
end
end

```

endmodule

Implications and trade-offs:

- Responsiveness versus stability: increase K_p/K_i for faster recovery but risk QP oscillation and visible flicker; tuning must consider frame-rate and GOP length.
- Granularity: frame-level PI is cheap (small area), while CTU-level control improves local quality but increases control-path complexity and RDO interactions with motion estimation (more memory and compute).
- Interaction with motion estimation: accurate cost statistics reduce misestimation of $R(QP)$ and reduce VBV excursions; therefore low-latency sharing of block SADs from the ME accelerator improves bitrate accuracy.
- System costs: tighter VBV margins require more frequent QP updates and larger counters on the entropy path, impacting area and power.

6.4.3 Entropy encoder design

The entropy encoder sits downstream of motion estimation and rate control: motion vectors and residuals from the motion engine determine symbol streams, while rate-control supplies target bit budgets and quantization parameters that influence symbol statistics used by the encoder. This subsection analyzes a hardware-friendly entropy coder suitable for modern GPU video encode engines, and presents a synthesizable binary range encoder implementation optimized for tile-parallel throughput and low-latency stream emission.

Problem: entropy coding must convert adaptively-distributed symbols (prediction residuals, transform coefficients, syntax elements) into compact bitstreams while sustaining per-tile throughput targets (e.g., multiple hundreds of MB/s aggregated across slices) and respecting GPU power/area constraints. Sequential arithmetic coders (CABAC-style) achieve high compression but are inherently serial; modern designs therefore partition frames into independent tiles/slices and instantiate parallel encoder engines, or use renormalization-friendly coders such as range coders and rANS (asymmetric numeral systems) to balance compression and parallelism.

Analysis: the theoretical lower bound is Shannon entropy

$$[H]H(X) = - \sum_i p_i \log_2 p_i, \quad (6.21)$$

and practical encoders aim to produce average code length L close to H . A binary range coder maintains an interval size R and splits it by probability p for symbol 1; the coder emits bytes when the high-order bits of the interval stabilize. Renormalization repeatedly shifts R and outputs bytes:

$$[H]\text{while } R < R_{\min} : \quad R \leftarrow R \ll s, \text{ emit_byte}(\lfloor \text{low}/2^{24} \rfloor), \text{ low} \leftarrow (\text{low} \& 0xFFFFFFFF) \ll s. \quad (6.22)$$

Practical fixed-point probability models (11–12 bits) simplify multipliers into shift-and-add engines or small LUTs for high clock rates in an SM-adjacent encode block.

Implementation: below is a synthesizable Verilog module implementing a single binary range encoder engine intended as an entropy-encoder tile. It accepts a stream of binary symbols

with a fixed-point context probability and emits bytes with ready/valid handshaking. Multiple instances can be instantiated per GPU video engine to encode tiles in parallel; an output arbiter or DMA concatenates emitted bytes into the frame bitstream.

Listing 6.15: Binary range encoder engine (synthesizable).

```

module range_encoder #(
  parameter PROB_BITS = 11 // fixed-point probability resolution
) (
  input wire      clk,
  input wire      rst_n,
  // input symbol (1-bit) with valid/ready
  input wire      in_valid,
  output reg      in_ready,
  input wire      in_sym,          // symbol bit
  input wire [PROB_BITS-1:0] in_p, // probability of 1 in Q(PROB_BITS)
  // byte output (8-bit) with valid
  output reg [7:0] out_byte,
  output reg      out_valid,
  input wire      out_ready
);

// Internal state: low (32 bits), range (32 bits), buffer for pending bytes
reg [31:0] low;
reg [31:0] range;
reg [7:0] buffer; // last emitted byte staging (not a FIFO)
localparam R_MIN = 32'h01000000; // renorm threshold (~24-bit)
// combinational probability multiply -> split
wire [31:0] split; // range * p >> PROB_BITS
assign split = (range * in_p) >> PROB_BITS;

// Simple FSM: IDLE -> ENCODE -> RENORM -> OUTPUT
typedef enum reg [1:0] {IDLE=0, ENCODE=1, RENORM=2} state_t;
reg [1:0] state;

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    low    <= 32'd0;
    range  <= 32'hFFFFFFFF;
    state  <= IDLE;
    in_ready <= 1'b1;
    out_valid <= 1'b0;
    out_byte <= 8'd0;
  end else begin
    case (state)
      IDLE: begin
        out_valid <= 1'b0;
        if (in_valid && in_ready) begin
          // perform binary range update

```



```

        if (in_sym) begin
            low <= low + (range - split);
            range <= split;
        end else begin
            range <= range - split;
        end
        in_ready <= 1'b0; // consume input for this cycle
        state <= RENORM;
    end
end
RENORM: begin
    // renormalize: emit top byte(s) while range < R_MIN
    if (range < R_MIN) begin
        // emit top byte from low >> 24
        if (!out_valid || (out_valid && out_ready)) begin
            out_byte <= low[31:24]; // output MSB
            out_valid <= 1'b1;
            // shift low and range
            low <= (low << 8);
            range <= (range << 8);
            // allow next input when one byte emitted
            in_ready <= 1'b1;
            state <= IDLE;
        end else begin
            // back-pressure: wait until consumer accepts byte
            out_valid <= out_valid;
        end
    end else begin
        // no renorm needed; ready to accept next input
        in_ready <= 1'b1;
        out_valid <= 1'b0;
        state <= IDLE;
    end
end
endcase
end
endmodule

```

Implications and trade-offs:

- Parallelism: instantiate multiple `range_encoder` engines per video tile to match motion-estimate throughput; one engine per slice minimizes inter-tile synchronization.
- Compression vs latency: arithmetic/range coders deliver better compression than simple Huffman but require renormalization stalls and serial dependencies; area can be reduced using small LUT-based multiplication for fixed $PROB_{ITS}$.
- Integration: connect encoder outputs to a lightweight DMA or byte-merge unit; maintain

per-engine byte FIFOs to absorb back-pressure from shared output fabric.

- Power/area: increasing `PROB_BITS` improves model fidelity but costs multiplier complexity; choose 10–12 bits for a practical balance in SM-adjacent video engines.

6.4.4 Multi-format support

The previous discussion of entropy coding choices and the rate-control feedback loop motivates flexible front-end and back-end adapters that present a unified datapath to CABAC, rANS, or other entropy engines while preserving bitrate targets. Multi-format support therefore becomes an engineering problem of functional coverage, memory bandwidth budgeting, and datapath reuse across codecs and chroma/sample formats.

A practical design objective is to accept a wide set of input formats (planar, semiplanar, packed), bit depths (8/10/12), chroma subsampling (4:4:4, 4:2:2, 4:2:0), and metadata (colorimetry, HDR) while minimizing format-specific hardware islands. Analysis proceeds by decomposing into format negotiation, conversion, and shared acceleration units:

- Format negotiation: runtime caps and capability registers allow the host to select encoding profile, bit depth, chroma layout, and tile/stripe size. Negotiation ensures the TMU/DRAM accesses and ROP-like writebacks align to tile granularity.
- Conversion front-end: planar-to-semiplanar converters, chroma up/down samplers, and bit-depth packers normalize inputs to a canonical internal representation used by motion estimation, transforms, and quantizers.
- Shared datapath: 2D transforms (DCT/ASTC-like), quantizers, and RDO modules are parameterized for precision and scaling so the same silicon services H.264, HEVC, VP9, and AV1; only entropy stage switches between CABAC, CAVLC, rANS, or prefix-coded trees.
- Parallelization: tile-based partitioning maps tiles to parallel encode engines, enabling independent entropy streams where formats allow. AV1’s superblock independence benefits multiple entropy contexts; HEVC CABAC requires careful context reset policies at slice boundaries.

Operationally, memory bandwidth dominates. For a stream with width W , height H , frame rate F , bit depth D bits per sample, and C channels, the raw input bandwidth requirement is

$$[H]B = W \cdot H \cdot F \cdot \frac{D \cdot C}{8} \text{ bytes/sec.} \quad (6.23)$$

Choosing semiplanar layouts like NV12 reduces address generation complexity and cache pressure versus packed RGB, lowering effective B via cache-friendly accesses and tile-based reuse.

A compact, synthesizable format-adapter shows the implementation pattern: accept planar Y and UV streams and emit semiplanar outputs in adjustable bit-depth packing.

Listing 6.16: Planar-to-semiplanar adapter supporting 8/10-bit modes

```
module format_adapter #(
  parameter DATA_W = 32 // output bus width
) (
```

```

    input  clk,
    input  rstn,
    // Y-plane stream (byte lanes valid)
    input  y_valid,
    input  [7:0] y_data,
    output y_ready,
    // UV-plane stream (interleaved U then V bytes)
    input  uv_valid,
    input  [7:0] uv_data,
    output uv_ready,
    // Config: 0 = NV12 8-bit, 1 = P010 10-bit (packed into 16-bit lanes)
    input  mode_10bit,
    // Output stream (DATA_W aligned)
    output reg out_valid,
    output reg [DATA_W-1:0] out_data,
    input  out_ready
);
    // Simple FIFOs implied by ready signals; here flow-through pass-through
    // logic.
    assign y_ready  = out_ready; // backpressure prop
    assign uv_ready = out_ready;

    // Pack two pixels per DATA_W when 8-bit; in 10-bit mode pack into 16-bit
    // lanes.
    reg [15:0] pack_buf;
    reg        pack_half;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            out_valid <= 0;
            out_data  <= 0;
            pack_buf  <= 0;
            pack_half <= 0;
        end else begin
            out_valid <= 0;
            if (y_valid && uv_valid && out_ready) begin
                if (!mode_10bit) begin
                    // NV12: output 4 bytes per 32-bit word: Y0 Y1 U V
                    out_data <= {y_data, y_data, uv_data, uv_data}; // simple repeated
                        example
                    out_valid <= 1;
                end else begin
                    // P010-like: pack 10-bit into upper bits of 16-bit lanes
                    // Here we zero-extend 8->10 bits for simplicity.
                    pack_buf <= {6'b0, y_data}; // 16-bit lane containing 10-bit sample
                    out_data <= {pack_buf, {6'b0, uv_data}}; // two 16-bit lanes form
                        32-bit word
                    out_valid <= 1;
                end
            end
        end
    end

```

```

        end
    end
end
endmodule

```

Design implications and trade-offs:

- Supporting many formats reduces host preprocessing but increases format-conversion logic and verification cost.
- A canonical internal representation simplifies transform/quant blocks but may add conversion latency and temporary buffering, increasing DRAM pressure.
- Tile-based parallelism and semiplanar memory layouts are essential levers to lower bandwidth B in Eq. (1) and to scale entropy engines across SM/CU instances.

6.5 Video Processing Pipeline

6.5.1 Scaling and filtering

Building on the decode and motion-compensation primitives described earlier, scaling and filtering convert decoded picture samples into the spatial resolution and quality required by subsequent display or post-processing stages. The goal is a hardware-efficient, low-latency resampler that supports arbitrary scaling ratios and integrates with TMU-like sampling semantics used by texture units.

Scaling is typically implemented as two separable 1-D operations: horizontal resampling followed by vertical resampling. Bilinear interpolation is the lowest-complexity separable filter that yields acceptable quality for many video and UI workloads; its core operation computes a weighted average of four neighboring samples. For a continuous image intensity function f , the bilinear reconstruction at fractional positions $0 \leq u, v < 1$ is

$$[H]g(u, v) = (1 - u)(1 - v)f_{00} + u(1 - v)f_{10} + (1 - u)v f_{01} + uv f_{11}, \quad (6.24)$$

where f_{ij} are the integer-sampled pixels at the four surrounding grid points. Implementing (6.24) in fixed-point uses two cascaded linear interpolators to reduce multiplier count and latency: first horizontally produce two intermediates, then vertically interpolate those.

Analysis highlights:

- Fixed-point precision (choose F fractional bits) must balance quantization error vs. multiplier cost; $F = 12 - 16$ often suffices for 8-bit video.
- A separable design reduces complexity from four multiplies per channel to four multiplies across two stages when reusing intermediate results.
- Line buffering is required to provide the vertical neighbors; this is commonly a pair of line FIFOs sized to the input width, enabling streaming decode into the scaler.

The following synthesizable Verilog implements a streaming separable bilinear scaler with parameterized input width and fixed-point phase accumulators. It assumes input pixel stream of 24-bit RGB, and scale factors supplied as 16.16 fixed-point values.

Listing 6.17: Separable bilinear scaler (streaming).

```

module scaler_bilinear #(
    parameter IN_WIDTH = 1920,
    parameter FRAC = 16
) (
    input  wire      clk,
    input  wire      rst,
    input  wire [23:0] in_pixel,    // {R[23:16],G[15:8],B[7:0]}
    input  wire      in_valid,
    input  wire [31:0] scale_x,     // 16.16 fixed-point
    input  wire [31:0] scale_y,     // 16.16 fixed-point
    output reg [23:0] out_pixel,
    output reg      out_valid
);

    // line buffers for two most recent lines
    reg [23:0] line0 [0:IN_WIDTH-1];
    reg [23:0] line1 [0:IN_WIDTH-1];
    integer write_x;
    integer line_write_ptr;
    reg toggle; // 0: line0 is older, 1: line1 older

    // phase accumulators for output sampling
    reg [31:0] phase_x;
    reg [31:0] phase_y;
    reg [31:0] phase_x_step;
    reg [31:0] phase_y_step;

    // simple state: fill first two lines, then produce outputs
    reg [1:0] line_count;

    // initialize
    always @(posedge clk) begin
        if (rst) begin
            write_x <= 0;
            line_write_ptr <= 0;
            toggle <= 0;
            phase_x <= 0;
            phase_y <= 0;
            phase_x_step <= scale_x;
            phase_y_step <= scale_y;
            line_count <= 0;
            out_valid <= 0;
        end else begin
            phase_x_step <= scale_x;
            phase_y_step <= scale_y;
            if (in_valid) begin
                // write incoming pixel into the current write line

```

```

if (toggle == 0) line0[write_x] <= in_pixel;
else                line1[write_x] <= in_pixel;
write_x <= write_x + 1;
if (write_x == IN_WIDTH-1) begin
    write_x <= 0;
    line_count <= (line_count<2) ? line_count+1 : 2;
    toggle <= ~toggle; // advance ring
end
end

// produce output only when at least two lines buffered
if (line_count >= 2) begin
    // sample indices and fractional parts
    // ix = phase_x >> FRAC, fx = lower FRAC bits
    integer ix; integer iy;
    reg [FRAC-1:0] fx, fy;
    reg [23:0] p00, p10, p01, p11;
    reg [23:0] h0, h1;
    ix = phase_x[31:FRAC]; fx = phase_x[FRAC-1:0];
    iy = phase_y[31:FRAC]; fy = phase_y[FRAC-1:0];

    // clamp indices
    if (ix >= IN_WIDTH-1) ix = IN_WIDTH-1;
    // choose lines: top and bottom (older/newer depending on toggle)
    if (toggle == 0) begin
        p00 = line1[ix];          // top
        p10 = line1[(ix==IN_WIDTH-1)?ix:ix+1];
        p01 = line0[ix];          // bottom
        p11 = line0[(ix==IN_WIDTH-1)?ix:ix+1];
    end else begin
        p00 = line0[ix];
        p10 = line0[(ix==IN_WIDTH-1)?ix:ix+1];
        p01 = line1[ix];
        p11 = line1[(ix==IN_WIDTH-1)?ix:ix+1];
    end

    // horizontal interpolation for each channel
    // R channel
    integer r00,r10,r01,r11;
    integer rh0,rh1;
    r00 = p00[23:16]; r10 = p10[23:16];
    r01 = p01[23:16]; r11 = p11[23:16];
    rh0 = (( ( (1<<FRAC) - fx ) * r00 ) + ( fx * r10 )) >> FRAC;
    rh1 = (( ( (1<<FRAC) - fx ) * r01 ) + ( fx * r11 )) >> FRAC;
    // vertical
    integer rout;
    rout = (( ( (1<<FRAC) - fy ) * rh0 ) + ( fy * rh1 )) >> FRAC;

```

```

// repeat for G channel
integer g00,g10,g01,g11,gh0,gh1,gout;
g00 = p00[15:8]; g10 = p10[15:8];
g01 = p01[15:8]; g11 = p11[15:8];
gh0 = ((( (1<<FRAC) - fx ) * g00 ) + ( fx * g10 )) >> FRAC;
gh1 = ((( (1<<FRAC) - fx ) * g01 ) + ( fx * g11 )) >> FRAC;
gout = ((( (1<<FRAC) - fy ) * gh0 ) + ( fy * gh1 )) >> FRAC;

// B channel
integer b00,b10,b01,b11,bh0,bh1,bout;
b00 = p00[7:0]; b10 = p10[7:0];
b01 = p01[7:0]; b11 = p11[7:0];
bh0 = ((( (1<<FRAC) - fx ) * b00 ) + ( fx * b10 )) >> FRAC;
bh1 = ((( (1<<FRAC) - fx ) * b01 ) + ( fx * b11 )) >> FRAC;
bout = ((( (1<<FRAC) - fy ) * bh0 ) + ( fy * bh1 )) >> FRAC;

out_pixel <= {rout[7:0], gout[7:0], bout[7:0]};
out_valid <= 1'b1;

// advance phases
phase_x <= phase_x + phase_x_step;
phase_y <= phase_y + phase_y_step;
end else begin
    out_valid <= 1'b0;
end
end
end
endmodule

```

Design implications and trade-offs:

- Using separable bilinear yields low multiplier count and deterministic latency, suitable for real-time decode-to-display paths and TMU-like texture sampling.
- Higher-quality filters (bicubic, Lanczos) increase taps, memory footprint, and multiplier latency, hurting throughput and power in SM/TMU-limited designs.
- Fixed-point fractional width directly trades silicon area and dynamic range against visible banding; choose FRAC per target content (HDR requires more bits).
- Line-buffer sizing and bandwidth impose memory requirements; tile-based buffering can reduce BRAM pressure but increases control logic complexity.

6.5.2 Color space conversion

Following the discussion of sampling and filter kernels, color conversion is the next fixed-function step that maps sampled RGB values into the target color representation used for compression, storage, or display. This subsection analyzes the linear transforms used in video

pipelines, derives fixed-point implementation choices suitable for DSP-based hardware, and provides a synthesizable Verilog datapath for a pipelined RGB→YCbCr block.

Color conversion problem → analysis: many video codecs and displays use luma–chroma representations (YCbCr or Y′CbCr) because they separate luminance from chrominance and enable chroma subsampling (e.g. YUV_420). The conversion is linear in the gamma-corrected domain and can be expressed as a 3×3 matrix; using the luma coefficients K_r , K_g , K_b (e.g., BT.709 values $K_r = 0.2126$, $K_g = 0.7152$, $K_b = 0.0722$) yields the standard transform

$$[H] \begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} K_r & K_g & K_b \\ -\frac{K_r}{2(1-K_b)} & -\frac{K_g}{2(1-K_b)} & \frac{1}{2(1-K_b)} \\ \frac{1}{2(1-K_r)} & -\frac{K_g}{2(1-K_r)} & -\frac{K_b}{2(1-K_r)} \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}. \quad (6.25)$$

Operationally, a hardware module implements three weighted sums followed by fixed-point scaling, rounding, and clamping to the output bit-depth (commonly 8 or 10 bits). For hardware efficiency, coefficients are quantized to a Q-format and multiplications mapped to DSP slices; accumulation width must avoid overflow for max input values.

Implementation choices (practical):

- Use `COEFF_FRAC = 14` or `15` for Q1.14/Q1.15 to bound coefficient quantization error below 0.2% for BT.709.
- Pipeline the multiply stage and the add/shift stage to meet high clock frequencies (SM/T-MUs are irrelevant here; this is fixed-function video engine logic).
- Support both full-range and limited-range output by applying a small offset and clamping to [16,235] for 8-bit limited-range video.

A synthesizable Verilog implementation follows; it is fully pipelined and parameterized for input/output widths and coefficient fractional bits.

Listing 6.18: Pipelined RGB-to-YCbCr converter (BT.709 coefficients, Q14)

```

module color_convert #(
  parameter IW = 8,                // input per-channel bits
  parameter OW = 8,                // output per-channel bits
  parameter COEFF_FRAC = 14        // fractional bits for coeffs (Q1.COEFF_FRAC
)
) (
  input  wire      clk,
  input  wire      rstn,
  input  wire      valid_in,
  input  wire [IW-1:0] R,
  input  wire [IW-1:0] G,
  input  wire [IW-1:0] B,
  output reg       valid_out,
  output reg [OW-1:0] Y,
  output reg [OW-1:0] Cb,
  output reg [OW-1:0] Cr

```



```

);
// BT.709 coefficients scaled by 2^COEFF_FRAC
localparam integer KR = 3483;    // round(0.2126*2^14)
localparam integer KG = 11736;   // round(0.7152*2^14)
localparam integer KB = 1183;    // round(0.0722*2^14)

// Precomputed matrix entries for Cb and Cr in Q14 (signed)
// cb = a0*R + a1*G + a2*B
localparam integer CB_R = -((KR) / (2*(1.0-0.0722)) * (1<>> COEFF_FRAC) >
    ((1<>> COEFF_FRAC));

// For Cb/Cr use precomputed integer coefficients (Q14)
wire signed [IW+COEFF_FRAC+7:0] mC1 = $signed({1'b0,R}) * $signed(CB_R_I);
wire signed [IW+COEFF_FRAC+7:0] mC2 = $signed({1'b0,G}) * $signed(CB_G_I);
wire signed [IW+COEFF_FRAC+7:0] mC3 = $signed({1'b0,B}) * $signed(CB_B_I);
reg signed [IW+COEFF_FRAC+9:0] accCb;
always @(posedge clk) if (!rstn) accCb <= 0; else accCb <= mC1 + mC2 + mC3;

// final stage: round/shift + bias (128 center for chroma) + clamp
wire [OW-1:0] cb_out = $unsigned( (accCb + (1 << (COEFF_FRAC-1)) ) >>>
    COEFF_FRAC ) + 8'd128;
// Cr similar path (reuse logic omitted for brevity but must be implemented
    in production)

// output valid and registers
always @(posedge clk) begin
    if (!rstn) begin
        valid_out <= 1'b0; Y <= 0; Cb <= 0; Cr <= 0;
    end else begin
        valid_out <= valid_in; // one-cycle latency alignment with pipeline
            depth simplified
        Y <= y_out;
        Cb <= cb_out;
        Cr <= cb_out; // placeholder: in production compute Cr separately
    end
end
endmodule

```

Design implications and trade-offs:

- Mapping to DSP slices yields low-latency, high-throughput conversion but consumes scarce DSP resources; scalar compute on SMs trades DSP use for higher-latency threads and memory traffic.
- Choosing `COEFF_FRAC` balances quantization error versus accumulator width and power; 14–15 fractional bits are typical.
- Performing color conversion before chroma subsampling reduces interpolation error but increases memory movement if conversion expands chroma data width.

- Supporting multiple standards (BT.601, BT.709, BT.2020) requires coefficient banks and reconfiguration state, increasing control logic area.

6.5.3 Deinterlacing and denoising

Following color-space conversion and spatial scaling, deinterlacing and denoising operate on temporally adjacent samples and reconstructed chroma/luma planes, so their algorithms must respect chroma subsampling, TMU fetch alignment, and any motion vectors produced by the decoder or motion-estimation (ME) engine.

Deinterlacing problem: interlaced sources deliver alternating field samples; naive weaving causes combing on motion, bob halves vertical resolution. Motion-adaptive and motion-compensated methods detect motion and select per-pixel blending or motion-compensated reconstructions. Denoising problem: temporal noise benefits from accumulation across frames, but misaligned content (motion) requires compensation to avoid ghosting. Key resources are: the dedicated ME hardware in the video engine, a small SRAM temporal buffer per video pipeline for K -frame windows, TMU-assisted subpixel fetches, and SM compute for heavier filters or neural inference on tensor cores.

Analysis and algorithmic form. Motion-compensated temporal filtering gives the best quality/efficiency balance when ME vectors are available. For pixel position p at current frame t , the filtered estimate is

$$\hat{I}_t(p) = \frac{\sum_{k=0}^K w_k(p) I_{t-k}(p + v_{t,k}(p))}{\sum_{k=0}^K w_k(p)}, \quad (6.26)$$

where $v_{t,k}(p)$ are motion vectors mapping past samples into current coordinates, and weights $w_k(p)$ combine photometric similarity and motion confidence. Spatial denoising (e.g., bilateral or non-local means) can be applied either pre- or post-temporal accumulation to preserve edges.

Implementation patterns:

- Lightweight pipeline in display engine: use ME unit to produce block vectors at 8×8 or 16×16 granularity; perform per-block confidence test in logic; if static, use weave; if motion, apply per-pixel temporal blend computed in fixed-point DSPs or SMs.
- High-quality path on SMs or tensor cores: perform per-pixel motion compensation using TMU fetches into shared memory, then apply bilateral or CNN denoiser using tensor cores for matrix multiply lower-latency inference.

Example CUDA-style kernel (illustrative; uses TMU texture fetches and precomputed motion and weights):

Listing 6.19: Motion-compensated temporal accumulation kernel

```
__global__ void temporal_accum(cudaTextureObject_t tex[], // ring of textures
                             float2* motionVec,         // per-pixel vectors
                             float* weight,              // per-pixel weight
                             // per frame
                             float* out, int W, int H, int K) {
    int x = blockIdx.x*blockDim.x + threadIdx.x;
    int y = blockIdx.y*blockDim.y + threadIdx.y;
    if (x>=W||y>=H) return;
```

```

int p = y*W + x;
float num=0.f, den=0.f;
for (int k=0;k<tex[k], sx+0.5f, sy+0.5f); // TMU subpixel
    float w = weight[p + k*W*H];           // confidence+photometric
    num += w*sample; den += w;
}
out[p] = den>0 ? num/den : tex2D(tex[0], x+0.5f, y+0.5f);
}

```

Practical considerations and trade-offs:

- Latency vs quality: larger K yields better noise reduction but increases frame buffering and decode-to-display latency.
- Compute vs dedicated logic: SM/tensor-core denoisers yield highest quality but consume shader resources and power; lightweight fixed-point DSP implementations reduce power at quality cost.
- Memory bandwidth: temporal accumulation reuses data via on-chip SRAM and TMU caches; motion vector precision and subsampling affect cache locality.
- Interoperability: reuse ME vectors from decode engine avoids duplicated computation; provide fallbacks when ME is unreliable.

Design implications: allocate a small temporal SRAM window per pipeline to limit VRAM traffic; expose quality modes to OS compositor so latency-sensitive use cases can prefer low- K fast filters; architect ME outputs with block confidence to minimize ghosting and reduce unnecessary motion-compensated fetches.

6.5.4 HDR tone mapping

Continuing from deinterlacing/denoising cleanup and the conversion into a scene-linear working space, tone mapping becomes the deterministic step that compresses scene-referred HDR signals into the display-referred range while preserving perceptual detail and color consistency.

HDR tone mapping problem → analysis:

- Problem: map scene luminance L_{scene} spanning many orders of magnitude to display-referred luminance L_{disp} limited by panel peak and quantization, while avoiding clipping, crushing midtones, or hue shifts.
- Key constraints: target peak luminance, color gamut mapping, pipeline precision (native 10/12/16-bit buffers), and throughput matching TMU/ROP rates to avoid stalling the pixel pipeline.

Common mapping model (operational relevance): apply exposure scale, compute luminance, apply a curve with a controllable shoulder to preserve highlights, then reintegrate chroma. A widely used filmic-style operator usable in hardware is the modified Reinhard curve:

$$[H]L_{\text{mapped}} = \frac{L_{\text{exposed}} \left(1 + \frac{L_{\text{exposed}}}{W^2} \right)}{1 + L_{\text{exposed}}}, \quad L_{\text{exposed}} = E \cdot L_{\text{scene}} \quad (6.27)$$

where E is an exposure scalar and W is a display white-point control that adjusts the shoulder. This formula provides a soft roll-off avoiding hard clipping.

Implementation options and pipeline placement:

1. Fixed-function tone-mapper inside the video processing pipeline: lowest latency and power; implemented as a pipelined arithmetic unit with piecewise rational approximations and small LUTs for curve parameters. Suited for real-time decode/display of HDR video streams.
2. Compute-shader or fragment-shader implementation on SM/CU: most flexible for ACES/creative looks, but consumes SM resources and can contend with ML/compute workloads.
3. Hybrid: coarse LUT stored in TMU-accessible memory plus small polynomial correction in a fast datapath to trade off LUT memory vs ALU cost.

Practical considerations:

- Precision: keep intermediate in 16-bit float or 12-bit fixed-point to avoid banding; apply blue-noise dither before quantization when reducing to 10-bit.
- Gamut mapping: perform luminance-only compression, then apply saturations scaling in perceptual space such as CIECAM02 or PQ-to-XYZ chains to avoid hue shifts.
- Performance: a fully pipelined fixed-function unit can sustain ROP writeback rates; shader-based solutions may need batching and reduced-resolution prefiltering (grain-aware) to meet display refresh.

Example GPU-friendly GLSL fragment implementing exposure + modified Reinhard tone mapping (suitable for a post-decode pass):

Listing 6.20: GLSL tone-mapping fragment shader

```
#version 450
layout(location=0) in vec3 hdrColor; // scene-linear RGB
layout(push_constant) uniform PC { float exposure; float white; } pc;
out vec4 outColor;
float luminance(vec3 c){ return dot(c, vec3(0.2126,0.7152,0.0722)); } // Rec
    .709 Luma
void main(){
    vec3 c = hdrColor * pc.exposure; // apply exposure
    float L = luminance(c);
    float Lm = (L*(1.0 + L/(pc.white*pc.white)))/(1.0 + L); // eq. (1)
    vec3 mapped = c * (Lm / max(L,1e-6)); // scale chroma
    mapped = pow(mapped, vec3(1.0/2.2)); // display gamma
    outColor = vec4(clamp(mapped,0.0,1.0),1.0);
}
```

Design implications and trade-offs:

- Fixed-function units minimize latency and power but reduce artistic flexibility for creative looks.

- LUT-based approaches reduce ALU cost but increase TMU/DRAM bandwidth and meta-data complexity.
- Precision and dithering are required to avoid quantization artifacts; choosing 16-bit intermediates increases memory bandwidth but reduces visible banding for high-contrast HDR content.

6.6 Section 6: Verilog Implementation

6.6.1 Display timing generator

The preceding discussion of display controller responsibilities established the timing primitives (front/back porch, sync pulse, active region) used across multi-display and pixel pipelines; the implementation below converts those primitives into a synthesizable timing generator suitable for SM-level display engines and TMU/ROP synchronization.

We first formalize the timing totals used by the state machine and counters:

$$[H]H_{\text{total}} = H_{\text{active}} + H_{\text{fp}} + H_{\text{sync}} + H_{\text{bp}}, \quad V_{\text{total}} = V_{\text{active}} + V_{\text{fp}} + V_{\text{sync}} + V_{\text{bp}}. \quad (6.28)$$

Problem: generate deterministic HSYNC/VSYNC, horizontal/vertical blanking and pixel-valid strobes from programmable timing parameters at pixel clock rate so downstream stages (pixel pipeline, scaler, encoder such as DSC) sample correctly.

Analysis:

- Use free-running horizontal counter (x) and vertical counter (y) driven by pixel clock.
- HSYNC window begins after front porch: $x \in [H_{\text{active}} + H_{\text{fp}}, H_{\text{active}} + H_{\text{fp}} + H_{\text{sync}})$.
- VSYNC window analogous on y .
- Pixel valid when inside active rectangle: $x < H_{\text{active}}$ and $y < V_{\text{active}}$.

Listing 6.21: Display Timing Generator (DTG)

```

module dtg #(
    parameter H_ACTIVE = 1920,
    parameter H_FP      = 88,
    parameter H_SYNC   = 44,
    parameter H_BP      = 148,
    parameter V_ACTIVE = 1080,
    parameter V_FP      = 4,
    parameter V_SYNC   = 5,
    parameter V_BP      = 36,
    parameter HSYNC_POL = 1'b0, // active-low
    parameter VSYNC_POL = 1'b0
) (
    input  wire      clk,          // pixel clock
    input  wire      rst_n,
    output reg       hsync,

```

```

output reg      vsync,
output reg      hblank,
output reg      vblank,
output wire     pixel_valid
);

localparam H_TOTAL = H_ACTIVE + H_FP + H_SYNC + H_BP;
localparam V_TOTAL = V_ACTIVE + V_FP + V_SYNC + V_BP;

reg [$clog2(H_TOTAL)-1:0] x;
reg [$clog2(V_TOTAL)-1:0] y;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        x <= 0;
        y <= 0;
        hsync <= ~HSYNC_POL;
        vsync <= ~VSYNC_POL;
        hblank <= 1'b1;
        vblank <= 1'b1;
    end else begin
        // horizontal counter
        if (x == H_TOTAL - 1) begin
            x <= 0;
            if (y == V_TOTAL - 1)
                y <= 0;
            else
                y <= y + 1;
        end else begin
            x <= x + 1;
        end

        // timing signals
        hblank <= (x >= H_ACTIVE);
        vblank <= (y >= V_ACTIVE);
        // HSYNC pulse window after front porch
        hsync <= ((x >= (H_ACTIVE + H_FP)) && (x < (H_ACTIVE + H_FP +
            H_SYNC)))
            ? HSYNC_POL
            : ~HSYNC_POL;
        // VSYNC pulse window after vertical front porch
        vsync <= ((y >= (V_ACTIVE + V_FP)) && (y < (V_ACTIVE + V_FP +
            V_SYNC)))
            ? VSYNC_POL
            : ~VSYNC_POL;
    end
end
end

```

```
assign pixel_valid = (x < H_ACTIVE) && (y < V_ACTIVE);
```

```
endmodule
```

Operational implications and trade-offs:

- Programmable porches/sync let the same DTG support many panel timings, but counters' bit widths must cover maximum resolutions; larger counters increase area but avoid overflow.
- Pixel_valid gating reduces framebuffer read bandwidth during blanking; misalignment of DTG with memory fetch latencies can cause underruns, so coordinate with DMA prefetch depth in the display pipeline.
- Sync polarity and interlaced/frame field support add complexity; adding field toggles or double-buffered parameter registers enables glitch-free mode switches.

6.6.2 Video decoder FSM

The display timing generator provided frame and line timing guarantees that the decoder FSM must honour when scheduling output writes to framebuffers and when issuing DMA bursts to the display engine. The FSM below therefore coordinates bitstream fetch, parsing, entropy decode, motion compensation, and output write stages under backpressure from memory and the display scanout.

Problem: meet real-time decode of compressed frames while avoiding buffer underflow/overflow and respecting display timing. Analysis identifies stages with variable latency (entropy decode) and mostly deterministic compute (motion compensation). The controller must:

- sequence stages with valid-ready handshakes,
- allow pipeline stalls and flushes on seek or error,
- support burst DMA with alignment to tile/macroblock boundaries,
- expose simple telemetry for throughput tuning.

A common throughput constraint is how many hardware decode engines N are required for target playback:

$$[H]N \geq \left\lceil \frac{W \cdot H \cdot \text{fps}}{A_{\text{MB}} \cdot T_{\text{engine}}} \right\rceil \quad (6.29)$$

where W, H are frame dimensions, A_{MB} is macroblock area (e.g., 16×16), and T_{engine} is macroblocks decoded per second per engine. Equation (6.29) guides resource partitioning across CUs or dedicated decode cores.

Implementation: the FSM below is a synthesizable, parameterized Verilog module. It uses encoded states to minimize area and provides external handshake signals: `dma_req`, `parse_start`, `entropy_start`, `mc_start`, and `write_start`. It supports error recovery and flush. The module assumes ready/done pulses from downstream units and uses an internal credit counter to avoid DMA overcommit.

Listing 6.22: Video decoder top-level FSM controlling decode pipeline

```

module video_decoder_fsm #(
    parameter IDLE = 3'd0,
    parameter FETCH = 3'd1,
    parameter PARSE = 3'd2,
    parameter ENTROPY = 3'd3,
    parameter MC = 3'd4,
    parameter WRITEOUT = 3'd5,
    parameter ERROR = 3'd6
) (
    input wire      clk,
    input wire      rst_n,
    input wire      start,           // request to decode a frame
    input wire      dma_ack,         // DMA accepted
    input wire      fetch_done,      // fetch completed
    input wire      parse_done,      // parser finished
    input wire      entropy_done,    // entropy decoder finished
    input wire      mc_done,         // motion compensation finished
    input wire      write_done,      // output write finished
    input wire      stall,           // backpressure from memory/display
    input wire      flush,           // abort & flush
    output reg      dma_req,         // request DMA fetch
    output reg      parse_start,     // start parser
    output reg      entropy_start,   // start entropy decoder
    output reg      mc_start,        // start motion compensation
    output reg      write_start,     // start frame write
    output reg      done,            // frame done
    output reg [2:0] state_out      // debug state
);

reg [2:0] state, next_state;
reg [3:0] dma_credits; // prevent issuing too many DMA bursts

// next-state combinational logic
always @(*) begin
    // default deasserts
    dma_req = 1'b0;
    parse_start = 1'b0;
    entropy_start = 1'b0;
    mc_start = 1'b0;
    write_start = 1'b0;
    done = 1'b0;
    next_state = state;
    case (state)
        IDLE: if (start && !stall) begin
            dma_req = 1'b1; // fetch header/bitstream
            next_state = FETCH;

```



```

        end
    FETCH: if (fetch_done) begin
        parse_start = 1'b1;
        next_state = PARSE;
    end else if (flush) next_state = IDLE;
    PARSE: if (parse_done) begin
        entropy_start = 1'b1;
        next_state = ENTROPY;
    end else if (flush) next_state = IDLE;
    ENTROPY: if (entropy_done) begin
        mc_start = 1'b1;
        next_state = MC;
    end else if (flush) next_state = IDLE;
    MC: if (mc_done) begin
        write_start = 1'b1;
        next_state = WRITEOUT;
    end else if (flush) next_state = IDLE;
    WRITEOUT: if (write_done) begin
        done = 1'b1;
        next_state = IDLE;
    end else if (flush) next_state = IDLE;
    ERROR: begin
        // wait for reset or explicit restart
        if (flush) next_state = IDLE;
    end
    default: next_state = IDLE;
endcase
end

// sequential updates
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        dma_credits <= 4'd0;
    end else begin
        state <= next_state;
        // dma credit management when dma_ack asserted
        if (dma_req && dma_ack) dma_credits <= dma_credits + 1'b1;
        if (fetch_done && dma_credits != 0) dma_credits <= dma_credits - 1'b1;
    end
end

// expose debug state
always @(posedge clk) state_out <= state;

endmodule

```

Implications and trade-offs:

1. Use of credits reduces burst contention with DRAM controllers, trading hardware counters for robustness.
2. Encoded FSM minimizes logic area but may slightly increase critical-path latency compared to one-hot.
3. Scaling to multiple parallel decode engines requires arbitration for shared memory and output merges; equation (6.29) guides replication.
4. Supporting complex codecs (AV1) increases variability in `entropy_done` latency, favoring deeper buffering and backpressure-aware DMA scheduling.

6.6.3 Motion estimation module

The previous subsections established the decode FSM that supplies reconstructed reference frames and the timing generator that synchronizes pixel delivery; the motion estimator sits between these blocks and the entropy encoder, consuming frame-aligned pixels under the same timing constraints to produce motion vectors for motion compensation.

Problem: implement a synthesizable hardware motion-estimation engine for block-based video encoding suitable for H.264/H.265 pipelines. Requirements are low-latency per-macroblock (16×16) full-search SAD (sum of absolute differences) over a configurable search radius, streaming-compatible interfaces to the decoded-reference RAM, and bounded local storage for the current block.

Analysis: full-search SAD complexity grows with search radius W and block size B : the number of SAD operations per block is

$$[H]S = (2W + 1)^2 \cdot B^2. \quad (6.30)$$

For $B = 16$ and $W = 16$, S becomes large, so the design favors a single-pixel-per-cycle accumulator for area and a pipelined read from external reference RAM with one-cycle synchronous-read latency. The module supports:

- streaming input of one current-block pixel per cycle,
- external synchronous reference RAM read interface (address and read-enable outputs, pixel input),
- control handshake `start` and output valid with best motion vector and minimum SAD.

Implementation: the Verilog below is a complete, synthesizable single-search-engine suitable as a CU-local block in a multi-CU encoder. It stores the current 16×16 block in local BRAM, issues reference reads for each candidate, computes per-pixel absolute differences, accumulates SAD, and retains the best candidate vector.

Listing 6.23: Synthesizable 16x16 full-search SAD motion estimator (single engine).

```
module motion_estimator #(
  parameter MB_SIZE = 16,
  parameter PIX_W = 8,
  parameter ADDR_W = 32,
  parameter SEARCH_W = 16
```

```

) (
    input wire          clk,
    input wire          rst_n,
    // control
    input wire          start,          // start search for next
    MB
    input wire [ADDR_W-1:0] ref_base_addr, // top-left of search
    center (frame coords)
    // current block stream: one pixel per cycle, row-major, valid pulses
    input wire [PIX_W-1:0] cur_pixel_in,
    input wire          cur_pixel_valid,
    // reference RAM interface (synchronous read, 1-cycle latency)
    output reg [ADDR_W-1:0] ref_addr,
    output reg          ref_rd,
    input wire [PIX_W-1:0] ref_pixel_in,
    // results
    output reg          out_valid,
    output reg signed [15:0] mv_x, mv_y,
    output reg [31:0] sad_min
);

localparam MB_PIXELS = MB_SIZE*MB_SIZE;
localparam SEARCH_DIM = 2*SEARCH_W+1;

reg [PIX_W-1:0] cur_block [0:MB_PIXELS-1];
integer load_idx;
reg loading;

// search indices
integer dy, dx;
integer px_idx;
reg [31:0] sad_acc;
reg [31:0] candidate_sad;
reg [15:0] best_dx, best_dy;
reg signed [15:0] signed_dx, signed_dy;

typedef enum reg [1:0] {IDLE=0, LOAD=1, SEARCH=2, DONE=3} state_t;
reg [1:0] state, next_state;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        loading <= 0;
        out_valid <= 0;
        ref_rd <= 0;
        ref_addr <= 0;
        sad_min <= 32'hFFFFFFFF;
    end
end

```

```

mv_x <= 0; mv_y <= 0;
end else begin
state <= next_state;
case (state)
  IDLE: begin
    out_valid <= 0;
    sad_min <= 32'hFFFFFFFF;
    if (start) begin
      loading <= 1;
      load_idx <= 0;
    end
  end
  LOAD: begin
    if (cur_pixel_valid) begin
      cur_block[load_idx] <= cur_pixel_in; // store incoming pixels
      load_idx <= load_idx + 1;
      if (load_idx == MB_PIXELS-1) begin
        loading <= 0;
        // prepare search
        dy <= -SEARCH_W;
        dx <= -SEARCH_W;
        sad_min <= 32'hFFFFFFFF;
      end
    end
  end
  SEARCH: begin
    // issue read for current candidate pixel index
    if (px_idx < MB_PIXELS) begin
      // compute reference address: ref_base + candidate offset + pixel offset
      // here we assume linearized frame address; caller must map (x,y)-> addr
      ref_addr <= ref_base_addr + ((dy + SEARCH_W)*MB_SIZE + (dx + SEARCH_W)); // placeholder mapping
      ref_rd <= 1;
      // read returns next cycle
    end else begin
      ref_rd <= 0;
    end
    // capture returned pixel and accumulate
    if (ref_rd) begin
      // absolute difference
      if (ref_pixel_in > cur_block[px_idx])
        sad_acc <= sad_acc + (ref_pixel_in - cur_block[px_idx]);
      else
        sad_acc <= sad_acc + (cur_block[px_idx] - ref_pixel_in);
      px_idx <= px_idx + 1;
    end
  end

```

```

if (px_idx == MB_PIXELS-1) begin
    candidate_sad <= sad_acc;
    sad_acc <= 0;
    px_idx <= 0;
    // compare to best
    if (candidate_sad < sad_min) begin
        sad_min <= candidate_sad;
        best_dx <= dx;
        best_dy <= dy;
    end
    // advance candidate
    if (dx < SEARCH_W) dx <= dx + 1;
    else begin
        dx <= -SEARCH_W;
        if (dy < SEARCH_W) dy <= dy + 1;
        else begin
            // finished
            out_valid <= 1;
            mv_x <= best_dx;
            mv_y <= best_dy;
        end
    end
end
end
end
    DONE: begin
        out_valid <= 0;
    end
endcase
end
end

// simple next-state logic
always @(*) begin
    next_state = state;
    case (state)
        IDLE: if (start) next_state = LOAD;
        LOAD: if (!loading) next_state = SEARCH;
        SEARCH: if (out_valid) next_state = DONE;
        DONE: next_state = IDLE;
    endcase
end

endmodule

```

Implications and trade-offs:

- Area vs throughput: a single-engine full-search conserves logic but scales linearly in la-

tency with $(2W + 1)^2$ per-block (Eq. 6.30); multi-engine banks or early-exit hierarchical search reduce latency at higher area cost.

- Memory interface: synchronous read latency and reference-address mapping must align with the display timing generator and decode FSM to avoid stalls.
- Power: aggressive parallel SAD units increase power; selecting SEA (spatial early-adoption) heuristics trades off compression quality versus energy and latency.

6.6.4 Video pipeline testbench

The testbench described here continues the verification flow begun in the motion estimation and video decoder FSM subsections by exercising how decoded macroblocks and motion vectors traverse the video processing stages and by validating pipeline-level timing, backpressure, and buffer sizing under realistic workload patterns.

A rigorous testbench must solve three problems: produce a representative stream of decoded pixels and sideband data (motion vectors, frame boundaries); exercise the `video_pipeline` under backpressure and variable-latency conditions; and check functional correctness plus performance metrics (throughput, dropped frames). The approach below integrates a synthesizable stream generator and checker with a parametrizable DUT that models pipeline latency. We compute minimum FIFO depth from the steady-state throughput requirement: if the pipeline consumes R pixels per cycle on average and introduces a worst-case processing latency L cycles, the FIFO must satisfy

$$[H]D \geq R \cdot L \quad (6.31)$$

to avoid stalls. For bursty video workloads (compression-coded frames followed by motion compensation), add a safety factor S (typical $S = 1.5$ – 2) to (1) to account for jitter and clock-domain interactions.

Implementation outline and verification strategy:

- Stimulus: a frame generator emits color pixels and marks end-of-line (EOL) and end-of-frame (EOF) with sideband fields; motion vector tags are sent interleaved to model ME output.
- DUT: the example `video_pipeline` is parameterized by `LATENCY` and implements valid-ready handshaking to model backpressure from downstream TMUs or display timing units.
- Checker: counts pixels per-frame, asserts ordering constraints, verifies no valid beats lost when downstream asserts backpressure pulses with varied duty cycles.
- Metrics: measure achieved throughput (pixels/sec) and occupancy histogram for the FIFO to validate (1).

The following Verilog provides a compact, synthesizable test harness and a simple DUT used for verification. The generator uses counters to produce deterministic frames; the checker uses assertions and counters to raise a failure if pixels are dropped or misordered.

Listing 6.24: Verilog testbench: synthesizable stream generator, DUT (latency model), and checker.

```
`timescale 1ns/1ps
```

```

module stream_gen #(
    parameter H_PIX = 1280, V_LINES = 720
) (
    input wire clk, input wire rst_n,
    output reg [23:0] pixel, output reg valid, output reg last
);

    reg [31:0] x, y;
    always @(posedge clk) begin
        if (!rst_n) begin x <= 0; y <= 0; valid <= 0; last <= 0; pixel <= 0; end
        else begin
            valid <= 1;
            pixel <= {8'd0, x[7:0], y[7:0]}; // deterministic pattern
            last <= (x == H_PIX-1) && (y == V_LINES-1);
            if (x == H_PIX-1) begin x <= 0; if (y == V_LINES-1) y <= 0; else y <= y
                +1; end
            else x <= x+1;
        end
    end
endmodule

module video_pipeline #(
    parameter LATENCY = 4
) (
    input wire clk, input wire rst_n,
    input wire [23:0] in_pixel, input wire in_valid, output reg in_ready,
    output reg [23:0] out_pixel, output reg out_valid, input wire out_ready
);

    // simple shift-register latency pipeline with valid-ready
    reg [23:0] pipe_pix [0:LATENCY-1];
    reg pipe_v [0:LATENCY-1];
    integer i;
    always @(posedge clk) begin
        if (!rst_n) begin
            for (i=0;i0;i=i-1) begin
                if (~pipe_v[i] && pipe_v[i-1]) begin pipe_pix[i] <= pipe_pix[i-1];
                    pipe_v[i] <= 1; pipe_v[i-1] <= 0; end
            end
            // output stage
            if (pipe_v[LATENCY-1]) begin
                if (out_ready) begin out_pixel <= pipe_pix[LATENCY-1]; out_valid <= 1;
                    pipe_v[LATENCY-1] <= 0; end
                else out_valid <= 1; // backpressure holds valid high
            end else out_valid <= 0;
        end
    end
endmodule

```

```

module stream_check(
    input wire clk, input wire rst_n,
    input wire [23:0] pixel, input wire valid, input wire last
);
    reg [31:0] seen;
    always @(posedge clk) begin
        if (!rst_n) begin seen <= 0; end
        else if (valid) begin
            seen <= seen + 1;
            if (last) $display("Frame completed: %0d pixels seen", seen);
        end
    end
endmodule

module tb_top;
    reg clk = 0; always #5 clk = ~clk;
    reg rst_n = 0;
    wire [23:0] pix_a; wire v_a, l_a;
    wire [23:0] pix_b; wire v_b, r_b;
    stream_gen #(.H_PIX(640), .V_LINES(480)) gen(.clk(clk), .rst_n(rst_n), .
        pixel(pix_a), .valid(v_a), .last(l_a));
    video_pipeline #(.LATENCY(8)) dut(.clk(clk), .rst_n(rst_n), .in_pixel(pix_a)
        , .in_valid(v_a), .in_ready(r_b), .out_pixel(pix_b), .out_valid(v_b), .
        out_ready(1'b1));
    stream_check chk(.clk(clk), .rst_n(rst_n), .pixel(pix_b), .valid(v_b), .last
        (l_a));
    initial begin
        #20 rst_n = 1;
        #200000 $finish; // limit simulation time
    end
endmodule

```

Design implications and trade-offs:

- FIFO sizing must obey (1) with an added jitter safety factor; undersized buffers lead to dropped frames or degraded throughput under bursty decoder output.
- Modeling valid-ready handshakes in the testbench reveals backpressure propagation delays; longer LATENCY increases required FIFO depth and amplifies sensitivity to downstream stalls.
- For multi-clock domains (decode vs display), add CDC FIFOs and conservatively increase D ; for hardware emulation, prefer the synthesizable generator/checker to validate frame-level bandwidth and occupancy histograms.

Chapter 7

Interconnect and Communication

7.1 On-Chip Networks

7.1.1 Mesh and crossbar topologies

Building on earlier discussion of memory partitioning and L2-slice arbitration, we now compare two dominant on-chip interconnect primitives and quantify their impacts on throughput, latency, area, and power in modern GPUs. The analysis emphasizes how SIMT traffic patterns from SMs and tensor cores map to topology characteristics.

A mesh offers scalable physical layout and predictable wiring for a 2D array of compute complexes. For an N -node 2D mesh arranged as a grid of side $s = \sqrt{N}$, the worst-case hop count (network diameter) scales as

$$[H]\text{diameter}_{\text{mesh}} = 2(s - 1) = 2(\sqrt{N} - 1), \quad (7.1)$$

so average latency grows with $\mathcal{O}(\sqrt{N})$. Bisection bandwidth for a square mesh scales as

$$[H]B_{\text{bisect}} \propto s \cdot B_{\text{link}} = \sqrt{N} B_{\text{link}}, \quad (7.2)$$

where B_{link} is per-link bandwidth. Mesh advantages: wirelength locality, incremental tile replication, and simpler router design (minimal VC requirements for deadlock-free XY routing). Drawbacks: higher multi-hop latency for coherence-heavy or low-latency ML control traffic, and lower aggregate bisection per area than denser fabrics.

A full crossbar provides single-hop connectivity between any input and output, yielding minimal logical hop count and near-constant latency under light contention. However, a naive $N \times N$ crossbar has area and switch-complexity that grow as $\mathcal{O}(N^2)$, making it impractical beyond tens of ports for GPU-scale arrays. Practical implementations partition the crossbar into a hierarchical or buffered crossbar fabric:

- local crossbars per L2 slice that connect a small set of SMs,
- a two-stage crossbar (input-stage partial crossbar + rearrange stage) to reduce area,
- or concentrator-based designs that exploit traffic locality.

Latency model used for comparative sizing:

$$[H]L_{\text{net}} = L_{\text{router}} \cdot H + L_{\text{link}} \cdot H + L_{\text{ser/deser}}, \quad (7.3)$$

where H is hop count, L_{router} is per-router traversal delay, and $L_{\text{ser}/\text{deser}}$ captures packetization overhead. For GPU kernels with high arithmetic intensity, bandwidth dominates; for low-latency control (dispatch, atomics, small-message gathers), H and L_{router} matter.

Implementation example: a synthesizable parameterized crossbar connecting N inputs to M outputs with per-output priority arbitration and valid/ready handshake. This is a compact, production-ready building block suitable for small-scale L1/L2 crossbars or I/O fabrics.

Listing 7.1: Parametric N-to-M crossbar with per-output priority arbiter (synthesizable).

```

module crossbar #(
    parameter N = 4,                // number of inputs
    parameter M = 4,                // number of outputs
    parameter WIDTH = 64           // data width
) (
    input  wire      clk,
    input  wire      rstn,
    input  wire [N-1:0] in_valid,
    input  wire [N*WIDTH-1:0] in_data,
    output reg [N-1:0] in_ready,
    output reg [M-1:0] out_valid,
    output reg [M*WIDTH-1:0] out_data,
    input  wire [M-1:0] out_ready
);

    integer i, j;
    // internal one-hot selects per output
    reg [N-1:0] sel [0:M-1];

    // default assignments
    always @(*) begin
        for (i = 0; i < M; i = i + 1) begin
            sel[i] = {N{1'b0}};
            out_valid[i] = 1'b0;
            out_data[i*WIDTH +: WIDTH] = {WIDTH{1'b0}};
            for (j = 0; j < N; j = j + 1) begin
                if (in_valid[j] && !out_valid[i]) begin
                    sel[i][j] = 1'b1; // grant this input
                    out_valid[i] = 1'b1;
                    out_data[i*WIDTH +: WIDTH] = in_data[j*WIDTH +: WIDTH];
                end
            end
        end
    end

    // ready propagation
    always @(*) begin
        for (j = 0; j < N; j = j + 1) begin
            in_ready[j] = 1'b0;
            for (i = 0; i < M; i = i + 1) begin

```

```

        if (sel[i][j] && out_ready[i])
            in_ready[j] = 1'b1;
        end
    end
end
endmodule

```

7.1.2 Router architecture and buffering

Continuing from the topology discussion, router microarchitecture and buffering choices realize the logical links of a mesh or the centralized paths of a crossbar; topology shapes where congestion accumulates and thus where buffering must be provisioned. The following describes the router pipeline, buffering strategies appropriate for GPU workloads, and a compact Verilog implementation of a credit-aware input FIFO used in input-buffered routers.

A practical router datapath separates concerns into stages:

1. Routing computation (RC): determine next hop from packet header.
2. Virtual-channel allocation (VCA): assign a free virtual channel to avoid head-of-line (HOL) blocking.
3. Switch allocation (SA): arbitrate contenders for crossbar ports.
4. Switch traversal (ST): pass flit through crossbar fabric.
5. Link traversal (LT): transmit flit across physical link and update credits.

For GPU workloads—texture fetch bursts from TMUs, multicast-like shader message patterns, and sustained memory responses—buffer occupancy spikes are both bursty and correlated across SMs. Two dominant buffering strategies are used: input-buffered routers with virtual channels and credit-based flow control, and more area-costly buffered-crossbar or output-buffered designs used in small-scale crossbars.

Buffer sizing must cover round-trip flow-control latency and peak burstiness. Operationally, to avoid link stalls the minimum buffer depth per VC (in flits) should satisfy the link bandwidth times the round-trip credit latency:

$$[H]$$

$$D_{\min} \approx B_{\text{link}} \cdot RTT_{\text{cycles}}, \quad (7.4)$$

where B_{link} is flits per cycle. For example, a 1-flit-per-cycle link with a 20-cycle RTT needs at least 20 flits per VC to sustain full throughput without pipeline bubbles. Add a burst margin factor α (1.2–2.0) to tolerate instantaneous multi-SM bursts and on-chip memory controller jitter.

Virtual channels mitigate HOL blocking and improve throughput; however, they increase meta-data (per-VC pointers, occupancy bits). Credit-based flow control simplifies deadlock avoidance when combined with escape VCs and deterministic routing in meshes. Wormhole routing keeps router latency low by streaming flits, but increases sensitivity to VC availability and thus buffer sizing.

Practical implementation: a synthesizable input FIFO that returns credits when a flit departs. This module can be instantiated per VC in an input-buffered router.

Listing 7.2: Credit-aware input FIFO for a router VC.

```

module vc_fifo #(
    parameter FLIT_WIDTH = 128,
    parameter DEPTH = 32,          // must be power of two
    parameter PTR_W = $clog2(DEPTH)
) (
    input wire          clk,
    input wire          rst_n,
    input wire          push,      // write enable
    input wire [FLIT_WIDTH-1:0] din,    // flit input
    input wire          pop,      // read enable (on egress)
    output wire [FLIT_WIDTH-1:0] dout,    // flit output
    output wire         full,
    output wire         empty,
    output wire [PTR_W:0] credit_out // credits freed on pop
);

// BRAM-based circular buffer with pointers and occupancy counter
reg [FLIT_WIDTH-1:0] mem [0:DEPTH-1];
reg [PTR_W-1:0] wptr, rptr;
reg [PTR_W:0] occ;          // occupancy counter
assign full = (occ == DEPTH);
assign empty = (occ == 0);
assign dout = mem[rptr];
assign credit_out = (pop && !empty) ? 1 : 0; // return one credit per flit
read

// synchronous write
always @(posedge clk) begin
    if (!rst_n) begin
        wptr <= 0; rpтр <= 0; occ <= 0;
    end else begin
        if (push && !full) begin
            mem[wptr] <= din;
            wpтр <= wpтр + 1;
            occ <= occ + 1;
        end
        if (pop && !empty) begin
            rpтр <= rpтр + 1;
            occ <= occ - 1;
        end
    end
end
endmodule

```

Design implications and trade-offs:

- Buffer depth increases area and static power but reduces stalls and improves SM throughput under TMU bursts.

- More VCs improve utilization and reduce HOL blocking, at the cost of additional pointers and control logic.
- Wormhole routing with credit flow control minimizes router latency but requires careful credit sizing per (7.24).
- Centralized crossbars reduce per-router buffering but scale poorly; mesh input-buffered routers scale better for many-SM GPUs.

7.1.3 Flow control and arbitration

The previous subsections showed how router buffering and topology choices determine traffic patterns and contention points across the on-chip network. Flow control and arbitration implement the run-time mechanisms that convert those topologies and buffers into predictable latency, fairness, and throughput for SMs, memory controllers, TMUs, and ROPs.

Flow-control objectives are to prevent buffer overflow, bound latency under load, and avoid head-of-line blocking or deadlock. Two industry-proven schemes appear in modern GPUs: credit-based flow control, which tracks downstream buffer availability with explicit credits, and backpressure/on-off signals that stall upstream injectors. Credit-based schemes enable fine-grained injection control for high-bandwidth links, while on-off (stall) signals are cheaper but can amplify congestion across many hops.

Virtual channels (VCs) decouple physical link occupancy from logical flows and are required to prevent cyclic buffer dependency and livelock under wormhole routing. A minimal deadlock-avoidance policy is to assign at least two VCs per physical port and enforce escape VC routing restrictions. Arbitration inside routers and crossbars multiplexes competing inputs into limited output ports; choice of policy directly impacts SIMT-style workloads where bursts from a few warps can monopolize link bandwidth.

Quantitatively, microarchitects use queueing approximations to size buffers and set service rates. For a link with offered arrival rate λ and service rate μ , utilization is $\rho = \lambda/\mu$. Under M/M/1 assumptions, average queue length L and average waiting time W satisfy Little's law:

$$[H]L = \lambda W, \quad (7.5)$$

and in the M/M/1 model one can approximate $L \approx \frac{\rho}{1-\rho}$ for analytical sizing; treat this as conservative guidance, not definitive for bursty GPU traffic.

Arbitration policies common in GPU NoCs:

- Round-robin: fair, low hardware cost, prevents long-term starvation.
- Priority/age-based: reduces tail latency for older packets; useful for QoS to memory and prefetch.
- Grant-stationary matrix: used in multi-ported crossbars to combine per-output arbitration with per-input allocation.

Implementation example: a synthesizable parameterized round-robin arbiter that produces one-hot grants, suitable for router input schedulers or per-SM memory-port arbitration.

Listing 7.3: Parameterizable round-robin arbiter (synthesizable).

```
module rr_arbiter #(parameter N = 8) (
```

```

input wire      clk,
input wire      rst,
input wire [N-1:0] req,    // one bit per requester
output reg  [N-1:0] grant  // one-hot grant
);
reg [$clog2(N)-1:0] ptr;    // rotating priority pointer
integer i;
wire [N-1:0] masked;
// rotate requests so ptr is MSB-aligned
assign masked = (req << ptr) | (req >> (N - ptr));
always @(*) begin
    grant = {N{1'b0}};
    for (i = 0; i < N; i = i + 1)
        if (masked[i]) begin
            grant = ( ({N{1'b1}} >> (N-1-i)) & ~({N{1'b1}} >> (N-i)) ) >> ptr;
            disable for; // break
        end
    end
always @(posedge clk) begin
    if (rst) ptr <= 0;
    else if (!req) begin
        // advance pointer to next position after current grant
        ptr <= (ptr + (grant ? $clog2(N) : 1)) % N;
    end
end
endmodule

```

Integrate this arbiter with credit counters per output VC and grant only when credits exist. Performance trade-offs:

- More VCs and deeper buffers reduce head-of-line blocking but increase area and power.
- Strict priority improves latency for critical flows but risks starvation of background traffic.
- Round-robin yields fairness for SM warp traffic but increases worst-case tail latency.

Designers must balance buffer sizing, VC count, and arbitration policy to meet targeted throughput, latency, and QoS constraints.

7.1.4 Deadlock prevention

The previous discussion of credit-based flow control and per-port arbitration established how buffering and bandwidth allocation affect head-of-line blocking and throughput. Building on those mechanisms, deadlock prevention requires both routing discipline and resource-management primitives that guarantee absence of cyclic dependencies even under heavy SM-to-memory and peer GPU traffic.

Problem and analysis:

- Deadlock is a cyclic resource dependency where flits occupy buffers waiting for downstream resources held by other flits, producing livelock of SM, TMU, ROP, or RT core traffic. In wormhole routing the dependency graph includes: input buffers, virtual channels (VCs), and link credits.
- Two standard theoretical approaches remove cycles:
 1. Resource ordering (dimension-order routing or the turn model) which prevents cycles in the channel dependency graph.
 2. Duato-style escape channels: reserve at least one escape VC per port where routing is restricted to a simple deadlock-free algorithm; adaptive routing may use other VCs.

Practical constraints for GPUs:

- High-throughput streaming (texture fetches) emphasizes large sustained flit flows; control and cache-coherence traffic produces many short bursts. Mixed workloads increase the chance of transient buffer saturation.
- Buffer sizing must allow pipeline fill without starving links. A conservative flit-credit lower bound is

$$[H]C_{\min} = \left\lceil \frac{RTT \times BW}{\text{flit_bytes}} \right\rceil + 1, \quad (7.6)$$

where RTT is round-trip latency in cycles, BW is link bytes per cycle, and flit_bytes is the payload bytes per flit. This ensures at least one outstanding flit per round-trip plus headroom.

Implementation sketch (synthesizable Verilog):

Listing 7.4: VC credit manager with escape VC allocation

```

module noc_vc_manager #(
  parameter NUM_VC = 8,
  parameter C_WIDTH = 8
) (
  input  wire          clk,
  input  wire          rst,
  input  wire [NUM_VC-1:0] alloc_req,      // request per VC class (one-hot
      semantics)
  output reg  [NUM_VC-1:0] alloc_grant,    // grant per VC (one-hot)
  input  wire [$clog2(NUM_VC)-1:0] free_idx, // index returned when flit
      leaves downstream
  input  wire          free_valid,        // free strobe increments credit
  input  wire [C_WIDTH-1:0] init_credits // initial credit value load
);
  // per-VC credit counters
  reg [C_WIDTH-1:0] credits [0:NUM_VC-1];
  integer i;
  // initialize credits
  always @(posedge clk) begin

```

```

if (rst) begin
  for (i=0;i0 wins; escape VC = 0
  alloc_grant <= {NUM_VC{1'b0}};
  for (i=0;i0 && alloc_grant=={NUM_VC{1'b0}}) begin
    alloc_grant[i] <= 1'b1;
    credits[i] <= credits[i] - 1'b1; // consume credit on allocation
  end
end
// if no regular grant, try escape VC0 if requested and has credit
if (alloc_grant=={NUM_VC{1'b0}} && alloc_req[0] && credits[0]>0) begin
  alloc_grant[0] <= 1'b1;
  credits[0] <= credits[0] - 1'b1;
end
end
endmodule

```

Operational recommendations and trade-offs:

- Use one escape VC per port with strict dimension-order routing on that VC; permit adaptive routing on other VCs to improve utilization. This yields Duato-compliant deadlock freedom while preserving adaptivity.
- Trade-offs:
 1. Area/power: extra VCs and deeper buffers increase SRAM area and leakage; typical GPU designs budget several kilobytes per router port.
 2. Latency vs adaptability: escape VC forces some flits to follow suboptimal paths under contention, slightly increasing latency but guaranteeing forward progress.
 3. Performance knobs: tune $C_{\min}$ (Eq. 7.6) and number of VCs according to worst-case RTT from SM/RT cores to memory controllers; more credits reduce head-of-line stalls but cost area.
- For GPUs, prioritize escape-channel correctness for coherence and DMA control traffic, and allocate adaptive VCs to high-bandwidth streams (texture, tensor-core data) to maximize sustained throughput.

7.2 Memory Crossbar

7.2.1 SM-to-memory partition links

Building on prior discussion of on-chip router buffering and flow-control choices, we now focus on the point-to-point and aggregated links that connect each streaming multiprocessor (SM) to a memory partition slice in the crossbar. These SM-to-memory partition links are the critical boundary where SIMT-generated request bursts meet partitioned DRAM bandwidth and L2 slice arbitration, so their design directly affects sustained throughput, worst-case latency, and fairness across graphics, compute, and tensor workloads.

Problem: SMs produce high-rate, often bursty, loads (texture fetches from TMUs, atomics from compute kernels, and writeback streams from tensor cores). The crossbar must absorb bursts without head-of-line blocking, while avoiding excessive on-chip buffering that increases area and power.

Analysis:

- Each link is a packetized pipeline carrying request and response streams; many designs separate request (address+ctrl) and response (data) channels. For steady utilization, buffer capacity must cover the product of link bandwidth and round-trip latency (RTT) observed between SM and memory partition.
- For packet size S (bytes), link bandwidth B (bytes/s), and RTT T (s), minimum number of in-flight packets D to keep the link busy satisfies

$$[H]D \geq \frac{B \cdot T}{S}. \quad (7.7)$$

This lower bound guides sizing of transmit queues and per-SM response buffers inside a memory slice.

Implementation (practical link endpoint): a lightweight packet interface with credit-based backpressure and a small synchronous FIFO on the SM side prevents stalls in the SM pipeline while credits route from the partition to the SM. The module below is a synthesizable, parameterizable endpoint that accepts fixed-width packets from an SM and performs valid/ready handshakes with credit return. It omits protocol-specific address parsing so it can be integrated into different crossbar protocols.

Listing 7.5: SM-to-partition link endpoint with credit-based flow control

```

module sm_mem_link #(
    parameter DATA_W = 128, // payload width
    parameter DEPTH = 8      // fifo depth (number of packets)
) (
    input wire clk,
    input wire rst,
    // SM side (producer)
    input wire sm_valid,
    input wire [DATA_W-1:0] sm_data,
    output reg sm_ready,
    // Partition side (consumer)
    output reg part_valid,
    output reg [DATA_W-1:0] part_data,
    input wire part_ready,
    // Credit return from partition (for responses) - simple decrement
    input wire [$clog2(DEPTH):0] credits_in,
    output reg [$clog2(DEPTH):0] credits_out
);

// simple synchronous FIFO
reg [DATA_W-1:0] fifo_mem [0:DEPTH-1];

```

```

reg [$clog2(DEPTH):0] wr_ptr, rd_ptr, count;

// write side
always @(posedge clk) begin
  if (rst) begin
    wr_ptr <= 0; count <= 0;
    sm_ready <= 1'b1;
  end else begin
    if (sm_valid && sm_ready) begin
      fifo_mem[wr_ptr] <= sm_data; // write packet
      wr_ptr <= wr_ptr + 1;
      count <= count + 1;
    end
    // throttle SM when FIFO is full
    sm_ready <= (count < DEPTH-1);
  end
end

// read side to partition
always @(posedge clk) begin
  if (rst) begin
    rd_ptr <= 0; part_valid <= 1'b0; part_data <= {DATA_W{1'b0}};
    credits_out <= 0;
  end else begin
    // supply credits seen from partition upstream (simple pass-through)
    credits_out <= credits_in;
    if (!part_valid && (count > 0)) begin
      part_data <= fifo_mem[rd_ptr];
      part_valid <= 1'b1;
    end
    if (part_valid && part_ready) begin
      rd_ptr <= rd_ptr + 1;
      part_valid <= 1'b0;
      count <= count - 1;
    end
  end
end

endmodule

```

Implications and trade-offs:

- Buffer sizing per (7.24) must account for worst-case RTT including L2 arbitration, DRAM queueing, and serialization at the partition; under-sizing increases SM stalls, over-sizing increases area and power.
- Use of credit-based flow control reduces head-of-line blocking for responses but requires return-credit bandwidth; balance the credit granularity (packets versus beats) to trade control overhead against buffer savings.

- QoS: per-SM virtual channels or weighted arbiters in the partition reduce cross-SM interference at cost of additional buffers and arbitration logic.
- For ML workloads with large tensor-core writebacks, provision extra response buffering or prioritize write-combine paths to avoid degrading interactive graphics latency.

7.2.2 Bandwidth scheduling

The previous subsection described the physical SM-to-memory partition links and their topological constraints; we now address how to allocate the crossbar's aggregate DRAM bandwidth among competing SMs while accounting for QoS, latency, and throughput targets.

Bandwidth scheduling problem: multiple SMs (producers of cacheline-sized memory transactions) share P memory partitions through a crossbar. Goals are to (1) maximize utilization for bandwidth-bound kernels, (2) bound worst-case latency for latency-sensitive tasks, and (3) enforce tenant or priority-based fairness. Analysis begins by modeling each SM i with a weight w_i and request arrival process; the long-run bandwidth share B_i should satisfy proportional allocation

$$[H]B_i = \frac{w_i}{\sum_j w_j} B_{\text{total}}, \quad (7.8)$$

where B_{total} is the aggregate link bandwidth measured in cachelines per second. Enforcing (1)–(3) requires packet-level scheduling on the crossbar egress ports and optionally per-virtual-channel policing.

Design analysis examines common schedulers:

- Round-robin (RR): simple, low overhead, poor for weighted fairness.
- Weighted round-robin (WRR): approximates Eq. (1) using integer quanta; low state per SM.
- Deficit round-robin (DRR): handles variable-size transfers with deficit counters, useful for multi-beat transactions.
- Priority queues with token-bucket policing: satisfy latency bounds at cost of potential starvation.

Implementation choice: a DRR-style arbiter per memory-partition egress balances simplicity and correctness for cacheline-granular transactions. DRR requires per-SM deficit counters and a rotation pointer. The arbiter adds a quantum proportional to the configured weight each scheduling round, then grants as many cachelines as the deficit permits from the requesting SM.

The following synthesizable SystemVerilog module implements a parameterized DRR arbiter for N requesters. It assumes fixed-size grants of one cacheline beat per grant; quantum is interpreted as grant budget per round. Adaptation to multi-beat packets requires accounting for packet size in the subtraction.

Listing 7.6: DRR arbiter for crossbar egress (synthesizable SystemVerilog)

```

module drr_arbiter #(
    parameter int N = 8,                // number of SM requesters
    parameter int QW = 8                // quantum width
) (

```

```

input  logic          clk,
input  logic          rst_n,
input  logic [N-1:0]  req,      // one-hot active requests
input  logic [QW-1:0] quantum [N], // per-client quantum (weight)
output logic [N-1:0]  grant     // one-hot grant
);

// per-client deficit counters
logic [QW+8-1:0] deficit [N]; // extra bits for accumulation
int ptr; // rotation pointer

// reset
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ptr <= 0;
        for (int i = 0; i < N; i++) begin
            deficit[i] <= '0;
        end
        grant <= '0;
    end else begin
        grant <= '0;
        // deficit update and round-robin traversal
        for (int k = 0; k < N; k++) begin
            int idx = (ptr + k) % N;
            deficit[idx] <= deficit[idx] + quantum[idx]; // accumulate weight
            if (req[idx] && (deficit[idx] >= quantum[idx]) && (grant == '0)) begin
                grant[idx] <= 1'b1;
                deficit[idx] <= deficit[idx] - quantum[idx];
                ptr <= idx + 1;
            end
        end
    end
end
endmodule

```

7.2.3 Virtual channels

The previous discussion characterized SM-to-memory partition links and bandwidth scheduling policies that shape per-partition demand; virtual channels (VCs) are the complementary mechanism inside the crossbar that multiplex logical flows over physical links while enabling QoS, deadlock avoidance, and fine-grained arbitration across SMs, TMUs, ROPs and memory controllers.

Virtual channels (VCs) are lightweight logical lanes multiplexed over a single physical link. They separate competing request classes (e.g., high-priority read responses, writebacks, texture fetches) into independent FIFO buffers and arbitration domains. Problem: without VCs, head-of-line (HOL) blocking by a long transaction stalls unrelated flows, reducing SM throughput and increasing warp stall time. Analysis shows that VCs mitigate HOL blocking and enable credit-based flow control and virtual-channel allocation to avoid cyclic resource dependencies

that cause deadlock.

Key operational elements:

- Per-input buffering: each ingress port maintains N_{vc} separate FIFOs; a packet is tagged with `vc_id` at allocation time.
- Allocation and arbitration: VC allocation occurs at packet enqueue and crossbar arbitration chooses a granted pair (input VC, output port) based on QoS weights.
- Flow control: credit counters per VC prevent buffer overflow downstream; credits equal free entries.
- Deadlock avoidance: assign VCs so that at least one escape VC follows a strict acyclic ordering, or use virtual-channel ordering enforced by the allocator.

Sizing and latency: buffer depth must absorb link round-trip-time (RTT) and burstiness. For a link of raw bandwidth B (bytes/s), flit size F (bytes), and round-trip latency T_{rt} (s), per-VC minimum free-slots D to avoid backpressure-induced stalls satisfies:

$$[H]D \geq \left\lceil \frac{B \cdot T_{rt}}{F \cdot N_{vc}^{active}} \right\rceil \quad (7.9)$$

where N_{vc}^{active} is the number of concurrently active VCs sharing the link. Equation (1) shows depth grows with bandwidth and RTT; smaller flits reduce required depth but increase control overhead.

Implementation sketch: a synthesizable Verilog module below provides parameterized per-VC FIFOs with credit return and full/empty flags. It is intentionally minimal but synthesizable and suitable for integration into a crossbar input stage.

Listing 7.7: VC FIFO array with credit interface

```

module vc_fifo_array #(
  parameter N_VC = 4,
  parameter DEPTH = 8,
  parameter WIDTH = 64,
  parameter PTR_W = $clog2(DEPTH)
) (
  input  wire          clk,
  input  wire          rst,
  input  wire [N_VC-1:0] push,          // push per VC (one-hot allowed)
  input  wire [N_VC-1:0] pop,          // pop per VC
  input  wire [N_VC-1:0] push_valid,    // valid qualifier per VC
  input  wire [WIDTH-1:0] din [0:N_VC-1], // data per VC (bundled)
  output reg  [N_VC-1:0] full,
  output reg  [N_VC-1:0] empty,
  output wire [WIDTH-1:0] dout [0:N_VC-1], // output per VC
  output reg  [PTR_W:0] credits [0:N_VC-1] // free slots per VC
);

  // Per-VC RAMs and pointers
  reg [WIDTH-1:0] mem [0:N_VC-1][0:DEPTH-1];
  reg [PTR_W-1:0] wptr [0:N_VC-1];

```

```

reg [PTR_W-1:0] rptr [0:N_VC-1];
integer i;

// Initialization
always @(posedge clk) begin
  if (rst) begin
    for (i = 0; i < N_VC; i = i + 1) begin
      wptr[i] <= 0;
      rptr[i] <= 0;
      full[i] <= 0;
      empty[i] <= 1;
      credits[i] <= DEPTH;
    end
  end else begin
    for (i = 0; i < N_VC; i = i + 1) begin
      // Push
      if (push[i] && push_valid[i] && !full[i]) begin
        mem[i][wptr[i]] <= din[i];
        wptr[i] <= (wptr[i] + 1) % DEPTH;
        empty[i] <= 0;
        if ((wptr[i] + 1) % DEPTH == rptr[i]) full[i] <= 1;
        if (credits[i] > 0) credits[i] <= credits[i] - 1;
      end
      // Pop
      if (pop[i] && !empty[i]) begin
        rptr[i] <= (rptr[i] + 1) % DEPTH;
        full[i] <= 0;
        if ((rptr[i] + 1) % DEPTH == wptr[i]) empty[i] <= 1;
        if (credits[i] < DEPTH) credits[i] <= credits[i] + 1;
      end
    end
  end

// Output data per VC
genvar j;
generate
  for (j = 0; j < N_VC; j = j + 1) begin : OUT_ASSIGN
    assign dout[j] = mem[j][rptr[j]];
  end
endgenerate
endmodule

```

7.2.4 QoS enforcement

The previous discussion established how virtual channels isolate traffic classes and how bandwidth schedulers map those classes onto memory partitions; QoS enforcement is the mechanism

that turns those policies into concrete, time- and buffer-bounded behaviour at the crossbar. Here we analyze enforcement objectives, derive simple bounding math, present an implementable scheduler primitive, and state practical trade-offs.

Problem statement: multiple SMs, TMUs, and RT/tensor engines generate heterogeneous flows that share limited crossbar and DRAM bandwidth. Goals are to (1) guarantee minimum throughput for latency-sensitive flows, (2) prevent starvation of low-priority classes, and (3) bound worst-case queuing delay for real-time consumers such as display and RT cores.

Analysis proceeds from two building blocks commonly implemented in silicon:

- rate reservation (token-bucket or credit-based), which limits average service to a configured rate; and
- deficit/weight-based arbitration (WRR/DRR), which enforces proportional shares while avoiding head-of-line blocking.

If class i is assigned weight w_i and the crossbar can sustain aggregate service B_{total} , the long-term bandwidth guarantee is

$$[H]\text{share}_i = \frac{w_i}{\sum_j w_j} B_{\text{total}}. \quad (7.10)$$

For token-bucket enforcement, let $\text{tokens}_i(t)$ be tokens at time t , refill rate r_i , and bucket depth B . The discrete update per clock T_{clk} is

$$[H]\text{tokens}_i[t+1] = \min(B, \text{tokens}_i[t] + r_i T_{\text{clk}} - s_i[t]), \quad (7.11)$$

where $s_i[t]$ is service consumed. Equation (2) yields a maximum burst allowance equal to B , which directly bounds worst-case queuing if the recipient can drain at link rate.

Implementation: a hybrid design uses per-class token buckets at the aggregator inputs, plus a central WRR arbiter that consults token availability and virtual-channel occupancy. The following synthesizable SystemVerilog module implements a small, hardware-friendly WRR QoS arbiter. It maintains deficit counters and advances a round-robin pointer to avoid starvation. We assume unit-service quanta per cycle; weights scale quanta added each round.

Listing 7.8: Synthesizable WRR QoS arbiter (SystemVerilog) for N classes.

```

module wrr_qos #(
  parameter int CLASSES = 4,
  parameter int W = 8
) (
  input  logic          clk,
  input  logic          rst,
  input  logic [CLASSES-1:0] req,           // request per class
  input  logic [W-1:0]   weight [CLASSES], // configured weights
  output logic [CLASSES-1:0] grant         // one-hot grant
);
  logic [W+8-1:0] deficit [CLASSES]; // accumulate quanta
  int unsigned ptr;

  always_ff @(posedge clk) begin
    if (rst) begin

```

```

ptr <= 0;
grant <= '0;
for (int i = 0; i < CLASSES; i++) begin
    deficit[i] <= '0;
end
end else begin
    grant <= '0;
    for (int k = 0; k < CLASSES; k++) begin
        int idx = (ptr + k) % CLASSES;
        deficit[idx] <= deficit[idx] + weight[idx]; // add quantum each round
        if (req[idx] && (deficit[idx] >= weight[idx]) && (grant == '0)) begin
            grant[idx] <= 1'b1;
            deficit[idx] <= deficit[idx] - weight[idx];
            ptr <= idx + 1;
        end
    end
end
endmodule

```

7.2.5 Error handling

The previous subsections showed how DMA engines create high-throughput, ordered PCIe TLP streams and how peer-to-peer mappings can bypass OS-side error containment. Building on those ideas, this subsection treats detection, classification, and hardware response for PCIe errors that affect GPU DMA and P2P transfers.

Problem statement and analysis: undetected or late-detected errors on PCIe links cause data corruption, silent poisoning of GPU memory, or long stalls when DMA descriptors time out. Errors fall into three operational classes:

- Correctable link/endpoint errors (recoverable via resynchronization).
- Non-fatal transaction errors (ECRC or completion timeouts; require retry or completion handling).
- Fatal errors (link down, device malfunction; require isolation).

Quantitative criterion for retry budgeting comes from the transfer error probability. For a transfer of N bits and bit error rate p , the probability of a clean transfer is approximately

$$[H]P_{\text{ok}} \approx (1 - p)^N \approx e^{-pN}. \quad (7.12)$$

For large N , expected retries per transfer $\approx (1 - P_{\text{ok}})/P_{\text{ok}}$. Use this to dimension replay buffers and to set retry budgets that avoid thrashing.

Implementation analysis: hardware must combine three mechanisms:

1. Lightweight per-TLP/DMA descriptor tracking so the DMA engine can detect missing completions and reissue. Track descriptor ID, length, timestamp, and a poison bit set by completion with ECRC or user-level error.

2. A timeout unit calibrated to worst-case PCIe round-trip latency plus SM scheduling jitter. If outstanding bytes W and link bandwidth B , a minimal timeout $T_{\min}$ satisfies $T_{\min} \geq W/B + t_{\text{link}} + t_{\text{host_proc}}$.
3. AEC/AER reporting fabric that escalates to firmware for non-fatal or fatal errors, and injects poison markers into GPU page tables to avoid silent corruption for P2P accesses.

Example synthesizable Verilog for a compact error handler that tracks outstanding DMA descriptors, times out, counts errors, and asserts an AER request is below. Adjust parameters to system scale.

Listing 7.9: Compact PCIe error tracker and AER generator

```

module pcie_err_handler #(
    parameter DEPTH = 16,
    parameter IDW = 8,
    parameter LENW = 16,
    parameter TMW = 24
) (
    input wire          clk,
    input wire          rst,
    // incoming DMA issue
    input wire          issue_valid,
    input wire [IDW-1:0] issue_id,
    input wire [LENW-1:0] issue_len,
    // completion indication
    input wire          comp_valid,
    input wire [IDW-1:0] comp_id,
    input wire          comp_err, // 1 = error (ECRC/poison)
    // configuration and outputs
    input wire [TMW-1:0] tmo_thresh,
    output reg          aer_request,
    output reg [31:0]    fatal_count
);

    // simple tracker arrays
    reg          valid [0:DEPTH-1];
    reg [IDW-1:0] id   [0:DEPTH-1];
    reg [LENW-1:0] len  [0:DEPTH-1];
    reg [TMW-1:0] age   [0:DEPTH-1];

    integer i;
    // allocation pointer (wrap)
    reg [$clog2(DEPTH)-1:0] alloc_ptr;

    always @(posedge clk) begin
        if (rst) begin
            alloc_ptr <= 0;
            aer_request <= 0;

```

```

fatal_count <= 0;
for (i = 0; i < DEPTH; i = i + 1) begin
    valid[i] <= 0;
    id[i]      <= 0;
    len[i]     <= 0;
    age[i]     <= 0;
end
end else begin
    aer_request <= 0;

    // age and timeout detection
    for (i = 0; i < DEPTH; i = i + 1) begin
        if (valid[i]) begin
            age[i] <= age[i] + 1'b1;
            if (age[i] >= tmo_thresh) begin
                // escalate as fatal if beyond threshold
                fatal_count <= fatal_count + 1;
                aer_request <= 1'b1;
                valid[i] <= 1'b0; // retire entry
            end
        end
    end
end

    // allocate new descriptor
    if (issue_valid) begin
        // find free slot (next-fit)
        if (!valid[alloc_ptr]) begin
            valid[alloc_ptr] <= 1'b1;
            id[alloc_ptr]     <= issue_id;
            len[alloc_ptr]    <= issue_len;
            age[alloc_ptr]    <= {TMW{1'b0}};
            alloc_ptr         <= alloc_ptr + 1'b1;
        end else begin
            // allocation collision: treat as fatal
            fatal_count <= fatal_count + 1;
            aer_request <= 1'b1;
        end
    end

    // process completion
    if (comp_valid) begin
        for (i = 0; i < DEPTH; i = i + 1) begin
            if (valid[i] && id[i] == comp_id) begin
                valid[i] <= 1'b0;
                if (comp_err) begin
                    fatal_count <= fatal_count + 1;
                    aer_request <= 1'b1;
                end
            end
        end
    end

```

```

        end
    end
end
end
end
end
endmodule

```

7.3 High-Speed Serial Links

7.3.1 NVLink and Infinity Fabric

The previous discussion of on-chip routers and PHY equalization set the context for off-chip high-speed fabrics by stressing topology, flow-control, and signal-integrity trade-offs that carry over to inter-GPU links. This subsection analyzes architectural choices and performance trade-offs between two dominant implementations used in modern GPU systems: NVIDIA’s NVLink and AMD’s Infinity Fabric.

Problem: provide low-latency, high-bandwidth, coherent or loosely-coherent connectivity across dies and boards to enable scalable multi-GPU and CPU–GPU systems. Analysis focuses on protocol layering, aggregation, flow control, and application-level effect on graphics and ML workloads.

Main exposition.

- **Protocol and topology:** both fabrics are packetized, lane-aggregated serial links with link training, per-lane equalization, and CRC-based integrity. They expose:
 1. Multiple virtual channels to avoid head-of-line blocking for traffic classes (memory, coherency, message).
 2. Credit-based flow control to bound buffering and prevent overruns across SM/CU endpoints.
 3. Topologies ranging from point-to-point meshes to multi-rail fat-tree fabrics; topology choice impacts worst-case bisection bandwidth and collective operation latency.
- **Coherency and memory model:** newer NVLink generations support GPU memory coherency across devices within a unified address space, enabling direct load/store semantics and reducing software synchronization cost. Infinity Fabric typically integrates coherency between CPU caches and chiplet dies in AMD designs, enabling tighter CPU–GPU sharing in heterogeneous systems.
- **Performance modeling:** design and analysis commonly use an aggregated-bandwidth model. For N lanes at per-lane line rate R (bytes/s) and protocol efficiency η ($0 < \eta \leq 1$), effective bandwidth is:

$$[H]B_{\text{eff}} = N \cdot R \cdot \eta. \quad (7.13)$$

Latency for an injected packet of payload P (bytes) includes serialization, switch traversal, and queuing; serialization dominant term is $T_{\text{ser}} = P/(N \cdot R)$ assuming striping across lanes.

Implementation: simple calculator and microbenchmark estimator in Python to explore trade-offs (serialize overhead, link efficiency, and round-trip cost).

Listing 7.10: Link bandwidth and latency estimator

```
def eff_bandwidth(lanes, lane_rate_bytes, efficiency): # compute B_eff
    return lanes * lane_rate_bytes * efficiency

def round_trip_latency(payload_bytes, lanes, lane_rate_bytes, switch_hops,
                       per_hop_ns=20, fixed_ns=200, efficiency=0.9):
    # serialization + switch latency + fixed overhead
    ser_ns = payload_bytes / (lanes * lane_rate_bytes) * 1e9
    return ser_ns + switch_hops * per_hop_ns + fixed_ns

# example usage
lanes=8; lane_rate=3e9 # 3 GB/s per lane
B=eff_bandwidth(lanes, lane_rate, 0.92) # bytes/sec
rtt=round_trip_latency(64, lanes, lane_rate, 2) # ns
print(B, rtt) # bytes/sec, ns
```

Implications and trade-offs:

- Increasing lanes or per-lane rate raises raw throughput but increases PCB/connector complexity and energy per bit.
- Higher protocol efficiency (larger packets, lower header overhead) benefits bulk ML tensor transfers but worsens tail latency for small control messages.
- Coherent fabrics simplify programming and reduce explicit data movement but force stricter cache coherency traffic, raising fabric load under mixed workloads.
- Topology choices: mesh favors locality and lower wiring, while fat-tree supports uniform bandwidth at higher cost.

Designers should match link aggregation, protocol efficiency, and coherency policy to target workloads (graphics streaming vs large-model parameter sync) to balance latency, bandwidth, power, and scalability.

7.3.2 CXL and coherent interfaces

Continuing the comparison with NVLink and Infinity Fabric, we now examine how Compute Express Link (CXL) and related coherent interfaces extend heterogeneous system coherence to GPUs, and which microarchitectural choices matter for SM-level performance and model-parallel ML workloads.

CXL is a PCIe-based open standard that provides three protocol types: device memory access (CXL.mem), cache coherence (CXL.cache), and fabric management (CXL.io). For GPUs the key capability is CXL.cache: a host or accelerator may present cache lines as coherent, allowing remote CPU or GPU agents to read and write while preserving line-level coherence. The engineering problem is integrating that coherence into an SM-centric memory hierarchy without destroying SIMT throughput: maintaining low local access latency and high sustained bandwidth while enforcing global visibility across CPU, GPU, and other accelerators.

Analysis: two coherence architectures dominate for multi-agent systems:

- Snoob-based coherence keeps latency low for small node counts but floods fabric bandwidth as agent count grows.
- Directory-based coherence centralizes state per-line and scales better in bandwidth; it introduces directory lookup latency and additional metadata storage. Typical GPU L2/L3 slices act as coherence agents, and a GPU-side directory or snoop filter reduces fabric transactions.

Quantify the effective latency seen by a shader when remote coherence is possible. With local hit rate h , local latency L_{local} , remote-coherent access latency L_{remote} , and remote penalty P_{arb} (directory arbitration, fabric RTT), effective latency is

$$[H]L_{\text{eff}} = h \cdot L_{\text{local}} + (1 - h) \cdot (L_{\text{remote}} + P_{\text{arb}}). \quad (7.14)$$

This guides where to invest silicon: increasing cache capacity to raise h , or reducing P_{arb} via faster directory hardware, yields different power/area trade-offs.

Implementation patterns for GPUs:

- Place a per-L2 slice directory that tracks sharers at cache-line granularity; use bloom-filtered snoop filters to avoid expensive directory lookups for common private lines.
- Offload reordering and coarse-grain coherence operations into a hardware engine that batches invalidations to amortize fabric RTT.
- Map large model parameter regions as CXL.mem with explicit software-managed coherency to avoid frequent line-level coherence traffic.

A synthesizable, minimal directory filter for an L2 slice can be implemented as follows. It performs parallel tag compare and simple allocation; real designs augment this with sharer bitmaps and state machines for probe/response.

Listing 7.11: Minimal L2 directory filter (synthesizable).

```
module cxl_dir_filter #(
  parameter ENTRIES = 16,
  parameter TAGW    = 48,
  parameter IDW     = 4
) (
  input          clk,
  input          rst,
  input          req_valid,    // request present
  input          req_rw,      // 0=read,1=write
  input [TAGW-1:0] req_tag,    // line tag
  output reg     hit,
  output reg     grant_valid,  // grant to L2 or fabric
  output reg [IDW-1:0] grant_idx // matched/alloc index
);
  reg [TAGW-1:0] tags [0:ENTRIES-1];
  reg           val [0:ENTRIES-1];
  integer i;
  reg [IDW-1:0] alloc_ptr;
```

```

// parallel compare (combinational)
wire [ENTRIES-1:0] match;
genvar gi;
generate
  for (gi = 0; gi < ENTRIES; gi = gi + 1) begin : MATCH_GEN
    assign match[gi] = val[gi] && (tags[gi] == req_tag);
  end
endgenerate

// hit detection
always @(*) begin
  hit = |match;
end

// sequential update
always @(posedge clk) begin
  if (rst) begin
    alloc_ptr    <= 0;
    grant_valid <= 0;
    grant_idx    <= 0;
    for (i = 0; i < ENTRIES; i = i + 1) begin
      val[i]    <= 0;
      tags[i] <= 0;
    end
  end else begin
    grant_valid <= 0;

    if (req_valid) begin
      if (hit) begin
        // grant existing entry
        for (i = 0; i < ENTRIES; i = i + 1) begin
          if (match[i]) begin
            grant_idx    <= i[IDXW-1:0];
            grant_valid <= 1;
          end
        end
      end else begin
        // allocate new entry
        tags[alloc_ptr] <= req_tag;
        val[alloc_ptr]  <= 1'b1;
        grant_idx       <= alloc_ptr;
        grant_valid     <= 1'b1;
        alloc_ptr       <= (alloc_ptr + 1'b1) % ENTRIES;
      end
    end
  end
end

```

```

end
endmodule

```

7.3.3 PHY design and equalization

The previous discussion established system-level link training and coherence semantics used by host fabrics and die-to-die links; here we focus on the PHY-level mechanisms that realize those logical guarantees by shaping signals and recovering data on noisy, frequency-dependent channels.

PHY design must solve inter-symbol interference (ISI), frequency-dependent loss, and time-varying crosstalk while preserving latency and throughput targets for graphics and ML traffic. Practical PHY blocks include transmitter pre-emphasis (FFE), continuous-time linear equalizers (CTLE) in the RX front end, and decision feedback equalizers (DFE) or adaptive FFE in the digital domain. The engineering problem is to trade silicon area, power, and convergence time against residual bit-error-rate (BER) at target data rates and modulation orders (NRZ or PAM-N). Convergence and stability are typically handled by an adaptive algorithm; a simple, robust choice is the least-mean-squares (LMS) rule used in decision-directed or training modes. Operationally, training mode uses a known sequence for initial convergence, then switches to decision-directed LMS for tracking.

Before showing an implementation sketch, the LMS update for tap vector w reads:

$$[H]w_{n+1} = w_n + \mu e_n x_n \quad (7.15)$$

where e_n is the error (decision minus equalized output), x_n is the tap input sample vector, and μ is a small positive step-size controlling stability and convergence speed. For fixed-point hardware, μ is implemented as a power-of-two reciprocal (shift) to avoid costly multiplies.

A compact synthesizable Verilog module below implements a fixed-point multi-tap LMS equalizer suitable for a GPU link PHY training engine. It uses signed arithmetic, shift-based step size, and a serial-shift sample buffer. This module targets the digital equalizer stage that follows a CTLE and ADC; it is deliberately parameterized for tap count and bit widths.

Listing 7.12: Fixed-point multi-tap LMS equalizer (synthesizable).

```

module phy_eq_lms #(
    parameter integer TAPS = 5,
    parameter integer SAMPLE_W = 10,
    parameter integer TAP_W = 16,
    parameter integer MU_SHIFT = 4 // mu = 1/2^MU_SHIFT
) (
    input wire clk,
    input wire rst, // sync reset
    input wire train_en, // 1 during training or tracking
    input wire signed [SAMPLE_W-1:0] rx_sample, // new ADC sample
    input wire signed [SAMPLE_W-1:0] ref_symbol, // known symbol in training or
        decision
    input wire decision_mode, // 1 = decision-directed, 0 = training
    output reg signed [SAMPLE_W-1:0] eq_out
);
    // tap storage

```

```

reg signed [TAP_W-1:0] taps [0:TAPS-1];
reg signed [SAMPLE_W-1:0] shift_reg [0:TAPS-1];
integer i;
// compute equalized output  $y = \sum \text{taps} * x$  (accumulate with extra width)
always @(posedge clk) begin
  if (rst) begin
    for (i=0;i0;i=i-1) shift_reg[i] <= shift_reg[i-1];
    shift_reg[0] <= rx_sample;
    // compute Y
    reg signed [TAP_W+SAMPLE_W+4:0] acc;
    acc = 0;
    for (i=0;i>> (TAP_W-1); // simple normalization
    // determine error  $e = \text{reference} - y$  (training) or decision -  $y$  (DD)
    reg signed [SAMPLE_W-1:0] ref;
    ref = (decision_mode) ? ref_symbol : ref_symbol; // placeholder; device
      supplies decision when DD
    reg signed [SAMPLE_W+8:0] err;
    err = $signed(ref) - $signed(eq_out);
    // LMS update when allowed
    if (train_en) begin
      for (i=0;i> MU_SHIFT
        reg signed [TAP_W+SAMPLE_W+8:0] delta;
        delta = (err * $signed(shift_reg[i])) >>> MU_SHIFT;
        taps[i] <= taps[i] + delta[TAP_W-1:0];
      end
    end
  end
endmodule

```

Key implementation considerations and trade-offs:

- Step-size μ must satisfy stability bounds; empirically choose $\mu < 1/(\lambda_{\max})$, where $\lambda_{\max}$ is input power spectral peak.
- ADC resolution and sampling jitter set a floor on achievable BER; more taps reduce ISI but increase area and latency.
- Decision-directed LMS saves training bandwidth but can misadapt under high noise; hybrid training is recommended.
- Hardware multipliers and accumulator widths drive power; shift-based μ and quantized taps reduce complexity.

Concrete design implication: allocate area budget for at least 3–7 adaptive taps and a 10–12 bit ADC in high-speed GPU links to meet $\text{sub-}1 \times 10^{-12}$ BER at multi-Gbps rates, balancing convergence speed (larger μ , fewer bits) against steady-state noise immunity (smaller μ , more bits).

7.3.4 Latency and throughput tuning

The previous discussion of PHY equalization and CXL/coherent interface behavior showed how physical-layer tuning and cache-coherent protocols reduce symbol errors and maintain ordering; latency and throughput tuning applies those gains to system-level flow-control and buffering to meet GPU workload demands. Here we analyze the concrete problem of sizing buffers, setting credit limits, and choosing packetization to hide round-trip latency while maximizing link utilization for SM-to-memory, GPU-to-GPU, and accelerator-to-host paths.

Problem: modern GPU workloads present two conflicting needs. SIMT-heavy graphics and RT-core ray batches favor many small, low-latency control packets, while tensor-core ML training favors sustained large transfers to amortize DRAM and HBM access. The metric tying latency and throughput is the bandwidth–delay product (BDP); operation requires sufficient outstanding data to fully occupy the link:

$$[H]BDP = B \cdot RTT \quad (7.16)$$

where B is link bandwidth (bytes/s) and RTT the path round-trip time (s). The number of flits that must be outstanding is approximately $N = \lceil BDP/s_f \rceil$ for flit size s_f bytes. Example: a 50 GB/s link with RTT 1 μ s and $s_f = 256$ B requires $N \approx \lceil (50 \times 10^9 \times 10^{-6})/256 \rceil \approx 196$ flits.

Analysis: insufficient credits (for credit-based flow control) or too-small TX/RX buffers cause throughput collapse as the sender stalls waiting for credits, raising effective latency. Excessive buffering reduces head-of-line blocking but increases silicon area and increases worst-case flushing time during resets. Packetization influences efficiency: larger payloads reduce per-packet header overhead but raise tail latency for latency-sensitive messages.

Implementation considerations:

- Use separate virtual channels for control and bulk data to avoid blocking; prioritize control VCs in arbiter logic.
- Configure credit counters to at least N plus margin for reordering and in-flight ACKs.
- Apply dynamic coalescing: for tensor-core bulk transfers, aggregate small requests into large descriptors; for RT-core and TMU traffic, bypass coalescer.
- For multi-hop topologies (mesh or ring), add per-hop buffer of $\approx BDP_{\text{hop}}$ and account for router queuing delays.

The following Python snippet computes needed credits and tradeoff curves for varying RTT and flit size; use it to set firmware-configurable register defaults.

Listing 7.13: Compute required credits and buffer bytes for link tuning

```
# Simple credit calculator for link tuning
def required_credits(bw_bytes_s, rtt_s, flit_size=256, margin=1.2):
    bdp = bw_bytes_s * rtt_s                # bytes in flight
    credits = int((bdp / flit_size) * margin + 0.999) # ceil with margin
    buf_bytes = credits * flit_size
    return credits, buf_bytes

# Example: 50 GB/s, 1us RTT
```

```
credits, buf = required_credits(50e9, 1e-6, flit_size=256, margin=1.25)
print(f"credits={credits} flits, buffer={buf/1024:.1f} KiB") # debug output
```

Design implications and trade-offs:

- Increasing credits/ buffers improves sustained throughput but costs area and power; target per-link buffer proportional to BDP and workload mix.
- Prioritize separate VCs and low-latency fast-paths for small control packets to preserve SIMT responsiveness.
- Firmware should expose credit margin knobs to tune for topology changes (multi-GPU, PCIe bridges).
- Aggressive packet coalescing raises throughput for tensor workloads but worsens tail latency for RT and display paths; choose adaptive policies.

7.4 Multi-GPU Communication

7.4.1 GPU-to-GPU transfers

Building on the preceding discussion of PHY and PCIe protocol tradeoffs, this subsection focuses on the concrete problems and hardware patterns used to move large buffers directly between GPUs. It ties link-layer capabilities to DMA behavior in multi-GPU workloads such as model-parallel training and multi-GPU raster/compositing pipelines.

Problem: GPUs must transfer large tensors or framebuffer tiles with minimal CPU intervention and predictable latency. Analysis centers on three contributors to end-to-end transfer time: fixed link latency, per-transfer software/queueing overhead, and sustained bandwidth. Operationally, designers optimize for low latency on small control messages and high sustained throughput on bulk transfers. A simple transfer time model useful in architecture trade studies is

$$[H]T_{\text{transfer}} \approx L_{\text{link}} + L_{\text{sw}} + \frac{S}{B_{\text{eff}}}, \quad (7.17)$$

where L_{link} is physical link latency, L_{sw} is host or driver overhead, S is payload size, and B_{eff} is effective bandwidth after protocol and contention overheads.

Analysis: NVLink-like interconnects reduce L_{sw} by enabling peer-to-peer DMA and coherent mappings, and increase B_{eff} relative to PCIe Gen3/Gen4. For small S , latency dominates; for large S , maximizing B_{eff} and avoiding serialization points (CPU copy, staging buffers) is critical. Collective operations and rendezvous protocols change message patterns; hardware support for atomic and scatter-gather accelerates these patterns.

Implementation pattern: a hardware descriptor-driven DMA engine on each GPU that:

- Accepts descriptors (destination peer, address, beat count).
- Issues AXI/NoC read bursts to local memory.
- Packetizes beats to a link-layer FIFO (AXI-Stream) while observing backpressure.
- Supports zero-copy by operating on pinned VRAM and GPUDirect RDMA endpoints.

Below is a synthesizable Verilog example of a minimal scatter-gather read engine that issues read bursts and streams data to a link output. It shows key control states and ready/valid handshakes used in production designs.

Listing 7.14: Descriptor-driven AXI-to-stream DMA (simplified, synthesizable).

```

module sg_dma #(
    parameter ADDR_WIDTH = 64,
    parameter DATA_WIDTH = 512,
    parameter LEN_WIDTH = 16
) (
    input wire clk,
    input wire rst_n,
    // Descriptor input (one descriptor at a time)
    input wire desc_valid,
    input wire [ADDR_WIDTH-1:0] desc_addr,
    input wire [LEN_WIDTH-1:0] desc_len, // number of beats
    output reg desc_ready,
    // Simplified AXI4 read master (single outstanding burst)
    output reg arvalid,
    output reg [ADDR_WIDTH-1:0] araddr,
    output reg [7:0] arlen,
    input wire arready,
    input wire rvalid,
    input wire [DATA_WIDTH-1:0] rdata,
    input wire rlast,
    output reg rready,
    // AXI-Stream output to link
    output reg tvalid,
    output reg [DATA_WIDTH-1:0] tdata,
    input wire tready
);

localparam IDLE = 3'd0,
            ISSUE_AR = 3'd1,
            WAIT_RESP = 3'd2,
            STREAMING = 3'd3,
            DONE = 3'd4;

reg [2:0] state, next_state;
reg [LEN_WIDTH-1:0] beats_rem;
reg [7:0] burst_len;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        desc_ready <= 1'b0;
        arvalid <= 1'b0;
    
```

```

    rready <= 1'b0;
    tvalid <= 1'b0;
    beats_rem <= 0;
end else begin
    state <= next_state;
    // handshake updates
    if (state==IDLE && desc_valid) begin
        desc_ready <= 1'b1;
        beats_rem <= desc_len;
        araddr <= desc_addr;
    end else begin
        desc_ready <= 1'b0;
    end
    if (state==ISSUE_AR && arready) arvalid <= 1'b0;
    if (state==ISSUE_AR && !arvalid) begin
        arvalid <= 1'b1;
        // burst length = min(255,beats_rem)-1 per AXI spec
        burst_len <= (beats_rem > 8'd255) ? 8'd255 : (beats_rem[7:0]);
        arlen <= (burst_len==0) ? 8'd0 : (burst_len-1);
    end
    // streaming: forward read data to stream
    if (state==STREAMING) begin
        rready <= 1'b1;
        if (rvalid && rready && tready) begin
            tvalid <= 1'b1;
            tdata <= rdata;
        end
        if (rvalid && rready && tready && rlast) begin
            // decrement beats_rem by beats in this burst
            beats_rem <= beats_rem - (burst_len==0 ? 1 : burst_len);
            tvalid <= 1'b0;
            rready <= 1'b0;
        end
    end else begin
        rready <= 1'b0;
        tvalid <= 1'b0;
    end
end

always @(*) begin
    next_state = state;
    case (state)
        IDLE: if (desc_valid) next_state = ISSUE_AR;
        ISSUE_AR: if (arvalid && arready) next_state = WAIT_RESP;
        WAIT_RESP: if (rvalid) next_state = STREAMING;
        STREAMING: if (rvalid && rlast) begin

```

```

        if (beats_rem > (burst_len == 0 ? 1 : burst_len)) next_state =
            ISSUE_AR;
        else next_state = DONE;
    end
    DONE: next_state = IDLE;
endcase
end

endmodule

```

Implications and trade-offs:

- Hardware DMA reduces L_{sw} and CPU load, but requires pinned VRAM and coherent mappings for GPUDirect.
- Larger bursts increase B_{eff} but raise latency for small messages; tune burst size per workload.
- Backpressure integration between AXI and link stream is essential to avoid buffer overflow under crossbar contention.
- Supporting atomic or in-place reductions in the network reduces inter-GPU communication for synchronous ML collectives, at the cost of increased link-level complexity.

7.4.2 Collective operations

Collectives of interest include broadcast, reduce, all-reduce, all-gather, and reduce-scatter. Architecturally the choice of algorithm must match: small-message collectives favor tree-based algorithms to minimize latency (few hops), while large-message all-reduce for ML gradients typically uses bandwidth-optimal ring or hierarchical algorithms. To reason about performance, use the standard latency-bandwidth model where α is per-step latency and β is per-byte transmission time. For an all-reduce of N bytes per GPU on P GPUs using a ring algorithm, the dominant cost is $P - 1$ communication steps and each GPU transmits $2(P - 1)/P \cdot N$ bytes overall, giving the approximate time

$$[H]T_{\text{ring}} \approx (P - 1)\alpha + 2\frac{P - 1}{P}N\beta. \quad (7.18)$$

This formula highlights two operational trade-offs: increasing P raises the latency term linearly, while per-GPU effective bandwidth degrades only mildly for large N because the per-GPU serialized bytes scale as $\sim 2N$.

Implementation analysis must consider:

- Topology-awareness: NVLink meshes, switch fabric, or multi-socket NUMA affect hop counts and achievable β . Mapping logical ranks to physical ports reduces cross-socket traffic.
- Offload capability: NICs that implement collective offloads (e.g., SHARP-like) reduce SM and DMA engine involvement.
- Memory staging: zero-copy (GPUDirect) avoids host round-trips; when not available, staging adds extra $N \cdot \beta_{\text{host}}$ cost and CPU overhead.

- Compute overlap: use SM compute (including tensor cores) to fuse reduction compute with communication where precision allows (e.g., FP16 accumulate in FP32 accumulator).

A compact host-side example using NCCL and CUDA streams demonstrates orchestration: allocate device buffers, create a communicator, launch `ncclAllReduce` on a CUDA stream, then synchronize. Comments in the code explain the key steps.

Listing 7.15: Host-side NCCL all-reduce using CUDA streams (simplified).

```
#include
#include
#include

int main(int argc, char** argv){
    int rank=0, nranks=4; // assign ranks/size beforehand
    size_t N = 1<<20; // elements per GPU
    float *d_buf; cudaMalloc(&d_buf, N*sizeof(float)); // device buffer
    cudaStream_t s; cudaStreamCreate(&s); // per-rank stream

    ncclComm_t comm;
    ncclCommInitRank(&comm, nranks, /*ncclId*/(ncclUniqueId*)argv, rank);

    // launch collective on stream; NCCL handles NVLink/PCIe/GPUDirect
    // internally
    ncclAllReduce((const void*)d_buf, (void*)d_buf, N, ncclFloat, ncclSum, comm,
        s);

    cudaStreamSynchronize(s); // ensure completion before using results
    ncclCommDestroy(comm);
    cudaFree(d_buf); cudaStreamDestroy(s);
    return 0;
}
```

Design implications and trade-offs:

- Choose ring for large contiguous gradient transfers; choose tree for latency-sensitive control messages.
- Topology-aware rank placement often yields 20–40% effective bandwidth gains by avoiding cross-switch hops.
- Offloading collectives to network hardware reduces SM occupancy and power but increases NIC complexity and tight coupling between driver and hardware.
- Mixed-precision reduction using tensor cores can accelerate aggregation but introduces numerical considerations requiring accumulation in higher precision or compensated summation.

7.4.3 Topology optimization

7.4.4 Topology Optimization for GPU Interconnects

The previous subsections examined how collective primitives and point-to-point GPU transfers exercise the interconnect; topology optimization is the next lever that determines path lengths, congestion points, and achievable collective bandwidth for both DMA and collective kernels. Here we formalize the optimization objectives, analyze common topologies in GPU systems, and present a small evaluation utility to quantify trade-offs.

Problem statement and metrics. Given N GPUs, per-link physical port count P per GPU, and per-link raw bandwidth B_w , choose a graph topology $G(V, E)$ to maximize effective throughput for common communication patterns (allreduce, allgather, peer-to-peer DMA) while respecting wiring, PHY, and switch cost. Key metrics:

- Average shortest-path hops H_{avg} (latency proxy for small messages).
- Diameter D (worst-case hop count).
- Bisection bandwidth B_{bis} (limits throughput for global collectives).
- Link count and PHY budget $L_{\text{total}} = |E|$.

Bisection bandwidth is naturally expressed from the adjacency and per-link capacity: for a partition $S \subset V$,

$$[H]B_{\text{bis}}(S) = \sum_{i \in S} \sum_{j \in V \setminus S} b_{ij}, \quad (7.19)$$

and the topology's bisection bandwidth is the minimum of B_{bis} over balanced partitions. For homogeneous links $b_{ij} = B_w$, an upper bound is $B_{\text{bis}} \leq B_w \cdot |E_{\text{cut}}|$.

Topology tradeoffs (analysis):

- Full mesh (direct NVLink between all pairs): $H_{\text{avg}} = 1$, $D = 1$, $|E| = N(N - 1)/2$. Extremely low latency for collectives and P2P, but prohibitive PHY count and switch complexity for $N > 16$.
- Switch-based fabric (NVSwitch or InfiniBand leaf-spine): $H_{\text{avg}} \approx 2$ (GPU→switch→GPU), scalable; bisection determined by switch fabric oversubscription and port allocation.
- Ring/torus: link budget linear in N , simple wiring, but $H_{\text{avg}} \approx N/4$ for ring (large N) and $D \approx N/2$; efficient for streaming patterns but poor for latency-sensitive collectives.
- Fat-tree: designed to raise bisection; diameter depends on tree depth, and cost grows with root-level port count.

Practical evaluation: compute H_{avg} and a balanced bisection bandwidth from an adjacency matrix. The utility below is small and production-ready for topology evaluation.

Listing 7.16: Compute average hop count and bisection bandwidth

```
from collections import deque
def avg_shortest_path(adj):
    N = len(adj)
```

```

total = 0
for s in range(N):
    dist = [-1]*N
    q = deque([s]); dist[s]=0
    while q:
        u = q.popleft()
        for v, cap in enumerate(adj[u]):
            if cap and dist[v]==-1:
                dist[v]=dist[u]+1; q.append(v)
    total += sum(d for d in dist if d>0)
return total/(N*(N-1)) # average hops per ordered pair

def bisection_bandwidth(adj, Bw):
    N = len(adj); half = N//2
    cut = 0
    S = set(range(half))
    for i in S:
        for j in range(N):
            if j not in S:
                cut += adj[i][j] # capacity count
    return cut * Bw # total capacity across cut

# Example usage: adj as 0/1 capacity matrix; Bw in GB/s
# # adj = [[0,1,1,...], ...]; print(avg_shortest_path(adj),
#   bisection_bandwidth(adj, 25.0))

```

Design implications and trade-offs:

- For small- N multi-GPU nodes ($N \leq 8$), direct meshes give best latency and collective efficiency but consume many PHY lanes; budget cost scales $\propto N^2$.
- For scale-out ($N \gg 16$), switch fabrics or hierarchical fat-tree topologies deliver higher bisection per cost; they add a hop that raises latency for small messages but amortize in large-bandwidth collectives.
- Topology-aware routing and collective algorithms (ring allreduce vs tree allreduce) should be matched: minimize hop count for latency-critical control messages and maximize bisection utilization for high-throughput reductions.

7.4.5 Scalability challenges

The previous subsections established how topology choices (ring, fat-tree, NVSwitch fabric) interact with collective kernels and how algorithmic patterns (ring all-reduce, tree-based broadcast) map to physical links; the discussion below examines the scalability limits that arise when both topology and collective behavior are pushed to large node counts.

Contending goals create the core problem: maximize sustained bandwidth for large collectives while preserving low tail latency for small control messages (synchronization, kernel launch). Analysis shows two dominant scaling bottlenecks: (1) bisection bandwidth and congestion hotspots as effective aggregate throughput per GPU falls with shared links or oversub-

scribed switches; and (2) synchronization latency and software overhead that grows with node count, increasing stalls in SMs and lowering utilization of tensor cores and FPUs. For latency-dominated collectives the time model is commonly approximated as

$$[H]T_{\text{allreduce}}(N, S) \approx \alpha \log N + \beta \frac{N-1}{N} \frac{S}{B}, \quad (7.20)$$

where α is per-hop latency, β captures per-byte overhead, S is message size, B is link bandwidth, and N is GPU count. This expression clarifies that small- S workloads are latency-bound (logarithmic scaling), while large- S transfers are bandwidth-bound and limited by cross-section B and topology-induced contention.

Implementation strategies that mitigate these limits include:

- Topology-aware collective scheduling: map participating ranks to physical proximity to reduce α and cut number of congested links.
- Link aggregation with striping: split large payloads across multiple physical links, using credit-based flow control to avoid head-of-line blocking.
- Offload engines (RDMA/GPU-direct) and on-NIC collective offloads to reduce CPU/driver scheduling overhead and limit SM stalls.
- Adaptive algorithms: switch between tree and ring all-reduce depending on S and N .

A synthesizable Verilog module below shows a small link-aggregator with credit flow control suitable for striping flits across two physical links; it demonstrates the hardware building block for scalable multi-link transfers used in multi-GPU fabrics.

Listing 7.17: Two-link flit striping with credit-based flow control

```

module link_aggregator #(
    parameter FLIT_W = 128
) (
    input  wire          clk,
    input  wire          rst_n,
    input  wire [FLIT_W-1:0] in_flit,
    input  wire          in_valid,
    output wire          in_ready,
    // link 0
    output reg  [FLIT_W-1:0] link0_flit,
    output reg          link0_valid,
    input  wire          link0_credit, // credit returned when consumed
    // link 1
    output reg  [FLIT_W-1:0] link1_flit,
    output reg          link1_valid,
    input  wire          link1_credit
);

    // simple credit counters (1-bit per link for example)
    reg link0_busy, link1_busy;
    reg toggle; // round-robin striping bit

```

```

assign in_ready = ~((toggle & link1_busy) | (~toggle & link0_busy));

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    link0_flit <= {FLIT_W{1'b0}};
    link1_flit <= {FLIT_W{1'b0}};
    link0_valid <= 1'b0;
    link1_valid <= 1'b0;
    link0_busy <= 1'b0;
    link1_busy <= 1'b0;
    toggle <= 1'b0;
  end else begin
    // accept and stripe when input valid and target link free
    if (in_valid && in_ready) begin
      if (~toggle && ~link0_busy) begin
        link0_flit <= in_flit;
        link0_valid <= 1'b1;
        link0_busy <= 1'b1;
        toggle <= ~toggle;
      end else if (toggle && ~link1_busy) begin
        link1_flit <= in_flit;
        link1_valid <= 1'b1;
        link1_busy <= 1'b1;
        toggle <= ~toggle;
      end
    end
    // release when credits observed
    if (link0_credit) begin link0_valid <= 1'b0; link0_busy <= 1'b0; end
    if (link1_credit) begin link1_valid <= 1'b0; link1_busy <= 1'b0; end
  end
endmodule

```

Implications and trade-offs:

- Hardware striping increases aggregate B but raises per-packet reassembly cost and requires precise flow-control to prevent mis-ordering.
- Topology-aware scheduling reduces effective α but demands runtime placement intelligence in the driver/allocator, complicating job multiplexing.
- Offloads and NIC-level collectives reduce CPU stalls and SM starvation but add silicon area and complexity in error/recovery paths.
- Scaling beyond tens of GPUs typically requires non-blocking fabrics (NVSwitch, fat-tree) or hierarchical collectives to bound logarithmic latency growth.

7.5 Section 6: Verilog Implementation

7.5.1 Crossbar switch module

Building on the router buffering and arbitration principles discussed earlier in this chapter, this subsection provides a synthesizable crossbar switch designed for SM-to-memory partition fabrics and on-chip NoC aggregation. The module targets single-beat packet routing with per-output round-robin arbitration to guarantee fairness among contending SMs while preserving SIMT streaming throughput.

Problem: route N input channels from shader engines (SMs) to M outputs (memory partitions, CUs, or ROPs) without head-of-line blocking and with bounded arbitration latency. Analysis shows each output j cannot exceed its link bandwidth B_j ; if inputs issue at rates r_i toward j , the utilization U_j is bounded by

$$[H]U_j = \min\left(1, \frac{\sum_{i \in S_j} r_i}{B_j}\right), \quad (7.21)$$

where S_j is the set of inputs targeting output j . The crossbar must provide per-output arbitration that maps one winner input to an output when multiple requests collide, and assert `in_ready` only for the selected input to implement backpressure.

Implementation: a parameterized Verilog crossbar that accepts:

- per-input signals: `in_valid`, `in_dest` ($\log_2(M)$ bits), `in_data`;
- per-output signals: `out_ready`, `out_valid`, `out_data`.

Each output instantiates a small round-robin arbiter maintaining a priority pointer; selection logic is purely combinational, with the grant registered to avoid glitching. The module is fully synchronous and synthesizable.

Listing 7.18: Synthesizable parameterized crossbar with per-output round-robin arbitration

```

module crossbar #(
    parameter N = 8,                // number of inputs (SM links)
    parameter M = 4,                // number of outputs (memory partitions)
    parameter W = 64                // data width
) (
    input  wire      clk,
    input  wire      rst,
    input  wire [N-1:0] in_valid,
    input  wire [N*W-1:0] in_data,
    input  wire [$clog2(M)-1:0] in_dest [0:N-1], // per-input destination
    output reg [N-1:0] in_ready,
    output reg [M-1:0] out_valid,
    output reg [M*W-1:0] out_data,
    input  wire [M-1:0] out_ready

);

    // per-output priority pointer and combinational grant generation
    reg [$clog2(N)-1:0] prio_ptr [0:M-1];

```

```

integer i, j;

// build request matrix: req[j][i] == input i requests output j
wire req [0:M-1][0:N-1];
generate
  for (j = 0; j < M; j = j + 1) begin : BUILD_REQ
    for (i = 0; i < N; i = i + 1) begin : REQ_GEN
      assign req[j][i] = in_valid[i] && (in_dest[i] == j);
    end
  end
endgenerate

// arbitration and output selection
reg [N-1:0] grant [0:M-1];
always @(*) begin
  for (j = 0; j < M; j = j + 1) begin
    grant[j] = {N{1'b0}};
    out_valid[j] = 1'b0;
    out_data[j*W +: W] = {W{1'b0}};
    for (i = 0; i < N; i = i + 1) begin
      int idx = (prio_ptr[j] + i) % N;
      if (req[j][idx] && !out_valid[j]) begin
        grant[j][idx] = 1'b1;
        out_valid[j] = 1'b1;
        out_data[j*W +: W] = in_data[idx*W +: W];
      end
    end
  end
end

// ready feedback and round-robin pointer update
always @(posedge clk) begin
  if (rst) begin
    for (j = 0; j < M; j = j + 1)
      prio_ptr[j] <= 0;
    for (i = 0; i < N; i = i + 1)
      in_ready[i] <= 1'b0;
  end else begin
    for (i = 0; i < N; i = i + 1)
      in_ready[i] <= 1'b0;

    for (j = 0; j < M; j = j + 1) begin
      if (out_valid[j] && out_ready[j]) begin
        for (i = 0; i < N; i = i + 1) begin
          if (grant[j][i]) begin
            in_ready[i] <= 1'b1;
            prio_ptr[j] <= (i + 1) % N;
          end
        end
      end
    end
  end
end

```

```

        end
    end
end
end
end
end
endmodule

```

7.5.2 Round-robin arbiter

This module follows the crossbar switch discussion by specifying the per-port arbitration needed at each crossbar input so that competing SM/CU requestors (texture units, TMUs, ROPs, or memory controllers) receive fair service without head-of-line starvation. The round-robin arbiter presented below trades minimal logic for bounded latency and predictable fairness across many SIMT-driven request streams.

Problem: multiple initiators present one-hot requests; the crossbar can accept a single grant per cycle. Goals: (1) no starvation, (2) simple rotate-based priority for fairness, (3) parameterizable width for SM, L2 slice, or PCIe request fans.

Analysis:

- A canonical round-robin maintains a rotating pointer P that gives priority to requestor P , then $P + 1$, ..., wrapping to $P - 1$.
- Worst-case wait for any requester in a non-blocking single-grant arbiter is

$$[H]L_{\max} = N - 1, \quad (7.22)$$

where N is the number of requestors; average wait under uniform load is $\approx (N - 1)/2$ cycles.

- Throughput is one grant per cycle when requests are saturated; latency impact on GPU pipelines must be evaluated against SM issue/backpressure and memory controller buffering.

Implementation: the Verilog below is synthesizable and parameterized by WIDTH. It finds the first set bit after the pointer by iterating and produces a one-hot grant. The pointer advances to the next position after a successful grant, ensuring strict round-robin fairness.

Listing 7.19: Parameterizable round-robin arbiter (synthesizable).

```

module round_robin_arbiter #(parameter WIDTH = 8) (
    input wire      clk,
    input wire      rst_n,
    input wire [WIDTH-1:0] req,    // one-hot request vector
    output reg [WIDTH-1:0] grant   // one-hot grant vector
);
    // pointer width (computed at elaboration)
    function integer clog2(input integer x); integer i; begin i=0; x=x-1;
        while(x>0) begin x=x>>1; i=i+1; end clog2=i; end endfunction
    localparam PTR_W = (WIDTH<=1) ? 1 : clog2(WIDTH);

```

```

reg [PTR_W-1:0] pointer;           // current priority base
integer i;
integer sel_idx;
reg found;

// combinational selection: first request at or after pointer
always @(*) begin
    grant = {WIDTH{1'b0}};
    found = 1'b0;
    sel_idx = 0;
    for (i = 0; i < WIDTH; i = i + 1) begin
        // compute candidate index with wrap-around
        integer idx;
        idx = (pointer + i) % WIDTH;
        if (!found && req[idx]) begin
            grant[idx] = 1'b1;      // one-hot grant
            sel_idx = idx;
            found = 1'b1;
        end
    end
end

// pointer update and grant register (synchronous)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pointer <= {PTR_W{1'b0}};
        // grant reset handled by combinational block above; keep stable
        // on reset
    end else begin
        if (!grant) begin
            // advance pointer to next position after granted index
            pointer <= (sel_idx + 1) % WIDTH;
        end
    end
end
endmodule

```

Implications and trade-offs:

- Latency bound: equation (1) guarantees no requester waits more than $N - 1$ cycles; suitable for SM-to-L2 links where bounded arbitration reduces pipeline stalls.
- Critical path: combinational priority loop grows with WIDTH; for large N , pipeline the arbiter or implement hierarchical tree of smaller arbiters to meet timing.
- Area vs fairness: simple RR uses little state (pointer), but supporting locked grants or multi-issue per cycle increases complexity and requires extra logic and buffering.

7.5.3 PCIe transaction engine

The arbiter and crossbar modules established deterministic request ordering and path multiplexing; the PCIe transaction engine now consumes those ordered streams and turns GPU DMA commands into wire-form TLPs while enforcing link-layer constraints and outstanding-request limits.

The problem is to assemble PCIe Memory Read/Write TLPs, allocate and recycle numeric tags, split transfers to fit Max Payload Size (MPS), and pace transmissions to match PCIe link capacity. Analysis begins with the required outstanding-request depth to saturate a link given round-trip latency (RTT) and payload size P (bytes):

$$[H]N_{\text{req}} = \left\lceil \frac{B \cdot \text{RTT}}{P} \right\rceil \quad (7.23)$$

where B is effective link bandwidth in bytes/sec (after 128b/130b or 64b/66b encoding). For PCIe Gen4 x16, per-lane effective rate is approximately $r \approx 15.754$ Gb/s, so $B \approx 16 \cdot r/8$ bytes/sec. This formula guides tag pool sizing to avoid link underutilization.

Implementation choices:

- lightweight tag allocator using a bitmask for synthesizable priority encoding;
- header assembly that supports splitting by parameterized MPS;
- a simple flow-control counter to prevent more than one TLP per tag (ensures in-flight tracking);
- an AXI-Stream-like transmit interface to the link layer.

The following Verilog is a synthesizable, production-ready TX engine simplified for clarity. It performs tag allocation, header emit, and payload streaming. It expects completions to return tag identifiers to free resources.

Listing 7.20: Simplified PCIe transaction engine: tag allocation, TLP assembly, payload split.

```

module pcie_tx_engine #(
    parameter TAG_WIDTH = 4,                // number of tag bits
    parameter TAG_COUNT = (1< DATA_WIDTH/8) remain <= remain - DATA_WIDTH/8;
    else begin
        remain <= 16'd0;
        state <= S_IDLE;
        req_ready <= 1'b1;
    end
    end else tx_axis_tvalid <= 1'b0;
end
endcase
end
end
endmodule

```

Implications and trade-offs:

- Tag pool sizing must follow equation (7.23); undersized pools throttle throughput, oversized pools increase state and area.
- Splitting large transfers increases header overhead and PCIe transaction processing in the root complex.
- A hardware allocator using bitmask priority encoding is area-efficient and scales to typical tag counts (16–256).
- For ML and graphics (large DMA bursts), ensure MPS and tag depth match worst-case RTT to keep SM/Tensor cores fed without stalling.

7.5.4 NoC router and testbench

The preceding PCIe transaction engine and the round-robin arbiter provided transaction-level ordering and fair output selection respectively; this router uses a similar round-robin arbitration per output while integrating credit-based flow control to prevent buffer overrun across SM-to-memory links. The design goal is a synthesizable, low-latency 4-port mesh NoC router with single-Virtual-Channel FIFOs and per-output round-robin arbiters that map well to GPU SM and memory partition traffic patterns.

Problem: on-chip NoC routers must deliver high throughput for bursty SM traffic while avoiding head-of-line blocking and deadlock. Analysis shows that minimal buffering combined with per-output arbitration and credit-based backpressure yields predictable performance. Required buffer depth D must cover the worst-case round-trip flow-control delay; a lower bound is

$$[H]D \geq \left\lceil \frac{BW \cdot RTT}{F} \right\rceil \quad (7.24)$$

where BW is link bandwidth (bits s⁻¹), RTT is flow-control round-trip time (s), and F is flit width (bits). In a GPU tiled mesh, RTT includes router pipeline latency plus inter-router serialization delays.

Implementation: the router below implements:

- 4 bi-directional ports (N,E,S,W) as inputs and outputs, plus local injection/egress combined into the West port for SM correspondence.
- Per-input single-flit FIFO (parameterizable depth).
- Per-output round-robin arbiter selecting one winner per cycle.
- Credit-based flow control: outputs expose `out_ready` signals; inputs track a simple credit counter to stall injection when credits are exhausted.

Key properties:

- Single-cycle combinational switch allocation for low latency.
- Fairness via rotating priority in the arbiter.
- Synthesizable Verilog with parameters for flit width and depth.

Listing 7.21: Synthesizable 4-port NoC router with credit flow-control and RR arbitration.

```

module noc_router #(
    parameter FLIT_W = 32,
    parameter DEPTH = 4
) (
    input wire clk,
    input wire rst,
    input wire [3:0] in_valid,
    input wire [3:0][FLIT_W-1:0] in_data,
    output reg [3:0] in_ready, // credits available
    output reg [3:0] out_valid,
    output reg [3:0][FLIT_W-1:0] out_data,
    input wire [3:0] out_ready // downstream backpressure
);

// Simple shift-register FIFOs (single VC) per input
reg [FLIT_W-1:0] fifo [3:0][0:DEPTH-1];
reg [2:0] cnt [3:0]; // count per FIFO

integer i;
always @(posedge clk) begin
    if (rst) begin
        for (i = 0; i < 4; i = i + 1) begin
            cnt[i] <= 0;
            in_ready[i] <= 1;
        end
        out_valid <= 0;
        out_data <= '{default: 0};
    end else begin
        // enqueue
        for (i = 0; i < 4; i = i + 1) begin
            if (in_valid[i] && in_ready[i]) begin
                fifo[i][cnt[i]] <= in_data[i];
                cnt[i] <= cnt[i] + 1;
            end
            // update in_ready based on depth
            in_ready[i] <= (cnt[i] < DEPTH);
        end

        // clear outputs
        for (i = 0; i < 4; i = i + 1) begin
            out_valid[i] <= 0;
            out_data[i] <= {FLIT_W{1'b0}};
        end

        // fixed-priority routing (lowest index wins)
        integer j, src, k;

```

```

for (j = 0; j < 4; j = j + 1) begin
  for (src = 0; src < 4; src = src + 1) begin
    if (cnt[src] > 0) begin
      // destination encoded in lower 2 bits of flit
      if (fifo[src][0][1:0] == j && out_ready[j]) begin
        out_valid[j] <= 1;
        out_data[j] <= fifo[src][0];
        // dequeue by shifting
        for (k = 0; k < DEPTH - 1; k = k + 1) begin
          fifo[src][k] <= fifo[src][k + 1];
        end
        cnt[src] <= cnt[src] - 1;
        disable for; // one flit per output per cycle
      end
    end
  end
end
end
end
end
endmodule

```

Chapter 8

Performance Analysis and Optimization

8.1 Performance Metrics

8.1.1 Throughput (GFLOPS and TFLOPS)

This subsection follows the chapter’s metric taxonomy and moves from qualitative descriptions to the concrete arithmetic used to quantify GPU compute throughput and its relation to memory behavior. The exposition focuses on how to compute peak FLOPS, compare it to sustained measurements, and use those numbers to reason about architecture-level trade-offs.

Throughput for floating-point workloads is reported in FLOPS and commonly scaled to GFLOPS or TFLOPS. Architects distinguish peak theoretical throughput from sustained, application-level throughput. Peak throughput is an architectural property derived from the number of execution lanes, FMA (fused multiply-add) capability, and clock frequency. Define variables precisely and then give the canonical peak formula:

$$[H]\text{Peak FLOPS} = N_{\text{SM}} \times L \times R \times F \quad (8.1)$$

where N_{SM} is the count of streaming multiprocessors (SMs or CUs), L is the number of FP lanes (SIMD width) per SM, R is the number of floating-point results produced per lane per cycle (e.g., $R = 1$ for a single FMA), and F is clock frequency in flops/s per result. Note that an FMA typically counts as two floating-point operations (multiply + add), so F often includes this factor by setting $R \leftarrow R \times 2$.

Operational relevance: comparing peak to measured throughput identifies inefficiencies (memory stalls, divergence, poor ILP). Use the roofline arithmetic intensity threshold to classify bottlenecks:

$$[H]I_{\text{crit}} = \frac{\text{Peak FLOPS}}{B_{\text{mem}}} \quad (8.2)$$

where B_{mem} is sustained memory bandwidth in bytes/s. Workloads with arithmetic intensity I

8.1.2 Bandwidth utilization

Continuing from throughput metrics such as GFLOPS and TFLOPS, bandwidth utilization ties raw computational capability to the memory system’s ability to feed execution units (SMs, tensor cores, TMUs, ROPs) without starvation. Measuring and optimizing utilization exposes whether a workload is memory-bound or compute-bound and points to concrete microarchitectural fixes.

Bandwidth utilization quantifies how effectively the memory subsystem is used relative to its peak. Operationally this matters because many graphics and ML kernels are limited by data movement: texture fetches, scatter-gather for sparse tensors, or large GEMM input loads. Key contributors to utilization are access coalescing, request parallelism across memory channels and L2 slices, on-chip compression (DCC), and write-combine behavior at the ROP and DMA engines.

Analysis proceeds from counters exposed by PMUs: per-channel issued bytes, DRAM read/write cycles, L2 miss rate, and in-flight transaction counts. A compact metric is achieved bandwidth divided by theoretical peak:

$$[H]U = \frac{B_{\text{achieved}}}{B_{\text{peak}}} = \frac{\sum_i \text{bytes_issued}_i}{B_{\text{peak}} \cdot t} \quad (8.3)$$

where i indexes channels or memory controllers and t is the sampling interval. For example, if an HBM3 stack delivers 1.2 TB/s peak and counters show 600 GB transferred in one second, $U = 0.5$.

Practical measurement must correct for traffic amplification: cacheline fetches for narrow stores, read-for-ownership writebacks, and metadata from compression. Use derived counters to compute effective payload bytes versus protocol overhead. Latency hiding also affects interpretation: high utilization with long queue depths can indicate queuing bottlenecks rather than ideal streaming.

Implementation: sample a minimal set of counters and compute utilization and effective request size. The snippet below shows a simple analysis loop intended for on-host profiling tools that read hardware counters and report per-slice utilization and average request size.

Listing 8.1: Bandwidth-utilization calculation from PMU counters

```
# read counters: bytes_issued_per_channel, cycles, peak_bw_per_channel
# counters returned as dictionaries keyed by channel id.
def compute_util(bytes_issued, sample_time_s, peak_bw_per_ch):
    util = {}
    for ch, b in bytes_issued.items():
        peak = peak_bw_per_ch[ch] # bytes/sec
        util[ch] = b / (peak * sample_time_s) # fraction
    total_util = sum(bytes_issued.values()) / (sum(peak_bw_per_ch.values()) *
        sample_time_s)
    return util, total_util

# Example: bytes_issued={'ch0':3e11,'ch1':3e11}, sample_time_s=1.0
```

Interpretation requires correlating utilization with SM stall reasons, L2 miss counters, and queue depths. If utilization is low while SMs stall on memory, inspect request serialization (bank conflicts) or poor coalescing patterns. If utilization is near peak but performance stalls, consider latency spikes due to contention or insufficient concurrency.

Design implications and trade-offs:

- Increase peak BW (wider channels, higher DRAM rate) reduces U for same workload but raises power and area.
- Improve on-chip compression and L2 bandwidth amplification reduction to raise effective B_{achieved} without DRAM changes.

- Optimize access patterns (coalescing, tiling) to reduce small-request overhead at the cost of software complexity.
- Balance channel parallelism and QoS: prioritized traffic improves latency but may lower aggregate utilization under mixed workloads.

8.1.3 Cache efficiency

These observations on bandwidth and GFLOPS lead naturally to cache efficiency as the metric that connects raw arithmetic throughput and raw off-chip bandwidth through reuse and locality. Cache efficiency quantifies how well SMs and the memory subsystem convert DRAM bandwidth into useful data for SIMT execution.

Cache efficiency measures

- Problem: In modern GPUs, high arithmetic throughput is often starved by memory stalls when cache locality is poor. Cache efficiency captures hit rate, spatial and temporal reuse, and their impact on SM stall time.
- Analysis: Use the average memory-access time (AMAT) model adapted to GPU pipelines. Define T_{L1} and T_{L2} as L1 and L2 hit latencies (cycles), R_{L1} and R_{L2} as their miss rates (fraction of accesses that miss), and T_{DRAM} as off-chip latency in cycles. Then effective per-access latency is

$$[H]\text{AMAT} = T_{L1} + R_{L1}(T_{L2} + R_{L2}T_{\text{DRAM}}). \quad (8.4)$$

For SIMT warps, memory-stall cycles per load instruction approximate $S \approx \text{AMAT} \times N_{\text{memory_ops_per_warp}}$, scaled by outstanding request concurrency and memory-level parallelism (MLP). High MLP can hide part of AMAT, so effective stall is $S_{\text{eff}} = \max(0, \text{AMAT} - \text{overlap})$.

Practical measurement and microbenchmark

- Select counters: L1 loads, L1 misses, L2 hits, L2 misses, DRAM transactions, warp stalls. Compute hit rates: $H_{L1} = 1 - R_{L1}$ and effective bandwidth utilization = $\text{useful_bytes_transferred} / \text{peak_bytes_transferred}$.
- Use stride and working-set sweeps to map capacity and conflict behavior across L1/L2. Coalesced accesses reduce transaction count and lower R_{L2} ; texture and read-only caches yield higher spatial reuse for sampling workloads.

Implementation example: a CUDA microbenchmark that sweeps array size and stride to produce working-set vs hit-rate curves. Timing uses `cudaEvent` for reliable host timing; kernel reads with controlled stride to expose cache behavior.

Listing 8.2: CUDA microbenchmark: stride sweep for cache characterization

```
extern "C" __global__ void stride_read_kernel(int *a, int stride, int iters,
int n) {
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    // each thread walks a strided slice to probe spatial/temporal reuse
    int idx = (tid % n);
```

```

int sum = 0;
for (int i = 0; i < iters; ++i) {
    sum += a[idx];
    idx = (idx + stride) % n;
}
// prevent compiler optimization
if (tid==0) a[0]=sum;
}
// Host: allocate array sizes from L1DRAM and vary stride; read counters via
CUPTI or driver APIs.

```

Implications and trade-offs

- **Design:** Larger line sizes improve spatial locality but increase pollution for streaming kernels; higher associativity reduces conflict misses at area cost; partitioned L1/shared memory provides programmer-tunable locality.
- **Performance:** Improving H_{L1} or H_{L2} reduces AMAT per Eq. (1) and directly increases achieved GFLOPS for memory-bound kernels. However, larger caches add area/power and may lengthen hit latency T_{L1} , so the trade-off is between reduced miss rate and increased base hit time.
- **Operational:** For compute and ML workloads, prefer tiling and blocking to raise reuse; for texture-heavy graphics, leverage TMU caches and read-only caches to maximize hardware prefetch and spatial coalescing.

8.1.4 Power and thermal metrics

These metrics build directly on cache efficiency and bandwidth utilization: off-chip traffic raises power and shifts hot spots, while cache-friendly kernels reduce dynamic switching in memory controllers and ROPs. Quantifying power and temperature ties microarchitectural activity to DVFS and throttling policies.

Power and thermal metrics problem statement. For performance analysis we require:

1. accurate instantaneous power attribution (die, VRM, per-SM/CU, per-subsystem such as TMU/ROP/tensor cores),
2. temperature state (per-die sensor or per-region T_j estimates), and
3. a predictive thermal model to decide frequency/voltage and workload placement before throttling occurs.

Analysis: dynamic power per switching domain follows the well-known capacitive model $P_{\text{dyn}} = \alpha CV^2 f$, where α is activity factor, C is effective capacitance, V is supply, and f is frequency. Static leakage P_{leak} increases with temperature, approximated by an exponential dependence; for control purposes treat $P_{\text{total}} = P_{\text{dyn}} + P_{\text{leak}}(T)$. Thermal behavior of a die region is modeled by an RC thermal network:

$$[H]C_{\text{th}} \frac{dT(t)}{dt} + \frac{T(t) - T_{\text{amb}}}{R_{\text{th}}} = P_{\text{total}}(t), \quad (8.5)$$

giving steady-state temperature

$$[H]T_{ss} = T_{amb} + P_{ss} R_{th}. \quad (8.6)$$

This links power budgets to temperature envelopes and shows why bandwidth-heavy kernels (higher off-chip energy per access) can push T_{ss} above thermal limits even when compute-utilization is moderate.

Implementation: practical telemetry combines PMU counters (SM active cycles, tensor-core utilization, memory transactions), VRM current/voltage telemetry, and on-die temperature sensors. A lightweight runtime estimator projects P_{dyn} per domain from counters, folds in an empirical $P_{leak}(T)$ curve, and integrates (8.5) forward to predict temperature within the sensor time constant. The estimator then issues DVFS or kernel placement directives.

Example estimator (Python): simple per-region power + thermal predictor and DVFS decision.

Listing 8.3: Per-region power/thermal estimator and DVFS decision

```
# Inputs: counters dict keyed by region: {'SM0':{'active':cycles,...},...}
# sensors: {'T_amb':..., 'T_sensor_region':...}; vrm: {'V':..., 'f':...}
def estimate_and_control(counters, sensors, vrm, params):
    P = {} # power per region
    for r, c in counters.items():
        # activity -> dynamic power (alpha*C approximated by factor_k)
        P_dyn = params['k_dyn'][r] * (vrm['V']**2) * vrm['f'] * c['activity']
        P_leak = params['k_leak'][r] * vrm['V'] * exp(params['beta']*sensors[r])
        P[r] = P_dyn + P_leak
    # thermal integrate (forward Euler)
    T_next = {}
    for r in P:
        dT = (P[r] - (sensors[r]-sensors['T_amb']))/params['Rth'][r] / params['Cth'][r]
        T_next[r] = sensors[r] + dT * params['dt']
    # DVFS decision: throttle if any T_next exceeds threshold
    if any(T_next[r] > params['T_crit'][r] for r in T_next):
        return 'reduce_freq' # command to power controller
    return 'keep_freq'
# (real implementation reads counters/sensors and sends commands to firmware)
```

Implications and trade-offs:

- Per-region sensors and counters enable fine-grain control but cost area and routing complexity; coarse sensors reduce cost but increase conservative throttling.
- Aggressive DVFS yields energy savings but can increase latency due to voltage transition times and destabilize frequency-sensitive kernels.
- Thermal placement matters: concentrating work on cooler SMs improves throughput but may raise interconnect or VRM hot spots.
- For ML workloads, balancing tensor-core utilization against memory bandwidth often maximizes GFLOPS/W; optimized kernels that improve cache hit rate reduce P_{dyn} from memory subsystems and lower T_{ss} .

8.2 Bottleneck Identification

8.2.1 Memory-bound and compute-bound workloads

Continuing from the chapter’s metrics and roofline groundwork, this subsection applies those concepts to a practical decision tree for classifying kernels as memory-bound or compute-bound and prescribing mitigations appropriate to modern SM/CU microarchitectures.

Problem. On SIMT GPUs a kernel is memory-bound when off-chip DRAM transfers throttle throughput; it is compute-bound when ALU/Tensor throughput or instruction delivery limits achieved performance. Correct classification matters because optimizations that improve one class (e.g., adding more arithmetic for reuse) can be harmful for the other.

Analysis. Use operational intensity I (FLOPs per byte transferred) and the roofline relation to reason quantitatively. The roofline predicts

$$[H]P_{\text{achieved}} = \min(P_{\text{peak}}, B_{\text{mem}} \cdot I), \quad (8.7)$$

where P_{peak} is aggregate SM peak compute (including tensor cores if used) and B_{mem} is sustained DRAM bandwidth. The critical intensity separating regimes is

$$[H]I_{\text{crit}} = \frac{P_{\text{peak}}}{B_{\text{mem}}}. \quad (8.8)$$

If $I < I_{\text{crit}}$ the kernel is memory-bound; if $I > I_{\text{crit}}$ it is compute-bound. In practice, measured P_{achieved} and B_{mem} are used to compute an empirical $I_{\text{eff}} = P_{\text{achieved}}/B_{\text{mem}}$ and compare against I_{crit} .

Decision tree. The following steps classify a kernel and select mitigations.

1. Measure sustained B_{mem} with a streaming copy benchmark on the target device.
2. Measure P_{achieved} with the kernel under study.
3. Compute $I_{\text{eff}} = P_{\text{achieved}}/B_{\text{mem}}$.
4. Compute $I_{\text{crit}} = P_{\text{peak}}/B_{\text{mem}}$ using vendor-reported P_{peak} .
5. If $I_{\text{eff}} < I_{\text{crit}}$ the kernel is memory-bound; else compute-bound.
6. For memory-bound kernels:
 - Increase locality: use shared memory, texture/L1 cache, or register tiling.
 - Reduce traffic: compress data, use narrower types, or eliminate redundant loads/stores.
 - Exploit memory-level parallelism: increase occupancy to hide latency, use wider vector loads.
7. For compute-bound kernels:
 - Increase arithmetic intensity: fuse loops, reuse registers, or switch to tensor-core-accelerated math.
 - Reduce instruction overhead: eliminate divergent branches, use predication, or move constants to uniform registers.

- Exploit instruction-level parallelism: increase occupancy to hide pipeline latency, use independent instructions.

Example. Consider a simple vector add kernel.

```
__global__ void vec_add(float *c, const float *a, const float *b, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

Measured on an A100: $P_{\text{achieved}} = 1.2$ TFLOP/s, $B_{\text{mem}} = 1.55$ TB/s, $P_{\text{peak}} = 19.5$ TFLOP/s. Then $I_{\text{eff}} = 1.2/1.55 \approx 0.77$ FLOP/byte and $I_{\text{crit}} = 19.5/1.55 \approx 12.6$ FLOP/byte. Since $0.77 < 12.6$ the kernel is memory-bound. Mitigations: use vectorized loads/stores (float4), prefetch to L2, or overlap with other kernels.

Edge cases. When $I_{\text{eff}} \approx I_{\text{crit}}$ the kernel is balanced; small changes in either compute or memory can shift the bottleneck. In such cases profile both paths and choose the cheaper optimization.

8.2.2 Latency versus bandwidth limits

As established in the previous subsection, whether a kernel is memory-bound or compute-bound depends on arithmetic intensity and the roofline intersection; this subsection drills into the separate roles of latency and bandwidth in setting those limits and how GPU microarchitecture converts latency into an effective bandwidth that the compute datapath observes.

Modern GPUs convert raw DRAM latency into usable throughput by sustaining many outstanding requests across SMs, caches, and the memory crossbar. Two failure modes emerge: latency-limited behavior when insufficient concurrency exists to hide long round-trip times, and bandwidth-limited behavior when the memory subsystem cannot satisfy the aggregate demand even with many outstanding requests. Operationally, latency limits appear in workloads with dependent loads or small working sets and low warp-level parallelism; bandwidth limits appear in wide streaming kernels, large tensor GEMMs with moderate arithmetic intensity, and texture-heavy raster paths.

Quantify the conversion from latency to throughput before using it in performance models. Let

- L be the average memory round-trip latency in cycles,
- s be request size in bytes,
- f be GPU clock frequency in Hz,
- S be number of active SMs,
- W be average outstanding warps per SM (warps that can issue another memory request).

Then an upper bound on sustainable memory throughput T (bytes/s) from concurrency is

$$[H]T \leq \min\left(B_{\text{peak}}, \frac{S \cdot W \cdot s \cdot f}{L}\right), \quad (8.9)$$

where B_{peak} is raw interface bandwidth (HBM/GDDR). Combine this with arithmetic intensity I (flop/byte) in a practical roofline:

$$[H]P_{\text{achieved}} = \min(P_{\text{peak}}, I \cdot T). \quad (8.10)$$

Implications:

- For a fixed B_{peak} , increasing W (more warps, more thread blocks per SM, or explicit asynchronous memory ops) raises T until the bandwidth ceiling. If $S \cdot W \cdot s \cdot f / L \ll B_{\text{peak}}$, latency dominates and adding register capacity, reducing dependencies, or increasing concurrent blocks yields near-linear benefit.
- If $S \cdot W \cdot s \cdot f / L \gg B_{\text{peak}}$, the kernel is bandwidth-limited; optimizations should focus on reducing bytes transferred (compression, caching, tiling), not on adding concurrency.

A small microbenchmark helps differentiate these regimes by varying concurrency and measuring achieved throughput. The following CUDA kernel alternates dependent single loads and independent vector loads to show latency hiding effects versus aggregate bandwidth:

Listing 8.4: Microbenchmark toggling dependent vs independent loads to measure latency hiding.

```
extern "C" __global__ void mem_bench(char *data, size_t N, int iterations, int
    dep) {
    // dep == 1: dependent linked loads (latency-limited)
    // dep == 0: independent loads (bandwidth-favorable)
    size_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    size_t stride = blockDim.x * gridDim.x;
    unsigned long long acc = 0;

    for (int it = 0; it < iterations; ++it) {
        size_t i = (idx + it * stride) % N;
        char val = data[i];
        if (dep)
            idx = (idx + val) % N; // dependent chain
        acc += val;
    }

    // prevent compiler optimization
    if (idx == 0) data[0] = acc & 0xFF;
}
```

8.2.3 Roofline analysis

Following the discussion of latency versus bandwidth and the identification of memory-bound versus compute-bound behavior, roofline analysis provides a compact, quantitative view that links arithmetic intensity to achievable performance on a given GPU. It clarifies whether a kernel sits under the memory- or compute- ceiling, and helps prioritize optimizations such as increasing data reuse, changing precision to use tensor cores, or improving memory coalescing.

The roofline model defines arithmetic intensity (AI) as the ratio of useful arithmetic work to bytes transferred from main memory:

$$[H]AI = \frac{\text{FLOPs}}{\text{Bytes transferred}}, \quad P(AI) = \min(P_{\text{peak}}, B_{\text{mem}} \cdot AI), \quad (8.11)$$

where P_{peak} is the device peak compute throughput (consider per-precision ceilings: FP32, FP16/tensor cores, INT8), and B_{mem} is sustained memory bandwidth to VRAM after accounting for protocol and interconnect overheads. The ridge point separating memory- and compute-bound regimes is

$$[H]AI_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{mem}}}. \quad (8.12)$$

Operational steps to apply roofline on modern GPUs:

1. Measure kernel FLOPs and bytes moved to/from DRAM (exclude cache hits counted as DRAM if they avoid VRAM trips).
2. Compute AI and then predicted $P(AI)$ using Equation (8.11).
3. Compare measured throughput to prediction; if measured is much lower than $P(AI)$, inspect other bottlenecks (latency, occupancy, memory divergence, instruction mix).

A practical classification script helps automate decisions. The snippet below computes expected performance and classifies bound type for a kernel on a given GPU:

Listing 8.5: Compute roofline prediction and bound classification

```
def roofline_predict(flops, bytes_moved, peak_flops, bandwidth):
    # flops: total kernel floating-point ops; bytes_moved: DRAM bytes touched
    ai = flops / bytes_moved                                # arithmetic intensity (
        FLOP/byte)
    peak = peak_flops                                       # device peak (FLOP/s)
    mem_bound_perf = bandwidth * ai                         # achievable given memory
        bandwidth
    predicted = min(peak, mem_bound_perf)                   # roofline prediction (FLOP
        /s)
    bound = 'compute-bound' if mem_bound_perf >= peak else 'memory-bound'
    return {'AI': ai, 'predicted_Flops_per_s': predicted, 'bound': bound}
# Example: 1e12 FLOPs, 5e10 bytes, 20e12 FLOP/s peak, 1000e9 B/s bandwidth
```

Key GPU-specific considerations:

- Use separate P_{peak} values for tensor cores and scalar ALUs; mixed-precision kernels may move onto higher ceilings.
- Account for L1/L2 hit rates: reduce bytes_moved when data reuse keeps accesses in caches or shared memory.
- For texture-heavy graphics workloads, treat TMU and ROP throughput and cache misses as additional bandwidth ceilings.

Design implications and trade-offs:

- If $AI < AI_{\text{ridge}}$, prioritize reducing DRAM traffic via tiling, shared-memory reuse, or compression; these give the largest throughput gains.
- If $AI > AI_{\text{ridge}}$, focus on compute throughput: enable tensor cores, increase occupancy, and reduce instruction overhead.
- Trading precision (FP32→FP16/INT8) can move a kernel to a higher compute ceiling but may require increased numeric stabilization and data conversion costs.
- Roofline predictions assume steady-state streaming; real kernels with divergence, atomics, or synchronization may perform below roofline and need targeted profiling.

8.2.4 Profiling methodology

The roofline framing and the latency-versus-bandwidth discussion inform the profiler's focus: measure arithmetic intensity, realized bandwidth, and latency-induced stalls to locate the true bottleneck. Profiling methodology turns those theoretical constructs into repeatable measurements that map SIMT behavior, SM/CU utilization, and memory subsystem performance.

Problem: accurately attribute long kernel runtime to compute, memory, or synchronization causes when many subsystems interact. Analysis begins by decomposing observed wall-clock time T_{total} into dominant contributors and comparing against machine limits. A useful operational model is

$$[H]\text{Attainable FLOPS} = \min(\text{Peak FLOPS}, BW_{\text{eff}} \cdot AI), \quad (8.13)$$

where arithmetic intensity $AI = \frac{\text{FLOPs}}{\text{Bytes}}$ and BW_{eff} is effective sustained bandwidth.

Implementation: follow a disciplined five-step procedure.

1. Establish a deterministic baseline: single-kernel, synthetic inputs, fixed power/DVFS state.
2. Collect a minimal counter set guided by roofline: executed FLOPs, SM occupancy, warp stalls (memory, execution), DRAM bytes, L1/L2 miss rates, and synchronization events. Use vendor tools: `ncu` (Nsight Compute) or CUPTI on NVIDIA, and `rocp/rocTracer` on AMD.
3. Microbenchmark critical paths to get peak achievable numbers: pure-FMA loops for peak TFLOPS, streaming copy for sustained BW, and small-kernel latency probes for L1/L2.
4. Correlate counters across time slices to expose phase behavior and transient bottlenecks using sampling-based traces.
5. Validate hypotheses via targeted changes: increase block size, enable shared-memory tiling, or force different memory access patterns.

Practical profiling script (invokes Nsight Compute CLI; adjust metric list for target GPU):

Listing 8.6: Run Nsight Compute metrics for a kernel and parse CSV

```
import subprocess, csv
# run ncu to collect metrics; replace 'my_app' and kernel selector as needed
subprocess.run(["ncu", "--target-processes", "all", "--metrics",
```

```

        "sm__sass_thread_inst_executed_op_fma_sum,dram__bytes_read.sum
        ",
        "--csv","--export","report.csv","./my_app"]])
# parse CSV to extract metrics (very small parser)
with open("report.csv") as f:
    for row in csv.DictReader(f):
        # compute simple ratios; fields vary by GPU/tool
        insts = float(row.get("sm__sass_thread_inst_executed_op_fma_sum",0))
        bytes_read = float(row.get("dram__bytes_read.sum",1))
        ai = insts/bytes_read
        print(f"AI={ai:.3f}, FMA={insts:.0f}, DRAM_bytes={bytes_read:.0f}")

```

Analysis patterns to recognize:

- High DRAM throughput near channel limits plus low achieved FLOPS indicates memory-bound kernels.
- Low warp occupancy with many execution stalls suggests instruction-level or register-pressure bottlenecks.
- High L2 miss with moderate DRAM indicates cache size or access-pattern issues.

Design implications and trade-offs:

- Increasing shared-memory tiling improves AI but raises register pressure, possibly reducing occupancy.
- Enlarging caches reduces DRAM load but increases area and latency; trade cache size versus L1 latency.
- More SMs or tensor cores raise peak FLOPS, but NoC and memory bandwidth must scale to avoid new bottlenecks.
- Prefetching and asynchronous copies reduce latency but increase bandwidth demand and complexity.

8.3 Performance Counters

8.3.1 Counter and sampler architecture

The bottleneck-identification techniques just discussed rely on accurate, low-overhead telemetry; the counter and sampler architecture described here provides that telemetry at SM/CU granularity without perturbing SIMT execution. This subsection details problem statement, microarchitectural analysis, a synthesizable implementation pattern, and measurable trade-offs.

Problem: capture event rates across many SM/CU resources (e.g., ALU issue, TMU requests, ROP writes, tensor-core MACs) while supporting:

- high event cardinality (>64 event types),
- many simultaneous counters per SM,

- low area and power,
- read/clear atomicity and overflow detection,
- multiplexing when physical counters < event types.

Analysis: hardware counters must be edge- or level-sensitive and support configurable event selection per physical counter via a small event-mux. Samplers periodically snapshot counters into a low-bandwidth readout path to avoid stalling the on-chip network; the snapshot interval T_s trades temporal resolution for readout overhead. With clock frequency f_{clk} and desired sampling frequency f_s , choose integer downsample factor $N = \lfloor f_{\text{clk}}/f_s \rfloor$. Thus

$$[H]N = \left\lfloor \frac{f_{\text{clk}}}{f_s} \right\rfloor, \quad T_s = \frac{N}{f_{\text{clk}}}. \quad (8.14)$$

Atomic read-clear semantics are implemented by latching counter value to a shadow register before clearing the live counter to avoid losing events that occur during read.

Implementation: the following synthesizable Verilog implements a compact per-SM counter bank with event multiplexing, periodic sampler, snapshot handshake, and overflow bits. It targets production use in an SM PMU block; bus and privilege checks are left to top-level integration.

Listing 8.7: SM counter and sampler module (synthesizable).

```

module pmu_counter_sampler #(
  parameter NUM_EVENTS = 64,
  parameter NUM_CNTRS   = 8,
  parameter WIDTH      = 48
) (
  input  wire          clk,
  input  wire          rstn,
  input  wire [NUM_EVENTS-1:0] event_bus, // one-hot event signals
  // config interface (simple register write)
  input  wire          cfg_we,
  input  wire [7:0]      cfg_addr, // addr space: 0..NUM_CNTRS-1
      => event_sel, 128 => sample_period
  input  wire [WIDTH-1:0] cfg_wdata,
  // sampler output
  output reg          sample_valid,
  output reg [NUM_CNTRS*WIDTH-1:0] sample_data,
  input  wire          sample_ack
);

  // per-counter event select (encoded index into event_bus)
  reg [$clog2(NUM_EVENTS)-1:0] evt_sel [0:NUM_CNTRS-1];
  reg [WIDTH-1:0] counter [0:NUM_CNTRS-1];
  reg [NUM_CNTRS-1:0] overflow;

  // sampling period register (cycles)
  reg [31:0] sample_period;

```

```

reg [31:0] sample_cnt;

integer i;

// config writes
always @(posedge clk) begin
    if (!rstn) begin
        sample_period <= 32'd1000;
        for (i = 0; i < NUM_CNTRS; i = i + 1) begin
            evt_sel[i] <= 0;
            counter[i] <= 0;
            overflow[i] <= 0;
        end
        sample_cnt <= 0;
        sample_valid <= 0;
    end else begin
        if (cfg_we) begin
            if (cfg_addr < NUM_CNTRS)
                evt_sel[cfg_addr] <= cfg_wdata[$clog2(NUM_EVENTS)-1:0];
            else if (cfg_addr == 8'd128)
                sample_period <= cfg_wdata[31:0];
        end

        // increment counters on active events
        for (i = 0; i < NUM_CNTRS; i = i + 1) begin
            if (event_bus[evt_sel[i]]) begin
                if (counter[i] == {WIDTH{1'b1}})
                    overflow[i] <= 1;
                else
                    counter[i] <= counter[i] + 1'b1;
            end
        end

        // sampling interval
        if (sample_cnt >= sample_period) begin
            sample_cnt <= 0;
            sample_valid <= 1;
            for (i = 0; i < NUM_CNTRS; i = i + 1)
                sample_data[i*WIDTH +: WIDTH] <= counter[i];
        end else begin
            sample_cnt <= sample_cnt + 1;
            sample_valid <= 0;
        end

        // clear on sample acknowledgment
        if (sample_ack)
            for (i = 0; i < NUM_CNTRS; i = i + 1)

```

```

        counter[i] <= 0;
    end
end

endmodule

```

8.3.2 Multiplexing techniques

Building on the sampler and counter buffering described previously, we now examine practical multiplexing strategies that let a limited set of hardware counters measure a much larger set of SM-, TMU-, ROP-, tensor-core and memory events without overwhelming area or crossbar bandwidth.

Multiplexing problem: modern GPUs expose hundreds of distinct events per SM but cannot afford one full-width physical counter per event. The goal is to trade temporal resolution and software complexity for hardware resource efficiency while preserving statistical usefulness for bottleneck analysis. Techniques fall into three operational categories:

- Time-division (round-robin) multiplexing: cycle a physical counter through selected event lines, allocating a fixed timeslot per event. This minimizes logic by reusing one counter but introduces quantization error and potential aliasing for bursty events.
- Event-triggered sampling: latch counters only when a trigger condition occurs (e.g., warp retire rate drop). This reduces average overhead but needs programmable triggers and edge-detection hardware.
- Hierarchical aggregation: local per-SM short-term counters are multiplexed into an SM aggregator which periodically sends compact summaries to a global PMU fabric, reducing cross-chip bandwidth.

Analysis: For round-robin multiplexing with N events and a timeslot of T cycles per event at clock frequency f_{clk} , the per-event sampling frequency equals

$$[H]f_{\text{sample}} = \frac{f_{\text{clk}}}{N \cdot T}. \quad (8.15)$$

Lowering T increases temporal resolution but increases overhead on the crossbar and counter writebacks. Bursty SIMT behavior (e.g., divergence spikes) requires T small enough to avoid undercounting short-lived phenomena.

Implementation example: a synthesizable Verilog PMU multiplexer that performs configurable round-robin sampling and writes per-event shadow counters. It supports a software-readable shadow array and a programmable timeslot width.

Listing 8.8: PMU round-robin multiplexer — synthesizable.

```

module pmu_mux #(
    parameter NUM_EVENTS = 32,
    parameter CNT_WIDTH  = 48,
    parameter T_BITS     = 8           // timeslot size = 2^T_BITS cycles
) (
    input wire          clk,

```



```

input wire          rst_n,
input wire [NUM_EVENTS-1:0] event_pulse, // one-cycle pulses per source
input wire [T_BITS-1:0] timeslot_log, // programmable timeslot
    exponent
input wire [$clog2(NUM_EVENTS)-1:0] rd_idx, // read index
output reg [CNT_WIDTH-1:0] rd_data,
output reg          rd_valid
);
// per-event shadow counters (compact storage)
reg [CNT_WIDTH-1:0] counters [0:NUM_EVENTS-1];
reg [$clog2(NUM_EVENTS)-1:0] sel; // current selected event
reg [T_BITS-1:0] timeslot_cnt;

integer i;

// initialize and update counters
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sel <= 0;
        timeslot_cnt <= 0;
        rd_data <= 0;
        rd_valid <= 0;
        for (i = 0; i < NUM_EVENTS; i = i + 1)
            counters[i] <= 0;
    end else begin
        // increment counters on active event pulses
        for (i = 0; i < NUM_EVENTS; i = i + 1)
            if (event_pulse[i])
                counters[i] <= counters[i] + 1'b1;

        // timeslot control
        if (timeslot_cnt == (1 << timeslot_log) - 1) begin
            timeslot_cnt <= 0;
            sel <= (sel == NUM_EVENTS - 1) ? 0 : sel + 1'b1;
            rd_data <= counters[sel];
            rd_valid <= 1;
        end else begin
            timeslot_cnt <= timeslot_cnt + 1'b1;
            rd_valid <= 0;
        end

        // external read access
        if (rd_idx < NUM_EVENTS)
            rd_data <= counters[rd_idx];
    end
end
end

```

endmodule

8.3.3 Key hardware metrics

Building on the sampler architecture and multiplexing strategies discussed earlier, we now map those mechanisms to the concrete metrics you should collect and how each metric ties to a specific bottleneck in an SM/CU. This subsection translates abstract counter design into actionable measures for graphics and ML workloads.

Problem: choose a compact set of counters that reveals whether a kernel is compute-bound, memory-bound, or limited by control flow or resources. Analysis: counters must capture (a) instruction and functional-unit utilization, (b) memory-system behavior across hierarchies, (c) control and warp-level inefficiencies, and (d) thermal/power headroom. Key hardware metrics to expose:

- Compute throughput:
 1. SM active cycles and SM warp-issued (warp_issue_count) — show SIMT issue utilization.
 2. FP32/FP16/INT operations retired per cycle and tensor-core usage — directly measures arithmetic throughput.
 3. ALU/TMU/RT/ROP occupancy breakdown — isolates unit-specific saturation.
- Memory and cache:
 1. DRAM bytes read/write and PCIe transfers.
 2. L1, L2 hit/miss counters and request latencies.
 3. Texture (TMU) and ROP bandwidth counters.
 4. Shared memory bank conflict events.
- Scheduling and control:
 1. Active warps per SM and occupancy (fraction of maximum warps active).
 2. Warp divergence events, reconverge latency.
 3. Stalled cycles by reason: memory, execution, dispatch, or synchronization.
- System-level:
 1. Power draw and thermal throttling samples (NVML/CUPTI).
 2. Context-switch and kernel-launch overhead counters.

Operational relevance before equations: occupancy and roofline arithmetic-intensity thresholds are essential for diagnosing whether adding more compute units or higher bandwidth yields performance.

Let RegsPerSM, SharedPerSM, and MaxBlocksPerSM be hardware limits; for a kernel with RegsPerThread, SharedPerBlock, ThreadsPerBlock, the active blocks per SM is

$$[H]B_{\text{active}} = \min \left(\left\lfloor \frac{\text{RegsPerSM}}{\text{RegsPerThread} \cdot \text{ThreadsPerBlock}} \right\rfloor, \left\lfloor \frac{\text{SharedPerSM}}{\text{SharedPerBlock}} \right\rfloor, \text{MaxBlocksPerSM} \right). \quad (8.16)$$

Occupancy is then

$$[H]\text{Occupancy} = \frac{B_{\text{active}} \cdot \text{WarpsPerBlock}}{\text{MaxWarpsPerSM}}. \quad (8.17)$$

Roofline tie-in: sustained GFLOPS = $\min(\text{peak_GFLOPS}, \text{BW_bytes/s} \cdot \text{arithmetic_intensity})$. Use counters to compute $\text{arithmetic_intensity} = \text{FLOPs}/\text{DRAM_bytes}$.

Implementation: sample both high-rate streaming counters (SM cycles, issued warps) and multiplexed detailed counters (L1 misses, tensor-core events). Use coarse sampling for power/thermal and high-rate ring buffers for per-kernel counters. Example: quick NVML poll for SM/GPU utilization and memory throughput.

Listing 8.9: Poll lightweight GPU metrics via NVML

```
import pynvml, time
pynvml.nvmlInit()
h = pynvml.nvmlDeviceGetHandleByIndex(0)
for _ in range(10):
    util = pynvml.nvmlDeviceGetUtilizationRates(h)      # GPU and memory util
    mem = pynvml.nvmlDeviceGetMemoryInfo(h)             # total/used/free
    power = pynvml.nvmlDeviceGetPowerUsage(h)           # milliwatts
    print(util.gpu, util.memory, mem.used, power)       # simple inline comments
    time.sleep(0.2)
pynvml.nvmlShutdown()
```

Implications and trade-offs:

- Expose enough counters to map the roofline corner; multiplexing reduces overhead but can miss transient stalls.
- Instrumentation frequency trades visibility for perturbation: high-rate sampling reveals microbursts but adds overhead.
- Design PMU with dedicated low-cost counters for frequent metrics (SM cycles, L1 misses, tensor-core ops) and multiplexed slots for rare diagnostics to minimize performance impact.

8.3.4 PMU software interfaces

Building on the previous discussion of which hardware metrics matter and how counters are multiplexed across limited physical registers, we now address the software-facing problem: how the Performance Monitoring Unit (PMU) exposes those counters to profilers and the run-time so engineers can collect accurate, low-overhead measurements from SMs, tensor cores, TMUs, ROPs and memory controllers.

Problem — requirements and constraints:

- Provide both coarse counts (accumulators) and sampled streams with timestamps for event correlation.
- Minimize interference: reads and context switches must not perturb SM scheduling or memory timing materially.

- Support event encoding, grouping, simultaneous enable masks, overflow handling and multiplexing transparency to users.

Analysis — key elements of a robust PMU interface:

1. Event model and encoding: software needs a compact encoding for vendor events (e.g., L2_miss, SM_FP32_cycles, TMU_tex_fetch). The kernel/driver translates the encoding to per-SM select bits and counter program words.
2. Group semantics and atomic reads: related events (latency-critical counters from the same SM stage) must be placed in a group so the PMU can snapshot atomically across counters to avoid skew from interleaved SM activity.
3. Multiplexing and scaling: when there are more logical events than physical counters, the driver time-multiplexes events. Measured counts must be scaled to estimate full-interval values:

$$\text{count_scaled} = \text{count_raw} \times \frac{T_{\text{total}}}{T_{\text{enabled}}} \quad (8.18)$$

where T_{enabled} is the wall time the event was active on a physical counter and T_{total} is the profiling interval.

4. Sampling vs counting: sample-based interfaces capture stack, PC or warp id at overflow; accumulator reads return totals. Sampling requires precise timestamps (e.g., cycle counter or monotonic clock) to correlate with SM scheduler events.
5. Context and process isolation: per-context counters must be saved/restored on kernel GPU context switch or implemented per-VA space to avoid attribution errors between kernels or streams.
6. Low-overhead read paths: support mmap'ed ring buffers for sample streams and a fast ioctl or read for accumulators to avoid syscall overhead in tight loops.

Implementation — an example using Linux perf-style semantics (illustrative):

Listing 8.10: User-space enable/read loop using a perf-style PMU

```
#include <sys/syscall.h>
#include <linux/perf_event.h>
#include <unistd.h>
#include <stdint.h>
#include <stdio.h>

// open perf-like PMU (vendor encoding done in driver) and read counter.
int open_pmu(int type, int config) {
    struct perf_event_attr attr = {0};
    attr.type = type; attr.size = sizeof(attr);
    attr.config = config; // vendor event id
    attr.disabled = 1; attr.exclude_kernel = 1;
    return syscall(__NR_perf_event_open, &attr, 0, -1, -1, 0);
}

int main() {
```

```

int fd = open_pmu(/*PMU_TYPE*/0x4, /*EVENT_ID*/0x123);
// enable, run workload, then read 64-bit counter
ioctl(fd, PERF_EVENT_IOC_ENABLE, 0);
// run GPU kernel (submit commands)...
uint64_t val; read(fd, &val, sizeof(val)); // raw count
ioctl(fd, PERF_EVENT_IOC_DISABLE, 0);
printf("raw count = %llu\n", (unsigned long long)val);
close(fd);
}

```

Operational notes:

- Driver must expose `read_format` flags to include enabled-time and `time_enabled` to apply (8.18) reliably.
- For SM-local metrics, the PMU should allow affinity to a specific SM/CU to avoid averaging across SMs.

Design implications and trade-offs:

- Exposing many simultaneous events requires larger physical counter arrays or more aggressive multiplexing; the former increases area and power, the latter increases statistical error and requires careful scaling.
- Atomic group reads reduce skew but add hardware complexity; sampling on overflow gives fine-grain attribution at cost of interrupt and buffer management.
- Per-SM counters with low-cost context save/restore give the most precise kernel-level attribution but incur state-storage overhead per context and longer context-switch latency.

8.4 Workload Characterization

8.4.1 Instruction mix and balance

Building on prior discussion of bottleneck identification and low-level counters, we next quantify how instruction composition shapes whether an SM is compute-bound, memory-bound, or latency-bound. This subsection formalizes instruction-mix metrics, shows how to measure them from PMU samples, and derives actionable thresholds for optimization.

Instruction mix is the normalized frequency vector of dynamic instructions executed on an SM or warp. Representative categories are floating-point (FP) arithmetic, integer arithmetic, memory ops (loads/stores), special-function and transcendental ops, texture/TMU ops, and tensor-core/MMA ops. Let the raw counts be collected into vector $c = [c_{\text{FP}}, c_{\text{INT}}, c_{\text{MEM}}, c_{\text{SFU}}, c_{\text{TMU}}, c_{\text{MMA}}]$. The normalized mix is

$$[H]_m = \frac{c}{\sum_i c_i}, \quad (8.19)$$

which allows comparison between kernels of different execution lengths. Arithmetic intensity (AI) remains useful: $AI = \frac{\text{FLOPs}}{\text{bytes transferred}}$, appearing in the roofline model. The instruction-balance metric useful for microarchitectural matching is the L1 distance to a target mix m^* that

represents ideal hardware-balanced utilization:

$$[H]\Delta = \|m - m^*\|_1 = \sum_i |m_i - m_i^*|. \quad (8.20)$$

A high Δ indicates overprovisioned units sitting idle while another unit throttles throughput.

Analysis workflow:

1. Collect per-SM counters for the categories above across representative kernels and phases.
2. Compute m per sampling window to capture phase behavior.
3. Map m to likely bottlenecks using simple rules:
 - m_{MEM} large and AI small $\rightarrow$ memory-bound.
 - m_{MMA} dominant with low m_{MEM} $\rightarrow$ tensor-core compute-bound.
 - High m_{SFU} or m_{TMU} suggests specialized-unit saturation.

Implementation example: extracting mix from a perf log and computing imbalance against a hardware profile m^* . The code assumes counter export as a JSON mapping.

Listing 8.11: Compute instruction mix and imbalance

```
# Parse counters dict; keys are category names, values are counts.
def compute_mix(counters):
    total = sum(counters.values())
    return {k: v/total for k, v in counters.items()} # normalized mix

# Example hardware target mix for balanced SM usage (engineer-tuned).
target = {'FP':0.35, 'INT':0.15, 'MEM':0.20, 'SFU':0.05, 'TMU':0.10, 'MMA':0.15}

def ll_imbalance(mix, target):
    return sum(abs(mix.get(k,0)-target.get(k,0)) for k in target)

# Example usage
counters = {'FP':1_200_000, 'INT':300_000, 'MEM':700_000, 'SFU':100_000, 'TMU':200_000, 'MMA':500_000}
mix = compute_mix(counters) # normalized instruction mix
imbalance = ll_imbalance(mix, target) # scalar metric to prioritize tuning
```

Implications and trade-offs:

- If Δ is high and m_{MEM} dominates, invest in coalescing, prefetching, or increased occupancy to hide DRAM latency. These reduce bandwidth pressure but may increase register pressure.
- If m_{MMA} dominates, optimize data layout and tiling to saturate tensor cores; this can shift pressure to memory and caches.
- Balancing often requires iterative tuning: changing instruction mix via algorithmic reformulation may expose new bottlenecks, so profile-driven adjustments are essential.
- Use phase-level sampling to avoid misleading averages; small hot phases can dominate latency despite modest overall counts.

8.4.2 Cache and memory patterns

Building on the instruction-mix analysis, the next dominant axis for classifying workloads is how they touch memory: spatial locality, temporal reuse, stride regularity, and working-set size determine whether a kernel is compute- or memory-bound and which cache level will be critical. Below we analyze concrete patterns, show a simple optimization implementation, and state practical design implications.

Problem — observable patterns and why they matter:

- Sequential/coalesced linear scans (good spatial locality) map well to wide DRAM bursts and texture/L1 caching; they maximize effective bandwidth per transaction.
- Strided accesses with stride S that is not a small factor of the cache line size cause low useful bytes per fetch and create cache-line fragmentation.
- Gather/scatter and random accesses destroy prefetch effectiveness and increase L2/TLB pressure.
- Working sets larger than L1 but smaller than L2 favor streaming through L1 bypass or software-managed tiling (shared memory) to avoid repeated DRAM trips.

Analysis — simple performance model and heuristics: Operational relevance is captured by average memory access time (AMAT). For a single access stream:

$$[H]AMAT = T_{\text{hit}} + \text{MissRate} \cdot T_{\text{miss}}, \quad (8.21)$$

where T_{hit} is L1/L2 hit latency and T_{miss} includes DRAM latency plus queuing. A lower AMAT increases achieved bytes/sec; combined with arithmetic intensity $I = \text{FLOPs/Byte}$ it determines compute vs memory bound behavior via the roofline slope BW_{peak} and peak FLOPS.

Implementation — code showing a harmful strided pattern and a tiled (coalesced) fix. The example assumes CUDA-style SIMT; use `threadIdx.x` and `blockDim.x` semantics. Comments are brief inline.

Listing 8.12: Strided load vs tiled coalesced loads

```
__global__ void strided_add(float *A, float *B, float *C, int N, int stride) {
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    int i = gid * stride;           // bad: non-contiguous per-thread
    addresses
    if (i < N) C[i] = A[i] + B[i];
}

__global__ void tiled_add(float *A, float *B, float *C, int N) {
    extern __shared__ float tile[]; // shared memory tile
    int tid = threadIdx.x, gid = blockIdx.x * blockDim.x + tid;
    int base = blockIdx.x * blockDim.x;
    int idx = base + tid;
    // coalesced load into shared tile (atomic per-thread contiguous load)
    if (idx < N) tile[tid] = A[idx];
    __syncthreads();
    // local compute using tile reduces DRAM reads
    if (idx < N) C[idx] = tile[tid] + B[idx];
}
```

Implications and trade-offs:

- Hardware: larger cache lines and aggressive prefetchers increase throughput for dense scans but worsen bandwidth waste for sparse patterns; associativity reduces conflict misses but costs die area and power.
- Software: transform strided/gather patterns into contiguous per-warp accesses or apply blocking into shared memory to improve reuse; prefer memory layouts (tiling, swizzling) that align with SM lanes.
- System: TLB pressure from many small pages creates stalls—use large pages for high-throughput kernels but accept higher memory fragmentation risk.
- Performance trade-offs: increasing shared memory usage for tiling raises register/shared memory pressure and can reduce occupancy; pick tile sizes that balance reuse against reduced concurrent warps.

8.4.3 Thread divergence statistics

These divergence statistics follow naturally from the prior examination of cache/memory patterns and instruction-mix balance, since control-flow variability couples to memory access scatter and alters effective instruction throughput. Quantifying divergence converts observed warp masks into engineering metrics that guide scheduling, predication, and compiler transforms.

Problem: a binary conditional across a warp becomes serial when threads disagree, reducing SIMT efficiency. Analysis begins with per-branch outcome distribution measured at program counter (PC) granularity. Let W denote warp width (typically 32). For an independent per-thread branch-taken probability p , the expected number of distinct SIMT execution paths (i.e., serial subgroups) for that binary branch is

$$[H]S_{\text{exp}}(p, W) = 2 - p^W - (1 - p)^W, \quad (8.22)$$

which equals 1 only when all threads agree, and approaches 2 for moderate p in large warps. This closed form quantifies worst-case serialization for 2-way branches; multi-way control increases expected subgroup count according to the occupancy of each outcome.

Operational metrics to collect on silicon or in simulator:

- Per-PC branch frequency and outcome histogram, enabling computation of branch entropy $H = -\sum_k p_k \log_2 p_k$.
- Warp-level active-lane utilization averaged over kernel lifetime, measured as fraction of lanes executing useful instructions.
- Reconvergence distance and stack pressure: average dynamic depth and time-to-reconverge.

Implementation: a lightweight profiler can aggregate warp execution masks into counters per PC. The following Python snippet demonstrates mask-based aggregation and computes S_{exp} for a binary branch observed counts:

Listing 8.13: Compute divergence metrics from warp masks

```

# masks: list of 32-bit ints, one per warp execution at PC (1=taken,0=not)
from collections import Counter
masks = [...] # collected by sampler; each entry is a 32-bit warp mask
cnt = Counter(masks)
total = sum(cnt.values())

# per-PC taken fraction p and expected serial paths S_exp
taken_bits = sum(bin(m).count("1") * c for m, c in cnt.items())
p = taken_bits / (total * 32.0) # average per-thread taken prob
W = 32
S_exp = 2 - p**W - (1-p)**W # eqn (1) applied
# branch entropy per observation (binary)
import math
if p in (0.0,1.0):
    H = 0.0
else:
    H = - (p*math.log2(p) + (1-p)*math.log2(1-p))
print("p=",p,"S_exp=",S_exp,"H=",H)

```

Implications and trade-offs:

1. High S_{exp} or entropy signals that predication or code restructuring is beneficial, but predication increases instruction count and register pressure.
2. Thread reordering or grouping by branch outcome reduces divergence at scheduler cost, and may worsen memory coalescing unless locality-aware.
3. Hardware options include more aggressive reconvergence hardware, compactors that reduce active-lane idle cycles, or finer-grain WARPs (smaller W), each trading area/complexity for throughput gains.

8.4.4 Power behavior profiling

The previous subsections established how thread divergence and cache/memory patterns change execution density and memory pressure, both of which directly affect instantaneous power. Here we quantify and profile those effects to attribute power to functional domains and workload characteristics.

Power behavior profiling problem: measure how SM, memory, TMU, ROP, tensor core, and static/leakage components contribute to instantaneous and average power under representative graphics or ML workloads. Accurate domain attribution enables workload-aware DVFS, fine-grained power gating, and thermal control policies. The operational approach is: collect synchronized power samples and performance counters, build a domain-level model, and validate by predicting power for held-out kernels.

Physically, dynamic power for a switching domain follows

$$[H]P_{\text{dyn}} = C V^2 f \alpha, \quad (8.23)$$

where C is effective capacitance, V supply voltage, f clock frequency, and α the activity factor. For a GPU we decompose total power as

$$[H]P_{\text{tot}}(t) = P_{\text{leak}} + \sum_{d \in D} C_d V_d^2 f_d \alpha_d(t), \quad (8.24)$$

with domains $D = \{\text{SM}, \text{L2}, \text{DRAM}, \text{TMU}, \text{ROP}, \text{Tensor}\}$. We map measurable counters into activity estimates using a linear model:

$$[H]P_{\text{meas}}(t) \approx \beta_0 + \sum_{i=1}^n \beta_i x_i(t), \quad (8.25)$$

where x_i are normalized counters such as `gpu_util`, `mem_bw_util`, `cache-miss-rate`, and instruction-type mixes. Fit coefficients β using ordinary least squares:

$$[H]\hat{\beta} = (X^\top X)^{-1} X^\top y. \quad (8.26)$$

Practical profiling methodology:

1. Instrumentation: sample on-die power rails plus wall power if available; sample SM and memory counters at the highest feasible resolution (tens of milliseconds).
2. Workload design: generate microbenchmarks that isolate compute-bound, memory-bound, TMU-heavy, and tensor-core-heavy behaviors.
3. Synchronization: align timestamps and resample to a common interval before model fitting.
4. Validation: predict power for unseen kernels and measure residuals to guide model refinement.

Example lightweight sampler using NVIDIA NVML for power and utilization (Python). Comments indicate where to extend with custom perf counters or CUPTI integration.

Listing 8.14: NVML-based sampler for power and util; extend to collect counters via CUPTI

```
import time
from pynvml import nvmlInit, nvmlDeviceGetHandleByIndex, \
    nvmlDeviceGetPowerUsage, nvmlDeviceGetUtilizationRates

nvmlInit()
h = nvmlDeviceGetHandleByIndex(0)  # single-GPU test

def sample(duration_s=10, interval_s=0.05):
    t0 = time.time()
    rows=[]
    while time.time() - t0 < duration_s:
        p = nvmlDeviceGetPowerUsage(h) / 1000.0  # mW->W
        u = nvmlDeviceGetUtilizationRates(h).gpu  # percent
        m = nvmlDeviceGetUtilizationRates(h).memory  # percent
        rows.append((time.time(), p, u, m))
```

```

        time.sleep(interval_s)
    return rows

data = sample(30, 0.02)  # 30s trace at 20ms intervals
# serialize data for offline regression and counter fusion

```

Design implications and trade-offs:

- Simple linear models are interpretable but can misattribute non-linear frequency or temperature effects; include $V^2 f$ terms when varying DVFS.
- High-resolution sampling increases attribution accuracy but raises measurement overhead and synchronization complexity.
- Per-domain sensors and per-SM counters enable precise gating decisions, but increase area and design complexity.

8.5 Optimization Techniques

8.5.1 Occupancy tuning

Building on bottleneck identification and workload characterization, occupancy tuning translates resource budgets into how many warps can be resident on an SM to hide latency. The goal is to balance register pressure, shared memory use, and block sizing so the scheduler has enough ready warps to tolerate memory and execution latency without wasting SM resources.

Problem: given per-SM resources (register file size R_{SM} , shared memory S_{SM} , maximum threads T_{SM} , maximum concurrent blocks B_{SM}) and a kernel's per-thread and per-block usage (registers r_t , shared s_b , threads per block t_b), compute how many warps can be resident and the resulting occupancy.

Analysis. The number of resident blocks limited by each resource is

$$[H]B_{\text{regs}} = \left\lfloor \frac{R_{SM}}{r_t \cdot t_b} \right\rfloor, \quad B_{\text{sh}} = \left\lfloor \frac{S_{SM}}{s_b} \right\rfloor, \quad B_{\text{threads}} = \left\lfloor \frac{T_{SM}}{t_b} \right\rfloor. \quad (8.27)$$

The actual resident blocks per SM are

$$[H]B_{\text{res}} = \min\{B_{\text{regs}}, B_{\text{sh}}, B_{\text{threads}}, B_{SM}\}. \quad (8.28)$$

Resident warps equal $W_{\text{res}} = B_{\text{res}} \cdot (t_b/W_{\text{size}})$, and occupancy

$$[H]\text{occupancy} = \frac{W_{\text{res}}}{W_{\text{max}}} \quad \text{where } W_{\text{max}} = \frac{T_{SM}}{W_{\text{size}}}. \quad (8.29)$$

Implementation. Use the following utility to compute occupancy from measured or spec values; it is valuable during kernel tuning and autotuning loops to iterate over `t_b` and register usage (via compiler flags or `__launch_bounds__`).

Listing 8.15: Compute SM occupancy given resource parameters

```

def compute_occupancy(R_SM, S_SM, T_SM, B_SM, warp_size,
                      regs_per_thread, shared_per_block, threads_per_block):

```

```

# blocks constrained by each resource
b_regs = R_SM // (regs_per_thread * threads_per_block) # register limit
b_sh   = S_SM // shared_per_block                      # shared mem limit
b_thr  = T_SM // threads_per_block                     # thread limit
b_res  = min(b_regs, b_sh, b_thr, B_SM)                # resident blocks
warps_res = b_res * (threads_per_block // warp_size)  # resident warps
warps_max = T_SM // warp_size
return warps_res / warps_max
# Example: R_SM=65536 regs, S_SM=98304 bytes, T_SM=2048 threads, B_SM=16
# blocks, warp_size=32

```

Operational tips:

- If occupancy is low due to high `regs_per_thread`, try compiler flag reductions or rearranging code to enable register reuse.
- If limited by shared memory, restructure tiling to use fewer bytes per block or increase block parallelism with smaller tiles.
- High occupancy does not guarantee peak throughput; compute-bound kernels may be ALU-limited, and excessive register spilling reduces effective occupancy.

Design implications and trade-offs:

- Increasing SM resources (registers or shared) raises achievable occupancy but increases area and power.
- For memory-bound kernels, targeting occupancy above the latency-hiding threshold (often 50–75%) yields diminishing returns.
- Autotuning block sizes to the occupancy sweet spot reduces stalls while avoiding register spilling, improving overall throughput and energy efficiency.

8.5.2 Coalesced memory access

As discussed in the previous subsection on occupancy tuning, maximizing active warps on an SM reduces latency exposure but cannot recover bandwidth lost to inefficient memory transactions; coalesced access is the complementary optimization that converts many small requests into minimal, wide transfers.

Problem: SIMT warps issue memory requests per lane but the memory system services aligned segments, so misaligned or strided patterns inflate the number of DRAM/L1 transactions and waste cache bandwidth. Analysis must quantify transactions per warp so architects can compare bandwidth cost across access patterns.

Let W be warp width (typically 32), e the element size in bytes, and S the memory segment/cache-line size (commonly 128). For a contiguous, aligned access the minimum transactions per warp is

$$[H]T_{\min} = \left\lceil \frac{W \cdot e}{S} \right\rceil. \quad (8.30)$$

If the base address has offset o into a segment, the effective numerator becomes $W \cdot e + o$, increasing T . Example: $W = 32$, $e = 4$ (float) and $S = 128$ gives $T_{\min} = \lceil 128/128 \rceil = 1$. A stride of two doubles the span, yielding $T = 2$, halving effective bandwidth.

Practical implementation strategies:

- Align per-warp base addresses to S to avoid cross-segment accesses. Add padding at allocation time if necessary.
- Make each thread access consecutive elements so threads in a warp form a contiguous vector; use vectorized types (e.g., float4) when memory layout permits.
- Use shared memory as a reformatting buffer: coalesce many scattered loads into contiguous shared-memory writes, then service the compute with bank-aware shared reads.
- Minimize gather/scatter patterns in performance-critical kernels; replace with index transformations or prefetch + re-layout.

Concrete example (CUDA): the kernel below shows a coalesced load into shared memory followed by per-thread work. Comments are brief inline.

Listing 8.16: Coalesced load into shared memory to compact a strided global layout

```
extern "C" __global__ void compact_and_compute(float* dst, const float* src,
    int N) {
    __shared__ float sdata[256];           // shared buffer (size tuned)
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int lane = threadIdx.x;                // within-warp lane
    int W = 32;
    int base = (blockIdx.x * blockDim.x) + (lane / W) * W; // align per-warp
    base
    // coalesced load: each contiguous thread reads contiguous src element
    if (base + lane < N) sdata[lane] = src[base + lane]; // coalesced global
    load
    __syncthreads();
    // compute using shared memory to benefit from compact layout
    if (tid < N) dst[tid] = sdata[lane] * 2.0f; // simple example compute
}
```

Architectural and performance implications:

- Coalescing reduces DRAM transactions and L1/L2 pressure, improving sustained bandwidth and energy per useful byte.
- Reformatting with shared memory trades increased register/shared usage and possible occupancy reduction against fewer memory transactions; choose the balance per roofline analysis.
- Vectorized types and alignment improve throughput but can increase working set size and complicate memory allocator design.
- For irregular workloads, hardware gather engines or wider memory request aggregation reduce software complexity at area and logic cost.

Trade-offs must be evaluated per workload: memory-bound kernels often benefit most from coalescing even at the cost of slightly lower occupancy, while compute-bound kernels may tolerate less aggressive reformatting.

8.5.3 Shared memory utilization

Building on coalesced global accesses and occupancy tuning, shared memory is the on-SM scratchpad that converts DRAM bandwidth pressure into on-chip data reuse; tuning its size and access pattern directly affects latency hiding and SM throughput. The analysis below explains banked behavior, occupancy interactions, and practical coding patterns for ML and graphics kernels (e.g., tiled GEMM, convolution).

GPUs expose a banked shared memory (typical $B = 32$ banks, `bank_width` = 4 or 8 bytes). When a warp issues a 32-thread access, parallelism is limited by distinct bank addresses touched. If multiple threads target the same bank but different addresses, accesses serialize; identical addresses can often be broadcast. Let S be shared bytes per block, SM_S the total per-SM shared memory, W_b the warp size (usually 32), and W_p the warps per block. The device-limited blocks per SM due to shared memory are

$$[H]N_{\text{blocks}}^{(SM)} = \left\lfloor \frac{SM_S}{S} \right\rfloor. \quad (8.31)$$

Effective occupancy (fraction of max warps active) can be approximated by

$$[H]\text{Occ} = \frac{\min(N_{\text{blocks}}^{(SM)} \cdot W_p, W_{\text{max}}^{(SM)})}{W_{\text{max}}^{(SM)}}, \quad (8.32)$$

with $W_{\text{max}}^{(SM)}$ the hardware warp capacity. Increasing s raises reuse but reduces $N_{\text{blocks}}^{(SM)}$, lowering Occ and diminishing the GPU's latency hiding.

Bank conflict cost: if a warp touches k distinct banks concurrently, then cycle-serialized factor $\approx \lceil W_b/k \rceil$ when each bank can service a single request per cycle. Thus maximize distinct bank indices across threads to approach one-cycle service.

Implementation pattern: use tiling + padding (to avoid strided bank collisions) and cooperative loading with `__syncthreads()`. Example tiled GEMM kernel skeleton:

Listing 8.17: Tiled GEMM using shared memory and padding

```
__global__ void tiled_gemm(const float *A, const float *B, float *C,
                          int N) {
    __shared__ float As[BLOCK][BLOCK+1]; // +1 avoids bank conflicts
    __shared__ float Bs[BLOCK][BLOCK+1]; // padding trick
    int tx = threadIdx.x, ty = threadIdx.y;
    int row = blockIdx.y*BLOCK + ty;
    int col = blockIdx.x*BLOCK + tx;
    float sum = 0.0f;
    for (int m = 0; m < N; m += BLOCK) {
        // cooperative load into shared (coalesced global loads)
        As[ty][tx] = A[row*N + m + tx]; // aligned global read
        Bs[ty][tx] = B[(m+ty)*N + col];
        __syncthreads(); // ensure data ready
        #pragma unroll
        for (int k = 0; k < BLOCK; ++k) sum += As[ty][k]*Bs[k][tx];
        __syncthreads();
    }
    C[row*N + col] = sum; // write-back
}
```

Practical tuning checklist:

- Use small padding (e.g., +1 column) to break regular strides that map to same banks.
- Balance s to avoid reducing Occ via (1) and (2).
- Favor cooperative loads with coalesced global reads and then reuse in shared memory.
- Measure bank conflict counters and occupancy together: a small drop in Occ can be outweighed by higher arithmetic intensity from reuse.

Design implications and trade-offs:

- Larger shared allocations increase arithmetic intensity but reduce occupancy and potential latency hiding.
- Bank conflicts cause serialization that can surpass global memory latency penalties; padding and address remap are low-cost mitigations.
- For ML tensor cores, tile sizes should align to both shared-bank geometry and tensor-core tile footprints to maximize utilization.

8.5.4 Instruction scheduling

Building on shared-memory tiling and coalesced-access patterns, instruction scheduling translates those memory-level gains into sustained arithmetic throughput by selecting ready warps to maximize SM utilization while avoiding pipeline hazards.

Problem: under SIMT, a scheduler must hide long memory and functional-unit latencies across many warps while respecting dependencies and resource limits (registers, shared memory). Analysis begins with the latency-hiding constraint. If a memory or functional latency is L cycles and the scheduler can issue a warp every s cycles, the minimum number of active warps required to cover that latency is

$$[H]W_{\text{req}} = \left\lceil \frac{L}{s} \right\rceil. \quad (8.33)$$

For example, $L = 400$ cycles (DRAM miss) and $s = 1$ cycle implies $W_{\text{req}} = 400$, which often exceeds per-SM occupancy limits imposed by register or shared-memory allocation.

Key scheduling considerations:

- Dependency tracking: use a scoreboard to mark operand readiness and prevent RAW/WAW hazards; scoreboard logic must scale with warp count and issue width.
- Latency-class scheduling: separate short-latency (ALU) and long-latency (memory/RT/tensor) warp lists to avoid head-of-line blocking by warps waiting on long operations.
- Issue policies: round-robin gives fairness; greedy oldest-first reduces stall time for long-waiting warps; two-level schedulers combine both for throughput and latency control.
- Functional-unit matching: scheduler must be topology-aware (ALU, tensor core, TMU, RT core, ROP) and map warp instructions to available units without creating structural hazards.

- Divergence and reconvergence: active-mask-aware selection avoids issuing threads that will immediately mask out, improving useful-issue efficiency.

Implementation: a practical per-SM scheduler design includes

1. Ready queues partitioned by latency class and unit type.
2. A scoreboard array indexed by `warp_id` for operand-ready bits.
3. An age-register to implement oldest-first within each queue.
4. A small CAM or bitset to track outstanding long-latency misses to deprioritize warps generating many concurrent cache misses.

A stripped-down simulator that demonstrates round-robin ready selection and latency modeling:

Listing 8.18: Minimal warp-scheduler simulator (latency classes).

```
# Simple warp scheduler: ready_queue holds (warp_id, instr_latency)
ready_queue = [(0, 1), (1, 400), (2, 2)] # (id, latency)
cycle = 0
while ready_queue:
    # pick first ready warp (round-robin) -- real design uses age + priorities
    warp_id, lat = ready_queue.pop(0)
    # issue instruction: if long latency, reinsert after latency cycles
    if lat > 16:
        # simulate long-latency stall before next ready instruction
        ready_queue.append((warp_id, 1)) # next instruction will be short
        cycle += 1 # issue cost
    else:
        cycle += 1 # complete short op quickly
    # basic bookkeeping -- real hardware tracks many more states
print("Simulated cycles:", cycle) # // small model output
```

Implications and trade-offs:

- Increasing active warps raises throughput until resource constraints (registers/shared memory) reduce per-warp resources and ILP, producing diminishing returns.
- Aggressive oldest-first policies lower tail latency but can reduce fairness and increase cache thrashing.
- Hardware complexity (multi-queue schedulers, larger scoreboards, CAMs) trades silicon area and power for reduced stall cycles; designers must balance scheduler sophistication against area/power budgets.
- For ML workloads, exposing tensor-core availability and prioritizing compute-bound warps improves utilization; for graphics, prioritizing RT-core and ROP-bound warps reduces frame variance.

8.6 Section 6: Power and Thermal Management

8.6.1 Dynamic power reduction

Following workload characterization and roofline-based bottlenecking, reducing dynamic power targets the activity and switching that profiling exposes, and complements occupancy and memory optimizations previously discussed. The following develops the problem, analyzes cost/benefit quantitatively, and gives a small synthesizable control module example for runtime clock enable gating.

Dynamic switching dominates short-term power in modern GPUs. Operationally, switching power is proportional to switching activity, capacitance, voltage squared, and frequency; this motivates both activity reduction and voltage/frequency reductions:

$$[H]P_{\text{dyn}} = \alpha C V^2 f, \quad (8.34)$$

where α is per-node switching probability, C is switched capacitance, V is supply voltage, and f is clock frequency. For throughput-bound kernels where throughput $\propto f$, energy per operation is approximately proportional to V^2 , so lowering voltage yields quadratic energy reduction per op, at the cost of linear throughput loss if frequency is reduced proportionally.

Problem $\rightarrow$ analysis $\rightarrow$ implementation:

- Problem: high α arises from idle-thread toggles (warp divergence), frequent memory traffic causing TMU/ROP activity, and always-on control logic in SMs.
- Analysis: reduce α by consolidating active warps and avoiding toggling unused datapaths; reduce C by power domain partitioning and fine-grain gating; reduce V and f opportunistically with workload-aware DVFS.

Concrete runtime techniques and their operational relevance:

1. Clock-enable gating per SM/CU/TMU: prefer clock-enable to gated clocks in RTL synthesis. Use idle counters that detect long inactivity and deassert enables to entire pipeline lanes, reducing effective α .
2. Workload-aware DVFS: combine hardware counters (warp_active, cache_miss_rate, tensor_core_util) with a bounded DVFS controller to shift operating points without violating QoS.
3. Functional gating: disable TMU, texture decompression, RT cores or tensor cores when their utilization falls below thresholds, avoiding leakage-dominant tradeoffs during long idle times.
4. Algorithmic power reduction: VRS, coarse shading, early-Z, and compressed framebuffer writes reduce memory traffic and TMU/ROP toggling.

Practical controller example: per-SM clock-enable generator (synthesizable Verilog). It outputs `clk_en` to be used as the flip-flop enable in SM pipelines. The controller measures activity and asserts enable when active.

Listing 8.19: Per-SM clock-enable controller (synthesizable).

```

module sm_clk_en_ctrl #(
    parameter IDLE_THRESH = 1024 // cycles before disabling
)(
    input wire clk,
    input wire rstn,
    input wire activity,          // pulse when SM issues work
    output reg clk_en             // clock-enable to SM registers
);
    reg [15:0] idle_cnt;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            idle_cnt <= 0;
            clk_en <= 1'b0;
        end else begin
            if (activity) begin
                idle_cnt <= 0;
                clk_en <= 1'b1;          // re-enable on activity
            end else if (idle_cnt >= IDLE_THRESH) begin
                clk_en <= 1'b0;          // disable after threshold
            end else begin
                idle_cnt <= idle_cnt + 1;
            end
        end
    end
endmodule

```

Design implications and trade-offs:

- Aggressive gating reduces dynamic power but increases latency to re-enable units, impacting short bursts and latency-sensitive tasks.
- DVFS yields large energy-per-op gains but shrinks available performance headroom for peak compute and may require larger voltage regulators.
- Consolidating warps improves gating efficiency but can raise per-SM thermal density and reduce memory-level parallelism.

Overall, architect for hierarchical gating (warp, SM, slice, whole-GPU), combine counters for robust policies, and tune thresholds per workload class to balance energy, performance, and thermal headroom.

8.6.2 Workload-based DVFS

Building on dynamic power reduction techniques that targeted switching capacitance and clock gating, workload-based DVFS uses runtime workload signals to select voltage-frequency operating points for different GPU domains (SM/CU, memory, fabric, tensor/RT islands). This subsection describes the problem, derives a simple control-optimality condition, presents an implementable controller, and concludes with concrete design implications.

Problem: fixed-frequency DVFS or static governors either waste energy during memory-bound phases or throttle performance during compute-bound tensor-heavy phases. The controller must map high-level metrics — SM utilization, memory bandwidth saturation, IPC, tensor-core occupancy, and thermal headroom — to domain DF (frequency) settings while respecting transition latency and voltage regulator granularity.

Analysis: model power of a domain as the sum of dynamic and static components, with dynamic proportional to $V^2 f$ and performance proportional to f for throughput-bound work. Let $P(f) = \alpha V(f)^2 f + P_{\text{leak}}$ and throughput $T(f) = \beta f$; energy per operation $E_{\text{op}} = P/T$. With linearized voltage scaling $V(f) = V_0 + \kappa f$, minimize

$$[H]E_{\text{op}}(f) = \frac{\alpha(V_0 + \kappa f)^2 f + P_{\text{leak}}}{\beta f} = \frac{\alpha(V_0 + \kappa f)^2}{\beta} + \frac{P_{\text{leak}}}{\beta f}. \quad (8.35)$$

Differentiating and solving gives the operating point trade-off: raising f reduces the static-energy term $\frac{P_{\text{leak}}}{f}$ but increases the dynamic baseline $\propto (V_0 + \kappa f)^2$. For memory-bound kernels where $T(f)$ saturates to T_{max} independent of SM f , the optimizer pins SM f to the minimum that sustains T_{max} , shifting energy to memory domain.

Implementation: practical controllers combine:

- a lightweight classifier mapping counter vectors to "compute-bound", "memory-bound", or "tensor-bound";
- per-domain lookup tables (LUT) of safe (V, f) pairs;
- hysteresis and minimum residency timers to amortize transition latency;
- thermal headroom and VRM constraints enforcement.

Example runtime controller (production-style, reads perf counters and sets DVFS via firmware API):

Listing 8.20: Workload-based DVFS controller pseudocode

```
# Read counters (sampled every 20 ms)
sm_util, mem_bw_util, ipc, tensor_occ, temp = read_perf_counters()
# Classify workload quickly
if mem_bw_util > 0.85:                # memory-bound
    target_mem_f = mem_lut.high        # keep memory fast
    target_sm_f  = sm_lut.low          # slow SM to save energy
elif tensor_occ > 0.6 and ipc > threshold:
    target_sm_f  = sm_lut.high         # favor tensor throughput
    target_mem_f = mem_lut.med
else:                                # compute-bound
    target_sm_f  = sm_lut.med
    target_mem_f = mem_lut.med
# Enforce thermal headroom and apply with hysteresis
apply_freqs_if_stable(target_sm_f, target_mem_f, temp)
```

Implications and trade-offs:

- Latency: frequent DVFS changes harm interactive latency and can destabilize thermal control; use minimum residency (e.g., 50–200 ms).

- Granularity: per-SM or per-cluster DVFS yields finer energy savings but increases regulator complexity and area.
- Accuracy vs overhead: richer classifiers (ML models) improve mapping but cost counter bandwidth and compute.
- Performance bounds: aggressive downclocking during memory-bound phases reduces energy but risks underutilizing tensor cores if workload shifts rapidly.
- Safety: voltage headrooms, stability margins, and coordination with thermal throttling must be conservative to avoid timing violations.

8.6.3 Thermal throttling

The DVFS and dynamic-power reduction strategies described previously reduce average package power, but thermal throttling remains the ultimate safety control when die temperature approaches limits. Thermal throttling converts thermal sensor data and thermal models into immediate hardware or OS-enforced performance limits to protect silicon ($T_{j,\max}$) and maintain reliability.

Thermal problem — analysis

- Problem: localized hot spots (e.g., tensor-core arrays, memory controllers, TMUs, or power-dense SM clusters) drive junction temperature toward $T_{j,\max}$ faster than system-level DVFS reacts.
- Thermal dynamics: steady-state temperature follows the thermal-resistance model; transient behaviour follows an RC thermal network with thermal capacitance C_{th} and resistance R_{θ} .

$$[H]C_{th}\frac{dT(t)}{dt} = P(t) - \frac{T(t) - T_{amb}}{R_{\theta}}, \quad (8.36)$$

and steady-state yields $T_{ss} = T_{amb} + R_{\theta}P_{ss}$. From this, an upper power bound avoiding throttling is

$$[H]P_{\max} = \frac{T_{j,\max} - T_{amb}}{R_{\theta}}. \quad (8.37)$$

Operational control design

- Sensors and sampling: high-fidelity thermal sensors near hottest blocks (power-delivery, SM banks, HBM controllers) with sampling rates of hundreds of Hz reduce latency. MRT (moving-average) filters mitigate noise but add delay; keep time constants below thermal time constant to remain responsive.
- Policies:
 1. Reactive throttling: when $T \geq T_{throttle}$, reduce core clocks or voltage, gate tensor cores, or limit SM active warps.
 2. Proactive throttling: predict future temperature via short-window power integration; reduce power earlier to avoid entering hysteresis loops.

3. Spatial throttling: disable or downclock specific clock islands (per-SM or per-slice) to maintain throughput elsewhere.
- Scheduler-level actions: warp/SM scheduler can reduce issue rates, increase memory-bound workload multiplexing, or steer work away from hot SMs.

Implementation example The following governance loop, suitable for firmware running on the PMU or OS daemon, reads temperature and applies frequency caps. Comments are concise and pragmatic.

Listing 8.21: Simple thermal governor loop for GPU PMU

```
#include
int read_temp_millideg(void);           // read sensor (milli-deg C)
void apply_freq_cap(uint32_t mhz);      // enforce cap via registers
uint32_t current_cap = 2000;           // initial cap (MHz)
while (1) {
    int t = read_temp_millideg();        // e.g., 85000 == 85.0C
    if (t > 85000) {                     // throttle threshold (85 C)
        current_cap = current_cap > 200 ? current_cap - 200 : 200;
        apply_freq_cap(current_cap);     // reduce performance
    } else if (t < 80000) {              // hysteresis lower bound (80 C)
        current_cap = current_cap < 3000 ? current_cap + 100 : current_cap;
        apply_freq_cap(current_cap);     // relax cap
    }
    sleep_ms(50);                        // sampling cadence
}
```

Design implications and trade-offs

- Latency vs stability: faster sampling reacts sooner but risks oscillation; add hysteresis or model-based prediction to stabilize.
- Granularity: per-SM/spatial throttles preserve aggregate throughput better than global throttles but increase control complexity and require additional clock-domain infrastructure.
- Performance: throttling reduces sustained TFLOPS and may shift bottlenecks (compute→memory); plan scheduler and DVFS policies to minimize impact on latency-sensitive graphics and ML throughput.
- Reliability: conservative $P_{\max}$ extends silicon lifetime; aggressive performance pushes increase T_j and leakage, reducing efficiency.

8.6.4 Efficiency optimization

Linking DVFS-based workload shaping and temperature-aware throttling, this subsection frames practical efficiency optimization: how to select operating points and run-time control policies that maximize useful work per watt while respecting thermal limits identified earlier.

Problem: maximize useful throughput under power and temperature constraints. Analysis begins from first principles: dynamic power scales as voltage and frequency, and leakage grows with temperature. A compact model is

$$[H]P_{\text{total}}(V, f, T) \approx C \cdot \alpha V^2 f + I_{\text{leak}}(V, T) V, \quad (8.38)$$

where C is switched capacitance per cycle, α is activity factor (workload-dependent), and I_{leak} increases with temperature (approximately exponential). Performance for compute-bound kernels often scales linearly with core frequency until other resources (memory, TMU, ROP) bottleneck, so define throughput $S(f, \text{util}) \approx s_0(\text{util}) f$ for a given SM occupancy and instruction mix.

We therefore optimize the efficiency metric

$$[H]\eta = \frac{S(f, \text{util})}{P_{\text{total}}(V, f, T)} \quad (\text{useful work per watt}). \quad (8.39)$$

Practical optimization solves

$$[H]\max_{f, V} \eta \quad \text{s.t.} \quad P_{\text{total}}(V, f, T) \leq P_{\text{cap}}, \quad T \leq T_{\text{th}}. \quad (8.40)$$

Because V and f are quantized in DVFS steps and I_{leak} depends on recent thermal history, a runtime controller using sampling and lightweight models is required rather than closed-form continuous solutions.

Implementation: a workload-aware governor combines hardware counters and a PI loop to tune a power cap or frequency step. Below is production-ready pseudo-C used in embedded GPU firmware; it reads SM utilization, GFLOPS counters, and temperature sensors, and adjusts `freq_step` or `power_cap` to maximize a rolling estimate of η .

Listing 8.22: Runtime efficiency governor (firmware) example

```
/* sample counters and sensors */
uint32_t sm_util = read_hw_counter(SM_UTIL);           // percent
uint32_t gflops = read_hw_counter(GFLOPS);             // rolling GFLOPS
float temp = read_sensor(TEMP_SENSOR);                 // degrees C

/* compute estimates */
float perf = (float)gflops;                             // measured throughput
float power = read_power_meter();                       // instantaneous W
float eta = perf / max(power, 1e-3f);                   // GFLOPS/W

/* PI controller tries to increase eta by nudging freq or cap */
static float integrator = 0;
float error = target_eta - eta;
integrator += ki * error;
float control = kp * error + integrator;

/* map control output to discrete DVFS steps or cap */
int freq_step = clamp(curr_step + (int)roundf(control), MIN_STEP, MAX_STEP);
if (temp > MAX_SAFE_TEMP) freq_step = min(freq_step, THROTTLE_STEP); //
    thermal safety

apply_freq_step(freq_step); // fast switch supported by firmware
```

Operational implications and trade-offs:

- Measuring η directly (hardware GFLOPS counters + power meter) gives robust decisions across SIMT, tensor core, and memory-bound kernels; models relying only on utilization misclassify mixed workloads.
- Aggressively maximizing η can reduce peak throughput; choose objective by workload class (latency-sensitive vs throughput-oriented).
- Hardware support (fine-grained power telemetry, fast DVFS steps, per-SM power gating) reduces controller latency and improves stability but increases design complexity and area.
- Thermal hysteresis and leakage-temperature coupling demand conservative margins in model parameters to avoid thermal runaway; tune PI gains per GPU family.

Chapter 9

Physical Design and Manufacturing

9.1 Floorplanning

9.1.1 Die partitioning and hierarchy

Building on the prior floorplanning discussion of congestion, power distribution, and thermal constraints, this subsection drills into how logical GPU blocks are partitioned and hierarchically arranged on the die to meet performance and manufacturability goals. The problem is to map SMs, memory controllers, caches, I/O PHYs, and fixed-function units into a physical hierarchy that minimizes latency, meets power/thermal budgets, and yields high yield.

Analysis: partitioning decisions are driven by communication patterns, bandwidth demands, and manufacturability limits.

- Communication-driven placement: place strongly communicating units (e.g., SM clusters and their local L1/TMU banks or tile-based L2 slices) close together to reduce interconnect length and NoC contention.
- Bandwidth placement: memory controllers and PHYs must be placed near package I/O or HBM stacks to minimize off-die trace length and R/C loss.
- Power/thermal segmentation: divide the die into power domains and thermal zones so DVFS and power gating limit hot-spot propagation.

Key physical relation: interconnect delay grows quadratically with length for distributed R and C , so wire delay scales as

$$[H]\tau \approx \frac{1}{2}R'C'L^2, \quad (9.1)$$

where R' and C' are per-unit-length resistance and capacitance, and L is wire length. This motivates compact clustering of SMs and placing high-bandwidth paths on lower-resistance metal tiers.

Implementation: apply a hierarchical tiling approach to realize low-latency, manufacturable layouts.

1. Macro-level: split the die into tiles sized to a reticle-friendly granularity. Each tile contains: a SM cluster with SIMD/Tensor resources, associated TMU and L1 banks, a local L2 slice and NoC router port.

2. System-level: arrange tiles in a rectilinear grid with memory controllers and PHY macros aligned along I/O edges or over HBM interfaces.
3. Power/clock: carve power meshes and clock sinks per tile, limiting clock tree depth inside each tile to reduce skew and enabling localized clock gating.
4. Chiplet option: when using chiplets, partition by function: compute chiplets containing SM clusters, I/O and PHY chiplets, and a central coherency/IO hub. Place wide, short die-to-die links for coherence and high-bandwidth flows.

Practical heuristics:

- Balance tile size so average hop count between SMs and L2 remains low while maintaining routability.
- Co-locate ROP/ROP caches near frame buffer and display engines to reduce writeback traffic.
- Reserve continuous metal channels for high-throughput NoC trunks.

Example: simple area-partition helper to compute tile counts and area allocation.

Listing 9.1: Compute tile counts from area budget

```
# compute number of tiles given die area and per-tile areas
die_area_mm2 = 600.0      # total die area in mm^2 (example)
sm_area = 20.0           # area per SM cluster in mm^2
l2_slice = 4.0           # area per L2 slice in mm^2
io_macros = 30.0         # total area reserved for I/O/PHYs
# allocate area for tiles (SM + L2)
tile_area = sm_area + l2_slice
available = die_area_mm2 - io_macros
num_tiles = int(available // tile_area) # integer tiles that fit
print(f"Tiles: {num_tiles}, remaining_area_mm2: {available - num_tiles*
      tile_area}")
# further compute grid dims (closest square)
import math
cols = math.ceil(math.sqrt(num_tiles))
rows = math.ceil(num_tiles / cols)
print(f"Grid: {rows}x{cols} tiles")
```

Implications and trade-offs:

- Tighter clustering reduces NoC latency and power, but increases routing congestion and local thermal density.
- Larger tiles simplify coarse-grain timing and power delivery, but raise average inter-tile communication distance.
- Chiplet partitioning improves yield and heterogeneous integration, but adds die-to-die latency and protocol complexity.

Designers must trade average interconnect length, routability, thermal dissipation, and manufacturability when selecting partition granularity.

9.1.2 Block placement and routing

Building on the die partitioning choices and hierarchical clustering previously described, placement locates each macro and standard-cell cluster on the die while routing realizes the physical interconnects between them. These two problems must be solved together for modern GPUs because large regular arrays (SM clusters, tensor arrays) and few extremely wide, latency-sensitive nets (memory channels, clock, HBM PHYs) dominate performance and manufacturability.

Problem statement and analysis. Block placement must minimize interconnect delay, routing congestion, and thermal density while satisfying power-grid and clock-tree placement constraints. A standard objective uses weighted sums of wirelength, congestion, and timing slack penalties:

$$[H]C = \alpha \cdot \text{HPWL} + \beta \cdot \text{Congestion} + \gamma \cdot \sum_n \max(0, -s_n) \quad (9.2)$$

where HPWL is half-perimeter wirelength summed over nets, Congestion is a bin-based overflow metric, and s_n is timing slack for net n . For a net connecting pins with coordinates (x_i, y_i) , contribution to HPWL is $\text{HPWL}_n = (\max_i x_i - \min_i x_i) + (\max_i y_i - \min_i y_i)$.

Operational constraints specific to GPUs:

- SM clusters should be placed to minimize average distance to L2 slices and memory crossbar to reduce memory access latency and energy.
- Tensor arrays and TMUs form regular grids; preserving alignment simplifies local routing and power distribution.
- HBM stacks, PHYs, and package pins must be along die edges to minimize high-speed IO route length and maintain signal integrity.
- Critical nets (clock, reset, high-bandwidth AXI buses) require reserved wide routing channels and careful via budgeting.

Implementation approach. Follow a staged flow:

1. Global placement: force-directed or analytic placer minimizes Equation (9.2), producing legalized rectangles.
2. Macro legalization: snap large macros to legal sites while preserving hub connectivity (e.g., memory controllers near HBM).
3. Congestion-aware detail placement: iterative swaps to resolve hot bins and open routing channels for wide buses.
4. Clock-tree and power-grid synthesis: insert clock sinks and metal rails before final routing to avoid IR hotspots.
5. Routing: global routing assigns bins; detailed router respects timing-critical constraints and shielding for high-speed nets.

Example lightweight implementation of a cost evaluation and swap move, suitable for placement prototyping:

Listing 9.2: Simple placement cost and swap for prototyping

```

# blocks: dict name -> (x,y,w,h); nets: list of lists of pin coords
def hawl(blocks, nets):
    total=0.0
    for net in nets:
        xs=[]; ys=[]
        for (bname, px, py) in net: xs.append(blocks[bname][0]+px); ys.append(
            blocks[bname][1]+py)
        total += (max(xs)-min(xs)) + (max(ys)-min(ys))
    return total

def swap_and_cost(blocks, nets, a, b):
    # swap centers of blocks a and b
    ba=blocks[a]; bb=blocks[b]
    blocks[a], blocks[b] = (bb[0],ba[1],ba[2],ba[3]), (ba[0],bb[1],bb[2],bb
        [3])
    cost = hawl(blocks, nets)
    # undo swap for caller to decide
    blocks[a], blocks[b] = ba, bb
    return cost

# usage: simulated annealing loop would call swap_and_cost to accept/reject
moves

```

Design implications and trade-offs. Compact placement reduces HPWL and energy but increases routing congestion and thermal hotspots, which can degrade frequency and yield. Prioritizing bandwidth-critical paths and reserving wide routing channels simplifies timing closure at the cost of area and possibly wirelength. Early power-grid and clock-tree planning reduces late-stage reroutes, saving weeks in closure. In short, placement and routing choices directly trade area, timing, thermal behavior, and manufacturability; GPU architects must weight these against workload targets (graphics vs ML) to meet performance and yield goals.

9.1.3 Power grid and clock tree

Building on block placement constraints and the die partitioning hierarchy, the power grid and clock tree must be co-optimized with block floorplan boundaries to meet voltage regulation, IR drop, EM (electromigration), and timing budgets across SM clusters, ROPs, TMUs and memory interfaces. Physical placement determines current paths and clock sink distribution; mismatches between electrical intent and placement create hotspots that break timing or reliability.

The problem: deliver low-impedance power to hundreds of SMs while providing a low-skew, low-jitter clock to deep pipelined shader cores without excessive area or routing overhead. Analysis proceeds by modeling supply rails as a 2D resistive mesh and the clock network as a hierarchical buffered tree or grid-hybrid, then deriving sizing and decoupling requirements from worst-case load and transient currents observed in graphics and ML workloads.

Key analysis equations relate current density and allowable voltage droop. For a metal segment of length L , thickness t , resistivity ρ and width w carrying current I , the voltage drop

is $V = I \cdot R$ with $R = \rho \cdot L / (t \cdot w)$. Rearranging for minimum width given an allowable drop V_{allow} :

$$[H]w \geq \frac{\rho L I}{t V_{\text{allow}}} \quad (9.3)$$

This simple bound guides mesh pitch and number of metal stripes per rail. For electromigration, ensure current density $J = I / (t \cdot w) < J_{\text{max}}$ (process-specified); use parallel vias and redundant stripes where currents concentrate (e.g., near VRM pads and SM clusters).

Clock tree design choices:

- H-tree for low-skew center-out distribution to symmetric clusters (good for regular SM layouts).
- Balanced buffered trees with local deskew buffers inside each SM to absorb placement variations.
- Grid-hybrid (global grid + local H-trees) for large dies to trade routing area for skew robustness.

Decoupling strategy:

- Place hierarchical decoupling capacitors: on-die MIM/metal caps near SMs for fast transients; package decoupling for lower-frequency bulk; board-level VRMs handle slow transients and bulk current.
- Analyze effective impedance $Z(f)$ and ensure the resonance avoids dominant current harmonics from SM switching. Use S-parameter or step-response simulations.

Implementation sketch (power-island sequencer FSM, synthesizable Verilog) to coordinate power gating, isolation and retention during cornering and dynamic power management:

Listing 9.3: Power-island sequencing FSM for safe power gating

```

module power_island_seq(
    input  wire clk,           // always-on clock domain
    input  wire rst_n,        // active-low reset
    input  wire pwr_req,      // request to power-up island
    input  wire pwr_rail_ok,  // hardware power-rail good sensor
    output reg  pwr_en,       // enable power switch (PMOS/NMOS gate)
    output reg  iso_on,       // isolation cells enable
    output reg  ret_latch_en  // retention latch enable
);
typedef enum reg [1:0] {OFF=2'b00, POWERING=2'b01, ON=2'b10} state_t;
state_t state, next;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= OFF; else state <= next;
end
always @(*) begin
    // default outputs
    pwr_en = 1'b0; iso_on = 1'b1; ret_latch_en = 1'b0; next = state;
    case (state)

```

```

OFF: if (pwr_req) next = POWERING;
POWERING: begin
    pwr_en = 1'b1; // assert power switch
    if (pwr_rail_ok) begin
        iso_on = 1'b0; ret_latch_en = 1'b1; next = ON;
    end
end
ON: begin
    pwr_en = 1'b1; ret_latch_en = 1'b1; iso_on = 1'b0;
    if (!pwr_req) next = OFF;
end
endcase
end
endmodule

```

Design implications and trade-offs:

- Wider metal and denser via arrays reduce IR drop and EM risk but consume routing area and increase capacitance, harming clock insertion delay.
- Aggressive clock buffering reduces skew but increases dynamic power and complicates floorplan pin access; local deskew lets placement change without global rework.
- More on-die decoupling reduces transient droop but increases area and may shift resonance; co-design with package VRM yields best area/power balance.

9.1.4 Thermal optimization

Previous floorplanning steps—placing blocks and routing wires while planning a robust power grid and clock tree—establish the electrical and geometric constraints that thermal optimization must respect. Thermal work therefore operates on a fixed partitioning and routing skeleton, adding heat-aware placement, TSV/thermal-via planning, and DVFS control hooks to mitigate hot spots created by dense SM, tensor core, and memory-controller clusters.

Thermal optimization problem statement: minimize peak die temperature and thermal gradients subject to routing, power-grid, and timing constraints. Model the die as a discretized thermal network where conductance between nodes captures silicon, interconnect, package, and TIM heat paths. At steady state the conductance matrix G and power vector P satisfy

$$[H]GT = P \quad \Rightarrow \quad T = G^{-1}P, \quad (9.4)$$

so placement decisions that change adjacency and material stacks implicitly change G and P . Practical optimization replaces a full inverse with fast solvers (sparse Cholesky or multigrid) to compute node temperatures for objective evaluation inside a placement loop.

Analysis steps used in industry-grade floorplanning:

1. Build a coarse thermal grid aligned to macros: SM clusters, tensor arrays, TMU/ROP clusters, memory controllers and IO pads.
2. Approximate local power densities p_i for each macro from workload characterization (graphics vs ML kernels).

3. Solve $GT = P$ incrementally after proposed macro moves to estimate $T_{\max}$ and gradient metrics such as ∇T across neighbor cells.
4. Penalize placements with high $T_{\max}$ or large $\|\nabla T\|$ in the cost function fed to simulated annealing or partitioning-based placer.

Implementation techniques that integrate with routing and power planning:

- Spatially spread tensor cores and high-intensity SMs to avoid contiguous high-power clusters; interleave lower-power logic to act as heat sinks.
- Reserve "thermal channels": dedicated routing corridors for thermal vias and wide metal planes connected to package spreader.
- Co-optimize floorplan with power-grid topology to reduce local IR and align high-conductance power rails with heat sources.
- Insert on-die sensors and DVFS hooks at the macro level, enabling runtime mitigation for unforeseen workload hotspots.

Example synthesizable Verilog: simple thermal monitor that aggregates N on-die sensors and issues DVFS requests when max temperature crosses thresholds.

Listing 9.4: On-die thermal monitor (synthesizable)

```

module thermal_monitor #(
    parameter N = 8,                // number of sensors
    parameter W = 12                // temperature bitwidth
) (
    input  wire clk,
    input  wire rstn,
    input  wire [N*W-1:0] temp_bus, // concatenated temps MSB-first
    input  wire [W-1:0] th_warn,    // warning threshold
    input  wire [W-1:0] th_crit,    // critical threshold
    output reg  warn,               // warn signal
    output reg  crit,              // critical signal
    output reg  [$clog2(N)-1:0] idx_max // index of hottest sensor
);

    integer i;
    reg [W-1:0] max_t;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            max_t <= {W{1'b0}}; warn <= 1'b0; crit <= 1'b0; idx_max <= 0;
        end else begin
            max_t <= {W{1'b0}};
            idx_max <= 0;
            for(i=0; i < max_t) begin
                max_t <= temp_bus[(N-1-i)*W +: W];
                idx_max <= i[$clog2(N)-1:0];
            end
        end
    end

```

```

    warn <= (max_t >= th_warn);
    crit <= (max_t >= th_crit);
end
end
endmodule

```

Design implications and trade-offs:

- Spreading high-power units reduces peak temperature but increases interconnect cost and latency for sensitive paths.
- Thermal channels and vias consume routing area and complicate timing closure.
- Aggressive TIM and package choices lower die temperature but increase BOM and manufacturing risk.
- Runtime DVFS provides flexibility versus increased control complexity and potential performance loss during throttling.

9.2 Synthesis and Timing Closure

9.2.1 RTL synthesis flow

The preceding discussion of block placement, power-grid routing, and thermal budgeting sets constraints that the RTL synthesis flow must respect; synthesis choices directly determine cell counts, netlist shapes, and timing margins used by place-and-route tools. This subsection follows by describing a practical, timing-driven RTL synthesis flow tailored to large SIMT GPU blocks such as SMs, tensor cores, and shared-memory controllers.

Problem: translate cycle-accurate RTL into a gate-level netlist that meets frequency, area, and power targets under multi-mode multi-corner (MMMC) constraints for GPU datapaths. Analysis highlights:

- Critical path contributors in GPUs: wide register-file read ports, long combinational FMA chains in tensor cores, and arithmetic reduction trees feeding TMUs and ROPs.
- Timing budget must cover clk-to-Q, combinational delay, net delay, and setup time; path slack is

$$[H]\text{slack} = T_{\text{clk}} - (T_{\text{cq}} + T_{\text{comb}} + T_{\text{net}} + T_{\text{setup}}), \quad (9.5)$$

and synthesis must maximize positive slack across all timing paths.

Implementation steps (practical sequence used in GPU projects):

1. Front-end checks: lint, CDC analysis, and formal equivalence for microarchitecture transforms.
2. Constraint authoring: create SDC (.sdc) with multi-mode definitions for boost/power-save frequencies, false-path and multi-cycle path declarations for asynchronous interfaces.
3. Technology mapping: perform timing-driven mapping using library characterized for the target node, preferring high-drive cells on fanout-heavy nets feeding SM warp schedulers.

4. Retiming and register balancing: insert/shift pipeline registers to break long combinational chains in tensor cores while preserving handshaking semantics.
5. Optimization passes: gate-level restructuring, buffer insertion for slew control, and area-power trade-offs (cell sizing vs frequency).
6. Verification: static timing analysis (STA) across corners, power estimation, and lint on netlist.

Key practical techniques:

- Prefer explicit valid/ready pipelines for long-latency data paths to enable retiming without functional change.
- Mark non-interfering debug or low-priority paths as false paths to focus optimization on SM-critical loops.
- Use multi-voltage/multi-threshold cells selectively: low- V_t on tensor core critical loops, high- V_t elsewhere to save leakage.

Concrete Verilog implementation example: a synthesizable pipelined register stage with clock-enable, commonly used to add balancing stages generated by synthesis scripts.

Listing 9.5: Pipelined register stage with enable and synchronous reset

```

module pipeline_reg #(
  parameter WIDTH = 64
) (
  input   wire clk,
  input   wire rst_n,           // synchronous active-low reset
  input   wire en,             // clock-enable (synthesis-friendly)
  input   wire [WIDTH-1:0] d,
  output  reg  [WIDTH-1:0] q
);
  always @(posedge clk) begin
    if (!rst_n) q <= {WIDTH{1'b0}}; // reset to zero
    else if (en) q <= d;             // enable-controlled capture
  end
endmodule

```

Implications and trade-offs: aggressive register insertion reduces T_{comb} but increases area and power due to extra flops and routing; selective cell upsizing reduces path delay but worsens power and routing congestion. For GPUs, balance register insertion in tensor cores and SM datapaths against increased memory-interconnect latency and thermal budget during place-and-route.

9.2.2 Timing constraints and setup

Following synthesis, the netlist and timing reports produced in the previous step feed directly into constraint refinement and closure, since synthesis decisions must be validated against physical timing budgets for SM datapaths, TMU fetch units, ROP writebacks, and tensor-core compute lanes. The next paragraphs analyze common setup constraints, show how they map to a

simple synthesizable pipeline implementation, and list practical steps for closing timing across GPU macros and interfaces.

Problem — ensuring that data launched from a source register arrives at the destination register before its setup requirement, accounting for clocking and physical effects. Important contributors are register clock-to-Q (t_{cq}), worst-case combinational delay ($t_{comb,max}$), destination setup time (t_{su}), clock network skew (t_{skew}), and clock jitter (t_{jitter}). For high-frequency GPU datapaths, large register files and long interconnects in SMs inflate $t_{comb,max}$ and inter-stage routing delays.

Analysis — formalize the setup constraint used by static timing analysis (STA). The required clock period T_{clk} must satisfy:

$$[H]T_{clk} \geq t_{cq} + t_{comb,max} + t_{su} + t_{skew} + t_{jitter} \quad (9.6)$$

Slack for the path is computed as

$$[H]slack = T_{clk} - (t_{cq} + t_{comb,max} + t_{su} + t_{skew} + t_{jitter}). \quad (9.7)$$

Targets such as pixel pipelines or tensor cores may require different T_{clk} values, so constraints must support multiple clock definitions and mode-specific exceptions.

Implementation — encode constraints early and conservatively, then refine with post-route delays. Typical actions:

1. Define clocks and derive generated clocks for interfaces like `pcie_clk` and `mem_clk`.
2. Mark false paths (e.g., reset trees) and multi-cycle paths (e.g., handshake sequences between SMs).
3. Run MMMC (multi-mode multi-corner) STA for worst-case PVT corners relevant to GDDR/HBM interfaces and core voltage islands.
4. Apply physical-aware synthesis: retiming, register balancing, and buffer insertion guided by ECOs from place-and-route.

Example: a synthesizable pipeline stage module that STA will analyze for t_{cq} and t_{su} contributions. The module is intentionally simple to show a long combinational path that could violate setup if not pipelined.

Listing 9.6: Simple pipeline stage illustrating register-to-register timing.

```

module pipeline_stage #(
  parameter WIDTH = 32
) (
  input  wire      clk,          // core clock
  input  wire      rst_n,
  input  wire [WIDTH-1:0] din,   // source data
  output reg [WIDTH-1:0] dout    // destination register
);
  reg [WIDTH-1:0] stage_reg;

  // long combinational logic example (synthesizable: replicated XOR tree)
  wire [WIDTH-1:0] comb;

```

```

genvar i;
generate
  for (i = 0; i < WIDTH; i = i + 1) begin : gen_xor
    assign comb[i] = din[i] ^ stage_reg[(i + 1) % WIDTH];
  end
endgenerate

// pipeline register
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    stage_reg <= 0;
    dout <= 0;
  end else begin
    stage_reg <= comb;
    dout <= stage_reg;
  end
end

endmodule

```

9.2.3 Clock domain verification

The previous subsections established timing constraints and the RTL synthesis flow required for timing closure; clock domain verification extends those topics by ensuring crossing paths and constraints align across synthesized netlists and placed-and-routed designs. Correct CDC verification prevents elusive functional failures that survive synthesis and timing signoff, especially between SMs, memory controllers, and I/O domains such as PCIe or HBM.

Problem: GPUs have many independent clock domains (SM/compute, memory controller, display, video, interconnect), so signals cross asynchronous boundaries frequently. A missed synchronization or incorrect false-path constraint can corrupt texture descriptors in a TMU, confuse ROP write sequencing, or deadlock command queues. Metastability is the fundamental failure mode; design verification must quantify and mitigate its risk.

Analysis: Practical CDC verification combines static lint, formal checks, simulation with randomized domain phase relationships, and post-layout static timing checks with correctly specified `create_clock` and `set_false_path` SDC entries. For synchronizer design, Mean Time Between Failures (MTBF) approximates the expected time between unresolved metastability events. With two independent frequencies f_1 and f_2 , and synchronizer resolution time T_s , an often-used model is

$$[H]\text{MTBF} \approx \frac{\exp(T_s/\tau)}{f_1 f_2 \tau}, \quad (9.8)$$

where τ is the metastability time constant of the flop. Equation (1) shows exponential improvement with added resolution slack, highlighting why extra synchronizer stages or deeper FIFO elastic depth can be cost-effective.

Implementation: for data-path crossings carry either:

- single-bit signals through two-flop synchronizers, or

- multi-bit or burst transfers through asynchronous FIFOs using Gray-coded pointers and synchronized read/write pointer copies.

The listing below is a synthesizable, parameterized asynchronous FIFO used in many GPU datapaths (command queues, DMA descriptors). It uses Gray pointers and two-flop synchronizers for pointer transfer, producing safe empty/full flags and supporting block transfers without assuming phase alignment.

Listing 9.7: Asynchronous FIFO with Gray-coded pointers and 2-flop synchronizers

```

module async_fifo #(
  parameter DATA_WIDTH = 64,
  parameter ADDR_WIDTH = 6 // depth = 2^ADDR_WIDTH
) (
  input wire wr_clk,
  input wire rd_clk,
  input wire rst_n,
  input wire wr_en,
  input wire [DATA_WIDTH-1:0] wr_data,
  input wire rd_en,
  output reg [DATA_WIDTH-1:0] rd_data,
  output wire full,
  output wire empty
);

  localparam DEPTH = (1<> 1);
  end
  end
  end

  // read domain logic
  always @(posedge rd_clk or negedge rst_n) begin
    if (!rst_n) begin
      rd_ptr_bin <= 0;
      rd_ptr_gray <= 0;
      wr_ptr_gray_rdclk <= 0;
      wr_ptr_gray_rdclk_r <= 0;
      rd_data <= 0;
    end else begin
      // synchronize write pointer into read clock domain (2-flop)
      wr_ptr_gray_rdclk_r <= wr_ptr_gray_rdclk;
      wr_ptr_gray_rdclk <= wr_ptr_gray;
      if (rd_en && !empty) begin
        rd_data <= mem[rd_ptr_bin[ADDR_WIDTH-1:0]]; // read data
        rd_ptr_bin <= rd_ptr_bin + 1;
        rd_ptr_gray <= (rd_ptr_bin + 1) ^ ((rd_ptr_bin + 1) >> 1);
      end
    end
  end

```

```

// convert synchronized gray to binary for comparisons
function [ADDR_WIDTH:0] gray_to_bin;
    input [ADDR_WIDTH:0] g;
    integer i;
    begin
        gray_to_bin[ADDR_WIDTH] = g[ADDR_WIDTH];
        for (i = ADDR_WIDTH-1; i >= 0; i = i-1)
            gray_to_bin[i] = gray_to_bin[i+1] ^ g[i];
        end
    endfunction

wire [ADDR_WIDTH:0] rd_ptr_bin_sync = gray_to_bin(rd_ptr_gray_wrclk_r);
wire [ADDR_WIDTH:0] wr_ptr_bin_sync = gray_to_bin(wr_ptr_gray_rdclk_r);

assign full = ((wr_ptr_bin[ADDR_WIDTH:0] - rd_ptr_bin_sync) == DEPTH);
assign empty = (wr_ptr_bin_sync == rd_ptr_bin);
endmodule

```

Implications and trade-offs:

- Adding synchronizer stages or FIFO depth reduces metastability risk exponentially, but increases latency and area.
- Over-constraining with false-paths can mask real hold violations in place-and-route; co-ordinate SDC with floorplanning.
- Place synchronizers and FIFO registers near domain boundary logic to reduce routing-induced skew and timing uncertainty.
- For high-throughput SM-to-memory paths, favor asynchronous FIFOs; for control flags, use low-latency two-flop synchronization.

9.2.4 Multi-mode optimization

The preceding discussion established clock-domain verification practices and showed how setup constraints and timing arcs are expressed for static timing analysis; multi-mode optimization extends those ideas by enumerating operational scenarios and applying mode-specific synthesis and STA flows to close timing across all relevant PVT and power conditions.

Multi-mode optimization problem definition: GPUs must meet timing for multiple, sometimes conflicting, operating points such as high-frequency boost (performance) and low-voltage low-power modes. Additionally, functional modes (e.g., power-gated SM clusters, fused clock gating, and asynchronous interfaces) create mode-specific false paths and clock relationships. The engineering goal is to minimize pessimism while ensuring correctness across the union of modes, reducing extra buffering or area while avoiding timing escapes.

Analysis approach:

- Enumerate independent axes: clock domains C , voltage bins V , temperature corners T , and functional modes F . The worst-case number of mode combinations is

$$[H]N_{\text{modes}} = |C| \times |V| \times |T| \times |F|. \quad (9.9)$$

Practical flows group or prune combinations by equivalence (e.g., same library/constraint set).

- Classify timing differences into:
 1. Cycle-time changes: different $create_clock$ periods.
 1. Library swaps: timing libraries vary with voltage/temperature.
 2. Constraint diffs: mode-specific false paths or input/output delays due to disabled islands or retention.
- Use formal equivalence of register connectivity and preserve registers across synthesis runs to avoid functional mismatch when doing per-mode optimizations.

Implementation recipe (tool-agnostic, synthesizable flow):

1. Define canonical mode descriptors and a small set of representative STA scenarios.
2. For each mode, select an appropriate Liberty library and SDC constraint file.
3. Apply synthesis with mode-aware constraints and generate mode-specific netlists, or use multi-mode compilation features (e.g., multi-corner, multi-mode in commercial tools) to produce a single netlist optimized for all modes.
4. Run multi-mode STA; collect critical endpoints per mode and feed back to synthesis for targeted retiming or gate sizing.

Example Tcl snippet for automated mode iteration and constraint generation:

Listing 9.8: Tcl flow for multi-mode SDC and library selection

```
# modes definition (simple example)
set modes {\
  {name "perf"   clk_period 5.0 lib "lib_perf.db"   sdc "sdc_perf.sdc"} \
  {name "economy" clk_period 10.0 lib "lib_lp.db"    sdc "sdc_lp.sdc"} }

foreach m $modes {
  # parse map (tool-specific)
  dict with m {
    puts "Processing mode $name"
    # set timing library for this mode
    link_library $lib
    # generate SDC: create_clock and mode-specific false paths
    create_clock -name clk -period $clk_period -waveform {0 [expr {
      $clk_period/2}]}
    source $sdc ;# additional false paths/input delays
    # run synthesis and write mode-tagged netlist
    compile -incremental ;# mode-aware compile
    write_verilog ${name}_netlist.v
    write_sdf ${name}.sdf
  }
}
```

Implications and trade-offs:

- More modes increase design/verification cost linearly or worse per equation (9.9); therefore, group modes by shared libraries or constraints whenever safe.
- Conservative single-mode optimization wastes area and power; per-mode or true multi-mode compilation reduces overdesign but requires stronger toolchain support and disciplined constraint management.
- Runtime features like DVFS and power gating must be explicitly modeled as mode constraints; failing to declare functional false paths leads to spurious timing fixes and increased pessimism.

9.3 Place and Route

9.3.1 Placement algorithms

Building on prior floorplanning choices and power-grid constraints, placement algorithms convert logical blocks into spatial coordinates while balancing timing, routability, power distribution, and thermal hotspots. The following develops the placement problem, analyzes common algorithmic approaches, presents a compact analytic solver example, and states practical trade-offs for GPU layouts.

Placement problem and cost model. The core objective is to minimize interconnect length and timing slack violations while respecting density and macro constraints. A widely used surrogate for wirelength is half-perimeter wirelength (HPWL) over nets; for quadratic (analytic) placers we approximate the net contribution by a quadratic term. A canonical formulation for 1D placement (x coordinate) is

$$[H] \min_x \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2 + \lambda C_{\text{density}}(x), \quad (9.10)$$

where E is the netlist, w_{ij} are net-based weights (often proportional to net degree or criticality), and C_{density} penalizes local overcrowding. Writing the quadratic part in matrix form gives the Laplacian objective

$$[H] \min_x \frac{1}{2} x^T L x - b^T x, \quad (9.11)$$

with graph Laplacian L and anchor vector b encoding fixed pins and terminal constraints. Solving the sparse linear system $Lx = b$ yields a global minimum for the quadratic surrogate.

Algorithm classes and where they fit GPUs.

- Analytic placers (quadratic, NLopt variants) quickly produce routable global placement for millions of standard cells; they are suitable for dense SM/compute clusters with regular connectivity.
- Partitioning-based methods (min-cut) excel at preserving locality for large macros and I/O islands; used early to cut the die into routing-friendly regions.
- Simulated annealing and force-directed detailed placers provide legalization and cell-level swaps to remove overlaps and fine-tune timing-critical nets, including those feeding TMU or ROP blocks.

- Timing-driven placement integrates path criticality into w_{ij} , increasing weight on nets in critical SIMT or tensor-core data paths.

Implementation example: a minimal analytic solver that builds the Laplacian and solves for x positions with pinned anchors. The code uses SciPy sparse linear algebra and anchors macro/pad positions as boundary conditions.

Listing 9.9: Minimal analytic placement solver using sparse Laplacian

```
import numpy as np
from scipy.sparse import coo_matrix
from scipy.sparse.linalg import spsolve

# nodes: N, nets: list of (i,j,weight)
def analytic_place(N, nets, anchors):
    # build Laplacian L and RHS b
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    for i, j, w in nets:
        rows += [i, j, i, j]; cols += [i, j, j, i]
        vals += [w, w, -w, -w] # Laplacian contributions
    L = coo_matrix((vals, (rows, cols)), shape=(N, N)).tocsr()
    # incorporate anchors as large-diagonal penalties
    for idx, xpos, strength in anchors: # idx node pinned to xpos
        L[idx, idx] += strength
        b[idx] += strength * xpos
    x = spsolve(L, b) # solve sparse linear system
    return x # x positions (1D)

# Example usage: feed nets and anchors for each placement region.
```

Design implications and trade-offs:

- Analytic solvers scale to millions of cells but need strong density and macro legalization passes to avoid overlaps; they trade per-net accuracy for global convexity.
- Timing-driven weighting improves critical-path latency but can increase HPWL and routing congestion; balance via multi-objective weights.
- Macros and power rings require constraint-aware placement; failure to treat them early forces expensive rip-up and detailed placement iterations.
- For GPUs, prioritize low-latency paths between SM/CU clusters, L1/L2 slices, and memory controllers, accepting modest wirelength increases to preserve critical-bandwidth corridors.

9.3.2 Routing congestion management

Placement algorithms created the physical seed that defines local net density, macro adjacency, and channel constraints; routing congestion management operates on that seed to convert density maps and netlists into routable wiring while preserving timing and signal integrity. The

problem is determining where demand exceeds routing supply and choosing corrective actions that minimize timing or functional regressions.

Problem: routing congestion manifests when the aggregate routing demand in a channel bin exceeds available tracks or via capacity, producing hotspots that force nonoptimal detours, increase delay, or require rerouting of long critical nets. Analysis begins with a discrete congestion grid derived from placement and estimated net bounding boxes. For each bin i define demand D_i and capacity K_i ; a simple normalized congestion metric is

$$[H]C_i = \max\left(0, \frac{D_i - K_i}{K_i}\right), \quad (9.12)$$

and a global hotspot cost can be expressed as

$$[H]\text{Cost} = \sum_i w_i C_i^p, \quad (9.13)$$

where weights w_i reflect timing-critical net density and $p \geq 2$ amplifies hotspots.

Implementation choices follow an engineering hierarchy:

1. Early detection: derive D_i using Steiner estimation and pin access demand; include layer-aware capacities so vertical vias and top metal scarcity are captured.
2. Prioritization: weight nets by slack so critical SM-to-L2 interconnects and wide TMU buses get higher w_i , reducing the chance routing fixes harm performance.
3. Corrective actions, ordered by invasiveness:
 - Local rip-up and re-route of noncritical nets.
 - Layer reassignment to less used metal layers, considering via-congestion and RC impact.
 - Channel widening or track reassignment where placement allows.
 - Macro nudging or legalization iterations to redistribute density.

A practical tool-selection policy is implemented as a cost-driven loop that evaluates cost from (2) and applies fixes until Cost falls under a threshold or iteration budget expires. The following Python snippet computes a congestion map and selects candidate nets for rip-up based on weighted contribution; comments indicate where timing data plugs in.

Listing 9.10: Compute congestion and pick rip-up candidates

```
# compute D and K per bin; nets is list of nets with bbox and criticality
def compute_congestion(bins, nets):
    D = [0]*len(bins); K = [bin.capacity for bin in bins]
    for net in nets:
        bins_hit = net.estimate_bins() # bbox-based estimation
        for i in bins_hit:
            D[i] += net.track_demand * net.criticality_weight()
    C = [max(0, (D[i]-K[i])/K[i]) for i in range(len(bins))]
    return C
```

```

def select_ripup_candidates(C, bins, nets, thresh=0.2):
    # choose nets that contribute most to high-C bins, prefer noncritical nets
    score = {net:0 for net in nets}
    for i,c in enumerate(C):
        if c>thresh:
            for net in bins[i].nets: score[net] += c*net.width
    # return top-k noncritical nets
    return sorted(nets, key=lambda n: (score[n], n.slack), reverse=True)[:50]

```

Implications and trade-offs:

- Aggressive layer reassignment reduces planar congestion but raises via density and possible EM/IR hotspots.
- Penalizing critical nets preserves timing but may leave lower-priority congestion requiring placement iteration.
- Iterative rip-up adds run-time; GPU designs with dense SM-L2 fabrics may mandate placement relaxation heuristics early to avoid expensive routing iterations.

9.3.3 Signal integrity checks

These checks build directly on placement decisions (block adjacency, shielding lanes) and routing congestion mitigations (detours that change coupling neighborhoods); both affect worst-case aggressor sets and supply routing that the signal integrity (SI) sign-off step must analyze. The problem is to detect and bound functional failures due to crosstalk, simultaneous switching output (SSO) noise, and glitch-induced timing violations in a large multi-SM GPU fabric while minimizing impact on routing and timing closure.

Analysis: perform parasitic extraction after detailed placement and routing to capture mutual capacitances and resistances between nets. For capacitive coupling, the first-order induced voltage on a victim net from a single aggressor is approximated by

$$[H]\Delta V_{\text{victim}} \approx \frac{C_m}{C_m + C_{\text{load}}} \Delta V_{\text{aggressor}}, \quad (9.14)$$

where C_m is the mutual capacitance and C_{load} is the effective node capacitance. For multiple simultaneous aggressors, superpose worst-case polarity and timing alignment; capacitive coupling scales linearly with simultaneous transitions, so N coactive aggressors can amplify the victim swing up to $\approx N \frac{C_m}{C_{\text{load}}} \Delta V$ before shielding or buffering mitigates it.

Implementation checklist in place-and-route flow:

- Extract RC and Cpairs using sign-off parasitic extraction; generate a netlist with parasitics for PrimeTime SI or equivalent.
- Run noise-aware STA with aggressor/victim annotations to compute crosstalk-induced delay and injected noise; flag nets where injected noise crosses the input noise margin for receivers.
- Perform dynamic simulations for critical SM interconnects (e.g., shared memory banks, ROP data buses) to reveal glitching on combinational paths.

- Check SSO scenarios: model simultaneous bank read/write from many SMs to estimate peak dI/dt and corresponding supply bounce; correlate with IR drop/EM analysis.
- Insert targeted mitigations: shielding (ground/rail traces), spacing rules, buffer insertion (repeaters), net reordering to reduce parallel runs, and local decoupling capacitors in macro placement.

A simple on-chip RTL monitor can help during bring-up to detect stuck or excessive toggles (useful for correlating routing changes to SI events):

Listing 9.11: SI activity monitor for a sampled net

```

module si_activity_monitor #(
    parameter WIDTH = 1,           // number of monitored nets
    parameter CNT_BITS = 16,
    parameter THRESH = 1000       // toggle threshold
) (
    input  wire clk,
    input  wire rst_n,
    input  wire [WIDTH-1:0] sig, // sample of net(s)
    output reg  [WIDTH-1:0] alert // sticky alert per net
);
    reg [WIDTH-1:0] prev;
    reg [CNT_BITS-1:0] cnt [WIDTH-1:0];
    integer i;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            prev <= {WIDTH{1'b0}};
            alert <= {WIDTH{1'b0}};
            for (i=0; i< THRESH) alert[i] <= 1'b1; // flag high activity
            end
            prev[i] <= sig[i];
        end
    end
endmodule

```

Implications and trade-offs:

- Shielding reduces coupling but increases routing area and net capacitance, degrading timing; use selective shielding for long, parallel buses (e.g., TMU/ROP links) identified by SI analysis.
- Buffer insertion lowers coupling seen by long wires but costs power and area and can change arrival times, requiring STA re-closure.
- Conservative spacing and larger metal for power grids mitigate SSO and IR drop but raise congestion; therefore SI fixes must be balanced against placement/routing constraints and compute throughput targets for SM arrays.

9.3.4 IR drop and EM control

Building on the previous discussion of high-speed signal integrity and routing congestion, IR drop and electromigration (EM) control complete the electrical reliability picture by coupling power-distribution geometry to dynamic workload currents from SMs, tensor cores, TMUs, and ROPs. These effects are dominated by instantaneous current demand and the low-frequency impedance of the power-delivery network (PDN), so place-and-route decisions must target both steady-state and transient behavior.

Problem: localized voltage sag and metal migration reduce performance and lifetime. Analysis begins with the node-level voltage relation that includes PDN impedance $Z_{pd}(j\omega)$ and time-varying current $I(t)$:

$$[H]V_{node}(t) = V_{dd} - I(t) \cdot Z_{pd}(j\omega) \quad (\text{approximate in frequency band of interest}) \quad (9.15)$$

For DC or low-frequency worst-case estimates, use $V_{drop} \approx I \cdot R_{grid}$. For EM lifetime, apply Black's equation:

$$[H]\text{MTTF} = A J^{-n} \exp\left(\frac{E_a}{kT}\right) \quad (9.16)$$

where J is current density, E_a activation energy, T absolute temperature, and $n \approx 1-2$. Both equations drive physical decisions: mesh widths, via counts, decoupling distribution, and hot-spot placement.

Implementation checklist during place-and-route:

- Partition high-current macros (SM clusters, L2 slices, memory controllers) to balance instantaneous currents and avoid synchronized switching. Use clock-phase and power-aware placement to decorrelate peaks.
- Dimension metal layers and via arrays so per-via current density $J_{via} = I/(N_{vias}A_{via})$ stays below EM limits with margin. Add redundant vias at expected hot nodes.
- Insert distributed decoupling and place low-ESR caps near SM clusters to cut transient impedance at GHz frequencies.
- Run static DC IR analysis plus transient EM-aware IR (time-stepped currents) and SPICE-level simulation for critical corners.

Example synthesizable Verilog shows a minimal on-chip power monitor and throttle controller that aggregates sampled current counters and issues throttling signals when short-term current budgets exceed safe thresholds. This integrates with SM power managers for runtime mitigation.

Listing 9.12: Runtime power monitor and throttle controller (synthesizable).

```
module power_throttle #(
  parameter N_SAMPS=8, parameter WIDTH=16
) (
  input wire          clk,
  input wire          rstn,
  input wire [WIDTH-1:0] current_sample, // instantaneous current
                                estimate
  input wire [7:0]    temp_degC,        // thermal input

```

```

    output reg                                throttle_req    // throttle request to SM
        arbiter
);
// simple moving-window accumulator
reg [WIDTH+3:0] acc;
reg [3:0] idx;
reg [WIDTH-1:0] window [0:N_SAMPS-1];
integer i;
wire [WIDTH+3:0] threshold = (32'd1000 + temp_degC*8); // programmable
    margin

always @(posedge clk) begin
    if (!rstn) begin
        acc <= 0; idx <= 0; throttle_req <= 0;
        for (i=0;i < threshold) throttle_req <= 1;
        else throttle_req <= 0;
    end
end
endmodule

```

Design implications and trade-offs:

- Wider metal and more vias reduce IR/EM risk but increase area and routing congestion, impacting timing closure.
- Distributed decoupling lowers transient impedance but adds die area and BOM cost.
- Runtime throttling preserves lifetime and avoids overprovisioning the PDN, yet reduces peak throughput for latency-sensitive workloads.
- Early co-placement of high-current units and PDN co-optimization reduces iterations between P&R and signoff EM/IR runs.

9.4 Design for Test

9.4.1 Scan insertion

Following place-and-route and timing-closure decisions, scan insertion must be reconciled with routing congestion, clock-tree planning, and multi-domain gating already established for SMs and CUs. The following discussion analyzes scan insertion problems specific to GPUs and gives an implementation pattern and its performance implications.

Scan insertion problem and analysis:

- Problem: provide high stuck-at and transition fault coverage across millions of flip-flops while minimizing routing, power, test time, and impact on functional timing.
- Key GPU constraints: many clock domains, aggressive clock gating inside SMs, very large register files and SRAM arrays (handled better with BIST), and strict area and power budgets in tensor cores and ROP pipelines.

- Strategy: use hierarchical parallel scan chains (one or more per SM/CU cluster) with balanced chain length to bound shift time and minimize long global routing. Replace only sequential elements that are inexpensive to scan; use partial scan plus test points where full scan violates timing.

Test time model and compression: Operational relevance: shift time dominates ATE cost and must be reduced by chain parallelism and test compression. If a design has total scanned bits L , test vector count V , scan clock frequency f_{scan} , and compression factor C ($C > 1$ means compression), then:

$$[H]T_{\text{test}} \approx \frac{L}{f_{\text{scan}}} \cdot \frac{V}{C}. \quad (9.17)$$

Balancing L across P parallel chains yields per-chain length L/P , reducing shift time linearly with P . Compression adds hardware (decompressors/compactors) but reduces ATE vector count and pin usage.

Implementation details:

- Use a standard scan flip-flop cell with synchronous reset and scan enable. Place scan-enable muxing close to the flop to avoid extra timing arcs. Route chain signals within the same clock domain to avoid CDC issues.
- For gated clocks, use a scan clock path that bypasses gating during test via a test-mode clock mux; ensure the mux is secure and the gating latch does not introduce extra capture windows.
- Memory and register-file elements remain out of scan; apply BIST and March tests for SRAM arrays and dedicated read/write patterns for register banks.
- Integrate on-chip scan compression: LFSR-based decompressor for pattern expansion and XOR-based compactors for response compaction. Locate compressor near chip boundary or per-cluster to reduce interconnect.

Verilog scan cell (synthesizable, single-bit scan DFF):

Listing 9.13: Synthesizable single-bit scan DFF

```

module scan_dff (
  input  wire clk,           // functional clock
  input  wire rst_n,        // active-low synchronous reset
  input  wire se,           // scan enable (1 = shift)
  input  wire si,           // scan in
  input  wire d,            // functional data
  output reg  q,            // flop output
  output wire so            // scan out (next chain element)
);
  // synchronous logic: on scan enable load si, otherwise load d
  always @(posedge clk) begin
    if (!rst_n)
      q <= 1'b0;
    else if (se)
      q <= si;           // shift path
  end

```

```

    else
        q <= d;          // functional path
    end
    assign so = q;       // scan chain output
endmodule

```

Design implications and trade-offs:

- Increasing parallel chains P reduces ATE time but raises test routing and TAP complexity; optimal P trades off tester time cost vs silicon routing overhead.
- Compression reduces vector count but adds area and decompressor timing budget; locate compression per cluster to reduce high-fanout nets.
- Avoid scanning large memories; rely on BIST to preserve scan chain length and reduce test time.
- Ensure test clock bypass of gating circuits to preserve capture timing; otherwise, false rejects occur.

Overall trade-offs must balance ATE cost, silicon area, routing congestion, and timing closure risk for large GPU arrays.

9.4.2 Built-in self-test

Scan insertion prepares logic for controllability and observability by breaking long flops into scan chains; built-in self-test (BIST) picks up where scan leaves off, enabling fast on-chip exercise of logic and large SRAM banks that are otherwise impractical to test exhaustively at wafer sort and bring-up. The following explains why BIST is critical for GPUs, analyzes common algorithms and metrics, then gives a synthesizable MBIST implementation for per-bank SRAM testing.

Problem: modern GPUs contain thousands of on-chip SRAM arrays (register file slices, L1/L2 banks, texture caches) and complex datapaths (SM ALUs, tensor cores, RT cores). Off-chip test stimulus is bandwidth-limited; per-bank parallel BIST reduces test time but must balance area, aliasing risk, and impact on functional silicon.

Analysis:

- Logic BIST (LBIST) typically uses an n -bit linear feedback shift register (LFSR) PRNG as pattern generator and a multiple-input signature register (MISR) as compactor. A maximal LFSR has period $P = 2^n - 1$, so pseudo-random coverage scales with n but aliasing probability of a nonzero fault masking to a zero signature is near 2^{-k} for a k -bit MISR.
- Memory BIST (MBIST) relies on algorithmic tests (March families). For a single bank of depth N_{addr} and N_{passes} passes, test time is approximated by

$$[H]T_{\text{MBIST}} = \frac{N_{\text{addr}} \cdot N_{\text{passes}} \cdot C_{\text{op}}}{f_{\text{clk}}}, \quad (9.18)$$

where C_{op} is the average cycles per read/write operation and f_{clk} the BIST clock. Parallelizing across B banks reduces wall-clock time roughly by B (subject to power/current limits).

Implementation guidance:

- Hierarchical BIST: instantiate local MBIST controllers per SRAM bank to run March C- (write 0; read 0, write 1; read 1) in parallel under a global BIST manager which sequences LBIST for logic clusters (SM, TMU, ROP, tensor core) and coordinates power rails.
- Use LFSR seeds selectable via JTAG for deterministic replay of failing patterns; capture MISR signatures and scan-compress failing scan chains for root-cause debug.
- Throttle parallel BIST to meet wafer power/IR drop constraints and supply sequencer limits.

Concrete, synthesizable MBIST controller (parameterized; single-port memory interface). It implements a March C- variant: write 0 to all, read 0 write 1, read 1; asserts *done* or *fail*.

Listing 9.14: Parametric MBIST controller (synthesizable)

```

module mbist_controller #(
  parameter ADDR_WIDTH = 10,
  parameter DATA_WIDTH = 32
) (
  input  wire      clk,
  input  wire      rst_n,
  input  wire      start,          // start MBIST
  // single-port memory interface
  output reg [ADDR_WIDTH-1:0] mem_addr,
  output reg          mem_en,
  output reg          mem_we,
  output reg [DATA_WIDTH-1:0] mem_wdata,
  input  wire [DATA_WIDTH-1:0] mem_rdata,
  output reg          done,
  output reg          fail
);

localparam STATE_IDLE      = 3'd0;
localparam STATE_W0_ALL    = 3'd1;
localparam STATE_R0_W1     = 3'd2;
localparam STATE_R1_VERIFY = 3'd3;
localparam STATE_DONE      = 3'd4;

reg [2:0] state;
reg [ADDR_WIDTH-1:0] addr;

// synchronous state machine
always @(posedge clk) begin
  if (!rst_n) begin
    state <= STATE_IDLE;
    addr  <= {ADDR_WIDTH{1'b0}};
    mem_en <= 1'b0;
    mem_we <= 1'b0;
  
```



```

mem_wdata <= {DATA_WIDTH{1'b0}};
mem_addr <= {ADDR_WIDTH{1'b0}};
done <= 1'b0;
fail <= 1'b0;
end else begin
  case (state)
    STATE_IDLE: begin
      done <= 1'b0;
      fail <= 1'b0;
      if (start) begin
        addr <= {ADDR_WIDTH{1'b0}};
        state <= STATE_W0_ALL;
      end
    end
  end

  // WRITE 0 to all addresses
  STATE_W0_ALL: begin
    mem_en <= 1'b1;
    mem_we <= 1'b1;
    mem_addr <= addr;
    mem_wdata <= {DATA_WIDTH{1'b0}};
    if (addr == {ADDR_WIDTH{1'b1}}) begin
      addr <= {ADDR_WIDTH{1'b0}};
      state <= STATE_R0_W1;
    end else addr <= addr + 1'b1;
  end

  // READ 0, then WRITE 1
  STATE_R0_W1: begin
    mem_en <= 1'b1;
    mem_we <= 1'b0;           // perform read
    mem_addr <= addr;
    // compare read data against 0 on next cycle (assumes 1-cycle read
    // latency)
    if (mem_rdata != {DATA_WIDTH{1'b0}}) begin
      fail <= 1'b1;
      state <= STATE_DONE;
    end else begin
      // after successful read, write 1 to same address
      mem_en <= 1'b1;
      mem_we <= 1'b1;
      mem_wdata <= {DATA_WIDTH{1'b1}};
      mem_addr <= addr;
      if (addr == {ADDR_WIDTH{1'b1}}) begin
        addr <= {ADDR_WIDTH{1'b0}};
        state <= STATE_R1_VERIFY;
      end else addr <= addr + 1'b1;
    end
  end

```

```

    end
end

// READ 1 and verify
STATE_R1_VERIFY: begin
    mem_en <= 1'b1;
    mem_we <= 1'b0;
    mem_addr <= addr;
    if (mem_rdata != {DATA_WIDTH{1'b1}}) begin
        fail <= 1'b1;
        state <= STATE_DONE;
    end else begin
        if (addr == {ADDR_WIDTH{1'b1}}) begin
            state <= STATE_DONE;
        end else addr <= addr + 1'b1;
    end
end

STATE_DONE: begin
    mem_en <= 1'b0;
    mem_we <= 1'b0;
    done <= 1'b1;
    state <= STATE_IDLE;
end

default: state <= STATE_IDLE;
endcase
end
end
endmodule

```

Design implications and trade-offs:

- Parallel per-bank MBIST minimizes wafer test time but increases peak on-chip current; throttle with a global power-aware sequencer.
- LBIST LFSR/MISR reduce test vector storage but incur aliasing risk; increase MISR width or append deterministic vectors for critical paths (e.g., tensor core accumulators).
- Embedding BIST hardware consumes area and timing budget; place controllers near banks to reduce routing and allow bank-local parallelism, trading modest area for large reductions in ATE time and improved field diagnosability.

9.4.3 JTAG and boundary scan

The preceding discussion showed how built-in self-test (BIST) provides on-chip structural coverage and how scan insertion exposes flip-flops for vector-based ATPG; JTAG and boundary scan provide the standardized external access and daisy-chaining mechanism required to drive

those scan chains during manufacturing and in-system debug. This subsection analyzes the TAP interface and practical boundary-scan deployment for large GPU dies, then gives a compact synthesizable TAP+boundary register example and concrete design trade-offs.

JTAG is specified by IEEE 1149.1 and centers on a Test Access Port (TAP) with four mandatory pins (TCK, TMS, TDI, TDO) and one optional reset (TRST_n). The TAP sequences through defined states (RESET, RUNTEST/IDLE, SHIFT-IR, SHIFT-DR, UPDATE, etc.), enabling two principal operations:

- Instruction register (IR) shifting to select modes such as EXTEST, SAMPLE/PRELOAD, or BYPASS.
- Data register (DR) shifting to clock boundary-scan cells or other scan chains for observation and control.

For modern GPUs, multiple practical patterns emerge:

1. Partition scan chains per major functional block (SM/CU clusters, memory controllers, PHYs) to cap shift latency.
2. Use short BYPASS paths for unused regions during board-level tests, minimizing global chain depth.
3. Gate external access in production silicon with a secure fuse or authentication unit to prevent unauthorized read/write of internal state.

Operational performance depends on chain length B and TCK frequency f_{TCK} . The nominal shift time for a full boundary chain is

$$[H]t_{\text{shift}} = \frac{B}{f_{\text{TCK}}}, \quad (9.19)$$

so halving B or doubling f_{TCK} directly halves time. In practice, f_{TCK} is limited by IO pad SERDES and package signal integrity; typical production test TCK ranges from a few hundred kilohertz to tens of megahertz.

The following Verilog implements a compact, synthesizable TAP controller with parameterizable boundary register width and IR length. It supports RESET, SHIFT_IR, SHIFT_DR, UPDATE, and RUNTEST/IDLE states, and drives a simple boundary shift chain.

Listing 9.15: Synthesizable minimal TAP controller with boundary-scan chain

```

module jtag_tap #(
  parameter IR_LEN = 4,
  parameter BOUNDARY_LEN = 128
) (
  input   wire TCK,           // test clock
  input   wire TMS,          // test mode select
  input   wire TDI,          // serial input
  input   wire TRST_n,       // optional async reset (active low)
  output  reg  TDO           // serial output
);

// TAP state encoding

```

```

typedef enum reg [3:0] {
    RESET=4'd0, IDLE=4'd1, SELECT_DR=4'd2, CAPTURE_DR=4'd3,
    SHIFT_DR=4'd4, EXIT1_DR=4'd5, PAUSE_DR=4'd6, EXIT2_DR=4'd7,
    UPDATE_DR=4'd8, SELECT_IR=4'd9, CAPTURE_IR=4'd10, SHIFT_IR=4'd11,
    EXIT1_IR=4'd12, PAUSE_IR=4'd13, EXIT2_IR=4'd14, UPDATE_IR=4'd15
} tap_state_t;

reg [3:0] state, next_state;

// Instruction and boundary registers
reg [IR_LEN-1:0] ir;
reg [BOUNDARY_LEN-1:0] boundary_chain;

// State transition (sampled on rising TCK)
always @(posedge TCK or negedge TRST_n) begin
    if (!TRST_n) state <= RESET;
    else state <= next_state;
end

// Combinational next-state logic
always @(*) begin
    case (state)
        RESET:      next_state = TMS ? RESET : IDLE;
        IDLE:       next_state = TMS ? SELECT_DR : IDLE;
        SELECT_DR:  next_state = TMS ? SELECT_IR : CAPTURE_DR;
        CAPTURE_DR: next_state = TMS ? EXIT1_DR : SHIFT_DR;
        SHIFT_DR:   next_state = TMS ? EXIT1_DR : SHIFT_DR;
        EXIT1_DR:   next_state = TMS ? UPDATE_DR : PAUSE_DR;
        PAUSE_DR:   next_state = TMS ? EXIT2_DR : PAUSE_DR;
        EXIT2_DR:   next_state = TMS ? UPDATE_DR : SHIFT_DR;
        UPDATE_DR:  next_state = TMS ? SELECT_DR : IDLE;
        SELECT_IR:  next_state = TMS ? RESET : CAPTURE_IR;
        CAPTURE_IR: next_state = TMS ? EXIT1_IR : SHIFT_IR;
        SHIFT_IR:   next_state = TMS ? EXIT1_IR : SHIFT_IR;
        EXIT1_IR:   next_state = TMS ? UPDATE_IR : PAUSE_IR;
        PAUSE_IR:   next_state = TMS ? EXIT2_IR : PAUSE_IR;
        EXIT2_IR:   next_state = TMS ? UPDATE_IR : SHIFT_IR;
        UPDATE_IR:  next_state = TMS ? SELECT_DR : IDLE;
        default:    next_state = RESET;
    endcase
end

// Shift IR and DR on SHIFT states (TDI sampled at TCK posedge)
always @(posedge TCK or negedge TRST_n) begin
    if (!TRST_n) begin
        ir <= {IR_LEN{1'b0}};
        boundary_chain <= {BOUNDARY_LEN{1'b0}};
    end

```

```

    TDO <= 1'b0;
  end else begin
    if (state == SHIFT_IR) begin
      TDO <= ir[0]; // TDO driven from LSB of IR
      ir <= {TDI, ir[IR_LEN-1:1]}; // shift right, new bit at MSB
    end else if (state == SHIFT_DR) begin
      TDO <= boundary_chain[0]; // drive chain LSB
      boundary_chain <= {TDI, boundary_chain[BOUNDARY_LEN-1:1]}; // shift
    end else if (state == UPDATE_IR) begin
      // IR updated; in a full design, update control signals here
      TDO <= ir[0];
    end else begin
      TDO <= 1'bz; // float TDO when not shifting (tooling may drive pull)
    end
  end
end
endmodule

```

Design implications and trade-offs:

- Partitioning reduces B and test time but increases boundary logic and routing complexity.
- Higher f_{TCK} shortens test time but demands robust IO and package SI.
- Secure gating of JTAG avoids information leakage but complicates production debug and in-field diagnostics.
- For chiplet GPUs, standardized boundary scan across die interfaces enables per-die isolation and parallel test, improving wafer yield at cost of additional pad count.

9.4.4 Yield analysis

The previous subsections described how boundary-scan and built-in self-test expose and localize faults; yield analysis quantifies their impact on production throughput and guides redundancy and partitioning choices. Below we analyze probabilistic yield models, show how spare allocation changes the effective yield for large-die SIMD fabrics, and provide a synthesizable controller that maps detected column faults to spare columns.

Yield modeling. For initial engineering estimates use a Poisson defect model where the expected defect count per die is $\lambda = A \cdot D$, with die area A and defect density D (defects per unit area). The classical single-parameter yield model gives

$$[H]Y_{\text{Poisson}} = e^{-\lambda} = e^{-AD}, \quad (9.20)$$

which is accurate when defects are independent and small. For correlated defects or clustering use the negative binomial model with clustering factor α :

$$[H]Y_{\text{NB}} = \left(1 + \frac{AD}{\alpha}\right)^{-\alpha}. \quad (9.21)$$

Redundancy effect. If the design provides s identical spares per functional column group, and defects follow Poisson with mean λ , the probability that the number of defects in that group is tolerable ($\leq s$) is

$$[H]P_{\leq s} = \sum_{k=0}^s \frac{\lambda^k e^{-\lambda}}{k!}. \quad (9.22)$$

For a GPU with many replicated units (texture units, ROP islands, register-file banks) the overall die yield is the product of group yields when failures are independent. Use these equations to size spare counts and to evaluate die partitioning alternatives (single monolithic die versus chiplet break-up). Partitioning reduces A per partition and thus increases yield exponentially for Poisson behavior.

Practical analysis steps

1. Estimate D from foundry historical data and wafer maps.
2. Compute baseline Y from (9.20) and (9.21).
3. For each unit type, compute $P_{\leq s}$ from (9.22) and adjust global yield.
4. Perform sensitivity analysis of s , A , and D to determine marginal cost of additional spares versus expected yield gain.

Implementation example. Below is a synthesizable Verilog spare allocation controller that takes a BIST-produced column fault mask and assigns up to `NUM_SPARES` replacements. It demonstrates a production-ready FSM suitable for inclusion in an `ROP` or `TMU` bank controller.

Listing 9.16: Spare-column allocator: maps faulty columns to spare indices.

```

module spare_row_manager #(
  parameter NUM_COLS    = 256,
  parameter NUM_SPARES = 8
) (
  input  wire          clk,
  input  wire          rst_n,
  input  wire          start,           // trigger mapping from
    fault_mask
  input  wire [NUM_COLS-1:0] fault_mask, // 1 = faulty column
  output reg  [31:0]      spare_used_count, // number of spares
    consumed
  output reg          done,
  output reg  [$clog2(NUM_COLS)-1:0] remap_table [0:NUM_SPARES-1] // maps
    spare i -> replaced column idx
);

integer i;
reg [$clog2(NUM_COLS)-1:0] next_idx;

always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    spare_used_count <= 0;
  
```

```

done <= 1'b0;
for (i = 0; i < NUM_SPARES; i = i + 1)
    remap_table[i] <= 0;
end else if (start) begin
    spare_used_count <= 0;
    next_idx <= 0;
    done <= 1'b0;
    for (i = 0; i < NUM_COLS; i = i + 1) begin
        if (fault_mask[i] && (spare_used_count < NUM_SPARES)) begin
            remap_table[spare_used_count] <= i[$clog2(NUM_COLS)-1:0];
            spare_used_count <= spare_used_count + 1;
        end
    end
    done <= 1'b1;
end
end

endmodule

```

9.5 Packaging Technologies

9.5.1 Flip-chip and BGA packaging

Following place-and-route and IR-drop analysis, packaging choices close the physical-design loop by determining the PDN, signal integrity, and thermal path available to the many SMs and tensor cores on a large GPU die. The flip-chip + BGA combination solves the electrical and thermal needs of high-density SIMT processors by minimizing loop inductance, enabling large I/O counts, and providing a direct heat conduction path to the board-level heat spreader.

Problem: modern GPU dies deliver hundreds of watts in a few square centimeters, so packaging must satisfy three coupled constraints: low PDN impedance across DC–GHz, minimal signal and power inductance for fast TMU/ROP interfaces, and low thermal resistance from junction to ambient. Analysis:

- Electrical: flip-chip uses an array of solder bumps (microbumps/C4) to connect die pads directly to an organic substrate or interposer. Short vertical paths reduce current-loop area and series inductance compared to perimeter wire-bonds, improving transient voltage droop and enabling tighter decoupling budgets. The available I/O count N scales inversely with bump pitch p :

$$[H]N \approx \frac{A_{\text{die}}}{p^2}, \quad (9.23)$$

so reducing pitch increases routing complexity but enables distributed PDN and many high-speed SERDES lanes or memory channels.

- Thermal: the dominant metric is junction-to-case thermal resistance θ_{RJC} . For steady-state power P , junction rise is

$$[H]\Delta T = P \cdot R_{\theta\text{JC}}, \quad (9.24)$$

making low thermal-resistance solder bumps and thermal interfaces critical for SM frequency scaling.

Implementation notes (assembly and materials):

1. Die attach and reflow using controlled atmosphere; bumps formed as solder paste or preformed balls.
2. Substrate options:
 - Organic BGA substrates: lower cost, high layer count for routing and embedded capacitance.
 - Silicon interposer (2.5D): used for HBM stacks, supports TSVs and microbumps.
3. Underfill epoxy reduces CTE-driven shear on bumps, improving thermal cycling reliability.
4. TI materials: thermal interface material (TIM) + heatspreader + heatsink attach for spreading heat from bump region.

Practical calculation and quick checks: a small script computes junction temperature rise and approximate I/O count for a target pitch.

Listing 9.17: Package quick-check: junction temp and IO count

```
# simple calculator for junction rise and IO count
A_die_mm2 = 600.0          # die area in mm^2 (e.g., large GPU)
pitch_mm = 0.5            # bump pitch in mm
P_W = 300.0               # package power in watts
R_theta_JC = 0.08         # junction-to-case R (degC/W) -- engineering estimate

# I/O count estimate (ignore keepout and power pad allocation)
N_io = int((A_die_mm2) / (pitch_mm**2))
deltaT = P_W * R_theta_JC

print(f"Estimated I/O count: {N_io} balls")    # many balls enable distributed
PDN
print(f"Junction rise (degC): {deltaT:.1f}")    # drives TIM and heatsink design
```

Design implications and trade-offs:

- Reducing bump pitch increases N and lowers per-bump current, but raises substrate routing congestion and cost.
- Flip-chip reduces PDN inductance and thermal resistance, enabling higher SM frequencies and more aggressive DVFS, but demands underfill and careful CTE management to avoid fatigue.
- Choice of organic substrate versus interposer affects HBM integration, signal attenuation, and board-level routing; interposers improve density at higher cost. Balancing electrical, thermal, reliability, and cost constraints determines optimal package architecture for a given GPU generation.

9.5.2 Thermal interface materials

The flip-chip and BGA packaging discussion established the geometric and mechanical constraints that set the stage for selecting thermal interface materials (TIMs): die-to-heat spreader gaps, underfill regions, and allowed assembly pressures directly determine which TIM families are practical. Here we analyze TIM behavior, quantify thermal resistance contributions for high-power GPU hotspots (SMs, tensor cores, ROP clusters), and show a simple design tool for sizing TIM performance.

Problem: GPUs concentrate tens to hundreds of watts in small die regions, producing high heat fluxes that must traverse multiple interfaces. The dominant thermal penalty at the package level is often the TIM layer between the die or integrated heat spreader (IHS) and the heatsink. Key metrics are thermal conductivity k ($\text{W m}^{-1} \text{K}^{-1}$), bond-line thickness t (m), contact area A (m^2), and contact resistance from surface roughness and voids.

Analysis:

- The bulk conduction resistance of a uniform TIM layer is

$$[H]R_{\text{TIM}} = \frac{t}{kA}, \quad (9.25)$$

so halving t or doubling k reduces resistance proportionally. For a die power P , the temperature rise across the TIM is $\Delta T_{\text{TIM}} = PR_{\text{TIM}}$.

- Real interfaces add contact resistance R_{contact} due to microscopic asperities and trapped air; total interface resistance is

$$[H]R_{\text{total}} = R_{\text{contact}} + R_{\text{TIM}} + R_{\text{spreader}}. \quad (9.26)$$

Contact resistance is reduced by higher clamping pressure, surface planarization, and the TIM's ability to fill gaps (low-viscosity greases vs compliant gap fillers).

- Spreading resistance of an IHS becomes relevant for large dies and nonuniform hotspot distributions; spreading resistance decreases as the IHS area increases and with higher thermal conductivity materials.

Implementation (material families and trade-offs):

- Thermal greases/inks: high k ($1\text{--}8 \text{ W m}^{-1} \text{K}^{-1}$ for advanced fillers), low t achievable under compression, susceptible to pump-out and aging under thermal cycling.
- Phase-change materials (PCM): low-viscosity at reflow, solid at operating temperature; good initial contact, moderate long-term reliability.
- Thermally conductive pads and gap fillers: electrically insulating, compliant for nonplanar stacks, but typically thicker (hundreds of μm) and lower effective k ($1\text{--}3 \text{ W m}^{-1} \text{K}^{-1}$).
- Solder TIMs (e.g., indium, Sn-based alloys): lowest R , permanent, can act as mechanical bond, but introduce CTE mismatch stress and require process heat.
- Vapor-deposited or thin-film TIMs (metallic layers, graphene-enhanced films): excellent for extreme thin bond lines and high k , but may require planar surfaces and sophisticated deposition.

A compact tool to compare TIM choices for a hotspot area and power density (example Python snippet):

Listing 9.18: Simple TIM temperature-rise calculator (area in m^2 , k in W/mK , t in m).

```
# compute deltaT across TIM and total interface
def deltaT_tim(P, A, k, t, R_contact=0.0):
    R_tim = t / (k * A)          # conduction resistance
    R_total = R_contact + R_tim  # ignore spreader term here
    return P * R_total

# example: 200 W localized over 1e-4 m^2 (10x10 mm hotspot)
P = 200.0
A = 1e-4
print(deltaT_tim(P, A, k=5.0, t=50e-6))  # advanced grease, 50um
print(deltaT_tim(P, A, k=1.0, t=200e-6))  # gap filler, 200um
```

Implications and performance trade-offs:

- Minimize bond-line thickness and maximize effective k to reduce ΔT , but consider mechanical stress: solder/TIM choices that minimize R may increase die stress and assembly complexity.
- For large dies, prioritize high- k IHS and consider low- R thin TIMs to reduce spreading resistance; for nonplanar or multi-chip modules, compliant gap fillers ensure reliability despite higher thermal resistance.
- Assembly parameters (clamp force, surface finish) and TIM aging (pump-out, phase separation) materially affect long-term thermal budget and therefore GPU throttling margins; select TIM and process to balance peak thermal resistance, reliability, and manufacturability.

9.5.3 Multi-chip modules

The discussion of thermal interface materials and flip-chip/BGA assembly naturally leads to multi-chip modules (MCMs), where thermal paths and solder/bumps determine both mechanical integrity and electrical performance across multiple dies. MCMs combine packaging techniques with interposer or substrate-level die integration to optimize GPU throughput, memory proximity, and yield.

Problem: GPUs require massive aggregate memory and compute bandwidth while maintaining reasonable yield and thermal dissipation. Monolithic die scaling increases defect risk; MCMs split functions (SM clusters, HBM stacks, IO) into tileable dies. Key metrics are aggregate bandwidth, die-to-die latency, and thermal resistance.

Analysis: interconnect options include passive silicon interposer, active interposer, and organic substrate. Critical constraints:

- Routing density (pins/ mm^2) limits per-link peak bandwidth B_{link} .
- Serialization and link protocol overhead introduce latency L_{ser} per hop.
- Thermal resistance chain from junction to ambient affects max power per tile.

A simple capacity model for N identical compute tiles gives aggregate fabric bandwidth

$$[H]B_{\text{total}}(N) = \frac{N \cdot B_{\text{link}}}{1 + \alpha(N - 1)} \quad (9.27)$$

where α models cross-tile contention and protocol inefficiency; $\alpha \rightarrow 0$ for ideal nonblocking interposer, and increases when external interconnects (PCIe/NVLink) are oversubscribed.

Thermal modeling is equally important. Junction temperature can be approximated as

$$[H]T_j = T_a + P \cdot (R_{\theta JC} + R_{CT} + R_{TA}) \quad (9.28)$$

where $R_{\theta JC}$ is junction-to-case, R_{CT} is case-to-heat-sink (including TIM), and R_{TA} is heat-sink-to-ambient.

Implementation: practical MCMs use:

- Tile partitioning: map SM/CU clusters and memory controllers to separate dies to maximize local traffic and minimize cross-die hops.
- UCIe or proprietary PHYs on interposer with credit-based flow control to avoid head-of-line blocking.
- Thermal stitching: place high-power dies adjacent to large TIM and vapor chamber areas; route hot-spots away from fine-pitch TSV clusters.

Example synthesizable Verilog for a simple credit-based die-to-die link FSM (flow control) used at the PHY/PL level:

Listing 9.19: Credit-based die-to-die link FSM (synthesizable).

```

module die2die_link #(parameter CREDITS=8) (
    input   wire  clk, rstn,           // clock and active-low reset
    input   wire  tx_req,             // request to send (from producer)
    output  reg   tx_ack,             // link grants send
    input   wire  rx_credit,          // incoming credit returned
    output  reg   rx_consume          // consume signal to receiver
);
reg [$clog2(CREDITS):0] credits;    // credit counter
// initialize credits on reset
always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        credits <= CREDITS;
        tx_ack <= 1'b0;
        rx_consume <= 1'b0;
    end else begin
        // grant if credits available
        if (tx_req && credits > 0) begin
            tx_ack <= 1'b1;
            credits <= credits - 1;
        end else begin
            tx_ack <= 1'b0;
        end
    end

```

```

    // return credit when peer signals consumption
    if (rx_credit) credits <= credits + 1;
    // consumer pulse (example) - could be handshake-driven
    rx_consume <= tx_ack; // simple loopback consume for pipeline
end
end
endmodule

```

Implications and trade-offs:

- Bandwidth vs. latency: passive interposers offer high density and low hop latency, but active interposers enable protocol offload at higher cost.
- Yield vs. performance: smaller tiles improve yield but increase cross-die traffic and protocol overhead α in (10.15).
- Thermal budget: partitioning must respect (9.34); packing many high-power SM tiles increases R_{CT} demands and cooling cost.
- Design cost: UCIe and TSV technology reduce electrical losses but add process complexity and BOM cost.

9.5.4 TSV-based stacking

Following the discussion of multi-chip modules and thermal interface choices, vertical integration using through-silicon vias (TSVs) is the key enabler for tightly coupled HBM stacks and logic-on-memory assemblies beneath SM arrays and tensor cores. TSVs provide short, high-density die-to-die interconnects but introduce electrical, thermal, mechanical, and layout constraints that must be balanced against bandwidth and yield.

A TSV (through-silicon via) is a plated hole through a die that carries signals, power, or ground between stacked dies. In modern GPUs, TSVs are used for:

- HBM stacks: wide, low-latency memory channels adjacent to memory controllers.
- Logic partitioning: splitting large dies into logic and I/O tiers close to TMUs, ROPs, or memory controllers.
- Power delivery: dedicated TSV power/ground pillars for reduced IR drop and better PDN (power distribution network) locality.

Quantitatively, aggregate TSV link bandwidth is approximately proportional to the number of TSV lanes, lane width, and link frequency:

$$[H]B = n_{TSV} \cdot f_{link} \cdot w \quad (\text{bits/s}), \quad (9.29)$$

where n_{TSV} is count of active lanes, f_{link} is per-lane toggling rate, and w is bits per cycle. Parasitics limit f_{link} : each TSV contributes capacitance C_{TSV} (order of tens of femtofarads) and contact resistance R , producing a time constant $\tau \approx R \cdot C_{TSV}$ that constrains achievable rise times and equalization needs.

Architectural impact analysis:

- Signal integrity: tight TSV pitch increases coupling; differential signaling and on-die equalization are required for multi-GHz lanes near SMs that drive high-bandwidth reads for texture units.
- Thermal/mechanical: TSVs create stress concentration; place them away from active HF regions unless stress-aware design and filler rings are used, especially near ROP clusters with high sustained power.
- Layout tradeoffs: TSV pitch p consumes area roughly p^2 per via, reducing usable logic area; routing to TSV arrays must minimize congestion for L1/L2 paths serving tensor cores.
- Yield: TSVs that fail in a pillar can render an entire stack unusable; redundancy and error correction across die-to-die links are standard.

Practical implementation example: a synthesizable TSV link wrapper with parity and elastic buffering, suitable as a die-to-die handshake across a synchronous stack. The module is parameterized for width and implements simple error detection and valid/ready flow control.

Listing 9.20: Synchronous TSV link wrapper with parity and elastic FIFO

```

module tsv_link #(parameter DATA_W = 64, parameter DEPTH = 8) (
    input   wire           clk,
    input   wire           rst_n,
    // TX side (local die)
    input   wire [DATA_W-1:0] tx_data,
    input   wire           tx_valid,
    output  wire           tx_ready,
    // RX side (remote die)
    output  reg  [DATA_W-1:0] rx_data,
    output  reg           rx_valid,
    input   wire           rx_ready,
    output  reg           rx_error
);

    // Simple synchronous FIFO storage
    reg [DATA_W-1:0] fifo [0:DEPTH-1];
    reg [$clog2(DEPTH):0] wr_ptr, rd_ptr, count;
    reg parity_in, parity_out;

    // parity generation (even parity)
    wire pgen = ^tx_data;

    // Write logic
    assign tx_ready = (count < DEPTH);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            wr_ptr <= 0; count <= 0; parity_in <= 0;
        end else if (tx_valid && tx_ready) begin
            fifo[wr_ptr] <= tx_data;
            parity_in <= pgen;
        end
    end

```

```

        wr_ptr <= wr_ptr + 1;
        count <= count + 1;
    end
end

// Read logic with parity check
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0; rx_valid <= 0; rx_data <= 0; rx_error <= 0;
    end else if ((count > 0) && (!rx_valid || (rx_valid && rx_ready))) begin
        rx_data <= fifo[rd_ptr];
        parity_out <= ^fifo[rd_ptr];
        rx_error <= (parity_out != parity_in); // simple check across path
        rx_valid <= 1;
        rd_ptr <= rd_ptr + 1;
        count <= count - 1;
    end else if (rx_valid && rx_ready) begin
        rx_valid <= 0;
    end
end
endmodule

```

Design implications and trade-offs:

- Use TSVs to minimize SM-to-HBM latency but budget die area and thermal headroom for TSV arrays.
- Prefer differential lanes and on-die equalization when f_{link} approaches multi-GHz ranges indicated by (10.15).
- Add redundancy and lightweight link-layer error detection (parity/CRC) to improve effective yield at the cost of area and latency.

9.6 Section 6: Advanced Integration

9.6.1 Chiplet architectures

Following the prior discussion of package-level choices and die stacking techniques, chiplets move the problem from a single monolithic die to coordinated dies connected by high-performance die-to-die links. This subsection analyzes how partitioning, interconnect bandwidth, latency, and thermal constraints determine GPU system-level performance and manufacturability.

Problem: modern GPUs require massive compute and memory bandwidth while facing diminishing returns from monolithic scaling. Partitioning into chiplets reduces yield loss, enables heterogeneous process selection (e.g., compute on bleeding-edge node, IO/HBM on mature node), and allows reuse of large I/O or memory dies. The key design question becomes: how to provision inter-chiplet bandwidth and low enough latency to avoid new system-level bottlenecks.

Analysis: performance follows a variant of the roofline model where the whole-package memory bandwidth BW_{total} and arithmetic intensity AI determine whether the GPU is memory- or compute-bound. To avoid memory-bound execution for a peak compute throughput $\text{FLOPS}_{\text{peak}}$, required aggregated chiplet bandwidth must satisfy

$$[H]BW_{\text{total}} \geq \frac{\text{FLOPS}_{\text{peak}}}{\text{AI}}. \quad (9.30)$$

If compute dies are scaled out (N_{compute} chiplets), die-to-die interconnect BW scales with link count and per-link rate; effective per-chiplet memory bandwidth is $BW_{\text{total}}/N_{\text{compute}}$, raising contention and potential stalls.

Important physical parameters:

- Link latency: request/response round-trip affects SM stall time for loads; coherent or coherent-like protocols need sub-100 ns latencies to preserve throughput for fine-grained workloads.
- Topology: ring, mesh, or crossbar at package level impacts hop count and congestion. A mesh with low-diameter hops favors scalable SM clusters.
- Flow control: credit-based or elastic buffers reduce deadlock and enable pipelining without large buffering inside SMs.
- Thermal and PDN: distributing power across multiple dies reduces local IR drop but complicates VRM placement and thermal coupling; hot compute chiplets require robust TIM and heat-spreader design.

Implementation example: a synthesizable credit-managed FIFO endpoint suitable for a chiplet die-to-die streaming link. It implements fixed-depth buffering with credit return to upstream endpoint, using a circular buffer for data storage.

Listing 9.21: Credit-backed FIFO endpoint for chiplet link

```

module chiplet_link_endpoint #(
  parameter WIDTH = 64,
  parameter DEPTH = 16,
  parameter PTR_W = $clog2(DEPTH)
) (
  input wire          clk,
  input wire          rst,
  // Ingress (from remote chiplet)
  input wire [WIDTH-1:0] in_data,
  input wire          in_valid,
  output wire         in_ready, // credit-driven
  // Egress (to local consumer)
  output reg  [WIDTH-1:0] out_data,
  output reg          out_valid,
  input wire          out_ready
);
  // circular buffer storage
  reg [WIDTH-1:0] ram [0:DEPTH-1];

```

```

reg [PTR_W-1:0] wr_ptr, rd_ptr;
reg [PTR_W:0]    count;

assign in_ready = (count < DEPTH); // accept when space exists

// write side (ingress)
always @(posedge clk) begin
    if (rst) begin
        wr_ptr <= 0; count <= 0;
    end else if (in_valid && in_ready) begin
        ram[wr_ptr] <= in_data; wr_ptr <= wr_ptr + 1; count <= count + 1;
    end
end

// read side (egress)
always @(posedge clk) begin
    if (rst) begin
        rd_ptr <= 0; out_valid <= 0; out_data <= 0;
    end else begin
        if (!out_valid && (count > 0)) begin
            out_data <= ram[rd_ptr];
            out_valid <= 1;
            rd_ptr <= rd_ptr + 1;
            count <= count - 1;
        end else if (out_valid && out_ready) begin
            out_valid <= 0;
        end
    end
end
endmodule

```

Implications and trade-offs:

- Partitioning reduces NRE and improves yield, but requires BW provisioning per (9.30); under-provisioning produces memory-bound scaling.
- High-rate short links (e.g., UCIe, EMIB) reduce latency but increase power and signal-integrity cost; interposer-based solutions lower latency at higher substrate cost.
- Heterogeneous process choice simplifies integration of HBM and IO dies but forces careful PDN and thermal co-design.
- Flow-control and topology selection directly affect buffer sizing, head-of-line blocking, and coherence cost; conservative buffering raises area and power budgets.

9.6.2 Die-to-die interconnects

Building on the chiplet partitioning and IO-budgeting strategies just discussed, die-to-die interconnects are the physical and protocol substrate that make separated GPU function blocks

behave as a single coherent accelerator. The following examines the problem, analyzes key electrical and protocol constraints, shows a practical link-level implementation pattern, and states concrete design implications.

Problem statement and analysis. GPU dielets must exchange large, low-latency volumes of data between SM clusters, memory controllers, and tensor cores while meeting thermal, yield, and routing constraints. Key metrics are:

- bandwidth density (Gb/s per mm of edge or per bump),
- latency (serialization plus PHY and pipeline delay),
- energy per bit (pJ/bit), and
- reliability (BER and recovery cost).

Physical options include short-reach SERDES over organic substrate, silicon interposer or EMIB micro-bumps, and TSV stacks. Protocol choices include lane aggregation with lightweight link layers (credit flow control and optional FEC) or full-coherence-capable fabrics (UCIe for standardized PCIe-like transactions). Electrical design must budget equalization, pre-emphasis, and power per lane; typical on-package links target energy in the order of 1–10 pJ/bit depending on reach and speed.

A succinct performance model helps system tradeoffs. If N is lanes per interface, R is per-lane raw data rate, and η is encoding efficiency after link coding and FEC overhead, effective throughput T is

$$[H]T = N \cdot R \cdot \eta. \quad (9.31)$$

Latency includes serialization $L_{\text{ser}} = \text{flit_bits} / (N \cdot R)$ plus fixed PHY/pipeline L_{phy} ; for small flits the serialization term can dominate.

Implementation pattern. A production-friendly link-layer design uses:

1. wide flits sized to amortize CRC and header overhead for tensor-core data (e.g., 256–512 bytes per flit for bulk transfers, smaller for coherence ops),
2. credit-based flow control to avoid head-of-line blocking and to support in-order delivery across multiple virtual channels,
3. elastic FIFOs for clock-domain and deskew tolerance,
4. light-weight error detection (CRC/parity) and optional retransmit or FEC for longer reaches.

The Verilog module below implements a synthesizable die-to-die link endpoint with flit FIFOs and credit flow-control. It demonstrates the core handshake and buffering you would place at each die boundary.

Listing 9.22: Die-to-die link endpoint with credit-based flow control and parity

```
module ddi_link_endpoint #(
    parameter FLIT_WIDTH = 256,
    parameter FIFO_DEPTH = 8,
    parameter CREDIT_WIDTH = 4
) (
```

```

input wire          clk,
input wire          rst_n,
// Transmit side
input wire [FLIT_WIDTH-1:0] tx_flit,
input wire          tx_valid,
output reg          tx_ready,
output reg [CREDIT_WIDTH-1:0] tx_credits_out, // advertise free entries
// Receive side
output reg [FLIT_WIDTH-1:0] rx_flit,
output reg          rx_valid,
input wire          rx_ready,
input wire [CREDIT_WIDTH-1:0] rx_credits_in // remote credits returned
);

// Simple FIFO for elastic buffering
reg [FLIT_WIDTH-1:0] fifo [0:FIFO_DEPTH-1];
reg [$clog2(FIFO_DEPTH):0] wr_ptr, rd_ptr, count;
reg [CREDIT_WIDTH-1:0] local_credits;

// parity bit per flit for error detect
wire tx_parity = ^tx_flit;
reg parity_error;

// Transmit logic: push when tx_valid and credit available
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= 0; rd_ptr <= 0; count <= 0;
        tx_ready <= 1'b0; local_credits <= FIFO_DEPTH;
        tx_credits_out <= FIFO_DEPTH;
    end else begin
        // update credits when remote returns credits
        if (rx_credits_in != 0) begin
            local_credits <= local_credits + rx_credits_in;
        end
        tx_credits_out <= FIFO_DEPTH - count; // advertise available entries
        tx_ready <= (local_credits > 0) && (count < FIFO_DEPTH);
        if (tx_valid && tx_ready) begin
            fifo[wr_ptr] <= tx_flit;
            wr_ptr <= (wr_ptr + 1) % FIFO_DEPTH;
            count <= count + 1;
            local_credits <= local_credits - 1;
        end
        // transmit to receiver when available and receiver ready
        if ((count > 0) && rx_ready) begin
            rx_flit <= fifo[rd_ptr];
            rx_valid <= 1'b1;
            rd_ptr <= (rd_ptr + 1) % FIFO_DEPTH;
            count <= count - 1;
        end
    end
end

```

```

        end else begin
            if (rx_ready) rx_valid <= 1'b0;
        end
    end
end

// parity check on receive
always @(posedge clk) begin
    if (rx_valid && rx_ready) begin
        parity_error <= (^rx_flit) != tx_parity); // simple detect (example)
    end
end
endmodule

```

Implications and trade-offs:

- Higher lane count N reduces serialization latency but increases package area and power; choose N to match SM aggregate IO needs.
- Larger flits improve amortized efficiency η in (1), but increase tail-latency for small coherence messages.
- Credit depth and FIFO sizing trade area for throughput headroom under jitter and out-of-order reassembly.
- Error control choice (CRC+retransmit versus FEC) trades link latency predictability versus added energy per bit.
- Standardizing on UCIE-like protocols eases ecosystem integration but may constrain latency optimizations needed for coherent SM-level traffic.

9.6.3 UCIE protocol

The previous subsections showed why chiplet partitioning and robust die-to-die links are prerequisites for modern GPU scaling: chiplet architectures reduce monolithic die cost and die-to-die interconnects determine achievable bandwidth, latency, and coherence. UCIE (Universal Chiplet Interconnect Express) is the industry response that standardizes the package-level interconnect stack to enable interoperable GPU chiplets and heterogeneous IP blocks.

UCIE operationally defines a layered protocol stack and an electrical PHY profile optimized for short, high-density package channels. For architects, the key properties are:

- Modular layering: a physical PHY with lane aggregation, a link layer providing flow control and link training, and an upper-layer mapping that can carry PCIe, CXL, or vendor protocols.
- Flexible topologies: point-to-point, switch fabric, daisy-chain, and retimer-assisted longer-reach links. Each topology changes hop count and hence latency and power.
- Flow control and reliability: credit-based flow control with link-level CRC and optional retry paths to reduce packet loss on package interconnects.

- Protocol multiplexing: upper-layer protocols are encapsulated so a GPU can expose coherent memory to a host via CXL or expose DMA/IO via PCIe over the same physical substrate.

Architectural analysis uses a simple bandwidth model that makes trade-offs explicit. If N is lanes, R is per-lane raw rate in Gbit/s, and η is encoding/overhead efficiency ($0 < \eta \leq 1$), usable aggregate bandwidth B in GB/s is

$$[H]B = \frac{N \cdot R \cdot \eta}{8}. \quad (9.32)$$

Example: with $N = 16$, $R = 32$ Gbit/s and $\eta = 0.9$, $B \approx 57.6$ GB/s usable, which matches typical GPU inter-slice channel needs when multiplied across multiple links.

Implementation considerations for GPUs:

- Place memory-heavy tilelets (HBM controllers, large L2 slices, or tensor-core arrays) next to the highest-bandwidth UCIe root to minimize hop latency.
- For coherent shared memory across chiplets, implement a coherence manager at the UCIe endpoint or use a coherence mapping via CXL over UCIe; this adds protocol-state memory and increases latency budget per access.
- Link training and lane equalization require a PHY training finite-state machine and may add microsecond-scale initialization costs, which must be amortized across runtime sessions.

A small synthesizable Verilog utility commonly required in a UCIe endpoint is a credit counter for transmit flow control. The counter below is production-ready and safe for synthesis; integrate it with your link-layer FSM to gate flits when credits are zero.

Listing 9.23: UCIe transmit credit counter (synthesizable)

```

module ucie_credit_counter #(parameter WIDTH=8) (
    input  wire      clk,
    input  wire      rstn,
    input  wire      credit_inc, // +1 credit event (rx grant)
    input  wire      credit_dec, // -1 credit on flit sent
    output reg [WIDTH-1:0] credits, // current credits
    output wire      credit_empty // true if no credits
);
    // synchronous credit update, saturates at max value
    always @(posedge clk) begin
        if (!rstn) credits <= {WIDTH{1'b0}};
        else begin
            case ({credit_inc, credit_dec})
                2'b10: credits <= credits + 1; // increase
                2'b01: credits <= (credits != 0) ? credits - 1 : 0; // consume
                default: credits <= credits; // no change or
                    both
            endcase
        end
    end

```

```

end
assign credit_empty = (credits == 0);
endmodule

```

Design implications and trade-offs:

- Wider lane counts lower serialization latency but increase pin/power and routing complexity on the organic substrate or interposer.
- Choosing switch-based topologies yields flexible fanout but adds additional hop latency and area for switching logic; point-to-point maximizes latency predictability for latency-critical GPU paths.
- Coherent mappings over UCIe require additional state and injects non-trivial latency; use region-based coherence or software-managed coherence to bound complexity.
- Power-per-bit and thermal bead placement drive lane counts and physical routing choices; design for graceful degradation (rate negotiation) to handle thermal/power caps.

9.6.4 Heterogeneous integration

Building on the UCIe protocol’s standardized transport and the physical options for die-to-die interconnects, heterogeneous integration focuses on assembling disparate silicon tiles and memory into a cohesive GPU subsystem while preserving SM/CU locality and minimizing latency to TMUs, ROPs, and tensor cores. The problem is to reconcile electrical, thermal, and coherence constraints while enabling high aggregate bandwidth and acceptable yield.

Analysis: heterogeneous integration mixes chiplets, stacked dies, and interposers. Key variables are per-link bandwidth, inter-die latency, power density, and thermal resistance. Aggregate link bandwidth scales roughly with the number of physical lanes N and per-lane rate r , giving

$$[H]B_{\text{agg}} = N \cdot r, \quad (9.33)$$

but usable GPU throughput also depends on serialization overhead and protocol efficiency η , so effective bandwidth is $B_{\text{eff}} = \eta B_{\text{agg}}$. Thermal coupling imposes a temperature rise proportional to power and thermal resistance:

$$[H]\Delta T = P_{\text{die}} \cdot R_{\text{th,stack}}, \quad (9.34)$$

which directly limits peak SM/Tensor core frequency. Interposer-based approaches reduce pin pitch and lower link energy per bit, while 3D stacking trades longer thermal paths for shorter latency between logic and near-memory HBM.

Implementation considerations concentrate on three integrated subproblems: (1) PHY and link management to provide low-energy, low-latency lanes to SM clusters; (2) power-delivery network (PDN) partitioning across dies to avoid IR drop and electromigration; (3) coherence and memory-mapping for unified L2/L3 across chiplets. A small synthesizable Verilog controller below demonstrates lane aggregation, link training, and simple health reporting suitable for a chiplet-to-chiplet interconnect manager:

Listing 9.24: Chiplet link manager: lane aggregation and training

```

module link_manager #(

```

```

parameter LANES = 8,           // physical lanes
parameter ID_WIDTH = 4
)(
  input wire clk, reset_n,
  input wire [LANES-1:0] lane_rx_valid, // per-lane status
  output reg [LANES-1:0] lane_tx_enable, // enable lanes
  input wire start_training,
  output reg trained, // training done
  output reg [ID_WIDTH-1:0] status_id
);
// simple FSM: IDLE -> TRAIN -> ACTIVE
localparam IDLE=2'd0, TRAIN=2'd1, ACTIVE=2'd2;
reg [1:0] state, next_state;
always @(posedge clk or negedge reset_n) begin
  if (!reset_n) state <= IDLE; else state <= next_state;
end
always @(*) begin
  next_state = state;
  lane_tx_enable = {LANES{1'b0}};
  trained = 1'b0;
  case(state)
    IDLE: if (start_training) next_state = TRAIN;
    TRAIN: begin
      // enable lanes if receiver sees valid signal
      lane_tx_enable = lane_rx_valid;
      if (&lane_rx_valid) next_state = ACTIVE; // all lanes good
    end
    ACTIVE: begin
      lane_tx_enable = {LANES{1'b1}};
      trained = 1'b1;
    end
  endcase
end
// simple status identifier
always @(posedge clk) if (!reset_n) status_id <= 0; else status_id <=
  status_id + trained;
endmodule

```

Implications and trade-offs:

- Bandwidth vs. power: increasing N raises B_{agg} linearly but increases PDN and thermal load; energy-per-bit must be below a few pJ bit^{-1} for efficient GPU interconnects.
- Latency vs. yield: tighter die-to-die pitch (microbump, interposer) reduces latency but complicates yield and repairability compared to discrete chiplets.
- Coherency scope: full-cache coherency across chiplets simplifies programming but costs cross-die coherence traffic and energy; partitioned coherence reduces bandwidth needs at software cost.

- Thermal stacking: 3D stacking improves locality to HBM but requires aggressive heat spreading; R_{th} must be budgeted per SM/tile to avoid frequency throttling.

Designers must balance these trade-offs to meet SM-level latency requirements for graphics and ML kernels while keeping power density within thermal budgets.

Chapter 10

Future Directions and Emerging Technologies

10.1 Modern GPU Case Studies

10.1.1 NVIDIA Hopper and AMD RDNA3

Building on earlier discussion of compute-versus-bandwidth tradeoffs and the role of specialised units for ML and ray tracing, this case study contrasts two recent mainstream designs and how each resolves those tradeoffs in hardware and software. The analysis emphasizes operational consequences for ML training, real-time rasterization, and hybrid workloads.

NVIDIA's Hopper lineage targets maximum AI throughput by tiling hardware around dense matrix engines and a unified SM substrate. The architectural problem is converting transistor budget into sustained tensor MACs while avoiding memory-starvation of the tensor datapath. Hopper solves this by

- increasing on-chip data reuse through larger coherent caches and cross-SM L2 collaboration,
- widening and diversifying tensor cores to support mixed precisions including low-precision formats, and
- integrating interconnect and software (CUDA/cuDNN) optimizations that expose sparsity and operator fusion.

For ML workloads, practical effects are higher arithmetic intensity and reduced DRAM bandwidth per FLOP, improving TFLOPS/W on transformer-like models. For graphics, the investment yields higher compute headroom for denoising and AI-based upscaling, while RT cores remain dedicated to BVH traversal and intersection.

AMD's RDNA3 addresses a different set of constraints: die yield and power-efficient peak frequency. The primary engineering decision is to partition the GPU into smaller chiplets, allowing aggressive process choices for compute dies and separate memory/cache dies. This modularity lets AMD scale shader arrays and cache capacity independently, improving raster throughput and cache hit rates (beneficial for texture-heavy workloads). RDNA3 also refines ray accelerators per CU and focuses on balanced TMU/ROP throughput to keep pixel pipelines fed at higher clocks. The result is competitive raster performance per watt and improved scaling for multi-GPU-like configurations without requiring a single gigantic monolithic die.

To reason about per-SM occupancy and its performance impact across these designs, we use a resource-constrained occupancy model. Let R_{SM} be registers per SM, r_{warp} registers required per warp, S_{SM} shared memory per SM, s_{block} shared memory per block, W_{max} maximum warps per SM, and $W_{\text{per_block}}$ warps per block. Then

$$[H]\text{occupancy} = \frac{\min\left(\left\lfloor \frac{R_{\text{SM}}}{r_{\text{warp}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{s_{\text{block}}} \right\rfloor \cdot W_{\text{per_block}}\right)}{W_{\text{max}}}. \quad (10.1)$$

A small utility computes occupancy given per-kernel parameters; this helps tune kernels for either Hopper (favoring high tensor throughput, moderate occupancy) or RDNA3 (favoring high occupancy to feed high-clock CUs).

Listing 10.1: Simple occupancy calculator used for kernel tuning

```
#include
// compute occupancy fraction (0..1)
double occupancy(int R_SM, int r_warp, int S_SM, int s_block,
                 int W_per_block, int W_max) {
    int max_warps_by_regs = R_SM / r_warp;           // integer division
    int max_blocks_by_shm = S_SM / s_block;
    int max_warps_by_shm = max_blocks_by_shm * W_per_block;
    int active_warps = std::min(max_warps_by_regs, max_warps_by_shm);
    if (active_warps > W_max) active_warps = W_max;
    return double(active_warps) / double(W_max);
}
// Example: use real device constants when available.
```

Design implications and trade-offs:

- Hopper: prioritise large tensor core arrays and cache — good for high arithmetic-intensity ML, at cost of larger monolithic die complexity and thermal hotspots.
- RDNA3: chiplet modularity improves yield and cache scaling, providing energy-efficient raster throughput but requiring low-latency chiplet interconnect design.
- Software: Hopper benefits from operator fusion and sparsity-aware kernels, while RDNA3 benefits from occupancy-aware scheduling and cache-friendly data layouts.
- Tuning: use equation (1) and the occupancy tool to balance register/shmem usage versus active warps; optimal points differ between high-throughput tensor pipelines and high-frequency raster pipelines.

10.1.2 Intel Arc and Apple GPU designs

Building on the comparison of NVIDIA Hopper and AMD RDNA3 (ray/AI tradeoffs, large tensor arrays vs. flexible CUs), we now examine how Intel Arc and Apple GPU designs pursue different tradeoffs between ray tracing, AI acceleration, memory bandwidth, and energy efficiency. These two families illustrate alternative integration points: Intel targets feature parity with discrete GPUs while Apple emphasizes SoC-level coherence and power-constrained throughput.

Problem: deliver high graphics and ML throughput within distinct ecosystems.

- Intel Arc (Xe HPG lineage) emphasizes programmable CUs with hardware ray-tracing units, XMX-like matrix engines for ML, and a conventional discrete-GPU memory subsystem (GDDR/GPU VRAM). Operationally, this targets peak GFLOPS and feature compatibility with desktop APIs.
- Apple GPU designs (Apple Silicon) prioritize tightly-coupled unified memory, tile-based rendering to reduce external bandwidth, and aggressive power-performance scaling in a SoC envelope. ML workloads are often steered to the Neural Engine (ANE) or specialized matrix paths off the main GPU.

Analysis: use the Roofline model to reason about where each design is likely to be bound. For a kernel with arithmetic intensity A (FLOPs/byte), attainable performance is limited by compute peak $P_{\max}$ or memory bandwidth B :

$$[H] \text{Attainable FLOPS} = \min(P_{\max}, B \cdot A). \quad (10.2)$$

- Intel Arc’s larger external bandwidth and wider vector/TMU resources push more kernels into the compute-bound regime ($P_{\max}$). That benefits heavy ray-tracing and large-matrix inference on-CU.
- Apple’s tile-based approach effectively increases on-chip reuse, raising effective A by reducing bytes transferred off-chip; smaller B can suffice, improving energy efficiency for mobile/SoC workloads.

Implementation: a central part of both designs is CU/work-queue dispatch with fairness and backpressure. Below is a synthesizable Verilog round-robin dispatcher that maps incoming work packets to N_{CU} compute units with valid-ready handshaking—reflecting the practical need to balance latency hiding and power gating in both Intel and Apple pipelines.

Listing 10.2: Round-robin dispatcher for N_{CU} CUs (synthesizable).

```

module rr_dispatch #(
    parameter N_CU = 4,
    parameter IDW  = $clog2(N_CU)
) (
    input  wire          clk,
    input  wire          rst,
    input  wire          in_valid,
    input  wire [31:0]   in_pkt,
    output reg           in_ready,
    output reg [N_CU-1:0] cu_valid,
    output reg [31:0]    cu_pkt [0:N_CU-1],
    input  wire [N_CU-1:0] cu_ready
);

reg [IDW-1:0] ptr;
integer i;

always @(posedge clk) begin
    if (rst) begin

```

```

ptr <= 0;
in_ready <= 1;
cu_valid <= 0;
end else begin
  in_ready <= 0;
  cu_valid <= 0;

  // round-robin selection logic
  for (i = 0; i < N_CU; i = i + 1) begin
    if (cu_ready[(ptr + i) % N_CU] && in_valid && !in_ready) begin
      cu_valid[(ptr + i) % N_CU] <= 1;
      cu_pkt[(ptr + i) % N_CU] <= in_pkt;
      ptr <= (ptr + i + 1) % N_CU;
      in_ready <= 1;
    end
  end
end
end
endmodule

```

10.1.3 Mobile GPUs (Mali, Adreno)

Building on the previous case studies of desktop-class throughput optimizations and heterogeneous accelerator integration, mobile GPU designs shift emphasis to minimizing energy per frame and reducing DRAM traffic while retaining usable shader throughput for graphics and ML tasks.

Mobile vendors converge on three operational levers: on-chip tiling to exploit locality, aggressive bandwidth compression, and fine-grained power management. ARM Mali implementations center on a hardware tiler and tile-based deferred rendering (TBDR) that stages raster work into small on-chip tiles, allowing early-Z and blending to occur without round-trips to external DRAM. Qualcomm Adreno designs also use tiling and hierarchical binning but pair that tiler with low-latency ROP/TMU paths and proprietary bandwidth compression (e.g., UBWC) to sustain interactive frame rates. Both stacks integrate ISP/NPU interfaces so shader cores (SIMT-like units) can be repurposed for lightweight ML kernels, while NPUs handle heavier DNN inference.

Quantifying bandwidth requirements clarifies the trade-offs. For a framebuffer of width W , height H , bytes-per-pixel b , and target frame rate f , the raw bandwidth per second is approximately

$$[H]B_{\text{raw}} = W \cdot H \cdot b \cdot f. \quad (10.3)$$

With an on-chip tiler and compression factor $c > 1$, the external bandwidth requirement reduces to $B_{\text{ext}} \approx B_{\text{raw}}/c$, directly lowering system power at the cost of on-chip SRAM area and the logic for compression/decompression.

Architectural analysis and implementation considerations:

- Tile size selection trades on-chip SRAM versus parallelism: larger tiles increase cache reuse (good for complex shading, texture reuse) but require bigger tile buffers and incur

higher latency for partial-frame updates.

- Compression (AFBC, UBWC) reduces B_{ext} but adds encode/decode latency in the TMU/ROP datapaths; designs must pipeline these stages to avoid starving SM/CU execution units.
- Power management uses cluster-level power gating and DVFS; fast stitch points (tiler completion, early-Z rejects) create natural windows to gate TMU or shader clusters.

A compact, synthesizable RTL example of a tile buffer dual-port RAM used in a mobile tiler demonstrates an implementation pattern that balances area and single-cycle access for local ROP/TMU operations:

Listing 10.3: Simple synthesizable tile buffer (dual-port RAM).

```

module tile_buffer #(
    parameter TILE_PIXELS = 1024,
    parameter DATA_W = 32,
    parameter ADDR_W = $clog2(TILE_PIXELS)
) (
    input wire clk,
    input wire wr_en,           // write enable
    input wire [ADDR_W-1:0] wr_addr,
    input wire [DATA_W-1:0] wr_data,
    input wire rd_en,           // read enable
    input wire [ADDR_W-1:0] rd_addr,
    output reg [DATA_W-1:0] rd_data
);

// inferred dual-port BRAM (one port read, one port write)
reg [DATA_W-1:0] mem [0:TILE_PIXELS-1];

always @(posedge clk) begin
    if (wr_en) mem[wr_addr] <= wr_data;           // write port
    if (rd_en) rd_data <= mem[rd_addr];           // read port (registered)
end
endmodule

```

Design implications and performance trade-offs: choosing smaller tiles reduces SRAM cost but increases external bandwidth and shader stalls; adding complex compression improves energy efficiency but requires pipelined encode/decode logic and verification of corner-case formats; finally, aggressive power gating yields large energy savings for interactive workloads, at the cost of wake-up latency that must be amortized by sufficient tile/scene locality.

10.1.4 Design tradeoff comparisons

The prior comparisons emphasized power-constrained mobile choices and SoC integration tradeoffs seen in Intel Arc and Apple GPUs; those examples motivate a cross-product tradeoff framework that highlights compute, memory, and specialization decisions. Below we analyze concrete design axes, quantify their interactions with a simple performance model, and show a small utility to compare candidate configurations.

A practical performance model for throughput-oriented workloads is the roofline bound, which directly exposes the compute–memory tradeoff. Let P be achievable performance (FLOP/s), P_{peak} the peak SM/CU/Tensor performance, B the sustained memory bandwidth (bytes/s), and I the arithmetic intensity (FLOP per byte). Then achievable performance is

$$[H]P \leq \min(P_{\text{peak}}, B \cdot I). \quad (10.4)$$

Equation (1) clarifies two regimes: compute-bound when $I \geq P_{\text{peak}}/B$, and memory-bound otherwise. For real GPUs, P_{peak} aggregates SM/cores and tensor cores weighted by precision throughput; B must account for effective bandwidth after compression and protocol overhead.

Apply this to design comparisons:

- NVIDIA (Hopper-class): allocates large die area to tensor cores and high-rate interconnects, shifting more workloads into the compute-bound regime for ML kernels. This increases P_{peak} but requires large on-die accumulators and wide memory buses (HBM) to avoid memory bounds.
- AMD (RDNA3): prioritizes high-frequency CUs and ROP/TMU balance for graphics. RDNA3 trades some tensor specialization for higher single-thread throughput, improving raster and compute mixed workloads where I varies.
- Intel Arc and Apple: prioritize SoC memory coherency and unified memory latency, reducing effective B variability for CPU–GPU sharing but constraining raw P_{peak} per-watt.
- Mobile (Mali/Adreno): trade raw P_{peak} for aggressive tiling, on-chip caches, and TMU/ROP balance to maximize effective I through locality, thereby shifting many workloads toward the compute side without larger bandwidth.

Design decisions can be explored programmatically; the listing computes the roofline bound for candidate designs.

Listing 10.4: Compute roofline bound for candidate GPUs

```
def roofline_bound(P_peak, bandwidth, intensity):
    """Return achievable FLOP/s (min of compute and memory roof)."""
    # P_peak: FLOP/s, bandwidth: bytes/s, intensity: FLOP/byte
    return min(P_peak, bandwidth * intensity)

# Example compare: Hopper-like vs RDNA3-like (numbers illustrative)
hopper = roofline_bound(1.2e14, 1.2e12, 30)    # 120 TFLOPS, 1.2 TB/s,
intensity 30
rdna3 = roofline_bound(8.0e13, 0.7e12, 30)    # 80 TFLOPS, 0.7 TB/s, intensity
30
print(hopper, rdna3)    # numeric bound outputs
```

Implications and trade-offs:

- Increasing P_{peak} without proportional B yields diminishing returns for memory-bound kernels.
- Specialized units (tensor/RT) raise silicon efficiency for targeted workloads but reduce general-purpose ALU area, affecting graphics or mixed pipelines.

- Chiplet or HBM integration trades lower manufacturing cost or higher bandwidth for increased latency and inter-die coherence complexity.
- For SoC-integrated designs, optimizing on-chip cache hierarchy and locality raises effective I , often delivering better real-world throughput than raw P_{peak} increases.

10.2 Specialized AI Accelerators

10.2.1 Google TPU and Cerebras engine

Following the GPU case studies, we now contrast two divergent approaches to accelerating dense and sparse AI workloads: a tightly coupled systolic ASIC favored by Google, and a wafer-scale many-core engine from Cerebras. The comparison focuses on computational fabric, memory hierarchy, and sustained throughput for large transformer and CNN models.

Problem: modern deep learning kernels (GEMM, convolution, attention) demand extremely high multiply-accumulate (MAC) throughput while minimizing off-chip traffic. Analysis begins with the operating bound: an accelerator’s sustained throughput is limited either by peak compute or by memory system bandwidth multiplied by arithmetic intensity. Formally,

$$[H]\text{Sustained FLOPS} \leq \min(P_{\text{peak}}, B_{\text{mem}} \cdot I), \quad (10.5)$$

where P_{peak} is peak floating-point rate, B_{mem} is effective memory bandwidth, and I is arithmetic intensity (FLOP/byte). This roofline governs design choices: increase on-chip storage to raise I , or increase compute and interconnect to keep pipelines fed.

Google TPU (family) implements large systolic matrix units and a unified on-chip buffer, optimizing for high utilization on dense GEMM and convolution kernels with reduced control complexity relative to SIMT GPUs. Key characteristics:

- Systolic arrays map tiles of A and B matrices to local MAC pipelines, maximizing data reuse and minimizing load/store pressure on the global memory subsystem.
- A software-managed unified buffer (scratchpad) enables tiling strategies where weights or activations remain on-chip for several accumulation steps, raising I .
- Integration with host interconnect and TPU pods uses parameter-server or AllReduce topologies for multi-device model-parallel and data-parallel scaling.

Cerebras wafer-scale engine pursues a different trade-off: massive spatial parallelism via hundreds of thousands of simple compute cores, each with local SRAM and a mesh interconnect. Operational implications:

- Local SRAM per core dramatically increases aggregate on-chip capacity, effectively pushing B_{mem} for short-term reuse and raising arithmetic intensity for large models without frequent DRAM access.
- The fabric supports model-parallel placements where entire layers or shard partitions remain on-chip, reducing synchronization overhead that would otherwise traverse an NVLink or PCIe boundary.
- Mesh routing latency and congestion control become first-order design concerns; software mapping optimizes placement to reduce long-tail hops.

Implementation note: to quickly assess whether a workload is memory- or compute-bound on a given accelerator, the following Python snippet computes the bottleneck using simple inputs.

Listing 10.5: Compute vs memory bound check

```
def bottleneck(P_peak, B_mem, arithmetic_intensity):
    # P_peak in FLOPS, B_mem in bytes/s, arithmetic_intensity in FLOP/byte
    sustained = min(P_peak, B_mem * arithmetic_intensity)
    return ("compute_bound" if sustained == P_peak else "memory_bound",
            sustained)

# example usage: TPU-like and Cerebras-like parameters (illustrative)
print(bottleneck(1e15, 1e12, 800)) # high intensity -> compute bound
print(bottleneck(1e15, 2e11, 100)) # lower BW or intensity -> memory bound
```

Implications and trade-offs:

- Systolic/TPU-like designs minimize control and maximize dense GEMM efficiency but require careful tiling; they struggle with irregular sparsity and fine-grained branching common in some RL workloads.
- Wafer-scale engines excel at large-model residency and coarse-grain model parallelism, reducing off-chip synchronization; they introduce routing congestion and yield/packaging complexity.
- For architects: prioritizing on-chip capacity and interconnect reduces B_{mem} pressure, while increasing MAC array depth raises P_{peak} ; balance depends on target arithmetic intensity and workload sparsity.

10.2.2 Graphcore IPU and Groq LPU

Following the TPU and Cerebras discussion, we now compare two commercially deployed, production-focused AI accelerators whose design points diverge from conventional SIMT GPUs: Graphcore's IPU (Intelligence Processing Unit) and Groq's LPU (Linear/Lightweight Processing Unit). Both prioritize graph-structured model execution and low-latency inference, but they trade programmer-visible parallelism, memory layout, and control differently than SM/CU-based GPUs.

Graphcore IPU and Groq LPU — problem and analysis

- Problem: maximize model throughput and minimize end-to-end latency for irregular, sparse, or dynamic graph workloads while keeping power and area efficient.
- Analysis focus: execution model (MIMD vs deterministic streaming), on-chip memory and interconnect, compiler scheduling, and implications for ML training versus inference.

Key architectural contrasts (operational relevance first)

1. Execution model and programmability

- IPU: MIMD-style tiles (independent program counters) allow fine-grained parallel tasks and asynchronous data movement. This matches workloads with irregular control flow (dynamic graphs, GNNs).
- LPU: compiler-constructed, statically scheduled pipeline with deterministic timing and minimal control overhead; excellent for low-latency, predictable inference on feed-forward transformers and CNNs.

2. Memory and dataflow

- IPU: large distributed local SRAM per tile and a low-latency exchange fabric enable keeping working sets on-chip, reducing external DRAM bandwidth pressure and improving arithmetic intensity.
- LPU: emphasizes streaming weights and activations through a deep compute pipeline; small local state but predictable access patterns allow simpler buffering and exact latency calculation.

3. Compiler and runtime

- IPU requires aggressive placement and asynchronous DMA scheduling to exploit data locality and hide communication latency.
- LPU shifts complexity to the compiler, which emits a fixed schedule; hardware is simpler but less tolerant of mis-scheduled variability.

Throughput model (symbolic) Let T be number of tiles/lanes, M the MACs per tile, and f the clock frequency. Peak multiply-accumulate throughput in FLOPS is:

$$[H]\text{FLOPS}_{\text{peak}} = 2 \cdot T \cdot M \cdot f \quad (10.6)$$

(IPU designs increase effective $T \cdot M$ via many small tile engines; LPU attains similar product by deep pipelining and high utilization on dense, scheduled workloads.)

Practical implementation: compute tile-block sizing under SRAM constraint

Listing 10.6: Tile blocking under per-tile SRAM constraint

```
def choose_tile_block(S_tile_bytes, m, n, k, bytes_per_elem=2):
    # compute max block sizes bm,bn,bk that fit: (bm*bk + bk*bn + bm*bn)*
    # elem_size <= S_tile
    # simple greedy: fix bk, maximize bm and bn equally
    for bk in range(min(k,256),0,-1):
        max_pair = (S_tile_bytes/bytes_per_elem)/(bk) # approx bm+bn+bm*bn/bk
        # term simplified
        bm = bn = int((max_pair)**0.5)
        if bm>=1 and bn>=1:
            return bm, bn, bk
    return 1,1,1 # fallback minimal blocking
# Example: S_tile_bytes = 64*1024 # use \lstinline|S_tile_bytes| in comments
# above
```

Implications and performance trade-offs

- IPU-style MIMD with large per-tile SRAM improves performance on sparse or control-heavy ML workloads but costs more complex inter-tile scheduling and larger area for distributed memory.
- LPU deterministic streaming reduces control and achieves ultra-low latency and power-efficient inference, but compiler scheduling becomes critical and hardware is less flexible for dynamic graphs.
- For GPU architects: integrating IPU-like local SRAM and exchange fabric into SM/CU clusters can raise arithmetic intensity; adopting LPU-style deterministic pipelines for critical inference paths can lower tail latency at the expense of generality.

10.2.3 Comparison with general GPUs

Building on the prior discussion of Graphcore’s fine-grained IPU messaging and the TPU/Cerebras emphasis on systolic and wafer-scale arrays, this subsection contrasts those domain-specific architectures against general-purpose GPUs across measurable metrics and real workloads. The goal is to show when a dedicated AI accelerator outperforms a GPU and why a GPU still remains the pragmatic choice for mixed graphics/compute environments.

Problem statement: designers must choose between specialization for ML kernels and the flexibility of SIMT GPUs when optimizing for throughput, latency, power, and programmability. Analysis proceeds by applying the roofline model to representative workloads and by examining microarchitectural differences that affect the model’s parameters.

Analysis: the roofline relationship captures the compute-memory tradeoff relevant to both classes of devices. Let F_{peak} be peak MAC throughput, B memory bandwidth, and I arithmetic intensity (FLOP per byte). Then

$$[H]\text{Achieved FLOPS} = \min(F_{\text{peak}}, I \cdot B) \quad (10.7)$$

Specialized accelerators increase effective F_{peak} per watt by using:

- dense systolic arrays or massive MAC banks with narrow control logic, improving area and energy per operation,
- on-chip weight SRAM and activation buffers to raise effective I , and
- native support for low-precision formats and structured sparsity to compress compute and memory footprints.

GPUs improve sustained performance across diverse kernels by offering:

- large register files and high-bandwidth shared memory to expose data reuse across warps,
- flexible tensor cores plus general ALUs allowing mixed FP32/FP16/BF16/INT8 execution, and
- unified memory and mature compiler toolchains that amortize software development cost.

Implementation example: a quick calculator shows the roofline effect for a transformer attention layer and a dense GEMM. The snippet below compares a hypothetical GPU and TPU.

Listing 10.7: Roofline comparison for two devices (toy example)

```
# compute achieved FLOPS using roofline model (toy values)
def achieved_flops(F_peak, bandwidth, intensity):
    return min(F_peak, intensity * bandwidth)

# GPU: flexible SIMT with tensor cores (values in TFLOPS, TB/s)
gpu = {'F_peak': 100.0, 'bandwidth': 1.0}    # 100 TFLOPS, 1 TB/s
tpu = {'F_peak': 250.0, 'bandwidth': 0.8}    # 250 TFLOPS, 0.8 TB/s

# transformer attention intensity ~ 40 FLOP/byte; GEMM ~ 200 FLOP/byte
print(achieved_flops(gpu['F_peak'], gpu['bandwidth'], 40)) # GPU attention
print(achieved_flops(tpu['F_peak'], tpu['bandwidth'], 40)) # TPU attention
```

Implications and trade-offs:

- For memory-bound inference tasks with low arithmetic intensity, large on-chip caches and memory-bandwidth improvements matter more than raw MAC density.
- For training or high-intensity GEMMs, accelerators with very high F_{peak} and tiled dataflows provide superior energy efficiency.
- GPUs remain preferable when workloads mix graphics, irregular control flow, or custom kernels requiring ISA flexibility and a mature software stack.
- Design choices—weight-stationary vs output-stationary dataflow, support for structured sparsity, and precision modes—directly shift the operating point on the roofline, affecting both peak attainable throughput and area/power efficiency.

10.2.4 Domain-specific optimization

Building on the comparison to general GPUs and the IPU/LPU designs, the focus here shifts to concrete domain-specific optimizations that close the efficiency gap between commodity SMs and specialized tensor engines. The previous analysis of routing, fine-grained scheduling, and deterministic execution informs which architectural levers yield the largest returns for ML inference and training.

Problem: modern DNN layers are dominated by regular matrix multiplies, but practical workloads show exploitable structure: blocked activations, quantized operands, and structured sparsity (a pattern where fixed groups of elements are zero, e.g., 2:4 sparsity). The engineering objective is to maximize effective utilization of tensor cores while minimizing memory transfers and control overhead.

Analysis: evaluate the performance bound with a roofline model that captures compute and bandwidth limits. Operational intensity I is FLOPs per byte transferred; peak achievable performance P follows

$$[H]P = \min(P_{\text{peak}}, I \cdot B), \quad (10.8)$$

where B is DRAM bandwidth and P_{peak} is tensor core throughput. Structured sparsity with compression factor s increases I approximately by s (assuming zero-values are not loaded), so memory-bound kernels can move toward the compute bound when $I \cdot B$ approaches P_{peak} . For example, a 2:4 sparsity pattern ($s \approx 2$) halves memory transfers for weights, raising I and proportionally lifting performance for memory-limited layers.

Implementation: domain-specific optimizations combine these techniques:

- Dataflow specialization: narrow, tiled systolic arrays per CU reduce internal buffering and exploit locality.
- Hardware zero-skipping for structured sparsity: stream a small mask with each tile to gate MAC operations.
- Mixed-precision datapaths and accumulator widening: INT8/FP16 multiply with FP32/INT32 accumulation in tensor cores.
- On-chip compression and decompression units adjacent to L1/L2 reduce interconnect traffic.

The following synthesizable Verilog demonstrates a compact, parameterized systolic PE that accepts a sparsity mask to skip MACs for quantized inference (INT16 operands). This module would be instantiated in a 2D array on an SM or CU tile.

Listing 10.8: Systolic PE with zero-skip mask (INT16 MAC)

```

module pe_zero_skip #(
  parameter WIDTH = 16
) (
  input wire          clk,           // clock
  input wire          rst_n,        // active-low reset
  input wire          valid_in,     // input valid
  input wire signed [WIDTH-1:0] a,  // left operand
  input wire signed [WIDTH-1:0] b,  // top operand
  input wire          mask,         // 1=valid MAC, 0=skip
  input wire signed [31:0] acc_in,  // incoming accumulator
  output reg signed [31:0] acc_out, // outgoing accumulator
  output reg          valid_out     // output valid
);

// simple pipeline: register inputs, perform conditional MAC
reg signed [WIDTH-1:0] a_r, b_r;
reg mask_r, v_r;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    a_r <= 0; b_r <= 0; mask_r <= 0; v_r <= 0;
    acc_out <= 0; valid_out <= 0;
  end else begin
    a_r <= a; b_r <= b; mask_r <= mask; v_r <= valid_in;
    // conditional multiply-accumulate (quantized MAC)
    if (v_r) begin
      if (mask_r)
        acc_out <= acc_in + $signed(a_r) * $signed(b_r);
      else
        acc_out <= acc_in; // skip multiply, forward accumulator
      valid_out <= 1'b1;
    end else begin

```

```

        acc_out <= acc_out;
        valid_out <= 1'b0;
    end
end
end
endmodule

```

Implications and trade-offs:

- Benefits: structured sparsity and zero-skipping reduce memory traffic and increase tensor core utilization, improving energy per inference.
- Costs: additional mask bandwidth and control logic increase area and slightly complicate scheduling across SMs and CUs.
- Design trade-offs: choose granularity of sparsity (e.g., 2:4 vs fine-grained) to balance compression efficiency and mask overhead; align tile size to L1/L2 capacity to maximize I under Equation (10.32).

10.3 Beyond Moores Law

10.3.1 Process scaling limits

Building on the architectural tradeoffs illustrated by recent GPU case studies and specialized accelerators, this subsection analyzes the physical and economic constraints that halt classical transistor scaling and how they reshape SM-level and chip-level design choices. The treatment proceeds from the bottleneck physics to quantified models and implementation strategies relevant to graphics, tensor, and ray-tracing workloads.

The core problem is that device feature-size reduction no longer delivers proportional gains in performance-per-watt. A compact analytic statement useful for architects is the power density relation:

$$[H]P_{\text{area}} = D \cdot C \cdot V^2 \cdot f, \quad (10.9)$$

where D is transistor density (transistors per unit area), C is switching capacitance per transistor, V is supply voltage, and f is switching frequency. Under ideal Dennard scaling C , V , and f scale down as D increases, keeping P_{area} roughly constant. In modern nodes V and f have plateaued due to leakage and reliability limits; thus P_{area} rises with D , forcing thermal throttling on large arrays of SMs or tensor cores.

Key physical limits and their microarchitectural impacts:

- Leakage and variability: subthreshold leakage and random dopant fluctuations increase static power and yield variation, raising the cost of large register files and wide port counts used by SIMT hardware. Threshold voltage scaling yields exponential leakage growth, making large always-on arrays (e.g., L1, register file) expensive in power.
- Interconnect and RC delay: wire resistance grows as cross-section shrinks, so global latency and energy become dominated by metal wires rather than transistor switching. This drives tile-based and chiplet partitioning to localize traffic and favor on-die routers and short traces between SMs and caches.

- Reliability and electromigration: higher current densities limit peak operating frequency and necessitate DVFS and conservative power delivery for ROP and memory controllers.
- Quantum and ballistic limits: below a few nanometers electron transport departs from diffusive models, complicating transistor models and limiting further V scaling.

Practical evaluation for architects often reduces to budgeting power per SM. The following Python snippet computes a simple upper bound on the number of SMs given a die power budget, per-SM dynamic and leakage estimates, and density scaling; use as a parametric tool when considering wider SMs or more tensor cores.

Listing 10.9: Simple SM count vs power budget model

```
def max_sms(power_budget_w, per_sm_dyn, per_sm_leak):
    # per_sm_dyn: dynamic power at target utilization (W)
    # per_sm_leak: static leakage per SM (W)
    return int(power_budget_w // (per_sm_dyn + per_sm_leak))

# example: 350 W card, dynamic 0.6 W/SM, leakage 0.05 W/SM
print(max_sms(350, 0.6, 0.05)) # compute max SMs fitting thermal/power budget
```

Design and performance implications:

- Chipletization and advanced packaging are mandatory to continue area scaling without catastrophic power density increases; this trades off inter-die latency and adds coherence complexity.
- Increasing specialization (tensor cores, RT cores) concentrates compute where energy per op is lowest, improving MFLOPS/W for ML and ray tracing at the cost of generality.
- Architectural levers—tile sizing, local SRAM banks, aggressive DVFS, and near-memory compute—reduce long-wire communication and energy but complicate software mapping and SM scheduling.
- Economic trade-off: nodes with marginal transistor-density gain but higher mask and fabrication cost push designers toward system-level scaling (more dies, coherency fabrics) rather than monolithic die scaling.

10.3.2 GAA and CFET transistor evolution

Building on the limits to planar node scaling, the industry shifted to device architectures that restore electrostatic control and pack more transistors per unit area; here we examine gate-all-around (GAA) evolutions and complementary FET (CFET) stacking and their direct consequences for modern GPU microarchitectures. The analysis ties device physics to SM/CU-level choices so architects can quantify tradeoffs between density, power, and performance.

GAA devices (nanosheet, nanowire, ribbon FETs) surround the channel with gate material, sharply reducing short-channel effects and improving subthreshold slope. This permits lower V_{dd} or reduced threshold voltage for the same leakage target, which benefits high-frequency tensor cores and TMUs by lowering dynamic energy per operation. CFETs vertically stack an nFET atop a pFET in a single fin or pillar, enabling near-doubling of functional transistor density and shorter local routing for logic that alternates polarity (e.g., SIMD datapaths, register files).

However, CFETs introduce thermal coupling between stacked devices and complicate contacts and back-end routing; these factors increase parasitic capacitance and may offset nominal area gains for SRAM macros and ROP clusters.

Operational relevance is captured by the basic delay and power relations used in floorplanning and DVFS modeling. For a first-order compute datapath element:

$$[H]\tau \approx \frac{C_{\text{load}} V_{\text{dd}}}{I_{\text{drive}}} \quad (10.10)$$

and, for long-channel approximation of drive current,

$$[H]I_{\text{drive}} \propto \mu_{\text{eff}} C_{\text{ox}} \frac{W_{\text{eff}}}{L} (V_{\text{dd}} - V_{\text{th}})^2. \quad (10.11)$$

GAA improves electrostatics and effective threshold control, which increases the margin on the right side of (2) or allows V_{dd} reduction in (1). CFET increases W_{eff} per footprint by vertical stacking, boosting aggregate drive per planar area but complicating heat removal.

To quantify architecture impacts, a compact model helps estimate how transistor density gains translate to additional SMs or tensor cores, and how leakage and dynamic power scale. The following Python snippet computes a simple projection for SM count and system power given device density improvements and per-transistor power assumptions.

Listing 10.10: Simple density vs. power projection for GAA/CFET device choices

```
# baseline parameters (example values)
baseline_trans_per_mm2 = 1e9      # transistors per mm^2 at reference node
area_per_SM_mm2 = 100.0          # mm^2 per SM (including local cache)
density_gain = 1.8                # e.g., CFET ~1.8x effective density
C_per_trans = 2e-15              # Farads per transistor (typical order)
f_core = 1.5e9                   # core frequency (Hz)
V_dd = 0.8                      # supply voltage (V)
leakage_per_trans = 1e-9         # static leakage (A)
# derived
new_trans_per_mm2 = baseline_trans_per_mm2 * density_gain
SMs_per_die = int((new_trans_per_mm2 * 1e-6) / (area_per_SM_mm2 *
    baseline_trans_per_mm2/1e6))
P_dyn_per_trans = C_per_trans * V_dd**2 * f_core      # dynamic per-transistor
P_static_per_trans = leakage_per_trans * V_dd
# totals
total_trans = new_trans_per_mm2 * 100.0              # example die area 100
    mm^2
P_total = total_trans * (P_dyn_per_trans + P_static_per_trans)
print("Projected SMs per die:", SMs_per_die)         # rough capacity
    estimate
print("Projected total power (W):", P_total)
```

Design implications and trade-offs:

- GAA enables lower V_{dd} or higher performance per transistor, reducing dynamic energy per FLOP for tensor cores, but die-level power density remains a thermal limiter at high SM counts.

- CFET gives near-2x area efficiency for logic-dominated blocks (e.g., SIMD datapaths, control logic), enabling more SMs or larger register files, at the cost of harder thermal management and more complex BEOL/contact routing for caches and SRAM.
- Floorplanning must balance transistor gains against increased parasitics: SRAM cell stability and ROP/cache latency may degrade if local routing and bitline capacitance grow.
- Architectures should prioritize moving bandwidth-sensitive blocks (HBM interfaces, memory controllers) to maintain balance: adding SMs without matching memory bandwidth yields compute-bound inefficiency.

10.3.3 Advanced packaging methods

These packaging options extend the discussion of transistor evolution and process limits by shifting scaling effort from transistor density to die-to-die integration and heterogeneous stacking. Advanced packaging thus becomes the primary lever for increasing aggregate bandwidth, reducing off-chip latency, and enabling heterogeneous SM/CU specialization when node scaling slows.

Advanced packaging methods solve three GPU problems: (1) providing high-bandwidth, low-latency connectivity between SM-rich logic dies and HBM stacks; (2) managing thermal and power delivery across multiple heterogeneous dies; (3) enabling economical reuse of IP blocks across process nodes. Analysis highlights trade-offs in electrical and thermal domains:

- Electrical: interposer and EMIB links reduce channel length, improving achievable data rate and energy per bit versus organic substrates. TSV-based stacking yields minimal latency but concentrates heat.
- Mechanical and thermal: 3D stacking increases junction temperature and complicates heat spreading, affecting SM frequency and reliability.
- Economic: chiplet reuse reduces NRE cost but increases system integration complexity and yields sensitivity.

Quantitative sizing must use bandwidth and energy models. For a multi-die GPU using N lanes per die pair, with single-lane gross rate R (Gb/s) and encoding efficiency η (fraction after 8b/10b or 64b/66b), aggregate effective bandwidth B_{eff} is

$$[H]B_{\text{eff}} = N \cdot R \cdot \eta. \quad (10.12)$$

Per-bit energy E_b and link power P_{link} then give link power $P = B_{\text{eff}} \cdot E_b$. A design target might constrain P per mm^2 of interposer area to meet cooling budgets.

Implementation-level considerations include:

- Choosing interposer type (silicon interposer vs embedded bridge) based on required lane density and thermal path.
- Placing high-bandwidth channels close to HBM interfaces and routing control/status lanes to reduce cross-talk.
- Designing die-to-die protocols with credit-based flow control to avoid head-of-line blocking while minimizing buffering.

A minimal synthesizable Verilog link controller below demonstrates a parameterized, credit-based transmitter/receiver pair suitable for chiplet links. It is intentionally simple to illustrate implementation trade-offs: depth versus latency, and credit width versus in-flight packet capacity.

Listing 10.11: Credit-based die-to-die link controller (simplified).

```

module chiplet_link_ctrl #(
    parameter LANES = 4,
    parameter DATA_WIDTH = 32,
    parameter FIFO_DEPTH = 8,
    parameter CREDIT_WIDTH = 8
) (
    input wire                clk,
    input wire                rstn,
    // TX side
    input wire [DATA_WIDTH-1:0] tx_data,
    input wire                tx_valid,
    output wire                tx_ready,
    output reg  [CREDIT_WIDTH-1:0] tx_credit_out, // advertise credits to
        remote
    // RX side
    output reg  [DATA_WIDTH-1:0] rx_data,
    output reg                rx_valid,
    input wire                rx_ready,
    input wire [CREDIT_WIDTH-1:0] rx_credit_in  // remote credits
);

    // Simple FIFO (circular buffer) for TX
    reg [DATA_WIDTH-1:0] tx_fifo [0:FIFO_DEPTH-1];
    reg [$clog2(FIFO_DEPTH):0] tx_wr_ptr, tx_rd_ptr;
    reg [$clog2(FIFO_DEPTH):0] tx_count;
    reg [CREDIT_WIDTH-1:0] rem_credits;

    // TX write
    always @(posedge clk) begin
        if (!rstn) begin
            tx_wr_ptr <= 0; tx_count <= 0;
        end else if (tx_valid && tx_ready) begin
            tx_fifo[tx_wr_ptr[$clog2(FIFO_DEPTH)-1:0]] <= tx_data; // write data
            tx_wr_ptr <= tx_wr_ptr + 1;
            tx_count <= tx_count + 1;
        end
    end

    // TX readiness: can accept new data if FIFO not full
    assign tx_ready = (tx_count < FIFO_DEPTH-1);

```

```

// Remotes credit update (simple model)
always @(posedge clk) begin
    if(!rstn) begin
        rem_credits <= 0;
        tx_credit_out <= FIFO_DEPTH; // advertise available buffer
    end else begin
        rem_credits <= rx_credit_in; // assume credit feedback arrives timely
        tx_credit_out <= FIFO_DEPTH - tx_count; // advertise local free slots
    end
end

// Transmit over LANES when credits available and FIFO not empty
always @(posedge clk) begin
    if(!rstn) begin
        tx_rd_ptr <= 0; rx_valid <= 0; rx_data <= 0;
    end else begin
        if (tx_count > 0 && rem_credits > 0) begin
            // send one beat per clock per LANES combined; simplified to
            DATA_WIDTH
            rx_data <= tx_fifo[tx_rd_ptr[$clog2(FIFO_DEPTH)-1:0]];
            rx_valid <= 1;
            tx_rd_ptr <= tx_rd_ptr + 1;
            tx_count <= tx_count - 1;
        end else if (rx_ready) begin
            rx_valid <= 0; // consumer accepted
        end
    end
end
endmodule

```

Design implications and trade-offs:

- Higher lane counts reduce per-lane rate but increase area and power; use Eq. (10.15) to balance N and R for thermal limits.
- Silicon interposers yield highest electrical density but raise cost and thermal challenges versus EMIB.
- Credit-based protocols lower buffer requirements but require reliable low-latency feedback paths to avoid stalling SM/CU throughput.
- TSV stacking minimizes latency but forces aggressive thermal management, often limiting peak SM frequency.

10.3.4 Economic and design impact

The transistor evolution and advanced packaging discussion showed why scaling now combines device innovation with heterogeneous integration. This subsection quantifies how those constraints translate into economic pressures and concrete GPU design choices.

When aggressive node scaling yields diminishing area and power benefits, architects trade die monolithicity for heterogeneous chiplets and advanced interposers. Operationally, decisions hinge on cost per good die, yield sensitivity to die area, and packaging overhead. The standard Poisson yield model gives first-order guidance:

$$[H]Y = e^{-AD} \quad (10.13)$$

where A is die area and D is defect density; cost per functioning die approximates

$$[H]C_{\text{die}} \approx \frac{C_{\text{wafer}}}{N_{\text{die}} Y} + C_{\text{pack}}, \quad (10.14)$$

with $N_{\text{die}} \approx \pi R^2/A$ the wafer capacity and C_{pack} packaging and test costs. These equations show exponential sensitivity of cost to die area, motivating chiplet partitioning for large GPUs with many SMs, tensor cores, RT cores, and memory PHYs.

To evaluate trade-offs numerically, designers simulate scenarios for monolithic versus chiplet layouts, including interposer cost and yield multiplicative effects. The following Python snippet computes cost-per-good-die for two configurations, keeping model parameters explicit for sensitivity studies.

Listing 10.12: Cost-per-good-die comparison for monolithic vs chiplet GPU designs.

```
import math

# wafer and defect model params
C_wafer = 10000.0    # $ per wafer
R = 150.0           # wafer radius mm
D = 0.5e-6          # defects per mm^2

def cost_per_good_die(area_mm2, C_pack):
    N_die = math.pi * R*R / area_mm2                # approximate dies
    # per wafer
    Y = math.exp(-area_mm2 * D)                     # Poisson yield
    return C_wafer / (N_die * Y) + C_pack              # $ per good die

# monolithic GPU
mono_area = 800.0    # mm^2
mono_pack = 50.0
print(cost_per_good_die(mono_area, mono_pack))        # cost monolithic

# chiplet GPU: smaller compute die plus IO die on interposer
chiplet_area = 200.0 # mm^2 per compute chiplet
num_chiplets = 4
io_die_area = 100.0
interposer_pack = 200.0 # includes advanced packaging cost
# cost includes all chiplets plus IO die and higher packaging
total_chiplet_cost = num_chiplets * cost_per_good_die(chiplet_area, 20.0) \
    + cost_per_good_die(io_die_area, 20.0) + interposer_pack
print(total_chiplet_cost) # total assembled GPU cost estimate
```

Implications and trade-offs:

- Larger monolithic dies increase per-die performance potential (lower inter-chip latency) but inflate C_{die} nonlinearly via Eq. (10.13).
- Chiplets reduce area-per-piece sensitivity to defects, enable heterogeneous process-node partitioning (e.g., high-density logic on advanced node, I/O and memory PHYs on mature nodes), and amortize NRE costs across SKUs.
- Advanced packaging (interposers, EMIB, silicon bridges) reduces latency versus discrete multi-die but increases C_{pack} and introduces power/thermal constraints inside the package.
- Performance trade-offs include inter-chiplet bandwidth and added latency, affecting SIMT warp scheduling, cache coherence across SMs, and tensor-core input staging.

10.4 Emerging Memory Technologies

10.4.1 HBM4 and next-gen DRAM

Building on the packaging and transistor-scaling trends discussed earlier, HBM4 and next-gen DRAM designs aim to sustain GPU-level memory growth by raising per-pin data rates, increasing parallelism and improving energy per bit. This subsection analyzes the architectural levers, models aggregate bandwidth, and shows a small tool to evaluate design trade-offs.

Problem and analysis. Modern SM-rich GPUs (tensor cores, many FP/INT units, TMUs and ROPs) are increasingly memory-bandwidth bound for graphics and AI workloads. The central problem is how to convert process, signaling and stacking advances into usable system bandwidth without prohibitive energy, latency or signal-integrity cost. Key levers are:

- Channel count and channel width partitioning to increase concurrency and bank-level parallelism.
- Per-pin signaling rate and modulation (NRZ vs PAM4) to scale raw throughput.
- PHY equalization, forward error correction (FEC), and training to maintain BER at high rates.
- Bank-group and pseudo-channel design to preserve row-buffer hit rates and reduce queuing latency.

A simple engineering model relates stack parameters to aggregate bandwidth. Let C be channels per stack, W the bits transferred per cycle per channel, and R the transfer frequency (transfers per second per bit lane). Effective aggregate bandwidth B (bytes/s) with efficiency η (protocol/encoding overhead) is

$$[H]B = \eta \cdot C \cdot W \cdot R/8. \quad (10.15)$$

This form lets architects trade off higher R (faster PHY) versus larger C and W (more parallel lanes) to hit a target B under power and area constraints.

Operational considerations and microarchitecture impact:

- Memory controller complexity grows: more channels and higher rates demand wider arbitration, command/address timing management, and per-channel training state.

- Latency vs throughput: adding channels reduces queuing and increases parallelism, but higher per-pin rates can increase worst-case latency when link retraining or FEC retransmissions occur.
- Power envelope: energy per bit E_b must drop to make higher aggregate BW feasible. System energy scales roughly as $E = E_b \cdot B$, so lowering E_b is as important as increasing R .
- Error management: integrated FEC or CRC with retransmit paths influences throughput and tail latency; GPUs with tight QoS (real-time display) need fast error-handling regimes.

Implementation tool. Use this Python snippet to explore parameter sensitivities; change C , W , R , η , E_b to see bandwidth and power trade-offs.

Listing 10.13: HBM stack bandwidth and power calculator.

```
# Simple HBM bandwidth/power model -- change params for scenarios.
def hbm_stats(channels=8, width_bits=128, rate_hz=3.2e9, eta=0.9, eb_j_per_bit
=2e-12):
    # B: bytes/sec from Eq. (1)
    B = eta * channels * width_bits * rate_hz / 8.0
    # P: power for data transfers
    power_W = eb_j_per_bit * channels * width_bits * rate_hz
    return B, power_W

# Example: evaluate a stack
bw, p = hbm_stats(channels=8, width_bits=128, rate_hz=4.0e9, eta=0.88,
eb_j_per_bit=1.5e-12)
print(f"Bandwidth (GB/s): {bw/1e9:.2f}, Data power (W): {p:.2f}") # quick
check
```

Design implications and trade-offs:

- Favor higher channel parallelism when row-buffer locality is high; favor per-pin rate increases when die area or interposer lanes are constrained.
- Prioritize PHY and coding techniques that reduce E_b before pushing raw R to limit thermal impact on GPU die and package.
- Memory-controller QoS, bank partitioning and request coalescing must evolve to exploit dozens of pseudo-channels without inflating latency tails.

10.4.2 Processing-in-memory concepts

HBM4's expanded logic layer and wider, lower-latency channels make moving some compute into the memory stack practical; building on that bandwidth increase, processing-in-memory (PIM) targets the dominant cost in modern GPU workloads: data movement between SMs (streaming multiprocessors) and off-chip DRAM.

Problem: ML training, tensor tiling, and sparse graph traversal are memory-bound on GPUs because fetching operands across the memory hierarchy consumes far more energy and time

than the arithmetic. Analysis shows that when the ratio of energy to transfer one 32-bit word to energy to perform one MAC exceeds a threshold, relocating MACs into or adjacent to DRAM reduces total system energy and improves end-to-end throughput. Define B_{mem} as effective bytes/s available to a memory partition and O_w as operations performed per loaded word. The PIM-limited throughput is:

$$[H]\text{Throughput}_{\text{PIM}} = \min(F_{\text{PIM}}, B_{\text{mem}} \times O_w) \quad (10.16)$$

where F_{PIM} is the compute peak of the in-memory engines. For GEMV-like kernels O_w is small, so PIM must expose enough F_{PIM} or increase O_w via tiling to be beneficial.

Implementation: practical PIM for GPUs takes a hierarchical approach.

- Place low-complexity vector MAC engines in the memory logic layer to execute reduction kernels, convolutions, or bit-serial primitives close to DRAM banks.
- Expose a simple command/doorbell interface that SMs use to submit offload jobs; the SM remains the scheduler and verifier, while PIM engines perform data-parallel reductions or prefetch-filtering.
- Maintain memory consistency by treating PIM writes as DMA with well-defined fences; integrate with the GPU's directory or coherence domain to avoid stale caches in L1/L2.

Example offload primitive: in-memory dot-product (streaming read, accumulate, write result). The following synthesizable Verilog implements a simple PIM MAC engine assuming a synchronous read port to local banked SRAM. It uses a start handshake, sequential address stepping, and a writeback interface.

Listing 10.14: PIM streaming MAC engine (synthesizable).

```

module pim_mac #(
  parameter ADDR_W = 16, DATA_W = 32, LEN_W = 16
) (
  input  wire      clk,
  input  wire      rst,
  input  wire      start,           // start command
  input  wire [ADDR_W-1:0] base_addr, // base of vector pair
  input  wire [LEN_W-1:0] length,    // number of elements
  // simple memory port A (vector A)
  output reg [ADDR_W-1:0] rd_addr_a,
  output reg            rd_en_a,
  input  wire [DATA_W-1:0] rd_data_a,
  // simple memory port B (vector B)
  output reg [ADDR_W-1:0] rd_addr_b,
  output reg            rd_en_b,
  input  wire [DATA_W-1:0] rd_data_b,
  // result writeback port
  output reg [ADDR_W-1:0] wr_addr,
  output reg            wr_en,
  output reg [DATA_W-1:0] wr_data,
  output reg            busy

```

```

);
reg [LEN_W-1:0] cnt;
reg signed [63:0] acc; // accumulator wider than DATA_W for safe MAC
localparam IDLE=0, READ=1, ACC=2, WRITE=3;
reg [1:0] state;
always @(posedge clk) begin
    if (rst) begin
        state <= IDLE; busy <= 0; rd_en_a<=0; rd_en_b<=0; wr_en<=0;
    end else begin
        case(state)
            IDLE: begin
                if (start) begin
                    cnt <= length;
                    rd_addr_a <= base_addr; rd_addr_b <= base_addr + length; // layout
                        : A then B
                    acc <= 0; busy <= 1;
                    state <= READ;
                end
            end
            READ: begin
                if (cnt != 0) begin
                    rd_en_a <= 1; rd_en_b <= 1;
                    state <= ACC;
                end else begin
                    rd_en_a <= 0; rd_en_b <= 0; state <= WRITE;
                end
            end
            ACC: begin
                rd_en_a <= 0; rd_en_b <= 0;
                acc <= acc + $signed(rd_data_a) * $signed(rd_data_b);
                rd_addr_a <= rd_addr_a + 1; rd_addr_b <= rd_addr_b + 1;
                cnt <= cnt - 1;
                state <= READ;
            end
            WRITE: begin
                wr_addr <= base_addr - 1; wr_data <= acc[DATA_W-1:0]; wr_en <= 1;
                state <= IDLE; busy <= 0;
            end
        endcase
    end
end
endmodule

```

Implications and trade-offs:

- Benefits: reduces bandwidth pressure on SM-L2 channels, lowers DRAM-to-core energy, accelerates low-arithmetic-intensity kernels like GEMV and sparse reductions.
- Costs: adds logic to memory stacks, complicates cache coherence and programming mod-

els, and provides limited ISA flexibility; overall system throughput follows Eq. (1) and depends on partitioned bandwidth, PIM compute, and job granularity.

10.4.3 Persistent and near-data memory

Following the processing-in-memory and HBM4 discussions, this subsection examines how byte-addressable persistent memory (PMEM) and near-data persistent fabrics alter GPU data paths and reliability models. It focuses on integrating persistent and near-data memory with SMs, L2, and memory controllers to accelerate large-model training, checkpointing, and low-latency parameter server patterns.

Problem: GPUs require low-latency, high-bandwidth persistent storage for model shards and checkpointing without stalling SIMT execution. Analysis shows two dominant integration points:

- Direct-attached PMEM behind the memory controller for large-capacity, byte-addressable access; and
- Near-data persistent accelerators (PIM+PMEM) exposing atomic, durable operations adjacent to memory banks.

Operational considerations before equations:

- Persistence latency (t_{persist}) is orders of magnitude higher than volatile write-back, so paths must avoid SM stalls.
- Endurance (cycles per cell) and wear amplification from write patterns determine usable lifetime for large parameter updates.

Quantitative model for endurance-based lifetime (years) given sustained writes:

$$[H]\text{Lifetime (years)} = \frac{E}{W \cdot A \cdot 3600 \cdot 24 \cdot 365} \quad (10.17)$$

where E is endurance cycles, W is average writes per second, and A is wear amplification factor from controller-level write amplification.

Implementation sketch: a persistent-write engine colocated in the memory controller schedules GPU-originated writes to PMEM without blocking warp progress. It uses DMA-style handshakes, batched cache-line flushes, and a persistence completion handshake back to the SMs so kernel-level durable barriers can be implemented with minimal stall.

A minimal synthesizable Verilog persistent-write engine demonstrates an FSM that issues a write, requests a persist, and waits for acknowledgement. This module is intended as a controller macro instantiated per memory channel.

Listing 10.15: Persistent write engine for memory-channel-level durability

```
module persist_write_engine #(
  parameter ADDR_WIDTH = 48,
  parameter DATA_WIDTH = 64
) (
  input wire          clk,
  input wire          rst,
```



```

    input wire          start,          // start persistence
        transaction
    input wire [ADDR_WIDTH-1:0] addr,
    input wire [DATA_WIDTH-1:0] data_in,
    output reg          busy,
    output reg          done,
    // memory-side interface
    output reg          mem_req,        // request write
    output reg          mem_persist_req, // request persistence/flush
    output reg [ADDR_WIDTH-1:0] mem_addr,
    output reg [DATA_WIDTH-1:0] mem_wdata,
    input wire          mem_ack,
    input wire          mem_persist_ack
);
    // FSM states
    localparam IDLE = 2'd0, WRITE = 2'd1, FLUSH = 2'd2, WAIT = 2'd3;
    reg [1:0] state, next_state;

    always @(posedge clk) begin
        if (rst) state <= IDLE; else state <= next_state;
    end

    always @(*) begin
        // default outputs
        mem_req = 1'b0; mem_persist_req = 1'b0;
        mem_addr = addr; mem_wdata = data_in;
        busy = (state != IDLE); done = (state == IDLE && start == 1'b0);
        next_state = state;
        case (state)
            IDLE: if (start) next_state = WRITE;
            WRITE: begin mem_req = 1'b1; if (mem_ack) next_state = FLUSH; end
            FLUSH: begin mem_persist_req = 1'b1; if (mem_persist_ack) next_state =
                WAIT; end
            WAIT: next_state = IDLE;
        endcase
    end
endmodule

```

Implications and trade-offs:

- Integrating PMEM at the memory-controller level minimizes PCIe/NVMe overhead but requires new durable-atomic ISA or runtime primitives on the GPU to avoid large SM stalls.
- Near-data persistence reduces data movement and increases effective throughput for model updates, but increases controller complexity and requires wear-leveling and ECC logic proximate to HBM/PMEM stacks.
- Designers must budget for increased latency on persistent commits; batching and asyn-

chronous acknowledgment preserve SIMT throughput while trading deterministic durability latency.

10.4.4 Memory-centric system design

These ideas build directly on persistent/near-data memory and processing-in-memory (PIM), tying hardware PIM primitives and persistent storage semantics into a cohesive memory-centric GPU design. The goal is to minimize high-latency, high-energy data transfers between SMs (streaming multiprocessors) and off-chip memory by relocating computation and control closer to HBM stacks and near-persistent fabric.

Problem: modern ML kernels and ray-tracing workloads are often memory-bound by off-chip bandwidth. Analysis begins with the roofline relation, which predicts attainable throughput as the minimum of compute peak and memory-delivered work:

$$[H]\text{Throughput} = \min(P_{\text{peak}}, B_{\text{eff}} \cdot I), \quad (10.18)$$

where P_{peak} is tensor/tile compute peak, B_{eff} is effective memory bandwidth after accounting for contention and protocol overhead, and I is arithmetic intensity (FLOPs per byte). Memory-centric design increases B_{eff} locally and raises effective I by performing reductions, compressions, or sparse-skipping inside the memory domain.

Analysis and architecture: adopt a hierarchical memory-side compute model with these components:

- Memory-side vector/ML units (lightweight SIMT-alike or systolic islands) inside HBM or die-stacked logic with direct fabric access to ROP/TMU regions for graphics tasks.
- Smart memory controllers that implement in-line primitives: scatter/gather, atomic reductions, and compression/decompression to reduce cross-chip traffic.
- A memory-coherent DMA/command interface allowing SMs to offload kernels to memory-side kernels and receive completion events without moving large operands.
- Page- or tile-granularity placement policies, leveraging persistence semantics where useful for checkpointing and fast restart.

Implementation example: a lightweight host-side API that issues an in-memory vector reduction task; the memory controller performs partial reductions and returns a scalar, avoiding full operand round trips.

Listing 10.16: Host code issuing a memory-side vector reduction (conceptual).

```
#include
// issue PIM reduction: addr -> HBM tile, len elements, op=SUM
int pim_reduce(void *addr, size_t len) {
    // build command packet (DMA descriptor) -- driver maps to memory
    controller
    struct cmd { uint64_t base; uint32_t len; uint8_t op; } c = { (uint64_t)
        addr, (uint32_t)len, 1 };
    // submit to memory controller queue (kernel call); returns job id
    int job = submit_memcmd(&c); // blocking submit for simplicity
    // wait for completion and fetch result (small scalar read)
```

```

    int64_t result = fetch_result(job);
    return (int) result;
}
// Usage: offload large reductions to HBM-local PIM units.

```

Operational implications and performance trade-offs:

- Area/power: embedding compute near memory increases die-stacked logic area and thermal density, reducing available SM area for peak FLOPS.
- Latency vs throughput: PIM reduces bulk-transfer latency and energy but adds added setup and scheduling latency; best for high-volume streaming operations.
- Programmability: exposes new APIs and requires compiler/runtime support for placement and fallback to SM execution.
- Coherency/consistency: tight coherence increases protocol complexity; pragmatic designs use explicit offload semantics and relaxed consistency.
- Granularity tuning: tile- or page-sized offloads balance metadata overhead against payload efficiency; choose granularity to maximize $B_{\text{eff}} \cdot I$ per Eq. (10.18).

10.5 Novel Computing Paradigms

10.5.1 Neuromorphic and optical computing

Building on the analysis of specialized accelerators and memory-centric designs earlier in this chapter, this subsection examines how neuromorphic and optical paradigms address the GPU scaling and energy limits encountered by tensor cores and large SM arrays. The goal is to show specific operational mappings to modern GPU subsystems and quantify trade-offs for integration.

Problem: high-throughput ML inference and sparse event-driven workloads are increasingly power- and bandwidth-bound on conventional SIMT SMs and tensor cores. Neuromorphic hardware promises event-driven sparsity and ultralow energy per inference. Optical accelerators promise high-throughput analog matrix multiply with data movement amortized by wavelength-division multiplexing. Analysis begins by expressing the canonical leaky integrate-and-fire (LIF) neuron in discrete form and showing how fixed-point updates map to GPU-friendly datapaths. The continuous model satisfies $\tau dV/dt = -V + RI$; a backward-Euler discretization yields

$$[H]V_{t+1} = \alpha V_t + \beta I_t, \quad \alpha = \frac{\tau}{\tau + \Delta t}, \quad \beta = \frac{R\Delta t}{\tau + \Delta t}, \quad (10.19)$$

which is implementable with integer Q-format arithmetic compatible with SM ALUs or small custom neuron CUs.

Implementation: two integration patterns are practical:

1. Neuromorphic islands as low-power CUs attached to the on-chip crossbar. Each island implements many LIF neurons in fixed-point and exposes a packetized spike interface to SMs for hybrid workloads. SMs perform plasticity updates on tensor-core-backed weight banks when heavy matrix ops are needed.

2. Photonic tensor cores offload dense GEMM. Optical multiply-accumulate is realized with interferometric meshes or microring weight banks; conversion between electrical and optical domains is buffered by high-bandwidth on-chip FIFO memory and optical interposers for chip-to-chip links.

Practical Verilog shows a synthesizable fixed-point LIF neuron suitable for replication inside a neuromorphic CU. Parameters use Q8.8 representation; the module emits spikes when membrane crosses threshold and resets.

Listing 10.17: Synthesizable Q8.8 LIF neuron (single cell)

```

module lif_neuron #(
  parameter integer THRESH = 16'd256, // Q8.8 threshold = 1.0
  parameter integer ALPHA  = 16'd240, // Q8.8 alpha ~0.9375
  parameter integer BETA   = 16'd16  // Q8.8 beta ~0.0625
) (
  input wire      clk,
  input wire      rst,
  input wire [15:0] I_q8_8, // input current Q8.8
  output reg      spike,
  output reg [15:0] V_q8_8 // membrane Q8.8
);
reg [31:0] prod1, prod2;
always @(posedge clk) begin
  if (rst) begin
    V_q8_8 <= 16'd0;
    spike  <= 1'b0;
  end else begin
    // fixed-point multiply Q8.8 * Q8.8 -> Q16.16; then shift back by 8
    prod1 = V_q8_8 * ALPHA; // Q16.16
    prod2 = I_q8_8 * BETA;  // Q16.16
    V_q8_8 <= (prod1 + prod2) >> 8; // back to Q8.8
    if (V_q8_8 >= THRESH) begin
      spike <= 1'b1;
      V_q8_8 <= 16'd0; // reset
    end else begin
      spike <= 1'b0;
    end
  end
end
endmodule

```

Implications and trade-offs:

- Energy: neuromorphic islands reduce energy per sparse inference by orders of magnitude versus SMs for event-driven workloads, but require on-chip spike routing and compact weight storage.
- Precision and programmability: optical GEMM achieves high throughput with limited analog precision; error correction and calibration increase system complexity and latency.

- Integration: placing neuromorphic CUs near L1/L2 reduces spike latency; photonic cores favor packaging and interposer solutions to minimize E/O conversion overhead.

10.5.2 Quantum-accelerated systems

The preceding survey of neuromorphic and optical accelerators highlighted alternative information encodings and near-data processing; quantum-accelerated systems complement those paradigms by offering provable algorithmic speedups for particular problems, and they require tight orchestration with classical GPU pipelines for practical workloads.

Problem: classical GPUs excel at dense linear algebra, SIMT execution, and tensor-core-accelerated neural training, but certain combinatorial optimization, sampling, and some linear-algebra subroutines have asymptotic quantum advantages. Integrating a quantum processing unit (QPU) into a GPU-centric system must minimize latency and maximize throughput while masking quantum coherence limits and high per-query overheads.

Analysis: model the end-to-end latency per quantum-accelerated call as the sum of host orchestration, data transfer, and QPU execution:

$$[H]T_{\text{total}} = T_{\text{host}} + T_{\text{comm}} + T_{\text{QPU}}, \quad (10.20)$$

where T_{host} includes GPU-side kernel launches (tensor-core matrix multiplies for parameter updates), T_{comm} is classical-to-QPU RPC and queuing, and T_{QPU} is circuit runtime plus error mitigation. Speedup requires $T_{\text{total}} < T_{\text{best_classical}}$, which motivates batching and amortization strategies: evaluate many circuit parameterizations per QPU pulse or co-locate pre- and post-processing on the same node to reduce T_{comm} . For variational hybrid workloads (VQE, QAOA), the inner loop is dominated by GPU-accelerated linear algebra and classical optimizers, while the QPU supplies expectation values; thus throughput scales with the product of GPU vectorization and QPU shot-rate.

Implementation: practical systems use a quantum-classical runtime that:

1. Batches circuit shots to reduce per-call overhead.
2. Streams inputs/outputs via DMA or shared memory to minimize copies.
3. Uses asynchronous queues and priority arbitration so SMs and tensor cores can proceed while QPU execution is underway.
4. Applies statistical error mitigation postprocessing on the GPU to exploit tensor cores for large-sample processing.

Example orchestration (Python sketch for hybrid VQE; comments are brief inline):

Listing 10.18: Hybrid GPU–QPU orchestration sketch

```
import torch                # GPU linear algebra / optimizer
from qiskit import QuantumCircuit
from qiskit_runtime import QPUClient  # hypothetical runtime client

qclient = QPUClient()        # manages QPU queue and batching (abstract)
model_params = torch.randn(100, device='cuda') # GPU-held parameters

def build_circuits(params_batch):
```

```

# build batch of parameterized circuits on host, minimal copies
circuits = []
for p in params_batch:
    qc = QuantumCircuit(4)
    # parameterized gates (placeholder)
    qc.rx(float(p[0].cpu().numpy()), 0)
    circuits.append(qc)
return circuits

for epoch in range(100):
    params_batch = model_params.chunk(8) # create 8 parameter sets
    circuits = build_circuits(params_batch)
    # asynchronous submit to QPU; returns futures
    futures = qclient.run_batch(circuits, shots=1024)
    # meanwhile run GPU pre/post work (tensor-core ops)
    loss = torch.matmul(model_params, model_params) # placeholder
    loss.backward()
    # collect QPU results and perform mitigation on GPU
    results = [f.result() for f in futures]
    # convert to tensors for batch reduction on GPU
    expectations = torch.tensor([r.expectation for r in results], device='cuda
    ')
    # optimizer step uses combined classical+quantum signal
    optimizer_step(expectations, model_params)

```

Implications and trade-offs:

- Co-scheduling reduces T_{comm} but increases runtime system complexity and resource contention on PCIe/NVLink.
- Batching improves amortized throughput but lengthens wall-clock time to first result, harming interactive workflows.
- Error rates and limited qubit counts make QPU acceleration problem-specific; only workloads with provable quantum advantage or large constant-factor gains justify integration.
- Architecturally, design choices include a low-latency host-QPU fabric, shared-memory staging for circuit parameters, and GPU-resident mitigation pipelines leveraging tensor cores for statistical postprocessing.

10.5.3 Approximate computation

The previous subsections showed how inherently noisy physical substrates, whether quantum superposition or spiking analog signals, relax the requirement for exact arithmetic. Approximate computation extends that theme by intentionally trading correctness for lower latency, area, and power in SMs, tensor cores, and other GPU datapaths when workloads tolerate bounded error.

Approximate computation addresses two problems: reducing critical-path length of wide arithmetic, and lowering energy per operation for throughput-bound kernels. Analysis begins

by quantifying worst-case numerical error and its effect on application-level metrics. A common technique is truncated-carry addition, where the least significant k bits remain accurate and the carry from that block is discarded before computing the upper bits. For an N -bit adder with truncation width k , the maximum absolute error $E_{\max}$ satisfies

$$[H]E_{\max} \leq 2^k - 1, \quad (10.21)$$

because discarding the carry can at most lose the value contributed by the lower k bits. This simple bound allows architects to trade precision against timing: removing long carry propagation shortens the adder critical path roughly proportional to the removed bit-length.

Implementation considerations for GPUs include where to place approximate units and how to expose them to software:

- Place configurable approximate adders or multipliers in tensor cores and in specialized SIMD lanes for ML inference, where models exhibit high noise tolerance.
- Expose ISA flags or separate instruction opcodes so compilers schedule approximate ops only where the algorithm permits.
- Provide hardware monitoring counters to measure quality-of-result (QoR) against baseline.

A synthesizable, production-ready Verilog example of a parameterized truncated-carry adder follows. It performs exact addition on the lower k bits and ignores carry into the upper block, reducing logic depth and power in the high block.

Listing 10.19: Parameterized truncated-carry adder: k lower bits exact, upper bits ignore incoming carry.

```
module trunc_adder #(
    parameter N = 32,                // total width
    parameter K = 8                  // preserved LSB width
) (
    input wire [N-1:0] a,
    input wire [N-1:0] b,
    output wire [N-1:0] sum
);
    // lower K-bit exact addition (produces carry_out but ignored)
    wire [K:0] low_sum = a[K-1:0] + b[K-1:0];           // K+1 bits
    // upper bits computed without carry_in to avoid long chain
    wire [N-K-1:0] high_sum = a[N-1:K] + b[N-1:K];      // no carry_in
    assign sum = { high_sum, low_sum[K-1:0] };           // concatenate
endmodule
```

Operational implications for GPU engineers:

- Clock/throughput: truncation reduces adder depth, enabling higher SM/Tensor Core frequency or simpler pipeline stages.
- Accuracy impact: equation (1) bounds worst-case error but real QoR depends on data distribution; use application-level validation.

- Area/power: smaller routing and simpler carry logic lower energy per operation, improving GFLOPS/W for inference and graphics post-processing.
- Software control: require compiler and driver support to confine approximate ops to tolerant kernels.

Trade-offs: selecting K trades deterministic numerical error for latency and power savings; choose K per workload class, validated by QoR metrics and hardware counters.

10.5.4 Stochastic and hybrid models

Following the discussion of approximate computation and quantum-accelerated systems, stochastic and hybrid models explore a middle ground: using probabilistic representations or noisy analog compute to reduce energy and area while retaining digital fallback paths for precision-critical phases. This subsection analyzes how stochastic encodings and hybrid analog–digital datapaths can be integrated into modern GPU pipelines (SMs, tensor cores, memory hierarchy) to accelerate graphics, Monte Carlo simulation, and ML inference.

Problem statement: many workloads (path tracing, MC simulation, Bayesian inference, some DNN layers) tolerate statistical error but are bottlenecked by memory bandwidth and energy cost of precise arithmetic. Stochastic computing encodes a scalar $p \in [0, 1]$ as a stream of Bernoulli bits; simple logic (e.g., AND) implements multiplication with very small logic area. Hybrid models combine such low-cost stochastic primitives or analog in-memory multiplication with deterministic accumulation and correction on tensor cores.

Analysis:

- Stochastic bitstreams: representing value by N random bits gives unbiased estimate with variance that scales inversely with length. The single-value variance is

$$[H]\text{Var}(\hat{p}) = \frac{p(1-p)}{N}, \quad (10.22)$$

so root-mean-square error decreases as $O(1/\sqrt{N})$. This tradeoff directly links energy/area (longer streams or repeated sampling) to statistical precision.

- For Monte Carlo workloads, overall estimator variance often dominates, so reducing per-operation precision can be acceptable. For ML, stochastic rounding in low-precision accumulation reduces quantization bias and can be implemented with cheap RNG units colocated in SMs or as a shared stochastic tile.

Implementation patterns for GPUs:

1. Hybrid tile on NoC: a small accelerator tile implements stochastic encoders (random bitstream generators), massively parallel bitwise operators, and decoders (pop-count+normalization). It communicates via the SM-to-memory crossbar and can offload repetitive kernels like elementwise multiplications or sampling for path tracing.
2. Analog PIM front-ends: in-memory multiply–accumulate arrays (resistive crossbars) produce noisy dot-products with high arithmetic intensity; digital tensor cores perform final high-precision accumulation and bias correction.

3. RNG provisioning: hardware Philox/Threefry counters or LFSR banks per warp to avoid SIMT divergence in random streams and to permit reproducible debugging.

Implementation example — a compact CUDA-style kernel showing stochastic averaging performed per warp (comments minimal). This kernel fits as an offloadable routine to an SM or to a hybrid stochastic tile; production implementations replace the RNG with a hardware counter-based stream.

Listing 10.20: Per-warp stochastic averaging kernel (conceptual)

```
extern "C" __global__ void warpStochAvg(float *out, uint64_t seed, int Nbits){
    int wid = (blockIdx.x*blockDim.x + threadIdx.x) / 32; // warp id
    uint32_t lane = threadIdx.x & 31;
    uint64_t state = seed ^ (wid<<32) ^ lane; // simple per-lane seed
    unsigned int pop = 0;
    for(int b=0;b> 7; state ^= state << 17;
        unsigned int bit = (state & 1u);
        pop += bit;
    )
    float p_hat = float(pop)/float(Nbits); // decode
    if(lane==0) out[wid]=p_hat; // one value per warp
}
```

Design implications and trade-offs:

- Area/power: stochastic logic and RNG banks are tiny relative to FP units, enabling energy savings for highly parallel, tolerant workloads; analog PIM trades area for dramatic energy/throughput but requires calibration and correction logic.
- Bandwidth/latency: stochastic representations reduce transfer size if compressed (run-length or popcount reduction), but long bitstreams increase latency—unsuitable for latency-sensitive ROP paths.
- Precision management: hybrid designs must include digital fallback/error correction (tensor cores or SM accumulators) and QoS controls to allocate deterministic resources when statistical error budgets are exceeded.
- Programmability: expose stochastic primitives and error bounds in APIs so schedulers place tolerant kernels on stochastic tiles and critical kernels on deterministic SM/tensor cores.

10.6 Section 6: Sustainability and Green Computing

10.6.1 Power efficiency trends

Building on the power- and thermal-management mechanisms discussed in Chapter 4 and the efficiency metrics from Chapter 18, this subsection analyzes dominant trends reducing GPU energy per useful operation and the architectural choices that enable them.

Problem: modern GPUs scale transistor count and memory bandwidth to meet ML and graphics demands, but total system power constrains deployment in data-centers and edge devices. Analysis begins by decomposing dynamic power and its relationship to operating points:

$$[H]P_{\text{dyn}} = \alpha CV^2 f, \quad (10.23)$$

where α is switching activity, C effective capacitance, V supply voltage, and f clock frequency. For sustained throughput, efficiency is often expressed as performance per watt:

$$[H]\eta = \frac{\text{GFLOPS}}{W} = \frac{\text{FLOPS per cycle} \times f}{P_{\text{dyn}} + P_{\text{static}}}. \quad (10.24)$$

Implementation strategies (architectural and system-level) that materially improve η include:

1. Specialized datapaths: tensor cores and mixed-precision units reduce arithmetic energy by computing more FLOPs per cycle per unit energy compared to general SM ALUs.
2. Data-movement reduction: higher on-chip reuse (larger register tiles, tiled GEMM), HBM channels, and compression (DCC) cut memory energy, often dominating total consumption for bandwidth-bound kernels.
3. Fine-grained power domains: per-SM/CU power gating and clock gating allow idle SIMT lanes and TMUs/ROPs to go dark, reducing α and static leakage.
4. Voltage-frequency scaling: workload-aware DVFS and per-domain near-threshold operation trade raw throughput for much lower energy per operation; caution is required for increased delay variation and error rates.
5. Algorithm-hardware co-design: sparsity-aware tensor cores and zero-skipping logic exploit structured sparsity (e.g., 2:4) to save energy by avoiding useless MACs.
6. Rendering optimizations: variable-rate shading and tile-based rendering reduce shader invocations, directly lowering SM/TMU utilization.
7. System integration: chiplet partitioning and localized power delivery reduce IR drop and converter losses, improving end-to-end energy efficiency.

To illustrate trade-offs, the following Python snippet estimates performance-per-watt for two precision modes and DVFS points; it is a simple engineering model for design-space exploration.

Listing 10.21: Estimate GFLOPS/W for DVFS and precision modes

```
# Simple perf/watt model; adjust params for design exploration
def perf_per_watt(flops_per_cycle, freq_hz, C, V, alpha, static_W):
    P_dyn = alpha * C * (V**2) * freq_hz
    flops = flops_per_cycle * freq_hz
    return (flops/1e9) / (P_dyn + static_W) # GFLOPS/W

# example points: tensor core (mixed) vs SM (FP32)
points = [
```

```

    {'name': 'tensor_fp16', 'flops_per_cycle': 512, 'freq': 1.5e9, 'C': 1e-9, 'V': 0.8,
     'alpha': 0.2, 'static': 20},
    {'name': 'sm_fp32', 'flops_per_cycle': 256, 'freq': 1.5e9, 'C': 1.2e-9, 'V': 1.0,
     'alpha': 0.3, 'static': 25},
]
for p in points:
    print(p['name'], perf_per_watt(p['flops_per_cycle'], p['freq'], p['C'], p['V'],
                                   p['alpha'], p['static']))

```

Design implications and trade-offs:

- Specialization (tensor cores, RT cores) increases GFLOPS/W but raises area and reduces generality; verify common workloads to justify silicon cost.
- Aggressive DVFS and near-threshold voltages give large energy gains but complicate timing closure, increase design margin needs, and can degrade yield.
- Memory-energy reductions (compression, HBM) pay large dividends for memory-bound kernels but add controller complexity and latency.
- Runtime telemetry and software-managed policies are essential; static hardware optimizations alone cannot capture workload variability.

10.6.2 Carbon footprint reduction

Building on power-efficiency trends, we now quantify and target the source of emissions rather than only reducing wattage. Concrete carbon accounting links GPU power states, data-center PUE, and grid carbon intensity to provide actionable levers for architects and operators.

Problem: compute-driven carbon emissions are the product of energy consumed and the carbon intensity of the electricity source. Analysis must therefore couple microarchitectural power management (DVFS, power gating, SM parking) with system-level scheduling and site energy profiles. A simple operational model for a single GPU instance is:

$$[H]C \text{ (kgCO}_2\text{)} = P_{\text{device}} \text{ (W)} \cdot t \text{ (h)} \cdot \frac{1}{1000} \cdot \text{CI (kgCO}_2\text{/kWh)} \cdot \text{PUE}, \quad (10.25)$$

where PUE is power-usage-effectiveness and CI is time-varying grid carbon intensity. This formula makes two implications explicit: (1) reducing P_{device} via architectural measures yields linear carbon reduction; (2) scheduling to low-CI windows multiplies the effect.

Implementation approaches that materially lower C include:

- Fine-grained power gating: park entire SMs or CUs when utilization is low, reducing static leakage and lowering active power.
- Workload consolidation and batching: fit kernels to fewer GPUs to enable cold power-gating of idle boards.
- Carbon-aware job scheduling: shift flexible ML training or batch renders into low-CI intervals using grid forecasts.
- DVFS knob coordination: optimize frequency/voltage per SM to operate at the knee of energy-per-op for the given kernel.

- Renewable-backed provisioning: integrate site-level CI telemetry and on-site renewables to preferentially route work.

A practical scheduler sketch that consolidates jobs and delays flexible work until low-CI windows follows. It assumes per-GPU utilization telemetry and grid CI forecast.

Listing 10.22: Carbon-aware GPU job scheduler (simplified)

```
def schedule_jobs(jobs, gci_forecast, gpus): # gci_forecast: list of (hour,
kgCO2/kWh)
    # prioritize latency-sensitive jobs; batch flexible jobs to low-carbon
    windows
    urgent, flexible = partition(jobs, key=lambda j: j.deadline) # simple
    split
    assign_urgent(urgent, gpus) # place to meet latency SLAs
    low_ci_hours = select_low_ci_hours(gci_forecast, window=4) # choose 4h
    window
    for job in flexible:
        slot = find_consolidation_slot(job, low_ci_hours, gpus)
        if slot:
            place_job(job, slot) # pack onto few GPUs
            park_unused_gpus(gpus) # power-gate idle cards
        else:
            place_job(job, best_effort(gpus))
```

Operational metrics to instrument: per-SM power, wake/sleep latency, job carbon estimate, and aggregated C per training epoch. Use these to validate equation (1) against observed emissions.

Design implications and trade-offs:

- Power-gating granularity: finer granularity yields greater idle savings but increases area and wake-latency complexity (affects short kernels).
- DVFS aggressiveness: shifting frequency lowers energy but can increase time-to-solution, so carbon-optimal point may reduce throughput.
- Scheduling latency vs carbon: batching reduces carbon but increases job latency; acceptable for background ML training, not for interactive graphics.
- Infrastructure coupling: PUE reduction (liquid cooling, waste-heat reuse) multiplies CPU/GPU optimizations, but requires facility investment.

10.6.3 Lifecycle optimization

Building on carbon-footprint accounting and runtime power-efficiency techniques, lifecycle optimization extends the focus from instantaneous watts to total environmental and economic cost across manufacture, operation, and retirement. The goal is to minimize carbon per useful unit of work by design choices in hardware modularity, runtime management (DVFS, power gating), and policies for refurbishment and reuse.

Problem — GPUs incur embodied carbon C_{emb} during fabrication and operational emissions proportional to energy consumption. Analysis quantifies total lifecycle carbon as the sum of

embodied and operational contributions; with an average power P_{avg} and lifetime L (hours) under an emissions factor EF ($\text{kgCO}_2 \text{ kWh}^{-1}$), a simple model is

$$[H]C_{\text{total}} = C_{\text{emb}} + \text{EF} \int_0^L P(t) dt \approx C_{\text{emb}} + \text{EF} \cdot P_{\text{avg}} \cdot L. \quad (10.26)$$

More useful is per-work carbon: amortize C_{emb} over total useful work N (e.g., rendered frames or training FLOPs) so

$$[H]C_{\text{per_work}} = \frac{C_{\text{emb}}}{N} + \text{EF} \cdot E_{\text{work}}. \quad (10.27)$$

This formulation makes explicit tradeoffs: increasing reuse (larger N) or reducing E_{work} via higher arithmetic intensity (tensor core tiling, on-chip reuse) lowers lifecycle impact.

Implementation — hardware features and firmware policies that materially reduce P_{avg} and increase N include:

- Fine-grained power gating for SM/CU banks and memory islands to remove leakage during idle windows.
- Multiple DVFS points per SM cluster combined with fast voltage regulators to track utilization from SIMT workloads.
- Telemetry counters (energy, utilization, temperature) fed to schedulers so the runtime can batch or migrate workloads to optimal SM groups for energy-aware placement.
- Chiplet modularity allowing ROP/TMU or tensor-core chiplets to be replaced or upgraded, increasing N .

Practical control logic must be synthesizable and low-latency; the following Verilog implements a compact per-cluster power-mode controller that exercises DVFS and power gating based on utilization and inactivity time. It is production-ready, synchronous, and parameterized.

Listing 10.23: Per-cluster power-mode controller (synthesizable).

```

module power_mode_controller #(
  parameter IDLE_THRESH = 1000, // cycles before idle
  parameter WAKE_LATENCY = 50  // cycles to ramp up
) (
  input wire      clk, rst_n,
  input wire [7:0] util,      // utilization percent (0-100)
  input wire [15:0] temp,     // temperature sensor
  output reg [2:0] dvfs_level, // 0=lowest..7=highest
  output reg      pg_enable  // 1 = power on, 0 = gated
);

  // simple FSM states
  localparam S_OFF = 2'd0, S_SLEEP = 2'd1, S_ACTIVE = 2'd2;
  reg [1:0] state, next_state;
  reg [15:0] idle_ctr, wake_ctr;

  // state transition combinational
  always @(*) begin

```

```

next_state = state;
case (state)
  S_ACTIVE: if (util < 8) next_state = S_SLEEP;
  S_SLEEP:  if (util >= 16) next_state = S_ACTIVE;
             else if (idle_ctr > IDLE_THRESH) next_state = S_OFF;
  S_OFF:    if (util >= 8) next_state = S_SLEEP;
  default: next_state = S_OFF;
endcase
end

// sequential: counters, outputs
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    state <= S_OFF; idle_ctr <= 0; wake_ctr <= 0;
    dvfs_level <= 3'd0; pg_enable <= 1'b0;
  end else begin
    state <= next_state;
    // idle/wake counters
    if (state == S_ACTIVE && util < 8) idle_ctr <= idle_ctr + 1;
    else if (state != S_ACTIVE) idle_ctr <= idle_ctr + 1;
    else idle_ctr <= 0;

    if (state == S_OFF && util >= 8) wake_ctr <= wake_ctr + 1;
    else wake_ctr <= 0;

    // outputs: simple DVFS mapping and pg control
    case (next_state)
      S_ACTIVE: begin pg_enable <= 1'b1; dvfs_level <= (util > 80)?7:
                                     (util > 50)?5:
                                     (util > 20)?3:1; end
      S_SLEEP:  begin pg_enable <= 1'b1; dvfs_level <= 3'd0; end
      S_OFF:    begin pg_enable <= 1'b0; dvfs_level <= 3'd0; end
    endcase
  end
end
endmodule

```

Implications and trade-offs:

- Benefits: lower lifecycle carbon per work via reduced P_{avg} and longer useful life through modular replacement; enables workload consolidation for higher utilization.
- Costs: area and design complexity for power switches, voltage regulators, and telemetry; wake latency impacts real-time graphics responsiveness and must be bounded; increased firmware/software scheduling complexity. Quantitative optimization uses (1)–(2) to choose gate granularity and reuse targets balancing embodied carbon amortization against performance and cost.

10.6.4 Renewable-powered data centers

Building on lifecycle optimization and carbon accounting, renewable-powered data centers apply temporal and spatial controls to minimize operational emissions while preserving GPU throughput. Practical strategies combine carbon-aware job scheduling, local energy storage, and hardware-level power management across SMs and tensor cores to align demand with renewable supply.

Problem: schedule latency-tolerant GPU workloads (batch training, large-scale render jobs) to times or sites with low grid carbon intensity while satisfying deadlines and maintaining high SM utilization. Analysis begins by modeling emissions as the integral of energy draw and grid carbon intensity. For discrete time slots this is

$$[H]\mathcal{C} = \sum_{t=1}^T E_t \cdot \text{CI}_t \quad (10.28)$$

where E_t is facility energy consumption at slot t and CI_t is grid carbon intensity ($\text{gCO}_2 \text{ kWh}^{-1}$). Renewable+storage systems change the effective CI_t seen by the load via stored energy dispatch and curtailment avoidance.

Implementation-level controls:

- Demand shaping: delay non-critical kernels or split epochs of training into windows where predicted CI_t is low. Use checkpointing and asynchronous gradient accumulation to preserve model convergence while time-shifting work.
- Geographic load balancing: migrate containerized workloads between sites with contrasting solar/wind forecasts. Network and PCIe/NVLink transfer costs must be accounted as energy overhead.
- Fine-grain GPU power management: combine DVFS, SM power gating, and tensor core scheduling to reduce E_t without incurring large latency penalties. For ML inferencing, reduce active SM warps or downclock memory channels temporarily.
- Storage dispatch optimization: model battery state-of-charge (SoC) with constraints and include charging losses. The dispatch minimizes Equation (1) subject to SoC dynamics.

A simple near-term scheduler can use forecasted CI and energy budgets to place batch jobs greedily. Example Python implementation (production scheduler would include failure modes, security, and resource arbitration):

Listing 10.24: Greedy carbon-aware batch scheduler

```
def schedule_jobs(jobs, ci_forecast, energy_budget):
    # jobs: list of (energy_kWh, deadline_slot)
    # ci_forecast: array of CI per slot
    # energy_budget: array of available renewable kWh per slot
    schedule = {}
    for job in sorted(jobs, key=lambda j: j[1]): # earliest deadline first
        e, dl = job
        # pick slot <= dl with max renewable fraction
        candidates = [(s, min(energy_budget[s], e)) for s in range(dl+1)]
        # score by low CI and available energy
```

```

best = max(candidates, key=lambda x: (energy_budget[x[0]]/ (
    ci_forecast[x[0]]+1e-6)))
if best[1] >= e:
    schedule[job] = best[0]
    energy_budget[best[0]] -= e
return schedule

```

Implications and trade-offs:

- Energy-aware scheduling reduces operational emissions but can increase job completion time and network energy due to migration.
- Storage smooths renewables but adds capital, round-trip losses, and lifecycle emissions.
- Aggressive DVFS and SM gating save power but may reduce peak GFLOPS and raise latency tail, impacting SLA-bound workloads.

Designers must quantify CI forecasts, migration costs, and GPU occupancy sensitivity to find the Pareto frontier between emissions and performance.

10.7 Section 7: Research Frontiers

10.7.1 AI-driven hardware design

Building on the exploration of specialized AI accelerators and memory-centric designs, we now examine how machine learning methods close the loop between workload characteristics and GPU microarchitecture choices, enabling automated co-design of SM layouts, tensor cores, and memory hierarchies.

The problem: manual microarchitectural tuning for modern GPUs—balancing SIMT throughput, tensor-core throughput, TMU/ROP bandwidth, and thermal/power envelopes—is intractable across the combinatorial design space. Analysis shows that the optimization is a constrained, multi-objective search over discrete and continuous variables where each candidate requires expensive cycle-accurate simulation or silicon measurements. Practical AI-driven methods therefore replace or augment simulators with learned surrogates, use differentiable relaxations of discrete choices, and apply RL or Bayesian search to find Pareto-optimal designs.

A common operational formulation minimizes a weighted cost combining latency, power, and area under timing and correctness constraints:

$$[H]\mathcal{L}(\mathbf{x}) = \alpha \cdot \text{Latency}(\mathbf{x}) + \beta \cdot \text{Power}(\mathbf{x}) + \gamma \cdot \text{Area}(\mathbf{x}) + \mathcal{P}(\mathbf{x}), \quad (10.29)$$

where $\mathbf{x}$ encodes microarchitecture (SM count, register file banks, tensor-core width, L1 size, memory interconnect parameters) and $\mathcal{P}$ is a penalty for timing violations or constraint breaches. Differentiable surrogates s_ϕ approximate metrics, allowing gradient-based proposals: $\nabla_{\mathbf{x}} s_\phi(\mathbf{x})$ guides continuous relaxations of discrete choices (e.g., softmax over configuration options).

Implementation approaches that have shown practical success:

- Surrogate modeling: train neural predictors for latency/power from a compact feature vector of architectural knobs and workload fingerprints (e.g., GEMM tile sizes, texture access patterns).

- Reinforcement learning: policy networks propose placement of functional blocks or floorplan sequences to optimize (10.29), using fast approximators for reward estimation.
- Bayesian optimization with multi-fidelity evaluations: use cheap analytic models or low-accuracy sims to prune candidate sets before full RTL+timing flows.
- Differentiable architecture search (DARTS-like): relax integer choices to continuous parameters, then discretize best candidates, keeping SM-level constraints to preserve SIMT scheduling properties.

A compact surrogate-driven architecture proposal loop in PyTorch (illustrative and easily adapted to GPU features) follows:

Listing 10.25: Surrogate-driven architecture proposer

```
import torch, torch.nn as nn, torch.optim as optim
# simple surrogate: maps arch vector x -> latency estimate
class Surrogate(nn.Module):
    def __init__(self, n): super().__init__(); self.net=nn.Sequential(nn.
        Linear(n,64),nn.ReLU(),nn.Linear(64,1))
    def forward(self,x): return self.net(x).squeeze()
# x: continuous arch params (e.g., SM_count, TF_width, L1_kb) scaled
surrogate=Surrogate(n=6); opt=optim.Adam(surrogate.parameters(),lr=1e-3)
for epoch in range(200): # train on measured data (
    omitted)
    loss = ... # MSE to measured latency
    opt.zero_grad(); loss.backward(); opt.step()
# propose new arch by gradient descent on surrogate + penalty (toy example)
x = torch.randn(6, requires_grad=True) # initial proposal
for it in range(50):
    pred = surrogate(x)
    penalty = 0.1*torch.relu(x-10).sum() # enforce upper bounds
    obj = pred + penalty
    obj.backward()
    with torch.no_grad(): x -= 1e-2 * x.grad; x.grad.zero_()
# x rounded to nearest legal configuration before RTL evaluation
```

Design implications and trade-offs:

- Surrogates reduce wall-clock search time by orders of magnitude but introduce model bias; cross-validate with multi-fidelity flows.
- Differentiable relaxations enable continuous optimization but require careful discretization to preserve SIMT semantics and timing.
- RL and Bayesian methods excel in placement/floorplanning; surrogate accuracy and reward shaping determine convergence speed.
- Practical adoption demands integration with RTL, timing tools, and thermal models to avoid proposals that violate physical constraints.

10.7.2 Self-optimizing microarchitectures

The previous discussion showed how ML can guide architectural choices offline and during design-space exploration; self-optimizing microarchitectures extend that capability by closing the loop at runtime, letting each SM (streaming multiprocessor) or CU (compute unit) adapt hardware knobs continuously to workload behavior.

Problem: workloads across graphics, ray tracing, and ML kernels vary in arithmetic intensity, memory locality, and divergence, so a static allocation of frequency, thread slots, and TMU/ROP bandwidth creates persistent inefficiency. **Analysis:** treat per-SM adaptation as a constrained multi-objective control problem where the controller observes short-window performance counters and issues resource adjustments with low latency and bounded overhead. Formulate the objective as minimizing a weighted sum of power and latency while meeting throughput SLOs:

$$[H] \min_{u(t)} \alpha P(u(t)) + \beta L(u(t)) \quad \text{s.t.} \quad T(u(t)) \geq T_{\text{req}}, u(t) \in \mathcal{U}, \quad (10.30)$$

where $u(t)$ are discrete controls (frequency level, active-warp cap, tensor-core tiling mode), P is power, L is tail latency, and T is achieved throughput.

Implementation: practical, low-cost controllers use:

- lightweight estimators (EWMA or small Kalman filters) over hardware counters (warp stalls, memory throughput, tensor core occupancy),
- hysteretic thresholding for discrete DVFS states to avoid flapping,
- per-SM policy tables derived from offline RL or Bayes-optimized points, compressed into LUTs for on-chip lookup,
- a hierarchical actuator: per-SM fast control for warps and SM clocks; a higher-level governor aggregates to domain DVFS.

The following synthesizable Verilog implements an SM-local controller that computes an EWMA on a 16-bit performance counter and issues a 2-bit DVFS request plus a warp-cap signal. It is intentionally simple so it can be replicated per SM with low area and timing cost.

Listing 10.26: SM-local self-optimizing controller (synthesizable)

```
module sm_selfopt_ctrl #(
    parameter EWMA_SHIFT = 4,           // decay factor 1/16
    parameter TARGET = 16'd1000,       // target perf units per epoch
    parameter HYST_UP = 2'd3,          // hysteresis up threshold
    parameter HYST_DOWN = 2'd1         // hysteresis down threshold
) (
    input wire      clk,
    input wire      rst_n,
    input wire [15:0] perf_cnt,        // sampled counter (e.g., useful_inst)
    input wire [15:0] power_budget,
    output reg [1:0] dvfs_req,         // 0..3 frequency levels
    output reg [7:0] warp_cap          // max active warps
);
    reg [15:0] ewma;                   // fixed-point EWMA
```

```

reg [1:0] up_vote, down_vote; // simple hysteresis votes

// EWMA update: ewma <= (15/16)*ewma + (1/16)*perf_cnt
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) ewma <= 16'd0;
    else ewma <= ewma - (ewma >> EWMA_SHIFT) + (perf_cnt >> EWMA_SHIFT);
end

// Simple policy: compare ewma to TARGET, adjust dvfs with hysteresis.
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        dvfs_req <= 2'd0;
        up_vote <= 2'd0;
        down_vote <= 2'd0;
        warp_cap <= 8'd32;
    end else begin
        // voting to avoid transient changes
        if (ewma > TARGET + HYST_UP) up_vote <= up_vote + 1'b1; else up_vote <=
            2'd0;
        if (ewma < TARGET - HYST_DOWN) down_vote <= down_vote + 1'b1; else
            down_vote <= 2'd0;

        // adjust DVFS
        if (up_vote == 2'd3 && dvfs_req != 2'd3) dvfs_req <= dvfs_req + 1'b1;
        else if (down_vote == 2'd3 && dvfs_req != 2'd0) dvfs_req <= dvfs_req -
            1'b1;

        // warp cap tied to dvfs and power budget (simple affine mapping)
        warp_cap <= (power_budget < 16'd200) ? 8'd16 : (dvfs_req==2'd3 ? 8'd48 :
            8'd32);
    end
end
endmodule

```

Implications and trade-offs:

- Latency of adaptation must be shorter than workload phase length; too-fast control increases oscillation, too-slow loses efficiency.
- Area and timing budgets favor simple estimators and LUT policies over full RL agents on-chip; off-chip training with on-chip execution is a common compromise.
- Per-SM autonomy improves locality but complicates global power coordination; hierarchy (SM-local + global arbiter) balances responsiveness and system-wide constraints.

10.7.3 Secure and open GPU initiatives

These ideas build directly on the last subsections: self-optimizing microarchitectures and AI-driven hardware design informed hardware-level telemetry and adaptive policies that also

change the security and openness posture of a GPU. Secure and open initiatives must therefore be co-designed with adaptive schedulers and learned optimization loops to avoid introducing exploitable side channels.

Problem: modern GPUs expose a large trusted computing base (TCB) spanning firmware, microcode, driver stacks, and debug interfaces. Attack surfaces include SM state, register files, shared memory, texture memory and caches, MMU/TLB, DMA engines, and external interconnects. Quantifying exposure helps prioritize mitigations; define a simple weighted attack-surface metric where each component i has exposure s_i in $[0, 1]$ and weight w_i representing criticality.

$$[H]A = \sum_i w_i s_i \quad \text{with} \quad 0 \leq A \leq \sum_i w_i. \quad (10.31)$$

Reducing A requires lowering s_i (hardening, isolation) or w_i (reducing critical dependency).

Analysis: hardening strategies must be compatible with SIMT execution and low-latency MMU/TLB operations. Key approaches:

- Hardware-enforced context isolation: per-CU context IDs propagated through caches, TLBs, and DMA engines to prevent cross-context leakage.
- Memory encryption and integrity (MEE): authenticated encryption at DRAM boundaries to protect off-chip confidentiality and integrity, with a decryption/verification pipeline staged to avoid stalling SMs.
- Minimal trusted firmware: signed microcode and measured boot reduce attack scope of device firmware and RT core microcode.
- Open-source critical RTL: selectively open-sourcing SM front-end, memory controller metadata paths, or debug bridges enables third-party audits while keeping proprietary ALU/tensor core optimizations closed.

Implementation example: a compact, synthesizable hardware policy checker that enforces per-context address ranges for DMA/SM requests. The module below is intended as an insert in the MMU path and shows practical integration points (context tagging, range compare, fault signal).

Listing 10.27: Per-context memory region checker (synthesizable).

```
module mem_protection #(
    parameter ADDR_W = 32,
    parameter CTX_W  = 8,
    parameter N_RULE = 4
) (
    input  wire      clk,
    input  wire      rst,
    input  wire [CTX_W-1:0] ctx_id,      // context tag
    input  wire [ADDR_W-1:0] req_addr,    // request address
    input  wire      req_valid,
    output reg       grant,
    output reg       fault
);
// simple rule table: base/limit per context index (synthesizable RAM)
```

```

reg [ADDR_W-1:0] base    [0:N_RULE-1];
reg [ADDR_W-1:0] limit  [0:N_RULE-1];
integer i;
// rule init (synthesizable constants, replace with config path)
initial begin
    base[0]=32'h0000_0000; limit[0]=32'h0FFF_FFFF;
    base[1]=32'h1000_0000; limit[1]=32'h1FFF_FFFF;
    base[2]=32'h2000_0000; limit[2]=32'h2FFF_FFFF;
    base[3]=32'h3000_0000; limit[3]=32'h3FFF_FFFF;
end
always @(posedge clk) begin
    if (rst) begin grant <= 1'b0; fault <= 1'b0; end
    else if (req_valid) begin
        // simple match: ctx_id selects rule index (low bits)
        i = ctx_id % N_RULE;
        if (req_addr >= base[i] && req_addr <= limit[i]) begin
            grant <= 1'b1; fault <= 1'b0;
        end else begin
            grant <= 1'b0; fault <= 1'b1;
        end
    end
end
endmodule

```

Implications and trade-offs:

- Latency: MEE and policy checks add pipeline stages and per-transaction latency δ ; persistent workloads with high DRAM transaction rates see throughput drop roughly proportional to added service time.
- Area and power: context-tagging metadata and integrity engines cost area and dynamic power; weight against reduced software-level mitigation costs.
- Performance interactions: tighter isolation increases TLB pressure and can reduce SM occupancy; smart eviction and co-design with self-optimizing schedulers mitigate this.
- Openness vs IP risk: selective open RTL enables verification of critical paths (MMU, DMA control, debug bridge) while protecting high-value cores (tensor cores, compiler backends).

Concrete design implication: target isolating high-weight components first (MMU, DMA, debug), measure A per (1) and prioritize mitigations that reduce s_i with minimal δ latency.

10.7.4 Future scalability challenges

Both secure/open GPU initiatives and self-optimizing microarchitectures expose new axes for scaling: heterogeneous blocks require coherent management, and adaptive hardware pushes limits of telemetry and control. These trends compound classical scalability bottlenecks—power, memory, and connectivity—so the architecture must be evaluated holistically.

The practical problem is balancing sustained throughput across SMs, tensor cores, RT cores, and I/O while respecting power and thermal envelopes. Roofline analysis remains a compact way to reason about aggregate limits; operational intensity I (FLOPs per byte) determines whether a kernel is compute- or memory-bound. Operational relevance: designers must provision memory subsystems and interconnects to shift common kernels into the compute-bound regime where SMs and tensor cores are fully utilized. Formally,

$$[H]P(I) = \min\{C_{\text{peak}}, B \cdot I\}, \quad (10.32)$$

where C_{peak} is peak compute (GFLOP/s) and B is sustainable memory bandwidth (GB/s). As C_{peak} grows with more tensor cores, keeping B proportionate becomes increasingly costly in area and power.

Power and thermal physics further constrain scaling. Dynamic power scales as

$$[H]P_{\text{dyn}} = C_{\text{eff}} V^2 f, \quad (10.33)$$

so frequency boosts or voltage increases to raise per-core performance yield quadratic cost. For multi-SM chips, total power density limits die-level SM counts and clock targets, forcing trade-offs between SIMD width, pipeline depth, and voltage islands.

Other concrete scalability challenges:

- Memory bandwidth vs capacity: HBM channels scale area and PHY complexity; near-data processing reduces traffic but complicates coherence and software models.
- Interconnect latency and coherence: multi-die chiplets and CXL-like fabrics impose higher tail latencies; collective operations (all-reduce) degrade unless network topology and NIC offloads are co-designed.
- Verification and security surface area: heterogeneous accelerators and run-time reconfiguration expand verification state space and attack vectors, increasing development cost.
- Reliability at scale: electromigration, negative-bias temperature instability, and soft errors compound as die area and operating temperature rise.

A small reproducible tool helps quantify compute vs memory limits; the following Python snippet computes roofline-bound throughput for a set of kernels and shows pressure points.

Listing 10.28: Roofline throughput estimator

```
def roofline(C_peak, B, intensity):
    # C_peak: GFLOP/s, B: GB/s, intensity: FLOP/byte
    return min(C_peak, B * intensity) # GFLOP/s

# Example: tensor core pool and HBM channel scaling
C_peak = 40000.0 # GFLOP/s (aggregate)
B = 1500.0 # GB/s (sustainable)
kernels = {'conv':8.0, 'gemm_small':64.0, 'spmv':0.5}
for name, I in kernels.items():
    print(name, roofline(C_peak, B, I)) # shows bottleneck per kernel
```

Design implications and trade-offs:

1. Invest in bandwidth-proportional memory subsystems or programmable near-memory units to raise $B \cdot I$ for common workloads.
2. Partition power domains and accept lower per-SM frequency to increase aggregate SM/tensor core counts under thermal caps.
3. Co-design interconnect topologies and collective offloads to sustain multi-GPU ML scaling.
4. Prioritize verification and fault-mitigation measures as heterogeneity and runtime reconfiguration increase system complexity.

Glossary of Terms

General Concepts

- **ALU (Arithmetic Logic Unit):** A fundamental building block of a GPU or CPU responsible for performing arithmetic and logical operations.
- **Backface Culling:** A graphics technique that eliminates faces of a 3D object not visible to the camera, improving rendering efficiency.
- **BRAM (Block Random Access Memory):** Memory embedded in FPGA devices, commonly used for buffering and storage in GPU pipelines.
- **Clipping:** The process of discarding primitives or pixels that lie outside the viewport or a designated viewing frustum.
- **Combinational Logic:** A type of digital logic where the output depends solely on the current input, without memory.
- **Dithering:** A graphics process that reduces color banding in images by adding noise or approximating unavailable colors.
- **FPGA (Field-Programmable Gate Array):** A programmable hardware device used for prototyping and implementing digital designs, including GPUs.
- **Framebuffer:** A memory buffer that holds the final rendered image, typically used to interface with display hardware.
- **GPU (Graphics Processing Unit):** A specialized processor optimized for parallel processing tasks such as rendering graphics and performing general-purpose computations.
- **HDL (Hardware Description Language):** A programming language, such as Verilog or VHDL, used to describe and design digital logic circuits.

GPU and Rendering Processes

- **Interpolation:** The calculation of intermediate values (e.g., colors, texture coordinates) between known points, commonly used in rasterization.
- **Lambertian Shading:** A simple shading model based on the diffuse reflection of light, often used in basic lighting calculations.

- **Metastability:** An unstable state in digital circuits that occurs when signals do not meet setup or hold time requirements, leading to unpredictable behavior.
- **Parallelization:** The process of dividing a computational task into smaller subtasks that can be executed simultaneously, fundamental to GPU operation.
- **Pipeline:** A series of processing stages in hardware or software, where each stage performs a specific task on a data stream.
- **Rasterization:** The process of converting 3D primitives into 2D pixel data for display on a screen.
- **Shader:** A small program that runs on a GPU to perform per-vertex or per-fragment processing tasks, such as lighting and texture mapping.
- **SIMD (Single Instruction, Multiple Data):** A parallel processing model where a single instruction is applied to multiple data elements simultaneously.
- **SIMT (Single Instruction, Multiple Threads):** An execution model used in modern GPUs where multiple threads execute the same instruction, with each thread operating on different data.
- **Texture Mapping:** The application of a 2D image (texture) onto a 3D surface to add detail and realism.

Verilog-Specific Terms

- **Always Block:** A procedural block in Verilog used to describe sequential or combinational logic based on event triggers.
- **Continuous Assignment:** In Verilog, a method to assign values to `wire` data types using the `assign` keyword.
- **Generate Statement:** A Verilog construct used to create parameterized or repetitive hardware structures.
- **Reg (Register):** A Verilog data type used to store values in sequential logic circuits.
- **Synthesis:** The process of converting HDL code into a hardware implementation, such as FPGA or ASIC design.
- **Testbench:** A simulation environment used to verify the correctness of an HDL design by applying input stimuli and monitoring outputs.
- **Verilog:** A hardware description language used to model digital circuits at various levels of abstraction.
- **Z-Buffering:** A technique used to manage depth information in 3D rendering, ensuring that closer objects obscure further ones.

Advanced GPU Concepts

- **Clock Domain Crossing:** Techniques used to safely transfer signals between hardware components operating on different clock domains.
- **Framebuffer:** A memory buffer containing pixel data, used to store the final output image before display.
- **High-Bandwidth Memory (HBM):** A type of memory designed for high data throughput, commonly used in modern GPUs for handling large datasets.
- **Memory Arbitration:** Strategies for resolving access conflicts in memory subsystems, ensuring efficient use of bandwidth.
- **Memory Caching:** The use of small, fast memory to store frequently accessed data, reducing latency and improving throughput.
- **Multi-Sampling Anti-Aliasing (MSAA):** A graphics technique that reduces aliasing artifacts by sampling multiple points per pixel and averaging the results.
- **Parallel Arithmetic:** Performing multiple arithmetic operations simultaneously, essential for GPU efficiency in tasks like shading and matrix transformations.
- **Shader Pipeline:** The sequence of shader stages in a GPU, including vertex, geometry, and fragment shading, used to process rendering tasks.
- **SIMT Efficiency:** Optimizing the execution of threads in a GPU to minimize divergence and maximize parallel throughput.
- **Warp Scheduling:** The process of managing groups of threads (warps) for efficient execution in SIMT architectures.

Graphics and Rendering Techniques

- **Alpha Blending:** A technique for rendering transparency by combining foreground and background colors based on an alpha value.
- **Edge Functions:** Mathematical expressions used to determine pixel coverage during rasterization.
- **Gouraud Shading:** A shading technique that interpolates vertex colors across a polygon, providing a smooth gradient effect.
- **HDR (High Dynamic Range):** A rendering technique that allows a wider range of colors and luminance, enhancing image realism.
- **Perspective Projection:** The process of mapping 3D points onto a 2D plane to simulate depth, based on a camera's perspective.
- **Primitive Assembly:** The process of combining vertices into geometric primitives such as triangles or lines for rendering.

- **Raster Operations (ROP):** The final stage in rendering where pixel data is written to the framebuffer, including blending and depth testing.
- **Screen Space:** The coordinate system used for rendering images on a display, corresponding to the visible portion of the framebuffer.
- **Texture Filtering:** Techniques such as bilinear or trilinear filtering to smooth textures and reduce aliasing artifacts.
- **Z-Fighting:** A visual artifact caused by insufficient precision in depth buffers, resulting in overlapping objects flickering.

System-Level Considerations

- **Fixed-Function Pipeline:** A legacy GPU architecture with dedicated hardware for specific tasks, contrasted with modern programmable pipelines.
- **Floating-Point Arithmetic:** Numerical computations using floating-point representation, commonly employed in shaders and transformations.
- **Latency:** The delay between an input stimulus and the corresponding output response, critical in real-time rendering.
- **Metastability Mitigation:** Techniques to ensure reliable signal transfer between asynchronous clock domains, such as synchronization registers.
- **Pipeline Stalling:** A condition where a pipeline stage must wait for data, causing delays in subsequent stages.
- **Resolution Scaling:** Adjusting the rendered resolution to balance performance and visual quality.
- **Setup and Hold Times:** Timing constraints in digital circuits to ensure reliable data capture at clock edges.
- **Throughput:** The amount of data processed per unit of time, a key performance metric in GPU design.
- **Triangle Setup:** The preprocessing of vertex data to determine rasterization parameters for a triangle.
- **Unified Memory Architecture (UMA):** A memory architecture where CPU and GPU share a common memory pool, simplifying data access.

Verification and Debugging

- **Assertions:** Statements in testbenches used to check conditions and verify expected behavior during simulations.

- **Coverage Metrics:** Quantitative measures of how thoroughly a design has been tested, including code, functional, and toggle coverage.
- **Debugging Strategies:** Methods for identifying and resolving errors in HDL designs, such as using waveform analysis and signal dumps.
- **Formal Verification:** Techniques that use mathematical proofs to ensure correctness of hardware designs against specifications.
- **Functional Simulation:** Running a hardware model in a simulator to validate its logical correctness.
- **Regression Testing:** Repeated testing to ensure new changes do not introduce errors into previously verified functionality.
- **Signal Dump (VCD):** A file format used to capture and analyze signal waveforms during simulation.
- **Test Coverage:** A metric indicating how much of the design functionality has been exercised by the testbench.
- **Testbench Automation:** Using scripts and tools to generate and run testbenches efficiently, reducing manual effort.
- **Waveform Analysis:** Examining signal transitions over time using tools like ModelSim or Vivado to debug hardware designs.

Advanced AI and GPU Architectures

- **AI Workloads:** Computational tasks specific to artificial intelligence, such as training and inference in neural networks.
- **Deep Learning Accelerator:** Specialized hardware within GPUs designed to optimize neural network computations, such as tensor operations.
- **Dynamic Parallelism:** A feature in modern GPUs that allows threads to spawn additional threads during execution.
- **Gradient Descent:** An optimization algorithm used in training neural networks, requiring efficient support for matrix operations.
- **Instruction Set Architecture (ISA):** The set of instructions a processor can execute, including specialized operations for AI and graphics tasks.
- **Low-Precision Arithmetic:** Numerical formats like FP16 or INT8, optimized for AI workloads by balancing performance and accuracy.
- **Matrix Multiplication:** A key operation in AI and graphics, accelerated in GPUs through dedicated hardware like tensor cores.
- **Neural Network Accelerator:** A specialized processing unit optimized for neural network operations, such as backpropagation and convolution.

- **SIMT Divergence:** A condition where threads in a warp follow different execution paths, reducing parallel efficiency.
- **Tensor Cores:** Hardware units in modern GPUs optimized for high-performance tensor operations, critical for AI applications.

Emerging Technologies and Trends

- **Chiplet Architecture:** A modular approach to designing processors by combining smaller, functional chips (chiplets) to form a complete system.
- **High-Bandwidth Memory (HBM):** Advanced memory technology providing high data throughput, widely adopted in GPUs for demanding workloads.
- **Ray Tracing:** A rendering technique that simulates the physical behavior of light to produce realistic shadows, reflections, and lighting effects.
- **RISC-V Extensions:** Open-standard instruction set extensions enabling customization for specific workloads, including GPU and AI tasks.
- **Thermal Management:** Techniques to manage heat dissipation in GPUs, ensuring reliability and performance under high computational loads.
- **Tile-Based Rendering:** A rendering approach that divides the screen into smaller tiles to optimize memory access and improve efficiency.
- **Unified Shader Model:** A GPU architecture where the same shader units handle vertex, geometry, and fragment processing.
- **Variable Rate Shading (VRS):** A technique that adjusts the shading rate for different screen regions to optimize performance without significant quality loss.
- **Virtual Reality (VR) Optimization:** Enhancements in GPUs to handle the high frame rates and low latencies required for immersive VR experiences.
- **Volumetric Rendering:** A technique for visualizing 3D datasets, such as medical scans or scientific simulations, using GPUs.

Key Mathematical Equations

First Steps in Verilog Development

Boolean algebra for combinational circuits:

$$F(A, B, C) = A \cdot (B + \overline{C})$$

3D Geometry Basics

Affine transformation:

$$\mathbf{v}' = \mathbf{R} \cdot \mathbf{v} + \mathbf{t}$$

Rasterization Basics

Barycentric coordinates:

$$\lambda_1 = \frac{(x_2 - x)(y_3 - y_2) - (x_3 - x_2)(y_2 - y)}{(x_2 - x_1)(y_3 - y_2) - (x_3 - x_2)(y_2 - y_1)}$$

Combinational vs. Sequential Logic

Finite-state machine (FSM) transition equation:

$$S(t+1) = f(S(t), X(t))$$

Clock Domain Crossing Strategies

Metastability resolution probability:

$$P = e^{-\frac{t_{res}}{\tau}}$$

Power Estimation and Management

Dynamic power estimation:

$$P_{dynamic} = \alpha CV^2 f$$

Clipping and Culling

View frustum clipping:

$$ax + by + cz + d \geq 0$$

Scan Conversion

Line equation for Bresenham's algorithm:

$$y = mx + b$$

Interpolation of Attributes

Perspective-correct interpolation:

$$A(x, y) = \frac{\sum w_i A_i}{\sum w_i}, \quad w_i = \frac{1}{z_i}$$

Memory Arbitration Techniques

Bandwidth utilization:

$$U = \frac{N_{effective}}{N_{total}}$$

Memory Caching

Cache hit rate:

$$H = \frac{\text{Cache hits}}{\text{Total accesses}}$$

SIMT vs. SIMD

Warp occupancy:

$$O = \frac{\text{Active warps}}{\text{Max warps per SM}}$$

Basic Neural Network Inference

Convolution operation:

$$Y(i, j) = \sum_{m=-k}^k \sum_{n=-k}^k X(i+m, j+n) \cdot K(m, n)$$

Specialized Hardware for AI

Tensor core operation:

$$C_{ij} = \sum_k A_{ik} \cdot B_{kj} + C_{ij}$$